IC Validator User Guide

Version X-2025.06-SP2, October 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Feedback

Contents

New in This Release 17
Related Products, Publications, and Trademarks 17
Conventions 18
Customer Support 18
Statement on Inclusivity and Diversity 19

1. **IC Validator Basics** 20
Using the IC Validator Tool in Your Design Flow 20
Running the IC Validator Tool Within the IC Compiler Tool 21
Running the IC Validator Tool Within the IC Compiler II Tool 22
Running the IC Validator Tool With StarRC Parasitic Extraction 23
Running the IC Validator Tool From the Custom Compiler Platform 23
Setting Up and Running DRC 24
Setting Up and Running LVS 24
IC Validator Command-Line Options 25
Common 26
Host Configuration 26
Runset Configuration 29
DRC 32
LVS / PERC 34
Explorer 39
Input Database 40
Output 43
Debug 45
Optimization Controls 47
Licensing 48
General 48
Compiler Directives 49
Support for 64-Bit Coordinates 49
Run-Only Execution 51
Environment Variables 52
Providing Additional Information to the Compiler 53
Creating Preprocessor Definitions From Environment Variables 53

Pruning Layers 55
Prioritizing the Scheduling of Functions 55
Runset Configuration 56
Runset Insertion 58
Exit Codes 58
Inlined Functions 60
Distributed Processing 62
Specifying Host Names and CPU Counts 62
Modifying Resources for a Running Job 63
Running in LSF 64
Running in Univa Grid Engine (UGE/SGE) 65
Running in SLURM 66
License Requirements 67
Runset Caching 68
Cache Files 68
Configuring Runset Caching 69
Matching a Runset to a Cache File 71
Running Knowledge Assistant Using IC Validator 71
Using IC Validator Command-Line Options With Runset Caching 72
Compatible Command-Line Options 73
Command-Line Options That Affect Runset Caching 73
Incompatible Command-Line Options 74
Layer Mapping 74
Runset Layer Mapping 74
Data Layer Mapping 76
Using PXL Runset Encryption 78

2. Licensing and Resource Requirements 81
Licensing Schemes 81
License Waiting 82
User Controls 82
License Caching 83
Which IC Validator Licenses Do I Need? 84
Apex Licensing 84
Elite Licensing 84

Base Licensing . . . 85
Licensing Requirements by Technology . . . 85
Licensing Requirements by Function . . . 87
Multicore Licensing . . . 89
Apex Multicore Licensing . . . 89
Elite Multicore Licensing . . . 89
Base Multicore Licensing . . . 89
Token Licensing . . . 90
Hyperthreading . . . 90
Resource Requirements . . . 91
Space Requirements . . . 91
Memory Usage . . . 92
Runtime and Memory Use Report . . . 93
Operating Systems . . . 97
System Limits . . . 97
Networked Environments . . . 97

3. Cloud Enablement for IC Validator . . . 98
Overview . . . 98
Using IC Validator in the Cloud . . . 98
Hybrid Cloud . . . 99
No Public Cloud Access . . . 99
Existing Cloud Access Without Data Mirroring and Existing Compute Grid . . . 99
Existing Data Mirroring and Multi-Cluster Grid Environment . . . 99
IC Validator Cloud Functionalities . . . 100
Preparing Your Design for The Cloud . . . 100
Transferring Your Design . . . 100
Cloud Configuration Guidelines . . . 100
IC Validator on Hybrid Cloud . . . 102

4. Output Files . . . 104
Description of Output Files . . . 104
General DRC and LVS Output Files . . . 105
Results File . . . 105
Error File . . . 108
Summary File . . . 110
Rules File . . . 110

Technology File 111
Tree Structure Output File 111
Virtual Cell File 111
Pair Section 111
Set Section 112
Explode Section 112
Distributed Log File 112
Distributed Statistics File 112
Distributed Work Directory 112
Graphical Error Structures 112
LVS-Specific Output Files 114
LVS Debugging File 115
Extracted Layout Netlist File 115
Extracted Layout SPICE Netlist File 115
Account File 115
Bounding Box File 115
Equivalence Files 115
Device Leveling Summary File 116
Compare Log File 116
Compare Tree Files 116
Compare Directory Output Files 117
Individual Equivalence Summary File 117
Partitioned Schematic Netlist 118
Partitioned Layout Netlist 118
Editing Errors 118

5. IC Validator Dashboard 119
Introduction 119
Getting IC Validator Dashboard Email Updates From IC Validator 119
Starting IC Validator Dashboard Standalone 120
Initial Directory Setting 121
The IC Validator Graphical User Interface 121
IC Validator Dashboard Summary Page 121
IC Validator Dashboard Status 123
IC Validator Run Status Indicators 124
IC Validator Run Information 124
CPU Usage Panel 124
Memory Usage Panel 124

Run Progress Panel 125
Rule Progress Panel 125
IC Validator Dashboard CPU History Page 126
IC Validator Memory History Page 128
IC Validator Dashboard Options 128

6. IC Validator Live DRC 130
About the IC Validator Live DRC Tool 130
IC Validator Live DRC With Custom Compiler 130
Setting Up the Environment 131
Viewing Job Progress 137
Running IC Validator Live DRC 138
Viewing and Fixing Violations 139
Additional Options 139
Halos 139
Recipes 140
IC Validator Live DRC With Virtuoso 143
Setting Up the Environment 143
Starting the IC Validator Live DRC Check 144
Browsing Results in IC Validator Live DRC 146
Additional Options 149
Halos 149
Check Layers 150
Honor the P&R Boundary visibility 151
Recipes 151
Saving and Loading Configurations 154
Preference Settings 160
Bindkeys 161
IC Validator Live DRC With IC Compiler II and Fusion Compiler 162
Setting Up the Environment 162
Displaying Objects for Design Rule Checking 166
Setting Up Pre-Compiled Cache Directories 167
Reproducing an IC Validator Live DRC Run in IC Validator 169

7. IC Validator Explorer DRC 170
Introduction to IC Validator Explorer DRC 171
Types of Explorer DRC Modes 171

Executing Explorer DRC 172
Configuring Explorer DRC 173
Explorer DRC Output Files 175
Explorer DRC Results File 176
Explorer DRC Errors File 176
Explorer DRC VUE File 176
Debugging With Explorer DRC 177
Explorer DRC Licenses 178

8. IC Validator Explorer LVS 179
Introduction to IC Validator Explorer LVS 179
Configuring Explorer LVS 180
Executing Explorer LVS 181
Explorer LVS Licenses 182

9. PXL 184
PXL Architecture 185
General Definitions 186
Identifiers 186
String Literals 188
Numeric Constants 188
Keywords 189
Operators 203
Comments 204
Environment Variable Reference 205
Variables and Operators 205
Variables 206
Initializing Variables 206
Scope of Variables 206
Operators 208
Assignment Operators 208
Relational Operators 209
Logical Operators 209
Conditional Operator 210
Numeric and Bitwise Operators 210
Operators With Double-Only Outputs 211

Member Reference 212
Precedence and Associativity 212
Data Types and Typing 214
Supported Data Types 214
Primitive Types 214
User-Defined Structures and Types 218
Type Analysis 225
Standard Arithmetic Conversions 226
Comparing Doubles 226
Type Promotion 227
Parameter Passing 227
String Concatenation 227
Default Injection 228
Nested Containers 228
Flow Control 230
Conditional Statements 230
Loops 231
for Loop 231
foreach Loop 232
while Loop 232
Termination of Execution 233
Functions 233
Function Definitions 233
Function Prototypes 234
Function Calls 235
Standard Function Call 236
Operator-Style Function Call 237
Argument Bindings 238
Defaults 239
Anonymous Functions 239
Function Overloading 240
Function Recursion 241
Program Structure 241
Compound Statement 241
Violations 242
Violation Comments 242
Violation-Producing Functions 243
Violation Variables 244
Violation Blocks 244
Execution Model 245

Preprocessor 245
Predefined Identifiers 247
Release Version Macros 248
Date Comparison Macros 249
Qualifiers 250
Example of Modularized Code 251

10. Dynamic-Link Library Support 253
Dynamic-Link Functions 253
Using the Dynamic-Link Functions 254

11. Unified Fill 255
Overview 256
Types of Fill Patterns 257
Polygon Fill Patterns 258
Single Layer Pattern Definition With Multiple Rectangles 258
Multilayer Pattern Definition 259
Multiple Fill Pattern Definition 260
Adjustable Rectangle Fill Patterns 261
Stack Fill Patterns 262
Cell Fill Patterns 268
Expandable Fill Cell Patterns 269
Stripe Fill Patterns 272
Fill-to-Fill Spacing 273
Spacing Between Instances of the Same Patterns 274
Spacing Between Different Patterns 277
Rules and Restrictions for Spacing Options 279
Target Region and Fill-to-Signal Spacing 280
Range and Width Dependent Spacing 285
Polygon Grouping 286
Signal Aligned Fills 288
Defining the Fill Pattern for Signal Aligned Fills 291
Improving Pattern Insertion 292
Pitch Aligned Fills 292
Criteria Analysis 295

Output Layers 300
Layer Compression 303

12. Working With Edges 304
Associating Edge Layers With a Parent Layer 305
Topological Operations 305
Logical Operations 306
Edge Manipulation 307

13. Critical Area Analysis Flow 308

14. DRC Error Classification 315
Using DRC Error Classification 316
Hierarchy Considerations in CELL_LEVEL Mode 316
Hierarchical DRC Error Classification 317
Error Classification Best Practices 317
Database Naming Conventions 318
DRC Error Classification Flow 318
DRC Error Classification Flow Example 320
Running the IC Validator Tool Initially 320
Creating an Error Classification Database 323
Running the IC Validator Tool Again 324
Updating the Existing Error Classification Database 325
pydb_report Utility 327
Classify File Format 329
Using Regular Expression Matching 330
Examples 331
pydb_export Utility 332
Error Classification Database Format 337
Purge File Format 338
Examples 338
Flattening an Error Classification Database 339
Using PERC Error Classification 340
Matching Classified Errors 341
Hierarchical PERC Error Classification 342

15. DRC Waivers 344

Defining Waivers 345

Using VUE to Define Waivers 346

Defining Waivers Using the Command-Line Utilities 349

Custom Waiver Properties 349

Creating a Waiver Database 352

Using a Waiver Database 356

Matching Modes 357

Waiver Report File 358

16. Error Management Utility 361

Error Management Utility 361

DRC Error Grouping 370

Splitting Error Databases 371

Merging Error Databases 372

Applying Waivers to an Existing Error Database 373

VUE File Support 373

Modifying Databases in Place 373

Hierarchy Considerations 374

XML Command File 374

Classifying Errors With the XML Command File 382

Exporting Errors to a Layout 383

Managing Referenced cPYDB Paths 384

17. Pattern Matching 386

Overview 386

Using the Pattern Matching Flow 387

Creating the Pattern Library 388

Preparing the Input Layout 388

Required Input 389

Optional Input 390

Additional Pattern Specifications 391

Example of Creating a Pattern Library 392

Examining the Output of Pattern Library Creation 394

Viewing and Locking a Pattern Library 397
Checking a Layout 398
Runset Example of Pattern Matching 398
Examining Pattern Matching Output 400
Post Pattern-Matching Operations 401
Detecting and Repairing Hotspots Within the IC Compiler Tool 402
Setting Up the Physical Signoff Options 402
Running Pattern Matching for Hotspot Detection 402
Automatically Fixing Hotspot Patterns 402
Repairing Hotspots With User-Defined Fix Guidance 403

18. Pattern Library Manager 406

Getting Started 406
Supported Versions of IC Validator WorkBench 406
Invoking the Pattern Library Manager 406
Communicating With IC Validator WorkBench 409
Pattern Library Manager GUI 410

Creating a New Pattern Library 410
Creating a New Pattern Library Name 411
Adding New Patterns 412
Capturing a New Pattern by Clipping 413
Extracting Patterns in a Layout 415
Generating a New Pattern From Scratch 417

Editing a Pattern 422
Adding an Edge Tolerance to a Pattern 423
Attaching Pattern Properties to a Pattern 424
Editing a Pattern Layout in IC WorkBench EV Plus 426

Exporting and Importing a Pattern Library 429
Importing a Pattern Database Format 430
Exporting a Pattern Library Manager Format 431

Running Pattern Matching in the Pattern Library Manager 433

Restrictions and Limitations 435

A. IC Validator Architecture 436

Basic Architecture 437

B. LVL Utility 438

Description 439

Command-Line Options 439

Using the OpenAccess Library Format 447

OpenAccess Layer Mapping File 448

Cell Mapping 449

Error Output Format for OpenAccess Baseline Library 449

Setting the Cell View Name 450

Layer Files 450

Selecting a Subset of Layers 450

Mapping Layers 451

Comparing a Subset of Layers 452

Comparing by Name 453

Comparing by Number 453

Handling of Text 454

LVL Output 455

Exit Status 456

Error Files 456

Output Directories 457

Use Models 458

QuickLVL Utility 459

Cell-Level Quick LVL 460

Differences Between LVL and QuickLVL 460

Auto-Incremental DRC Focused on Differing Layers and Regions 461

C. Layout Integrity Management 463

Using Layout Integrity Management 463

Layout Integrity Management Examples 464

Basic Example 464

Running on a Different Layout Format 466

Reporting of Mismatches From a Merged LIDB 467

Hierarchical Mode 468

Hierarchical Cell Check Example 468

Hierarchical Marker Layer Check Example 470

Marker Layer Check 474

Mixed Milkyway and GDSII Flow 475
The icv_lidb Utility 477
Command-Line Options 477
Exit Status 479
Output Files 479
The icv_lidb_report Utility 479
Command-Line Options 480
Output Files 480

D. IC Validator PXL Debugger 481
Running the PXL Debugger 482
PXL Debugger Commands 482
break 485
cont 487
delete 488
disable 489
enable 490
finish 491
info break 492
list 493
next 495
print 496
quit 497
record 498
source 500
step 501
where 502

E. PYDB Perl API 503
PYDB Perl API Overview 504
Accessing the PYDB 505
Examples of Common Tasks 506
PYDB Schema 508

F. Third-Party Licenses 509

Licensing Overview 510
OSS Package Notices 511
ANTLR 511
Boost 512
cx_Freeze 513
Flatbuffers 514
libdp 514
libxml2 516
MariaDB Connector 516
MariaDB 523
MCPP Public Domain Code 527
NumPy* 528
patchELF 534
pyparsing* 534
Python 3.6 License 534
scikit-learn 536
SciPy* 536
Shroud-1.0 546
SQLite 547
TclLib 549
Tcl/Tk 549
zlib 550
Zstandard 551
Standard OSS License Text 552
Apache-2.0 552
Artistic-1.0-Perl 556
GPL-1.0 559
GPL-3.0 564

Glossary 579

About This User Guide

This user guide describes PXL, the IC Validator architecture, command-line options, and other basic information.

This user guide is designed to enhance both beginning and advanced users' knowledge of PXL, the IC Validator architecture, and command-line options.

This preface includes the following sections:

- New in This Release
- Related Products, Publications, and Trademarks
- Conventions
- Customer Support
- Statement on Inclusivity and Diversity

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the IC Validator Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the IC Validator tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

https://solvnetplus.synopsys.com

You might also want to see the documentation for the following related Synopsys products:

- Synopsys® Custom Compiler™
- Synopsys IC Compiler™
- Synopsys IC Compiler™ II
- Synopsys Fusion Compiler™
- Synopsys StarRC™

Feedback

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
`Courier`	Indicates syntax, such as `write_file`
`Courier italic`	Indicates a user-defined value in syntax, such as `write_file` *`design_list`*
`Courier bold`	Indicates user input—text you type verbatim—in examples, such as `prompt> write_file top`
`Purple`	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as `include_enclosing = true | false`
[ ]	Denotes optional arguments in syntax, such as `write_file [-format fmt]`
...	Indicates that arguments can be repeated as many times as needed, such as *`pin1 pin2 ... pinN`*.
|	Indicates a choice among alternatives, such as `low | medium | high`
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the **Edit** menu and choosing **Copy**.
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

https://solvnetplus.synopsys.com

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to https://solvnetplus.synopsys.com.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

IC Validator Basics

This chapter gives a brief overview of the IC Validator physical verification tool. It also describes the IC Validator command-line options and distributed processing.

The basics of the IC Validator tool are described in the following sections:

- Using the IC Validator Tool in Your Design Flow
- IC Validator Command-Line Options
- Compiler Directives
- Exit Codes
- Inlined Functions
- Distributed Processing
- Runset Caching
- Using IC Validator Command-Line Options With Runset Caching
- Layer Mapping
- Using PXL Runset Encryption

Using the IC Validator Tool in Your Design Flow

All IC Validator runs are executed based on your IC Validator runset. You can execute the IC Validator tool using command-line execution, interactive execution using VUE, or from various other Synopsys tools, such as the StarRC or IC Compiler tools. To load IC Validator, you must set two environment variables:

- Set ICV_HOME_DIR to the IC Validator installation directory
- Set PATH to include $ICV_HOME_DIR/bin

Feedback

To run the IC Validator tool successfully in your design flow,

- Create and debug a runset file. The runset file includes the following information:
  - Layout database specifics
  - General options
  - Layer assignments
  - Database checks, such as snap and grid checks
  - Design rule checking (DRC), electrical rule checking (ERC), fill patterns, or layout-versus-schematic (LVS) checking
- Run IC Validator DRC and ERC on your layout database, verifying your design against the design rule document.
- Review and debug errors generated in the IC Validator DRC and ERC run using IC Validator error files or VUE.
- Run the IC Validator tool to perform fill-pattern generation, including basic DRC checks on the layers that are filled.
- Run IC Validator LVS checking.
- Review and debug errors generated in the IC Validator LVS run using IC Validator error files or VUE.

See Appendix A, IC Validator Architecture.

Running the IC Validator Tool Within the IC Compiler Tool

You can run the IC Validator tool from inside the IC Compiler tool; this flow is called In-Design physical verification. For this flow, use the following IC Compiler commands:

- `signoff_drc`. Performs DRC checking using a foundry-qualified IC Validator DRC runset.
- `signoff_autofix_drc`. Performs automatic DRC repair. In this flow, the IC Validator tool.
  - Performs signoff DRC checking.
  - Generates route guidance that assists the router in performing targeted repairs.
  - Validates the repair done by the router.
- `signoff_metal_fill`. Inserts metal and via fill in either pattern-based mode, which uses a foundry-qualified IC Validator metal-fill runset, or in track-based mode, which

Feedback

uses information from the technology file. The fill shapes are written into a FILL view of the Milkyway design.

- `signoff_fix_isolated_via`. Performs automatic fixing of isolated vias.
- `report_metal_density`. Calculates the metal density and density gradient values and annotates the values on the design.

Note:

You must make sure that the IC Validator tool can parse and run the runset before using it within the IC Compiler tool.

See the *IC Compiler Implementation User Guide*, which is available on SolvNetPlus, or the IC Compiler man pages for detailed command syntax and usage.

Running the IC Validator Tool Within the IC Compiler II Tool

You can run the IC Validator tool from inside the IC Compiler II tool; this flow is called In-Design physical verification. For this flow, use the following IC Compiler II commands:

- `signoff_check_drc`. Performs DRC checking using a foundry-qualified IC Validator DRC runset.
- `signoff_fix_drc`. Performs automatic DRC repair. In this flow, the IC Validator tool
  - Performs signoff DRC checking.
  - Generates route guidance that assists the router in performing targeted repairs.
  - Validates the repair done by the router.
- `signoff_create_metal_fill`. Inserts metal and via fill in either pattern-based mode, which uses a foundry-qualified IC Validator metal-fill runset, or in track-based mode, which uses information from the technology file. The fill shapes are written into the design view of the IC Compiler II input design.
- `signoff_fix_isolated_via`. Performs automatic fixing of isolated vias.

Note:

You must make sure that the IC Validator tool can parse and run the runset before using it within the IC Compiler II tool.

See the *IC Compiler II Implementation User Guide*, which is available on SolvNetPlus, or the IC Compiler II man pages for detailed command syntax and usage.

Feedback

Running the IC Validator Tool With StarRC Parasitic Extraction

You can generate IC Validator data for input to the StarRC tool, using the following functions as part of the IC Validator runset:

- The functions for parasitic extraction (PEX) layer mapping are
  - `pex_conducting_layer_map()`
  - `pex_via_layer_map()`
  - `pex_marker_layer_map()`
  - `pex_remove_layer_map()`
  - `pex_viewonly_layer_map()`
  - `pex_ignore_cap_layer_map()`

  These functions map IC Validator `polygon_layer` objects to StarRC TCAD_GRD_FILE layers to generate a layer mapping file for use with the StarRC tool.
- The function for generating a connected layout database and mapping files for input to StarRC parasitic extraction is
  - `pex_generate_results()`

Table 1 shows the LVS settings that are not supported for use in the IC Validator–StarRC flow. These settings should be set to `false`.

Table 1 LVS Settings Not Supported for IC Validator–StarRC Flow

Unsupported setting	Argument default	Recommended setting
`compare(memory_array_compare=true)`	`true`	`false`
`merge_series(multiple_paths=true)`	`false`	`false`

See the *IC Validator Reference Manual* for detailed syntax and usage of the parasitic extraction functions.

Running the IC Validator Tool From the Custom Compiler Platform

You can run the IC Validator tool for DRC and LVS from the Custom Compiler platform using its Verification menu or from the Custom Compiler command line. See the Custom Compiler documentation, which is available on SolvNetPlus, for more information.

Setting Up and Running DRC

You can set up and run DRC from the Custom Compiler platform using its Verification menu:

1. In your Layout window, choose Verification > DRC > Setup and Run.
2. Select the IC Validator tool and specify all the required runtime parameters.
3. Click Apply or OK to run the IC Validator tool.

Note:

To view the output, select the View Output check box to open the Custom Compiler Text Viewer. Using tabs in the viewer, you can see the runset, results, errors, and a log of the run.

To run the IC Validator tool with parameters previously set in the Setup and Run menu, choose Verification > DRC > Run.

You can start an IC Validator VUE debugging session from the Custom Compiler platform.

- After an IC Validator run, to run VUE from the Custom Compiler platform, choose Verification > DRC > Debug.
- To run VUE when the Setup menu has not been set to ICV,
  - Set the preference `db::setPrefValue xtDRCTool -value icv`.
  - Choose Verification > DRC > Debug.

To open the Custom Compiler Text Viewer, which provides access to the runset, *cell*.RESULTS, *cell*.LAYOUT_ERRORS, and the stdout.drc.log files, choose Verification > DRC > View Output.

Setting Up and Running LVS

You can set up and run LVS from the Custom Compiler platform using its Verification menu:

1. In your Layout or Schematic window, choose Verification > LVS > Setup and Run.
2. Select the IC Validator tool and specify all the required runtime parameters.
3. Click Apply or OK to run the IC Validator tool.

Note:

To view the output, select the View Output check box to open the Custom Compiler Text Viewer. Using tabs in the viewer, you can see the runset, results, errors, and a log of the run.

Feedback

To run the IC Validator tool with parameters previously set in the Setup and Run menu, choose Verification > LVS > Run.

You can start an IC Validator VUE debugging session from the Custom Compiler platform.

- After an IC Validator run, to run VUE from the Custom Compiler platform, choose Verification > LVS > Debug.
- To run VUE when the Setup menu has not been set to ICV,
  - Set the preference `db::setPrefValue xtLVSTool -value icv`.
  - Choose Verification > LVS > Debug.

To open the Custom Compiler Text Viewer, which provides access to the runset, *cell*.RESULTS, *cell*.LAYOUT_ERRORS, *cell*.LVS_ERRORS, and the stdout.lvs.log files, choose Verification > LVS > View Output.

IC Validator Command-Line Options

The IC Validator command-line syntax is as follows:

```
icv [command-line-options] [runsetfile]
```

where *runsetfile* is a set of instructions written in PXL, which tells IC Validator how to perform the required checking operations. In most cases, a runset file is required. There are some exceptions, for example, when comparing two netlists, or when adding or removing hosts to a run.

This section of the manual describes the command-line options that can be used with IC Validator. They are organized into categories based on their purpose as follows:

- Common
- Host Configuration
- Runset Configuration
- DRC
- LVS / PERC
- Explorer
- Input Database
- Output
- Debug

Feedback

- Optimization Controls
- Licensing
- General

Common

Table 2 IC Validator Common Command-Line Options

Option	Definition
Common	Options used often in most flows.
runsetfile	IC Validator runset.
`-c` *cell*	Overrides the `cell` argument in the `library()` function and the `layout_top_cell` argument in the `compare()` function. By specifying `-cell "*"` you can run the IC Validator tool using the OASIS or GDSII format without specifying the top cell. The IC Validator tool finds the top cell in the database. If there is more than one potential top cell, the IC Validator tool chooses the first one it finds in the database and prints a warning message in the summary file. Do not use `-cell "*"` in these situations: • LVS runsets, that is, runsets with a call to the `compare()` function. • Runsets that call the `get_top_cell()` function before the end of the ASSIGN section. • Along with the `-ndg` or `-C` command-line options. • Runsets that set the `select_window` or `exclude_window` argument of the `incremental_options()` function. • Databases that are not in GDSII or OASIS format.
`-f` *format*	Overrides the library format (GDSII, Milkyway, NDM, OASIS, or OpenAccess) specified in the library() function.
`-vue`	Creates VUE output. Creating VUE output can also be enabled by setting the `create_vue_output` argument in the `error_options()` function to `true`.

Host Configuration

Table 3 IC Validator Host Configuration Command-Line Options

Option	Definition
Host Configuration	Options to specify the hosts, CPUs, memory, and disk usage.

Feedback

Table 3 IC Validator Host Configuration Command-Line Options (Continued)

Option	Definition
`-host_init hostname | number_of_cpus | hostname:number_of_cpus | LSF | SGE | NB | SLURM`	Starts an IC Validator job on a single host or multiple hosts. See Specifying Host Names and CPU Counts for more information. Examples: • `-host_init` *`hostname`*. Runs on the given host and uses all of the CPUs. • `-host_init` *`number_of_cpus`*. Runs on the local host and uses the specified number of CPUs. • `-host_init` *`hostname:number_of_cpus`*. Runs on the given host and uses the specified number of CPUs. • `-host_init LSF`. Automatically determines host and number of CPUs from Platform LSF environment. • `-host_init SGE`. Automatically determines host and number of CPUs from the Univa Grid Engine environment • `-host_init NB`. Automatically determines host and number of CPUs from the NetBatch environment. • `-host_init SLURM`. Automatically determines host and number of CPUs from the SLURM environment. **Note:** Use the `sbatch` batch submission command to submit jobs and not the `srun` command which is not suitable for multihost submissions. **Note:** When using the `hostname` and `number_of_cpus` syntax, the argument to the `-host_init` command-line option can be specified multiple times, for example, `-host_init hosta:8 hostb:8`
`-host_disk`	Specifies a local group directory for all hosts specified with `-host_init` or `-host_add`. Examples: • `icv -host_init hostA:32 -host_disk /SCRATCH/user/group_1` • `icv -host_add hostB:32 -host_disk /SCRATCH/user/group_1` **Note:** If the local disk space becomes 100 percent full while the tool is running, the tool no longer uses the local directory path, but instead uses only the network path of the run_details directory. By default, the `write_vue_info` function does not use the directory specified by the `-host_disk` command-line option. To override this behavior, set the `_ICV_NI_USE_HOST_DISK` environment variable. Use this option when the `-host_disk` directory provides significantly better performance than the run directory.

Feedback

Table 3 IC Validator Host Configuration Command-Line Options (Continued)

Option	Definition
`-host_login`	Enables the IC Validator tool to log in to remote hosts to start processes for a distributed job. Ensure that the tool locates the binary/script supplied to `-host_login`. The default is `rsh`. Examples: • `icv -host_login ssh` • `icv -host_login "qrsh -inherit"` **Note:** Your specification of `-host_login` always takes precedence over other settings including overridding the login mechanism used for `-host_init/-host_add LSF|SGE|NB|SLURM`.

`-host_memory` *`Memory_Value`* `M|G|T`	Attempts to maintain the user-specified target memory for the hosts used in the IC Validator job by rescheduling and threading. This switch cannot be used to reduce the memory of a single command.
	`-host_memory 1000M`
	`-host_memory 100G`
	`-host_memory 10T`
	The maximum value of this command-line option is the actual physical memory of the machine.
	Note:
	The `-host_init` and `-host_add` command-line options support the `-host_memory` command-line option. Unless the `-host_memory` command-line option is specified on the command line, the IC Validator tool never sets any memory targets. The `-host_add` command-line option does not inherit any memory settings from the main job.

`-host_add hostname | number_of_cpus | hostname:number_of_cpus | LSF | SGE | NB | SLURM`	Adds more hosts (including remote hosts) to an IC Validator job that is already running. The usage is the same as the usage for `-host_init`. See Specifying Host Names and CPU Counts for more information. **Note:** By default, `-host_add` only supports single host grid requests. Use multiple `host_add` submissions to add more than one host.
`-host_remove` *`list_of_hosts`*	Removes the list of hosts from the IC Validator job. All IC Validator processes and commands linked to the running job are shut down for the listed hosts (if this host was acquired through a host reservation system, the host can be released).
`-host_thread_mode` *`SMT | <value>`*	• `-host_thread_mode SMT`. Uses simultaneous multithreading (SMT) enabled CPUs for threading. • `-host_thread_mode <value>`. Dedicates a percentage of CPUs to be used for threading. The value must be between 0 and 100. **Note:** `SMT` is evaluated before `<value>` when both options of `-host_thread_mode` are provided.

Feedback

Table 3 IC Validator Host Configuration Command-Line Options (Continued)

Option	Definition
`-host_elastic_cpu_shutoff` *#cpus*	Specifies the maximum number of CPUs a running IC Validator job acquires automatically. The default is 2000.
`-host_elastic user_command`	Automatically adds or removes hosts (including remote hosts) to or from an IC Validator job that is already running. This option enables the functionality and specifies the method for acquiring new hosts. This option is specified when a job is originally started or applied to a running job. See Distributed Processing for more information. Example: `-host_elastic user_command`. Runs the specified command to add hosts. The command can include options and must include an icv `-host_add` invocation.
`-i name`	Overrides the library names specified in the `library_name` and `merge_libraries` arguments of the `library()` function. When you use the `-i` command-line option, the `library_name` and `merge_libraries` arguments are redefined. You can specify multiple `-i` command-line options. The `library_name` is replaced by the first `-i` command-line option. The merged libraries are created from subsequent `-i` values, with the first merged library created from the second `-i` command-line option. The format for the merge libraries is set to the same format as for the main library. There is no layer mapping.
`-container`	Wrapper option that runs the IC Validator tool in a container. For more information about the container setup, see the Synopsys Containers page at https://www.synopsys.com/containers or the Synopsys Installation Guide page at https://www.synopsys.com/install.
`-filecache`	Reduces disk usage by storing some files in memory instead of on the disk.
`-3dic` *3dic_design_file*	Specifies a 3DIC design file, such as a top-level **.3dbx** file from 3Dblox, or an IC Validator native format file, to use instead of a runset.

Runset Configuration

Table 4 IC Validator Runset Configuration Command-Line Options

Option	Definition
Runset Configuration	Options to control runset switches, caching, and command line files.

Feedback

Table 4 IC Validator Runset Configuration Command-Line Options (Continued)

Option	Definition
`-D name[=value]`	Defines symbolic names with an optional value. The default is 1. **Note:** When using the `-D` command-line option to pass string values (`icv -D FOO="BAR"`), you must ensure the quotation marks are not removed by the shell (`icv -D FOO='"BAR"'`), or put`-D FOO="BAR"` into a clf file.
`-U name`	Undefines symbolic names that were previously defined with the `-D` command-line option.
`-I directory`	Specifies the preprocessor include directories. This command-line option takes only one path, but you can use multiple `-I` options on a command line. When you specify multiple paths, order matters. For example, you have the file aaa.rh in both my_path1 and my_path2. • When you specify `-I my_path1 -I my_path2`, the aaa.rh file from my_path1 is used. • When you specify `-I my_path2 -I my_path1`, the aaa.rh file from my_path2 is used. Include directories are searched after the current working directory is searched. If an include directory is not in the install directory, then for the `-I` command-line option, you must specify the absolute path of the directory. See File inclusion in Table 57 of the Preprocessor section for more information.

Feedback

Table 4 IC Validator Runset Configuration Command-Line Options (Continued)

Option	Definition

`-clf` *file*	Allows you to add command-line options to your IC Validator run using a text file. It performs a left-to-right replacement with the options from the specified file. This command-line option takes only one file, but you can use multiple `-clf` options on the command line.
	The content of the text file, such as wildcard expansion and the line continuation character (\), are not processed by the UNIX shell.
	For example,
	`% icv -clf normal.clf -il "one"`
	`-clf select_violation_rules.clf drc22nm.rs`
	The text file normal.clf contains
	`# Run Options`
	`-I PXL`
	`# Run on three cores`
	`-host_init 3`
	The text file select_violation_rules.clf contains
	`# Select Options`
	`-svc "*vC1xor*" -svc vC1not`
	Using these two files is equivalent to the command line
	`% icv -host_init 3 -I PXL -il "one"`
	`-svc "*vC1xor*" -svc vC1not drc22nm.rs`
	The IC Validator tool expands text in the file that has a $ sign at the start of a variable as an environment variable. For example, the MY_INCLUDE environment variable set on the command line as
	`% setenv MY_INCLUDE /u/path`
	could be used in the text file to set the path for runset include files:
	`-I $MY_INCLUDE/include`

`-append_runset`	Specifies that additional runset files must be concatenated on to the end of the main runset.
`-runset_config` *config_file*	Specifies the runset configuration, which can be used to change options and compare functions. The configuration file might contain options and compare functions which can change the behavior of the runsets. The functions behave as if they were inserted into the runset between the end of the options and the beginning of the assigns. The compare functions behave as if they were inserted into the runset just before the `compare()` function, with the exception of the `compare()` function itself, which is modified by the contents of the configuration file. In general, variables are not supported, except in the case of the few required arguments in compare functions in which they are ignored, for example, the compare matrix.
`-cache-only`	Generates a cache file in the local cache directory for a runset without running it through the engine.

Feedback

Table 4 IC Validator Runset Configuration Command-Line Options (Continued)

Option	Definition
`-ece`	Enables the reporting of additional error codes to the shell. See Exit Codes. See the documentation for the shell you are using regarding how to capture the error code status.
`-norscache`	Disables runset caching. See Runset Caching for more information.

DRC

Table 5 IC Validator DRC Command-Line Options

Option	Definition
DRC	Options to select and unselect specific violations, delta voltage tracing, and incremental DRC.
`-svc` *violation_comment* `-uvc` *violation_comment* `-svn` *violation_name* `-uvn` *violation_name* `-sfn` *function* `-ufn` *function*	Selects the violations that are executed during the run. There are three categories of selection rules: select or suppress by violation comment (`-svc` and `-uvc`), by violation name (`-svn` and `-uvn`), and by function name (`-sfn` and `-ufn`). The arguments • Can include the wildcard * and range expressions using [ ]. • Must be enclosed in quotation marks if it uses a wildcard *. (This enclosure prevents processing by the UNIX shell.) These options take only one argument but you can specify multiple options on a command line. For example, `% icv -svc "*96A1*" -svc "*11B16*"` • For each category, selection is based on what is selected and not suppressed. Final selection is the intersection of the violations selected in each category. • If the selection option is not used for a category but the suppression option is used, then all violations excluding those suppressed are selected for that category. • If both selection and suppression options are missing for a category, then for that category all violations are selected. If you do not use any of the options, the whole runset is selected. • If the final selection of violations is empty, the run terminates with an error.
`-smu`	The `-svc` and `-uvc`,`-svn` and `-uvn`, and `-sfn` and `-ufn` specifications (along with any runset-based `violation_name_[un]select()` or `violation_comment_select()` specifications), construct a matched list of rules by comment, name, and function, respectively. By default, these three lists are intersected to produce the finalized "selected" violation list. Using the `-smu` command-line option, this list is joined by union to produce the finalized "selected" violation list.

Feedback

Table 5 IC Validator DRC Command-Line Options (Continued)

Option	Definition
`-bct` *option*	Blocks forward data flow analysis through the outputs of specified commands. The selections for *option* are: • `extent`. The extent functions whose output is blocked are `cell_extent()`, `cell_extent_layer()`, `layer_extent()`, and `chip_extent()`. • `connect`. The connect functions whose output is blocked are `connect()`, `incremental_connect()`, `stamp()`, and `stamp_edge()`. • `both`. The output of both extent and connect functions is blocked. • `none`. Forward data flow analysis is not blocked. This value is the default. The `-bct` command-line option must be used with the `-il` or `-iln` command-line options. You can use all three options together. The selection of the specified commands, however, is not blocked. For example, if there is a data flow path from an incremental assign layer to a check that requires output from a `connect()` function, the `connect()` functions are not blocked even if the `-il layer_name -bct connect` command-line options are used.

`-il "`*layer ... layer*`"`	Defines the incremental layers to be used in an incremental-by-layer run by layer name.
	• Put the set of layer names within quotation marks.
	• Specify multiple names with spaces as separators.
	For example,
	`-il "POLY"`
	`-il "POLY DIFF METAL"`
	Note:
	Layer names specified in the `name` argument of any assign function are used for matching incremental assign layer names.
	When the `name` argument of an assign function is not used, the layer name for the assign is from the left side assignment, for example, *`layer_name`* `= assign(...).`
	If more than one layer name is specified, incremental processing occurs if one or more layer names are found in the ASSIGN section. Layer names that are not found are ignored. If none of the specified layers are found in the ASSIGN section, the IC Validator tool exits with an error message.
	The violations that are based, directly or indirectly, on the incremental layers are executed in the IC Validator run. The other violations are not executed in the IC Validator run.

Feedback

Table 5 IC Validator DRC Command-Line Options (Continued)

Option	Definition
`-iln` *`layer_number1`*`[-`*`layer_number2`*`]` `[,`*`datatype1`*`[-`*`datatype2`*`]] ...`	Defines the incremental layers to be used in an incremental-by-layer run by number in the runset. • Put the set of layer numbers within quotation marks. • Specify multiple layers with spaces as separators. • Use dashes to specify a range of layers or data types. • Use commas to separate the layer numbers from the data types. For example, to specify (layers 4 to 5, data types 0 to 1), (layer 3), and (layers 1 to 2, datatype 1) use the following command-line option: `-iln "4-5,0-1 3 1-2,1"` If more than one layer number is specified, incremental processing occurs if one or more layer numbers are found in the assign function. Numbers that are not found are ignored. If none of the specified layer numbers are found in the assign function, the IC Validator tool exits with an error message. Violations that are based, either directly or indirectly, on incremental layers are executed in the IC Validator run. The other violations are not executed in the IC Validator run.
`-dv_trace_error_nets`	Enables delta voltage net tracing automatically. **Note:** When using this command-line option, IC Validator automatically inserts a `dv_trace_error_nets()` call at the end of the input runset. If the runset already contains `dv_trace_error_nets()`, the call is ignored.
`-idrc_f` *`format`*	Defines the input format of baseline layout for auto-incremental DRC flow.
`-idrc_i` *`path`*	Defines path to baseline layout for auto-incremental DRC flow.
`-idrc_c` *`cell_name`*	Defines top cell in baseline layout for auto-incremental DRC flow.

LVS / PERC

Table 6 IC Validator LVS / PERC Command-Line Options

Option	Definition
LVS / PERC	Options to specify netlist path, format, and top cell, netlist comparison, short extraction, equiv. files and other LVS execution options.

Feedback

Table 6 IC Validator LVS / PERC Command-Line Options (Continued)

Option	Definition
`-s` *schematic*	Overrides the file name specified in the `schematic_file` argument of the `schematic()` function. The `schematic_library_file` argument of the `schematic()` function is not overwritten. When you use the `-s` command-line option, you must also use the `-sf` command-line option. If the schematic file is not found when LVS compare is run, the IC Validator tool exits with a message stating that it cannot open the file to read.
`-sdn` *device_names*	Runs only those device commands whose device names match the specified `device_names` patterns.
`-sf SPICE | VERILOG | ICV`	Specifies the input schematic netlist format, overriding the format specified in the `schematic_file` argument of the `schematic()` function. The `schematic_library_file` argument of the `schematic()` function is not overwritten.
`-stc` *cell*	Overrides the top cell in the schematic netlist, overriding the `schematic_top_cell` argument of the `compare()` function and the `cell` argument of the `schematic()` function.
`-C`	Runs only a netlist comparison between the existing schematic netlist and a previously generated layout netlist, using the information specified in the runset. DRC and extraction are not done. This command-line option is intended for rerunning compare when the design data has not changed but some compare settings are changed. Do not place compare-related functions, such as extract functions, filter, and merging, in a conditional statement if its execution depends on some condition that is derived by layers. In this situation, the data dependency triggers the restart of data creation, and the IC Validator tool reruns almost everything in the runset.
`-ex`	Runs device extraction only. The `compare()` function is not invoked, and as a result, all functions that use the `compare()` function output are not run.
`-ndcOnly`	EERC netlist domain checking only, which reuse previously generated netlist file and disable all polygon, device and compare functions.
`-nvn`	Enables the IC Validator tool to run a netlist-versus-netlist (NVN) flow without a runset. When this option is used, any runset on the IC Validator command line is not read.
`-check_all_properties`	Enables the IC Validator Compare to automatically perform property checking on all extracted properties of the devices, regardless of whether they are specified in `check_property`. All existing `check_property()` and `check_property_off()` are ignored.

Feedback

Table 6 IC Validator LVS / PERC Command-Line Options (Continued)

Option	Definition
`-disable_compare_device_settings [FILTER | MERGE | SHORT | ALL]`	Depending on the specified value, some or all compare device settings are ignored so that the netlists are compared without those settings.
`-nvn_netlist_mode mode`	Specifies the netlist-versus-netlist (NVN) flow mode as layout-versus-layout (LVL) or schematic-versus-schematic (SVS). When set to LVL, the valid pin and device names used in both input netlists are always referenced to layout names in the runset. When set to SVS, the names are always schematic names. When this command-line option is not specified, the pin and device names are schematic and layout, respectively.
`-create_lvs_short_output [output_limit|ALL]`	Creates VUE output and generates LVS Short Finder output. You can specify to report only a set number of shorts or all shorts. The default is 200. A *cellname*.vue file is created. The output is in the ./run_details/short_out_db directory. When there is a set limit, the shorts are reported in the following order, with the lowest level within the hierarchy reported first: 1.Shorts between power and ground 2.Shorts between power and power, or between ground and ground 3.Shorts between power and signal, or between ground and signal 4.Shorts between signal and signal This output can also be enabled by using the `create_lvs_short_output` argument of the `error_options()` function. However, you can only change the number of shorts using the command-line option.
`-define_empty_cell_as_device [NONE|BOTH]`	Specifies whether empty cells (X-card instances) are ignored (the default) or recognized as GENDEV primitive devices provided that all required conditions are met. See the `compare()` function in the *IC Validator Reference Manual* for more information. For an empty cell to be treated successfully as a device, the IC Validator tool must be able to equate it automatically between the schematic and layout netlists.
`-dhe true | false`	Enables or disables dual hierarchy device extraction. Use this command-line option to overwrite `init_device_matrix(dual_hierarchy_extraction = )` in the runset.
`-dvp LVS | ALL`	Extracts `ALL` (the default) properties or the ones used only by LVS compare. Use this command-line option to overwrite `init_device_matrix(device_properties = )` in the runset.
`-use_alternate_starrc`	Switches between the user-specified and default-embedded versions of StarRC.

Feedback

Table 6 IC Validator LVS / PERC Command-Line Options (Continued)

Option	Definition

`-e` *file*	Adds the `equiv_options()` and `lvs_black_box_options()` function calls in the specified file into the runset. The file can contain one or more calls to these functions.
	The only IC Validator functions that can be in the file are the `equiv_options()` and `lvs_black_box_options()` functions, but other valid PXL commands can be used. The following example shows the use of a `foreach` loop:
	`equiv_cells : list of string = {"cellA", "cellB", "cellC", "cellD", "cellE", "cellF"};`
	`foreach( cell in equiv_cells ){`
	`  equiv_options({{cell, cell}});`
	`}`
	If there are calls to the `equiv_options()` and `lvs_black_box_options()` functions in the runset, the equivalence points from the runset and the equivalence file are accumulated.

`-gse true | false`	Enables or disables the generation of system-generated equivalence cell pairings. Use this option to override `lvs_options(generate_system_equivs = …)`.
`-inter_die`	Enables inter-die LVS checking on 2.5-D and 3-D designs.
`-ln` *filename*	Overrides all of the file names specified in the `layout_file` argument of the `read_layout_netlist()` function. You must use the `-ln` command-line option in conjunction with the `-lnf` command-line option.
`-lnf` *layout_netlist_format*	Overrides all the formats specified in the `layout_file` argument of the `read_layout_netlist()` function.
`-lvs_property_epsilon` *10-5/10-6*	Defines the epsilon value used for floating point comparison when comparing the property value.

Feedback

Table 6 IC Validator LVS / PERC Command-Line Options (Continued)

Option	Definition

`-nwl` *`wire_log_file`*	Provides a wireLog file to guide IC Validator on how to resolve shorts. Overrides the wireLog file generated automatically by NetTran during IC Validator run.
	You can generate a wireLog file from your schematic netlist. See the "Creating a WireLog File Using NetTran" in the *IC Validator LVS User Guide* for more information about how you can manually generate a wireLog file running NetTran with its command-line option.
	The wireLog file allows the IC Validator tool to use the same net as NetTran when there is a short. The IC Validator tool assigns the *newNetName* to the net.
	Example of wireLog file syntax:
	`# format: cellName origNetName newNetName`
	` add4 VDD2 VDD`
	` cs_add VDD5 VDD`
	` cs_add n36 VDD`
	` nor2b VDD6 VDD5`

`-starrc_cores [`*`number_of_cores`*`]`	Sets the number of cores to use when automatically launching StarRC from an IC Validator run.
`-udn` *`device_names`*	Does not run those device commands whose device names match the specified `device_names` patterns.
`-vueshort [`*`integer`*`|ALL]`	Extracts either a specified number of shorts or all shorts. The default is 200. The extracted data is in the ./run_details/short_out_db directory. When there is a set limit, the shorts are reported in the following order, with the lowest level within the hierarchy reported first: 1.Shorts between power and ground 2.Shorts between power and power, or between ground and ground 3.Shorts between power and signal, or between ground and signal 4.Shorts between signal and signal You can also set the shorts to be extracted using the following arguments of the `text_net()` function: `create_short_finder_nets` and `short_debugging = {vue_short_finder=true, output_limit=integer}`.
`-zn`	Compresses all netlists generated by the `netlist()`, `write_spice()`, `write_customized_spice()` and `write_xref_spice()` functions. Compresses equivalence netlists generated by the `compare()` function.

Feedback

Explorer

Table 7 IC Validator Explorer Command-Line Options

Option	Definition
Explorer	Options to control Explorer mode for DRC and LVS.
`-explorer standalone|auto[:TierNumber]| tiers[:TierNumber]`	Runs explorer mode with either `standalone`, `auto`, or `tiers` mode. See IC Validator Explorer DRC for more information.
`-explorer_lvs_layers [METAL | ALL |user_custom_cell_list]`	Specifies which layers to check in Explorer LVS. See IC Validator Explorer LVS for more information.
`-explorer_lvs`	Runs the Explorer mode for LVS. See IC Validator Explorer LVS for more information.
`-explorer_lvs_cells [AUTO | ALL]`	Specifies which cells to check in Explorer LVS. See IC Validator Explorer LVS for more information.
`-explorer_lvs_checks [SHORTS | ALL]`	Performs only text-based short checks or text-based short checks along with LVS compare. See IC Validator Explorer LVS for more information.
`-explorer_lvs_preserve_cells "<space separated cellnames>"|*"`	Specifies the cells that are to be preserved from being deleted or black-boxed in Explorer LVS so that those cells can be included in Explorer LVS. See IC Validator Explorer LVS for more information.
`-explorer_lvs_preserve_cell_file "file path"`	Specifies the file input version of `-explorer_lvs_preserve_cells`. One line per cell to be preserved.
`-explorer_lvs_preserve_hierarchy_cells "space separated cellnames"|*`	Specifies the equiv_cells and their child equiv_cells that should not be deleted or black-boxed when running with -explorer "*lvs*|*lvs:1*|*lvs:2*".
`-explorer_lvs_preserve_hierarchy_cells_file "file path"`	Specifies the file input version of `-explorer_lvs_preserve_hierarchy_cells`. One line per cell.
`-explorer_lvs_text_report SHORT|OPEN|ALL`	Specifies which types of text violation are to be reported. `SHORT` (default) for text short, `OPEN` for text open, or `ALL` for text short/open/unused.
`-explorer_script path_to_file`	Specifies the file used to add resources after tier 0 and tier 1 are completed. See IC Validator Explorer DRC for more information.

Feedback

Table 7 IC Validator Explorer Command-Line Options (Continued)

Option	Definition
`-explorer_tiers_file` *`path_to_file`*	Specifies the file to customize so that user-defined tiers can be passed to the Explorer DRC run. See IC Validator Explorer DRC for more information.

Input Database

Table 8 IC Validator Input Database Command-Line Options

Option	Definition
Input Database	Options to control input layout path, layer mapping, magnification, small cell mode, as well as NDM and OA options.
`-64`	Runs the IC Validator tool with 64-bit coordinates support. See Support for 64-Bit Coordinates for more information. This option is available with Elite licensing. See Which IC Validator Licenses Do I Need? for information about IC Validator licensing.
`-add_shape_file`	Replaces the list of new polygon files to the `add_shape_files` in the `library()` function. This command-line option can be used more than one time and all specified polygon files are used.
`-lf` *`layermappingfile`*	Specifies the layer mapping file. This file maps the runset layer numbers to the corresponding layer numbers in the library. See Layer Mapping.
`-m` *`factor`*	Overrides the database magnification factor in the `library()` function.
`-ml` *`filename`*	Puts into the runset a call to the *`format`*`_merge_library_options()` function that is defined in *`filename`*. The *`format`* can be `milkyway`, `ndm`, or `openaccess`. The syntax for this file is exactly that of the *`format`*`_merge_library_options()` function. Any *`format`*`_merge_library_options()` function in the runset is overwritten.
`-p` *`library_path`*	Overrides the library path specified in the `library()` function.

Feedback

Table 8 IC Validator Input Database Command-Line Options (Continued)

Option	Definition
`-rsi` *`list_of_requested_info`*	Reports input library information. Supports only GDSII and OASIS input formats. Reports • GDSII last modified and last accessed • md5sum computed with md5sum -b *file* • format of the input library • path to the input library • command used for the library input Enumerators must be in a comma-separated list, and the entire list must be within quotation marks. See the `report_streamfile_information` argument of the `run_options()` function. **Note:** The `-rsi` command-line option overrides its runset setting in the `run_options()` function.
`-small_cell`	Overrides automatic small cell determination and forces small cell mode to run as fast as possible on small designs. To prevent slowdown on larger designs, the IC Validator tool exits with an error if it determines that the input design is too large.
`-sub_cell`	Indicates a run on a small subcell inside a larger OASIS layout. Avoids reading the entire layout.
`-milkyway_version`	Prints Milkyway schema versions supported by the current IC Validator version.
`-mw_default_layers` *`mw_layer_number(s)`*	Defines Milkyway layers by number that if colored, also is mapped as if uncolored.
`-ndm_version`	Prints NDM schema versions supported by the current IC Validator version.
`-ndm_default_layers` *`number(s)`*	The `-ndm_default_layers` command-line option functionality is equivalent to the `-write_default_layers` option functionality of the `write_gds` command in the IC Compiler II tool. This command-line option specifies that the listed layers for which the geometry and text information corresponding to any color-mask configuration are read by the IC Validator tool as default layer data as well. When this command-line option is specified along with layer mapping information for color and mask data, all of the geometry and text layer information is read twice; first as the layer-datatype pair obtained from the layer map considered as color-mask data, and second as the layer-datatype pair obtained from the layer map considered as non-color or mask data.
`-ndm_design_label` *`label`*	Design label for the input NDM library.

Feedback

Table 8 IC Validator Input Database Command-Line Options (Continued)

Option	Definition
`-ndm_exclude_layers` *`name(s)`*	Defines NDM technology layer names to exclude when reading from NDM libraries. This is a comma separated list of technology layer names with wildcard matching supported and does not apply to replacement libraries specified by `ndm_merge_library_options()`.
`--ndm_layer_map_format icc2 | icc_default | icc_extended`	The `-ndm_layer_map_format` command-line option functionality is equivalent to the `-layer_map_format` option functionality of the `write_gds` command in the IC Compiler II tool. This command-line option specifies the way the input layer mapping file is interpreted by the IC Validator tool for NDM input layouts, particularly for Milkyway-formatted layer mapping files that have different layer numbers for system layers in normal versus extended layer mode. The default if the `-ndm_layer_map_format` command-line option is not called is to automatically detect between the Milkyway and NDM formats. If the current layer map file being used is in the Milkyway format, the `-ndm_layer_map_format` command-line option must be used, and either the `icc_default` or the `icc_extended` command-line option must be selected.
`-ndm_multiple_rules_per_layer`	Specifies that data read from an NDM layout with a layer mapping file must map to all the destinations specified in the layer mapping file.
`-ndm_shape_use_for_pg_mesh`	Specifies a list of NDM shape uses to be mapped with the `pg_mesh` use type in the NDM layer mapping file. This can be any combination of “lib_cell_pin_connect macro_pin_connect ring stripe”. Specify empty string (“”) to turn off mapping of the `pg_mesh` use type. The default is “ring stripe”.
`-oa_version`	Prints the current OpenAccess dynamic library version.
`-oa_cell_map` *`cellmap`*	Overrides the `cell_mapping_file` argument specified in the `openaccess_options()` function.

Feedback

Table 8 IC Validator Input Database Command-Line Options (Continued)

Option	Definition
`-oa_color_mapping` *option*	Specifies how to use the locked and unlocked anchor bit for the OpenAccess color layer mapping. The selections for *option* are • `ignore`. Uses the color mappings for which the locked or unlocked keyword is not specified, and ignores the color mappings for which locked or unlocked is specified. • `strict`. Uses the color mappings for which the locked or unlocked keyword is specified, and ignores the color mappings for which locked or unlocked is not specified. • `full`. Sets all shapes that are read with color to locked. The mapping for this setting is the same as the mapping for the `compatibility` setting. • `compatibility`. Follows the industry standard. Ignores color mappings for which the colorAnnotate keyword is located after the color specification. When no color mapping rule matches a color shape, the mapping is used without color. • `lockOnly`. Specifies that for a colorable layer, if the `-fullColor` option is `true` (the default) in the technology database for that layer, only shapes that are colored and locked is mapped. Uncolored or unlocked shapes on that layer is discarded. If the `-fullColor` option is `false` for a layer, the layer mapping is honored for that layer.
`-oa_dm6`	Uses OpenAccess shared libraries that are compatible with DM6 plug-ins. The default is `-oa_dm6`.
`-oa_layer_map` *layer_mapping_file*	Overrides the `layer_mapping_file` argument setting specified in the `openaccess_options()` functions.
`-oa_object_map` *oa_object_mapping_file*	Overrides the `object_mapping_file` argument specified in the `openaccess_options()` function.
`-oa_lib_defs` *library_definition_file*	Overrides the OpenAccess `library_definition_file` argument setting specified in the `library()` function.
`-oa_view` *view_name*	Overrides the `view` argument specified in the `openaccess_options()` function.

Output

Table 9 IC Validator Output Command-Line Options

Option	Definition
Output	Options to specify directory locations for group files and `run_details`, error limit per check, heat map, and other output options.

Feedback

Table 9 IC Validator Output Command-Line Options (Continued)

Option	Definition
`-g group_path`	Overrides the group path name specified in the `group_path` argument of the `run_options` function. The IC Validator tool does not allow you to set this value to the current working directory (.). This command-line option takes only one directory. If you use multiple `-g` options, the IC Validator tool uses the directory from the last one. **Note:** The directory provided to this command-line option must be visible to all hosts. If you provide a local cache directory, see the `-host_disk` command-line option.
`-rd run_details_path`	Overrides the `run_details` path. IC Validator tool does not allow you to set this value to the current working directory (.). The tool reports an error if the current directory is specified for `run_details`.
`-w path`	Redefines the current working directory,
`-email dashboard`	Sends an email to the user using ICV Dashboard every time the percent complete line is updated in the icv.log file.
`-progress_script`	Specified script is run every time the percent complete line is updated in the icv.log file.
`-cpydb db_name db_path`	Appends an error classification database to the error classification databases specified in the runset. • *db_name* specifies the error classification database name. • *db_path* specifies the error classification database path.
`-elpc non negative number | ERROR_LIMIT_MAX`	Error limit per check (default is 100).
`-slpc true | false`	Enable or Disable the stop limit per check. Use it to override `error_options(stop_check_at_error_limit = ...)`. For information on functions that do not support `error_options(stop_check_at_error_limit=true)`, see the `error_options()` function in the IC Validator Reference Manual.
`-ep prefix`	Overrides the prefix for all error cell names specified in the write functions.
`-hm`	Enables the generation of violation heat maps. **Note:** This command-line option triggers the `error_sampling_per_check` argument with the same integer limit as the `error_limit_per_check` argument if not specified. See the `error_sampling_per_check` argument of the `error_options()` function for more information.

Feedback

Table 9 IC Validator Output Command-Line Options (Continued)

Option	Definition
`-hm_cell` *`cell_name`*	Enables the generation of violation heat maps for cells which match the given cell name patterns, in addition to the top cell. If any specified cell is exploded due to performance optimization, the heat map is not be generated for that cell. See the `heat_map_cells` argument of the `error_options()` function for more information. **Note:** The `-hm` command-line option must also be provided for any heat maps to be generated.
`-ned`	Overrides the `report_error_details` argument of the `error_options()` function. Use this command-line option to not report violation details to the LAYOUT_ERRORS and TOP_LAYOUT_ERRORS files. The information is still stored in the error database (PYDB), but only the violation summary section is written to the LAYOUT_ERRORS file.
`-errdm_file` *`output_file`*	Specifies that the tool writes all violations found during the run to the given file which might be imported into the Fusion Compiler error browser.
`-errdm_hierarchy` *`TOP|INST`*	When writing Fusion Compiler error output, specifies how errors reported in lower level cells are to be treated. The default is `INST`, which translates a single instance of each error to the top cell in the error file. Specifying `TOP` only outputs violations that are reported in the top cell.
`-errdm_resolution` *`microns`*	Specifies the resolution in microns of the Fusion Compiler error file. This option is required for writing Fusion Compiler error files when the NDM design in Fusion Compiler has a resolution other than the default resolution of 0.0001.

Debug

Table 10 IC Validator Debug Command-Line Options

Option	Definition
Debug	Options to output additional debug information (cpp and pcr files), run selected runset lines and turn on the layer debugger.
`-cpp` *`outputfile`*	Reads the input runset, writes a preprocessed version of the runset to the output file, and then immediately terminates. All preprocessor directives and macros are fully processed in the output runset, and all included user (non-IC Validator) files are expanded. All comments are removed from the output file and encrypted sections are preserved.

Feedback

Table 10 IC Validator Debug Command-Line Options (Continued)

Option	Definition
`-pcr` *`output_file`*	Reads the runset and writes compiler-checked PXL output after function inlining and loop unrolling. The output can be pruned by using violation selections, run-only execution, or the incremental-by-layer `-il` and `-iln` command-line options. See Run-Only Execution for more information.
`-ro`	Enables run-only processing. See Run-Only Execution.
`-rofl` *`file:line`*	Enables run-only command selection based on the file and line number.
`-verbose`	Prints the output of the individual commands to the screen while the IC Validator tool is running rather than only percentage complete messages. The information printed is the same as the information reported to the summary file. However, the information is not displayed in the same order. The screen output reports the information in the order of processing by the IC Validator tool, while the content of the summary file is sorted to match the sequential order of the runset. The default is 3.

`-vfn` *`function_name`*
`-vfn ALL`
	Reports evaluated parameter values of the specified runset function in the summary file, and on the screen when the `-verbose` option is used. This command-line option takes only one function name but you can use multiple `-vfn` options on a command line. You can also use the `-vfn ALL` option to report the values for all runset functions.
	The message contains the function name, actual value of each parameter of the function, and the return type of the function. This information is useful for debugging the input parameter value of any function call. Here is an example of the output of the `-vfn library` option:
	`<runset>:<line_no>: library (`
	`  library_name = CELL_LEVEL`
	`  format = MILKYWAY`
	`  cell = TOP`
	`  library_path = <path>`
	`  library_definition_file =`
	`  magnification_factor = 1.`
	`) returning void`

`-layer_debugger`	Enables the storing of layer debugger output without the use of the VUE. For compatibility with VUE, this command-line option enables the creation of VUE output (as if by the `-vue` command-line option). Also, to match the VUE layer debugger behavior, runset selection is performed based on the union of commands selected by each of the following categories: violation comment (`-svc` and `-uvc`), violation name (`-svn` and `-uvn`), function name (`-sfn` and `-ufn`), and run-only selection.
`-debug`	Use IC Validator PXL Debugger.
`-debug_source` *`fileName`*	File containing commands for the IC Validator debugger. See IC Validator PXL Debugger.

Feedback

Optimization Controls

Table 11 IC Validator Optimization Controls Command-Line Options

Option	Definition
Optimization Controls	Options to control runset and hierarchy optimization, and operation priority.
`-ndg`	Enables use of streamlined processing flow, which bypasses some stages of runset compilation, generation of dependencies between functions, and runset optimizations that require dependency analysis. Without dependency generation, the IC Validator tool cannot run in distributed processing mode, and all functions in the runset are executed sequentially on a single host. Use the `-ndg` option with small cells, where the incremental cost of running all functions is significantly lower than the time and memory needed to fully optimize a complex runset. When using the `-ndg` option, any user function that invokes an inline runset function must be tagged with the `inline` qualifier. In general, IC Validator OPTIONS and ASSIGN section functions fall into this category. See Inlined Functions for a complete list of the runset functions that are tagged with the `inline` qualifier. See the `inline` qualifier description in Qualifiers. When using the `-ndg` option, you cannot use the dual-hierarchy extraction flow.
`-nro`	Disables runset optimizations.
`-pec KEEP | EXPLODE`	Overrides the setting of the `partially_exploded_cells` argument in the `error_options()` function.
`-prune_assign`	The IC Validator tool reads only the required runset layers from input design, and creates the required runset layers. When this command-line option is not used, the IC Validator tool reads all the runset layers from input design. Whether or not you use the `-prune_assign` command-line option, unnecessary runset layers are not reported in the summary file. See Pruning Layers for information about the `prune_assign` compiler directive.
`-rfc`	Reduces fill cell creation in back end fill compression.
`-prioritize_file_line`	Enables prioritization based on the file and line number.
`-prioritize_violation_comment` *violation comment*	Enables prioritization based on the violation comment.

Feedback

Licensing

Table 12 IC Validator Licensing Command-Line Options

Option	Definition
Licensing	Options to specify licensing schemes, license waiting, and keys.
`-lic_token2023`	Forces token based licensing scheme.
`-lic_apex`	Forces use of Apex licensing scheme. See Apex Licensing.
`-lic_base`	Forces use of Base licensing scheme. See Base Licensing.
`-lic_elite`	Forces use of Elite licensing scheme. See Elite Licensing.
`-license_mode` *`run_available`* `|` *`run_and_add`* `|` *`exit`* `|` *`exit_and_retry`*	Defines the tool behavior when all multicore licenses are not available at startup. The default is `run_and_add`. See User Controls for more information.
`-license_wait_duration` *`value`* `[d|h|m|s]`	Specifies the maximum time the tool waits for licenses to become available. The default is `0`. See User Controls for more information.
`-keys` *`key(s)`*	Defines the keys that control environment variable access. Use a comma-separated list with no spaces to provide multiple keys.
`-nlc`	Turns off license caching.

General

Table 13 IC Validator General Command-Line Options

Option	Definition
General	Options to control when IC Validator exits, flat execution, recipe inclusion, and output usage and version information.
`-exit`	Terminates a running IC Validator process and provides partial results. The `-exit` command-line option • Allows IC Validator to clean up before exiting. This includes: - Any file on a local disk or network disk - Any file in /dev/shm • Allows IC Validator to generate partial results. The LAYOUT.ERRORS and RESULTS files contain a banner indicating that IC Validator exited early.
`-copilot`	Executes the Knowledge Assistant System

Feedback

Table 13 IC Validator General Command-Line Options (Continued)

Option	Definition
`-time_limit` *hours*	Supports exiting with partial results after a user-specified time limit is reached (one-hour minimum) **Note:** This command-line option requires the Elite license.
`-usage`	Displays usage message and exits.
`-flat`	Enables a flattened run. This command-line option works identically to adding `hierarchy_options(flatten={"*"});` to the runset. See the `flatten` argument of the `hierarchy_options()` function for more information.

`-recipe`	Adds a recipe from a recipe file in the following format:
	`-recipe "/path/to/recipe:recipe name"`
	`-recipe "local_recipe_file: recipe name 2"`

`-V`	Prints the program version.
`-Version`	Prints version numbers for the individual components and the library versions of Synopsys software.

Compiler Directives

Support for 64-Bit Coordinates

The IC Validator tool converts coordinate data to a signed 32-bit integer coordinate space. Large die sizes with a small database resolution might require a larger integer space, causing a coordinate overflow error to occur.

To increase the coordinate space, use the `-64` command-line option. Only use the `-64` option when it is required; this is because the memory usage increases.

The value limits for a 32-bit integer are:

- Maximum positive value: 2,147,483,647
- Maximum negative value: -2,147,483,648

In the following example,

- R_{coord} is a real number xy coordinate value.
- I_{coord} is the coordinate converted to an integer.

Feedback

Assume the following values:

- R_{coord} = 25,000.1234 mm (2.50001234 cm from 0.0000)
- Working resolution = 2x

For a database resolution of 0.0001 mm, the R_{coord} value does not overflow the 32-bit integer coordinate space:

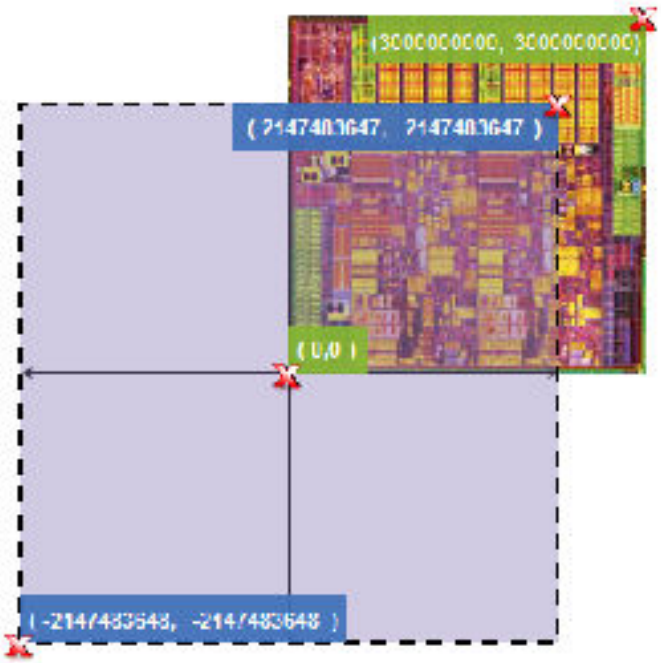
I_{coord} = 25,000.1234*(1/0.0001)*2 = 500,002,468 < 2,147,483,647

For a database resolution of 0.00001, the R_{coord} value overflows the 32-bit integer coordinate space:

I_{coord} = 25,000.1234*(1/0.00001)*2 = 5,000,024,680 > 2,147,483,647

Not all overflows, however, are created equally. Before using the `-64` option, make sure that you are using all of the available coordinate space. As shown in Figure 1, using only positive coordinate values artificially limits the maximum die size.

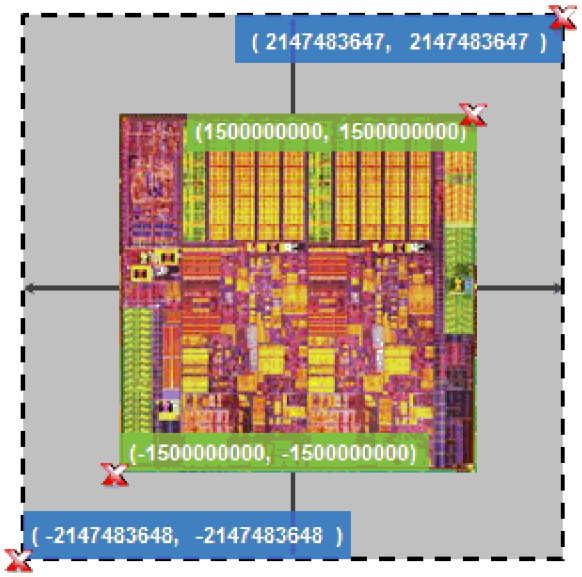
Figure 1 Overflow of the 32-Bit Integer Coordinate Space



Center the die about the origin to maximize available coordinate space, as shown in Figure 2. Using the `-64` option is not needed to prevent an overflow.

Feedback

Figure 2 No Overflow of the 32-Bit Integer Coordinate Space



The IC Validator error database is enhanced with a variable-width coordinate format that can accommodate either 32-bit or 64-bit integer coordinates. This applies to both the error databases (PYDB) and the error classification databases (cPYDB), which might be exported for DRC error classification.

The pydb_export utility is enhanced with the ability to convert error classification databases (cPYDB) among the various coordinate formats to maintain compatibility as necessary.

Note:

Current and future IC Validator tools retain support for reading error databases with the older fixed 32-bit or 64-bit coordinate formats. However, IC Validator tools prior to version L-2016.06 are not able to read the new coordinate format.

Run-Only Execution

Follow these steps for run-only execution.

1. Identify the sections of the runset for which you want the required runset checks with these directives:

   ```
   #pragma runonly begin

   #pragma runonly end
   ```

   You can define multiple run-only sections within a runset.

2. Call the IC Validator tool with `-ro`, the run-only command-line option, to
   a. Parse the runset.
   b. Prune the runset functions that do not have data flow into run-only code sections.
   c. Execute the pruned runset.

   Note:

   If the IC Validator tool does not find a run-only section within a runset, the runset is not executed.

3. Alternatively, to stop execution after the runset is parsed and pruned for run-only execution, use the `-pcr` command-line option along with `-ro`. For example,

   ```
   % icv -ro -pcr outfile
   ```

4. Use the `-nro` command-line option when using pruned run-only output as the source runset. If you do not use the `-nro` option, default pruning optimizations can eliminate functions inside run-only code sections.

   Note:

   All `#pragma` run-only directives remain in the pruned run-only output.

   The presence of `#pragma` run-only directives in the pruned run-only output enables the `icv -ro runonly.rs` command line to be used for execution of the output pruned runonly.rs runset.

Environment Variables

The $ sign preceding a variable indicates the variable is an environment variable. For example,

```
$ENV_ABC
```

The variable must be set in the environment; otherwise a parse error occurs. All environment variables are considered strings. For example,

```
if( $ENV_ABC == "XYZ") {
   x = abc(XYZ);
}
else {
   x = abc(NOT_XYZ)
}
```

Feedback

Providing Additional Information to the Compiler

The compiler directive `#pragma envvar` interfaces with environment variables to

- Print a list containing the names and the values of each specified environment variable.

  `#pragma envvar report` *`variable ... variable`*

  If any variable does not exist, an error occurs. For example,

  ```
  error: #pragma envvar references undefined environment variable 'e'
  ```

- Probe the environment for the named variable.

  ```
  // input file
  #pragma envvar default x "aa"
  #pragma envvar default y "bb"
  #pragma envvar default z $x "__" $y // Use $X notation to
                                      // reference defined
                                      // variables
  #pragma envvar report x y z

  // output from compiler
  a.rs:4: x=aa
  a.rs:4: y=bb
  a.rs:4: z=aa__bb
  ```

  - If the named variable exists, no action is taken.
  - If the named variable does not exist, the value of the variable in the environment is set to the concatenation of the rest of the line. The concatenation must be a sequence of string literals or references to other environment variables, or both.

  `#pragma envvar default` *`variable ... variable`*

Note:

If there is more than one `#pragma envvar default` statement that defines the same variable, subsequent occurrences see the variable as already set in the environment, as set by the first occurrence.

Creating Preprocessor Definitions From Environment Variables

The preprocessor `#pragma PXL define_from_envvar` creates a preprocessor definition from an environment variable, if the environment variable is set. The syntax is

`#pragma PXL define_from_envvar` *`cpp_var env_var`*

The preprocessor definition named by *cpp_var* is given the value of the environment variable named by *env_var*, as if by a `#define` command. If the environment variable is

Feedback

not set, no action is taken, such that *cpp_var* remains undefined or retains its previous definition.

The preprocessor `#pragma PXL define_from_envvar` takes two arguments:

- The first argument is the name of the preprocessor definition to redefine if the environment variable is set.
- The second argument is the name of the environment variable to get the value from.

```
#include <icv.rh>

#define str(x) #x
#define qstr(x) str(x)

#if defined(FOO_DEF)
note("Before pragma, defined as: " + qstr(FOO_DEF));
#else
note("Not defined before pragma");
#endif

#pragma PXL define_from_envvar FOO_DEF FOO_ENV

#if defined(FOO_DEF)
note("After pragma, defined as: " + qstr(FOO_DEF));
#if FOO_DEF == 2
note("... and satisfies test for == 2");
#elif FOO_DEF == 3
note("... and satisfies test for == 3");
#endif
#else
note("Not defined after pragma");
#endif
```

When the run is as follows:

```
% export FOO_ENV=3
% icv -verbose -D FOO_DEF=2 define_from_envvar.rs
```

The output is:

```
define_from_envvar.rs:10: Before pragma, defined as: 2
define_from_envvar.rs:17: After pragma, defined as: 3
define_from_envvar.rs:21: ... and satisfies test for == 3
```

Notice the original value is `2` because of the preprocessor definition that passed on the command line. The `define_from_envvar` pragma redefines `FOO_DEF` to a value of `3`. If `FOO_ENV` is not set, `FOO_DEF` is has a value of `2`. If `FOO_DEF` is also not defined on the command line, it remains undefined for the remainder of the preprocessing.

Feedback

Use the `#if definition(identifier, string, case_sensitivity)` function in conjunction with the `define_from_envvar` pragma to support the querying of the *$envvar_name* environment variable to determine if it has a value of *string*.

In the user environment, set this:

```
setenv BEOL_STACK 9M_3Mx_2Fx_2Dx_1Gx_1Iz_LB
```

The runset looks like this:

```
#pragma PXL define_from_envvar d_ENV_BEOL_STACK BEOL_STACK

#if definition(d_ENV_BEOL_STACK, "9M_3Mx_2Fx_2Dx_1Gx_1Iz_LB", 0)
...
#endif
#if definition(d_ENV_BEOL_STACK, "8M_2Mx_2Fx_2Dx_1Gx_1Iz_LB", 0)
...
#endif
```

- The third argument, `#case_sensitivity`, is used on *define_identifiers* `(definition(identifier, string, case_sensitivity)`, and is explicitly `#defined` in the runset, without the need for *envvar*. With a value of `0`, this argument performs a case-insensitive comparison between the identifier's value and the specified string. With a value of `1`, this argument performs a case-sensitive comparison.

Pruning Layers

The `#pragma prune_assign` compiler directive tells the IC Validator tool to read-only the required runset layers from input design and create the required runset layers. You can use this compiler directive to enable or to disable pruning:

```
#pragma icv compiler [enable|disable] prune_assign
```

When this compiler directive is not used or is `disable` and the `-prune_assign` command-line option is not used, the IC Validator tool reads all of the runset layers from the input design. In all cases, unnecessary runset layers are not reported in the summary file. The setting of the `prune_assign` directive in the runset has priority over the `-prune_assign` command-line option.

Prioritizing the Scheduling of Functions

The `#pragma prioritize` directive increases the scheduling priority for the function and the functions it depends on. Place the `#pragma prioritize` directive immediately before

Feedback

the targeted command or code block. This directive might also assign extra threads to those prioritized functions. For example,

```
#pragma prioritize
 not_rectangles(M5);
```

The `#pragma prioritize` directive increases the priority for both of the `not_rectangles()` functions, as shown in the following example:

```
#pragma prioritize
ERR_M4 @= { @ "M4 non-rectangles";
  not_rectangles(M4);
  not_rectangles(M5);
}
```

See Table 15 for related command-line options.

Runset Configuration

Runset configuration changes the configuration of Options and Compare functions. See the `-runset_config` command-line option in IC Validator Command-Line Options for more information.

Runset configuration supports the following IC Validator functions:

Table 14 Supported Runset Configuration Functions

OPTIONS functions	Compare setup functions	compare()
`compatibility_options()`	`check_property()`	`compare()`
`drc_black_box_options()`	`check_property_off()`	
`debug_options()`	`filter()`	
`error_options()`	`filter_off()`	
`gds_options()`	`get_filtered_devices()`	
`hierarchy_auto_options()`	`map_capacitor()`	
`hierarchy_options()`	`map_gendev()`	
`incremental_options()`	`map_inductor()`	
`layout_drawn_options()`	`map_nmos()`	
`layout_grid_options()`	`map_np()`	

Feedback

Table 14 Supported Runset Configuration Functions (Continued)

OPTIONS functions	Compare setup functions	compare()
milkyway_options()	map_npn()	
ndm_options()	map_pmos()	
oasis_options()	map_pn()	
openaccess_options()	map_pnp()	
pattern_options()	map_resistor()	
prototype_options()	match()	
resolution_options()	merge_parallel()	
run_options()	merge_parallel_off()	
text_net_global_options()	merge_series()	
text_options()	merge_series_off()	
equiv_options()	recognize_gate()	
lvs_black_box_options()	recognize_gate_off()	
lvs_options()	recaclulate_property()	
net_options()	short_equivalent_nodes()	
waiver_options()	short_equivalent_nodes_off()	
milkyway_error_options()	short_equivalent_stack_nodes()	
	short_equivalent_stack_nodes_off()	

The functions in the `-runset_config` file are processed in accordance with the following criteria:

- All of the OPTIONS functions are effectively inserted into the runset after the last option.
- All of the “compare setup” functions are inserted just after the last “compare setup” function.
- Any options specified in `compare()` itself are used to update compare in the runset.

Feedback

Runset Insertion

Runset insertion allows you to customize the main runset using secondary runsets without the need for modifying the main runset. If the main runset includes the necessary pragma statements to indicate where code can be inserted, you can provide additional runset code in a separate file.

The syntax in the main runset is *#pragma icv insert <label>*. This pragma can only be used one time per *<label>*. All of these pragmas must be on their own line.

After the insertion pragma is specified, you can mark a block of code by setting *#pragma icv label <label> begin* and *#pragma icv label <label> end*. These pragmas must come after the corresponding insert pragma. They cannot be nested or overlap. Multiple blocks of code using the same *<label>* are allowed and are inserted into the same destination in the order they are defined.

	Inputs		Output

foundry_runset.rs	#include <icv.rh>
	
	#pragma icv insert options
	M1 = assign({{1}});
	V1 = assign({{2}});
	M2 = assign({{3}});
	#pragma icv insert assigns
	
	V1_M1 @= { V1 not M1; }
	V1_M2 @= { V1 not M2; }
	#pragma icv insert checks
	
	final_checks();
		internal runset to be parsed	#include <icv.rh>
			
			M1 = assign({{1}});
			V1 = assign({{2}});
			M2 = assign({{3}});
			boundary = assign({{255}});
			
			V1_M1 @= { V1 not M1; }
			V1_M2 @= { V1 not M2; }
			CHECK_BOUNDARY @=
			{
			not_inside(M1,boundary);
			not_inside(M2,boundary);
			}
			
			final_checks();

user_patch_file.rs	#pragma icv label assigns begin
	boundary = assign({{255}});
	#pragma icv label assigns end
	
	#pragma icv label checks begin
	CHECK_BOUNDARY @=
	{
	not_inside(M1,boundary);
	not_inside(M2,boundary);
	}
	#pragma icv label checks end		

Exit Codes

Table 15 lists the general exit codes.

Feedback

Table 15 General Exit Codes

Number	Type	Explanation
0	EXIT_COMPLETE	Run completed.
30	EXIT_COMPLETE_WERROR	Run completed with errors reported in the *cell*.LAYOUT_ERRORS file or an LVS failure reported in the *cell*.LVS_ERRORS file. This exit code is enabled only when the `-ece` command-line option is used. A clean layout with only waived errors or methodology warnings reported does not issue this exit code.
32	EXIT_USER_INT	Run was interrupted by the user with Ctrl+C.
33	EXIT_USER_ERR	Error caused by the user, such as a parser error or a file, was not found.
34	EXIT_OS	System error, such as division by zero, bus error, disk full, out of swap space, or could not create directory or file.
35	EXIT_PANIC	An internal fatal error occurred.
44	EXIT_DATABASE_ERROR	Error indicating that a database processing error has occurred and only partial results are available.

Table 16 lists the license manager related exit codes.

Table 16 License Manager Related Exit Codes

Number	Type	Explanation
64	EXIT_LIC_TOO_MANY	Too many licenses checked out.
65	EXIT_LIC_NOT_ALLOWED	Station not authorized to run this product.
66	EXIT_LIC_NO_REPLY	Could not contact license server.
67	EXIT_LIC_DENIED	License denied, no explanation given.
(-1)	EXIT_NONE	An uninitialized exit status variable.

Feedback

Inlined Functions

When you run the IC Validator tool with the `-ndg` command-line option,

- Any user function that invokes an inline runset function needs to be tagged with the `inline` qualifier. See Table 17 for a list of inline runset functions. For example,

```
my_options : inline function ( void ) returning void
{
   . . .
   error_options(...);
   gds_options(...);
   . . .
}
```

- User functions that call runset functions with cell lists and call runset functions used to construct cell lists also must be tagged with the `inline` qualifier. See Table 18 for a list of functions. For example,

```
add_strings_to_list : inline function (string_list : in_out list of
 string, new_strings : list of string) returning void
{
   string_list = string_list.merge(new_strings);
}

get_cell_extent : inline function (layer1 : polygon_layer, cells :
 list of string) returning result : polygon_layer
{
   result = cell_extent(layer1, cells);
}

cells : list of string = {};

add_strings_to_list(cells, {"AX", "AY", "AZ"});
add_strings_to_list(cells, {"BX", "BZ"});

part_extent = get_cell_extent(part, cells);
```

If a user function needs to be tagged with the `inline` qualifier but it does not have the qualifier, the runtime error is:

```
error: unable to extract cell list for no explode options
```

Table 17 is a list of the runset functions that are tagged with the `inline` qualifier.

Table 17 Runset Functions Tagged With the inline Qualifier

`assign()`	`assign_edge()`
`assign_openaccess()`	`assign_openaccess_edge()`

Feedback

Table 17 Runset Functions Tagged With the inline Qualifier (Continued)

assign_openaccess_text()	assign_text()
equiv_options()	error_options()
exclude_milkyway_cell_types()	exclude_milkyway_net_types()
exclude_milkyway_route_guide_layers()	exclude_milkyway_route_types()
exclude_ndm_blockage_types()	exclude_ndm_design_types()
exclude_ndm_net_types()	exclude_ndm_shape_uses()
gds_options()	get_netlist_connect_database()
get_top_cell()	hierarchy_auto_options()
hierarchy_options()	incremental_options()
init_device_matrix()	instance_property_number()
layout_drawn_options()	layout_grid_options()
layout_integrity_by_cell()	layout_integrity_by_marker_layer()
layout_integrity_options()	library()
library_create()	library_import()
lvs_black_box_options()	lvs_options()
milkyway_merge_library_options()	milkyway_options()
ndm_options()	ndm_merge_library_options()
net_options()	oasis_options()
openaccess_options()	
pattern_options()	prototype_options()
read_layout_netlist()	resolution_options()
run_options()	text_options()

Table 18 lists the runset functions with cell lists and functions used to construct cell lists.

Table 18 Runset Functions With Cell Lists and Used to Construct Cell Lists

cell_extent()	cell_extent_layer()

Feedback

Table 18 Runset Functions With Cell Lists and Used to Construct Cell Lists (Continued)

`compare()`	`copy_by_cells()`
`create_ports()`	`flatten_by_cells()`
`text_origin()`	

Distributed Processing

You can run the IC Validator tool in a variety of different distributed processing situations, including using multiple CPUs on a single host and using multiple hosts.

Running the IC Validator tool with distributed processing provides the following features:

- Scalability increases with the number of CPUs used
- Flexibility for adding CPUs to an existing IC Validator job
- Simultaneous multithreading

Distributed processing is described in the following sections:

- Specifying Host Names and CPU Counts
- License Requirements

Specifying Host Names and CPU Counts

The IC Validator tool takes advantage of multiple CPUs on a host by determining automatically whether to start parallel tasks or multithread. You must communicate to the tool how many CPUs on the host you want to use. You can do this by using the `-host_init` command-line option.

For example, to use 8 CPUs on a single host, use the following command-line option:

```
icv -host_init 8 runset.rs
```

It is possible to specify which host to run on. In the preceding example, the host name was omitted, so the local host was assumed. If you specify a different host name, the IC Validator tool starts on that host. In the following example, 8 CPUs are used on hostA.

```
icv -host_init hostA:8 runset.rs
```

Because a remote host is specified, the IC Validator tool must make a connection to that host. By default, the tool attempts to use the `rsh` command to make this connection. If a different connection method is needed, use the `-host_login` command-line option (some

Feedback

IT environments might disable `rsh` access or require a different connection method). In the following example, 8 CPUs are used on hostA and `ssh` is used to connect to the host.

```
icv -host_init hostA:8 -host_login /bin/ssh runset.rs
```

Note:

IC Validator uses `rsh` as the host login program by default when available. Using `rsh` can be considered a security risk and is frequently discouraged and disabled. If `rsh` is disabled in your environment, IC Validator automatically uses `ssh`. Disabling `rsh` at the system level is the recommended way to enforce the security policy for all users on your network. You can also set the `-host_login` command-line option to always use `ssh` when launching IC Validator (for example, `-host_login ssh`).

You can list more than one host. In this case, the IC Validator tool uses all of the hosts listed, with the number of CPUs specified. In the following example a total of 32 CPUs are used: 8 CPUs on the local host; 8 on hostA; and 16 CPUs on hostB (because the `-host_login` command-line option is NOT specified, `rsh` connects to the hosts).

```
icv -host_init 8 hostA:8 hostB:16 runset.rs
```

You can enable additional threads on the hosts that support simultaneous multithreading (SMT). In the following example, on a host with two logical CPUs per core, the IC Validator tool dedicates 8 CPUs for commands and 8 additional CPUs for threading.

```
icv -host_init 8 -host_thread_mode SMT
```

See the `-host_init` and `-host_login` command-line options in IC Validator Command-Line Options for more information.

Note:

If the host name is specified but the CPU count is omitted, ALL of the CPUs are used on the host.

After the job is running, you can add more CPUs. Hosts and CPUs are specified in the same way using the `-host_add` command-line option. Notice that this job must be run from the same working directory as the original IC Validator job. If the run_details directory was specified in the initial run using `-rd`, this directory must also be specified on this run using `-rd`.

Modifying Resources for a Running Job

You can add more CPUs to a running IC Validator job. Hosts and CPUs are specified with the `-host_add` command-line option, in the same way that they are specified with the `-host_init` command-line option.

The following example adds 16 CPUs on hostC to the running job.

Feedback

```
icv -host_add hostC:16
```

Note:

When using `-host_add`, most of the other command-line options are unnecessary and not allowed. See `-host_add` in Table 3 for more information.

You can also remove hosts from a running job. Hosts are specified using the `-host_remove` command-line option. It is only possible to remove entire hosts. The following example removes all of hostC (all of its CPUs) from the running job.

```
icv -host_remove hostC
```

You can instruct IC Validator to automatically add and remove hosts from the running job. Hosts are added when too many commands are queued and are removed when the hosts are idle. This can be specified when the job is originally started or after the job is running. You must provide the command that IC Validator should use to acquire a new host. The command must include an `icv -host_add` on the acquired host.

For the `-host_init`, `-host_add`, and `-host_elastic` command-line options, this IC Validator call must be run in the same working directory as the running job. Furthermore, if the run_details directory was specified in the initial running run using `-rd`, this directory must also be specified on this run using `-rd`. All command-line options are ignored with the following exceptions: `-rd` is used by all three of these options, and `-host_login`/`-host_disk` is used by `-host_add`.

Running in LSF

When running in LSF, the syntax and preceding examples are still appropriate. The following example shows how to launch an LSF job that starts IC Validator on 16 CPUs on a single host, using the `bsub` command.

```
bsub -n 16 -R "span[ptile=16]" <bsub options> icv -host_init 16
runset.rs
```

Similarly, to add 16 CPUs on one host to a running IC Validator job, use the `-host_add` command-line option. For example,

```
bsub -n 16 -R "span[ptile=16]" <bsub options> icv -host_add 16
```

As a convenience, the IC Validator tool can automatically determine the host configuration and sets `-host_login` to `blaunch`. To enable this feature, specify `LSF` as the as the argument to `-host_init`. In the following example, 16 CPUs are requested on a single host using `bsub` and are identified using the `LSF` argument of the `-host_init` command-line option.

```
bsub -n 16 -R "span[ptile=16]" <bsub options> icv -host_init LSF
runset.rs
```

Feedback

This feature is most useful when requesting jobs on multiple hosts. The following example works with multiple hosts as well (2 hosts, 16 CPUs each):

```
bsub -n 32 -R "span[ptile=16]" <bsub options> icv -host_init LSF
runset.rs
```

The `LSF` argument also works with the `-host_add` command-line option. In this example, a total of 64 CPUs are added to a running job: 8 hosts each with 8 CPUs.

```
bsub -n 64 -R "span[ptile=8]" <bsub options> icv -host_add LSF
```

To elastically add hosts to the job, specify a bsub command to the `host_elastic` command-line option that acquires a machine and starts the IC Validator tool. In the following example, a single host is requested with 8 CPUs. (Depending on your environment, you might need to provide additional arguments to bsub.)

```
bsub -n 8 'span[hosts=1]' icv -host_add LSF
```

Add this entire string to the `host_elastic` command-line option or specify in a separate script. The complete IC Validator command can look as follows:

```
icv -host_init 16 -host_elastic "bsub -n 8 'span[hosts=1]' icv -host_add
 LSF" runset.rs
```

Running in Univa Grid Engine (UGE/SGE)

When running in SGE, the syntax and preceding examples are still appropriate. The following example shows how to launch an SGE job that starts IC Validator on 16 CPUs on a single host using the `qsub` command.

```
qsub -pe mt 16 <qsub options> icv -host_init 16 runset.rs
```

Similarly, to add 16 CPUs on one host to a running IC Validator job, use the `-host_add` command-line option. For example,

```
qsub -pe mt 16  <qsub options> icv -host_add 16
```

As a convenience, the IC Validator tool can automatically determine the host configuration and sets `-host_login` to `qrsh -inherit`. To enable this feature, specify `SGE` as the argument to `-host_init`. In the following example, 16 CPUs are requested on a single host using qsub and are identified using the `SGE` argument of the `-host_init` command-line option.

```
qsub -mt 16 < qsub options> icv -host_init SGE runset.rs
```

This feature is most useful when requesting jobs on multiple hosts. The following example works with multiple hosts as well (2 hosts, 16 CPUs each):

```
qsub -pe dp16 32 <qsub options> icv -host_init SGE runset.rs
```

Feedback

The SGE argument also works with the -host_add command-line option. In this example, a total of 64 CPUs are added to a running job: 8 hosts each with 8 CPUs.

```
qsub -pe dp8 64 <qsub options> icv -host_add SGE
```

To elastically add hosts to a running job, use the host_elastic command-line option with a qsub command that requests a machine and launches IC Validator. For example, to request a single host with 8 CPUs, use the following command:

```
qsub -cwd -V -pe mt 8 icv -host_add SGE
```

Include this entire string as the argument to the host_elastic option, or specify it in a separate script. The complete IC Validator command can look like the following:

```
icv -host_init 16 -host_elastic "qsub -cwd -V -pe mt 8 icv -host_add SGE"
 runset.rs
```

Running in SLURM

When running in SLURM, the syntax and preceding examples are still appropriate. The following example shows how to launch an SLURM job that starts IC Validator on 16 CPUs on a single host using the sbatch command.

```
sbatch --cpus-per-task=16 --nodes=1 <sbatch options> run_add.sh

run_icv.sh:

icv -host_init 16 runset.rs (or) icv -host_init SLURM runset.rs
```

Similarly, to add 16 CPUs on one host to a running IC Validator job, use the -host_add command-line option. For example,

```
sbatch --cpus-per-task=16 --nodes 1 <sbatch options> run_add.sh

run_add.sh:

icv -host_add 16 (or) icv -host_add SLURM
```

As a convenience, the IC Validator tool can automatically determine the host configuration and set -host_login to srun with the appropriate command-line options. To enable this feature, specify SLURM as the argument to -host_init. For example, to request 16 CPUs on a single host using sbatch:

```
sbatch --cpus-per-task=16 --nodes 1 <sbatch options> run_icv.sh

run_icv.sh:

icv -host_init SLURM runset.rs
```

Feedback

This feature is most useful when requesting jobs on multiple hosts. The following example works with multiple hosts as well (2 hosts, 16 CPUs each):

```
sbatch --cpus-per-task=16 --nodes 2 <sbatch options> run_icv.sh

run_icv.sh:

icv -host_init SLURM runset.rs
```

Similarly, for adding hosts to a running job, use:

```
icv -host_add SLURM
```

To elastically add hosts to a running job, use the `host_elastic` command-line option with an `sbatch` command that requests a machine and launches IC Validator. For example, to request a single host with 8 CPUs, use the following command:

```
sbatch --cpus-per-task=8 --nodes 1 <sbatch options> run_add.sh

run_add.sh:

icv -host_add SLURM
```

Include this entire string as the argument to the `host_elastic` option, or specify it in a separate script. The complete IC Validator command can look like the following:

```
icv -host_init 16 -host_elastic "sbatch --cpus-per-task=8 --nodes 1
 <sbatch options> run_add.sh" runset.rs
```

License Requirements

For Elite licensing, see Table 19 for examples of the number of licenses needed.

Table 19 Elite Licensing for Multicore Distributed Processing

Situation	Licenses required	Explanation
DRC on 16 cores	4 licenses	First license enables 4 cores, plus 3 additional licenses that enable 4 cores each
LVS on 4 cores	1 license	
Metal fill on 9 cores	3 licenses	First two licenses enable 4 cores each, plus 1 core from third license
Pattern matching on 62 cores	16 licenses	First 15 licenses enable 4 cores each, plus 2 cores from the 16th license

For more information about IC Validator licensing, see the Licensing and Resource Requirements chapter in the *IC Validator User Guide*.

Feedback

Runset Caching

The IC Validator runset caching feature reduces the startup time of the IC Validator tool. After a runset is parsed and optimized, the internal data structures for the runset are saved to a runset cache file. Subsequent IC Validator runs that use the same runset load the runset cache file rather than parsing and optimizing the runset again.

You can turn runset caching off by using the `-norscache` command-line option. You might want to use this option when debugging a run.

Note:

If you use runset caching with parasitic extraction (PEX) functions for multiple corners, the IC Validator tool does not flag any file naming conflicts until runtime.

See Using IC Validator Command-Line Options With Runset Caching on page 72 for information about using command-line options with runset caching.

Cache Files

When the IC Validator tool writes a runset cache file, the following messages are written to screen indicating the path of the cache file:

```
Saved runset to cache: /remote/us54h1/myname/.icvrscache/
e3b32d5de337d888106672bbba370041-rod-3908.rscache
Parsing Finished
Runset Compile Time=0:00:03 User=3.37 Sys=0.16 Mem=0.182 GB
```

When the IC Validator tool reads a runset cache file, the following messages are written to the screen indicating the path of the cache file that was read:

```
Loaded runset from cache:
/remote/us54h1/myname/.icvrscache/
e3b32d5de337d888106672bbba370041-rod-3908.rscache
Cached Runset Compile Time=0:00:00 User=0.25 Sys=0.04 Mem=0.108 GB
```

You can determine which cache file in your cache directory is associated with a specific runset. When you look at the content of the cache file, you'll see something like this text:

```
Cached runset file created from ICV runset:
/remote/seg-scratch/myname/S475264/pxl.ev
. . .
```

The second line is the full path of the runset that was used to generate the cache file.

Feedback

Configuring Runset Caching

You can use several environment variables to configure runset caching.

`ICV_RUNSET_CACHE_DIR_SHARED`

Sets the shared runset cache directory used by the IC Validator tool.

```
% setenv ICV_RUNSET_CACHE_DIR_SHARED directory
```

This cache directory is not user-specific. Multiple users can point to the same shared cache directory. When enabled, the IC Validator tool performs the following checks:

- Checks the shared cache directory for cache that either matches or misses the runset criteria
- If missed, the IC Validator tool checks the local cache directory
  - If matched, the IC Validator tool uses the local cache file.
  - If missed, the IC Validator tool compiles and stores cache in the local cache directory (local to the user).

You must move cache files to the shared directory manually so that they can be used by multiple users.

`ICV_RUNSET_CACHE_DIR`

Sets the runset cache directory used by the IC Validator tool when storing cache files.

```
% setenv ICV_RUNSET_CACHE_DIR directory
```

The default runset cache directory is ~/.icvrscache.

Runset cache directories are user-specific. That is, multiple users cannot point to the same runset cache directory by setting their ICV_RUNSET_CACHE_DIR variables to the same directory.

`ICV_RUNSET_CACHE_MAXFILES`

Limits the number of runset cache files that the IC Validator tool writes to the runset cache directory.

```
% setenv ICV_RUNSET_CACHE_MAXFILES num_files_to_keep
```

When the IC Validator tool is run, it does not delete any preexisting runset cache files until either the `ICV_RUNSET_CACHE_MAXFILES` or the `ICV_RUNSET_CACHE_MAXSIZE` limit is reached. When either limit is reached, the

Feedback

IC Validator tool deletes files in the least-recently used order; that is, cache files that have been loaded recently are retained while the older files are deleted.

`ICV_RUNSET_CACHE_MAXSIZE`

Limits the amount of disk space the IC Validator tool can use to store runset cache files. By default, the IC Validator tool limits the space used for runset cache files to 200 MB.

```
% setenv ICV_RUNSET_CACHE_MAXSIZE max_size
```

The *max_size* value is a numeric value followed by an optional letter indicating the unit type. You can use a unit type of K for kilobytes, M for megabytes, G for gigabytes, or T for terabytes. If no unit type is used, the *max_size* value is assumed to be in megabytes.

`ICV_RUNSET_CACHE_DEBUG`

Reports messages that help you pinpoint a configuration problem or the source of the unexpected behavior. Use this variable if runset caching is behaving in an unexpected way. The messages show configuration parameters and give a reason for loading or not loading each candidate cache file in the runset cache directory.

```
% setenv ICV_RUNSET_CACHE_DEBUG
```

For example,

```
...
RUNSET-CACHE: cache directory : ./RSCACHE/
RUNSET-CACHE: cache max size  : 307200
RUNSET-CACHE: cache max files : -1
RUNSET-CACHE: version string  : <platform> <ICV version> <date>
RUNSET-CACHE: opendir() opened cachedir : ./RSCACHE/
RUNSET-CACHE: readdir() read filename :
3cb51a5a7c3d1da800be5d6ee862da29-rod-16372.rscache
RUNSET-CACHE: 0 cache files matched runset MD5:
1de3cbc1c76cd3f7b05877718fa83cae
RUNSET-CACHE: no matching cache file found!
```

`ICV_DISABLE_RUNSET_CACHE`

Disables runset caching.

```
% setenv ICV_DISABLE_RUNSET_CACHE
```

You can also use the `-norscache` command-line option.

Feedback

Matching a Runset to a Cache File

The IC Validator tool matches a runset to an existing runset cache file only when the following criteria are met:

- Preprocessing directives must match the directives stored in the cache file. Some of these directives are internally generated by the IC Validator tool, but others come directly from the IC Validator command line via the `-D`, `-U`, and `-I` options.
- The `-svn`, `-uvn`, `-svc`, `-uvc`, `-sfn`, and `-ufn` command-line options have not changed between runs.
- The preprocessed runset must match that of the cache file, as determined by a hashing algorithm.
- The current IC Validator version must match that of the cache file.
- The endianness of the architectures that IC Validator is run on during the cache save and the current run (the cache load attempt) must match. For example, AIX and Sun are big endian platforms and, therefore, can share a cache file.
- UNIX environment variables referenced in the runset must have the same values as they had when the cache file was generated.

Note:

Using the `-ln` and `-lnf` to override file names and formats specified in the `layout_file` argument of the `read_layout_net()` function does not cause cache misses.

Running Knowledge Assistant Using IC Validator

To launch the IC Validator Knowledge Assistant from the command line, use the `icv -copilot` command.

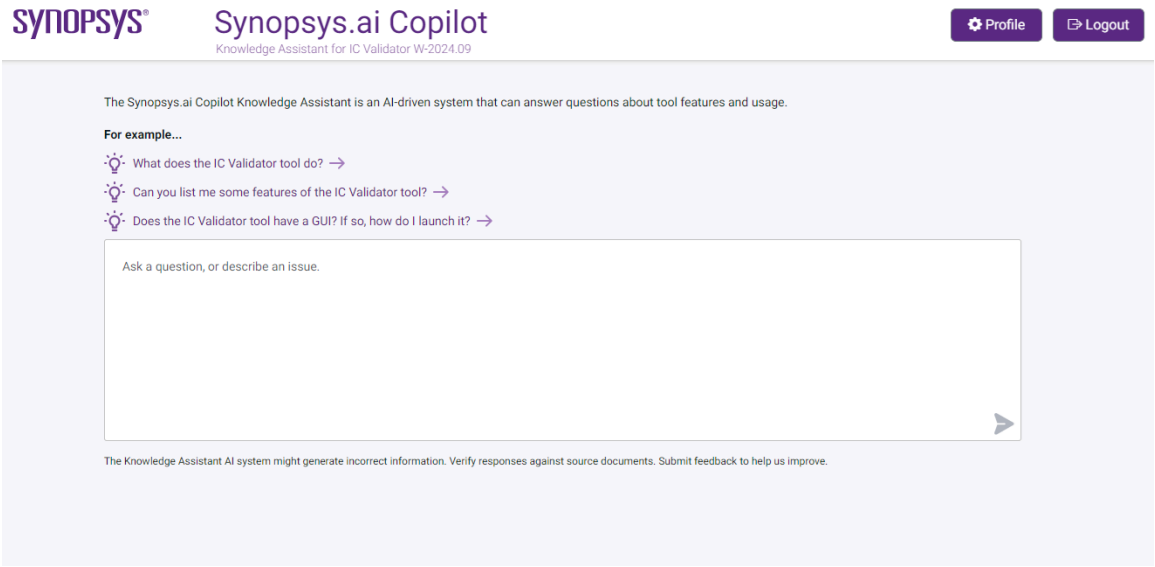
Before running this command, ensure that the `SNPSAI_COPILOT_HOME` environment variable is correctly set. If the `snpsai_copilot` binary cannot be located through the `SNPSAI_COPILOT_HOME` environment variable or the system's search path, IC Validator displays the following error message and terminates:

```
Error: 'snpsai_copilot' could not be found in env var SNPSAI_COPILOT_HOME
```

No additional command-line arguments should be used with the `-copilot` option, as any extra arguments are ignored.

You can run the Knowledge Assistant in the foreground using `icv -copilot` or in the background using `icv -copilot &`. After approximately 10–15 seconds, the Knowledge Assistant is launched, as shown in Figure 3.

Figure 3 IC Validator Knowledge Assistant



IC Validator automatically exits when you close the Knowledge Assistant GUI.

The IC Validator Knowledge Assistant verifies the presence of any valid IC Validator license but does not check out an IC Validator license. Instead, it checks out a `GenAI_KA_Copilot` license to operate.

Using IC Validator Command-Line Options With Runset Caching

After a runset is run the first time with runset caching enabled, subsequent runs of the same runset normally result in a cache file being read rather than a full parse of the runset.

The content of the runset can be altered by certain IC Validator command-line options, which affects the use of runset caching. These changes cause the IC Validator tool to write a new cache. Anytime the runset is modified, runset caching are affected.

Note:

Options not specifically mentioned in the following sections are generally compatible with runset caching.

Feedback

Compatible Command-Line Options

The following IC Validator options are compatible with runset caching. These options can be added, removed, or modified on the command line. The IC Validator tool still uses the previously generated cache.

```
-c
-g
-i
-p
-f
-rd
-stc
```

Command-Line Options That Affect Runset Caching

The following options cause changes in the content of the preprocessed runset. Therefore, when you add these options to the command line or change their arguments, the preprocessed runset does not hash to the same cache file. The runset parses normally and writes a new cache file. If these options are not changed, subsequent runs of the same runset hash to this cache file.

```
-64
-D
-U
-il
-iln
-sfn
-svc
-svn
-ufn
-uvc
-uvn
-verbose
```

Feedback

Incompatible Command-Line Options

Because of the unique flows used to handle the following options, these options disable runset caching when used:

```
-debug

-debug_source

-ndg
```

The following option affects runset caching because it changes internal options in the parser. When you use this option, the IC Validator tool generates a separate cache file:

```
-nro
```

Layer Mapping

There are two types of layer mapping:

- Runset Layer Mapping: This mapping modifies your *runset* to create a new runset.
- Data Layer Mapping: This mapping modifies the *data* as it is read in by changing the layer and datatype attributes of the data.

You can use both types of mapping when you are running on multiple libraries.

Runset Layer Mapping

Runset layer mapping is the process of substituting layer/datatype numbers that are specified in the ASSIGN section of a runset with the actual layer/datatype numbers of data in a database. You need to map layers, for example, when the layer/datatype numbers in your database are not consistent with the layer/datatype numbers specified in a foundry supplied runset.

Specify the mapping file with the `-lf` command-line option. See IC Validator Command-Line Options on page 25.

The first line of the mapping file must always be

```
RUNSET(*,*) = Library(*,*)
```

This statement maps each layer to itself (identity mapping). This identity mapping ensures that there is a mapping for each layer. Subsequent lines of the mapping file override the identity map for selected layers.

Feedback

For example,

```
Runset(*,*) = Library(*,*)
Runset(15,0) = Library(1,0)
Runset(16,0) = Library(2,0)
Runset(17,0) = Library(3,0)
```

Mapping is specified within the file using the following formats:

Note:

`Runset(i,j)` refers to layer/datatype numbers in the ASSIGN section of the runset.

`Library(i,j)` refers to the actual layer/datatype numbers of the data in the database.

- `Runset(A,B) = Library(C,D)`

  In the following example, an assign reference to layer/datatype (1,5) is replaced with layer/datatype (2,0).

  ```
  Runset(1,5) = Library(2,0)
  ```

- `Runset(A,B) = Library(C1...Cm,D1...Dn)`

  In the following example, any assign reference to layer/datatype (5,0) is replaced with the four layer/datatype pairs (10,0), (10,4), (12,0), and (12,4).

  ```
  Runset(5,0) = Library(10 12,0 4)
  ```

- `Runset(A1...Am,B1...Bn) = Library(C1...Cm,D1...Dn)`

  Note:

  The number of Runset layer elements and Library layer elements must be equal.

  The number of Runset datatype elements and Library datatype elements must be equal.

  In the following example, four layer/datatype substitutions are made:

  assign (2,6) is replaced with (10,0)

  assign (2,8) is replaced with (10,4)

  assign (4,6) is replaced with (12,0)

  assign (4,8) is replaced with (12,4)

  ```
  Runset(2 4,6 8) = Library(10 12,0 4)
  ```

Feedback

The syntax rules for a mapping file are

- Wildcards are allowed in place of a list. For example,

  ```
  Runset(1,*) = Library(2,*)
  ```

  means that assign layer 1, all datatypes, is replaced with layer 2, all datatypes.

- Layer/datatype ranges are specified by a dash. For example,

  ```
  Runset(2 5-8,*) = Library(10-12 17 20,*)
  ```

- List elements must be in ascending order.

When the IC Validator tool is executed with a layer mapping file, any substitution of assign layer/datatype numbers is reported below the assign statement in the *cell*.sum file. Here is an example report:

```
L1_all = assign({ { 11 } })
 Function: assign
 Inputs: L1_all.polygonlayer.0001
 Outputs: L1_all.polygonlayer.0002
Layer list {{11}} was replaced with {{1}}
```

Data Layer Mapping

Use data layer mappings when you merge multiple libraries with corresponding data on different layers or datatypes. For example, if you have a main design library with metal1 on layer 41 and a secondary library with metal1 on layer 31, you need to merge these libraries using data layer mapping so that all metal1 data is read in on the same layer. This layer is layer 41 in this example.

Typically, you specify the main design library with the `library()` function. The `library()` function does not provide data layer mapping capabilities for the main library. However, data layer mapping is available when you specify additional libraries. Therefore, you can map the data from the additional libraries to the corresponding layers and datatypes used by the main design library.

If you use the `library_create()` function instead of the `library()` function, all of the libraries are specified with the `library_import()` function and have data layer mapping available. For the purposes of data layer mapping, you can use any library as the main library and layer map the remaining libraries to that main library.

In addition to data layer mapping, you can also use runset layer mapping in the same run to map the runset layers to the layers used by your main library. The following examples illustrate this scenario.

Feedback

In this example, the main and secondary libraries are specified in the `library()` function:

- The main library, one.oasis, is specified with the `library()` function. It has metal1 on layer 41.
- A secondary library, two.oasis, has metal1 on layer 31.
- The runset assumes that metal1 is on layer 5.

The metal1 data on layer 31 in the two.oasis library is mapped to layer 41 as it is read in. This mapping ensures that all input metal1 data is read onto layer 41. The runset layer mapping then transforms the `assign({{5}})` assignment into `assign({{41}})` to ensure that the run executes as expected. Here is a snippet of the runset:

```
library(library_name    = "one.oasis",
        cell            = "A",
        format          = OASIS,
        merge_libraries = {{library_name = "two.oasis", format=OASIS,
                            layer_map_file="two_layer_map"}});

metal1 = assign({{5}});
```

The two_layer_map file has the data layer mapping for two.oasis:

```
41  31
```

The runset layer mapping file, specified with the `-lf` command-line option, has this mapping:

```
RUNSET(*,*) = LIBRARY(*,*)
RUNSET(5,*) = LIBRARY(41,*)
```

In the following example, the main library is specified in the `library()` function and the secondary library in the `library_import()` function. Use this method to import multiple libraries when you do not want cell merging to occur. Each library remains distinct from the others so that your assigns functions can import data from one library but not another.

- The main library, one.oasis, is specified with the `library()` function. It has metal1 on layer 41.
- A secondary library, two.gds, has metal1 on layer 31.
- The runset assumes that metal1 is on layer 5.

The two_layer_map file and the runset layer mapping are the same as in the previous example. Here is a snippet of the runset:

```
lib1 = library(library_name = "one.oasis",
               cell         = "A",
               format       = OASIS);

lib2 = library_import(library_name   = "two.gds",
```

Feedback

```
                        cell           = "A",
                        cell_prefix    = "TWO_",
                        format         = GDSII,
                        layer_map_file = "two_layer_map");

metal1 = assign({{5}});
```

In the following example, the main library is specified in the `library()` function and the secondary library in the `milkyway_merge_library_options()` function. The libraries are not merged nor kept separate. Milkyway cell data is replaced with the equivalent GDSII data on a cell-by-cell basis. See the `milkyway_merge_library_options()` function for more information.

- The main Milkyway library, one, is specified with the `library()` function. It has metal1 on layer 41.
- A secondary GDSII library, two.gds, has metal1 on layer 31.
- The runset assumes that metal1 is on layer 5.

The two_layer_map file and the runset layer mapping are the same as in the first example. Here is a snippet of the runset:

```
library(library_name = "one",
        cell         = "A",
        format       = MILKYWAY,
        library_path = "./");

milkyway_merge_library_options(
   libraries    = {{"one", "./", {{"two.gds", GDSII,
                                   layer_map_file="two_layer_map"}}}},
   missing_cell = ABORT);

metal1 = assign({{5}});
```

Using PXL Runset Encryption

Runset encryption is a method used to conceal the content of the runset from an end user. Use encryption to limit the end user's knowledge of foundry process parameters or of the steps necessary to perform certain design rule checks or transformations.

Operation

After a runset is created as plain text, certain content can be transformed into encrypted blocks. The content of the encryption blocks can contain PXL constructs and any of the standard preprocessor directives. Plain text is also referred to as clear text.

The pxlcrypt utility is used to encrypt the runsets. It encrypts only blocks of cleartext. This utility is included in the IC Validator release install directory. The syntax is

Feedback

```
% pxlcrypt cleartext_runset encrypted_runset
```

The pxlcrypt utility cannot reverse the encryption process.

The encrypted runset is then delivered to the end user, who executes the IC Validator tool using it.

All runset functions that are encrypted are reported as “encrypted function” in the summary file and on standard output when the `-verbose` option is used. Although the functions are listed as encrypted functions, the machine usage, number of polygons generated, and names of smart-deleted layers (layers that are no longer needed because nothing depends on them) are still reported.

Encryption Directives

The directives that mark the beginning and ending points of an encryption section are

```
#pragma PXL encrypt begin

#pragma PXL encrypt end
```

More than one section of a runset can be encrypted in a runset. However, an encrypted section cannot be within another encrypted section. Here is an example of an encryption section in a runset:

```
...
#pragma PXL encrypt begin
e2 @= { @ "e2 violation";
        external2( l1, l2, distance < 2.0, extension = NONE,
                   look_thru = COINCIDENT);}
#pragma PXL encrypt end
...
```

Partial constructs cannot be encrypted. For example, this partial construct is not allowed:

```
...
external2( l1, l2, distance < 2.0, extension = NONE,
#pragma PXL encrypt begin
          look_thru = COINCIDENT);}
#pragma PXL encrypt end
...
```

The pxlcrypt utility automatically breaks encryption blocks around included files. For example, if the runset has this code:

```
#pragma PXL encrypt begin
...
#include "file1.rh"
...
#pragma PXL encrypt end
```

Then, the code is encrypted as:

```
#pragma PXL encrypt begin
...
#pragma PXL encrypt end
#include "file1.rh"
#pragma PXL encrypt begin
...
#pragma PXL encrypt end
```

2

Licensing and Resource Requirements

This chapter contains the setup instructions for the IC Validator tool. Make sure that you have all the correct directories and files, that your environment accommodates the IC Validator tool, and that you have incorporated the proper licensing information.

The licensing and resource requirements are described in the following sections:

- Licensing Schemes
- License Waiting
- Which IC Validator Licenses Do I Need?
- Resource Requirements

For additional information about Synopsys licensing and installing the IC Validator tool, see the following documents:

- The Synopsys Licensing QuickStart Guide, available at

  http://www.synopsys.com/Support/LI/Licensing/Pages.
- Information about installing the IC Validator tool is available at

  http://www.synopsys.com/Support/LI/Installation/Pages/default.aspx.

Licensing Schemes

The IC Validator tool has three types of licensing schemes. The IC Validator tool is run using one of these schemes:

- Apex Licensing
- Elite Licensing
- Base Licensing

Base licensing is the default scheme. If the job does not use license waiting or the `-lic_apex`, `-lic_elite`, or `-lic_base` command-line options, the IC Validator tool first checks for the lowest tier license required for the run and if not available then checks

Feedback

higher tiers. If the run can use any tier, it first checks for Base licenses. If Base licenses are not available, the tool checks for Elite, and then Apex licenses. If none of these licenses are available, the job ends with a license denied error.

The three licensing schemes can exist on a system as available licensing schemes, but an IC Validator job uses only one of them. They cannot be used together.

If multiple licenses are available, you can use the `-lic_apex`, `-lic_elite` or `-lic_base` command-line options to select the licensing scheme to use for a specific job. See IC Validator Command-Line Options on page 25 for more information.

License Waiting

The IC Validator tool supports license waiting in accordance with the standard Synopsys Common Licensing (SCL) guidelines.

When you run the IC Validator tool with license waiting,

- If only Apex licensing is installed, the job queues for Apex licenses.
- If only Elite licensing is installed, the job queues for Elite licenses.
- If only Base licensing is installed, the job queues for Base licenses.
- If multiple licensing schemes are installed, the job queues for the first of Base, Elite, or Apex that is installed. Other schemes are ignored.
  - If you use the `-lic_base` command-line option, the job queues for Base licenses and the Apex and Elite licenses are ignored.
  - If you use the `-lic_elite` command-line option, the job queues for Elite licenses and the Apex and Base licenses are ignored.
  - If you use the `-lic_apex` command-line option, the job queues for Apex licenses and the Elite and Base licenses are ignored.

User Controls

The IC Validator tool provides user controls for when not all multicore licenses are available at the time a job is submitted.

License_wait_duration. Specifies the maximum time the tool waits for licenses to become available.

License_mode. Determines what happens when multicore licenses are not all available at startup.

Feedback

- `run_available`. The job runs with the partial subset of licenses that is available.
  - If the `-license_wait_duration` command-line option is also set and no licenses are available, the job queues for a license until a non-zero quantity is available, and then runs with those licenses.
- `run_and_add`. The job runs with the partial subset of licenses that is available, and then queues and adds as licenses become available, attempting to achieve all requested licenses.
  - If the `-license_wait_duration` command-line option is also set and no licenses are available, the job queues for licenses until a non-zero quantity is available, and then starts with those licenses. The job then queues and adds as licenses become available, attempting to achieve all requested licenses.
- `exit`. The job exits if the full quantity is not available.
  - If the `-license_wait_duration` command-line option is also set, the job queues and incrementally acquires licenses, and starts if the full quantity of licenses is achieved before the duration expires; otherwise the job exits.
- `exit_and_retry`. The job exits if the full quantity is not available, releases any licenses acquired, and retries.
  - If the `-license_wait_duration` command-line option is also set, the job queues and incrementally acquires licenses, and starts if full quantity is achieved before the duration expires; otherwise, the job exits, release any licenses acquired, and retries.

License Caching

The IC Validator tool caches information as it learns about which license keys are installed on which license servers. In subsequent runs, the cached information might be used to change the order of the license server list used by the job or the order in which the various license schemes are checked, potentially reducing the time it takes to successfully check out licenses for the job.

The cached information expires periodically and is also updated as the tool learns more about which licenses are available on which servers. When the tool changes the priority of license servers or license schemes, it is logged in run_details/licmsg.

By default, the license cache is in the following directory:

```
$XDG_CACHE_HOME/Synopsys/icv/license
```

If $XDG_CACHE_HOME is not set, the license cache is in the following directory:

```
~/.cache/Synopsys/icv/license
```

Set the ICV_LICENSE_CACHE_DIR environment variable to override the location of the license cache.

License caching can be disabled by using the `-nlc` command-line option.

Which IC Validator Licenses Do I Need?

The IC Validator functions in your runset determine the required licenses. The following sections discuss the needed licenses:

- Apex Licensing
- Elite Licensing
- Base Licensing
- Licensing Requirements by Function
- Licensing Requirements by Technology
- Multicore Licensing
- Token Licensing
- Hyperthreading

Apex Licensing

Apex licensing uses the following keys:

- ICValidator-Manager-Apex
- ICValidator2-GeometryEngine
- ICValidator2-CompareEngine

The ICValidator-Manager-Apex key is checked out every time the IC Validator tool is run, and it uses a tiered licensing system, high (Apex), middle (Elite), and low (Base) depending on the technology or function.

Elite Licensing

Elite licensing uses the following keys:

- ICValidator-Manager
- ICValidator2-GeometryEngine
- ICValidator2-CompareEngine

Feedback

The ICValidator-Manager key is checked out every time the IC Validator tool is run, and it uses a tiered licensing system, high (Apex), middle (Elite), and low (Base) depending on the technology or function.

Base Licensing

Base licensing uses the following keys:

- ICValidator-Manager-2020
- ICValidator2-GeometryEngine
- ICValidator2-CompareEngine

The ICValidator-Manager-2020 key is checked out every time the IC Validator tool is run, and it uses a tiered licensing system, high (Apex), middle (Elite), and low (Base) depending on the technology or function.

Licensing Requirements by Technology

Technology groupings can require one or more licenses, as shown in Table 20.

Table 20 Technology Requirements

Technology	License required
DRC functions	Available with all tiers
LVS extraction functions	Available with all tiers
LVS compare functions	Available with all tiers. Uses ICValidator2-CompareEngine
Metal fill functions	Available with all tiers
Antenna	Available with all tiers
Fusion with IC Compiler II/Fusion Compiler (InDesign)	Available with all tiers
Netlist Visualizer	Available with all tiers
Layer Debugger	Available with all tiers
LVS Short Finder	Available with all tiers
DRC Heat Map and Balanced Error Distribution	Available with all tiers
IC Validator Launch	Available with all tiers

Feedback

Table 20 Technology Requirements (Continued)

Technology	License required
Error database (PYDB) queries	Available with all tiers
PERC	Available with all tiers
Live DRC - Custom	Available with all tiers
Live DRC - Digital	Available with all tiers
16-Threaded Commands	Available with all tiers
VUE	Available with all tiers. Uses ICValidator2-GeometryEngine[1]
NetTran and other utilities	Available with all tiers. Uses ICValidator2-GeometryEngine[2]
Pattern matching functions	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
Multiple-Patterning Technology (MPT) functions	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
Explorer DRC	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
Explorer LVS	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
Elastic CPU Management	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
Blackbox DRC	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
3DIC	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
Unified fill	Various subcomponents are restricted to Elite | Apex. Various other subcomponents are restricted to Apex
ML resource estimation	Apex (ICValidator-Manager-Apex)

1. *Only checks for the presence of a valid license. No license is checked out.*
2. *Only checks for the presence of a valid license. No license is checked out.*

Feedback

Licensing Requirements by Function

Individual functions can require one or more licenses, as shown in Table 21.

Table 21 Functions Requirements

Functions (Options)	License required
`adjust_color_graph()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`external2(`	Available with all tiers

`look_thru`
`blocking_layer`
`)`
	Apex (ICValidator-Manager-Apex)

`fill_to_text()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`four_color()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`layout_text_to_net_property()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`net_color_check()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`net_select_error()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`not_within_distance_by_property()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`optional_pattern_markers()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_extract()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_learn()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_library_compare()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_library_lock()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_library_merge()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)

Feedback

Table 21 Functions Requirements (Continued)

Functions (Options)	License required
`pattern_library_read()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_match()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_predict()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pattern_predict_marker()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pex_color_layer_map()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pex_qtf_layer_map()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`pex_qtf_layers()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`reduce_four_color_graph()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`reduce_three_color_graph()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`remove_vias()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`three_color()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`two_color()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`unified_fill(`	Available with all tiers
`different_size_fill_spacing`	Apex (ICValidator-Manager-Apex)
`forbidden_layer`	Apex (ICValidator-Manager-Apex)
`merging_limit_exclude_layers`	Apex (ICValidator-Manager-Apex)
`merging_limit_exclude_layers_secondary`	Apex (ICValidator-Manager-Apex)
`min_fill_region_concave_distance`	Apex (ICValidator-Manager-Apex)
`projecting_layer_shrink`	Apex (ICValidator-Manager-Apex)
`remove_point_touch`	Apex (ICValidator-Manager-Apex)
`)`	Apex (ICValidator-Manager-Apex)
`unified_fill_region(`	Available with all tiers

Feedback

Table 21 Functions Requirements (Continued)

Functions (Options)	License required
`deangle_max_deviation`	Apex (ICValidator-Manager-Apex)
`region_type = UF_BLOCKAGE_REGION )`	Apex (ICValidator-Manager-Apex)
`xref_to_double_property()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`xref_to_double_property()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)
`xref_to_property()`	Elite (ICValidator-Manager) or Apex (ICValidator-Manager-Apex)

Multicore Licensing

The IC Validator tool has Apex, Elite, and Base Multicore licensing requirements, as shown in the following sections:

Apex Multicore Licensing

The IC Validator tool can be run with one or more ICValidator-Manager license. Each license allows you to use up to four CPUs. For example:

A 100 CPU job requires 25 Apex (ICValidator-Manager-Apex) licenses

Elite Multicore Licensing

The IC Validator tool can be run with one or more ICValidator-Manager licenses. Each license allows you to use up to four CPUs. For example:

A 100 CPU job requires 25 Elite (ICValidator-Manager) licenses

Base Multicore Licensing

The IC Validator tool can be run with one or more ICValidator-Manager-2020 licenses. Each license allows you to use up to two CPUs. For example:

A 100 CPU job requires 50 Base (ICValidator-Manager-2020) licenses

Token Licensing

The token based licensing describes the approach for checking out multiple licenses. The token licensing is similar to Multicore. Multiple ICV Manager licenses are consumed based on the number of CPUs, according to a defined ratio.

The following components contribute to the total number of licenses needed for a job.

Total number of token licenses = (Per-CPU Flow * Multicore) + Per-Session Functionality

- **Per-CPU Flow Functionality**: Depending on the type of job, a certain number of licenses are needed which multiply with the multicore licensing.
- **Multicore**: The number of multicore result is a defined multiplier of the Per-CPU functionality
- **Per-Session Functionality**: Some functionality do not multiply with multicore. They consume an additional static license quantity regardless of multicore.

Hyperthreading

The IC Validator tool consumes licenses until the quantity of CPUs is equal to the number of physical cores. For example:

A 100 CPU job requires 25 Apex (ICValidator-Manager-Apex) licenses

Hyperthreading Example

A 20-core host with Hyperthreading enabled presents itself as 40 CPUs, where 4 CPUs = 1 license:

8 CPUs (-host_init 8) : Consumes 2 licenses

12 CPUs (-host_init 12) : Consumes 3 licenses

16 CPUs (-host_init 16) : Consumes 4 licenses

20 CPUs (-host_init 20) : Consumes 5 licenses

24 CPUs (-host_init 24) : Consumes 5 licenses (4 CPUs reserved for Threading)

28 CPUs (-host_init 28) : Consumes 5 licenses (8 CPUs reserved for Threading)

32 CPUs (-host_init 32) : Consumes 5 licenses (12 CPUs reserved for Threading)

36 CPUs (-host_init 36) : Consumes 5 licenses (16 CPUs reserved for Threading)

40 CPUs (-host_init 40) : Consumes 5 licenses (20 CPUs reserved for Threading)

Feedback

Resource Requirements

The initial database file and the files that the IC Validator tool creates and uses determine the system requirements. With some knowledge of the design to be checked, you can estimate physical memory, disk space, and possible swap space requirements for your IC Validator runs.

A contrib directory, which is included with IC Validator releases, provides scripts for analyzing IC Validator performance. The contrib directory is at the top level of the release install directory. *These scripts are not supported.* See the README file in the contrib directory for more information.

Space Requirements

The IC Validator program performs layout checking operations on group files that are created from the design database and stored in the group directory. The IC Validator tool first creates the primary group files, one file for each layer in an assign function of the runset. Each primary group file is a database describing that layer's existence throughout the design, maintaining all hierarchy. As the IC Validator run progresses, output files are generated from certain checks. These outputs are also written as group files. For example, the files in a group directory could be:

```
HRCHY.ERROR     inst_indx.dat   pgate.group    toxcont.group
cont.group      met1.group      poly.group     via.group
gate.group      met2.group      psel.group     well.group
inst_data.dat   ngate.group     tox.group
```

A general rule is to have a minimum of two to three times the space of the initial database, if it is in GDSII file format, reserved for group file creation, assuming the design has a good hierarchical style. A mostly flat design, or one with unwanted data interactions, might require four to six times the initial database space for its group file creation. Thus, for a GDSII file database of 10 MB, a minimum of 20 to 30 MB is required for group file creation space, if the design is hierarchical in nature. By contrast, at least 40 to 60 MB are required for a flat design.

In addition to the group files, the other significant output files from the IC Validator run are the run summary files and the output database files. The summary files are the *cell*.LAYOUT_ERRORS and *cell*.sum files. These files are ASCII log files that document the results of the IC Validator run on the cell named in the runset.

The size of the *cell*.LAYOUT_ERRORS file and the output database depends on the number of errors encountered in the IC Validator run. As with the group files, if the design is hierarchical, the graphical output database is smaller than the graphical output for a more flat design. The graphical output database is smaller because the IC Validator tool only needs to report errors internal to repeated modules one time, while for a flat design the error is reported for every instance of the module.

Feedback

Memory Usage

You can manage the memory space needed for error output reporting by using a logical approach to locating errors. Start by running the IC Validator tool on small modules of the design and address errors as they are encountered. After the errors are fixed, run the IC Validator tool on hierarchically higher modules. Using this procedure keeps your *cell*.LAYOUT_ERRORS file and output database reasonably small. Working up through the hierarchy in this way manages the use of memory needed for error output storage space.

The IC Validator tool also requires physical memory and swap space to complete its checks. The tool attempts to load into memory all data required to complete the checks currently being performed. Any overflow of physical memory uses swap space. Generally speaking, any time the IC Validator program needs to access swap space, you can expect performance degradation, especially if swap space is distributed over a large network with considerable network traffic. If enough physical memory is available to load all data and complete a check without accessing swap space, you can expect maximum throughput. A machine with three to four times as much memory as the GDSII file size should perform well with a hierarchical design. Flat designs require more hardware. Set the swap space so that any overflow of physical memory does not exhaust the swap space and cause the IC Validator run to terminate.

The output *cell*.sum file includes a list of the memory used and the number of page faults required to complete each check. If you see that certain checks require more memory than the machine has allocated, causing large amounts of data to be swapped out to disk, you might need a machine with more memory for future runs. Example 1 shows an example of the reported memory usage. The following section, Runtime and Memory Use Report, gives information about the reported runtimes and memory usage.

Example 1 Partial Example of Memory Usage Information in cell.sum File

```
length_edge() at sf.rs:162
gate_gt_3_5 = length_edge(gate_e, distance > 3.5)
  Function: length_edge
  Inputs: gate_e.edgelayer.0001
  Outputs: gate_gt_3_5.edgelayer.0001
8 unique edge sets written.
  Check time = 0:00:00  User=0.00 Sys=0.00 Mem=0.015 GB
external2() at sf.rs:166
external2(gate_le_1_5, POLY, distance < 0.15, extension = RADIAL,
look_thru = COINCIDENT)
  Comment: "R.POLY.G1: Min Space POLY to Gate for Gate Width <=1.5 must be > 0
.15"
  Function: external2
  Inputs: gate_le_1_5.edgelayer.0001, POLY.polygonlayer.0002
WARNING - 6 spacing violations found.
   Check time = 0:00:00  User=0.00 Sys=0.00 Mem=0.016 GB
external2() at sf.rs:169
external2(gate_gt_1_5_le_2_5, POLY, distance < 0.25, extension = RADIAL,
look_thru = COINCIDENT)
```

Feedback

```
  Comment: "R.POLY.G2: Min Space POLY to Gate for 1.5 < Gate Width <=2.5 must
 be
> 0.25"
  Function: external2
  Inputs: gate_gt_1_5_le_2_5.edgelayer.0001, POLY.polygonlayer.0002
WARNING - 6 spacing violations found.
   Check time = 0:00:00  User=0.00 Sys=0.00 Mem=0.016 GB
external2() at sf.rs:172
external2(gate_gt_2_5_le_3_5, POLY, distance < 0.35, extension = RADIAL,
look_thru = COINCIDENT)
  Comment: "R.POLY.G3: Min Space POLY to Gate for 2.5 < Gate Width <=3.5 must
 be
> 0.35"
  Function: external2
  Inputs: gate_gt_2_5_le_3_5.edgelayer.0001, POLY.polygonlayer.0002
WARNING - 3 spacing violations found.
   Check time = 0:00:00  User=0.00 Sys=0.00 Mem=0.016 GB
external2() at sf.rs:175
external2(gate_gt_3_5, POLY, distance < 0.45, extension = RADIAL,
look_thru = COINCIDENT)
  Comment: "R.POLY.G4: Min Space POLY to Gate for Gate Width > 3.5 must be > 0
.45"
  Function: external2
  Inputs: gate_gt_3_5.edgelayer.0001, POLY.polygonlayer.0002
WARNING - 6 spacing violations found.
   Check time = 0:00:00  User=0.00 Sys=0.00 Mem=0.016 GB
```

For maximum performance on a networked system, keeping as many files as required on the local machine results in faster completion. Enough disk space local to the machine on which the job is run should be available to accommodate the Milkyway library (or, GDSII or OASIS files), the group files being created, and all ASCII or physical error files being created.

Runtime and Memory Use Report

This section has information about the runtime and memory usage reported in the summary file for each stage of an IC Validator run. Note that some processing falls outside of the cumulative memory reporting.

Note:

Generally, only memory allocated by the IC Validator tool is reported in the peak usage or accumulative memory usage. For example, Milkyway memory is allocated by the Milkyway library, so it is not included in the report of IC Validator peak memory usage or machine accumulative memory usage.

Table 22 shows the definitions of the times and memory usage terms used in the summary file.

Feedback

Table 22 Terms Used in the Summary File

Term	Definition
Time	Wall time
User	CPU time for calculation
Sys	CPU time for I/O
Mem	Maximum memory usage

The following examples are from different stages in a summary report:

1. **Runset compile stage**
   - When the IC Validator tool is run without runset caching, the report looks like this:

     ```
     Runset Compile Time=0:01:03 User=61.08 Sys=0.81 Mem=1.065 GB
     ```

   - When the IC Validator tool is run with the `-rscache` option for runset caching, the report looks like this:

     ```
     Cached Runset Compile Time=0:00:04 User=0.81 Sys=0.25 Mem=0.203 GB
     ```

2. **Distributed processing setup stage**

   ```
   Virtual Machine Init Time=0:00:02 User=1.09 Sys=0.12 Mem=1.065 GB
   System Startup Time=0:00:02 User=0.00 Sys=0.52 Mem=1.065 GB
   ```

3. **Preprocessing stages**
   - **Hierarchy reading and optimization stage:** The Milkyway and OASIS memory usage is not accounted for in the peak memory report.

     ```
     Oasis memory used: 27.0M
     Milkyway memory usage: 4714.789M
     Reading hierarchy Time=0:00:10  User=8.29 Sys=2.87 Mem=0.207 GB
     Updating hierarchy Time=0:00:00  User=0.07 Sys=0.00 Mem=0.056 GB
     Exploding Time=0:00:00  User=0.44 Sys=0.02 Mem=0.057 GB
     ```

   - **Virtual cell pairing stage**

     ```
     Pairing Iteration 1 Time=0:00:00  User=0.53 Sys=0.04 Mem=0.098 GB
     Pairing Iteration 2 Time=0:00:00  User=0.54 Sys=0.05 Mem=0.097 GB
     Pairing Iteration 3 Time=0:00:00  User=0.51 Sys=0.04 Mem=0.096 GB
     Pairing Iteration 4 Time=0:00:00  User=0.45 Sys=0.03 Mem=0.098 GB
     Pairing Time=0:00:01  User=2.03 Sys=0.16 Mem=0.098 GB
     Combined VCELL Time=0:00:02  User=2.67 Sys=0.23 Mem=0.098 GB
     Post VCell Time=0:00:00  User=0.12 Sys=0.00 Mem=0.060 GB
     Prototype Pass 2 Time=0:00:00  User=0.04 Sys=0.00 Mem=0.061 GB
     ```

- **Regioning stage**

```
Determine Region Time=0:00:00  User=0.02 Sys=0.00 Mem=0.059 GB
```

- **Create Layer stage**

```
Create Layer Setup Time=0:00:00  User=0.06 Sys=0.01 Mem=0.058 GB
Hierarchy Cleanup Time=0:00:01  User=0.89 Sys=0.04 Mem=0.059 GB
```

4. **Command execution**

- **Command statistic report**

```
Check Time=0:00:00  User=0.00 Sys=0.00 Mem=0.017 GB
```

- **Reading error hierarchy**

```
Elapsed Time=0:00:00  User=0.08 Sys=0.00 Mem=0.024 GB
```

- **text_net()**

```
Total Text Time=0:00:00  User=0.05 Sys=0.00 Mem=0.064 GB
```

- **extract_devices()**

```
Device Connect Time=0:00:04  User=1.41 Sys=2.84 Mem=0.147 GB
```

- **netlist()**

```
Netlisting Time=0:00:03  User=1.63 Sys=0.68 Mem=0.069 GB
```

- **write_milkyway():** The Milkyway memory usage is not accounted for in the peak memory report.

```
Milkyway memory usage: 4129.207M
Compression Time=0:05:07  User=275.34 Sys=27.80 Mem=3.529 GB
Writing Time=0:10:32  User=516.55 Sys=47.36 Mem=3.812 GB
```

- **write_oasis():** The OASIS memory usage is not accounted for in the peak memory report.

```
Merging input layout from <input OASIS>
Merge Time=0:00:00  User=0.00 Sys=0.00 Mem=0.025 GB
Writing Time=0:00:00  User=70.00 Sys=0.00 Mem=0.025 GB
```

- **write_gds():**

```
Writing Time=0:00:00  User=0.00 Sys=0.00 Mem=0.025 GB
```

5. **Host summary**
   - **Per host summary of total runtime and peak memory usage.** This time excludes memory usage for reading and writing Milkyway data and OASIS files.

     ```
     Host number 0 total check Time=0:04:40  User=182.60 Sys=37.42
     Mem=0.791 GB
     ```

   - **Per host summary of total runtime.** This time includes preprocessing and peak memory usage but excludes memory usage for reading and writing Milkyway data and OASIS files.

     ```
     Host number 0 total check and preprocess Time=0:04:56  User=195.38
     Sys=40.62 0.791 GB
     ```

6. **Error writing summary report**

   ```
   Overall error storage time: User=0.06 Sys=0.11 Mem=0.014 GB
   Generation Time=0:00:00  User=0.01 Sys=0.00 Mem=0.0147 GB
   ```

7. **Overall memory and runtime reports**
   - **Per machine accumulative peak memory usage.** This memory usage excludes reading and writing Milkyway data and OASIS files.

     ```
     IC Validator Peak Machine Memory Report
     igcae209     : Peak = 2.061 GB
     igcaew008    : Peak = 3.813 GB
     igcae191     : Peak = 1.272 GB
     igcae177     : Peak = 1.396 GB
     igcae188     : Peak = 1.403 GB
     igcae205     : Peak = 1.269 GB
     igcae178     : Peak = 1.399 GB
     igcae174     : Peak = 1.404 GB
     ```

   - **Overall runtime and peak memory report that is for all stages and functions.** This report excludes memory usage for reading and writing Milkyway data and OASIS files.

     ```
     Overall engine Time=0:06:12 Highest command Mem=0.791 GB
     ```

   - **Master memory usage.** This usage is not accounted for in the peak memory report.

     ```
     Overall Master Mem=0.246 GB
     ```

Operating Systems

The IC Validator tool supports many hardware architectures and operating systems. See the installation guide for a list of the supported operating systems:

http://www.synopsys.com/support/licensing-installation-computeplatforms/compute-platforms/release-specific-support.html

System Limits

If system limits are set incorrectly, you might encounter an error during the IC Validator run. For example, the following commands set the proper system limits for an IC Validator run on a Linux operating system. (The numerical values are in kilobytes.)

```
unlimit filesize

unlimit datasize

limit descriptors 1024

limit stacksize 8192
```

Note:

The descriptors setting controls how many files you can have open at any time during a run.

To verify the limits on a host, run the `limit` shell command.

Networked Environments

Accessing files over a networked environment, such as when a remote group directory is used, can degrade performance. You might observe large differences in the wall time (overall time) and total CPU time (user time plus system time) reported by the IC Validator tool.

Note:

These differences could also be caused by:

- Other processes running on the same system.

- System overhead.

The read- and write-transfer sizes (`rsize` and `wsize`) of the NFS mounts should be at least 32768. For example, for static mounts set:

```
% mount -t nfs -o rsize=32768,wsize=32768 server:/path /mnt/nfs
```

3

Cloud Enablement for IC Validator

This chapter explains about hybrid cloud and how to run IC Validator in a hybrid cloud environment.

The information described in this chapter is as follows:

- Overview
- Hybrid Cloud
- IC Validator Cloud Functionalities
- IC Validator on Hybrid Cloud

Overview

Using cloud resources allows you to handle peak workloads, access powerful hardware, and scale your compute resources as needed. IC Validator supports multiple cloud service providers and integrates with hybrid environments, making it easier to manage large and complex designs.

Using IC Validator in the Cloud

The advantages of using IC Validator in the cloud is as follows:

- Elasticity: Access virtually unlimited compute and storage resources.
- Hardware Optimization: Choose the best CPUs and GPUs for your project.
- Multi-Cloud Compatibility: Work with providers like Microsoft Azure, AWS, Google Cloud, Oracle Cloud, and HPE GreenLake.
- Pay-Per-Use: Use resources as needed and pay only for what you use.
- Seamless Hybrid Integration: Scale up with additional compute resources when required.

Feedback

- Cost-Effective Instances: Use spot instances and save-and-restore features for long jobs.
- Continuous Performance Improvements: Benefit from ongoing memory and I/O optimizations.

Hybrid Cloud

Organizations seeking to establish or enhance a hybrid cloud environment can review various supported configurations:

No Public Cloud Access

When cloud connectivity is unavailable, the Synopsys Cloud platform can be used to deploy a hybrid cloud environment. Refer to the Synopsys Cloud section for additional information.

Existing Cloud Access Without Data Mirroring and Existing Compute Grid

Establishing a hybrid cloud environment requires a data mirroring service. The Synopsys Data Service offers an electronic design automation-focused alternative to third-party data mirroring solutions. Details are available in the Synopsys Data Service section.

A multi-cluster grid environment is necessary to fully leverage hybrid cloud capabilities. Consult the information technology department for guidance on configuring a multi-cluster grid. After assembling the required components, hybrid mode can be enabled for electronic design automation workloads.

Existing Data Mirroring and Multi-Cluster Grid Environment

In this configuration, a data mirroring service (such as NetApp FlexCache) is already implemented, and the compute grid environment (such as IBM Load Sharing Facility or Simple Linux Utility for Resource Management) supports multi-cluster operation. Most Synopsys tools operate efficiently in this environment. Tool-specific options are available for distributed execution. Refer to the Product-Specific Cloud Functionalities section for further details.

Feedback

IC Validator Cloud Functionalities

This section provides guidelines for running IC Validator in a cloud environment.

Preparing Your Design for The Cloud

Create the runset and resolve all dependencies by copying all necessary files, including the layout database, to a local directory before transferring the design to the cloud environment.

Note:

Refer to the IC Validator Reference Manual for detailed usage information.

Transferring Your Design

Transferring the design files may require significant time. It is advisable to prepare a small block design first to test the cloud environment before transferring large databases. Use the small block design to verify that the runset, cloud setup, and host configurations are complete and functional in the cloud environment.

Cloud Configuration Guidelines

Launch IC Validator on Cloud

Run IC Validator in the cloud using standard host configurations specified on the command line. Refer to the Distributed Processing section in the IC Validator User Guide for guidance on host configuration.

Environment variable setup:

- Set `ICV_HOME_DIR` to the IC Validator installation directory.
- Set `PATH` to include `$ICV_HOME_DIR/bin`

IC Validator uses multiple central processing units (CPUs) on a host, as well as multiple hosts, to form a distributed job. Specify the number of hosts and CPUs for a job or allow IC Validator to determine the host pool size from the job scheduler platform, such as Load Sharing Facility (LSF) or Simple Linux Utility for Resource Management (SLURM).

Example: Submitting an IC Validator job to LSF using two hosts with 16 CPUs each:

```
bsub -n 32 -R "span[ptile=16]" <bsub options> icv -host_init LSF
 runset.rs
```

Feedback

Specifying LSF as the argument to `-host_init` instructs IC Validator to automatically determine the host configurations and launch distributed engines to occupy two hosts with 16 CPUs each.

Additional hosts can be added to a running IC Validator job in the cloud using the `-host_add` command-line option, in the same manner as with `-host_init`. Hosts can also be removed from a running job by specifying them with the `-host_remove` command-line option.

Note:

Some cloud providers enforce strict memory usage policies for running jobs. Use the `-host_memory` command-line option to limit IC Validator memory usage per host.

Optimize NFS Usage on Cloud

IC Validator stores intermediate data in the group directory, typically located on a network disk under the **run_details** directory. Although IC Validator automatically deletes intermediate data, the data volume may increase the cost of running IC Validator in the cloud. The following sections describe methods to reduce or eliminate IC Validator runtime network file system (NFS) footprint.

Cache Files on Local Disk

Specify a local group directory for all hosts using the `-host_disk` command-line option. In this mode, IC Validator manages file transfers among distributed hosts in the cloud. If local disk space becomes full, IC Validator stops writing to the local disk and instead uses the network path of the **run_details** directory to store overflow data.

Substitute NFS with Cloud Input/Output Layer (CIOL)

Configure IC Validator to automatically set up a CIOL mount and create a temporary network directory.

Two configurations are recommended as follows:

1. To minimize NFS usage, configure a CIOL mount for the IC Validator run_details directory:
   - Use NFS as the IC Validator job working directory to store only results and log files.
   - Add the following command-line options:
     - `-ciol`. Enables CIOL mode for IC Validator.
     - `-rd CIOL`. Requests CIOL to create run_details for IC Validator.
     - `-host_disk /local/disk/path`. Caches intermediate data on local disk to improve performance.

Feedback

2. To run ICV without NFS usage:
   - The current working directory can reside on any disk.
   - Add the following command-line options:
     - `-ciol`
     - `-host_disk /local/disk/path`
   - Use absolute paths for all input data files in IC Validator command-line options.

To configure the CIOL data store location, set the `ICV_CIOL_LOCAL_DIR` environment variable to a directory on disk. If this environment variable is not set, IC Validator sets up an in-memory database for CIOL data storage.

IC Validator on Hybrid Cloud

Distributed Processing Across On-Premises and Cloud Hosts

Running a distributed IC Validator job across both on-premises and cloud hosts requires the following prerequisites:

- A job scheduler platform capable of submitting jobs to both on-premises and cloud hosts.
- A network that allows establishing Transmission Control Protocol/Internet Protocol (TCP/IP) connections between on-premises and cloud hosts.
- Runset files and layout databases must be accessible with identical paths on both on-premises and cloud hosts.

Two file storage configurations are recommended:

1. Use an NFS disk shared by on-premises and cloud hosts to launch IC Validator. This is the preferred configuration.
2. If a shared NFS disk is unavailable, create a shared network disk with CIOL and use it as the working directory for IC Validator. The `-host_disk` command-line option is recommended for improved performance. Two options are available to configure the CIOL disk as follows:
   - Launch the CIOL server first, then launch the CIOL client on all on-premises and cloud hosts to create the CIOL mount. Run IC Validator on this CIOL mount.
   - Use IC Validator distributed processing to propagate the execution of CIOL clients to the hosts where IC Validator is launched.

Feedback

Note:

A `-host_login` script is required when starting an IC Validator job on multiple hosts. Configure the `-host_login` script to execute the CIOL client before starting IC Validator on remote hosts.

Similarly, when using the `-host_add` or `-host_elastic` flow, ensure that the script executes the CIOL client before starting the IC Validator executable.

Computing on Cloud Hosts With On-Premises Data Storage

To execute an IC Validator job using only cloud hosts while keeping the runset and layout database in on-premises storage, do as follows:

1. Configure the CIOL server on an on-premises host and copy the layout and runset data to the CIOL mount.
2. Execute the CIOL client on cloud hosts before starting the IC Validator job to provide access to runset and design data.
3. After completing the IC Validator job, transfer results from the cloud to on-premises storage.

Optionally, skip the transfer step and retrieve IC Validator results from on-premises hosts by:

1. Using the CIOL mount as the working directory for the IC Validator job.
2. Using the `-host_disk` command-line option for improved performance.
3. Executing the CIOL client with the on-premises host to monitor the running IC Validator job and review the results.

4

Output Files

This chapter provides an overview of IC Validator design rule checking (DRC) and layout-versus-schematic (LVS) output files.

The information described in this chapter is:

- Description of Output Files
- General DRC and LVS Output Files
- LVS-Specific Output Files

Description of Output Files

The output files produced by the IC Validator tool are specific to the type of run performed. If you are performing DRC, the tool produces error and summary files. The error files list the DRC errors in your design. The summary files list information about your design and the completed run.

If you are performing device extraction, the IC Validator tool produces error, summary, and layout netlist files. The layout netlist file is the file that is compared to a schematic netlist during LVS comparison. Both DRC and device extraction runs produce tree structure, virtual cell, and technology files in the run_details and run_details/hierarchy directories.

If you are performing LVS comparison, the IC Validator tool produces error, log, and equivalence summary files. The error file lists the nets and devices from the layout that do not match the schematic.

The error, summary, and layout netlist files are written to current working directory. There are additional files, which are located in the run_details directory, that help in debugging LVS errors.

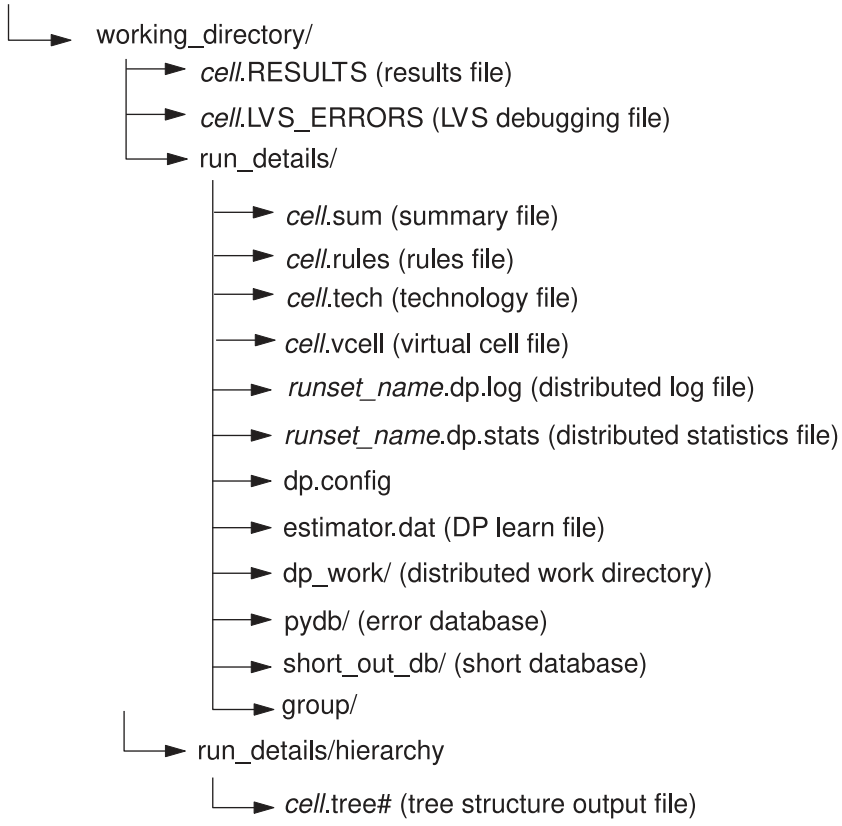
All output files are named *cell*.abbreviation, where *cell* is the name specified by the `cell` argument in the `library()` function. The extension following *cell* is an abbreviation for the explanation of the file. For example, the account file name is *cell*.acct.

Feedback

General DRC and LVS Output Files

Figure 4 shows the output files and directories common to both DRC and LVS runs. The files are described in the following sections.

Figure 4 Output Files and Directories Common to DRC and LVS



Results File

The IC Validator tool creates a RESULTS file, *cell*.RESULTS, that provides the results from key areas of the run and a summary of the search and grep key results. The RESULTS file also contains the IC Validator command line options used for the run. This file is the starting point for analyzing the results of an IC Validator run.

Feedback

The *cell*.RESULTS file is divided into the following sections:

- Results Header

  If the DRC run completed with no violations, the IC Validator tool returns a single line that states, “RESULTS: CLEAN”.

  If the DRC run completed, but with violations, the IC Validator tool returns a single line that states, “RESULTS: NOT CLEAN”.

  If the LVS run completed, the IC Validator tool returns two results in the header section:

  - The first result is a single line that states, “LVS Compare Results: PASS | FAIL”.
  - The second result is a single line that states, “DRC and Extraction Results: CLEAN | NOT CLEAN”.

  A DRC or LVS run that does not complete returns a single line that states, “RESULTS: RUN ABORTED”.

- Execution

  This section consists of two parts:

  - The first part prints the IC Validator banner and copyright information, and how the IC Validator tool was called.
  - The second part consists of basic run information, which includes the following items: input library file name, top cell name, layout format of input library, runset file name, working directory path, run details directory path, LVS error file name, layout error file name, user name, and start and end time of the IC Validator run.

- Results Summary: DRC Run

  This section consists of two parts:

  - The first part reports the rule and DRC error statistics, including the total number of rules that are run, the number of rules that are not executed, the number of rules that contain violations, and the total number of violations. See the *cell*.LAYOUT_ERRORS file for detailed violation data.
  - The second part consists of a table showing each rule name and the total number of violations found for that rule. This part is output only if the `verbose_results_file` argument of the `run_options()` function contains the `ERRORS` option. Rule names are extracted from rule comments based on the `rule_name_delimiter` argument of the `error_options()` function. If you run DRC error classification, the number of waived violations is excluded from the number of violations found. Any rule that was not executed shows a “Not Executed” status. Violation counts for individual functions under a rule are not reported, with the

Feedback

exception of functions without a rule. These functions are reported (indented by two spaces) under the default rule name, “Violation.” Violation counts are preceded with the “v =” string for easy searching of a specific violation count.

- Results Summary: LVS Run

  This section consists of two parts:

  - The first part is the table of LVS compare statistics. This section reports the following LVS compare statistics: LVS compare results, top equivalence points, number of checked black-box cells, number of passed black-box cells, number of failed black-box cells, number of checked equivalences, number of successful equivalences, and number of failed equivalences. See the *cell*.LVS_ERRORS file for the detailed compare violation data.
  - The second part lists the LVS results of all equivalence and black box pairs; the first being the failed equivalences and black-box cells, followed by passed equivalences and black-box cells. You can search or grep the list to quickly find the results of equivalence and black-box cells.

- Assign Layer Statistics

  This section consists of two parts:

  - The first part reports four items: total number of used assign layers, number of empty used assign layers, number of non-empty used assign layers, and number of pruned assign layers.
  - The second part consists of a table showing each layer name, hierarchical and flat polygon counts for the layer (or “Pruned” if the layer was pruned), and the layer or datatype assignments for the layer. This part is output only if the `verbose_results_file` argument of the `run_options()` function contains the `ASSIGNS` option.

    The table is divided into three sections: 1) non-empty used layers 2) empty used layers 3) pruned layers. The assign statement for each layer contains layer datatype assignments and all other user-supplied assignment options.

- Run Configuration

  This section consists of the following seven parts:

  - The `run_options()` function reports the `verbose_results_file` argument.
  - The `hierarchy_options()` function reports the `delete` argument.
  - The `error_options()` function reports the following items: `error_limit_per_check`, `match_errors`, `report_empty_violations`, and `clean_classifications` arguments.

Feedback

    - The `drc_black_box_options()` function reports the following items: `black_box_cells`, `black_box_region_cells`, `keepout_layers_check`, and `keepout_violation_comment` arguments.
    - Production Environment Variables report all production environment variables that are set. A total count of these environment variables is printed at the end of the list.
    - Debug Environment Variables report all debugging environment variables that are set. A total count of these environment variables is printed at the end of the list.
    - Distributed processing reports the distributed cores and turbo, followed by a list of host machines with their specifications (model name, CPU, cores, RAM, swap space). Each host machine also displays the number of distributed processes and the number of CPU on the host next to the host name.
- Performance Statistics

  This section reports five items:

    - IC Validator runtime
    - Peak single command memory
    - Peak disk usage
    - Distributed host peak memory usage (this is a table showing each host, the distributed processing and number of CPU on that host, and the host peak memory and RAM)
    - Overall distributed utilization

Error File

The IC Validator tool creates an error file, *cell*.LAYOUT_ERRORS, during DRC or netlist extraction runs. These types of errors are referred to as layout errors. The LAYOUT_ERRORS file contains the results of a run, a summary list of the errors, and a more detailed list of the errors.

The top of the file states either CLEAN or ERRORS so that you can quickly determine if there are layout errors in the design.

The next section is the summary section. This section provides the comment for a function, the function itself, and the number of errors for that function if it had errors.

The last section is the details section. Each error report contains the following: an error description that identifies the check being performed; a structure name where the error occurred; a position or a bounding box value where the error occurred; and other information specific to a check. The position coordinates are reported hierarchically, that

Feedback

is, relative to the named structure and only one time for each referenced structure. These errors can also be viewed graphically using the VUE debugging tool.

During the run, the LAYOUT_ERRORS file is kept up-to-date with the results and the summary sections. At the end of the run, a final version of the file is written including the details section, as specified by `error_options(report_error_details)`.

The following section of the LAYOUT_ERRORS file shows a minimum area rule for metal1 (Met1). The rule was coded as follows in the runset:

```
//R.Met1.A
{
   @"R.Met1.A:For Metal 1, minimum area must be 0.04";
   area( Met1, value < 0.04);
};
```

The LAYOUT_ERRORS file reports this rule as follows:

```
                LAYOUT ERRORS RESULTS: ERRORS

              ##### ####  ####   ###  ####   ###
              #     #   # #   # #   # #   # #
              ####  ####  ####  #   # ####   ###
              #     #  #  #  #  #   # #  #      #
              ##### #   # #   #  ###  #   # ####

=========================================================================

Library name:    ../../lab_data/INLIB_TRAINING.GDS
Structure name:  top
Generated by:   IC Validator <platform> <release> <date>

Runset name: icv runset.rs
User name: juser
Time started: 2016/03/02 07:35:12AM
Time ended: 2016/03/02 07:35:51AM

Called as: icv runset.rs
                  ERROR SUMMARY

 R.Met1.A: Minimum area must be 0.04
   area ..................................... 1 violation found.

                  ERROR DETAILS

-------------------------------------------------------------------------
R.Met1.A: Minimum area must be 0.04
-------------------------------------------------------------------------

sf.rs:238:area
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Structure ( lower left x, y ) ( upper right x, y) Area
```

Feedback

```
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
top        (37.2700, 4.0300)    (37.4800, 4.1700)    0.0294
```

Summary File

The IC Validator tool creates a summary file, *cell*.sum, during DRC or netlist extraction runs.

If you are using the `-verbose` command-line option, the information printed to the screen is the same as the information reported to the summary file. The screen output, however, prints the information in the order of processing by the IC Validator tool, while the content of the summary file sorted to match the sequential order of the runset.

Rules File

Rules to be executed are written to the *cell*.rules file in the run_details directory. Violation names and violation comments that remain in the runset after all forms of runset selection have been performed are reported in the *cell*.rules file. Examples of runset selection include preprocessor directives, selectable rules, incremental layers, and run-only pruning.

Violation names and violation comments are harvested for all rules in the final parse tree. Because runtime conditional commands (`if`/`then`/`else` statements and `for`/`foreach`/`while` loops) might remain, rules in the final parse tree are not guaranteed to execute. Rule output is included for all commands in the final parse tree, including commands that do not execute due to runtime conditions.

Violation names and violation comments might not always be statically resolved. In this case the file and line number are provided in the output file instead of the rule name.

Only the violation comment is present in the output if there is no “@=” violation block for the runset rule.

The default comment is “Violation” and “@ \“Violation\”” appears in the rules to be output when a rule uses the default comment.

The following is an example *cell*.rules file:

```
                          ICV_Master (R)
        Copyright (c) 1996 - 2015 Synopsys, Inc.
   This software and the associated documentation are proprietary to
Synopsys,Inc. This software might only be used in accordance with the
 terms and conditions a written license agreement with Synopsys, Inc. All
 other use, reproduction, or distribution of this software is strictly
    prohibited.

Called as: icv runset.rs -i test.gds
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
User name:        synopsoid
```

Feedback

```
Layout format:    GDSII
Input file name:  test.gds
Top cell name:    top
Time started:     2015/11/03 02:43:08PM
---------------------------------------------------------------------------
                          Rules to be Executed
---------------------------------------------------------------------------
     Unresolved violation name/comment found at runset.rs:100
@ "Rule comment with no violation block"
@ "Violation"
VIOL_101 @= { @ "VIOL_101_comment" }
VIOL_102 @= { @ "VIOL_102_comment" }
VIOL_103 @= { @ "VIOL_103_comment" }
VIOL_104 @= { @ "VIOL_104_comment" }
VIOL_105 @= { @ "VIOL_105_comment" }
```

Technology File

The IC Validator tool creates a technology file, *cell*.tech file, that displays the options which are set for the IC Validator run. The list of options includes the default as well as the user-specified values. The technology file contains three sections: exploded cells, deleted cells, and placement explode statistics.

Tree Structure Output File

The IC Validator tool creates tree structure files, named *cell*.tree, during DRC or netlist extraction runs. Tree structure files contain information about the design hierarchy tree and reference statistics for every cell in the design tree. The *cell*.tree0 file represents the original data. The *cell*.tree1 file is created after all of the exploding options are completed. Additional tree files are created when using the hierarchy options. The *cell*.tree*N* file with the highest number (*N*) is the hierarchy that was processed during the run. The tree files are printed if the `print_preprocess_files` argument of the run_options() function specifies the `TREE` option, the `compress_tree_files` argument is used to determine if the tree files are compressed when written to disk.

Virtual Cell File

The virtual cell file, *cell*.vcell, contains information about pairing, creating sets, and exploding of virtual cells depending on the options you use in the IC Validator runset and the data in your design. It includes the Pair, Set, and Exclude sections.

Pair Section

The Pair section reports the cell pairs that were created during each virtual cell pass iteration. The `iterate_max` option of the `pairs` argument of the `hierarchy_options()` function sets the maximum number of iterations.

Feedback

Set Section

The Set section reports the sets that were created during each virtual cell pass iteration. The `iterate_max` option of the `sets` argument of the `hierarchy_options()` function sets the maximum number of iterations.

Explode Section

The Explode section of the output file displays all of the branches that were exploded to bring together interacting placements under a common parent. For more information, see the `explode` argument of the `hierarchy_options()` function in the *IC Validator Reference Manual*.

Distributed Log File

The distributed log file, named with the runset name and a dp.log extension, contains more information than a summary file, as it includes details about when a function is sent to a host, and the results produced when the host completed the function. The distributed log file is in order of execution rather than in order of the runset.

Distributed Statistics File

A distributed statistics file, named with the runset name and a dp.stats extension, contains distributed processing statistics.

Distributed Work Directory

A distributed work directory, named dp_work, is created for a distributed run in the run_details directory. The directory contains subdirectories and files used during the distributed run. If this directory exists before a run, it and all of its content are deleted and new files are created. Because this directory is deleted before the run, do not name any individual directories dp_work in the run_details directory of a run.

Graphical Error Structures

The IC Validator tool generates an error database, PYDB, that can be viewed in a layout editor, such as VUE. It contains error polygons and coordinates that are located in proximity to design rule violations. The shapes produced by different checks can be written to separate layers and data types in the error structure, as specified in the runset file. See DRC Error Classification, for a description of the error database and of the pydb_report and pydb_export utilities that you can use to classify errors.

The error database contains all the error data generated by violation blocks and all the information needed for a VUE debugging session. By default, this database is in

Feedback

run_details/pydb. You can change the location by using the `db_path` argument of the `error_options()` function. This database allows for data mining, and you can create custom reports.

Note:

You can start reviewing the error database before the IC Validator tool completes the run.

The IC Validator tool generates a hierarchy of error structures that mimics the database hierarchy that is being checked. Thus, any errors that are found in a particular structure in the database appear in a corresponding error structure in the error hierarchy. Error structures are generated only for the database structures that contain errors or that reference other structures with errors. The IC Validator tool does not create an error hierarchy if no errors were found.

Error structures are named with the default error prefix, ERR_. This prefix precedes the structure name. For example, the default error structure for the cell CLOCK is ERR_CLOCK. You can redefine the error prefix using the `error_cell_prefix` argument in the write functions, such as the `write_gds()` function.

For example,

Database hierarchy	Error hierarchy

CHIP {
  ALU {
  BIT0
  BIT1 }
  CLOCK {
  CLK1
  CLK2
  CLK3 }
}
	ERR_CHIP {
	  ERR_ALU {
	  ERR_BIT0
	  ERR_BIT1 }
	  ERR_CLOCK {
	  ERR_CLK1
	  ERR_CLK3 }
	}

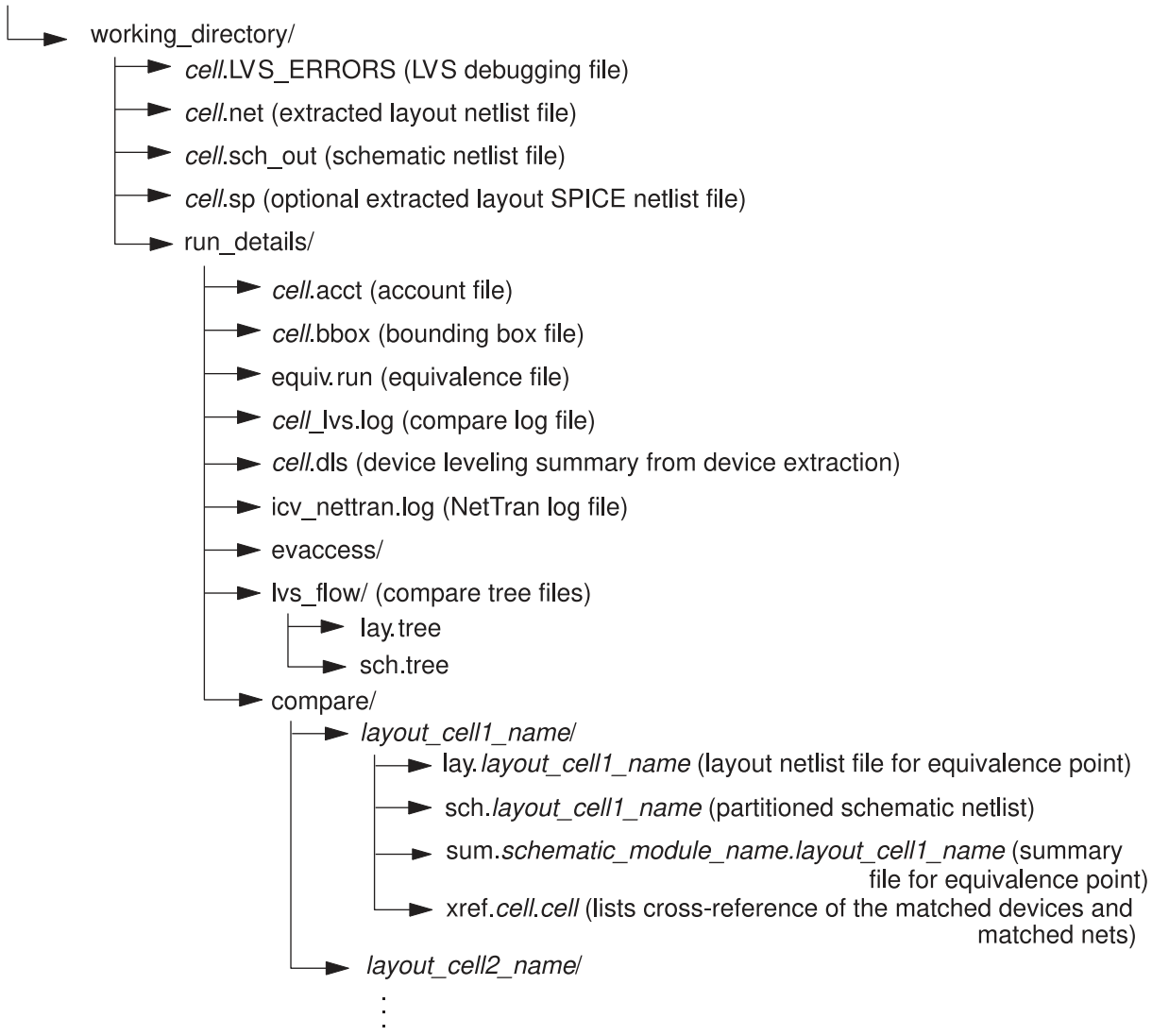
In the error hierarchy, there is no occurrence of the error structure ERR_CLK2. Either the original CLK2 cells were exploded during preprocessing, or the structure CLK2 did not contain any errors or structures that contain errors. If a previous IC Validator run with errors is corrected and rerun, previous error hierarchy structures are retained. The IC Validator tool does not overwrite these structures if no errors are found, but the new error hierarchy does not have references to these previous error containing cells.

Feedback

LVS-Specific Output Files

Figure 5 shows the output files and directories for LVS runs. The files are described in the following sections.

Figure 5 Output Files and Directories for LVS



Feedback

LVS Debugging File

The `compare()` function creates a debugging file, *cell*.LVS_ERRORS, that indicates whether or not the schematic-versus-layout comparison was successful. This file is the starting point for debugging LVS issues.

Depending upon whether the comparison was successful or not, the file lists the PASS/FAIL information and layout errors if any. The LVS_ERRORS file then lists the important failed equivalence points, sorted in high and low priorities, and provides diagnostics designed to assist you in troubleshooting the design.

Extracted Layout Netlist File

The IC Validator tool creates an extracted layout netlist file, *cell*.net, whenever a `netlist()` extraction function is specified in the runset file. This file is a hierarchical description of the structures found in the layout database.

Extracted Layout SPICE Netlist File

The IC Validator tool creates an extracted SPICE layout netlist file, *cell*.sp, whenever the `write_spice()` function is specified in the runset file. This file is a hierarchical SPICE description of the structures found in the layout database. See the `write_spice()` function in the *IC Validator Reference Manual* for further information.

Account File

The IC Validator tool creates an account file, *cell*.acct, that lists the number of devices and their types extracted from each cell in the hierarchy. The cumulative count shown is the total count of all devices contained in that structure.

Bounding Box File

The IC Validator tool creates a bounding box file, *cell*.bbox, that lists the total number of cells and the bounding box coordinates of each cell in the design.

Equivalence Files

If you do not supply an equivalence file, then an LVS compare run automatically produces an equivalence file that is used to compare the layout and schematic netlists. When producing an equivalence file, the IC Validator tool recognizes that some equivalence points are extraneous and do not aid in the comparison process. After all analysis is completed, the IC Validator tool generates an equivalence file, equiv.run, that lists all the

Feedback

equivalence cells that were used during the run. See the `equiv_options()` function in the *IC Validator Reference Manual* for more information.

Device Leveling Summary File

The device leveling summary (DLS) file gives you a way to trace how a device was extracted in terms of hierarchy level. You can determine the root cause of why a device was extracted at the wrong level of hierarchy (that is, leveled) by the diagnostic information in the *cell*.dls file, which is in the run_details directory.

The following is an example *cell*.dls file.

```
+-------------------------------------------------------------------------+
|                          ICV Device Leveling Summary                    |
+-------------------------------------------------------------------------+

Device Leveling Summary Results:

Body Layers           | Cell | Layer Interacted:
----------------------+------+--------------------
ngt.polygonlayer.0001  A      nsd.polygonlayer.0001, NW.polygonlayer.0003
pgt.polygonlayer.0001  B      psd.polygonlayer.0001
...
```

The device leveling summary file shows body layers for devices that cannot be pushed back to their original cells after device recognition and property calculations. The device body layers that are successfully pushed back are not shown in this file. The file shows

- Body Layers. Derived layer of device body that is leveled out of the cell in which it originally resides.
- Cell. Original cell of the leveled devices.
- Layer Interacted. Layer that interacted with device body, causing device leveling. If there is an interaction among body layer polygons from the same body layer across multiple levels of hierarchy, the body layer name is printed in this column.

Compare Log File

The IC Validator tool creates a log file, *cell*_lvs.log, during an LVS run. This compare log file contains all of the messages related to the `compare()` function.

Compare Tree Files

The `compare()` function produces two tree files. One is named lay.tree; it provides the hierarchical structure of the layout netlist. The second is named sch.tree; it provides the

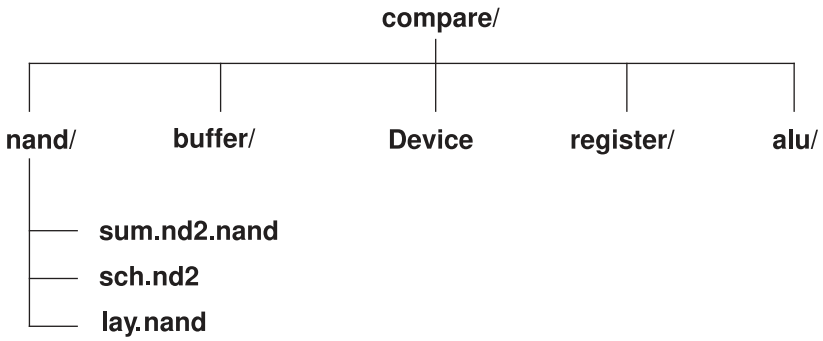
Feedback

hierarchical structure of the schematic netlist. These files are in the run_details/lvsflow directory.

Compare Directory Output Files

The detailed output files from an LVS comparison are placed in the compare directory run_details/compare. A subdirectory of this compare directory is created for each equivalence point in the equivalence file as the equivalence points are evaluated. These subdirectories are named based on the layout structure name. Figure 6 shows an example of a compare directory.

Figure 6 Compare Directory Structure Example



Individual Equivalence Summary File

The results of the comparison of each structure are written to a summary file named sum.*schematic_module_name*.*layout_structure_name*, generally referred to as the sum.*cell*.*cell* file. The file is located in the compare subdirectory of the working directory, as shown in Figure 6. This file is not generated if the `remove_equiv_sum_files` argument of the `compare()` function is `NONE` or if the particular equivalence point is completed without errors or warnings. The information in the summary file includes the following: the number of occurrences of each device found; the total number of nets that were found; and the number of devices and nets that were removed or created by merging. If all of the devices and nets could not be matched, the summary includes a listing of the discrepancies.

The sum.*cell*.*cell* file is divided into the following sections:

- Comparison Result
- Diagnostic Error
- Unmatched Nodes
- Error Information

Feedback

- Warning Information
- Preprocessing Information
- Comparison Information
- Merged Node Information
- Cross-Referencing Information
- Statistics Report

Partitioned Schematic Netlist

The schematic data used for this portion of the comparison process is written to the sch.*cell* file. This file is located in the compare subdirectory of the working directory, as shown in Figure 6. This file is not generated if the `write_equiv_netlists` argument of the `compare()` function is `FAILED` and the particular equivalence point is completed without errors or warnings, or if the argument is `NONE`. This file is the schematic netlist that results after exploding nonequivalent structures; it generally provides little or no useful information for analyzing comparison results.

Partitioned Layout Netlist

The layout data used for this portion of the comparison process is written to a file called lay.*cell*. This file is located in the compare subdirectory of the working directory, as shown in Figure 6. However, this file is not generated if the `write_equiv_netlists` argument of the `compare()` function is `FAILED` and the particular equivalence point is completed without error or warnings, or if the argument is `NONE`. This file is the layout netlist that results after exploding nonequivalent structures; it generally provides little or no useful information for analyzing comparison results.

Editing Errors

After the error structure has been placed into the database, you can begin editing and correcting the errors. The LAYOUT_ERRORS file lists the location and the structure name for each error encountered. The summary file, *cell*.sum, lists the checks that have errors.

Because the output of the IC Validator tool is hierarchical, you need to correct an error only one time, no matter how many times the structure is placed. You continue this process until all errors are eliminated.

5

IC Validator Dashboard

This chapter describes the IC Validator Dashboard capabilities.

The IC Validator Dashboard is described in the following sections:

- Introduction
- Getting IC Validator Dashboard Email Updates From IC Validator
- Starting IC Validator Dashboard Standalone
- The IC Validator Graphical User Interface

Introduction

IC Validator Dashboard provides an easy way for you to see how a run is progressing, how many rules exist and how many have completed, how many violations have been reported, and how IC Validator is using the resources it has been allocated.

As chips near completion, physical verification runs are in the critical path for tapeout, and it becomes increasingly important to ensure they complete on time. IC Validator Dashboard can be used effectively together with the IC Validator `-host_elastic`, `-host_add`, and `-host_remove` command-line options to help you optimize your physical verification runs and hardware resources.

Getting IC Validator Dashboard Email Updates From IC Validator

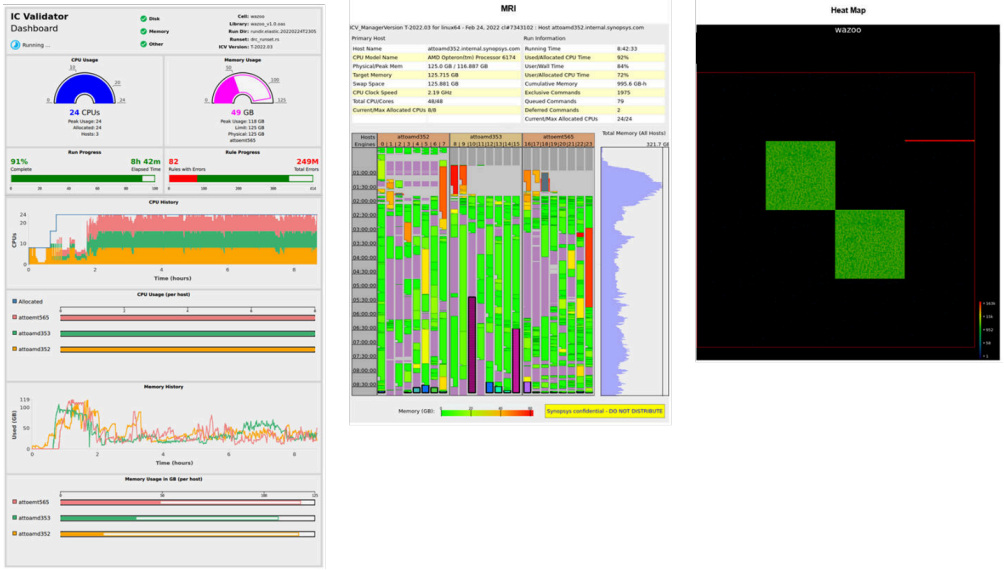
IC Validator Dashboard can send you updates by email. IC Validator sends them automatically based on the percentage of completed updates written to the icv.log file. You can enable these updates by specifying `-email dashboard` on the IC Validator command line when you start the run. For example:

```
icv ... -email dashboard ... runset.rs
```

The emails contain the three pages of content, in which IC Validator Dashboard shows an MRI snapshot colored by memory usage, and a heat map of violation density.

Feedback

Figure 7 IC Validator Dashboard Email Contents



Starting IC Validator Dashboard Standalone

You can start the IC Validator Dashboard using the following command:

```
icv_dashboard [-email ["user1,user2,..."]] [-V] [-h]
 [<run_details_path>|<dp_log_file>]
```

If you do not provide any arguments, Dashboard looks for a distributed processing log file in the ./run_details directory and opens the graphical user interface (GUI).

The command-line options are described in Table 23.

Table 23 IC Validator Dashboard Command-Line Options

Option	Description
`-email ["user1,user2,…"]`	Sends a single email update to the user and exits.
`-V`	Prints the product version and exits. Other command-line options are ignored.
`-h`	Prints program options and exits. Other command-line options are ignored.

Feedback

Table 23 IC Validator Dashboard Command-Line Options (Continued)

Option	Description
run_details_path|dp_log_file	Displays the GUI using either the specified run details directory or the DP log file. The default is ./run_details/. When the run details directory is specified, the distributed processing log file in that directory is used. If more than one distributed processing log file exists in that directory, the log file to use must be specified explicitly.

Note:

To start IC Validator Dashboard in this way, ensure that your PATH variable contains $ICV_HOME_DIR/bin, and that either LM_LICENSE_FILE or SNPSLMD_LICENSE_FILE environment variables point to a valid Synopsys license server with an IC Validator license.

Initial Directory Setting

The IC Validator Dashboard automatically creates an initial directory to store intermediate files. You can specify a path for the initial directory by setting the environment variable, `DASHBOARD_TMP`. If this path is set and valid, then the IC Validator Dashboard creates the initial directory in this path, otherwise the current directory is the next choice, followed by your home directory. If the IC Validator Dashboard fails to create the initial directory, an error is shown.

The IC Validator Graphical User Interface

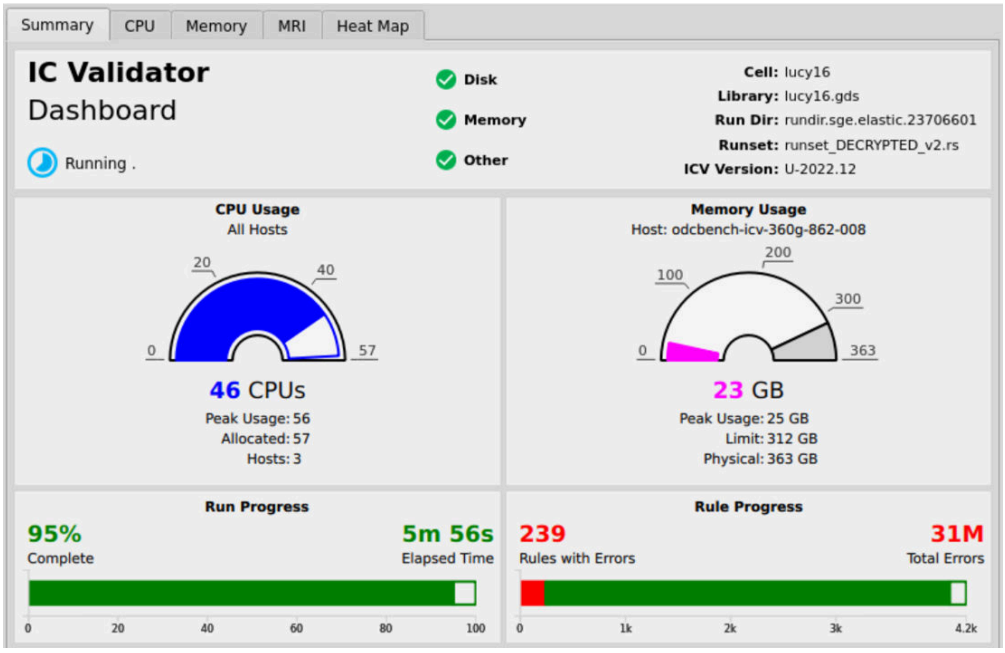
When you start the GUI is started interactively using the command-line options described in the previous section, it continuously monitors the IC Validator run it found or was pointed to. The GUI opens displaying the Summary page which, after it has loaded the information from the run, displays the contents in Figure 8.

IC Validator Dashboard Summary Page

The Summary page shows five panels, which provide information on the status, CPU usage, memory usage, run progress, and rule progress. You can see additional information about the rule progress or status “lights” by hovering the pointer over them until the tooltip appears.

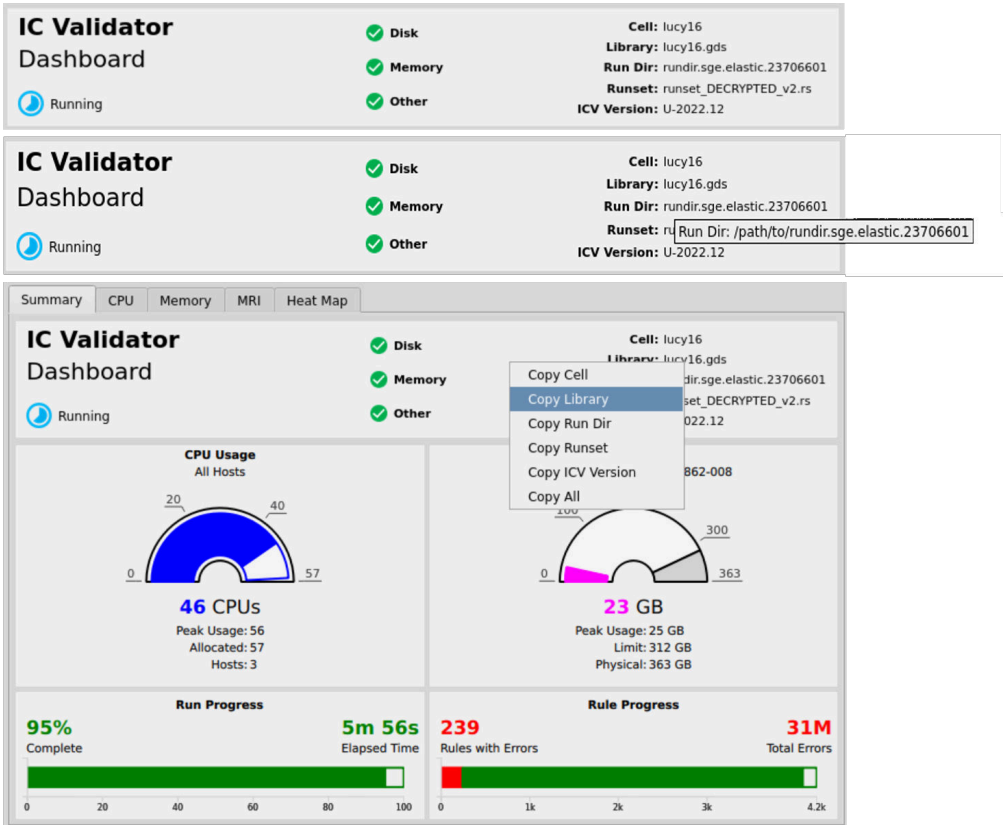
Feedback

Figure 8 IC Validator Dashboard Summary Page



Feedback

Figure 9 IC Validator Dashboard Status Panel



IC Validator Dashboard Status

There are five IC Validator Dashboard status values:

- Waiting

  This is shown when IC Validator Dashboard GUI has started and is waiting for the IC Validator run to start and the distributed processing log file to appear.

- Running

  Shown while the IC Validator run is running and IC Validator Dashboard is waiting for an update.

- Completed

  Shown when the IC Validator run has completed successfully.

- Stopped

  Shown when the IC Validator run has ended with an error

Feedback

- Unknown

  Shown when IC Validator Dashboard has read an incomplete distributed processing log file, but cannot tell if the job is still running. This can occur if a partial distributed processing log file has been loaded when IC Validator is no longer running.

IC Validator Run Status Indicators

In general, these status indicators are green. When IC Validator is short on disk space or memory, the respective indicator turns red. The “Other” status indicator turns red if a command has crashed, a host has been dropped, or if a significant portion of the total runtime is consumed by deferred commands. Hover the pointer over the indicator to display the reason for the status.

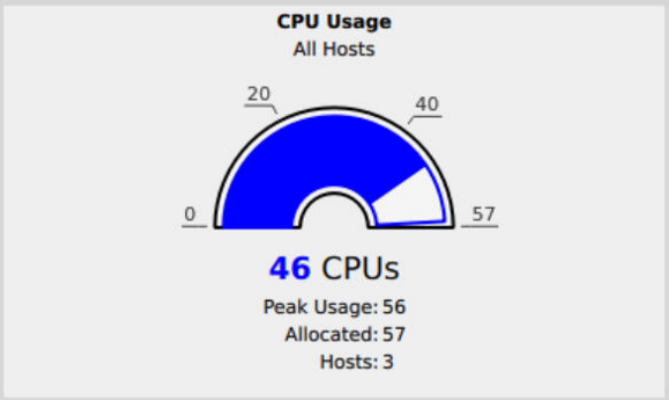
IC Validator Run Information

This section of the status panel displays the primary inputs to IC Validator.

CPU Usage Panel

The CPU usage panel shows the total CPUs in use across all hosts for the IC Validator run. When you run IC Validator in elastic mode, some CPUs might be released while the job is running. In that case, it is possible for the peak usage to exceed the allocated CPUs.

Figure 10 IC Validator Dashboard CPU Usage Panel

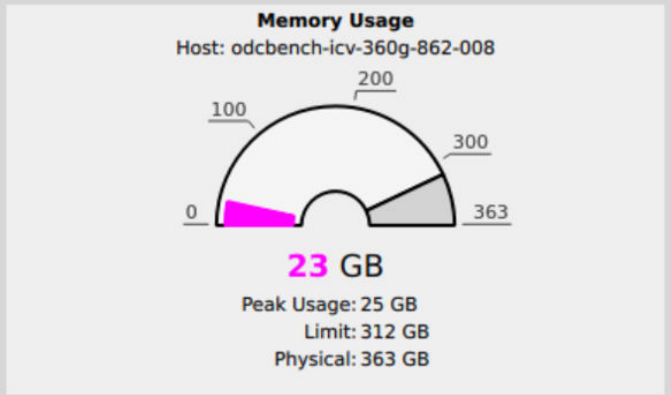


Memory Usage Panel

The Memory Usage panel shows the memory usage of the host using the most memory. When more than one host is allocated to the IC Validator run, this might change throughout the course of the run.

Feedback

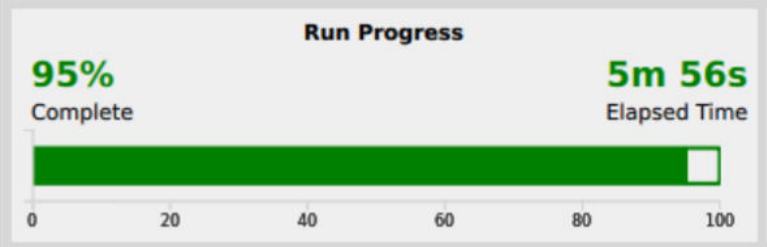
Figure 11 IC Validator Dashboard Memory Usage Panel



Run Progress Panel

The Run Progress panel shows the length of time since the start of the run (elapsed time) and the percentage of operations that IC Validator has completed. The progress bar is a graphical representation of the percent complete.

Figure 12 IC Validator Dashboard Run Progress Panel

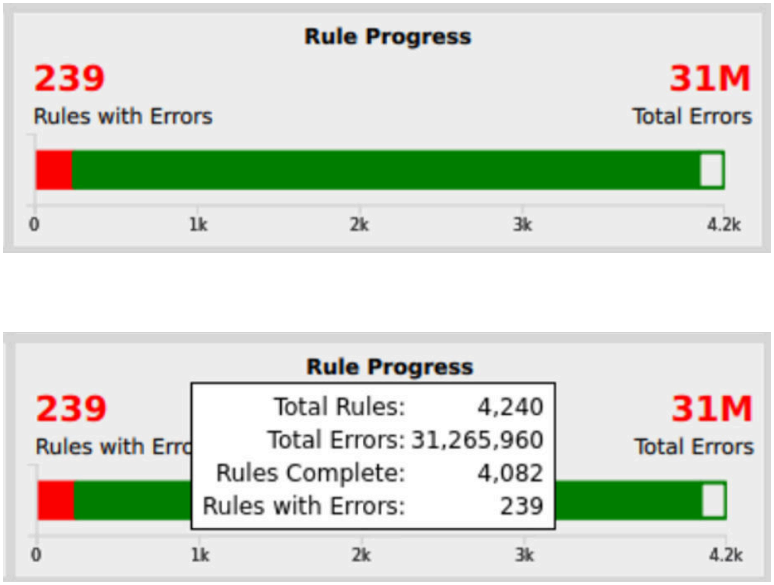


Rule Progress Panel

The Rule Progress panel shows how many rules have been executed and how many errors have been created. As shown in Figure 8, when you hover the pointer over the progress bar, a tooltip displays the precise values.

Feedback

Figure 13 IC Validator Dashboard Rule Progress Panel



When you run IC Validator in Explorer mode, the summary page shows the most recent Explorer Tier messages in the Rule Progress panel. You can see more detailed information by hovering over the message to display the Tooltip.

IC Validator Dashboard CPU History Page

The CPU page, as shown in Figure 14 shows the history of the total CPU usage color-coded by host. The upper panel shows a graph over time of the total number of CPUs allocated and those being used by each host. The lower panel serves as a legend for the graph and shows a current snapshot of the status for each host. When you hover the pointer over different elements, tooltips appear with additional information.

Feedback

Figure 14 IC Validator Dashboard CPU History Page

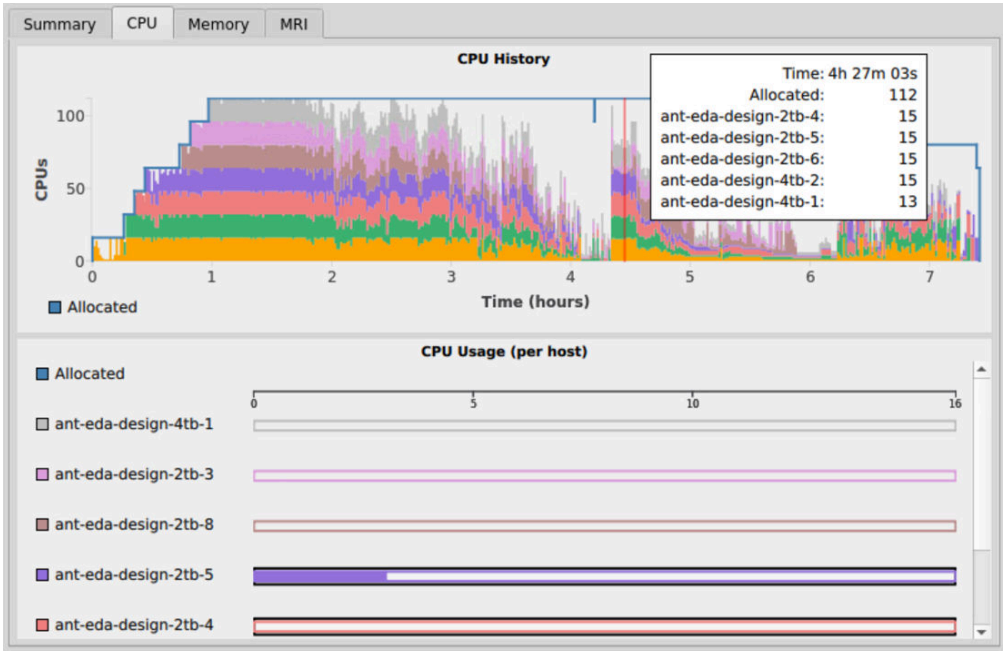
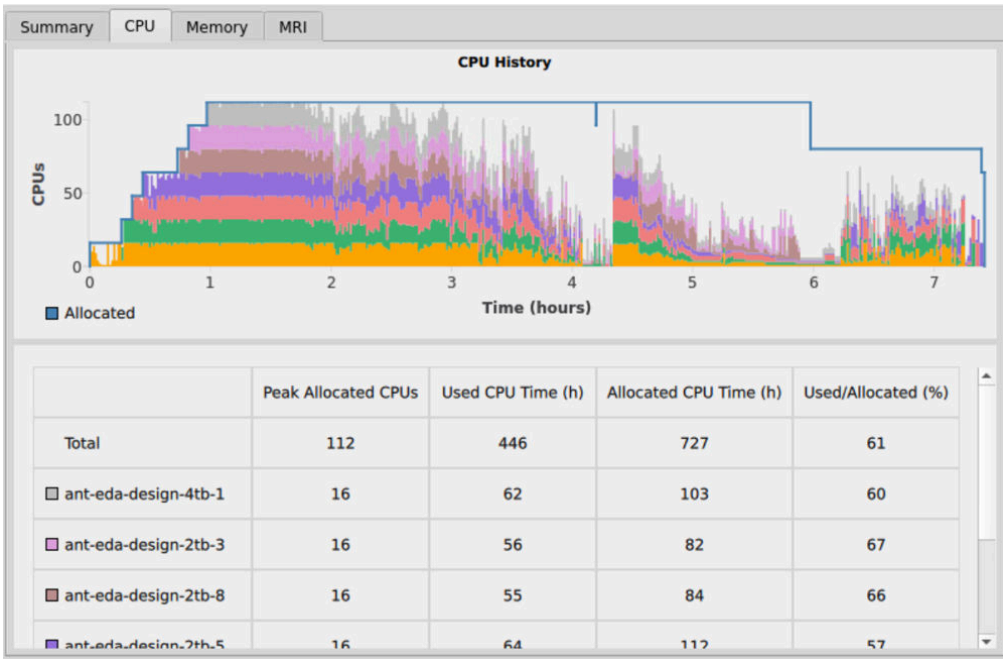


Figure 15 CPU History Page When Job Completes



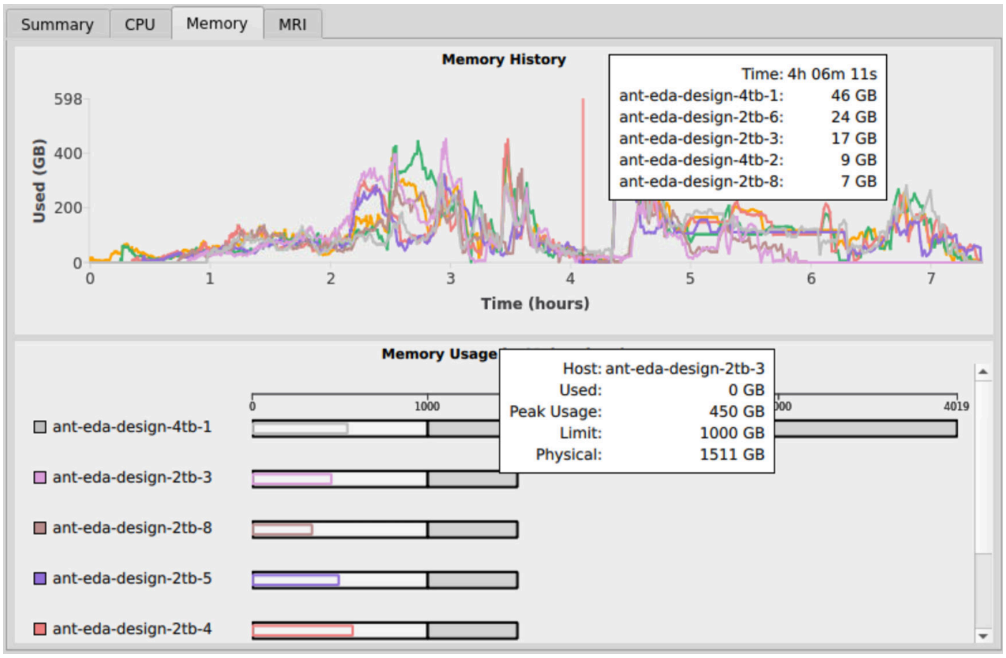
	Peak Allocated CPUs	Used CPU Time (h)	Allocated CPU Time (h)	Used/Allocated (%)
Total	112	446	727	61
ant-eda-design-4tb-1	16	62	103	60
ant-eda-design-2tb-3	16	56	82	67
ant-eda-design-2tb-8	16	55	84	66
ant-eda-design-2tb-5	16	64	112	57

Feedback

IC Validator Memory History Page

The Memory page, as shown in Figure 16 shows the history of the memory usage by host. The upper panel shows the memory usage by host over time. The lower panel shows a current snapshot of the memory usage for each host. The solid bar shows the current memory usage, the colored outline shows the peak memory usage, and the black outline shows the total physical memory of the host. If you specified a host memory limit using the IC Validator `-host_memory` command-line option, the portion of the black outline above that limit is shaded gray. When you hover the pointer over different elements, tooltips appear with additional information.

Figure 16 The IC Validator Dashboard Memory History Page

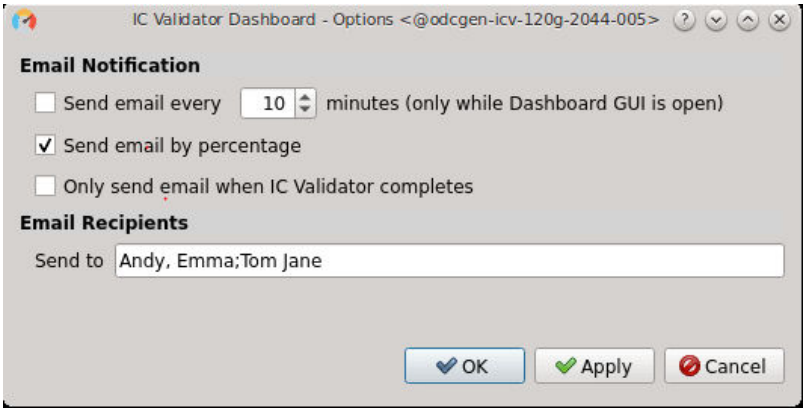


IC Validator Dashboard Options

Click the mail icon on the menu bar to open the IC Validator Dashboard – Options dialog box. Here, you can select up to three different email notification options. Time-based emails are sent every 10 minutes while the GUI is open. Event-based emails are sent each time a new progress line is printed. If IC Validator was started with the `-email dashboard` command-line option, this box is checked. You can disable event-based emails by unchecking that option or a notification email is sent when the ongoing ICV job completes or fails.

Feedback

Figure 17 IC Validator Dashboard - Options Dialog Box



Dashboard emails contain snapshots of the information displayed on each tab of the GUI.

6

IC Validator Live DRC

This appendix discusses the IC Validator Live DRC tool usage within the Custom Compiler, IC Compiler II, Fusion Compiler, and Cadence Virtuoso tools.

The IC Validator Live DRC tool is described in the following sections:

- About the IC Validator Live DRC Tool
- IC Validator Live DRC With Custom Compiler
- IC Validator Live DRC With Virtuoso
- IC Validator Live DRC With IC Compiler II and Fusion Compiler

About the IC Validator Live DRC Tool

Long DRC runtimes on advanced nodes and large designs can affect productivity. The IC Validator Live DRC tool enables you to run DRC directly within a layout editor, helping you fix violations actively and eventually generate a much cleaner layout that reduces signoff DRC runtime.

The IC Validator Live DRC tool uses the IC Validator signoff rule file so that the checks are comprehensive. However, because you are limiting the checks to the viewport and visible layers only, the runtime is significantly faster. The IC Validator Live DRC tool is supported within the Custom Compiler™, IC Compiler™ II, Fusion Compiler™, and Cadence® Virtuoso® Layout Editors.

IC Validator Live DRC With Custom Compiler

IC Validator Live DRC in Custom Compiler allows designers to run IC Validator physical verification checks on active layouts. IC Validator Live DRC allows you to get instant DRC results using the same runset and files as IC Validator. Advanced options like recipes allow you to create subsets of rules which can be selected and executed from a pull-down menu. For better ease of use, the violations are displayed within Custom Compiler without the need for a separate window to view violations. The following sections describe the steps to set up IC Validator Live DRC, execute a run, and view violations.

Feedback

Setting Up the Environment

To run IC Validator Live DRC within Custom Compiler, follow these steps:

1. *Set up the environment*

   Some IC Validator environment variable settings are required before starting Custom Compiler. To load the IC Validator Live DRC menus, you must set two environment variables:

   - Set ICV_HOME_DIR to the IC Validator installation directory
   - Set PATH to include $ICV_HOME_DIR/bin

   For example:

   ```
   setenv ICV_HOME_DIR </path/to/icv>
   setenv PATH $ICV_HOME_DIR/bin:$PATH
   ```

   To make the IC Validator Live DRC toolbar visible, select **Window > Toolbars > DRC**.

2. *Open the IC Validator Live DRC Setup Window*

   To access IC Validator Live DRC using the **Verification** menu, select **Tools > Verification > Live DRC** or click  .

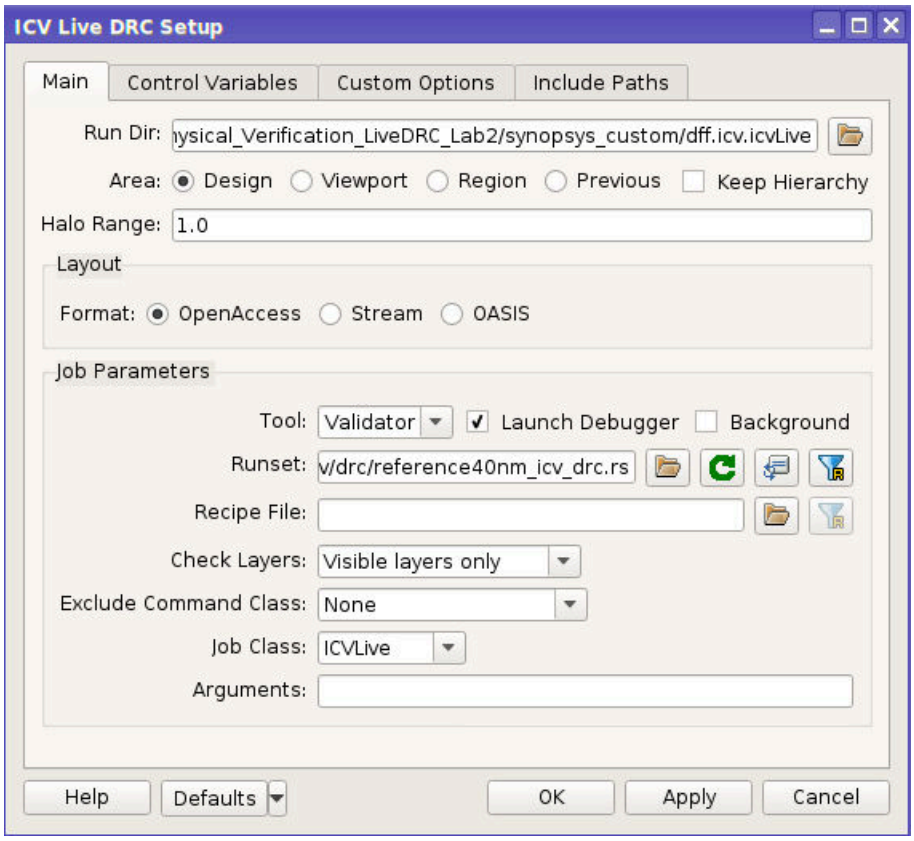
3. *Required files and settings*

   From the **Live DRC Setup** dialog box, select one of the following tabs: **Main**, **Control Variables**, **Custom Options**, **Include Paths**.

   a. **Main** tab

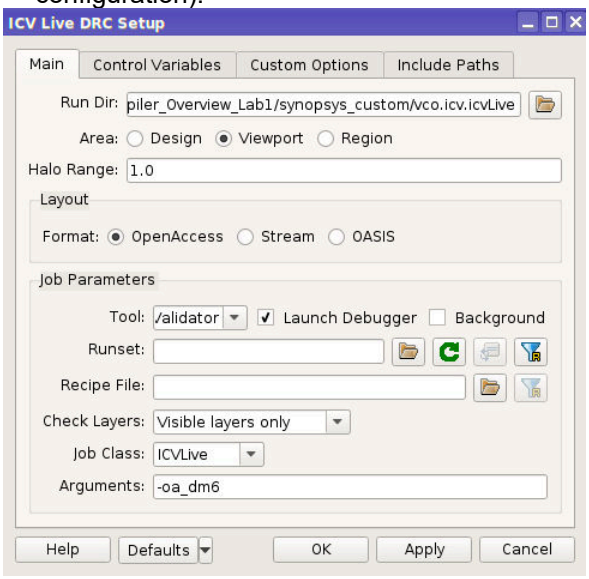
Feedback

Figure 18 Main Tab



Settings	Description
Run Dir	Select the directory in which you want to run IC Validator Live DRC. The output files are stored in this directory.
Area	Area on which to run the DRC check as design, viewport, or region, specified by coordinates of its rectangle in microns, in this order [x1, y1, x2, y2] for example: [-15.5, -20.0, 15.5, 20.0]. Click the interactive selection button to draw a selection region in the design.
Layout Format	For the **OpenAccess** format, verification runs on the current cell view.

Feedback

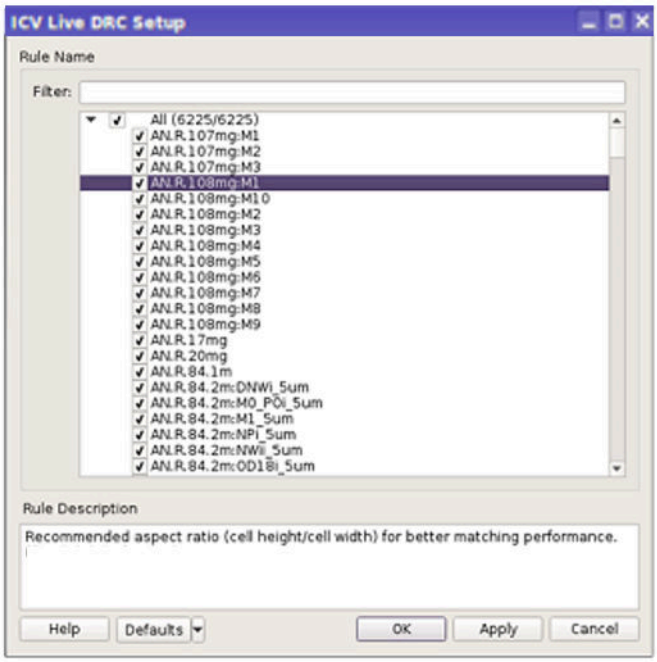
Settings	Description
	For the **Stream** and **OASIS** formats, the tool exports **Stream** or **OASIS** and runs IC Validator Live DRC on the generated GDSII or OASIS file. Set the following: ◦ **Format**: **OpenAccess**, **Stream**, or **OASIS**: Exports the layout ce llview used as input to the **File** you specify. ◦ Click the **Config** button to open the open the **Export Stream** or **Export OASIS** dialog box. To set the options in the dialog box, see the “Exporting Design Data in GDSII Stream Format” or “Exporting Design Data in OASIS Format” sections of the *Custom Compiler Environment User Guide*. The options you specify are integrated with PVE (preferences in the `pdk.xml` configuration). 
Job Parameters	Select the following: ◦ **Runset:** The design rule (runset) file used for IC Validator Live DRC. To select a subset of the rules, click  . If you want to use a rule file that has been modified since it was loaded, reopen the dialog box and click  to load the modified file. ◦ **Arguments**: The command-line arguments that control the behavior of the IC Validator Live DRC. You do not need to select the **Tool** and the **Job Class**. **Launch Debugger** launches (or updates existing) **Error Viewer** when the process completes. In **Background** mode, you can continue using Custom Compiler while IC Validator Live DRC checking runs in the background. Use the preference `xtICVLiveNotifyWhenComplete` to get a notification dialog box when IC Validator Live DRC checking completes.

Feedback

To select specific rules from the runset (rule) file, click  . In the **Live DRC Rule Selection** dialog box, select the rules you want to use for the rule checking, and click **OK**.

In the filter box, enter the full or partial name of the item or rule you want to select using a simple string match, a glob-style pattern, or a regular expression which is controlled by the **Filters** option in the **General Options** dialog box. To clear a filter box, click X as shown in the text field. (See “Entering General Option Settings” section of the *Custom Compiler Environment User Guide*).

Figure 19 Rule Selection



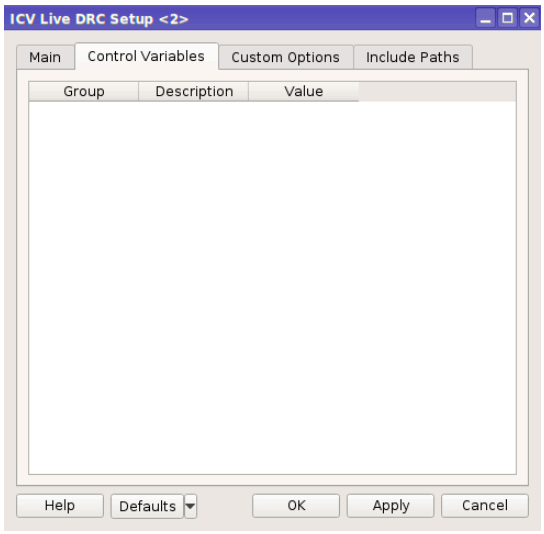
b. **Control Variables** tab

In the **Control Variables** tab, review, and optionally modify the current values for DRC runset control variables as preferred for this DRC run. The control variables are preset for a PDK. To modify a preset value, click with the alternate (non-selection) mouse button, and edit the entry field.

The control variable setup in this dialog box is similar to the setup in the **DRC Setup and Run** dialog box. See the “Control Variables Tab” and the “Using the Integrated Tools for Design Rule Checking” sections of the *Design Checking and Physical Verification User Guide* for more information.

Feedback

Figure 20 Control Variables Tab



c. **Custom Options** tab

The layer mapping file maps OpenAccess (OA) layer-purpose names to GDSII or OASIS layer number and data types. The file is used by IC Validator Live DRC for incremental DRC.

You can specify the **Layer Map** file in the **Custom Options** tab. In addition, you can add **Options** to parse the IC Validator Live DRC runset. For the list of options, see the “Using the Integrated Tools for Design Rule Checking” section of the *Design Checking and Physical Verification User Guide*.

You can use the same layer map file as is used for streaming out GDS or OASIS files. Often these can be found from previously defined preferences by typing:

```
db::getPrefValue dbExportStreamLayerMapFile
```

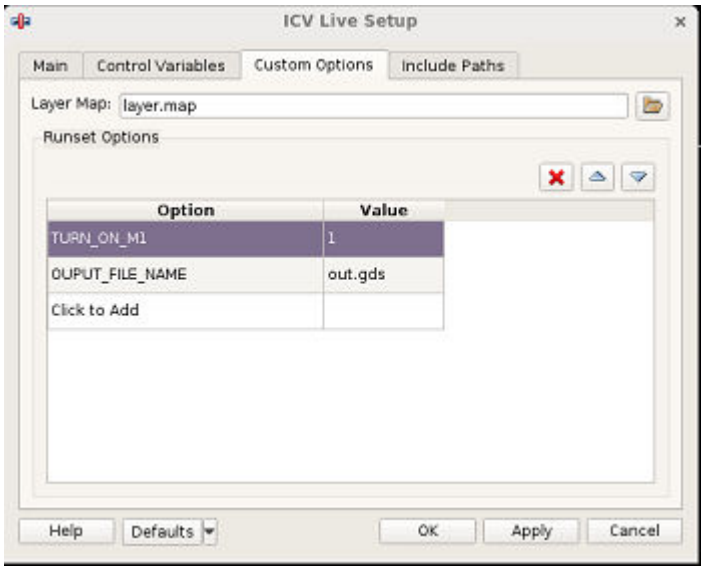
or

```
db::getPrefValue dbExportOasisLayerMapFile
```

in the console.

Feedback

Figure 21 Custom Options Tab

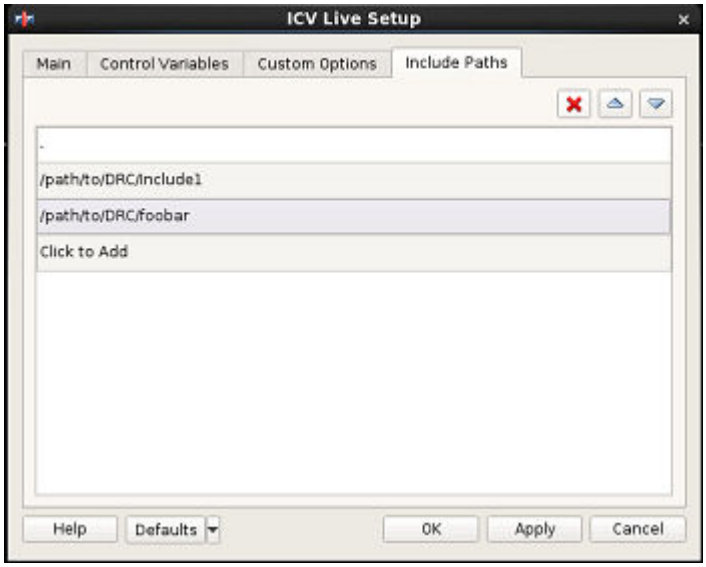


Note:

d. **Include Paths** tab

In the **Include Paths** tab, you can specify the folder that contains additional files to pass to IC Validator Live DRC during the DRC operation.

Figure 22 Include Paths Tab

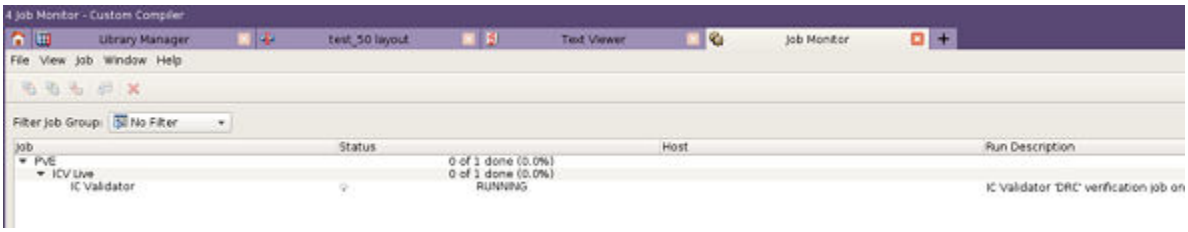


Feedback

Viewing Job Progress

You can view your IC Validator Live DRC jobs in the job monitor, which are displayed with the correct **Status** and **Run Description**.

Figure 23 Job Monitor



When you right-click a selected IC Validator Live DRC job, a menu appears from which:

- Running jobs can be terminated using the **Kill** command
- You can view the output file (icvl_manager.log)

Figure 24 Kill Command



If the total number of polygons in the current view window is more than 50,000, a warning appears because IC Validator Live DRC runtime performance can degrade beyond this point. If you select, “Do not show this message again,” the future runs use IC Validator Live DRC unless the maximum polygon count limit (currently 2,000,000) is reached. In this case, IC Validator Live DRC cannot be run.

Feedback

Running IC Validator Live DRC

To start an IC Validator Live DRC run:

1. In the Layout Editor, arrange the layout so that the region you want to check is within the viewport.
2. Make sure the layers you want to check are visible. You can change the visibility of layers using one of the following methods:
   - On the **Hierarchy toolbar**, set the Stop Level.
   - In the **Object/Layer Panel**, display or hide layers.
3. On the **Live DRC** toolbar, click 

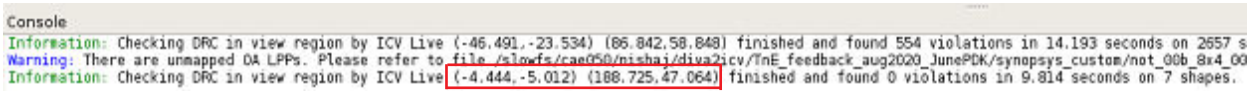
:

   The DRC process starts. When it is finished, the violation markers appear on the canvas. The violations are also listed in the Error Viewer.

The OA markers are displayed in the Error Viewer (choose **Windows** > **Assistants** > **Error Viewer**).

You can control whether IC Validator Live DRC displays a notification dialog box when checking completes by specifying the xtICVLiveNotifyWhenComplete preference.

Figure 25 IC Validator Live DRC Console



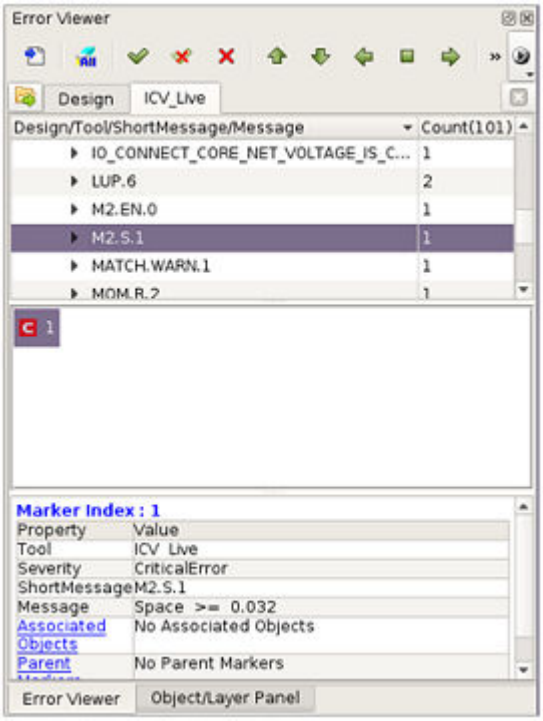
Feedback

Viewing and Fixing Violations

To view the DRC violations:

1. Display the **Error Viewer**.
2. Click the **ICV_Live** tab to view the list of DRC errors.

Figure 26 Error Viewer Assistant



Additional Options

Additional options like halos and recipes allow you to better control the run. For example, halos expand the data that is being checked to include some area immediately outside the view window and recipes allow you to create and run subsets of rules that can be selected from a pull-down menu.

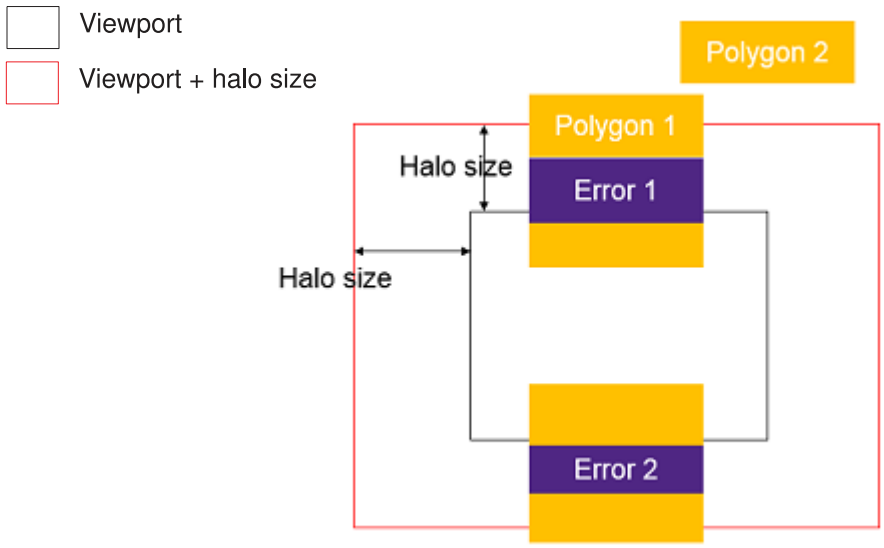
Halos

To run IC Validator Live DRC on data that is immediately outside the data in the view window, add a halo to the run. Adding a halo increases the size of the check window, as shown in Figure 27. By expanding the data checked to include the polygons immediately outside the view window, you can verify that design changes within the view window

Feedback

did not create any new DRC violations with the polygons immediately outside the view window.

Figure 27 IC Validator Live DRC Configuration - Halo Size



The black window is the viewport and the red window is the viewport plus the halo. If IC Validator Live DRC is executed with the halo setting:

- Only polygons that have some portion of their area inside the red window are included in the check. (Polygon 1 is included, but Polygon 2 is not).
- Only error shapes that overlap the black window are included in the result (Error 1 included, but Error 2 is not).

You can define the halo range using IC Validator Live DRC by specifying the xtICVLiveHaloRange preference. You can control whether IC Validator Live DRC writes out two OA markers to indicate the checking range by specifying the xtICVLiveDumpCheckingRange preference, in which case one marker shows the checked viewport, and the other marker considers the halo range.

Recipes

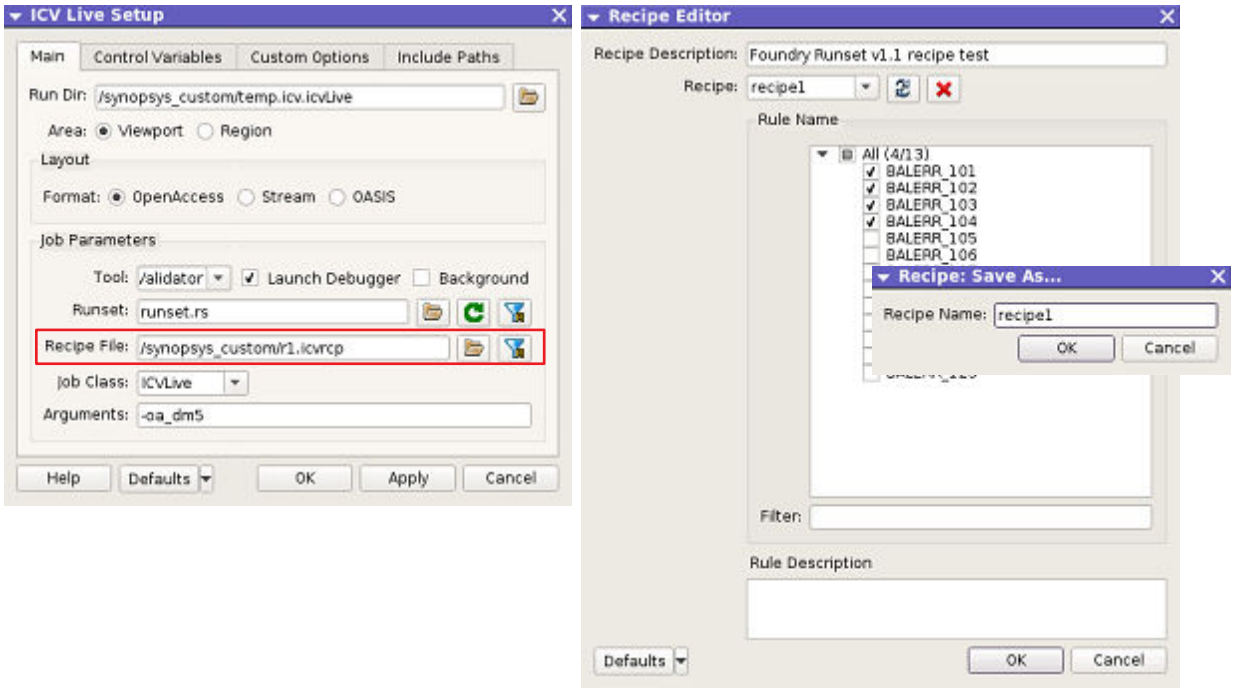
Recipes are JSON text files that contain a subset of rules from the signoff runset. Recipes allow you to run specific rules based on the design stage. For example, a recipe containing spacing rules or rules related to front-end layers can be executed at the very early stages of a design. Recipes with more advanced rules or rules related to back-end layers can be executed toward the end of the design cycle. You can create new recipes or modify existing recipes by setting the **Recipe File** path, which contains multiple recipes that can be defined using the **Recipe Editor**.

Feedback

To invoke the **Recipe Editor**:

1. From the **Live DRC Setup** dialog box, **Main** tab, select an existing **Recipe File** and click  , as shown in Figure 28, to load the **Recipe Editor**.
2. Click  , as shown in Figure 28, to choose a recipe from the **Select Recipe File** dialog box. Click **OK** to load the **Arguments** field with the selected recipe.

Figure 28 Recipe Editor With Arguments

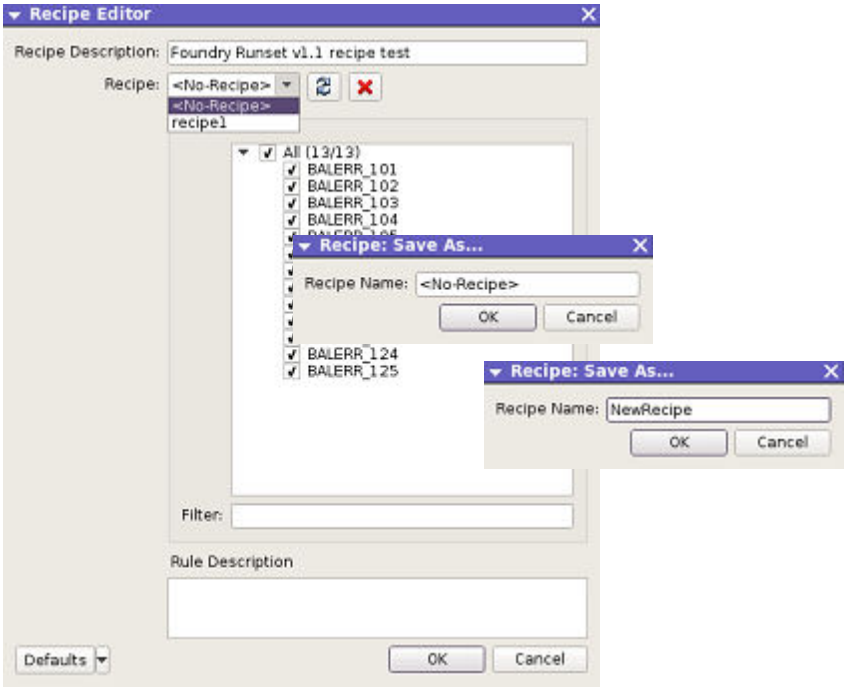


3. Choose **<No-Recipe>**, as shown in Figure 29, to remove the recipe settings from the **Arguments** path and create a new recipe.

Feedback

4. Check or uncheck the rules from the **Rule Name** list that you want to be included in your recipe. Click  to save a new recipe or the settings of an existing recipe. Click  to remove the existing recipe of the specified Recipe File.

Figure 29 Recipe Editor



1. Capture the coordinates of your run from the Console:
2. Create a config.rs file in your run directory using the following lines, and substitute the arguments in `select_window` with the coordinates of the IC Validator Live DRC run from the previous step:

```
hierarchy_options(
   explode_all = {"*"}
);
incremental_options(
   select_window={{-4.444, -5.012, 188.725, 47.064}},
   clip_window=false,
   window_error_filter = KEEP_ENCLOSED
);
```

IC Validator Live DRC runs on flat data. Therefore, you must use the `hierarchy_options()` function to flatten the input to the IC Validator run. Although IC Validator Live DRC does not clip polygons at the edge of the view window, it does not report any violations outside of the view window. Therefore, the `clip_window` and

Feedback

`window_error_filter` arguments must be used with the `incremental_options()` function.

3. Add this file to your IC Validator run by setting `-runset_config config.rs`.
4. Run IC Validator.

IC Validator Live DRC With Virtuoso

IC Validator Live DRC in Virtuoso allows you to run IC Validator physical verification checks on active layouts. IC Validator Live DRC allows you to get instant DRC results using the same runset and files as IC Validator. Advanced options like recipes allow you to create subsets of rules which can be selected and executed from a pull-down menu. Violations are displayed within Virtuoso without the need for a separate window to view violations. The following sections describe the steps to set up IC Validator Live DRC, execute a run, and view violations.

Setting Up the Environment

To set up IC Validator Live DRC with Virtuoso, some environment variables must be set so the IC Validator installation can be located by IC Validator Live DRC.

1. *Set up the environment*

   Set the ICV_HOME_DIR environment variable to the location where IC Validator is installed.

   Set the SYNOPSYS_SYSTYPE environment variable appropriately for your system. Typically, it should be set to linux64.

   Start Virtuoso, and in the CIW window type,

   ```
   load( strcat( getShellEnvVar("ICV_HOME_DIR") \
   "/etc/icvlive/virtuoso/icvl.il"))
   ```

   Alternatively, this command can be added to the .cdsinit file to automatically load the IC Validator Live DRC toolbar in Virtuoso.

   When Virtuoso is opened, the IC Validator Live DRC toolbar is displayed, as shown in Figure 30. It might be necessary to stretch the toolbar so that all of the buttons can be seen.

Figure 30 IC Validator Live DRC Toolbar



2. Open the IC Validator Live DRC Configuration dialog box.

Feedback

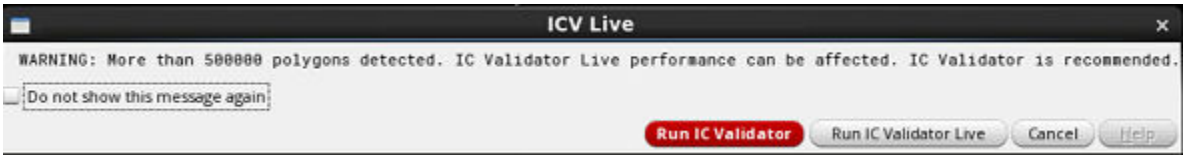
Click  : from the toolbar (or keyboard shortcut, Shift+F12)

The Configuration consists of three parts:

- Run Settings: Determines the parameters that are used to check.
- Check Filter: Determines which layers and rules are going to be checked (or not checked).
- Highlight Control: Determines the behavior when browsing IC Validator Live DRC results.

If the total number of polygons in the current view window is more than 50,000, a warning appears, as shown in Figure 31. If you select, the **Do not show this message again** checkbox," future runs will use IC Validator Live DRC unless the maximum polygon count limit (currently 2,000,000) is reached. In this case, IC Validator Live DRC cannot be run.

Figure 31 Configuration - Polygon Count Limit



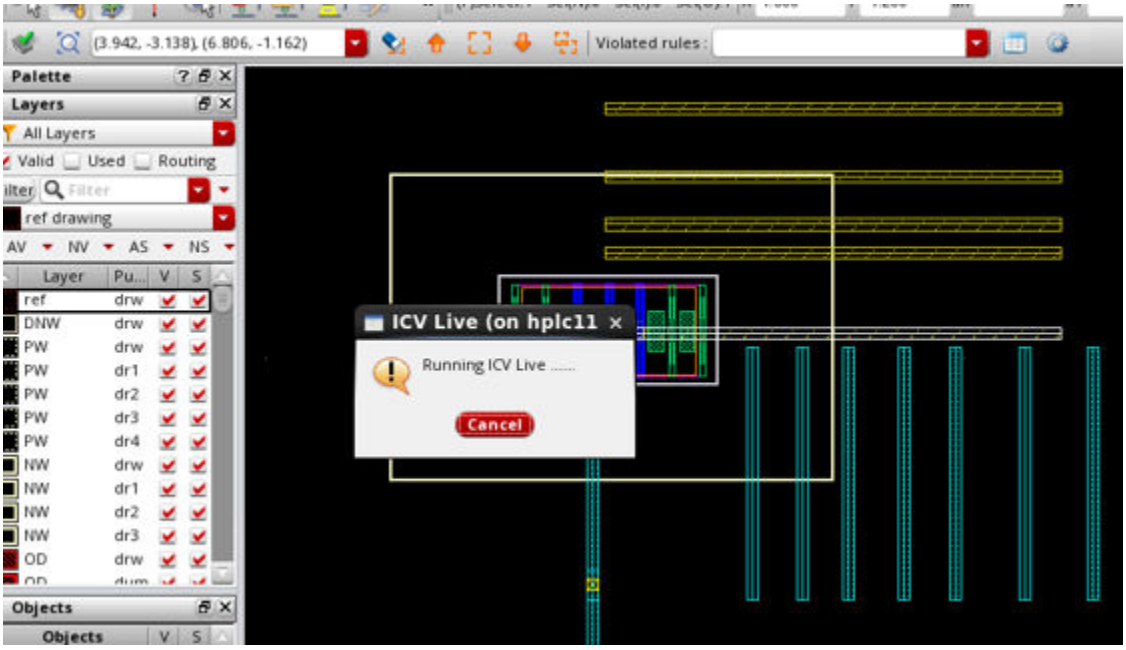
Starting the IC Validator Live DRC Check

Click  to start the IC Validator Live DRC Check (or keyboard shortcut, F12).

During the check, there is a modal dialog bbox. Click Cancel to terminate the check, as shown in Figure 32.

Feedback

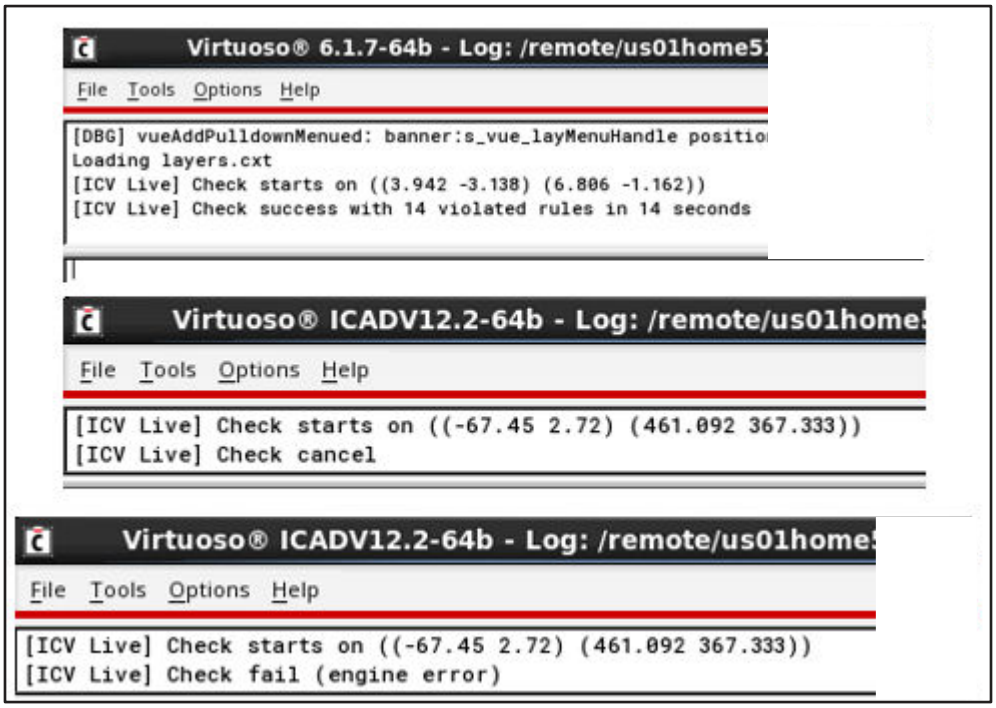
Figure 32 Running IC Validator Live DRC



There are related messages in the Virtuoso command window. IC Validator Live DRC messages are prefixed with "[ICV Live]". They communicate to the user what IC Validator Live DRC is doing and give information about why a job might have terminated. Some examples are shown in Figure 33.

Feedback

Figure 33 IC Validator Live DRC Messages



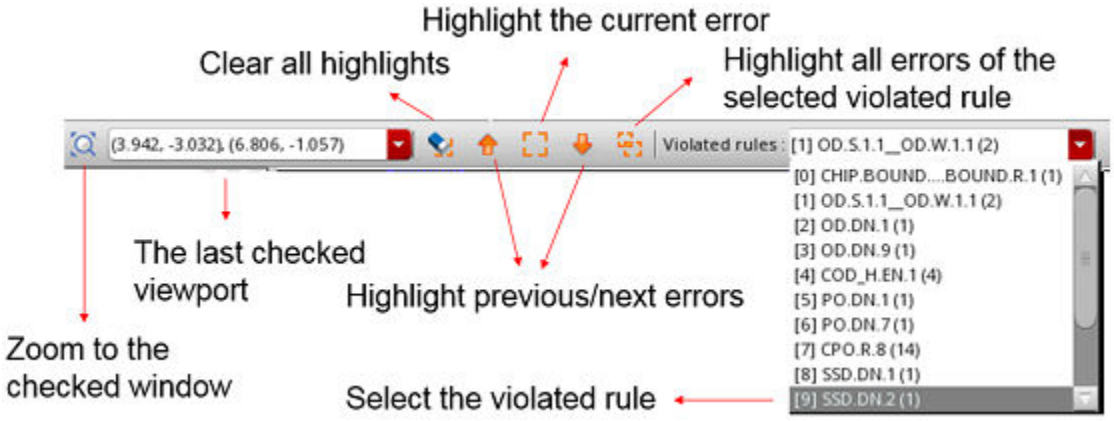
Browsing Results in IC Validator Live DRC

Use the IC Validator Live DRC toolbar to:

- Select a violated rule (by the violated rules box on the right-hand side)
- Browse the errors of the selected rule (by the five buttons in the middle)
- Zoom to the viewport (by the magnifier and coordinate box on the left side)

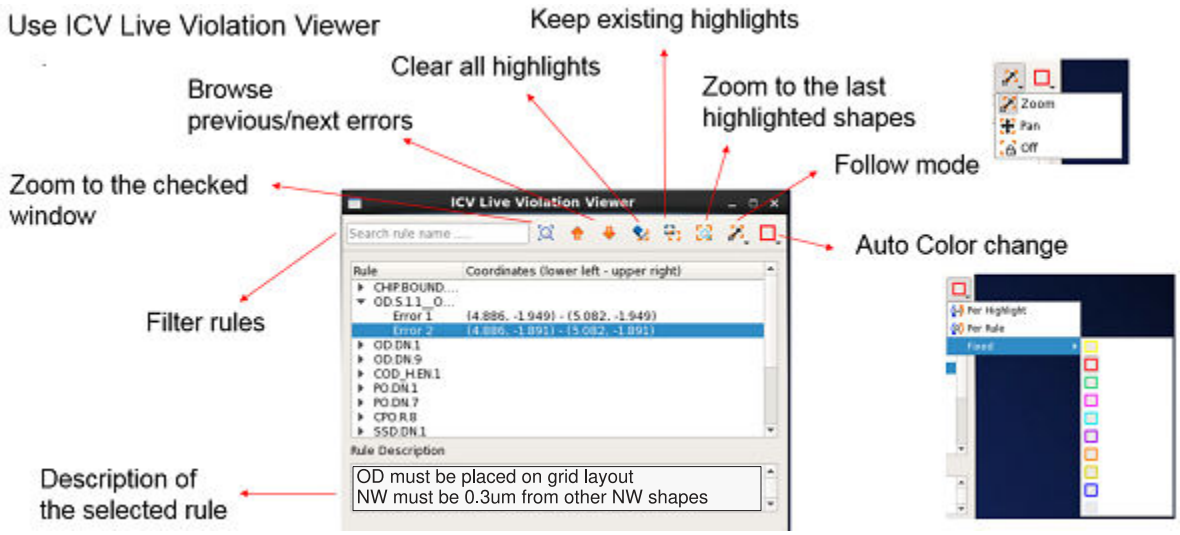
Feedback

Figure 34 IC Validator Live DRC Toolbar



IC Validator Live DRC also has a pop-up violation viewer with a detailed description of the violated rules and more violation highlight options. The IC Validator Live DRC Violation Viewer is shown in Figure 35. To bring up this violation viewer, click the icon in the IC Validator Live DRC toolbar.

Figure 35 Browsing Live DRC Results



Use the Highlight Control to:

- Keep Existing Highlights

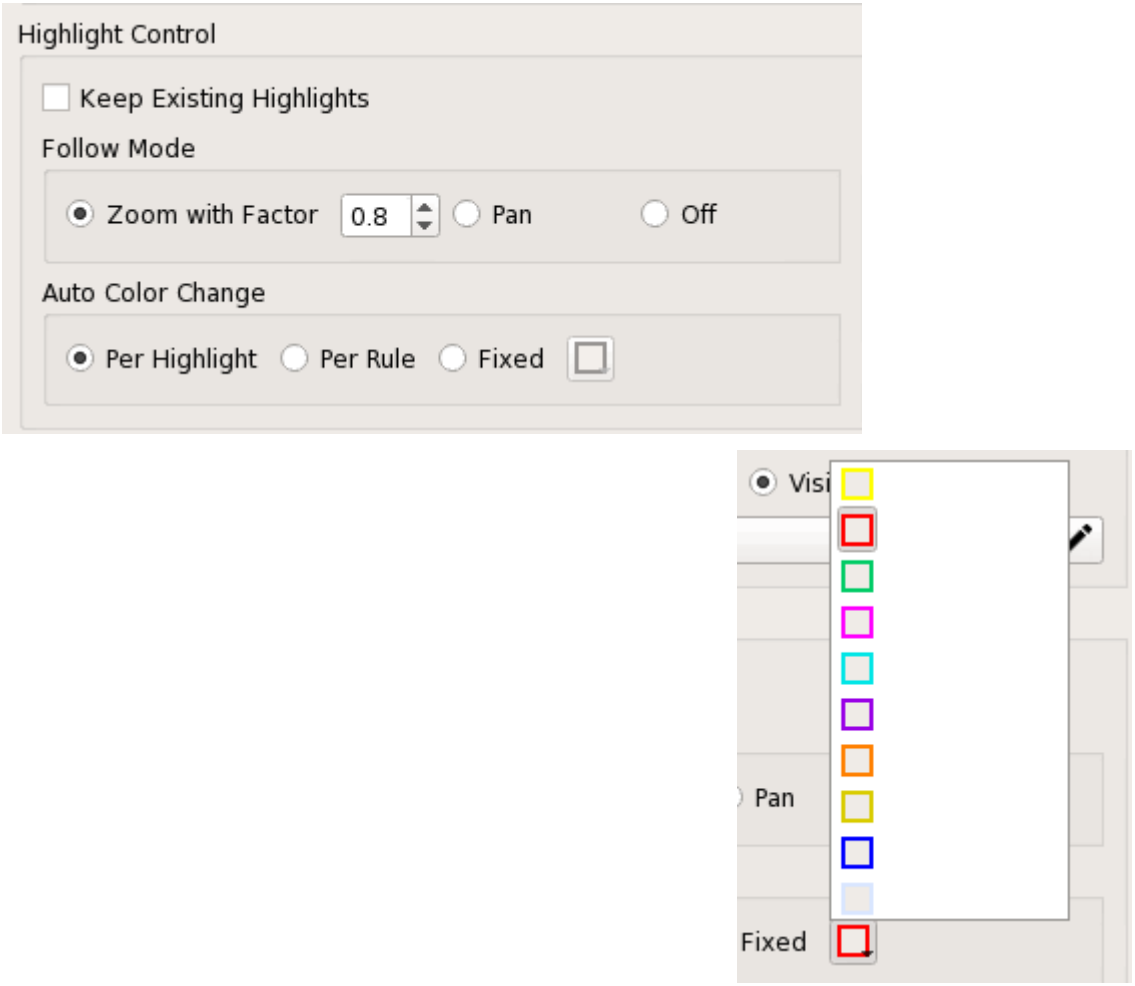
  - If checked, the previous highlight is not cleared.

- Follow Mode

    - Zoom with Factor: Zoom to the highlight shape with factor
    - Pan: Move to the center of highlight shape without changing zoom
    - Off: Do nothing
- Auto color change (Use Virtuoso's y0~y9 layers to highlight)
    - Per Highlight: Change color every highlight
    - Per Rule: Change color only when the current and previous highlights do not belong to the same rule.
    - Fixed: Use the chosen color to highlight
- The settings in Highlight Control are synchronized with the settings in the Violation Viewer.

Feedback

Figure 36 IC Validator Live DRC Highlight Control in the Configuration Window



Additional Options

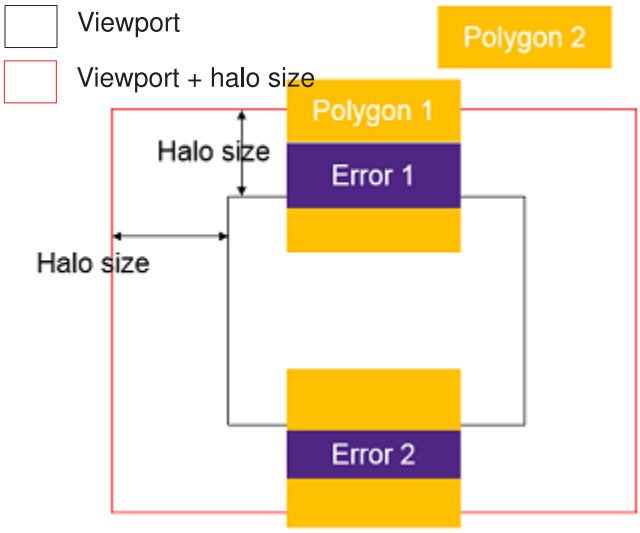
Additional options like halos and recipes allow you to better control the run. For example, halos expand the data that is being checked to include some area immediately outside the view window and recipes allow you to create and run subsets of rules that can be selected from a pull-down menu.

Halos

Adding a halo increases the size of the check window, as shown in Figure 37. By expanding the data checked to include the polygons immediately outside the view window, you can verify that design changes within the view window did not create any new DRC violations with the polygons immediately outside the view window.

Feedback

Figure 37 IC Validator Live DRC Configuration - Halo Size



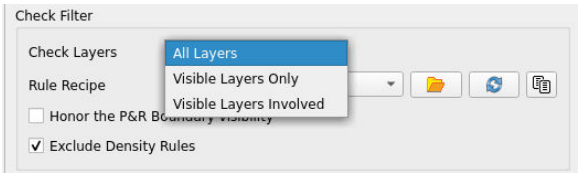
The black window is the viewport and the red window is the viewport plus the halo. If IC Validator Live DRC is executed with the halo setting:

- Only polygons that have some portion of their area inside the red window are included in the check. (Polygon 1 is included, but Polygon 2 is not).
- Only error shapes that overlap the black window are included in the result (Error 1 included, but Error 2 is not).

Check Layers

The **Check Layers** option controls which rules to include when checking according to visible layers. The available options are shown in Figure 38.

Figure 38 IC Validator Live DRC Configuration - Check Layers



The various options are as follows:

- **All Layers**: Includes all rules to check
- **Visible Layer Only**: Includes rules using visible layers only
- **Visible Layer Involved**: Includes rules that have at least one visible layer involved

Feedback

For example, consider the following three rules and assume only VIA is visible:

```
Rule1 @= {
       @ "Rule1";
       area(VIA, value = [2.0,2.0]);
}
Rule2 @= {
       @ "Rule2"; VIA and M1;
}
Rule3 @= {
       @ "Rule3"; M1 and M2;
}
```

When **All Layers** is selected, Rule1, Rule2, and Rule3 are included.

When **Visible Layer Only** is selected, only Rule1 is included.

When **Visible Layer Involved** is selected, both Rule1 and Rule2 are included.

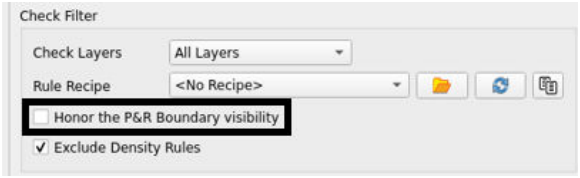
Honor the P&R Boundary visibility

The **Honor the P&R Boundary visibility** option is effective when **Check Layers** is set to **Visible Layer Only** or **Visible Layer Involved**.

When the option is not set, the P&R Boundary object is always treated as visible.

When the option is set, Live DRC honors the visibility the P&R Boundary object on the Virtuoso editor.

Figure 39 IC Validator Live DRC Configuration - Honor the P&R Boundary visibility

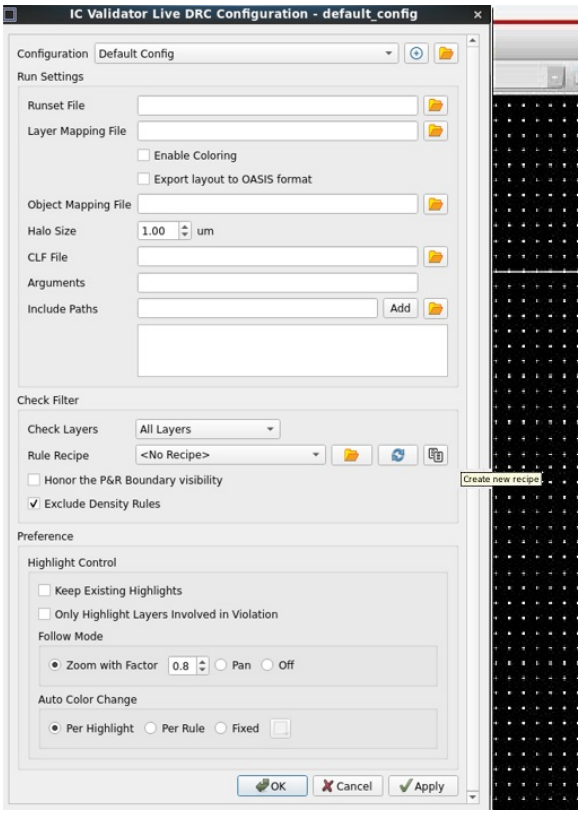


Recipes

Recipes are JSON text files that contain a subset of rules from the signoff runset. Recipes allow you to run specific rules based on the design stage. For example, a recipe containing spacing rules or rules related to front-end layers can be executed at the very early stages of a design. Recipes with more advanced rules or rules related to back-end layers can be executed toward the end of the design cycle. To create a recipe in IC Validator Live DRC, use the following features as shown in Figure 40.

Feedback

Figure 40 Create New Recipe



- Rule Recipe: Defines the rules that are checked in the IC Validator Live DRC run. Live DRC categorizes recipes into two types as shown in Figure 41.
  - User recipes: Recipes in $HOME/.icvlive/recipes.
  - Others recipes: Recipes in $ICVLIVE_RECIPE_PATH.
  - Click  : to refresh the content in the drop-down list.
  - Click  : to open the Recipe Editor and edit the current selected recipe.

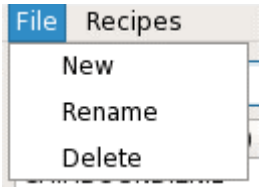
Figure 41 Recipe Categories: User and Others



- File menu has three new actions: New, Rename, and Delete

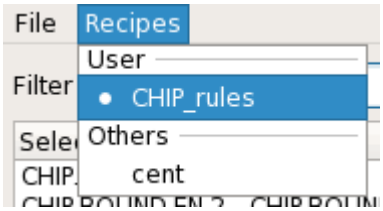
Feedback

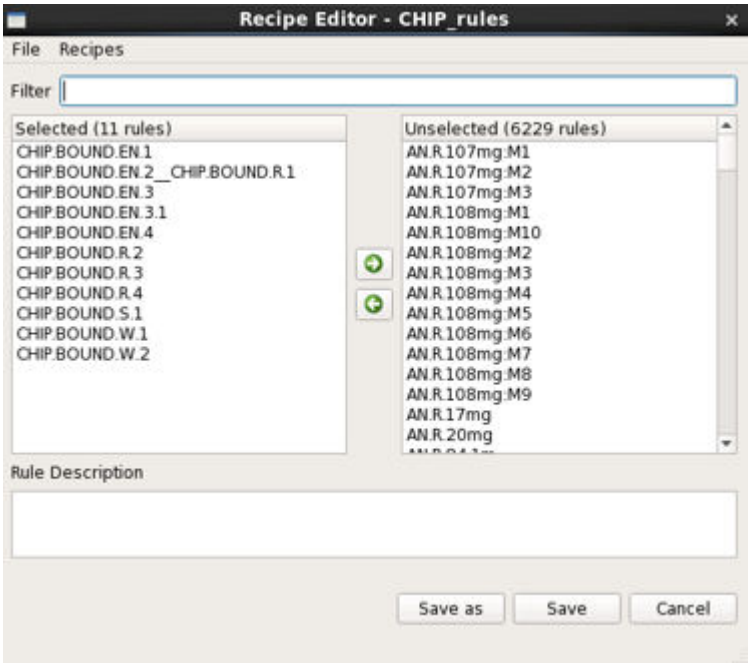
Figure 42 File Menu



- The Recipes menu lists all available recipes and allows you to switch between recipes.

Figure 43 Recipes



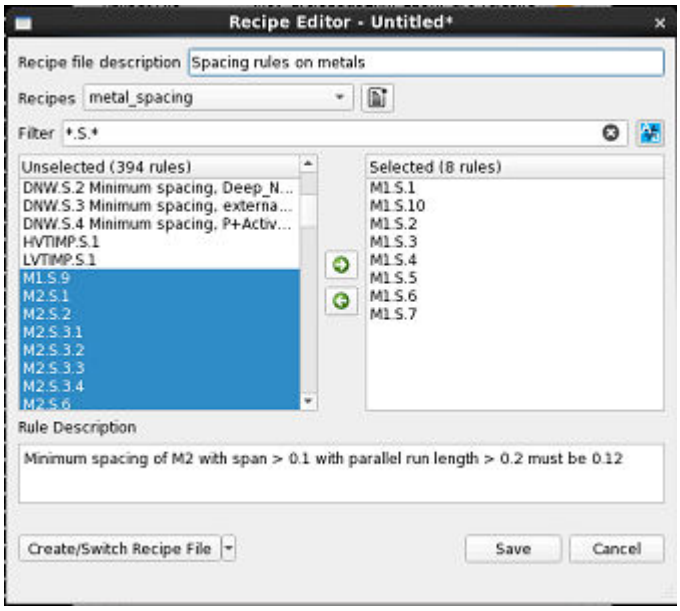


- Selected and Unselected rule list:
  - The total number of rules in each group is show in parentheses in the header.
  - To move rules between Selected and Unselected rule list

Feedback

- Choose rules in the rule list by clicking the items
- Support contiguous selection (press Shift), extended selection (press Ctrl) and select all (Ctrl + a)
- Click  or  to move the rule items

Figure 44 Recipe Creating Rules



Saving and Loading Configurations

Configuration files allow you to save the runset and other settings used in an IC Validator Live DRC run. Saving the settings allows you to get back to your work even when a previous session is closed and a new layout or session is opened. Configuration files are JSON-based text files similar to the IC Validator Recipe files. This section describes how to create and load these files to your IC Validator Live DRC run. To load a configuration file before you run, set the ICV_LIVE_CONFIG_FILE followed by the file or directories separated by colons. For example,

```
setenv ICV_LIVE_CONFIG_FILE file1:/path/to/dir1:file2:$cwd
```

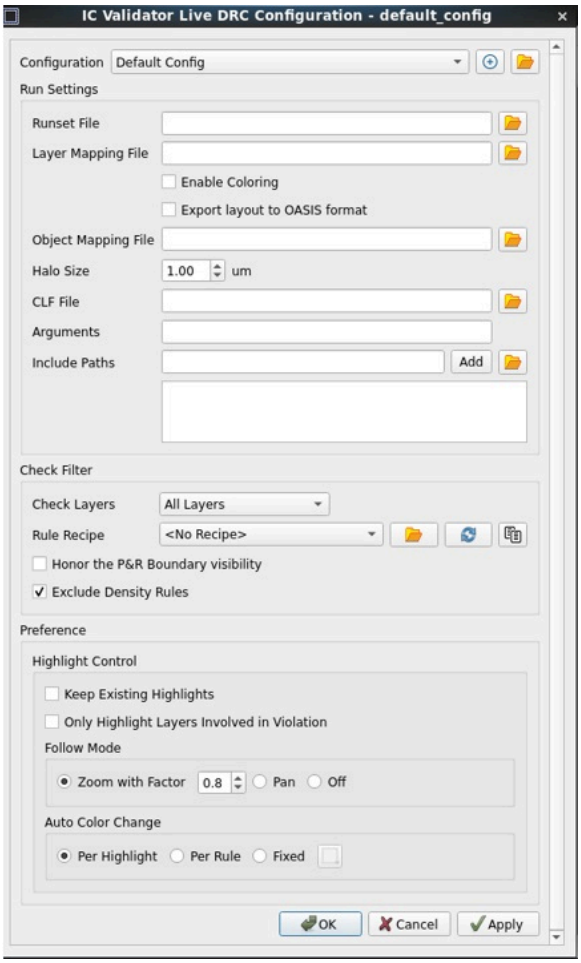
You can edit any files that have write permissions directly from the IC Validator Live DRC Configuration Window.

After you set up a configuration, you can save it by clicking the save button and using the pop-up dialog.

Feedback

If this configuration is commonly used, it might be useful to load it automatically. You can do this with the ICV_LIVE_CONFIG_FILE environment variable. Set this to a colon separated list of files or directories. When the IC Validator Live DRC configuration menu is opened, IC Validator Live DRC reads the configuration files in the ICV_LIVE_CONFIG_FILE path. You can then select a configuration from the configuration drop-down list at the top of the Configuration menu.

Figure 45 User Configuration



Add the runset and the layer map. The configuration drop-down menu displayed at the top of the Configuration window can be used to switch between different configuration settings.

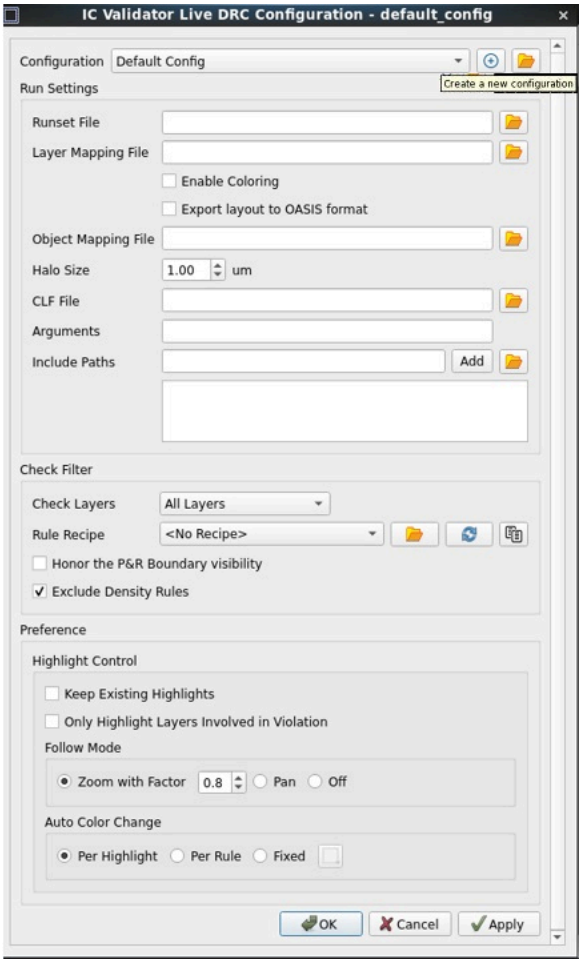
Feedback

Saving Configurations

To save configurations:

1. In the **ICV Live Configuration**, click **OK** or **Apply**.
2. Switch to another configuration using the **Configuration** drop-down list.

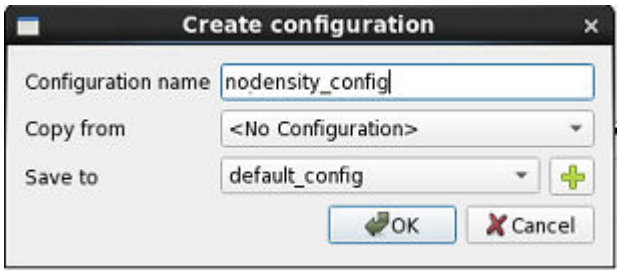
Figure 46 Saving Configurations



3. Click the **Create Configuration** or **Load Configuration** file.

Feedback

Figure 47 Create Configuration



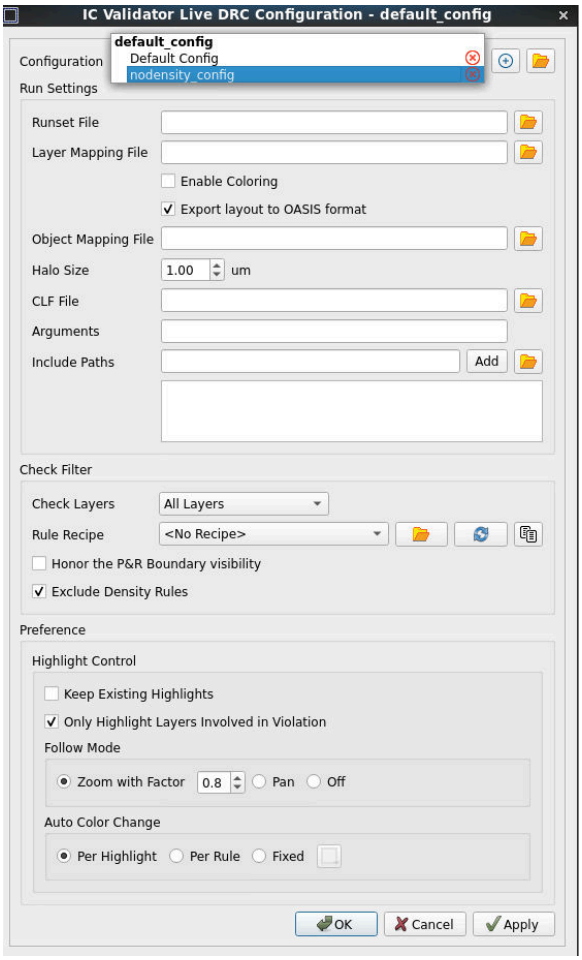
IC Validator Live DRC checks the write permissions of the configuration as follows:

a. If the unsaved configuration is writable, IC Validator Live DRC saves changes into a writable configuration file.

b. If the unsaved configuration is read-only, a message appears to prompt you to save the configuration in a new writable configuration file or overwrite an existing writable configuration file.

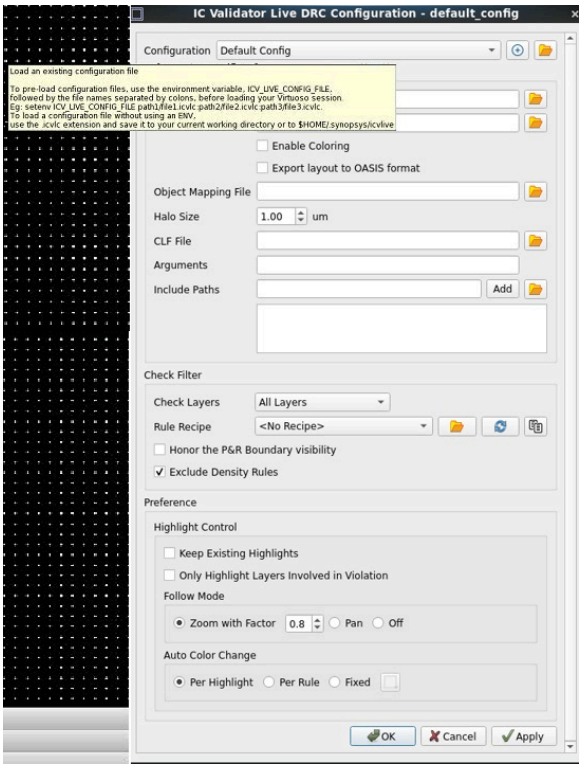
Feedback

Figure 48 User Configuration



Feedback

Figure 49 Loading an Existing Configuration File



Contents of a Configuration File

A configuration file is written in JSON format. The configuration files in IC Validator Live DRC are designed to remember the initial user settings in subsequent new sessions. One configuration file can have multiple configurations. For example, an N7 configuration file might have two configurations – one for BEOL runs and one for FEOL runs.

For example:

```
%vi n7_configurations.icvlc

{
    "configurations": [
        {
"checkFilter": {
                "checkLayers": "Visible Layers Involved",
                "excludeDensityRules": true,
                "loadedRecipeFiles": [
                    "/path/to/beol_spacing.icvrcp",
                    "/path/to/beol_density.icvrcp"
                ],
                "selectedRecipe": "Spacing Rules",
                "selectedRecipeFile": "/path/to/beol_spacing.icvrcp"
            },
```

Feedback

```
                "name": "BEOL Configuration",
                "runSettings": {
                    "clfFile": "/path/to/beol_clf.txt",
                    "enableColoring": true,
                    "exportLayoutToOasis": true,
                    "haloSize": 1.5,
                    "layerMappingFile": "n7.layermap",
                    "objectMappingFile": "n7.objmap",
                    "runsetFile": "n7.rs"
                }
            },
            {
                "checkFilter": {
                    "checkLayers": "All Layers",
                    "excludeDensityRules": true,
                    "loadedRecipeFiles": [
                    ],
                    "selectedRecipe": "",
                    "selectedRecipeFile": ""
                },
                "name": "All Layers ",
                "runSettings": {
                    "clfFile": "",
                    "enableColoring": false,
                    "exportLayoutToOasis": false,
                    "haloSize": 1,
                    ],
                    "selectedRecipe": "Custom Checks",
                    "selectedRecipeFile": "/path/to/feol.icvrcp"
                },
                "name": "FEOL Configuration",
                "runSettings": {
                    "clfFile": "/path/to/feol_clf.txt",
                    "enableColoring": true,
                    "exportLayoutToOasis": true,
                    "haloSize": 1.5,
                    "layerMappingFile": "n7.layermap",
                    "objectMappingFile": "n7.objmap",
                    "runsetFile": "n7.rs"
                }
            }
        ],
        "majorVersion": 1,
        "minorVersion": 0
}
```

Preference Settings

Live DRC stores preference-related settings into a per-user file. These settings do not affect a Live DRC check but affect GUI and highlighting.

For example, the Highlight Control section is stored as preference.

Feedback

By default, this information is stored to `$HOME/.synopsys/icvlive/prefs.icvlc`.

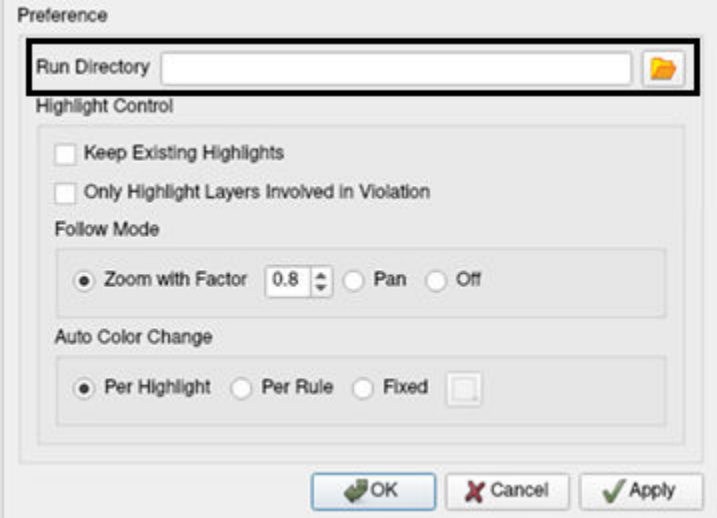
To change the file that you want to store, set the `ICV_LIVE_PREF_FILE` followed by a file name.

For example, `setenv ICV_LIVE_PREF_FILE /path/to/my_prefs.icvlc`

Run Directory

Set the `_ICV_LIVE_CLIENT_ENABLE_GUI_RUN_DIR` environment variable to enable the **Run Directory** field as shown in Figure 50. Use this field to specify the directory where Live DRC writes the intermediate files. If you do not specify a directory, Live DRC saves the intermediate files to `$HOME/.synopsys/icvlive` by default.

Figure 50 Run Directory Field In Preference Settings



To override the GUI setting, set the `ICV_LIVE_OUTPUT_DIR` environment variable. This environment variable has a higher priority than the **Run Directory** field in the GUI.

Bindkeys

IC Validator Live DRC bindkeys allow for quick access to common functions. There are two default bindkeys that can be written at the end of the icvl.il file, which is located in `$ICV_HOME_DIR/etc/icvlive/virtuoso/icvl.il`.

- Shift+F12 brings up the config window.
- F12 runs IC Validator Live DRC.

Feedback

You can define bindkeys by using the `hiSetBindKey()` function to set the Layout, Key, and SKILL function to call, as shown here:

- Fill `icvlSetup(hiGetcurrentWindow())` to bring up the config window.
- Fill `icvlExecute(hiGetCurrentWindow())` to run IC Validator Live DRC.
- Fill `icvlBrowse(hiGetCurrentWindow())` to bring up violation viewer.

For example:

```
Set hotkey for IC Validator Live DRC configuration dialog box and
 execution
hiSetBindKey("Layout" "Shift<Key>F12" "icvlSetup(hiGetCurrentWindow())")
hiSetBindKey("Layout" "<Key>F12" "icvlExecute(hiGetCurrentWindow())")
```

IC Validator Live DRC With IC Compiler II and Fusion Compiler

You can perform interactive signoff design rule checking in the IC Compiler II or Fusion Compiler GUI using the IC Validator Live DRC tool.

Setting Up the Environment

To perform interactive signoff design rule checking using IC Validator Live DRC,

1. Set up the IC Validator environment. Specify the location of the IC Validator executable by setting the `ICV_HOME_DIR` environment variable. You can set this variable in your .cshrc file. To specify the location of the IC Validator executable, use commands similar to those shown in the following example:

   ```
   setenv ICV_HOME_DIR /root_dir/icv
   set path = ($path $ICV_HOME_DIR/bin)
   ```

   You must ensure that the version of the IC Validator executable that you specify is compatible with the IC Compiler II or Fusion Compiler version that you are using. To report the version compatibility, use the `report_versions` IC Compiler II or Fusion Compiler command.

2. Set the application options for interactive signoff design rule checking.

   At a minimum, you must specify the foundry runset to use for design rule checking by setting the `signoff.check_drc.runset` application option and the layer mapping file that maps the technology file layers to the runset layers by setting the `signoff.physical.layer_map_file` application option.

Feedback

Before you run interactive design rule checking, configure the run by setting the application options for interactive signoff design rule checking, as shown in Table 24. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Table 24 Application Options for Interactive Design Rule Checking

Application option	Default	Description
`signoff.check_drc_live.default_layers`	(none)	Specifies the layers with color masks to write into the default layers for IC Validator Live DRC.
`signoff.check_drc_live.default_halo`	1	Specifies the distance in microns by which to expand the region in which IC Validator Live DRC performs checking.
`signoff.check_drc_live.keep_enclosing_results`	false	Specifies whether to keep DRC violations that are outside of the checking region. By default, IC Validator Live DRC keeps only those violations that are fully within the checking region. If you set this application option to true, IC Validator Live DRC keeps the DRC violations that are fully inside the trim region. The trim region is the checking region plus half of the distance specified by the signoff.check_drc_live.halo application option. Violations that are fully outside the trim region are always discarded.
`signoff.check_drc_live.run_dir`	signoff_check_drc_live_run	Specifies the run directory, which contains the files generated by the interactive signoff design rule checking. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.

Feedback

Table 24 Application Options for Interactive Design Rule Checking (Continued)

Application option	Default	Description
`signoff.check_drc_live.runset` (required)	(none)	Specifies the foundry runset to use for design rule checking.
`signoff.check_drc_live.user_defined_options`	signoff.check_drc_live.user_defined_options	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II/Fusion Compiler tools do not perform any checking on the specified string.
`signoff.physical.layer_map_file` (required)	(none)	Specifies the name of the layer mapping file that maps the layer names in the technology file to those in the foundry runset.

- Ensure that the layout window displays the objects on which you want to perform design rule checking.

  By default, interactive design rule checking uses the frame view of each cell. If you need more detail than what is provided in the frame view, you can use the design or layout view for specific cells. To specify the view used for specific cells, use the `gui_set_display_view` command. For example, to use the layout view for all hierarchical cells, use the following command:

  ```
  fc2_shell> gui_set_display_view \
      -cells [get_cells -hierarchical *] -view layout
  ```

  For information about controlling the objects displayed for design rule checking, see Displaying Objects for Design Rule Checking.

- Display the DRC toolbar by right-clicking in the GUI menu bar and selecting DRC Toolbar, as shown in Figure 51.

Figure 51 DRC Toolbar



The DRC Toolbar icons are listed in Table 25

Feedback

Table 25 DRC Toolbar Buttons

Application option	Description
	Runs design rule checking using IC Validator Live DRC.
	Opens the dialog box to view and sets the IC Validator Live DRC application options.
	Opens the dialog box to configure the rules to use for design rule checking.
	Selects the region in which to perform design rule checking.
	Resets the design rule checking region to that used in previous runs.
	Clears or highlights all DRC violations detected by IC Validator Live DRC.
	Iterates through the DRC violations for the current rule from IC Validator Live DRC.
	Controls zoom or pan when iterating through DRC violations.
	Shows the design rule being highlighted.

- Configure the rules to use for design rule checking by choosing **Edit** > **Options** >**ICV Live** or by clicking the gear icon  in the DRC toolbar.
- Run design rule checking by choosing **Edit** >**ICV** > **Run ICV-Live** on Current View or by clicking the ICV Run icon  in the DRC toolbar.

  The first time that you run IC Validator Live DRC (or after changing layers or rules), the tool caches the runset, which can take a few minutes. Subsequent runs do not perform this caching and therefore are much quicker.

Feedback

Displaying Objects for Design Rule Checking

When you run interactive design rule checking, the IC Validator tool performs design rule checking on what is displayed in the layout window, plus an extension of 1 micron.

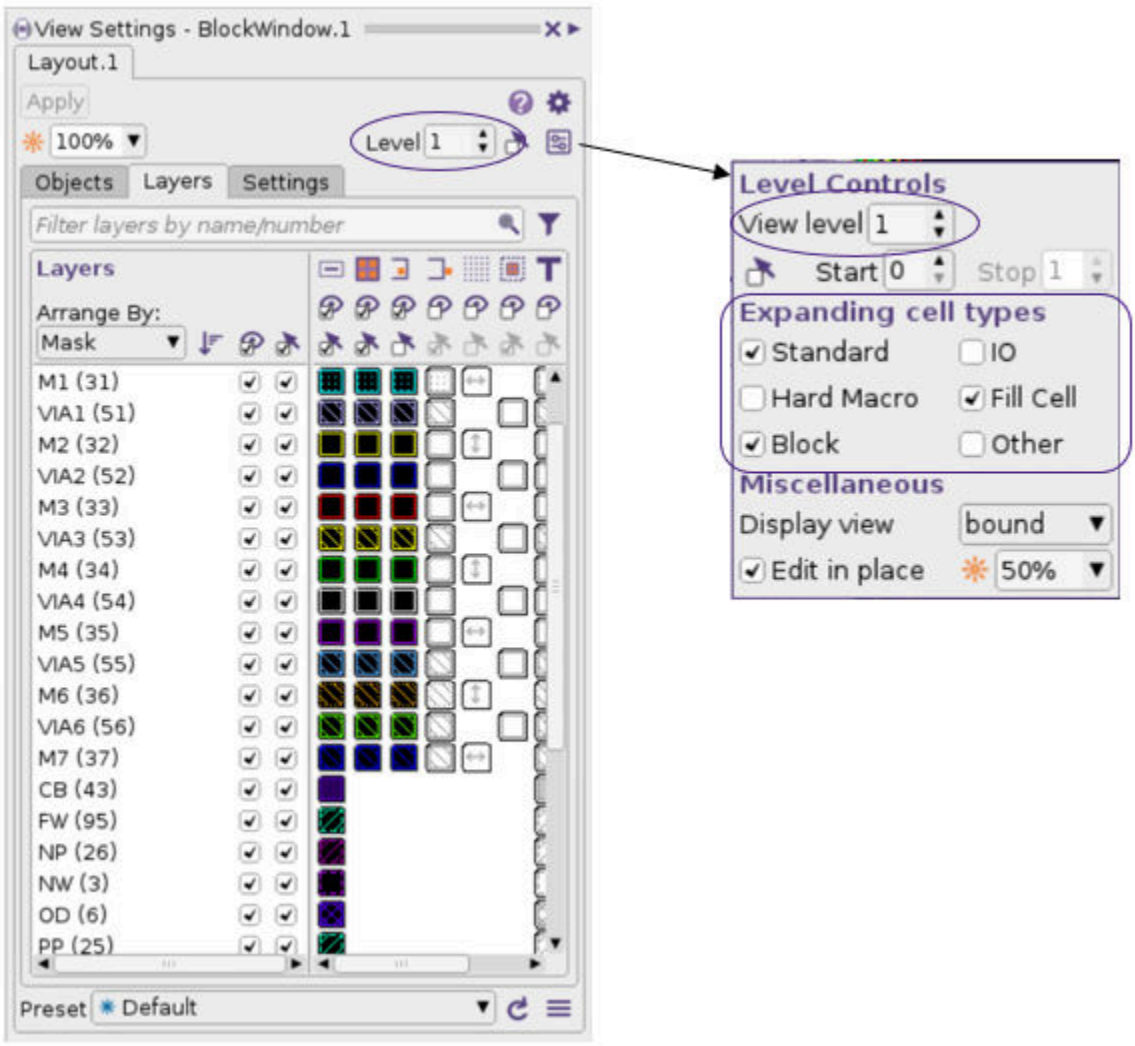
You can modify the size of the extension by setting the `signoff.check_drc_live.halo` application option. Use the View Settings and Hierarchy Settings panels to control the layers, levels, and cell types displayed in the layout window.

- Display the View Settings panel by clicking the View Settings icon in the panel area.
  - Specify the number of levels to display in the Level field.
  - Select the Layers tab and enable visibility for the required layers.
- Display the Hierarchy Settings panel by clicking the Hierarchy Settings icon  in the View Settings panel or the menu bar.
  - Specify the number of view levels in the “View level” field.
  - Select Standard, “Fill Cell,” and Block in the “Expanding cell types” section.

Figure 52 shows the View Settings and Hierarchy Settings panels used to control the layout window display for design rule checking.

Feedback

Figure 52 View and Hierarchy Settings



Note:

For more information about interactive signoff design rule checking, see the *Fusion Compiler™ User Guide* or the *IC Compiler™ II Implementation User Guide*, which are available on SolvNetPlus.

Setting Up Pre-Compiled Cache Directories

The IC Validator Live DRC feature recompiles the runset each time it detects new layers in the view window or when the rule selection changes. If you change the view windows and visible layers frequently, this can cause frequent compilations, which can slow down

Feedback

the IC Validator Live DRC runtime. To improve performance, prepare a cache directory containing cache files generated during the design process. When the window changes in the future, the IC Validator Live DRC feature can search for and reuse an older cache file from this directory if the layers, rule-selection, and settings match the current run. These cache directories can be shared with other users. To create runset cache directories and reuse or share files:

1. Set up the local cache directory and run IC Validator Live DRC to compile the runset with different configurations.

   ```
   unset ICV_DISABLE_RUNSET_CACHE
   setenv ICV_RUNSET_CACHE_DIR <first_user local>
   // Start ICV Live
   ```

2. Transfer the cache files from <first_user local> to the <new_user> shared cache directory, <new_user shared>.

3. Set up both local and shared cache directories in <new_user> to run IC Validator Live DRC so that it can pick up the precompiled files as well as store newly created cache files in the local path.

   ```
   unset ICV_DISABLE_RUNSET_CACHE
   setenv ICV_RUNSET_CACHE_DIR <new_user local>
   setenv ICV_RUNSET_CACHE_DIR_SHARED <new_user shared>
   // Start ICV Live
   ```

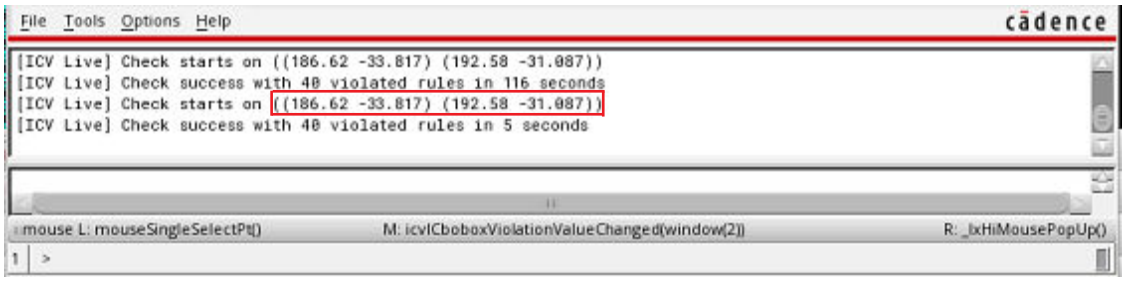
Feedback

Reproducing an IC Validator Live DRC Run in IC Validator

To reproduce IC Validator Live DRC run results in IC Validator,

1. Capture the coordinates of your run from the Console:

Figure 53 IC Validator Live DRC Console



2. Create a config.rs file in your run directory using the following lines, and substitute the arguments in `select_window` with the coordinates of the IC Validator Live DRC run from the previous step:

```
hierarchy_options(
   explode_all = {"*"}
);
incremental_options(
   select_window = {{186.62, -33/817, 192.58, -31.087}},
   clip_window = false,
   window_error_filter = KEEP_ENCLOSED,
   window_ambit = double
);
```

   IC Validator Live DRC runs on flat data. Therefore, you must use the `hierarchy_options()` function to flatten the input to the IC Validator run. Although IC Validator Live DRC does not clip polygons at the edge of the view window, it does not report any violations outside of the view window. Therefore, the `clip_window` and `window_error_filter` arguments must be used with the `incremental_options()` function.

3. Add this file to your IC Validator run by setting `-runset_config config.rs`.
4. Run IC Validator.

7

IC Validator Explorer DRC

This chapter explains the IC Validator Explorer DRC capabilities.

IC Validator Explorer DRC is described in the following sections:

- Introduction to IC Validator Explorer DRC
- Types of Explorer DRC Modes
- Configuring Explorer DRC
- Explorer DRC Output Files
- Debugging With Explorer DRC
- Explorer DRC Licenses

Feedback

Introduction to IC Validator Explorer DRC

Debugging DRC violations early in the design process can be complex and time consuming. For example, complicated DRC rules (multiple patterning and delta voltage) run for a very long time and do not add value when other simpler violations exist.

A basic set of design checks can be used to debug typical early design issues: finding cell overlaps, improper fill, and routing issues does not require complicated rules. IC Validator Explorer DRC automatically determines these basic rules from the foundry runset. It also adds extra methodology rules and debugging capabilities. This results in a significant reduction in turnaround time.

In addition, IC Validator Explorer DRC detects when design layers flag too many violations and deselects related slow-running rules. Such dynamic selection and execution of checks achieves a faster turnaround time than traditional full DRC runs.

Types of Explorer DRC Modes

There are three Explorer DRC modes:

- **Explorer DRC Standalone Mode**

  This mode runs automatically selected design rules from the foundry runset and IC Validator-added methodology checks.

  Use the `standalone` mode for designs with violations that require fast checking with low computing resources.

- **Explorer DRC Tiers Mode**

  Explorer DRC tiers provide a systematic framework for executing and reporting the high-priority checks first. In the `tiers` mode, checks are completed regardless of performance. All rules are selected and executed based on the tier order and up to the specified tier.

  Performance is not the main consideration when you use this mode.

  Some tiers are reserved for Explorer priority rules and methodology checks. Most of the tiers however, are user-customizable. See Table 26 for more information.

  The tiers are executed in order of assigned priorities. Lower-numbered tiers have the highest priority. Undefined checks between Tier 0 and Tier 10 are executed after Tier 10, as part of an “All remaining rules” tier. Results are updated during the run, as each tier finishes.

  Example of Running in Tiers Mode

Feedback

```
Run Explorer DRC auto mode on all Tiers with User defined Tiers
specified
using -explorer_tiers_file mytier.txt
% icv -explorer auto -explorer_tiers_file mytier.txt runsetfile
```

- **Explorer DRC Auto Mode**

  Similar to the `tiers` mode, this mode also uses the Explorer DRC tiers. However, this mode also has the ability to stop slow-running rules.

  When design layers with violations are found in `auto` mode, related rules are stopped if their functions are slow-running. This allows for results from earlier tiers to be used to stop later functions/rules from running too long (rules from tier 0 and tier 1 are never stopped early).

  The performance of this mode depends on the number of DRC errors. The larger the number of DRC errors, the more likely it is that `auto` mode deselects rules and exits early to achieve a faster turnaround time.

  Use the `auto` mode when the number of violations in the design is unknown and you want to complete as many checks as possible. If the design is clean, `auto` mode does not stop any of the checks, and it produces full signoff results.

Executing Explorer DRC

To execute Explorer DRC, set `-explorer standalone|auto[:`*`TierNumber`*`]|tiers[:`*`TierNumber`*`]` on the command line,

where *TierNumber* specifies the highest tier to be executed.

All tiers up to and including *TierNumber* are executed. If *TierNumber* is not specified, all of the checks are candidates for execution.

See the following section on configuring IC Validator Explorer DRC for more information.

Examples of Running in Explorer DRC Mode

```
Run Explorer DRC standalone mode
% icv -explorer standalone runsetfile

Run Explorer DRC auto mode up to Tier 4
% icv -explorer auto:4 runsetfile
```

Table 26 shows the different Explorer DRC tiers, pre-included checks, and customizable categories.

Feedback

Table 26 Explorer Tiers

Tier	Can the user add checks to this tier?	Can the user name this tier?	Included checks	Optional checks
0	No	No	Methodology warnings, fill overlap diagnostics, priority rules,	N/A
1	Yes	No	None	Priority rules (user-specified)
2	Yes	Yes	Voltage-based diagnostics (set 1)	User-specified rules
3	Yes	Yes	Voltage-based diagnostics (set 2)	User-specified rules
4 to 10	Yes	Yes	None	User-specified rules
N/A	N/A	N/A	All remaining rules	N/A

Configuring Explorer DRC

The Explorer DRC mode command-line options are described in Table 27.

Table 27 Explorer DRC Mode Command-Line Options

Option	Definition
`-explorer standalone|auto[:`*`TierNumber`*`]|tiers[:`*`TierNumber`*`]`	Runs Explorer mode with either `standalone`, `auto`, or `tiers` mode. *TierNumber* is optional for the `auto` and `tiers` modes to limit the run up to *TierNumber* specified. If not specified, all of the tiers are candidates for execution.
`-explorer_tiers_file` *`path_to_file`*	Specifies the file to customize so that user-defined tiers can be passed to the Explorer DRC run. The `ADD_RULES_TO_TIER` and `NAME_TIER` directives can be used in this file.
`-explorer_script` *`path_to_file`*	Specifies the file used to add resources after Tier 0 and Tier 1 are completed. You can start an `auto` run with fewer resources and dynamically add resources when Tier 0 and Tier 1 are completed. Use these resources for a faster turnaround time of the remaining checks.

The `-explorer_tiers_file` *`path to file`* command-line option points to an optional script file that you can use to customize tiers. An example of this file might look like the following:

```
ADD_RULES_TO_TIER 1 "*.1:" "*G1*"
ADD_RULES_TO_TIER 2 "*.S.*" "**DN*"
```

Feedback

```
NAME_TIER 4 "Handoff Rules"
ADD_RULES_TO_TIER 4 "*delta*"
ADD_RULES_TO_TIER 4 "*LU*"
DISABLE EXPLORER.OVERLAP
```

The following directives can be used to create this file:

- **ADD_RULES_TO_TIER *TierNumber Rule comment … Rule comment***

  Use this directive for tiers that allow user-specified rules. You can add rules to the tiers based on the violation comments. String matching is supported.

  Rules are added to tiers in ascending priority order. If a rule has already been added to a higher priority tier, it is not added to any lower priority tiers. This rule applies regardless of the order of specification in the Explorer tiers file.

  Skipped or empty tiers are allowed. When Explorer DRC encounters an empty tier, execution moves to the next priority tier. Reporting is in tier priority order.

- **NAME_TIER *TierNumber TierName***

  Use this directive for tiers that allow a user-specified name. This name is used when reporting to the LAYOUT_ERRORS file, RESULTS file, or VUE.

  Unnamed tiers are allowed. They are reported with the default Explorer tier name, *TierNumber*.

- **DISABLE_EXPLORER.DiagnosticName**

  Use this directive to disable Explorer proprietary diagnostics. The supported values are:

  - `EXPLORER.OVERLAP`. Disables fill overlap diagnostics that are included by default in Tier 0.
  - `EXLORER.ASSIGN`. Disables assign-layer based priority rules that are included by default in Tier 0.
  - `EXPLORER.VOLTAGE`. Disables voltage-based diagnostics that are included by default in Tier 3 and Tier 4.

The `-explorer_script` *`path_to_file`* command-line option runs after both the Tier 0 and Tier 1 checks are completed. You can start DRC `auto` mode with fewer resources than necessary to complete the entire run. If more resources are needed to run additional tiers, this command-line option adds additional resources following the basic checks. An example of a script file looks like this:

```
#!/bin/csh -f

echo "Explorer Tier 0 and Tier 1 checks are done. Adding resources and
running remaining Tiers"
```

Feedback

```
# Following command adds host "A" to ongoing run
icv -rd run_details -host_add A
```

Explorer DRC Output Files

Explorer DRC creates ASCII files that are similar to an IC Validator DRC run. See Output Files for more information on DRC run files.

In both the LAYOUT_ERRORS and RESULTS files, errors from an Explorer run are categorized in Table 28.

Table 28 Explorer Error Categorization

Category	Description
Methodology warnings	Specifies Explorer methodology warnings aimed to flag potential design or performance issues.
Fill overlap diagnostic	Specifies DRC errors that are diagnosed as fill overlap issues (runset selected fill overlap rules).
Priority rules	Specifies DRC errors that are available from Explorer priority checks (runset selected basic rules).Some of the rules in this category might get an EXPLORER prefix, which denotes that the rule has been modified (faster version) from the original runset. The original runset version runs as part of DRC Errors in the “All remaining rules” category.
Priority Rules (user-defined)	Specifies DRC errors available from user-defined priority rules.
Voltage dependent diagnostics	Specifies Explorer methodology warnings that flag text-based voltage conflicts on polygons or net objects. In addition, stray voltage texts and voltage checks are included.
Explorer tier *TierNumber*	Specifies DRC errors from unnamed tiers. By default, these errors are categorized under Explorer tier, *TierNumber*. You can modify the tier name using the `NAME_TIER` directive in the file.
All remaining rules	Specifies the available DRC errors from checks that are classified as predefined or user-defined tiers.

In addition to the error categorizations in Table 28, DRC errors are categorized by their user-defined category names (defined using the `NAME_TIER` directive in the file).

Note:

Error categories are dependent upon the type of Explorer mode and diagnostics flagged in that mode.

Feedback

Explorer DRC Results File

Explorer DRC creates the RESULTS file, cell.RESULTS. This file provides results from the run and also the IC Validator command-line option used for the run.

The RESULTS file is used as the starting point for analyzing the results of an Explorer DRC run. The Results summary section reports the DRC error statistics, as categorized in Table 28. Statistics include the total number of rules and violations in the different Explorer categories.

Explorer DRC Errors File

Explorer DRC creates a LAYOUT_ERRORS file, which provides layout errors that are flagged during the run. These errors are categorized in Table 28.

To ensure the error report is concise, error coordinate information is suppressed in the LAYOUT_ERRORS file. This is equivalent to running `error_options (report_error_details = false)`. The information is stored in the error database (PYDB), but only the violation summary section is written to the LAYOUT_ERRORS file.

The LAYOUT_ERRORS file is generated after each tier/category so that you can reference it during the Explorer DRC run. The icv.log file indicates when a tier/category is completed, as follows:

```
ICV_Engine run is 5% complete. Elapsed Time=0:07:24
All Priority Rules are finished! 19103703 violations were found.
```

Note:

While storing error markers in the error database (PYDB), Tier 0 and Tier 1 (Explorer priority checks and user-defined priority checks) enforce the limit set by the `error_sampling_per_check` argument (the default is 1 million). The limit of other tiers is based on the runset. See the *IC Validator Reference Manual* for more information about the `error_sampling_per_check` argument.

Explorer DRC VUE File

IC Validator Explorer creates the VUE file, cell.vue, to view Explorer DRC error details and the Heat Map.

Feedback

Debugging With Explorer DRC

To analyze an Explorer DRC run,

1. Run IC Validator with Explorer DRC.
2. Review the RESULTS file.
   - If the results are clean, go to signoff DRC. If the results are not clean, go to step 3.
3. Load the Explorer DRC .vue file into VUE. Attach the layout editor to VUE.
4. Click the Heat Map tab to start debugging. The Error Heat Map is a graphical interface used to visualize error distributions.

   Heat Maps are automatically enabled for Explorer DRC. See the "Introduction to the IC Validator Error Heat Map" chapter in the *IC Validator VUE User Guide* for more information.
5. Similar to the RESULTS and LAYOUT_ERRORS files, the Explorer DRC errors are categorized by default in Table 28. In general, start debugging in order from methodology warnings, diagnostics, predefined priority rules, user-defined tiers and, finally, to all remaining rules.

   In addition, you can select the Clustered Errors category mode from the menu in the Violation Category window. In this mode, identical errors are clustered together so that multiple rules can be debugged at one time. See the "Introduction to the IC Validator Error Heat Map" chapter in the *IC Validator VUE User Guide* for more information on clustered errors.
6. After rules are selected from their respective categories, isolate the areas with a high number of error counts (red regions in the heat map) and zoom into these dense regions.
7. Probe the heat map and layout errors in these regions to analyze the underlying cause for the high error count. Probing might include:
   - Zooming into regions
   - Highlighting error markers in those regions
   - Selecting or deselecting specific rules
8. Repeat steps 6 and 7 until all categories/clusters are exhausted.

Feedback

Explorer DRC Licenses

IC Validator Explorer DRC requires Elite licensing. This feature requires the following IC Validator licenses:

ICValidator-Manager

ICValidator2-GeometryEngine

See the Licensing and Resource Requirements chapter in the *IC Validator User Guide* for more information.

8

IC Validator Explorer LVS

This chapter explains the IC Validator Explorer LVS capabilities.

IC Validator Explorer LVS is described in the following sections:

- Introduction to IC Validator Explorer LVS
- Configuring Explorer LVS
- Executing Explorer LVS
- Explorer LVS Licenses

Introduction to IC Validator Explorer LVS

Finding the root cause of LVS errors early in signoff LVS is critical in reducing the overall signoff period and meeting a tapeout schedule. Explorer LVS targets the System-On-Chip (SOC) full chip, and provides a solution where you can find the root cause of LVS issues with a quick turnaround time and efficient machine resources. This reduces the overall signoff time and effort.

Explorer LVS focuses initially on LVS issues that stem from SOC integration, and then gradually expands its analysis to cover other potential LVS issues, such as LVS errors from the device or base layers or cells in the lower hierarchy.

To check the issues from SOC integration, by default, Explorer LVS performs a text-based short check with only the equivalence cells and metal layers that make up the SOC Top (for example, P&R, PowerGrid, placeholder polygon fill cells that build up the main SOC structure, as opposed to macros and IPs that are instantiated underneath the main SOC structure, and so on). The scope of checking LVS issues can be expanded as needed using flexible options provided by Explorer LVS.

Explorer LVS helps increase the productivity and efficiency of signoff LVS. It does not replace the traditional LVS tool. Final signoff LVS must run with the traditional LVS tool.

Feedback

Configuring Explorer LVS

Table 29 describes the command-line options available for configuring Explorer LVS.

Table 29 Explorer LVS Command-Line Options

Syntax	Description
`-explorer lvs`	Runs the Explorer mode for LVS. If `-explorer_lvs_cells`, `-explorer_lvs_layers`, or `-explorer_lvs_checks` are specified, Explorer LVS uses the `AUTO`, `METAL`, or `SHORTS` defaults for these command-line options and performs text-based short checks on SOC Top cells. See Introduction to IC Validator Explorer LVS for examples of SOC Top cells. The default behavior can be altered using these three command-line options as well, as described later in this table. Explorer LVS skips device extraction regardless of these command-line options.
`-explorer_lvs_cells [AUTO | ALL]`	Optional. Specifies which cells to check in Explorer LVS. `AUTO`. The default. Considers only top-related cells. When `-explorer_lvs_checks=SHORTS` or `PG_SHORTS`, delete all layout cells defined in `equiv_options()`, `lvs_black_box_options()`, or generated by `lvs_options(generate_user_equivs)`. `ALL`. Considers all cells to include all cells.
`-explorer_lvs_layers [METAL | ALL |`*`user_custom_cell_list`*`]`	Optional. Specifies which layers to check in Explorer LVS. `METAL`. The default. Considers only metal layers, or empty FEOL layers, such as device terminal layers. `ALL`. Considers all layers. *`layer2, layer2, ..., etc.`*. Considers only metal layers, or empty FEOL layers except `layer1`, `layer2`, and so on.
`-explorer_lvs_checks [SHORTS | ALL | PG_SHORTS]`	Optional. Performs only text-based short checks or text-based short checks along with LVS compare. `SHORTS`. The default. Reports only text_short violations. `ALL`. Reports text_short violations and LVS Compare results `PG_SHORTS`. Reports power and ground text_short violations. PG shorts definition: <*power, power*>, <*power, ground*>, <*ground, ground*> non-PG shorts definition: <*power, signal*>, <*ground, signal*>, <*signal, signal*>
`-explorer_lvs_preserve_cells [`*`preserve_cell_list`*`]`	Optional. Specifies the cells that are to be preserved from being deleted or black-boxed in Explorer LVS, so that those cells can be included in Explorer LVS.
`-explorer_lvs_preserve_cell_file "`*`file path`*`"`	Specifies the file input version of `-explorer_lvs_preserve_cells`. One line per cell to be preserved.

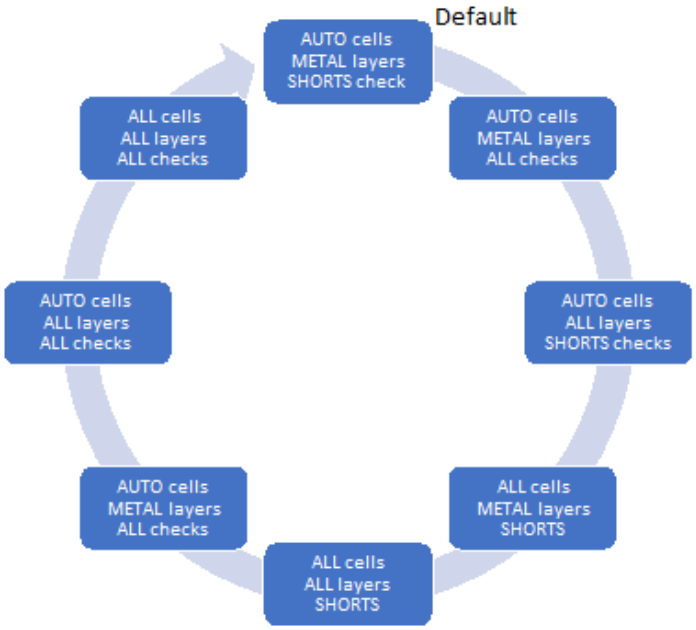
Feedback

Table 29 Explorer LVS Command-Line Options (Continued)

Syntax	Description
`-explorer_lvs_preserve_hierarchy_cells "space separated cellnames"|*`	Specifies the equiv_cells and their child equiv_cells that should not be deleted or black-boxed when running with -explorer "*lvs|lvs:1|lvs:2*".
`-explorer_lvs_preserve_hierarchy_cells_file "file path"`	Specifies the file input version of `-explorer_lvs_preserve_hierarchy_cells`. One line per cell.

All possible usages of the Explorer LVS command-line options are shown in Figure 54.

Figure 54 Combinations of Explorer LVS Command-Line Options



Executing Explorer LVS

To execute Explorer LVS, set `-explorer lvs` on the command line. This command-line option takes the `AUTO`, `METAL`, or `SHORTS` defaults of the `-explorer_lvs_cells`, `-explorer_lvs_layers`, or`-explorer_lvs_checks` command-line options. Explorer LVS performs text-based short checks on the SOC Top cells with only metal layers.

Feedback

The default behavior can be altered with the three Explorer LVS options `-explorer_lvs_cells`, `-explorer_lvs_layers`, and `-explorer_lvs_checks`, as described in Table 29.

Certain cells specified with `-explorer_lvs_preserve_cells` can be preserved from being deleted or black-boxed in Explorer LVS so that those cells can be included in Explorer LVS.

SeeTable 30 for examples of how to execute Explorer LVS.

Table 30 Examples of Executing Explorer LVS

Mode	Command Line	Description
Default	`% icv -explorer lvs \ runsetfile`	Text-based short checks on the SOC Top cells with metal layers only
Add all of the cells to the default	`% icv -explorer lvs \ -explorer_lvs_cells ALL \ runsetfile`	Text-based short checks on all of the cells with metal layers
Add all of the layers to the default	`% icv -explorer lvs \ -explorer_lvs_layers ALL \ runsetfile`	Text-based short checks on the SOC Top cells with all of the layers
Add all of the cells and layers to the default	`% icv -explorer lvs \ -explorer_lvs_cells ALL \ -explorer_lvs_layers ALL \ runsetfile`	Text-based short checks on all of the cells with all layers
Add LVS compare check to the default	`% icv -explorer lvs \ -explorer_lvs_checks ALL \ runsetfile`	Text-based short checks along with LVS compare on the SOC Top cells with metal layers only
Preserve CellA and CellB in Explorer LVS	`% icv -explorer lvs \ -explorer_lvs_preserve_cells \ "CellA CellB" runsetfile`	Same as the default behavior with the only difference being where Explorer LVS guarantees CellA and CellB to be preserved

Explorer LVS Licenses

IC Validator Explorer LVS requires Elite licensing. This feature requires the following IC Validator licenses:

ICValidator-Manager

ICValidator2-GeometryEngine

Feedback

See the Licensing and Resource Requirements chapter in the *IC Validator User Guide* for more information.

9

PXL

This chapter describes PXL, a programmable and extensible language, as well as the basics of function definitions and the use of functions in the IC Validator tool.

PXL is described in the following sections:

- PXL Architecture
- General Definitions
- Variables and Operators
- Data Types and Typing
- Flow Control
- Functions
- Program Structure
- Example of Modularized Code

Feedback

PXL Architecture

PXL is fully programmable, with general programmability features such as

- Variables
- Flow Control
- Lists (arrays) with standard indexing
- Structures
- Hashes
- Enumerated types
- User-defined functions
- Mathematical, logical, and string operations
- Macros with command-line option support
- Environment variables
- Static scope

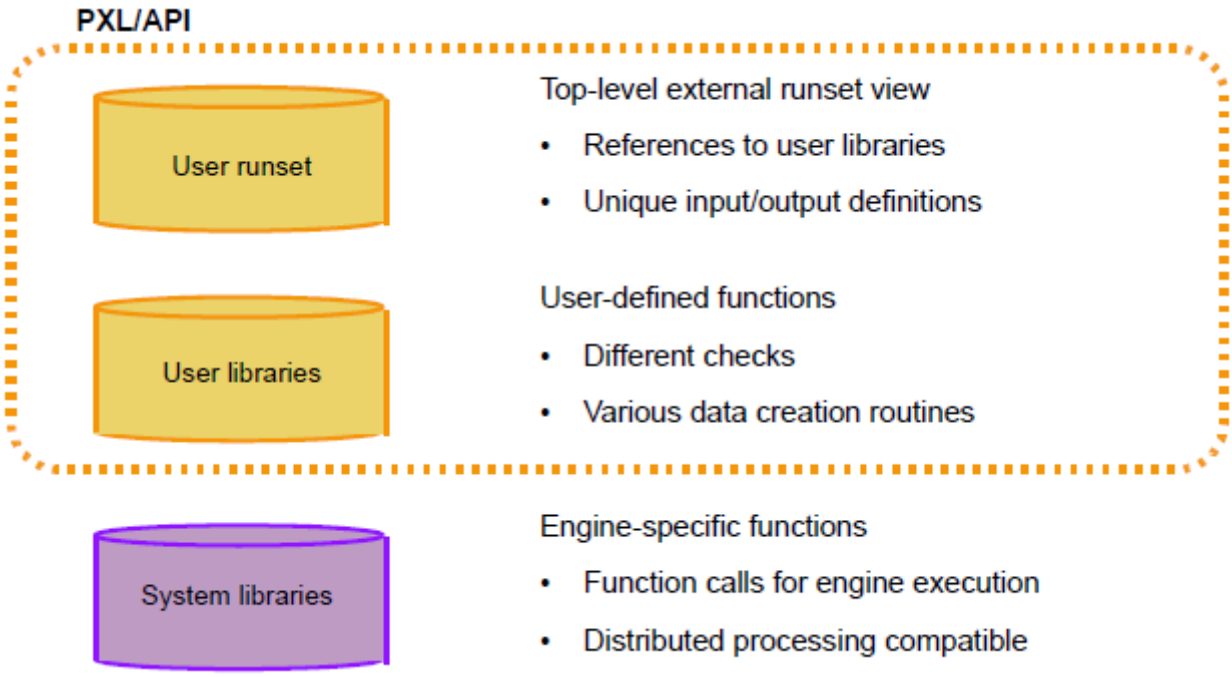
These features allow for

- Straightforward runset development
- Easy to maintain runsets
- Succinct code
- Code reuse with subroutines
- Support user-defined functions

Figure 55 shows the basic structure of PXL.

Feedback

Figure 55 PXL Structure



General Definitions

This section provides definitions and characteristics of some syntactic elements in PXL, such as identifiers, string literals, keywords operators, and comments. Understanding the purpose and function of syntactic elements is helpful while creating runsets.

Identifiers

Identifiers for any PXL components can include alphabetic characters, numbers, and underscores in any order.

PXL is case-sensitive. The identifiers listed here are all different:

- abCD
- ABCD
- abcd

Feedback

The following naming restrictions apply to identifiers:

- Two leading underscores (__). Identifiers with two leading underscores are not allowed. They are reserved for internal language use.
- One leading underscore (_). Identifiers with one leading underscore are not allowed. They are reserved for product use.
- All uppercase. Reserved for use by the application programming interface. You can, however, use uppercase identifiers as enumerated type elements.
- Reserved keywords. Reserved keywords must not be used as identifiers. For a list of reserved keywords in PXL, see Keywords on page 189."

You cannot use these names as identifiers:

- Function names: The functions are defined in the *IC Validator Reference Manual* and the IC Validator header files. Example function names are capacitor, check_property, internal1, and xor.
- Container type names: These names of lists, hashes, and structures are defined in the IC Validator header files. Here are some examples of container type names:
  - Lists: `coordinate_l` is used in defining the `property()` function
  - Hashes: `layer_list_h` is used in defining the `density()` function
  - Structures: `check_property_s` is used in defining the `check_property()` function

Examples of valid identifiers are

```
This IsAnIdentifier

another_identifier_using_underscores

layer20
```

Examples of reserved (invalid) identifiers are

```
__SYSTEMNAME
(Two leading underscores; reserved for internal language usage)

_productname
(One leading underscore; reserved for product use)

HIERARCHICAL
(Reserved keyword)
```

Feedback

String Literals

A string literal is a group of characters beginning and ending with a double quotation mark (").

Note:

The first character of a text string cannot be an integer or a space followed by an integer.

String literals must not contain a new line. If a new line is needed in a string literal, the new-line character (`\n`) must be used.

The following groups of characters are examples of string literals:

```
"hello"

"\""

"a string with\nmultiple lines"

"abc" "def"
```

String literals are automatically merged with any adjacent string literals. In particular, they are merged with preprocessor variables that contain string values, such as `__FILE__`, as well as references to environment variables, such as `$<id>`.

Use escape sequences within strings to set aside parts of the string as instructions to the compiler. The available set of escape sequences in PXL are shown in Table 31.

Table 31 Escape Sequences

Escape	Definition
`\n`	New-line character
`\"`	Double quotation mark character
`\\`	Single backslash character
`\`*`NNN`*	Octal value of the character
`\`*`t`*	Tab character
`\x`*`NN`*	Hexadecimal value of the character

Numeric Constants

Numeric constants in PXL are defined as shown in Table 32.

Feedback

Table 32 Numeric Constants

Number type	Condition	Example
double	Contains an exponent.	4e5, 10.305e+10
double	Contains a decimal.	3.141592
decimal integer	Contains a leading numeral (1-9).	1023
octal number	Contains a leading zero (0) and any of the digits from zero (0) to seven (7).	0346
hexadecimal number	Contains a leading 0x or 0X, and can contain only the hexadecimal digits (0-9a-fA-F).	0xDEADBEEF

Keywords

Keywords are built-in words that are recognized by the language. Keywords are reserved and therefore cannot be used as identifiers.

A few of the uses of keywords are

- Flow controllers, such as *if*, *foreach*, and *while*
- Operators, such as *in* and *contains*
- Constants, such as *true* and *false*

The following tables list the reserved words in PXL.

Note:

See Identifiers on page 186" for more information about naming restrictions.

Table 33 lists keywords that are reserved by the system and are unavailable for external use.

Table 33 System Keywords

barrier	broadcast	built in
deprecated	intrinsic	method
obsolete	preprocess	primary
proxy_out	published	required
shutdown	unsafe	unique

Feedback

Table 33 System Keywords (Continued)

volatile

Table 34 lists keywords use by the In-Design flow. Do not use variables with these names in an IC Validator runset.

Table 34 In-Design Flow Keywords

INCREMENTAL_MILKYWAY_OPTIONS_INCLUDE_VIEW	INCREMENTAL_OPTIONS_SELECT_WINDOW
INDESIGN_DO_FLOATING_VIA_FILL_PURGE	INDESIGN_DONT_UPDATE_TECHNOLOGY_FILE
INDESIGN_ENABLE_MY_ASSIGN	METAL_FILL_BLOCKAGE_LAYERS
METAL_FILL_BLOCKAGE_VIEW	METAL_FILL_DATATYPES
METAL_FILL_DENSITY_DEBUG	METAL_FILL_DENSITY_CHECK
METAL_FILL_DENSITY_DELTA_WIN_SIZE	METAL_FILL_DENSITY_MIN
METAL_FILL_FLATTEN	METAL_FILL_INLIB
METAL_FILL_INLIB_CELL	METAL_FILL_INLIB_EXCLUDE_FILL
METAL_FILL_INLIB_PATH	METAL_FILL_INSTALL_ICV_RH
METAL_FILL_INSTALL_PRIMEYIELD_RH	METAL_FILL_LAYER_NUMBERS
METAL_FILL_OUTLIB	METAL_FILL_OUTLIB_APPLY_PREFIX
METAL_FILL_OUTLIB_CELL	METAL_FILL_OUTLIB_CELL_PREFIX
METAL_FILL_OUTLIB_MODE	METAL_FILL_OUTLIB_PATH
METAL_FILL_OUTLIB_TEMP_CELL	METAL_FILL_OUTLIB_VIEW
METAL_FILL_OUTLIB_RF_CELL_PREFIX	METAL_FILL_RUNSET2LIBRARY_LAYER_MAP
METAL_FILL_SELECT_WINDOW	METAL_FILL_USER_RUNSET
_METHODOLOGY_FUNCTIONS_RS_	SAVE_METAL_FILL_INLIB_EXCLUDE_FILL
_SMF_ICV_RH_	SNPSINDESIGN
VIA_FILL_ALLOWED_ONE_SIDE	VIA_FILL_ASSOCIATED_METAL
VIA_FILL_DATATYPES	VIA_FILL_LAYER_NUMBERS
VIA_FILL_MASK_NAMES	VIA_FILL_MET_ENCLOSURE

Feedback

Table 34 In-Design Flow Keywords (Continued)

VIA_FILL_OUTLIB_RF_CELL_PREFIX	_WRITE_AREFS

Table 35 lists reserved application identifiers that cannot be used as identifiers.

Table 35 Reserved Application Identifiers

binary	boolean	break
by	const	constraint
constraint_category	CONSTRAINT_EQ	CONSTRAINT_GE
CONSTRAINT_GELE	CONSTRAINT_GELT	CONSTRAINT_GT
CONSTRAINT_GTLE	CONSTRAINT_GTLT	CONSTRAINT_LE
CONSTRAINT_LT	CONSTRAINT_NE	continue
DELETE_TEXT	DELETE_AND_REPORT	defined
double	elif	else
elseif	enum	false
for	foreach	function
hash	if	in
in_out	integer	list
newtype	of	on
out	return	returning
step	string	struct
substrate	SUBSTRATE	thru
to	true	unary
undefined	violation	void
while		

Enumerated type elements cannot be used as identifiers. Table 36 lists the enumerated type elements used by the IC Validator tool.

Feedback

Table 36 Enumerated Type Elements

ABORT	ABSOLUTE	ABSTRACT_DESIGN
ACCEPT	ACUTE	ADJUST
ALIEN	ALL	ALL_CELLS
ALL_POLYGONS	ALLOW	ALWAYS
AMBIT_ON_BLOCK	ANALOG_DESIGN	ANALOG_GROUND_NET
ANALOG_POWER_NET	ANALOG_SIGNAL_NET	ANNOTATION_FILE
ANTENNA_CELL	ANTIDISHING_BRIDGE	ANY_DEVICE_NAME
ANY_LEVEL	APPEND	APPEND_NEW_VERSION
AREA_FILL	AREA_FILL_USE	ARC
ARC_PLAN_CELL	ASSIGNS	
ASYMMETRIC	ASYMMETRIC_INTERSECTING	AUTO
AUTO_DETECT	AUTOMATIC	AXIS
BACKSLASH	BALANCED	BASE
BISECTOR	BLACK_BOX_DESIGN	
BLOCK_RADIAL	BLOCK_SQUARE	BLOCKAGE_OBJECT
BODY_EXTENTS	BODY_POLYGONS	
BOTH	BOUNDARY	BOTTOM
BOTTOM_HEIGHT	BREAK_PATH	BRIEF
BULK	BUS	BV_CELL
CAPACITOR	CASE_INSENSITIVE_NAME	CATEGORY_BLOCKAGE
CDL	CELL_LEVEL	CELL_ROW
CENTER	CENTER_TO_CENTER	CENTERLINE
CHECK	CHIP	CHIP_CONTEXT
CHIP_CELL	CHIPWIDE_AVERAGE	CLASSIFY
CLEAN	CLIP	CLIPPED_DELTA_WINDOW
CLIPPED_CENTER	CLOCK	CLOCK_0_SKEW

Feedback

Table 36 Enumerated Type Elements (Continued)

CLOCK_AND_SPLIT_CLOCK	CLOCK_BUFFER_CELL	CLOCK_NET
CLOCK_RING	CLOCK_STRIPE	CLOSED_OUTER_BOUNDARY
CLOSEST	CMPSIM	COINCIDENT
COLLECTOR	COLOR_1	COLOR_2
COLOR_LAYER_TYPE	COLOR_LINE_END	COLOR_RING
COLOR_SIDE	COMBINE_COLOR	
COMMENT	CONCAVE_TO_CONCAVE	CONCAVE_TO_EDGE
CONDUCTING_LAYER_TYPE	CONDUCTIVE_HEIGHT	CONNECT
CONFLICT	CONTACT	
CONTACTARRAY	CONTINUE	CONVERT_TO_POLYGON
CONVERT_TO_RECT	CONVEX_TO_CONCAVE	CONVEX_TO_CONVEX
CONVEX_TO_EDGE	COORDINATES_AND_DEVICE_TYPES	CORE_AREA_OBJECT
CORE_OF_BLOCK	CORE_WIRE	CORE_WIRE_USE
CORNER	CORNER_PAD_CELL	CORNER_PAD_DESIGN
CORNER_TO_CORNER	CORNER_TO_CORNER_OR_EDGE	CORNER_TO_EDGE
COUNT	COVER_CELL	COVER_DESIGN
CREATED_PORTS	CROSS	
CUMULATIVE_TOPOGRAPHY	CUSTOM	CUTTING
DATATYPE_ZERO_ONLY	DEEP_NWELL_NET	DEEP_PWELL_NET
DEFOCUS	DELTA_WINDOW	DENSITY
DENSITY_GRADIENT	DENSITY_NORMAL	
DENSITY_PERIMETER	DENSITY_SMARTSTEP	DENSITY_TOTARGE
DENSITY_UNDER_1	DENSITY_UNDER_2	
DEPTH_OF_FOCUS_COLUMNS	DEPTH_OF_FOCUS_COLUMNS_HIGH	DEPTH_OF_FOCUS_COLUMNS_LOW
DEPTH_OF_FOCUS_ROWS	DEPTH_OF_FOCUS_ROWS_HIGH	DEPTH_OF_FOCUS_ROWS_LOW
DESIGN	DESIGN_VIEW	

Feedback

Table 36 Enumerated Type Elements (Continued)

DETAIL_ROUTE	DETAIL_ROUTE_USE	DEVICE_NAME
DEVICE_PIN	DIAMOND	DIELECTRIC_HEIGHT
DIFFERENT_NET	DIFFERENT_POLYGON	DIFFERENT_SIZE_RECTANGLES
DIODE_DESIGN	DISCARD	
DONUT	DOT	
DOUBLE	DOUBLE_HEIGHT_STANDARD_CELL	DOUBLE_LIST
DRC	DROP	DUAL
DYNAMIC_SHIELD	EDGE	EDGES
EDGE_TO_CONCAVE	EDGE_TO_CONVEX	EDIF
EIGHT	ELLIPSE	ELLIPTICAL
ELPC	EMITTER	EMPTY
ENCLOSE	END_CAP_DESIGN	
EQUATE_NETS	ERROR	ERRORS
ERROR_CLASSIFICATION	ERROR_NO_ABORT	EXCLUSIVE
EXPANDED_EDGE	EXPLODE	EXTENDED
EXTENTS	EXTERIOR	EXTERIOR_INTERIOR
EXTRACT	FAIL	
FAILED	FAILED_INSERTION	
FC_BUMP_CELL	FC_DRIVER_CELL	FEEDTHRU_BLOCKAGE
FEMTO	FILL	FILL_BLOCKAGE
FILL_BLOCKAGE_USE	FILL_DESIGN	
FILL_RINGS	FILL_TRACK	
FILL_USE	FILLER_CELL	FILLER_DESIGN
FIRST_PATTERN_LAYER	FIXED	FLIP_CHIP
FLIP_CHIP_DRIVER_DESIGN	FLIP_CHIP_PAD_CELL	
FLIP_CHIP_PAD_DESIGN	FLIP_FLOP_CELL	FLOATING

Feedback

Table 36 Enumerated Type Elements (Continued)

FOLLOW_PIN	FOLLOW_PIN_USE	FORTY_FIVE
FOUR	FRAM	FRAME_VIEW
FULL	FULL_NAME	FULL_NAME_CASE_INSENSITIVE
FULL_NAME_CASE_SENSITIVE	GALAXY_AND_XO_CELL	GALAXY_CELL
GDSII	GENERIC	GLOBAL
GLOBAL_REPLACE	GLOBAL_ROUTE	GLOBAL_ROUTE_USE
GROUND	GROUND_NET	GROW
GUIDANCE	HALF_WIDTH	HIERARCHICAL
HIGH	HIGHEST_TEXT	HOLDING_CELL
HORIZONTAL	HORIZONTAL_AXIS	HORIZONTAL_TRACK
HORIZONTAL_TRACK_OBJECT	HORIZONTAL_WIRE_TRACK	
HORIZONTALWIRE	HYBRID	HYBRID_BAIL_EARLY
HYBRID_PATTERN_ONLY	ICV	ICV_LCC_AOI
ICV_LCC_BLOCKING	ICV_LCC_CO	ICV_LCC_CONTOUR_AREA
ICV_LCC_M1	ICV_LCC_M1_DUMMY	ICV_LCC_M1_OPC_DUMMY
ICV_LCC_M2	ICV_LCC_M2_DUMMY	ICV_LCC_M2_OPC_DUMMY
ICV_LCC_M3	ICV_LCC_M3_DUMMY	ICV_LCC_M3_OPC_DUMMY
ICV_LCC_M4	ICV_LCC_M4_DUMMY	ICV_LCC_M4_OPC_DUMMY
ICV_LCC_M5	ICV_LCC_M5_DUMMY	ICV_LCC_M5_OPC_DUMMY
ICV_LCC_M6	ICV_LCC_M6_DUMMY	ICV_LCC_M6_OPC_DUMMY
ICV_LCC_M7	ICV_LCC_M7_DUMMY	ICV_LCC_M7_OPC_DUMMY
ICV_LCC_M8	ICV_LCC_M8_DUMMY	ICV_LCC_M8_OPC_DUMMY
ICV_LCC_M9	ICV_LCC_M9_DUMMY	ICV_LCC_M9_OPC_DUMMY
ICV_LCC_M10	ICV_LCC_M10_DUMMY	ICV_LCC_M10_OPC_DUMMY
ICV_LCC_MAX		
ICV_LCC_OD	ICV_LCC_OD_DUMMY	ICV_LCC_OD_OPC_DUMMY

Feedback

Table 36 Enumerated Type Elements (Continued)

ICV_LCC_PO	ICV_LCC_PO_DUMMY	ICV_LCC_PO_OPC_DUMMY
ICV_LCC_VIA1	ICV_LCC_VIA2	ICV_LCC_VIA3
ICV_LCC_VIA4	ICV_LCC_VIA5	ICV_LCC_VIA6
ICV_LCC_VIA7	ICV_LCC_VIA8	ICV_LCC_VIA9
IF_INSERTION_FAILS	IGNORE	IGNORE_CAP_LAYER_TYPE
IMAGE_CELL	IN	INCLUSIVE
INCLUSIVE_PARALLEL	INDIVIDUAL	INDUCTOR
INNER	INPUT_LAYOUT	INSERTION
INSIDE	INSIDE_TO_OUTSIDE	
INSTANCE_NAME	INSTANCES	
INTERACT	INTERACTING	INTERIOR
INTERNAL_DESIGNS	INTERSECTION	
IO_PAD_CELL	KEEP	
KEEP_ALL	KEEP_ENCLOSED	KEEP_INTERACTING
KEEP_ONE	KILO	LATCH_CELL
LAYER	LAYER_INTENT_BASE	LAYER_INTENT_DEVICE
LAYER_INTENT_FILL	LAYER_INTENT_INTERCONNECT	LAYER_INTENT_METAL
LAYER_INTENT_VIA	LAYER_MAPS	LAYER1
LAYER2	LAYOUT_UNTEXTED	LAYOUT_VIEW
LCC	LEVEL_SOURCE_DRAIN	LI_BASE
LI_FILL	LI_INTERCONNECT	LI_METAL
LI_VIA	LIB_CELL_DESIGN	LIB_CELL_PIN_CONNECT
LIB_CELL_PIN_CONNECT_USE	LIBRARIES_AND_VIEWS	LINE
LINE_END_DIFF_COLOR	LINE_END_SAME_COLOR	LINE_END_TO_LINE_END
LINE_END_TO_SIDE	LINE_TO_LINE	LINEAR_CONTEXT
LINK	LOCAL	LOGIC_0_NET

Feedback

Table 36 Enumerated Type Elements (Continued)

LOGIC_1_NET	LOOP	LOW
LOWER	LOWER_CELLS	LOWEST
LVS	LVS_NETLIST_FLOW_ICV	LVS_NETLIST_FLOW_SPICE
LVS_USER_UNIT_METER	LVS_USER_UNIT_MICRON	
M1_ROUTE_GUIDE	M10_ROUTE_GUIDE	M11_ROUTE_GUIDE
M12_ROUTE_GUIDE	M13_ROUTE_GUIDE	M14_ROUTE_GUIDE
M15_ROUTE_GUIDE	M2_ROUTE_GUIDE	M3_ROUTE_GUIDE
M4_ROUTE_GUIDE	M5_ROUTE_GUIDE	M6_ROUTE_GUIDE
M7_ROUTE_GUIDE	M8_ROUTE_GUIDE	M9_ROUTE_GUIDE
MACRO_CELL	MACRO_DESIGN	MACRO_PIN_CONNECT
MACRO_PIN_CONNECT_USE	MAIN	MARKER_LAYER_TYPE
MASK_EIGHT	MASK_ELEVEN	MASK_FIFTEEN
MASK_FIVE	MASK_FOUR	MASK_FOURTEEN
MASK_NINE	MASK_ONE	MASK_ONE_HARD
MASK_ONE_SOFT	MASK_SEVEN	MASK_SIX
MASK_TEN	MASK_THIRTEEN	MASK_THREE
MASK_TWELVE	MASK_TWO	MASK_TWO_HARD
MASK_TWO_SOFT	MAX	MAX_AREA
MAX_CONDITION	MAX_CONDITION	MAX_INSERTION
MEEF	MEGA	MERGE
MERGED	MERGE_ALL	MERGE_CONNECTED
MERGE_CONNECTED_AND_TOP	MERGE_TOP_THEN_CONNECTED	MERGE_TOP_PORTS_THEN_CONNECTED
MERGED	METAL_HEIGHT	
METER	MICRO	MICRON
MID_CELL	MILKYWAY	MILLI

Feedback

Table 36 Enumerated Type Elements (Continued)

MIN	MIN_AREA	MISSING_CELLS
MISSING_COLOR	MIXED	MODULE_CELL
MODULE_DESIGN	MORE_THAN_TRIPLE_HEIGHT_CELL	MUTUAL
MUTUAL_NON_ORTHOGONAL	NAME_MATCHED	NANO
NDB_ALL	NDB_ANY	NDB_BJT
NDB_CACHE	NDB_CAP	NDB_CELL
NDB_DIODE	NDB_IGNORE	NDB_IND
NDB_INST	NDB_MOS	NDB_NET
NDB_NETLIST	NDB_NMOS	NDB_NONE
NDB_NP	NDB_NPN	NDB_ONE_WAY
NDB_PIN	NDB_PMOS	NDB_PN
NDB_PNP	NDB_RES	NDB_SUBCKT
NDB_UNDEFINED	NDB_TWO_WAY	NDB_UNCACHED
NDM		
NEGATIVE_45	NEIGHBORING_GAP	NEITHER
NET_NAME	NET_PORTPROPERTY	NETLIST_ANNOTATION_FILE_SPICE
NETLIST_ONLY	NETLIST_PEX_SPICE	NETLIST_XTR_SPICE
NETS	NEVER	
NEW_AREF_ONLY	NINETY	
NMOS	NO_COLOR	NO_EXPLODE
NODE	NOMINAL	NOMINAL_VARIATION
NON_CONT_PG	NON_ORTHOGONAL	NON_RECTANGULAR_SHAPES
NONE	NONE_DENSITY	NONE_DENSITY_PERIMETER
NONE_INCLUSIVE	NORMAL	NOT_A_DOUBLE
NOT_ADJACENT	NOT_CONTAINED	NOT_CORNER

Feedback

Table 36 Enumerated Type Elements (Continued)

NOT_DEFINED_LAYER_TYPE		
NOT_TEXTED	NOTCH	NP
NPN	NULL_CONNECT_DATABASE	NWELL_NET
NULL_PARAMETRIX_LAYER	OASIS	OCTAGONAL
ODD_LOOP_ONE_LINK	ODD_LOOP_ALL_LINKS	ODD_LOOP_ALL_NODES
ODD_LOOP_ALL_LOOP	ODD_LOOP_INSIDE_RING	OFF
OLD_AREF_ONLY	ON	ONE
ONE_EIGHTY	ONE_OR_NEITHER	ONE_POLYGON_PER_LAYER
ONE_X	ONLY_COLOR_1	ONLY_COLOR_2
ONLY_COORDINATES	OPC	OPC_USE
OPEN_OUTER_BOUNDARY	OPENACCESS	ORTHOGONAL
OTHER_NETS	OUT	OUTPUT_CELL
OUTPUT_ERRORS	OUTSIDE	OVERLAP
OVER_EXPOSURE	OVERSIZE	OVERWRITE
OWNER_NET	OXIDE_HEIGHT	PAD_DESIGN
PAD_FILLER_CELL	PAD_SPACER_DESIGN	PARALLEL
PARALLEL_POINT_PROJECTION	PARTITION	PARTITION_CONTEXT
PATH	PATH_OBJECT	
PATH_SEGMENT	PAUSE	PERCENT
PERIMETER	PERIMETER_DENSITY	PERPENDICULAR
PG_FOLLOW_PIN	PG_PIN	PG_PIN_ONLY_CELL
PG_RING	PG_STRIPE	PHYSICAL_ONLY_DESIGN
PICO	PIN	PIN_BLOCKAGE
PIN_NAME	PIN_OBJECT	PIN_TEXT
PINS	PLACEMENT_BLOCKAGE	PLACEMENT_HARD_BLOCKAGE

Feedback

Table 36 Enumerated Type Elements (Continued)

PLACEMENT_HARD_MACRO_BLOCKAGE	PLACEMENT_PARTIAL_BLOCKAGE	
PLACEMENT_SOFT_BLOCKAGE	PM_EXACT	
PM_EDGE_NONUNIFORM	PM_EDGE_UNIFORM	PM_EQ
PM_FUZZY	PM_GE	PM_GT
PM_LE	PM_LT	PM_MARKER_CENTER
PM_MARKER_EDGE	PM_NONE	PM_ONE_DIMENSIONAL
PM_TWO_DIMENSIONAL	PM_TWO_DIMENSIONAL_RUN_LENGTH	PM_UNIFORM
PMOS	PN	PNP
POINT	POINT_TO_POINT	POINT_TOUCH
POLY_ROUTE_GUIDE	POLYGON	POLYGON_OBJECT
POLYGON_TEXT	PORT_NAME	PORT_TEXT_MATCHED
PORT_TOPOLOGY_MATCHED	POSITIVE_45	POWER
POWER_AND_GROUND	POWER_NET	POWER_OR_GROUND
POWER_XOR_GROUND	PRE_COLOR_ALL_NODES	PRE_COLOR_ENVELOPE_PATH
PRE_COLOR_MIN_PATH	PRE_THREE_COLOR_ALL_NODES	PRE_THREE_COLOR_MIN_PATH
PREFIX_NAME	PRIMARY_AXIS	
PRINT	PRINT_ALL	PRINT_RESET_ALL
PROJECTING	PROJECTION	PROPERTY
PROPERTY_ERRORS_ONLY	PROPERTY_TEXT	PWELL_NET
QCMP	QTF_LAYER_TYPE	RADIAL
RADIAL_INSIDE	RADIAL_INTERSECTION	RADIAL_OUTSIDE
RAM_CELL	REASSIGN	
REASSIGNED_SHORTED	RDL_USE	
RECT	RECTANGLE	RECTANGLE_OBJECT
REDUCTION	REGION	REGION_CONTEXT

Feedback

Table 36 Enumerated Type Elements (Continued)

REGULAR_TEXT	RELATED_COINCIDENT	RELATIVE
REMAINDER	REMOVE_LAYER_TYPE	REMOVE_MULTI_EQUIVS
RENAME	RENAMED	REPLACE
REPORT	RESET	RESET_NET
RESISTOR	RETAIN_45	RIGHT
RING	RING_USE	ROM_CELL
ROTATE_180	ROTATIONAL	ROUND
ROUTE_GUIDE_OBJECT	ROUTE_TYPE	ROUTING_BLOCKAGE
ROUTING_FOR_TOP_LEVEL_BLOCKAGE	RPGROUP_BLOCKAGE	SAME
SAME_COLOR	SAME_DEVICE_NAME_ONLY	SAME_MASK
SAME_NET	SAME_POLYGON	SCAN_NET
SCREEN_BLOCKAGE	SERIES	SERIES_PARALLEL
SHAPING_BLOCKAGE	SHIELD_ROUTE	
SHIELD_ROUTE_USE	SHORT_BY_LONG	SHORTED
SHRINK	SIDE_DIFF_COLOR	SIDE_SAME_COLOR
SIDE_TO_SIDE	SIGNAL	SIGNAL_CONTEXT
SIGNAL_DETAIL	SIGNAL_GLOBAL	SIGNAL_LOCAL
SIGNAL_NET	SIGNAL_USER	SIMPLE
SIMULATE	SINGLE	SINGLE_DEVICE_EQUIVS
SINKS	SIXTEEN	SKIP
SKIP_PARENTS	SLASH	SLEEP_CONTROL
SLOT_BLOCKAGE	SOFT_MACRO_IO_FIXED_CELL	SOG_DESIGN_CELL
SOURCES_ON_SAME_NET	SPECIAL_POWER	SPECIAL_VIA_CELL
SPICE	SPICE_BJT	SPICE_CAP
SPICE_CELL	SPICE_DIODE	SPICE_IND

Feedback

Table 36 Enumerated Type Elements (Continued)

SPICE_MOS	SPICE_RES	SPLIT_CLOCK
SQUARE	SQUARE_CENTERED	SQUARE_EXTENDED
SQUARE_INSIDE	SQUARE_OUTSIDE	STACK_VIA_CELL
STANDARD_CELL	STANDARD_FILLER_CELL	START
STATISTICAL_COUNT	STOP	STRING
STRIPE	STRIPE_USE	SUBSTRING_NAME
SUM	SYMMETRIC	SYMMETRIC_NON_INTERSECTING
TAP_CELL	TECH	TERMINAL
TERMINAL_NAME	TERMINAL_TEXT	TEXT
TEXT_DISPLAY	TEXTED	TIE_HIGH
TIE_LOW	TOP	TOP_CELL
TOP_OF_NET	TOP_PORT	TOUCH
TREE	TRIANGLE	TRIANGLE_ONE_LINK
TRIANGLE_ALL_LINKS	TRIANGLE_ALL_NODES	TRIPLE_HEIGHT_STANDARD_CELL
TRUNCATE	TSV_CELL	
TWO	TWO_SEVENTY	TWO_X
UF_ADJUSTABLE	UF_CELL	UF_CHECKERED
UF_DPT_SPACING	UF_EXPANDABLE	UF_LAYER
UF_LINE_END	UF_LINE_SIDE	UF_PATTERN
UF_POLYGON	UF_STACK	UF_STRIPE
UNCONNECTED	UNDER_EXPOSURE	
UNDERSIZE	UNSET_NET	UNKNOWN_L_MODEL_CELL
UNMATCHED_PINS	UNMATCHED_TEXT	UNSPECIFIED
UNSPECIFIED_DOUBLE	UNUSED	UNUSED_CELLS
UNUSED_MARKERS	UPDATE	UPPER
USE	USE_ALL_EQUIV	USE_MILKYWAY

Feedback

Table 36 Enumerated Type Elements (Continued)

USE_OPENACCESS	USE_NDM	USED_CELLS
USER_DEFINED	USER_ROUTE	USER_ROUTE_USE
VCELL	VCMP	VERBOSE
VERILOG	VERTICAL	VERTICAL_AXIS
VERTICAL_TRACK	VERTICAL_TRACK_OBJECT	VERTICAL_WIRE_TRACK
VERTICALWIRE	VERTICES	VIA_BLOCKAGE
VIA_LAYER_TYPE	VIA_OBJECT	VIEWONLY_LAYER_TYPE
VIEWS	VIRTUAL_AMBIT	VIRTUAL_CORE
WAIVE	WARN	WARNING
WATCH	WELL_TAP_DESIGN	WINDOW
WIRING_BLOCKAGE	WITH_PORTS	X_BY_Y
XO_CELL	ZERO_SKEW	ZERO_SKEW_USE

Operators

Operators are symbols that have a particular meaning. For example, + indicates that two numbers are added. The operator types are shown in Table 37.

Table 37 Operator Types

Operator type	Number of arguments	Example
unary	1	Logical not (!)
binary	2	Addition (+) Exponentiation (**)
ternary	3	Question operator (e1 ? e2 : e3)

Feedback

Comments

Comments are text used to document the program. Comments are useful in indicating the intent of the code to readers. Comments are treated as white space. The comment types, shown in Table 38, are based on how many lines each comment contains.

Table 38 Comment Types

Comment type	Starts with	Ends with	Length
line	`//`	Next new line	One line
block	`/*`	`*/`	Any length

Block comments do not nest. If */ is seen within a comment, the preprocessor reports a warning. For example, if the runset contains

```
/*      - 1st comment start
   /*      - 2nd comment start
   */      - 2nd comment end
*/      - 1st comment end
```

The "1st comment start" matches with "2nd comment end"; the "1st comment start" does not match with "1st comment end".

Here are some examples of comments:

```
/* this is a single-line block comment */

// this is a line comment, so /* is ignored

/* block comments /* cannot nest. Therefore this comment evokes a
warning from the preprocessor */

/*
 * block comments can span
 * several lines, such
 * as this one does.
 */

/*
  A leading * is not required on each line of a
  block comment but does make the comment stand out
  from the program.
*/
```

Feedback

Environment Variable Reference

Environment variables are a set of dynamic values that can be used to control the way processes are run.

Environment variable references have the dollar sign character ($) followed by any normal identifier. If an undefined environment variable is referred to, it is rejected with a compile-time error.

Environment variables are replaced by a string literal containing the value of the environment variable. Replacement is only done on tokens of the form $<id>. Specifically, there is no replacement within string literals; replacement is not required because adjacent string literals are concatenated.

Note:

If the value of the environment variable is an empty string, it is converted to a string literal whose value is an empty string.

Here are some examples of environment variable usage:

```
x : string = $y;

/*
 * assuming environment variable my_value contains the value abcd,
 * the following code evaluates to true; otherwise evaluates to false.
 */
res : boolean = ( $my_value == "abcd" );

/*
 * if environment variable aaa contains the value
 * /home/dir/path, the following code produces a string
 * containing a path to the file "cases" in that directory.
 */
path : string = $aaa "/cases";

/*
 * if environment variable bbb is defined but empty, the
 * following code produces the string abcdef.
 */
mystr : string = "abc" $bbb "def";
```

Variables and Operators

This section introduces the concepts of variables and operators in PXL.

Feedback

Variables

Variables are used to store information temporarily in the program.

When declaring variables, the variable name must be followed by a colon (:) and the variable type. Variables can also be given an initial value when declared. See the examples in the following section.

Note:

See the Identifiers section for information about naming restrictions.

Initializing Variables

You can initialize variables to any of the supported data types in two ways:

- Using a literal value.

```
y : boolean = true;           // initialize by literal value
```

- Using an expression.

```
z : boolean = !y;             // initialized by expression
```

The following examples show different methods of initialization:

```
x : integer;                      // no initial value

pi : double = 3.141592;           // initialized by value

pi2 : double = pi * 2;            // initialized by expression

s : string = "abc" "def";         // concatenated

s1 : string = "abc" + pi;         // converted to string,then concatenated

c : constraint of double = (10.0,200];  // initialized by value
```

Scope of Variables

Scope in PXL is the meaning that variables and expressions have within several contexts in a program. The various rules and conditions that relate to scope of variables are described here by using examples.

Local scope is defined by a set of braces ({ }). That is, compound statements, defined with curly braces, define a new variable scope. See Compound Statement on page 241 for the use of braces.

Flow control statements create a new scope.

```
if ( var == 1) {          // Start of new scope
   my_var: integer = 1;
```

Feedback

```
   ...
}                          // End of scope.
```

- Variables declared in a local scope do not exist outside of that scope. That is, if a variable is declared inside of a particular scope, the variable cannot be used outside of that scope. For example,

```
{
   x = y + b;
}
z = x;      // x is not defined; it only exists in the local scope
```

- Variables are global from outermost scope to innermost scope. For example,

```
{
   x : integer = 1;
      {
      a = x + b;  // x exists and can be seen within this scope
                  // (global from  outer scope)
      x = y + b;  // global value of x is overwritten
      }
   z = x;         // x is defined here and contains the value of y + b
}
z = x;            // x is not defined here - parser error
```

- Variables declared in an outer scope that are redeclared in an inner scope are shadowed to prevent value collisions.
  - If variable shadowing occurs and the scope of the shadowed variable is not defined by a function declaration, the program generates a warning.
  - Within function calls, shadowing is expected and there is no warning. For example,

```
y: integer = 1;
x: integer = 1;
{
   y : integer = 3;  // The original y value is shadowed.
                     // This is a new y.
   x = x + 1;        // x is equal to 2 here
   y = y + 1;        // y is equal to 4 here
}
                     // y is equal to 1 here - the shadowed value
                     // returns when exiting the inner scope
                     // x is equal to 2 here
```

Here is another example of variable shadowing. The function declaration:

```
fxy : function (
   x : integer;
   y : integer;
) returning z : integer {
      x = x + 1;
      y = y + 1;
```

Feedback

```
      z = x + y;
};
```

The function call:

```
x1 : integer = 1;
y : integer = 1;

z1 = fxy(x1, y);  // within the function y is shadowed
                  // y in the local scope is "2"
x2 = x1;          // x1 is "1"
y1 = y;           // y is still "1"
z2 = z1;          // z1 is "4"
```

Operators

Operators are symbols used within an expression to specify certain operations that are to be performed while evaluating that expression.

PXL supports the following operators:

- Assignment Operators
- Relational Operators
- Logical Operators
- Conditional Operator
- Numeric and Bitwise Operators
- Operators With Double-Only Outputs

Assignment Operators

An assignment operator evaluates an expression and saves it to a target lvalue.

An lvalue is an expression that can contain a value and can be used on the left side of an assignment statement.

Assignments involving the use of > or >= in constraint values must have white space before and after the assignment operator. You can also place the constraint value within parentheses following the assignment =.

For example,

```
abc = >=3;
xyz = (>5);
```

Feedback

Note:

Assignment operators cannot cascade. This example is not legal:

```
a = b = c
```

The following examples show proper assignment:

```
x:   integer;  // declare a variable named x

point: newtype struct of { x: integer; y: integer; }; // declare a struct
                                                      // of integers
p: point;
x   = 42;      // assign x a value of 42
p.x = 10;      // assign p.x a value of 10
p.y = 27;      // assign p.y a value of 27
```

Relational Operators

A relational expression evaluates an expression to a Boolean value, either true or false.

Table 39 defines the relational operators.

Table 39 Relational Operators

Syntax	Description	Supported for
`e1 == e2`	true if e1 is equal to e2, false otherwise	All completely defined data types
`e1 != e2`	true if e1 is not equal to e2, false otherwise	All completely defined data types
`e1 < e2`	true if e1 is less than e2, false otherwise	Numbers
`e1 <= e2`	true if e1 is not greater than e2, false otherwise	Numbers
`e1 >= e2`	true if e1 is not less than e2, false otherwise	Numbers
`e1 > e2`	true if e1 is greater than e2, false otherwise	Numbers

Logical Operators

A logical expression evaluates to a Boolean value. A logical operator requires that all arguments to be of boolean type. Expressions are evaluated from left to right, and stop when the result is defined. Table 40 defines the logical operators.

Table 40 Logical Operators

Syntax	Description
`e1 && e2`	if e1 is false, then false, else e2

Feedback

Table 40 Logical Operators (Continued)

Syntax	Description
`e1 || e2`	if e1 is true, then true, else e2
`! e1`	if e1 is true, then false, else true

Conditional Operator

A conditional expression takes three arguments; the first argument must be of boolean type. Table 41 defines the conditional operator.

Table 41 Conditional Operator

Syntax	Description
`e1 ? e2 : e3`	if e1 is true, then e2, else e3

Consider a conditional operator with three arguments e1, e2, and e3. The conditional operator is evaluated in the following order:

1. The first argument (e1) is evaluated.
2. If the first argument (e1) evaluates to true, the second argument is evaluated.
3. If the first argument (e1) evaluates to false, the third argument (e3) is evaluated.

Numeric and Bitwise Operators

All standard mathematical operators are supported in PXL. The bitwise operators are supported only for integers. Table 42 defines the numeric and bitwise operators.

Table 42 Numeric and Bitwise Operators

Syntax	Description	Allowed for	Comments
`e1 + e2`	Addition	Integer and double	Returns a double if either argument is a double.
`e1 - e2`	Subtraction	Integer and double	Returns a double if either argument is a double.
`e1 * e2`	Multiplication	Integer and double	Returns a double if either argument is a double.
`e1 % e2`	Integer remainder	Integer and double	Returns a double if either argument is a double.

Feedback

Table 42 Numeric and Bitwise Operators (Continued)

Syntax	Description	Allowed for	Comments
`e1 << e2`	Logical left shift (zeros shifted in)	Integer	
`e1 >> e2`	Logical right shift (zeros shifted in)	Integer	
`e1 & e2`	Bitwise AND	Integer	
`e1 | e2`	Bitwise OR	Integer	
`e1 ^ e2`	Bitwise XOR	Integer	
`! e1`	Bitwise NOT	Integer	

Operators With Double-Only Outputs

Division (`/`) and exponentiation (`**`) treat the arguments as doubles and always return only doubles. Table 43 defines these operators.

Table 43 Operators With Double-Only Outputs

Syntax	Description	Accepts	Comments
`e1 / e2`	Division	Integer and double	e1 and e2 are forced to be doubles. The operation always returns a double.
`e1 ** e2`	Exponentiation	Double	e1 and e2 are forced to be doubles. The operation always returns a double.

Note:

Any expression with a division operator (/) or exponentiation operator (**) cannot be used as the index of a list or hash table.

To perform an integer operation, you must wrap the operation in a call of the `dtoi()` function. This function converts a double into an integer. For example,

```
var : integer = dtoi(10/3);
```

Feedback

Member Reference

A member reference expression accesses one member of a compound object. The member reference takes the type of that single member. The member is an lvalue and functions as the target of an assignment. Table 44 defines member reference expressions. See Assignment Operators on page 208." for more information about lvalue.

Table 44 Member Reference

Syntax	Description
`e1 . e2`	Structure member access by name
`e1 [ e2 ]`	List or hash member access by index

Precedence and Associativity

Table 45 lists the precedence and associativity of all operators in PXL. The top of the table is the highest precedence, and the bottom is the lowest.

Table 45 Precedence and Associativity of Operators (Highest Precedence First)

Operators	Associativity
`() [] .`	Left to right
`**`	Right to left (exponentiation)
`! + -`	Right to left (unary operators)
`== != < <= > >=`	Illegal (unary operators)
`* / %`	Left to right
`+ -`	Left to right
`<< >>`	Left to right
`< <= > >=`	Non associative
`== !=`	Non associative
`&`	Left to right
`^`	Left to right
`|`	Left to right

Feedback

Table 45 Precedence and Associativity of Operators (Highest Precedence First) (Continued)

Operators	Associativity
`&&`	Left to right
`||`	Left to right
`?:`	Right to left
`= @= ?=`	Illegal
User-defined operators	Non associative

Table 46 defines each type of associativity.

Table 46 Associativity

Associativity	Definition
Left to right	Expressions are evaluated from left to right.
Right to left	Expressions are evaluated from right to left.
Non associative	Parenthesis are required.
Illegal	The operator does not cascade. Operators do not produce the same type as they consume.

The following examples show how associativity works:

```
/*
 * pairs of expressions show evaluation order
 */

A + B + C
( A + B ) + C

A ** B ** C
A ** ( B ** C )

A * B + C >> 2
( ( A * B ) + C ) >> 2

A >= B == C
( A >= B ) == C

A < B == C > D
( A < B ) == ( C > D )
```

Feedback

Data Types and Typing

This section describes PXL-supported data types and the factors that determine how typing works within PXL.

Variables can be explicitly declared with a type. For example,

```
i : integer = 1 ;  // i is declared as an integer
```

Variables can be implicitly typed based on the return type of the operator. For example,

```
a : double = 1.1203;
b : double = 2.0213;
c = a + b;            // c is implicitly the type of "double"
```

Supported Data Types

PXL supports most primitive types and some tool-specific data types.

Primitive Types

Table 47 shows the primitive data types supported by PXL.

Table 47 Primitive Types

Data type	Description
integer	Integer; size is implementation-defined, generally 32-bit signed values.
double	Double-precision floating-point; size is defined at the time of implementation, generally 64-bit values.
boolean	True or false
string	Sequence of characters; supporting escape sequences to include special characters.
handle	An opaque data type, used to track data objects external to the PXL program itself.
constraint of double and constraint of integer	Concise representation of a set of contiguous double values or integer values.
predicate_handle	Boolean expression with special consideration for evaluating layer identifiers as empty or non-empty.

Feedback

integer

Here are examples of integer declarations:

```
i : integer;     // declared but not initialized
i : integer = 1; // declared and initialized
```

string

Defines a string of zero or more characters. The string type is initialized from a string literal. For example,

```
s : string = "xyzzy";
```

The string type supports the operations shown in Table 48 on any expression of string type.

Table 48 String Type Operations

Operation	e2 type	Return	Modifies e1	Comment
`e1.empty()`	--	boolean	no	Returns true if the string is empty, that is the string is `""`; otherwise returns false.
`e1.size()`	--	integer	no	Returns the number of characters in the string.

handle

An internal representation for an externally managed object. The type handle has a string representation that is unique for each externally managed object.

constraint of double and constraint of integer

Constraint of double and constraint of integer are mathematical ranges for dimensional rule values. These constraints provide an intuitive mechanism for describing parameters such as DRC spacing distances.

If the type of constraint is unspecified, it implies that the range is a constraint of double.

The constraint data types support the operations shown in Table 49 on an expression of this type.

Table 49 Constraint of Double and Constraint of Integer Operations

Operation	e2 type	Return	Modifies e1	Comment
`e1.contains(e2)`	item	boolean	no	Returns true if the e2 value is within the range specified by the e1 constraint.

Feedback

Table 49 Constraint of Double and Constraint of Integer Operations (Continued)

Operation	e2 type	Return	Modifies e1	Comment
`e1.category()`	--	enum	no	Returns the enumerator of the constraint from the predefined enumeration constraint category. See Table 50.
`e1.lo()`	--	integer/double	no	For binary constraints, returns the lower bound of the constraint: a double for a constraint of double; an integer for a constraint of integer.[3]
`e1.hi()`	--	integer/double	no	For binary constraints, returns the upper bound of the constraint: a double for a constraint of double; an integer for a constraint of integer.[3]

The constraint data type can only be created by assignment from another like-typed constraint or from a literal constant, as shown in Table 50.

Table 50 Constraint Categories

Literal	Description	Constraint category
`[a,b]`	Closed range from a to b, including both endpoints	`CONSTRAINT_GELE`
`(a,b)`	Open range from a to b, omitting the endpoints	`CONSTRAINT_GTLT`
`[a,b)`	Half-open range from a to b, including a but not b	`CONSTRAINT_GELT`
`(a,b]`	Half-open range from a to b, omitting a but including b	`CONSTRAINT_GTLE`
`>= a`	Range of all numbers greater than or equal to a	`CONSTRAINT_GE`
`> a`	Range of all numbers greater than a	`CONSTRAINT_GT`
`<= a`	Range of all numbers less than or equal to a	`CONSTRAINT_LE`
`< a`	Range of all numbers less than a	`CONSTRAINT_LT`
`== a`	Range containing only the number a	`CONSTRAINT_EQ`
`!= a`	Range containing all numbers except a	`CONSTRAINT_NE`

Both `lo()` and `hi()` are valid for all constraints. For single-ended constraints, both functions are defined to return the single endpoint, 'a'. For double-ended constraints, `lo()` returns 'a' and `hi()` returns 'b'.

3. *For unary constraints, such as == and >, both `e1.lo()` and `e1.hi()` return the single limit.*

Feedback

Examples

Here are some examples.

- Constraint of integer

  `[2,8]` contains the values 2, 3, 4, 5, 6, 7, 8

  `(2,8)` contains the values 3, 4, 5, 6, 7

- Constraint of double

  `[2,8]` contains all real numbers in the range including 2.0 and 8.0

  `(2,8)` contains all the real numbers in the range except 2.0 and 8.0

- Declaration example

```
a_value : constraint of double = [2.0,8.0];
```

- Assignment examples

```
distance <= 2.0          // All values less than or equal to 2.0
distance = <= 2.0        // All values less than or equal to 2.0
distance = [ 2.0, 8.0 )  // All values >= 2.0 and < 8.0
count = 5                // Count is exactly equal to 5
count = ==5              // Count is exactly equal to 5
```

Note:

The symbols such as == and <= are part of the constraint. The assignment operator = is optional for variables expecting a constraint type. However, if a variable of the constraint type is on the right side, the assignment operator is necessary:

```
my_constraint: constraint of double = < 5.0;
external1(layer1, distance = my_constraint );
external1(layer1, distance <5);                // same as above
external1(layer1, distance = < 5);             // same as above
```

predicate_handle

A Boolean expression enclosed in double brackets[[…]]. Any sort of Boolean logic containing at least one layer name or violation name is allowed. A layer name is special inside the predicate expression because at runtime, it evaluates to `true` if the layer has data, or `false` if the layer is empty. Violation objects also evaluate to `true` if there are violations and `false` if there are no violations.

Feedback

In runset functions, predicate_handle arguments control the operation of a given function based on the expression evaluation:

- `true`: The function operates as normal.
- `false`: The function produces an empty output layer.

For example:

```
temp_layer = and(poly, diff, predicate = [[metal1]]);
```

If `metal1` is empty, the `and()` function is skipped, and `temp_layer` is empty.

User-Defined Structures and Types

In addition to the primitive types, PXL supports the structures and types shown in Table 51.

Table 51 User-Defined Types

User-defined type	Salient features
newtype...	Defines a type.
enum of...	Defines a list of enumerated elements.
list of...	Defines a list of elements.
hash of...	Converts a list of elements of one data type to another data type.
struct of...	Defines a composite type consisting of multiple variables.
function	Defines a user-defined function that can be passed to another user-defined function and can be called as if it were an ordinary variable.

newtype...

A newtype declares a type of new variable. It does not define a variable. Each type is unique.

The following example is of a valid type definition:

```
/*
 * define a new type, direction.
 */

direction: newtype enum of {
   up, down, left, right, all // trailing comma is allowed
};

/* define two distinct types */
T1: newtype integer;
```

Feedback

```
T2: newtype integer;

/* define objects of the two types */
o1: T1;
o2: T2;
```

The following example is of an invalid type definition:

```
/* given the definitions above, o1 and o2 are of
 * different types, therefore they cannot be compared
 * and this is an error
 */

if (o1 == o2) { ... }
```

Here is an example of defining and using a new data type called direction. It has four allowed values.

```
direction : newtype enum of {
   NORTH,
   SOUTH,
   EAST,
   WEST
};
compass : direction;  // Declare compass as type "direction"
compass = EAST;       // Assign a legal value to "compass"
```

enum of...

Defines a set of names (enum literals) distinct from other types. The PXL coding standard is for enum literals to be all uppercase characters.

For example,

```
t3: newtype enum of {
   X,
   Y,
   Z,               // Trailing comma is not needed; silently discarded.
};
```

Here is another example:

```
direction : newtype enum of {
   NORTH, SOUTH, EAST, WEST
};

compass : direction;         // Declare compass as type "direction"
compass = EAST;              // Assign a legal value to "compass"
```

An enum can share names with other enumeration types without error:

```
t1_e : newtype enum of {
   NONE, TOP, BOTTOM, ALL
};
```

Feedback

```
t2_e : newtype enum of {
   NONE, LEFT, RIGHT, ALL
};
```

Cross-type operations are not allowed. Also, within one type definition, you cannot use the same name more than one time.

This example shows a cross-type operation. This operation is not legal:

```
v1 : t1_e;     // NONE, TOP, BOTTOM, ALL
v1 = NONE;     // Legal assignment
v1 = TOP;      // Legal assignment
v1 = RIGHT;    // Illegal assignment

v2 : t2_e;     // NONE, LEFT, RIGHT, ALL
v2 = NONE;     // Legal assignment

if (v2 == v1) // Illegal
```

list of...

Lists are arrays of objects.

- All of the objects are of the same type. This type is allowed to be empty or to contain duplicate objects.
- It can be processed using the `foreach()` looping construct.
- Lists can be addressed by an index number using brackets ([ ]).

See Nested Containers on page 228" for information about a list of lists.

This type supports the operations shown in Table 52.

Table 52 List Operations

Operation	e2 type	Return	Modifies e1	Comment
`e1.empty()`	--	boolean	no	Returns true if the list contains no elements; otherwise returns false.
`e1.size()`	--	integer	no	Returns the number of elements in the list.
`e1.contains(e2)`	item	boolean	no	Returns true if e2 is an element of the e1 list.
`e1.front()`	--	item	no	Returns a copy of the first element of the list.
`e1.back()`	--	item	no	Returns a copy of the last element of the list.

Feedback

Table 52 List Operations (Continued)

Operation	e2 type	Return	Modifies e1	Comment
`e1.chop_back()`	--	list	no	Returns a copy of the e1 list with the last element removed.
`e1.chop_front()`	--	list	no	Returns a copy of the e1 list with the first element removed.
`e1.append(e2)`	item	list	no	Returns a copy of the e1 list with e2 added to the back of the list.
`e1.prepend(e2)`	item	list	no	Returns a copy of the e1 list with e2 added to the front of the list.
`e1.push_front(e2)`	item	void	yes	Adds a copy of the e2 expression to the front of the list.
`e1.push_back(e2)`	item	void	yes	Adds a copy of the e2 expression to the back of the list.
`e1.pop_front()`	--	void	yes	Removes the first element of the list.
`e1.pop_back()`	--	void	yes	Removes the last element of the list.
`e1.merge(e2)`	list	list	no	Returns a copy of the e1 list with the e2 list concatenated to the end.
`e1.remove(e2)`	item	void	yes	Removes all values in the e1 list that are equal to the e2 expression.

The following are examples of legal code:

```
x: layer = ... ;

MyPartition: list of layer = PartitionLayer(L1);

foreach (part in MyPartition) { ... }

if (MyPartition.contains(x)) { ... }

i : integer = MyPartition.size();
```

The following example shows how to concatenate a list:

```
list1 : list of integer = { 1, 2, 3 };
list2 : list of integer = { 10, 20, 30 };
list3 : list of integer = list1.merge(list2);
```

list3 now has the value { 1, 2, 3, 10, 20, 30 };

Feedback

Here is an example of initializing and using a list of strings.

```
// declare myList to be a list of strings, uninitialized
myList : list of string;

// declare myList to be a list of strings, initialized
myList : list of string = { "this", "is" , "a", "list" };
// myList[0] = "this"
// myList[3] = "list"
```

Built-in list operator functions are supported. For example,

```
myList : list of string = { "this", "is" , "a", "list" };

s = myList.size();          // returns the size of a list: s is "4"
a = myList.front();         // a contains a copy of L1[0]: a is "this"
myList.pop_front();         // removes the first element of the list,
                            // the list is now { "is", "a", "list" }
myList.pop_back();          // removes the last element of the list,
                            // the list is now { "is", "a" }
myList.push_front("xyz");   // adds a new element containing "xyz"
                            // to the front of the list.
                            // the list is now { "xyz", "is", "a" }
myList.push_back("bar");    // adds a new element containing "bar"
                            // to the back of the list.
                            // the list is now { "xyz", "is", "a", "bar" }
```

hash of...

Defines a mapping from one data type to another. The source type is called the key type; the destination type is called the value type. That is, the hash defines an array of key/value pairs.

This type requires that the key type be string, integer, or enum, or a type derived from one of these three types. It cannot be a double. The value type can be of any previously defined type.

The following example creates a hash that converts integers to strings. In this case, the key type is integer and the value type is string. This type is initialized using the => operator.

```
t1_h : newtype hash of integer to string;
t2_h : newtype hash of string to integer;

h1 : t1_h = {
   1  => "a",
   2  => "b",
   26 => "z"
};

h2 : t2_h = {
   "xyzzy" => 1,
```

Feedback

```
   "plugh" => 10,
   "help"  => 100
};
```

Here is an example of how a derived type can be used:

```
t1 : newtype integer;
h1_h : newtype hash of t1 to string;
x : h1_h = { 1 => "a" } ;
```

The `hash of...` type can be assigned element-wise using subscripts:

```
x = h1[203];
h1[30] = "ad";
```

The `hash of...` type supports the member operations shown in Table 53.

Table 53 Hash Operations

Operation	e2 type	Return	Modifies e1	Comment
`e1.contains_key(e2)`	key type of hash		no	Returns true if e2 is a key to the e1 hash table. Effectively the same as `e1.keys().contains(e2)` but with possibly better runtime performance.
`e1.contains_value(e2)`	value type of hash		no	Returns true if e2 is a value for some entry in the e1 hash table. Effectively the same as `e1.values().contains(e2)` but with possibly better runtime performance.
`e1.empty()`	--	boolean	no	Returns true if the hash contains no elements; otherwise returns false.
`e1.keys()`	--	list of key type	no	Returns a list of all key values; the list does not have duplicate values.
`e1.size()`	--	integer	no	Returns the number of elements in the hash.
`e1.values()`	--	list of value type	no	Returns a list of all values; the list can contain duplicates, based on the content of the hash.
`e1.merge(e2)`	hash matching e1 type	hash	no	Returns a copy of the e1 hash with the keys and values of e2 concatenated to the end; any key that exists in e1 and e2 takes its value from e2.

Feedback

Table 53 Hash Operations (Continued)

Operation	e2 type	Return	Modifies e1	Comment
`e1.remove(e2)`	item	void	yes	Removes the e2 key from the e1 hash. There is no effect if the e2 key does not exist.

Here is another example:

```
alpha_h : newtype hash of string to integer;
alpha : alpha_h = {       // declare and initialize key => value pairs
        "a" => 1,
        "b" => 2,
        "c" => 3
        };
y = alpha["b"];          // The value of y is "2"
keyList = alpha.keys();  // A list of the keys in alpha
```

struct of...

Defines a distinct type identified by the user-supplied name. It must have at least one member and member names must be unique. The members can be of different types.

Here is an example structure that defines a point by its geometric coordinates:

```
point: newtype struct of { x: integer; y: integer; };
```

The `struct of...` type can contain initial values that are used as defaults for structure creation.

```
t3_s : newtype struct of {
   x: integer;
   y: integer = 76;   // initializer
   z: string = "hi";  // initializer
};
t4_s : newtype struct of {
   a: integer = 106;  // initializer
   b: string = "boo"; // initializer
};
```

The `struct of...` type can be assigned from a C-style initializer or from a list of name-value pairs. The following example shows assignment using the types defined in the previous example.

```
s3 : t3_s = { 10, z = "bye" };  // note that 'y' defaults
                                // to the value 76

// the defaults cause s4 to have a=106, b="boo"
s4 : t4_s = { };
```

Feedback

Here is another example.

```
shoebox : newtype struct of {
      style : string = "Tennis Shoe";
      shoe_size : double = 10.5;
   };

box : shoebox;
shoe = box.style;          // shoe is "Tennis shoe", the default
box.style = "Dress Shoe";  // Reassign the member box.style
// create a new shoebox with a different kind of shoe
box2 : shoebox = {"Boat Shoe", 9};
```

function

Allows a function to be passed to another user function. The function that is passed can be used as though it were a global function. For example,

```
// input file
#include <icv.rh>
xyzzy : function (void) returning void { note("abcd"); }
plugh : function ( f : function(void) returning void ) returning void {
f(); }
plugh(xyzzy);  // summary file shows "abcd" in output when plugh runs
```

Nested Types

Types can be combined to form nested types, such as

- list of struct
- list of enum

For example, using the shoebox struct definition in the example in struct of... on page 224:

```
box : list of shoebox = {
         { "Walking", 12 },
         { "High Heel", 7 }
      };

A = box[0].style;           // A is "Walking"
B = box[1].shoe_size;       // B is 7
```

Type Analysis

This section describes how PXL handles data types in special cases, such as when one data type is passed in an argument but another data type is expected.

In cases where PXL encounters one data type when another data type is expected, PXL performs automatic type conversion whenever appropriate. The rules that are used to perform automatic type conversion are discussed in the following sections.

Feedback

Standard Arithmetic Conversions

When there is a mismatch between the arithmetic type demanded by an expression and the arithmetic type found, conversions are done as shown in Table 54.

Table 54 Arithmetic Conversions

While expecting ...	Allow integer	Allow constraint of integer	Allow double	Allow constraint of double
integer	Yes[4]	No[5]	No	No
constraint of integer	Yes	Yes	No	No
double	Yes	No	Yes	No
constraint of double	Yes	Yes	Yes	Yes

These conversions are only applied when required by the types of the expression. In particular, integer-typed expressions are not converted to double unless required by the context.

When converting an integer or double to a constraint, the resultant constraint appears as if it had been written using the == unary constraint (CONSTRAINT_EQ).

```
f: function ( c: constraint ) returning void { ... }

f( (1.0, 26.0) );       // constraint is ok
f( 2.0 );               // single value is ok also
f( 42 );                // single integer is accepted
```

Comparing Doubles

Because the type double is an approximation of the real numbers, there are occasionally accumulated errors that cause apparently equivalent operations to produce different results. PXL has special functions for comparing doubles while accounting for such small errors. These special functions are described in the Equivalence Comparison Functions and Double Comparison Functions of the Math Functions section in the “Utility Functions” chapter of the *IC Validator Reference Manual*.

Note:

A warning is reported whenever there is a direct comparison (either == or !=) involving one of the double-related types.

4. *Yes indicates that the conversion is allowed.*
5. *No indicates that the conversion is not allowed. The operation resulted in an error.*

Feedback

Type Promotion

When a list is expected and an expression of the same type as that contained by the list is found, the expression is promoted from a single item to a list containing that item.

Parameter Passing

Function parameters that are qualified with either output or input-output must have matching argument values. The rules previously described do not apply. For example, when a function expects a constraint argument that is marked as out, providing the name of an integer-typed variable is not legal because the conversion that is required is not legal.

String Concatenation

Any expression of the types shown in Table 55 can be appended to a string expression using the standard addition operator (+).

Table 55 String Concatenation

Type	Resultant string appended
integer	Value of the integer expression
double	Value of the double expression
handle	Textual representation of the handle; unique for each distinct object
string	Content of the string
boolean	Either true or false
constraint of integer	Equivalent constraint literal expression
constraint of double	Equivalent constraint literal expression
user-defined enum	Name of the enum literal represented by the expression

When you are working with strings, remember

- When the left side of an addition operator (+) is a string, the result is a string.
- The addition operator (+) is not symmetric for string-valued expressions: a + b != b + a
- There are no spaces added between subsequent items appended to a string, so you might see unexpected results, such as in the following examples:

```
s : string;
s = "hello " + 10 + " " + (1 == 0);
// s now contains the string "hello 10 false"
```

Feedback

```
c : constraint = ( 10, 40 ];
s = "" + c;
// s now contains the string "(10.,40.]", because the
// type of "c" is constraint of double

mye : newtype enum of { A, B, C, D, X };
t : mye = X;
s = "" + t + A + B;
// s now contains the string "XAB"

s = "" + 11 + 3.14 + (1 == 1);
// s now contains the string "113.14true"

s = "" + (11 + 3.14) + (1 == 1);
// s now contains the string "14.14true"
```

Default Injection

When processing a function call, parameters for which no value is given assume the defaults, if any, from the function prototype. A parameter with no default or specified value results in an error.

Similarly, when processing a literal structure, members for which no value is given assume the defaults, if any, from the newtype structure definition. A member with no default or specified value results in an error.

Nested Containers

Lists, hashes, and structures are also known as containers. Containers can be nested in many different combinations. An example of a nested container is a list of structures.

Here is an example of a hash of a hash. The runset code is

```
hash1_h : newtype hash of string to integer;
hash2_h : newtype hash of string to hash1_h;

multi_hash : hash2_h = {};

multi_hash["A"] = {};          // Initialize hash of "A" to empty
multi_hash["A"]["ONE"] = 1;
multi_hash["A"]["TWO"] = 2;
multi_hash["B"] = {
   "THREE" => 3,
   "FOUR" => 4
};

foreach(key1 in multi_hash.keys() ){
   foreach(key2 in multi_hash[key1].keys()){
      note( "multi_hash[" + key1 + "][" + key2 + "] = "
            + multi_hash[key1][key2] + "\n");
```

Feedback

```
   }
}
```

The output is

```
runset.rs:123 multi_hash[A][ONE] = 1
runset.rs:123 multi_hash[A][TWO] = 2
runset.rs:123 multi_hash[B][FOUR] = 4
runset.rs:123 multi_hash[B][THREE] = 3
```

Here is an example of a list of a list. The runset code is

```
list1_l : newtype list of integer;
list2_l : newtype list of list1_l;

list2 : list2_l = {
   { 0, 1, 2, 3 },
   { 10, 11, 12, 13 }
   };

list2.push_back({ 20, 21, 22, 23 });
list2[1].push_back( 14 );

for( j=0 to (list2.size() - 1) ){
   for( i=0 to (list2[j].size() - 1) ){
      note( "list2[" + j + "][" + i + "] = " + list2[j][i] + "\n");
   }
}
```

The output is

```
runset.rs:143 list2[0][0] = 0
runset.rs:143 list2[0][1] = 1
runset.rs:143 list2[0][2] = 2
runset.rs:143 list2[0][3] = 3
runset.rs:143 list2[1][0] = 10
runset.rs:143 list2[1][1] = 11
runset.rs:143 list2[1][2] = 12
runset.rs:143 list2[1][3] = 13
runset.rs:143 list2[1][4] = 14
runset.rs:143 list2[2][0] = 20
runset.rs:143 list2[2][1] = 21
runset.rs:143 list2[2][2] = 22
runset.rs:143 list2[2][3] = 23
```

Here is another example of a list of a list. The runset code is

```
list3 : list of list of integer = {};

list3.push_back( { 0, 1, 2, 3 } );
list3.push_back( { 4, 5, 6 } );
list3.push_back( { 7, 8 } );
list3.push_back( { 9 } );
```

Feedback

```
for( j=0 to (list3.size() - 1) ){
   for( i=0 to (list3[j].size() - 1) ){
     note( "list3[" + j + "][" + i + "] = " + list3[j][i] + "\n");
   }
}
```

The output is

```
runset.rs:159 list3[0][0] = 0
runset.rs:159 list3[0][1] = 1
runset.rs:159 list3[0][2] = 2
runset.rs:159 list3[0][3] = 3
runset.rs:159 list3[1][0] = 4
runset.rs:159 list3[1][1] = 5
runset.rs:159 list3[1][2] = 6
runset.rs:159 list3[2][0] = 7
runset.rs:159 list3[2][1] = 8
runset.rs:159 list3[3][0] = 9
```

Flow Control

Flow can be controlled using

- Conditional Statements
- Loops
- Termination of Execution

Conditional Statements

Conditional statements in PXL are constructs that allow you to perform different actions depending on whether a specified condition evaluates to true or false.

The `if` statement is used in a program to perform an action only when a predefined condition is fulfilled.

The `if` statement is used in conjunction with at least two other components:

- An expression that can be evaluated to a Boolean value, true or false.
- A component that is executed if the evaluated expression returns a true value.

Feedback

Optionally, along with the required components, you can build a more complex conditional statement by including the following components:

- An `else` construct followed by a component that is executed if the first expression is evaluated to false.
- An `elseif` construct that indicates that a looped `if` condition or one or more `if` conditions are nested within a parent if conditional statement. PXL also supports the `elif` construct.
- The `if` statement uses the following syntax:

```
if (e1) s1;
if (e1) s1 elif (e2) s2;
if (e1) s1 elseif (e2) s2;
if (e1) s1 else s2;
```

The `if` statement is evaluated, as follows:

- The first Boolean expression (e1) is always evaluated first.
- The first component (s1) is processed if e1 evaluates to true.
- If the `else` statement is encountered, the statement that follows (s2) is executed.
- If `elif` or `elseif` statements are encountered, the statements following those are executed.

Loops

As in other programming languages, loops can be used in PXL to repeat an action a number of times.

PXL provides the following methods to create looping constructs in programs:

- for Loop
- foreach Loop
- while Loop

for Loop

The `for` loop is used to repeat a particular action a specified number of times. It consists of at least one integer (e1) and an action (s1) that must be repeated. It could also contain additional integers (e2, e3).

It uses the syntax

```
for (var = e1 to e2) s1;          //s1 is a statement; it can be compound
for (var = e1 to e2 step e3) s1;
```

Feedback

- Requires e1, e2, and e3 to be integer-valued expressions.
- Defines var as an integer variable visible only within statement s1.
- Evaluates e1, e2, and e3 one time at the beginning of the loop; subsequent assignments that affect the values of these expressions have no effect on looping.

For example,

```
for(i=0 to 10 step 2){              // Increment by 2: 0, 2, 4, 6, . . .
   if( i < 6) {
      a[i] = b[i] + c[i];
   } elif( ( i >=6 ) && ( i < 10)){
      a[i] = b[i] + c[i] + 1;
   } else {
      a[i] = b[i] - c[i];
   }
};
```

foreach Loop

The `foreach` loop is similar to a `for` loop but it is used to perform a set of actions with reference to the elements of a container, such as a list or an enum. It does not use a variable counter to determine the number of iterations of the action.

It consists of a list of type (e1) and a component (s1) that is evaluated for each element in e1.

It uses the syntax

```
foreach (var in e1) s1;  //e1 is of the type "list of"
```

- s1 is evaluated one time for each element of e1, with var copied from the list.
- s1 is bypassed if the input list e1 is empty.
- var is a constant variable of the correct type visible only within statement s1.

while Loop

The `while` loop is used to repeat a particular action until a condition is met. It consists of a Boolean expression (e1) and an action (s1) that must be repeated. The while loop always evaluates the Boolean expression e1 at least one time.

It uses the syntax

```
while (e1) s1;  // e1 is evaluated as a Boolean;
                // loop continues until e1 is false
```

Feedback

The `while` statement is evaluated, as follows:

- Each time e1 evaluates to true, s1 is executed.
- When e1 evaluates to false, the statement ends.

Termination of Execution

The error(“message”) is a Diagnostics Function. It writes the provided message and then terminates execution. For example:

```
#ifdef ABC
#ifdef DEF
error("ABC and DEF are mutually exclusive. Do not set both.");
#endif
#endif
```

Functions

Functions in PXL are categorized based on their usage. The function types are described in Table 56.

Table 56 Function Types

Function type	Description
Runset function	Standard IC Validator published function.
Remote function	Function written by the user that can be called by runset functions. There are also default remote functions supplied by the IC Validator tool.
User function	Function that is written by a user.
Utility function	Function supplied by the IC Validator tool that provides access to primitive data in a remote function.

Function Definitions

A function definition defines a set of operations associated with a name. A function definition identifies the names, types, and defaults for all parameters and for a returning value.

You can create your own functions and use them along with the provided functions within your runsets.

As is shown in the following example, all function definitions must be at the top level.

Feedback

```
// a simple function definition

myinversion: function ( arg: double ) returning rv : double
{
   rv = 1.0 / arg;
}

x = myinversion(2.0);  // simple function call
```

Function definitions provide a separate scope for all processing. Scoping is determined in the context of where things are written (static scoping). For information about scope see Scope of Variables on page 206.

Inside of a function definition these scoping rules apply:

1. If a declaration cannot be found at the current scope, higher scopes are searched until the entire function definition has been searched, stopping at a match.
2. Then, if necessary, a global variable that is declared before the function definition and has the same name is used.
3. Lastly, if the variable is being assigned to a function call or is in the position of an output parameter to a function call, it is implicitly declared in the scope of the current statement. However, if the variable is in the position of an input or input-output parameter to a function call even if it is being assigned to a function call, it is not implicitly declared in the scope of the current statement.

Outside of a function definition these scoping rules apply:

1. If a declaration cannot be found at the current scope, higher scopes are searched until the entire runset has been searched, stopping at a match.
2. Then, if necessary, a global variable that is previously declared and has the same name is used.
3. Lastly, if the variable is being assigned to a function call or is in the position of an output parameter to a function call, it is implicitly declared in the scope of the current statement. However, if the variable is in the position of an input or input-output parameter to a function call even if it is being assigned to a function call, it is not implicitly declared in the scope of the current statement.

Function Prototypes

A function prototype defines the parameters to a function and defaults for optional parameters. Function prototypes identify call-by-reference parameters. Function prototypes do not define the operation of the function.

Feedback

The following example is of a function prototype:

```
my_xor : function (
   a : polygon_layer,
   b : polygon_layer
) returning xorOut: polygon_layer;
```

The following example shows a function declaration.

```
my_xor : function (
   a : polygon_layer,
   b : polygon_layer
) returning xorOut: polygon_layer {
      tmp1 = not(a,b);
      tmp2 = not(b,a);
      xorOut = or(tmp1, tmp2 );
   };
```

The following example shows calling the `my_xor()` function, which was defined in the preceding example.

```
a1 : polygon_layer;
b1 : polygon_layer;
xorA1B1 = my_xor(a1, b1);
```

Function Calls

Function calls define how functions operate. A function call consists of the name of the function followed by instructions on how and where the function must be executed through arguments.

Values can be passed to the arguments in a function call in two different styles:

- By argument value alone (pass-by-value). The order of values must match the order of arguments in the function prototype. When you use this style, you must specify values for all optional arguments preceding an optional argument that is of interest to you because values are matched to arguments based on order.
- By name in `argument_name = argument_value` format (pass-by-name). The order of arguments is irrelevant. When you use this style, you do not need to specify values for all optional arguments preceding the optional argument that is of interest to you because values are matched to arguments based on name.

Both styles can be used in a single function call. However, in a function call, all argument values specified with the pass-by-value style must precede all argument values specified with the pass-by-name style. After you use the pass-by-name style in a function call, all following argument calls must be in the same style.

Feedback

For example, the syntax of the `interacting()` function is:

```
interacting : binary published function(
   layer1 : polygon_layer,
   layer2 : polygon_layer,
   count : count_constraint_t = >0,
   include_touch : more_include_touch_e = EDGE,
   processing_mode : processing_mode_e = HIERARCHICAL
) returning result : polygon_layer;
```

The following is an example of calling the `interacting()` function with the pass-by-name style:

```
y = interacting( layer1 = a, layer2 = b, include_touch = ALL );
   // The count argument is skipped, and the value is the default.
   // The value of the processing_mode argument is the default.
```

The following is an example of calling the `interacting()` function with the pass-by-value style:

```
y = interacting( a, b, 2 );
   // The value of count is 2.
   // The values of the include_touch and processing_mode arguments
   // are the defaults.
```

Here are two examples of calling the `interacting()` function with mixed styles:

```
y = interacting( a, b, include_touch = ALL );
   // The count argument is skipped, and the value is the default.
   // The value of the processing_mode argument is the default.

y = interacting( a, b, count = 2);
   // The values of the include_touch and processing_mode arguments
   // are the defaults.
```

There are two styles of function calls:

- Standard Function Call

  ```
  y = and(a, b);
  y = and(a, b, CELL_LEVEL);
  y = or(not(a,b),not(b,a));
  ```

- Operator-Style Function Call

  ```
  y = a and b;
  y = a and[CELL_LEVEL] b;
  y = (a not b) or (b not a);
  ```

Standard Function Call

A standard function call is always allowed for any function.

Feedback

A standard function call consists of the name of the function followed by a parenthesized and comma-separated list of at least one expression, followed by a list of at least one name-value pair.

In the following example, the function has three parameters.

```
g: function ( x : integer, y : integer, z : integer)
   returning void;
```

Of the four function calls listed here, one is not legal:

```
g(x=3, y=10, z=10);   // allowed

g(z=3, y=10, x=4);    // allowed

g(10, 20, 30);        // allowed; x=10, y=20, z=30

g(z=3, 10, 20);       // ILLEGAL - expressions must not
                      // follow name-value pairs
```

If defaults are present for any parameters, the parameters are not required to be specified.

In the following example, the function has three parameters, two of which have defaults.

```
f: function
   ( x : integer,
     y : integer = 27,
     z : integer = 42
   ) returning void { ... }
...
```

Of the five function calls listed here, two are illegal:

```
f(10, y=2);    // allowed, z is 42

f(10);         // allowed, y is 27, z is 42

f(z=3, x=4);   // allowed, y is 27

f();           // ILLEGAL - x requires a value

f(z=20);       // ILLEGAL - x requires a value
```

Operator-Style Function Call

An operator-style function call requires arguments to the left and right of the function name for binary functions. It requires other arguments to be specified as name-value pairs in square brackets immediately following the name of the function.

Operator-style function calls are allowed only for functions declared or defined with the binary qualifiers. Using operator-style function calls helps prevent the use of the function name in other contexts, such as variable names and member names.

Feedback

The following syntax defines how a function can be called by using the binary qualifier:

```
length: binary function (l1: layer, c1: constraint of double) returning
result: layer {...}
...
LONGMET = MET1 length > 5;
```

With careful interface definition, binary allows for easy-to-read programs. For example,

```
// A simple declaration that takes two arguments.
// The LHS is the layer to be selected from; the RHS is the
// constraint defining the criteria area.
...
// ... as is a variable of appropriate type.
c : constraint = (27.0, 42.5);
x : lyr1 area c;
```

Argument Bindings

Argument bindings determine which arguments at a call site are bound to which parameters in the function prototype.

- Select candidate prototypes based on the name of the function and the type required by the context. If the type cannot be determined, use only the function name. When an operator-style function call is made, the external arguments are bound to parameters in the prototype. For binary functions, the left argument is bound to the first parameter, and the right argument is bound to the second parameter.
- Each name-value argument is bound to the remaining like-named parameter.
- Any arguments that do not have names are bound to the remaining undefaulted parameters from left to right.
- If all remaining unbound parameters have defaults, the function is used.
- It is an error if argument binding fails to match exactly one function prototype.

For example, given the function MyFunc defined, as follows:

```
MyFunc: binary function (
   fromLayer: lhs layer,
   touching: rhs layer,
   corners: boolean = true,
   whole_polygon: boolean
) returning e: edge_layer;
```

The following function calls are all equivalent:

```
T1 = MyFunc (metalLayer, selectLayer, true);
T2 = MyFunc (metalLayer, selectLayer, whole_polygon=true);
T3 = metalLayer MyFunc[whole_polygon=true] selectLayer;
```

Feedback

The following code produces a compile-time error because the `whole_polygon` argument does not have a default:

```
/* illegal function call */
Tbroken = metalLayer MyFunc selectLayer;
```

Defaults

Defaults are specified as right side assignments to option variables in the function prototypes. For example, here is the syntax of the `and()` function:

```
and : function(
   layer1 : polygon_layer,
   layer2 : polygon_layer,
   processing_mode : processing_mode_e = HIERARCHICAL,
   remove_hierarchical_overlap : boolean = true
) returning and_result : polygon_layer;
```

The following call of the `and()` function uses the defaults for the `processing_mode` and `remove_hierarchical_overlap` arguments.

```
c = and( a, b );
```

The following call of the `and()` function uses an optional setting for the `processing_mode` argument and the default for the `remove_hierarchical_overlap` argument.

```
c = and( a, b, CELL_LEVEL );
```

Resetting Defaults

You can override the default of any function argument. Any argument can have its default changed; it can be numerical, a Boolean value, or an enumerator.

In this example, the default for the argument x of the abc function is set:

```
abc : literal function ( x : double = 123) returning z : double;
A = abc();   // == abc(123)
```

Caution:

When you reset defaults, you are redefining the function, and all of its arguments must be listed.

Anonymous Functions

Several IC Validator commands allow a user function to be passed in as an argument. Anonymous functions provide a mechanism for simple behavior or as part of organizing more complex behavior. You can use anonymous functions where function names are expected. The compiler generates an internal name and declares the function at the location in the code immediately preceding its invocation.

Feedback

Traditionally, global variables can be used and set to different values for different calls depending on the need. For example:

```
x : integer;
y : integer;
calc_nmos_props : function (void) returning void { … }

x = 0;
y = 5;
nmos(…, calc_nmos_props, …);

x = 1;
y = 4;
nmos(…, calc_nmos_props, …);
```

However, anonymous functions allow this code to be more compact and well organized. In the following example, a hidden function is declared just before each of the NMOS calls which then use it.

```
calc_nmos_props : function (x : integer, y : integer) returning void
 { … }

nmos(…, function { calc_nmos_props(0, 5); }, …);
nmos(…, function { calc_nmos_props(1, 4); }, …);
```

Function Overloading

PXL allows you to have more than one function with the same name in some cases.

Multiple functions can have the same name if PXL can determine which function is to be called by using the types and parameter names. If PXL is not able to determine the function to be called, it results in an error.

You can have two different functions with the same name if one returns a value and the other returns a void.

When a function appears at the statement level, it is assumed to be the void-returning function; when it appears in an expression, it is assumed to be the value-returning function:

```
MyFunction(a,b,c);       // use the void-returning function
                         // "MyFunction"
x = MyFunction(a,b,c);   // use the value-returning function
                         // "MyFunction"
```

Note:

PXL only supports function overloading based on different return types.

Feedback

Function Recursion

Recursive function calling is not supported in PXL. For example:

```
noCanDo : function( … ) returning integer {
  …
  x = noCanDo(…);
  …
}
```

Program Structure

A PXL program is a sequence of declarations, function definitions, and statements. A program is executed from top to bottom.

PXL sets these preconditions for a program to be valid:

- All objects must be defined before they are used.
- All statements are evaluated only for side effects. An expression must be part of an assignment.

  ```
  10 + 5;  // illegal because the result (15) is not captured
  ```

Compound Statement

A compound statement defines a nested scope in which local variables can be defined. Compound statements can be used anywhere a single statement is allowed. For information about scope, see Scope of Variables on page 206.

- Compound statements allow local variables to hide previously defined variables at other scoping levels. The compiler reports a warning. Hiding is also known as shadowing.
- Compound statements use braces ({ }) to identify the compound statement.

  ```
  { ; }  // Smallest legal compound statement
  ```

- Compound statements follow the following syntax:

  ```
  {
     s1;
     s2;
     ...
     sN;
  }
  ```

Feedback

Violations

Some IC Validator functions can produce violation data, which is also referred to as error output, instead of or in addition to layer output. These violations are stored in an SQLite database (the error database, PYDB), which is available to VUE for debugging, and written to the LAYOUT_ERRORS file at the end of the run. There are also some PXL functions that take violation data as input, such as the `write_gds()` and `violation_empty()` functions.

If you want to directly access the error data produced by the IC Validator tool, see Appendix E, PYDB Perl API for information about using the IC Validator Perl application programming interface (API).

Violation Comments

All violation output is associated with a comment string. This comment appears as a heading in the LAYOUT_ERRORS file as well as in VUE. Any number of functions can output to the same violation comment.

The violation comment for current and nested PXL scopes is specified by using the `@` operator. If no comment is specified in the runset, violations are given a default comment of "Violation".

The following example shows how to program violation comments that are unique for each layer in a set of layers. The violations are named “Metal1 spacing <= 0.1um”, “Metal2 spacing <= 0.1um”, and so forth for each layer.

```
for (i=0 to metal_layers.size()-1) {
   @ "Metal" + (i+1) + " spacing <= 0.1um";
   external1(metal_layers[i], distance<=0.1);
}
```

The following example shows the behavior of violation comments with respect to scope.

Note:

Typically, the violation comment definition (`@` operator) should not be separated from the functions to which it applies. The following example highlights the behavior of violation comments with respect to scope.

```
@ "First Error Message";
{ @ "Second Error Message";             // A new scope is created here
  local_var = abc(layer2);
  internal1(local_var, distance < 0.2); // Errors from internal1()
                                        // are stored in the database
                                        // under "Second Error Message"
}  // The original scope is restored here

external1(layer1, distance < 0.1); // Errors from external1() are stored
```

Feedback

```
                                        // in the database under "First Error
                                        // Message"
```

The following example shows how to program violation comments that are unique for each layer in a set of layers. The violations are named “Metal1 spacing <= 0.1um”, “Metal2 spacing <= 0.1um”, and so forth for each layer.

```
for (i=0 to metal_layers.size()-1) {
   @ "Metal" + (i+1) + " spacing <= 0.1um";
   external1(metal_layers[i], distance<=0.1);
}
```

The `text_net()`, `texted_with()`, and `not_texted_with()` functions can output several different types of violations. For these functions, there is a way to override the current runset violation comment for individual types of errors. In this example, text shorts are output to the violation comment “Rule 4.5.6: No Shorts”, while all other types of text errors are output to “Rule 1.2.3: No text errors”.

```
@ "Rule 1.2.3: No text errors";

cdb1 = text_net(cdb1,
                { { M1, M1_text } },
                shorted_violation_comment = "Rule 4.5.6: No shorts");
```

The IC Validator tool processes violation comments using command-line options such as, `-svc`. For example:

```
{@ "A";
  {@ "B";
    internal1(…);
  };
};
```

The tool considers only the lower violation comment for selection. In this case, setting either `-svc B` or `-uvc A` executes the `internal1()` function, however, setting `-svc A` or `-uvc B` does not.

Violation-Producing Functions

Most DRC and data generation functions are overloaded; that is, they have two versions. The version that returns a polygon layer does not produce error output. The void-returning version produces error output but not a polygon layer. Here are examples of the two versions of the `external1()` function.

```
// This external1 outputs to a polygon layer, no errors are produced
t1 = external1(layer1, distance < 0.1);

// This external1 outputs error data to the comment "Ext1 Check"
@ "Ext1 Check";
external1(layer1, distance < 0.1);
```

Feedback

There are also some PXL functions that do not return void, but can also produce error output. For example, the `text_net()` function returns a connect database but also produces error output for text shorts, opens, and unused text.

Violation Variables

The built-in PXL type violation can be used for variables to hold error data if there is a need to use it as input to another PXL function later in the runset. For example, violation variables can be used as input to functions such as `write_gds()` to output error polygons, or `violation_empty()` to determine if the violation contains any errors.

As the following example shows, some PXL functions return violations directly:

```
// These functions return violations for which there is not an associated
// PXL function
v1 = get_layout_grid_violation();
v2 = get_layout_drawn_violation();

// Write errors to GDSII
g = gds_library("err.gds");
write_gds(output_library = g,
          errors = { { v1, { 1 } },
                     { v2, { 2 } } });
```

Note:

Error output can be restricted by arguments such as `error_limit_per_check` in the `error_options()` function. Those options can influence how much data is actually stored in the error database. Discarded errors are not available for `write_gds()` and other functions.

Violation Blocks

Violation variables can also be assigned by using violation blocks. Violation blocks are specified by using the `@=` operator. The left side is a violation variable. A code block enclosed by curly braces follows the `@=` operator. Within this block, all functions which produce error output are associated with the violation variable. For example,

```
// v1 contains the error output from the external1() check
v1 @= {
   @ "Rule 1";
   l1 = a and b not c;
   external1(l1, distance < 2);
}

// v2 contains the error output from both internal1() checks and
// the external1() check
v2 @= {
   @ "Rule 2";
   internal1(l2, distance < 1);
   internal1(l3, distance < 1);
```

Feedback

```
   @ "Rule 3";
   external1(l2, distance < 1);
}

tdb : connect_database; // Declared outside the violation block scope
                        // so it can be used after the violation block
// v3 contains all text errors found by text_net()
v3 @= {
   @ "Rule 4";
   tdb = text_net(cdb, {{ M1, M1_text }});
}

// These functions can be used to get only certain types of errors
// from a violation. The result is a subset of the input violation.
v4 = get_text_shorted(v3); // v4 contains only text shorts from v3
v5 = get_text_unused(v3);  // v5 contains only unused text from v3

// Take some action if v4 is not empty
if (! violation_empty(v4)) {
   note("Found text shorts!");
}
```

Execution Model

The execution model for PXL demands that resources be available. Any operation in a particular program is available for execution at any time if its preconditions are satisfied. There is a strong bias to execute diagnostics early, meaning that all the diagnostics that are ready are executed immediately during the initial processing of the program if possible. Therefore, you might see out-of-order execution of statements in a PXL program. The final results are unaffected by this reordering; however, intermediate results can appear at different times or simultaneously. This out-of-order execution is also known as parallel execution.

The same program, when executed different times, always generates the same results. These results can appear in a different order, however, based on availability of resources.

Possible runtime errors are

- Attempting to use any variable that has not been assigned a value.
- Indexing beyond the end of a list or with a negative index.
- Reading from a hash with a key value for which there is no element.

Preprocessor

PXL uses a preprocessor to make the development and execution of programs easier and faster. A standard preprocessor module is integrated into the PXL compiler. Table 57 shows the preprocessor capabilities.

Feedback

Note:

The line length limit for the preprocessor is 64K characters. Exceeding this limit can usually be worked around by inserting a carriage return in the place of white space or a delimiter, such as a comma (,).

Table 57 Standard Preprocessor Modules

Typical preprocess capability	Syntax used
Conditional compilation	`#if . . . #ifdef`, `#ifndef`, `#elif`, `#else`
Error generation	`#error`

File inclusion	`#include` is used to insert the contents of additional files into the runset. The file name must be enclosed in either double quotation marks or angle brackets. The syntax controls the search path.
	Use angle brackets to include IC Validator files:
	`#include <icv.rh>`
	Directories are searched in this order:
	1. Directories specified on the command line with the `-I` option, in the order in which they are specified
	2. *ICV_INSTALL_DIRECTORY*/include
	Use double quotation marks to include your own files:
	`#include "my_include.rh"`
	Directories are searched in this order:
	1. The same directory as the file with the #include statement
	2. Directories specified on the command line with the by the `-I` option, in the order in which they are specified
	3. *ICV_INSTALL_DIRECTORY*/include
	When an absolute path is specified, only that path is searched.
	`#pragma once` can be used to prevent a file from being included more than one time. Any file containing `#pragma once` is only included one time, even if it appears in multiple `#include` statements.
	For example, if the runset directory structure looks like this:
	`Runset`
	`|-- rules.rs`
	`|-- Include`
	`|-- assign.rs`
	`|-- other_assign.rs`
	To include `other_assign.rs` in `rules.rs`, use
	`#include "Include/other_assign.rs"`
	To include `other_assign.rs` in `assign.rs`, use
	`#include "other_assign.rs"`
	See IC Validator Command-Line Options on page 25.
	See the *IC Validator Installation Notes* available on SolvNetPlus for more information about the IC Validator home directory.

Feedback

Table 57 Standard Preprocessor Modules (Continued)

Typical preprocess capability	Syntax used
Location tracking	`#line`
Standard macro support	`#define . . . #undef`, including token construction (macro body `#` binary operator) and stringification (macro body `##` unary operator). You can also use the `-D` command-line option to define variables. See IC Validator Command-Line Options on page 25.

Use preprocessor directives to remove large blocks of code; that is, to enclose the section within a pair of `#if 0` and `#endif` statements. For example,

```
#if 0
// everything through the matching #endif is treated as a comment.
if (something == true)
{
   a line comment.
   do_something(useful = true);
}
#endif
```

Here is an example of using directives to enforce one time inclusion. The first time this file is encountered MYFILE_RH is not defined. The included #define MYFILE_RH sets this variable to be defined and the code body is included. The code body is never included again because MYFILE_RH exists and the #ifndef evaluates to false.

```
#ifndef MYFILE_RH
#define MYFILE_RH

// Body of code to include goes here

#endif   // end if the #ifndef MYFILE_RH
```

When using #if statements,

- Use standard comparison operators:

  ==, <=, >=, &&, ||

- Macros that are not defined are interpreted as 0 (zero). For example, if SOMEMACRO is *not* defined, then `#if SOMEMACRO == 1` is translated as `#if 0 == 1`.
- Only compare integer values. Strings cannot be compared.

Predefined Identifiers

During preprocessing, the predefined identifiers shown in Table 58 are supported.

Feedback

Table 58 Predefined Identifiers

Identifier	Content
__LINE__	Line number in the current source file as an integer
__FILE__	Name of the current source file, as a string constant
__DATE__	Date of compile, as an 11-character string literal, for example, “Apr 25 2006”
__TIME__	Time of compile as a string literal (“hh:mm:ss”)
__PXL_VERSION_MAJOR__	Major version of the language as an integer
__PXL_VERSION_MINOR__	Minor version of the language as an integer
__PXL_VERSION_PATCH__	Patch level of the language as an integer
__PXL_DATE__	Date of compile, as an eight-digit integer constant as yyyymmdd, for example, 20060425
__PXL_TIME__	Time of compile, as a four-digit integer constant as hhmm, for example, 0014 or 2300

For example, __PXL_DATE__ and __PXL_TIME__ can be used to make code that changes behavior based on the current date. Because these variables are preprocessor variables, they can be used in #if and #define statements as well.

```
/**
 * define a new token, expire_20061231, that is be replaced
 * by the token deprecated before 2006.12.31 but expands
 * to obsolete thereafter.
 */
#if __PXL_DATE__ < 20061231
#define expire_20061231 deprecated
#endif

#ifndef expire_20061231
#define expire_20061231 obsolete
#endif
```

Release Version Macros

Published macros allow you to compare the current release version to other release versions.

For example, you can determine if the current release version is at least as recent as the release version specified with integer values for year, month, service pack, and hot fix.

Feedback

```
#if VERSION_GE(2009, 6, 1, 0)
. . .       // code that uses new features introduced in 2009.06.SP01
#else
. . .       // code that uses older method (pre 2006.06.SP01)
#endif
```

The VERSION macros are defined in Table 59.

Note:

The *year*, *month*, *service_pack*, and *patch* are integer values.

Table 59 VERSION Macros

Macro	Definition
`VERSION_GE(year, month, service_pack, patch)`	Tests to see if the current release version is greater than or equal to the specified release version.
`VERSION_LE(year, month, service_pack, patch)`	Tests to see if the current release version is less than or equal to the specified release version.
`VERSION_GT(year, month, service_pack, patch)`	Tests to see if the current release version is greater than the specified release version.
`VERSION_LT(year, month, service_pack, patch)`	Tests to see if the current release version is less than the specified release version.
`VERSION_EQ(year, month, service_pack, patch)`	Tests to see if the current release version is equal to the specified release version.

Date Comparison Macros

Date comparison macros allow you to change the behavior of a runset based on the current date.

For example, you can use the new macros to enable or disable certain PXL code based on the current date.

```
if (DATE_EQ(2015, 6, 4)) {
    note("Today == 2015-06-04");
} else {
    note("Today != 2015-06-04");
}
```

The DATE macros are defined in Table 60.

Note:

The *year*, *month*, and *day* are integer values.

Feedback

Specifying invalid or out-of-range values, such as a day value greater than 31 or a month value greater than 12, results in undefined behavior.

Table 60 DATE Macros

Macro	Definition
`DATE_EQ(year, month, day)`	Tests to see if the current date is equal to the date specified by the *year*, *month*, and *day* integers.
`DATE_GE(year, month, day)`	Tests to see if the current date is greater than or equal to the date specified by the *year*, *month*, and *day* integers.
`DATE_GT(year, month, day)`	Tests to see if the current date is greater than the date specified by the *year*, *month*, and *day* integers.
`DATE_LE(year, month, day)`	Tests to see if the current date is less than or equal to the date specified by the *year*, *month*, and *day* integers.
`DATE_LT(year, month, day)`	Tests to see if the current date is less than the date specified by the *year*, *month*, and *day* integers.

Qualifiers

A qualifier affects an item it marks by changing its behavior or the operations that are allowed on it. Table 61 defines the qualifiers.

Table 61 Qualifiers

Qualifier	Definition
const	The object causes an error each time it is assigned a new value. Initialization does not cause this error. Only attempts at direct assignment or passing through a function parameter qualified with out or in_out.
deprecated	The object causes a warning each time that it is defined or referenced. A function parameter being assigned an initial value in the function prototype or being used within the body of the function of which it is a parameter does not generate this warning.
in_out	The function parameter requires an assignable item (an lvalue) in each function call. A nonassignable item or an item of the wrong type causes an error. The value of that item is also changed as a result of the function call. See Assignment Operators on page 208 for more information about lvalue.
inline	Tags a function as requiring expansion in an IC Validator run using the `-ndg` command-line option. See IC Validator Command-Line Options on page 25.

Feedback

Table 61 Qualifiers (Continued)

Qualifier	Definition
literal	The function parameter requires that the function argument be a literal expression. That is, the compiler must be able to resolve the value of the function argument at parse time.
obsolete	The item causes an error each time it is defined or referenced. A function parameter being assigned an initial value in the function prototype does not generate this error.
out	The function parameter requires an assignable item (an lvalue) in each function call. A nonassignable item or an item of the wrong type causes an error. This parameter is assumed to have no value when the function body is processed. The value of that item is also changed as a result of the function call. See Assignment Operators on page 208 for more information about lvalue.

A qualifier can be used to qualify only relevant items. The qualifiers are shown in Table 62.

Table 62 Qualifier Items

Qualifier	Variable	Function	Parameter	Struct member
binary		yes		
const	yes		yes	
deprecated	yes	yes	yes	yes
in_out			yes	
inline		yes		
literal			yes	
obsolete	yes	yes	yes	yes
out			yes	

Example of Modularized Code

- function.rh

```
// Use pragma once to prevent including
// this file multiple times
#pragma once

#include <icv.rh>
```

Feedback

```
// Function prototypes
fx1 : function (
   lyr : polygon_layer,
   w   : double
) returning out: polygon_layer;

fx2 : function (
   lyr : polygon_layer,
   d   : double
) returning out: polygon_layer;

#include <functions.rs>
```

- function.rs

```
// Function definitions
fx1 : function (
   lyr : polygon_layer,
   w   : double
   ) returning out: polygon_layer {
      // code body
      out = f( lyr, w);
   };

fx2 : function (
   lyr : polygon_layer,
   d   : double
   ) returning out: polygon_layer {
      // code body
      out = f( lyr, d);
   };
```

- main.rs

```
#include <icv.rh>
#include <function.rh>

// assign(), library(), etc.

y = fx1( a, 0.1);
z = fx2( b, 0.3);
```

10

Dynamic-Link Library Support

This chapter explains using the dynamic-link feature of the IC Validator tool.

Use the dynamic-link feature specifically for device extraction. This feature allows you to pass measurement data to C libraries that are external to the IC Validator tool, and then return the results from the C libraries back to the IC Validator tool. The C libraries must be shared (dynamic) libraries.

Dynamic-link library support is described in the following sections:

- Dynamic-Link Functions
- Using the Dynamic-Link Functions

Dynamic-Link Functions

The following IC Validator functions support the dynamic-link library feature:

- `dev_dlink_library_close()` function
- `dev_dlink_library_open()` function
- `dev_dlink()` utility function
- `dev_dlink_library()` utility function
- `dlink_libraries` argument of all device configuration functions, such as the `capacitor()` function.

Feedback

Using the Dynamic-Link Functions

To use dynamic linking:

1. In your IC Validator runset,
   a. Open the C library using the `dev_dlink_library_open()` function.
   b. Specify the library handles in the IC Validator runset using the `dlink_libraries` argument in the device configuration functions.
   c. Close the library using the `dev_dlink_library_close()` function.

   For example,

```
dlink_handle = dev_dlink_library_open("library_name");

nmos(
   property_function=my_mos_func,
   dlink_libraries = {dlink_handle}
);

dev_dlink_library_close(dlink_handle);
```

2. Define the calculations for the device measurement data using the utility functions. The dynamic-link remote function is typically organized like this:

```
my_mos_func : function(void) returning void {
   //(1) Get measurement data for the device using the
   //    device utility functions
   //(2) Encode the measurement data into a one-dimensional array
   //(3) Retrieve dynamic-link library file handle using the
   //    dev_dlink_library() utility function
   //(4) Call user-defined dynamic-link function using the dev_dlink()
   //    utility function, and pass essential information to the
   //    external shared libraries
         IC Validator data
}
```

3. The user-defined wrapper function, written in the C language, has this typical flow:

```
customer_wrapper_function() {
   //(1) Decode the measurement data into customers' format
   //(2) Decide function pointer according to string parameter
   //(3) Call internal function
   //(4) Return computed results
}
```

11

Unified Fill

Using the functions and PXL, the programming language of the IC Validator tool, you can create both simple and complex procedures to meet your fill requirements. This chapter describes the basics of how to use the `unified_fill()` *function, an application-oriented, fill insertion function. This chapter also shows several usage models for runsets using the unified fill feature.*

Note:

The `Unified Fill` function is described in the *IC Validator Reference Manual*.

A fill pattern defined in the `unified_fill()` function can have either a single fill layer or multiple fill layers. That is, the function can output more than one polygon layer. Each fill-layer definition can have a single rectangle or any number of nonrectangular polygons. Additionally, every pattern can be viewed as a structure that is repeated inside the target region as defined by the spacing constraints.

The unified fill feature is described in the following sections:

- Overview
- Types of Fill Patterns
- Fill-to-Fill Spacing
- Target Region and Fill-to-Signal Spacing
- Range and Width Dependent Spacing
- Polygon Grouping
- Signal Aligned Fills
- Improving Pattern Insertion
- Pitch Aligned Fills
- Criteria Analysis
- Output Layers
- Layer Compression

Feedback

Overview

The `unified_fill()` function provides a high level of automation and flexibility in developing a fill procedure. The fill requirement might be as simple as generating an array of rectangular patterns inside a layer, or as complex as filling a design with a variety of patterns to meet a certain density target.

The `unified_fill()` function supports many options that you can configure in different ways for the required fill requirement. Some of the high level features of this function are

- Ability to define different types of patterns
- Ability to define fill insertion sequences that use fill cells read from an external database
- Equation-based criteria analysis
- Layer coloring
- Signal-aligned fills
- Automated target layer creation

Table 63 defines the terms used for the unified fill feature.

Table 63 Unified Fill Terminology

Term	Definition
Design layer and Signal layer	Existing layer in the design
Fill cell	Fill pattern definition that is imported to an IC Validator runset from an external layout database
Fill layer	Polygon layer output generated by the `unified_fill()` function
Target region and Fillable region	Region where fill can be added
Keepout region and Exclusion region	Region where fill must not be added
Fill pattern and Fill structure	Fill configuration composed of one or more fill layers. A pattern can be defined as either • A single fill layer • Multiple fill layers

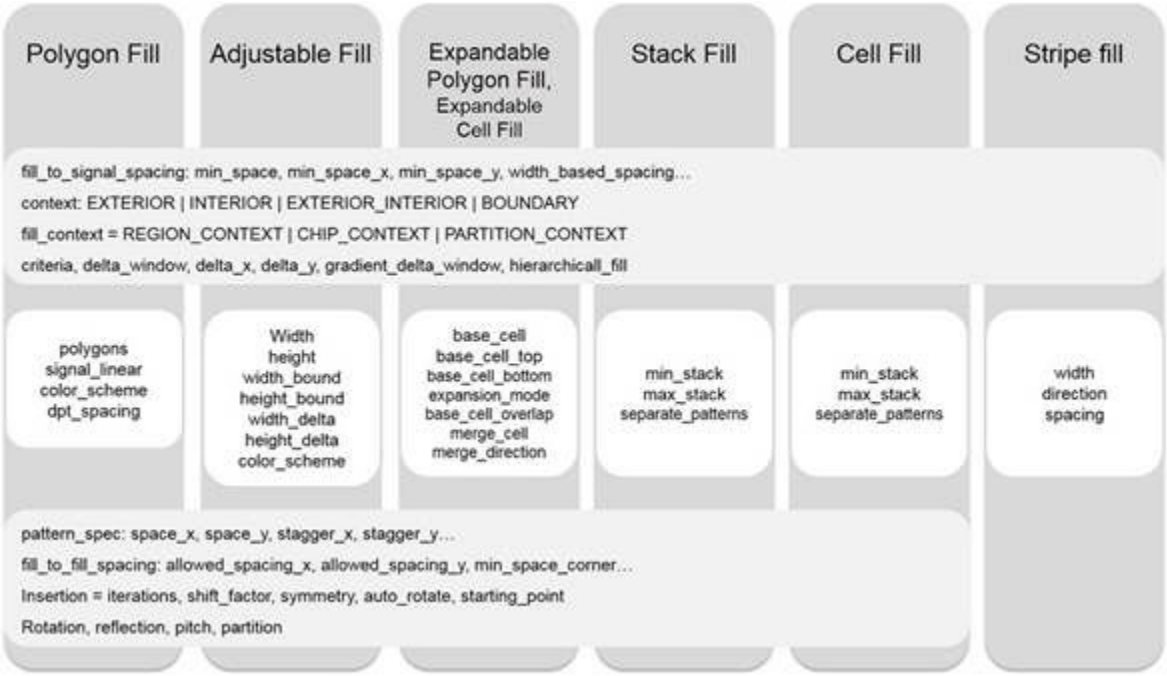
Feedback

Table 63 Unified Fill Terminology (Continued)

Term	Definition
Fill-to-signal spacing	Spacing from the fill pattern, or from the fill layer, to the design layer
Fill-to-fill spacing	Spacing between fill patterns or between fill layers

Figure 56 summarizes the features that are available for each type of fill pattern.

Figure 56 Fill Pattern Overview



Types of Fill Patterns

The `unified_fill()` function supports the following types of fill pattern definitions:

- Polygon Fill Patterns
- Adjustable Rectangle Fill Patterns
- Stack Fill Patterns
- Cell Fill Patterns

Feedback

- Expandable Fill Cell Patterns
- Stripe Fill Patterns

Polygon Fill Patterns

A polygon fill pattern is a coordinate based definition of a fill pattern. For this type of fill pattern, set the `type` option to `UF_POLYGON`.

The fill pattern can have any number of fill layers. For a multilayer pattern definition, each fill layer has its own layer definition. As shown in Example 2, a fill layer is identified with the name specified in the `output_layer_key` option of the `layers` option.

Example 2 Single Layer Pattern Definition With Single Rectangle

```
DATA_M1_fill : list of list of coordinate_s = {
      { {0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}, {0.0, 1.0} }
};
SHAPE_M1_fill : polygon_layer_s = {
      layer_spec = {output_layer_key = "M1_fill"}
      polygons = DATA_M1_fill,
};

fill_patterns = {{
      type = UF_POLYGON,
      polygon_fill = {
         layers = {SHAPE_M1_fill}
      }
}}
```

Single Layer Pattern Definition With Multiple Rectangles

The pattern definition shown in Example 3 and Figure 57 has one fill layer, M1_fill, that is defined by two rectangles. The dashed line indicates the bounding box of the layer definition. This pattern is repeated inside the target region defined by the spacing constraints. The bounding box of the pattern is the same as that of the layer definition because the pattern consists of only one layer.

Example 3 One Fill-Layer Example Using a Fill Pattern of Type UF_POLYGON

```
DATA_M1_fill : list of list of coordinate_s = {
      { {0.0, 0.0}, {1.0, 0.0}, {1.0, 2.0}, {0.0, 2.0} },
      { {2.0, 0.0}, {3.0, 0.0}, {3.0, 2.0}, {2.0, 2.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
      layer_spec = {output_layer_key = "M1_fill"}
      polygons = DATA_M1_fill,
};

fill_patterns = {{
```

Feedback

```
      type = UF_POLYGON,
      polygon_fill = {
         layers = {SHAPE_M1_fill}
      }
}}
```

Figure 57 One Fill-Layer Example



Multilayer Pattern Definition

The pattern definition shown in Example 4 and Figure 58 has two fill layers, M1_fill and M2_fill. The dashed line indicates the bounding box of the pattern definition. This pattern is repeated inside the target region as defined by the spacing constraints. In this example, bounding box of the pattern is not same as that of the individual layer definitions. When repeating the pattern, the bounding box of the pattern as a whole is considered.

Example 4 Two Fill-Layer Example Using a Fill Pattern of Type UF_POLYGON

```
DATA_M1_fill : list of list of coordinate_s = {
      { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
      layer_spec = {output_layer_key = "M1_fill"}
      polygons = DATA_ M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
      { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} }
};

SHAPE_M2_fill : polygon_layer_s = {
      layer_spec = {output_layer_key = "M2_fill"}
      polygons = DATA_ M2_fill,
};

fill_patterns = {{
      type = UF_POLYGON,
      polygon_fill = {
         layers = {SHAPE_M1_fill,
                   SHAPE_M2_fill}
      }
}}
```

Feedback

Figure 58 Two Fill-Layer Example



Multiple Fill Pattern Definition

You can specify multiple pattern definitions in the `unified_fill()` function. The input target region is processed for each fill pattern definition in the order specified. Figure 60 and Figure 59 show a fill procedure that first inserts the multilayer pattern, M1 and M2, followed by the single layer pattern, M1, in the remaining target region.

Example 5 Multiple Patterns Example Using fill_patterns Option of unified_fill() Function

```
DATA_M1_fill : list of list of coordinate_s = {
      { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
      layer_spec = {output_layer_key = "M1_fill"}
      polygons = DATA_ M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
      { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} }
};

SHAPE_M2_fill : polygon_layer_s = {
      layer_spec = {output_layer_key = "M2_fill"}
      polygons = DATA_ M2_fill,
};

fill_patterns = {
        {type = UF_POLYGON,
         polygon_fill = {
            layers = {SHAPE_M1_fill,
                      SHAPE_M2_fill}
         }
        },
        {type = UF_POLYGON,
         polygon_fill = {
            layers = {SHAPE_M1_fill}
         }
        }
}
```

Feedback

Figure 59 Multiple Patterns Example



Adjustable Rectangle Fill Patterns

An adjustable rectangle fill pattern is a single-layer rectangle pattern definition that can automatically stretch or shrink the dimensions of the rectangle to maximize the density of the fill. For this type of fill pattern, set the `type` option to `UF_ADJUSTABLE`.

You can start with a specific rectangle dimension and choose to either grow or shrink all the shapes along the boundary of the target region. Growing and shrinking is done in user-specified increment values. Adjustment of the fill shapes is done only to the shapes along the boundary of the target region.

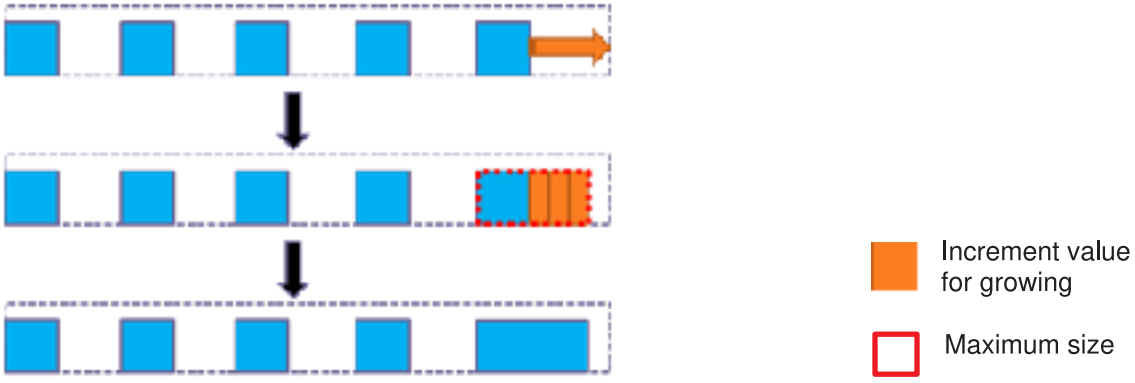
Example 6 defines a minimum rectangle of dimension 1 µm x 1 µm. Rectangles along the boundary of the target region are grown in the x-direction, in increments of 0.05 µm, to fit the maximum possible shape. As shown in Figure 60, the maximum possible shape is 4 µm x 1 µm.

Example 6 Adjustable Rectangle Fill Pattern Example Using a Fill Pattern of Type UF_ADJUSTABLE

```
fill_patterns = {{
      type = UF_ADJUSTABLE,
      adjustable_fill = {
         layer_spec = {output_layer_key = "M1_Fill"}
         width = 1.0,
         height = 1.0,
         width_bound = 4.0,
         width_delta = 0.05,
      }
}}
```

Feedback

Figure 60 Adjustable Rectangle Fill Pattern Example



Stack Fill Patterns

A stack fill pattern is similar to the `UF_POLYGON` pattern type but with more automation in developing a fill procedure. For this type of fill pattern, set the `type` option to `UF_STACK`.

You can use the `UF_STACK` type for applications that require multiple variations of a multilayer pattern to be inserted in sequence without having to explicitly define all the different variations. Using the options of the `unified_fill()` function, you provide a single pattern definition, consisting of multiple fill layers. Based on the fillable region, the tool tries to insert different variations of the fill pattern, starting with the original definition that contains all of the layers, and then proceeding to other variations of the pattern that contain a subset of the layers.

The goal is to find a variation of the pattern with the highest number of layers that can fit in a given fillable area. Use the `required_layer_keys` option of the `stack_fill` option to define dependencies between the fill layers, for example, via1 requires metal1 and metal2. Example 7 shows an example.

Example 7 Stack Fill Pattern Example Using stack_fill Option of unified_fill() Function

```
SHAPE_M1_fill : stack_layer_s = {
      layer_spec = {output_layer_key = "M1_fill"}
      polygons = DATA_M1_fill,
};
SHAPE_M2_fill : stack_layer_s = {
      layer_spec = {output_layer_key = "M2_fill"}
      polygons = DATA_M2_fill,
};
SHAPE_M3_fill : stack_layer_s = {
      layer_spec = {output_layer_key = "M3_fill"}
      polygons = DATA_M3_fill,
};
SHAPE_V1_fill : stack_layer_s = {
```

Feedback

```
      layer_spec = {output_layer_key = "V1_fill"},
      polygons = DATA_V1_fill,
      required_layer_keys = {"M1_fill", "M2_fill"}
};
SHAPE_V2_fill : stack_layer_s = {
      layer_spec = {output_layer_key = "V2_fill"},
      polygons = DATA_V2_fill,
      required_layer_keys = {"M2_fill", "M3_fill"}
};

fill_patterns = {{
      type = UF_STACK,
      stack_fill = {
         layers = {SHAPE_M1_fill,
                   SHAPE_V1_fill,
                   SHAPE_M2_fill,
                   SHAPE_V2_fill,
                   SHAPE_M3_fill},
         min_stack = 1,
         max_stack = 5
      }
}}
```

SHAPE_M1_fill, SHAPE_V1_fill, SHAPE_M2_fill, SHAPE_V2_fill, and SHAPE_M3_fill are individual layer definitions of a single multilayer pattern. The `unified_fill()` function processes the fillable region with the following patterns in the order shown:

M1-V1-M2-V2-M3

M1-V1-M2

M2-V2-M3

M1

M2

M3

The previous `UF_STACK` example is equivalent to writing multiple pattern definitions of the `UF_POLYGON` type, as shown in Figure 63.

Example 8 Writing Multiple Pattern Definitions Using the UF_POLYGON Type

```
fill_patterns = {
     {type = UF_POLYGON,
      polygon_fill = {      // First pattern for insertion
         layers = {SHAPE_M1_fill,
                   SHAPE_V1_fill,
                   SHAPE_M2_fill,
                   SHAPE_V2_fill,
                   SHAPE_M3_fill}
      }
```

Feedback

```
      },
      {type = UF_POLYGON,
       polygon_fill = {      // Second pattern for insertion
          layers = {SHAPE_M1_fill,
                    SHAPE_V1_fill,
                    SHAPE_M2_fill}
       }
      },
      {type = UF_POLYGON,
       polygon_fill = {     // Third pattern for insertion
          layers = {SHAPE_M2_fill,
                    SHAPE_V2_fill,
                    SHAPE_M3_fill}
       }
      },
      {type = UF_POLYGON,
       polygon_fill = {     // Fourth pattern for insertion
          layers = {SHAPE_M1_fill}
       }
      },
      {type = UF_POLYGON,
       polygon_fill =       // Fifth pattern for insertion
         {layers = {SHAPE_M2_fill}
       }
      },
      {type = UF_POLYGON,
       polygon_fill =       // Sixth pattern for insertion
         {layers = {SHAPE_M3_fill}
       }
      }
}
```

The order of the individual layer definitions determines the order of pattern variations that are placed. If the layers are defined as shown in Example 9.

Example 9 Order of Individual Layer Definitions Example

```
fill_patterns = {{
         type = UF_STACK,
         stack_fill = {
            layers = {SHAPE_M3_fill,
                      SHAPE_V2_fill,
                      SHAPE_M2_fill,
                      SHAPE_V1_fill,
                      SHAPE_M1_fill},
            min_stack = 1,
            max_stack = 5
         }
}}
```

Feedback

The order of pattern insertion sequence is changed to:

M1-V1-M2-V2-M3

M2-V2-M3

M1-V1-M2

M3

M2

M1

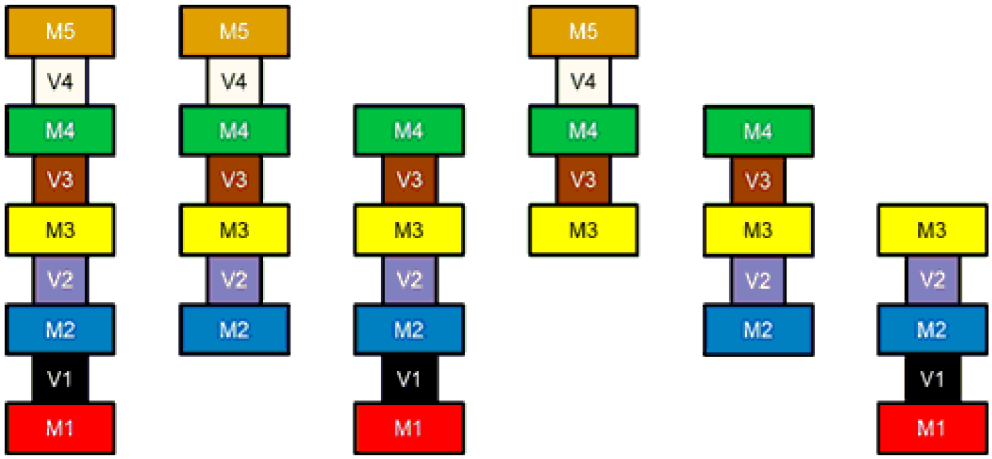
The number of variations of a pattern that can be constructed by the tool depends on the values of the `min_stack` and `max_stack` options.

- The `min_stack` value specifies the minimum number of fill layers required in any variation.
- The `max_stack` value specifies the maximum number of fill layers allowed in a variation.

Figure 61 shows an example of all pattern variations for a fill application in which the pattern definition consists of 5 metals and 4 vias as the fill layers. In this example,

```
max_stack = 9, min_stack = 5
```

Figure 61 Example of All Pattern Variations for a Fill Application



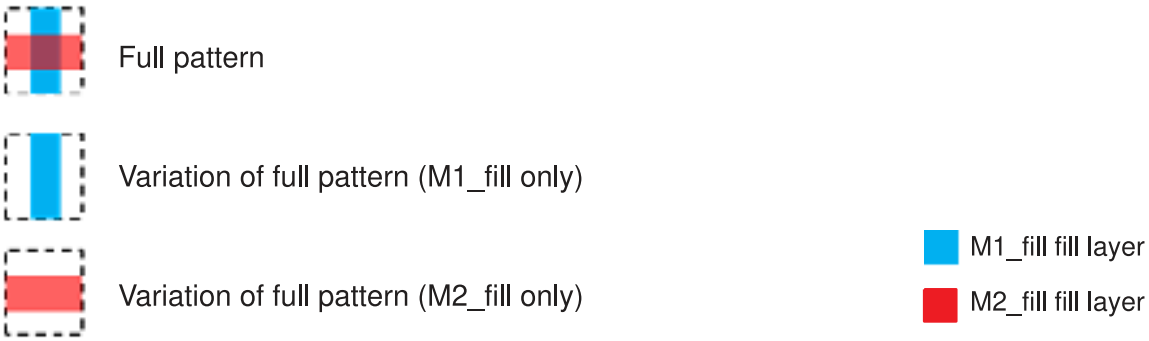
Depending on the layer definitions in the pattern, the bounding box of the pattern might be larger than the individual bounding box of each layer definition. In such cases, the bounding box of the smaller variations of the pattern might be different from that of the complete pattern definition, which consists of all the layers.

Feedback

By default, the `unified_fill()` function retains the bounding box of the complete pattern for all of its variations. All automatic variations of the pattern by default align with the bounding box of the complete pattern, as shown in Figure 61.

Example 10 and Figure 63 show an example where the full pattern is a structure with two fill layers, M1_fill and M2_fill, which are shown in Figure 62. There are two variations of this pattern, a pattern with only M1_fill and a pattern with only M2_fill. However, all patterns have the same bounding box as that of the full pattern. As a result, it might appear that the individual rectangles on M1_fill are spaced farther apart than the specified spacing value. This is because the spacing is based on the bounding box.

Figure 62 Two Fill Layers Example

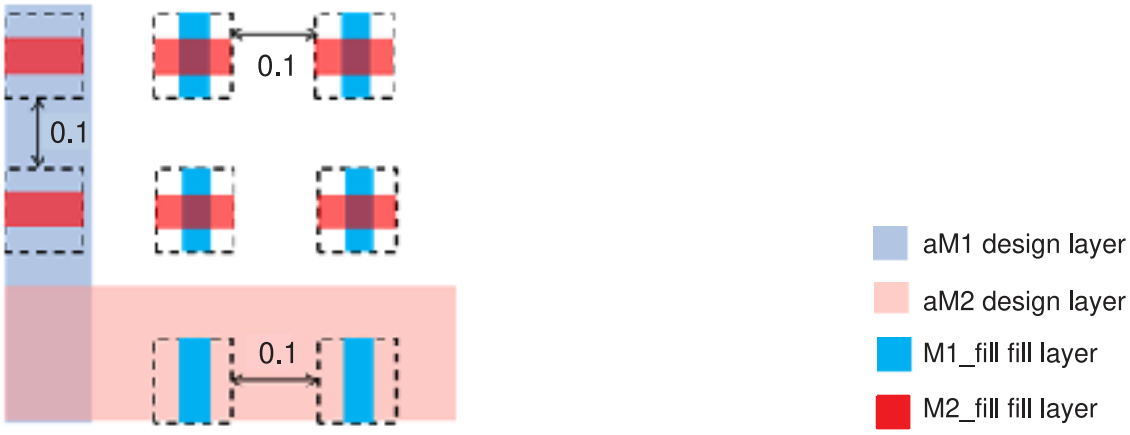


Example 10 Pattern Structure With Two Fill Layers Using stack_fill Option of unified_fill() Function

```
fill_patterns = {{
      type = UF_STACK,
      stack_fill = {
         pattern_spec = {space_x = 0.1,
                         space_y = 0.1},
         layers = {SHAPE_M1_fill,
                   SHAPE_M2_fill},
         min_stack = 1,
         max_stack = 2
      }
}}
```

Feedback

Figure 63 Pattern Structure With Two Fill Layers

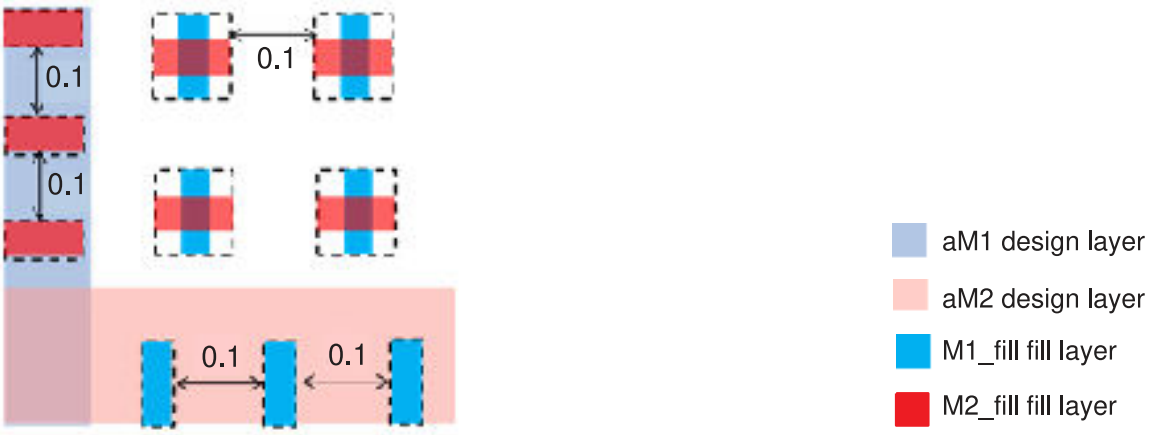


By default, the `unified_fill()` function retains the bounding box of the complete pattern for all of its variations.

You can change this alignment behavior by setting the `separate_patterns` option to `true`. In that case, every pattern variation has its own bounding box based on the extents of the subset of layers in it. When inserting a pattern variation, the bounding box of that specific pattern variation is used instead of the bounding box of the complete pattern.

In Figure 64, the two variations of the full pattern have their own bounding box. As a result, they appear to be placed closer when compared to the behavior when the `separate_patterns` option is `false`.

Figure 64 Pattern Structure With Two Fill Layers With separate_patterns Option True



Feedback

When the `separate_patterns` option is `false`,

- The `fill_context`, `iterations`, and `shift_factor` options are ignored.
- The `min_stack` and `max_stack` values are ignored. In other words, the tool works with all possible variations of the pattern.

Cell Fill Patterns

A cell fill pattern lets you import patterns from an external database instead of manually defining them in the runset. For this type of fill pattern, set the `type` option to `UF_CELL`.

The `UF_CELL` type provides an additional interface to import a cell, from an external GDSII or OASIS library, into the runset and map the layers in that cell to the fill layers. All other functionality is the same as with the `UF_POLYGON` and `UF_STACK` types. The imported pattern definition can be used either as is, as with the `UF_POLYGON` type, or chosen to be broken down to smaller patterns, as with the `UF_STACK` type.

A fill cell or a library of fill cells can be maintained along with the fill runset. These fill cells contain the fill layers that the runset can import and insert in the target region based on the specified constraints. See the `import_gds_cell()` and `import_oasis_cell()` functions in the *IC Validator Reference Manual* for information about importing a cell into the runset.

In Example 11, the M1_fill, V1_fill, M2_fill, V2_fill and M3_fill layer definitions are imported from the FillCell cell in the Fill_Cell.gds file. For example, Layer (1;0) in Fill_Cell.gds is designated for the M1_fill layer. See Stack Fill Patterns for examples of additional functionality that are conceptually equivalent.

Example 11 Cell Fill Pattern Example

```
Fill_Cell_lib = gds_library("./Fill_Cell.gds");
Fill_Cell = import_gds_cell(Fill_Cell_lib, "FillCell");

STACK_metal_fill : cell_fill_s =
{
   cell = Fill_Cell
   layers = {          // Map Individual layers of the cell to Fill layers
       {ldt_list = {{1, 0}},layer_spec = {output_layer_key = "M1_fill"}},
       {ldt_list = {{2, 0}},layer_spec = {
                                output_layer_key = "V1_fill"},
                                required_layer_keys = {"M1", "M2"}},
       {ldt_list = {{3, 0}},layer_spec = {output_layer_key = "M2_fill"}},
       {ldt_list = {{4, 0}},layer_spec = {
                                output_layer_key = "V2_fill"},
                                required_layer_keys = {"M2", "M3"}},
       {ldt_list = {{5, 0}},layer_spec = {output_layer_key = "M3_fill"}}
   },
   min_stack = 1,
   max_stack = 5,
   rotation = NONE,
```

Feedback

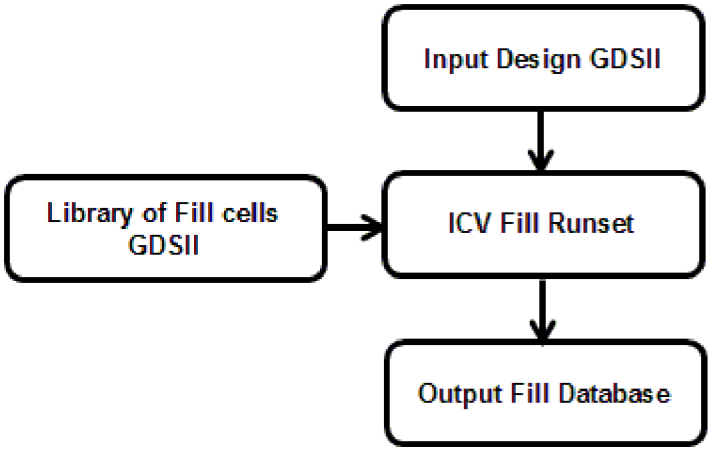
```
    reflection = false
};
```

Note:

Depending on your requirements, the `UF_STACK` type provides more automation in developing a fill procedure over the `UF_POLYGON` type. Similarly, the `UF_CELL` type provides more automation over the `UF_STACK` and `UF_POLYGON` types.

Figure 65 shows an example usage model for the `UF_CELL` type.

Figure 65 Usage Model for the UF_CELL Type



Expandable Fill Cell Patterns

An expandable fill-cell pattern lets you insert fill cells that get expanded along the boundaries of a fill target area. For this type of fill pattern, set the `type` option to either `UF_EXPANDABLE` or `UF_EXPANDABLE_CELL`.

The `UF_EXPANDABLE_CELL` type provides an additional interface to import a cell, from an external GDSII or OASIS library, into the runset and map the layers in that cell to the fill layers. All other functionality is the same as with the `UF_EXPANDABLE` type.

To use an expandable fill-cell type pattern, first define a base fill cell with incremental unit cells. For example, as shown in Figure 66, a simple base cell is made up of three incremental units. In this example, only the middle unit is repeatable.

Feedback

Figure 66 Repeatable Cell Example

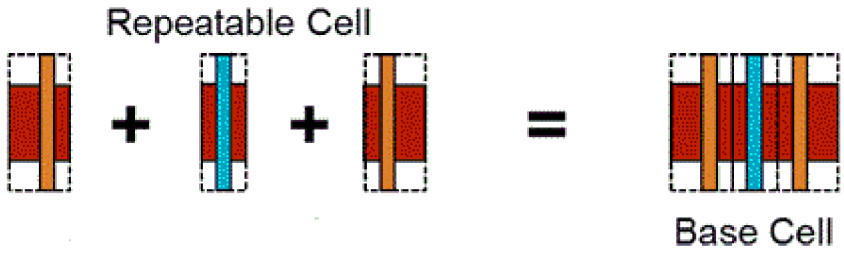
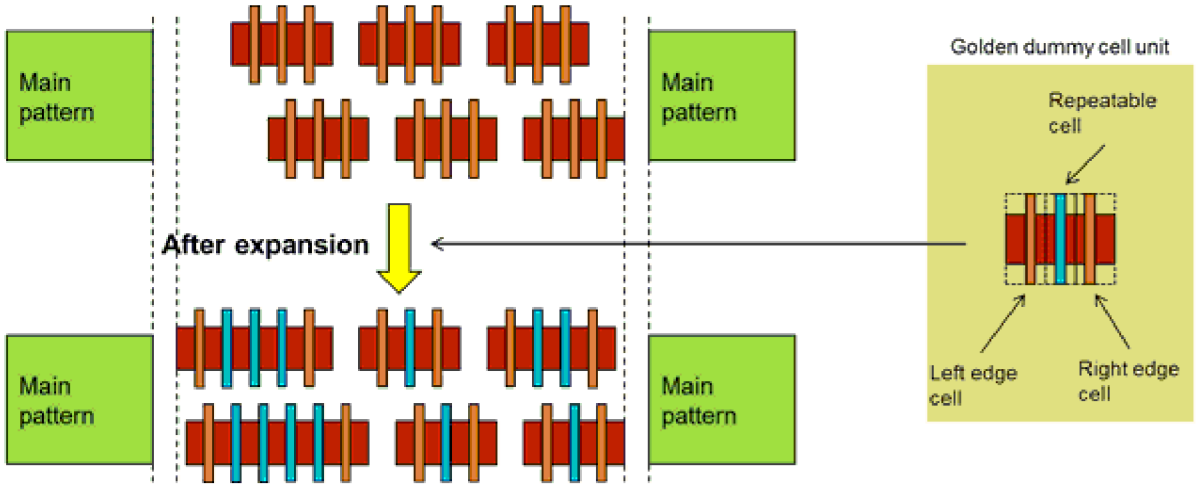


Figure 67 shows the expansion of a fill cell. The top half of the figure shows the base cell being inserted before the expansion. The bottom half of the figure shows the base cells that were expanded by repeating the middle unit.

Figure 67 Expansion of a Fill Cell



Example 12 shows how to program the `UF_EXPANDABLE` type.

1. Define the layers used in each incremental unit.
2. Declare the actual three incremental unit cells that make up the base cell.
3. Call the `unified_fill()` function and retrieve the output layers.

Example 12 UF_EXPANDABLE Type Example

```
// Define the layers used in each incremental unit
EXPAND_CELL_LEFT_LAYER_1 : polygon_layer_s = {
   layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_LEFT_LAYER_1"},
   polygons = {{ {0.045, 0.193}, {0.261, 0.103}, {0.261, 0.287}, {0.058, 0.247
} }}
};
EXPAND_CELL_LEFT_LAYER_2 : polygon_layer_s = {
   layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_LEFT_LAYER_2"},
```

Feedback

```
   polygons = {{ {0.045, 0.128}, {0.191, 0.128}, {0.191, 0.432}, {0.155, 0.232
} }}
};

EXPAND_CELL_MIDDLE_LAYER_1 : polygon_layer_s = {
   layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_MIDDLE_LAYER_1"},
   polygons = {{ {0.045, 0.193}, {0.261, 0.103}, {0.261, 0.287}, {0.058, 0.247
} }}
};
EXPAND_CELL_MIDDLE_LAYER_2 : polygon_layer_s = {
   layer_spec ={ output_layer_key = "OUTPUT_EXPAND_CELL_MIDDLE_LAYER_2"},
   polygons = {{ {0.045, 0.128}, {0.191, 0.128}, {0.191, 0.432}, {0.155, 0.232
} }}
};

EXPAND_CELL_RIGHT_LAYER_1 : polygon_layer_s = {
   layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_RIGHT_LAYER_1"},
   polygons = {{ {0.045, 0.193}, {0.261, 0.103}, {0.261, 0.287}, {0.058, 0.247
} }}
};
EXPAND_CELL_RIGHT_LAYER_2 : polygon_layer_s = {
   layer_spec = {output_layer_key = "OUTPUT_EXPAND_CELL_RIGHT_LAYER_2"},
   polygons = {{ {0.045, 0.128}, {0.191, 0.128}, {0.191, 0.432}, {0.155, 0.232
} }}
};

// Declare the actual three incremental unit cells that make up the base cell
cell_A_left : base_cell_s = {
   layers = {EXPAND_CELL_LEFT_LAYER_1, EXPAND_CELL_LEFT_LAYER_2 },
   repeatable = false
};
cell_B_middle : base_cell_s = {
   layers = { EXPAND_CELL_MIDDLE_LAYER_1, EXPAND_CELL_MIDDLE_LAYER_2 },
   repeatable = true,
   maximum = 10
};
cell_C_right : base_cell_s = {
   layers = { EXPAND_CELL_RIGHT_LAYER_1, EXPAND_CELL_RIGHT_LAYER_2 },
   repeatable = false
};

// Call unified_fill()
expand_fill_cell_output = unified_fill(
   fill_patterns = {
      {type = UF_EXPANDABLE,
       expandable_polygon_fill = {
          pattern_spec = {
             space_x = 0.12,
             space_y = 0.06,
             stagger_x = 0.2 },
          base_cell = {cell_A_left, cell_B_middle, cell_C_right} }
      }
   },
   fill_boundary = { type = LAYER, layer = REGION_CELL2_CORE }
);

// Retrieve the output layers
expand_fill_cell_layer_1 =
      expand_fill_cell_output ["OUTPUT_EXPAND_CELL_LEFT_LAYER_1"][0]
   or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_MIDDLE_LAYER_1"][0]
   or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_RIGHT_LAYER_1"][0]
```

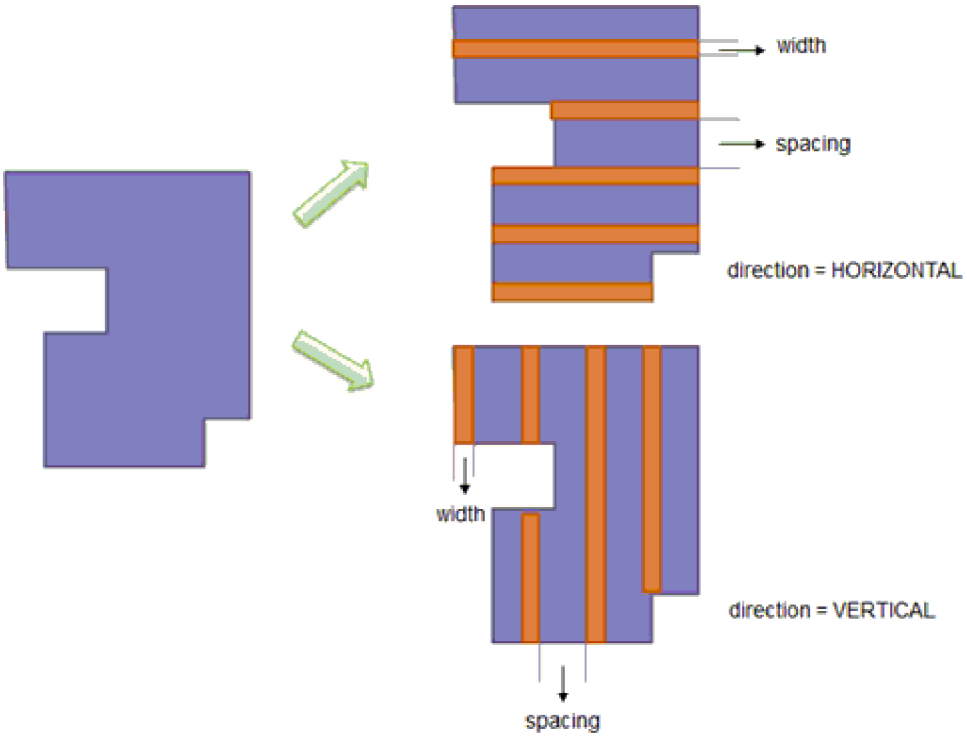
Feedback

```
expand_fill_cell_layer_2 =
      expand_fill_cell_output ["OUTPUT_EXPAND_CELL_LEFT_LAYER_2"][0]
   or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_MIDDLE_LAYER_2"][0]
   or expand_fill_cell_output ["OUTPUT_EXPAND_CELL_RIGHT_LAYER_2"][0]
```

Stripe Fill Patterns

A stripe fill pattern lets you insert a stripe-shape fill across target areas. For this type of fill pattern, set the `type` option to `UF_STRIPE`. You specify the direction, width, and spacing of the stripes, as shown in Example 13 .

Figure 68 Stripe Fill Patterns



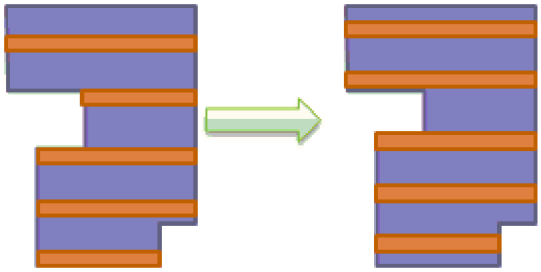
When symmetry is specified for stripe fill, the `unified_fill()` function

1. Gets the bounding box
2. Finds the center of the bounding box
3. Places the first stripe with its center line on the center of the bounding box
4. Places subsequent stripes at equal distances from the center stripe
5. Saves only full width stripes

Feedback

Figure 69 shows the result of symmetry for stripes.

Figure 69 Symmetry of Stripes



Example 13 shows a `unified_fill()` call using the `UF_STRIPE` type.

Example 13 UF_STRIPE Type Example

```
result = unified_fill(
   fill_patterns = {
      {type = UF_STRIPE,
       stripe_fill = {
          layer_spec = {
             output_layer_key = "DM1_FILL",
          },
          direction = HORIZONTAL,
          width = 0.1,
          spacing = 0.2,
       }
      }
   },

   fill_boundary = {
      type = LAYER,
      layer = M1
   },
   extents_output = {
      {output_layer_key = "OUTPUT_DM_EXTENTS"}
   }
);
```

Fill-to-Fill Spacing

As described earlier in this chapter, you can configure the `Unified Fill` function to insert different types of patterns into specified given target regions. By providing a list of pattern definitions, you instruct the tool to fill the target region by repeating each pattern definition in sequential steps.

Feedback

The fill-to-fill spacing feature lets you define:

- Spacing Between Instances of the Same Patterns
- Spacing Between Different Patterns

Spacing Between Instances of the Same Patterns

Spacing between instances of the same pattern is controlled by the `space_x`, `space_y`, `stagger_x` and `stagger_y` options in the `pattern_spec` option.

Note:

These values can be negative in certain conditions to support overlap of pattern placements.

Additionally, for the spacing between fill polygons and patterns that are placed inside different target regions, you can set the spacing between patterns by using the `pattern_spacing` option inside the `pattern_spec` option, or you can set the spacing between individual fill layers by using the `fill_to_fill_spacing` option inside the `layer_spec` option. The options are:

- `allowed_spacing_x`
- `allowed_spacing_y`
- `min_space_corner`
- `extension`

See the `Unified Fill` function for descriptions of these options. Use these options to trigger more complex layer-specific spacing checks during fill treatment. The `space_x`, `space_y`, `stagger_x`, and `stagger_y` options apply to the bounding box of the entire pattern definition in accordance with the `separate_patterns` option.

By design, the `unified_fill()` function makes sure that no two patterns get closer than the `space_x` and `space_y` values. In Example 14, `space_x` and `space_y` values are specified for the multilayer pattern definition. If patterns falling in adjacent polygons of the target region come too close, one of them is not inserted to meet the fill-to-fill spacing constraint, as shown in Figure 70.

Example 14 Spacing Between Same Patterns Using the space_x and space_y Options

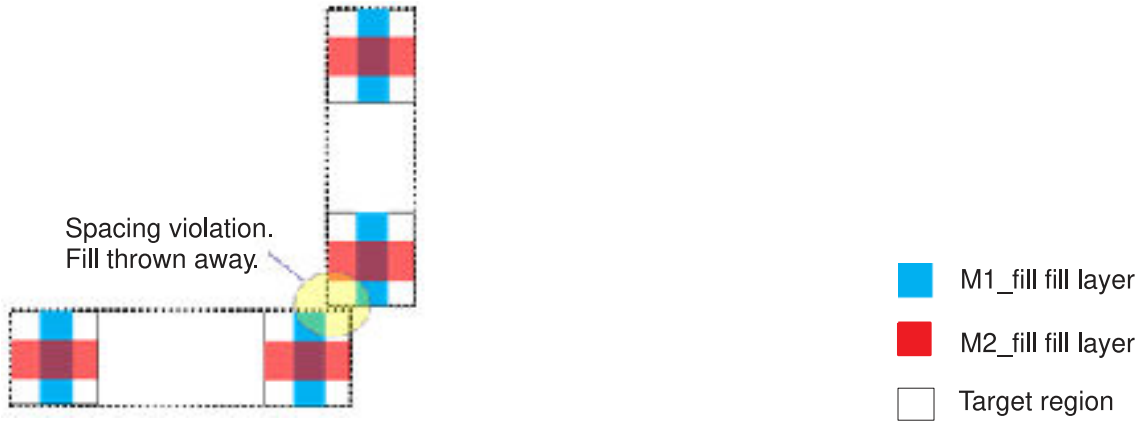
```
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {output_layer_key = "M1_fill"}
   polygons = DATA_ M1_fill,
};
SHAPE_M2_fill : polygon_layer_s = {
   layer_spec = {output_layer_key = "M2_fill"}
   polygons = DATA_ M2_fill,
```

Feedback

```
};

fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      pattern_spec = {space_x = 0.1,
                      space_y = 0.1},
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill}
   }
}}
```

Figure 70 Spacing Between Same Patterns Example



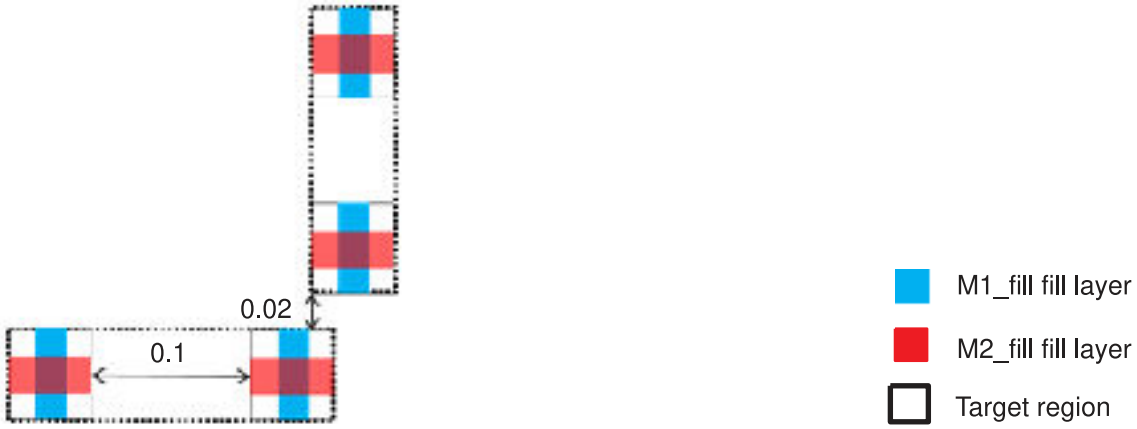
In Example 15, the `allowed_spacing_x` and `allowed_spacing_y` values are specified in the `pattern_spacing` option for the multilayer pattern definition. This ensures the necessary spacing between pattern instances from different target areas. As shown in Figure 71, both pattern instances are placed because the spacing in the x- and y-directions respects the `allowed_spacing_x` and `allowed_spacing_y` values.

Example 15 Spacing Between Multilayer Patterns With Allowed Spacing Values Smaller Than Space Values

```
fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      pattern_spec = {space_x = 0.1, space_y = 0.1,
                      pattern_spacing = {
                         allowed_spacing_x = {>= 0.02},
                         allowed_spacing_y = {>= 0.02}
                      }
      },
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill}
   }
}}
```

Feedback

Figure 71 Spacing Between Multilayer Patterns With Allowed Spacing Values Smaller Than Space Values Example



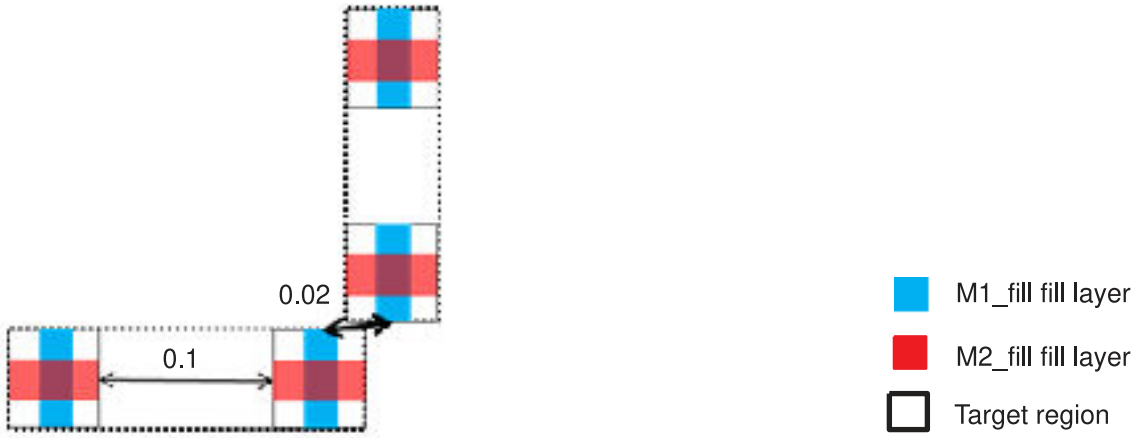
In Example 16, the `allowed_spacing_x` and `allowed_spacing_y` values are specified in the `fill_to_fill_spacing` option for the M1_fill fill layer. Even though the space between the bounding boxes violates the `space_x` and `space_y` values, the tool does not remove patterns if the individual layers inside the pattern satisfy the `allowed_spacing_x` and `allowed_spacing_y` values, as shown in Figure 72.

Example 16 Using the Allowed Spacing Values in the fill_to_fill_spacing Option

```
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
      fill_to_fill_spacing = {
         allowed_spacing_x = {>= 0.02},
         allowed_spacing_y = {>= 0.02}
      }
   }
   polygons = DATA_ M1_fill,
};
fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      pattern_spec = {space_x = 0.1, space_y = 0.1},
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill}
}}
```

Feedback

Figure 72 Using the Allowed Spacing Values in the fill_to_fill_spacing Option Example



Spacing Between Different Patterns

When processing a list of fill patterns, you can specify spacing values between different patterns using the `other_pattern_spacing` option. This option is a hash of integer to double constraint where the integer represents the order of the pattern within the list of patterns. The integer 0 represents the first pattern. For example, to set the spacing to the third pattern to be 0.5 or greater, write

```
other_pattern_spacing = {2 => >= 0.5}
```

Note:

The spacing is always applied to the bounding box of every pattern.

Figure 73 shows an application that uses 3 patterns, A, B, and C. The spacing between the patterns is expected to be as follows:

Pattern A:

- Space to Pattern B = 2
- Space to Pattern C = 1

Pattern B:

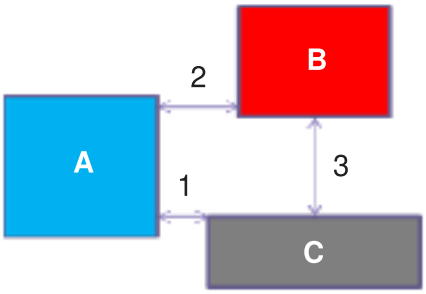
- Space to Pattern A = 2
- Space to Pattern C = 3

Pattern C:

- Space to Pattern A = 1
- Space to Pattern B = 3

Feedback

Figure 73 Spacing Between Different Patterns Example



Use the appropriate indexes in the `other_pattern_spacing` option, shown in Example 17, to set spacing from the current pattern to other patterns.

Example 17 Using the other_pattern_spacing Option

```
fill_patterns = {
// First Pattern A
   {type = UF_POLYGON,
    polygon_fill = {
       pattern_spec = {
          space_x = 0.1, space_y = 0.1,
          other_pattern_spacing = {1 => >= 2, 2 => >= 1}
       },
       layers = {SHAPE_A}
    }
   },

// Second Pattern B
   {type = UF_POLYGON,
    polygon_fill = {
       pattern_spec = {
          space_x = 0.1, space_y = 0.1,
          other_pattern_spacing = { 0 => >= 2, 2 => >= 3}
       },
       layers = {SHAPE_B}
    }
   },

// Third Pattern C
   {type = UF_POLYGON,
    polygon_fill = {
       pattern_spec = {
          space_x = 0.1, space_y = 0.1,
          other_pattern_spacing = {0 => >= 1, 1 => >= 3}
       },
       layers = {SHAPE_C}
    }
   }
}
```

Feedback

Rules and Restrictions for Spacing Options

The following rules and restrictions apply for the spacing options:

- The `allowed_spacing_x` and `allowed_spacing_y` values can be set in both the `layer_spec` and `pattern_spacing` options at the same time.
- The `allowed_spacing_x` and `allowed_spacing_y` values in the `layer_spec` option do not support multiple space ranges. That is, the list can only have one value. Only the > and >= constraints are supported.
- The `allowed_spacing_x` and `allowed_spacing_y` values in the `pattern_spacing` option support multiple space ranges. The < and <= constraints are not supported.
- The `allowed_spacing_x`, `allowed_spacing_y` and `min_space_corner` options do not support negative values.
- When the `allowed_spacing_x` and `allowed_spacing_y` values are not specified, valid fill-to-fill spacing is any distance greater than or equal to (>=) the `space_x` and `space_y` values. The tool makes sure that patterns or fill layers in adjacent fill-region polygons have the minimum spacing of the `space_x` and `space_y` values.
- When the `allowed_spacing_x` and `allowed_spacing_y` values are specified, the tool makes sure that patterns or fill layers in adjacent target layer polygons have the minimum spacing of the `allowed_spacing_x` and `allowed_spacing_y` values.
- When the `fill_context` option is `PARTITION_CONTEXT`, patterns or fill layers in adjacent partitions use the `allowed_spacing_x` and `allowed_spacing_y` values, if specified.
- The `other_pattern_spacing` option does not apply to variations of a pattern when the `type` option is `UF_STACK` or `UF_CELL`. In this case, the minimum space between different variations of a given pattern is based on the `space_x` and `space_y` values.
- By default, the `unified_fill()` function performs corner checking between patterns in different polygons of the fill region that do not share any common projection.
- When the `type` option is `UF_STACK`, the `unified_fill()` function does not apply either the `pattern_spacing` or `fill_to_fill_spacing` option between patterns of different permutations.
- The `space_x` and `space_y` values can be less than or equal to 0 (<= 0) if the `type` option is `UF_POLYGON`, `UF_STACK` or `UF_CELL`, `UF_EXPANDABLE`, or `UF_EXPANDABLE_CELL`. In addition for `UF_STACK` and `UF_CELL`, the `separate_patterns` option must be `false`, and the `grouping` option must be `NONE`.
- Corner checking, using the `min_space_corner` and `extension` options, is disabled if either the `space_x` or `space_y` option is negative. By default, a corner spacing check between fill patterns in different fill regions is based on max(`space_x`, `space_y`).

Feedback

Target Region and Fill-to-Signal Spacing

The `unified_fill()` function does not require you to explicitly derive the target region in the runset. The target region is automatically constructed by the tool using the various fill-to-signal spacing constraints. Specify these spacing values using the `fill_to_signal_spacing` option either in the definition of an individual fill layer or in the definition of the fill pattern.

In Example 18, the M1_Fill fill layer is kept away from the aM1 design layer by 0.1 µm. Similarly, the

- Minimum space from the M1_fill fill layer to the aM1_fill design layer is 0.5 µm
- Minimum space from the M2_fill fill layer to the aM2 design layer is 0.1 µm
- Minimum space from the M2_fill fill layer to the aM2_fill design layer is 0.5 µm
- Minimum space from the (M1_fill, M2_fill) multilayer pattern to BLKG_1 is 0.3 µm
- Minimum space from the (M1_fill, M2_fill) multilayer pattern to BLKG_2 is 0.2 µm

Example 18 fill_to_signal_spacing Option Example

```
DATA_M1_fill : list of list of coordinate_s = {
   { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
      fill_to_signal_spacing = {
         {aM1,      min_space = 0.1},
         {aM1_FILL, min_space = 0.5}
      }
   }
   polygons = DATA_ M1_fill,
};

DATA_M2_fill : list of list of coordinate_s = {
      { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} }
};
SHAPE_M2_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M2_fill",
      fill_to_signal_spacing = {
         {aM2,      min_space = 0.1},
         {aM2_FILL, min_space = 0.5}
      }
   }
   polygons = DATA_M2_fill,
};
```

Feedback

```
fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill},
      fill_to_signal_spacing = {
         {aBLKG_1, min_space = 0.3},
         {aBLKG_2, min_space = 0.2}
      }
   }
}}
```

For fill patterns of `UF_POLYGON` type, the multilayer pattern is not broken down to smaller variations of the pattern. As shown in Figure 74, the pattern is repeated in the target region by keeping a minimum distance from the design layers as specified in the `fill_to_signal_spacing` option.

By default, the `unified_fill()` function measures spacing from the edge of the design layer to the bounding box of the pattern. The keepout region for the pattern is constructed using all constraints of the `fill_to_signal_spacing` option defined for this pattern, as well as for every individual layer definition in that pattern.

There are two methods of measuring spacing between fill and signal layers. The `spacing_context` option controls which method of measurement is used for a given fill pattern:

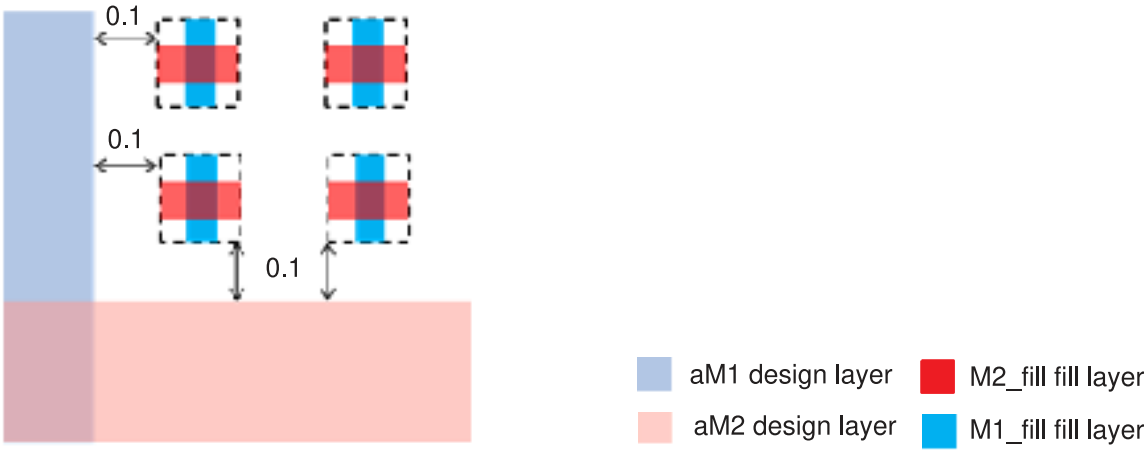
- When set to `UF_PATTERN`, spacing is measured from the signal to the bounding box of a fill pattern.
- When set to `UF_LAYER`, spacing is measured from the signal to individual polygons of a layer inside the pattern.

Note:

When the `space_x` and `space_y` values are negative, the `spacing_context` option setting of `UF_PATTERN` is not supported.

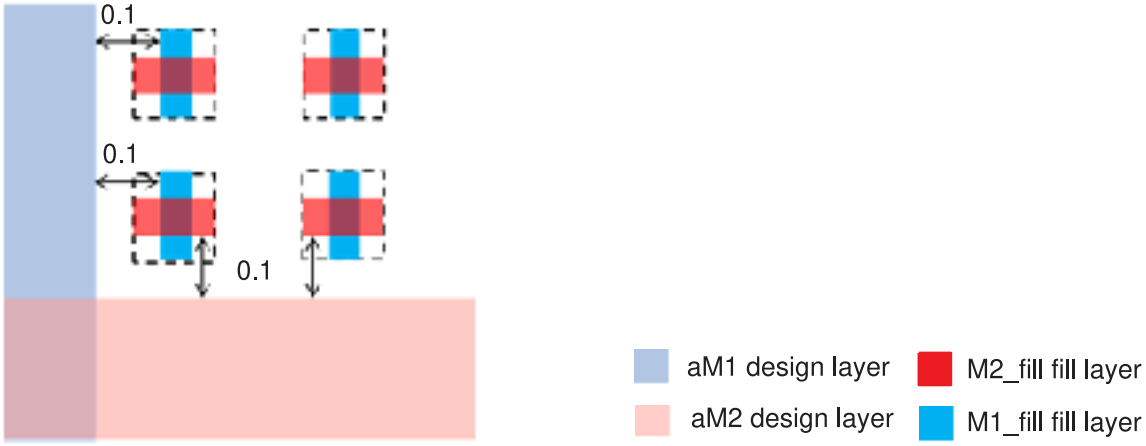
Feedback

Figure 74 spacing_context = UF_PATTERN Example



As shown in Figure 75, when the `spacing_context` option is `UF_LAYER`, the `unified_fill()` function measures fill-to-signal spacing from the edge of the design layer to the bounding box of an individual layer definition.

Figure 75 spacing_context = UF_LAYER Example



Example 19 and Figure 76 illustrate the behavior of the `spacing_context` option.

Example 19 fill_to_signal_spacing in Fill Layer Definition

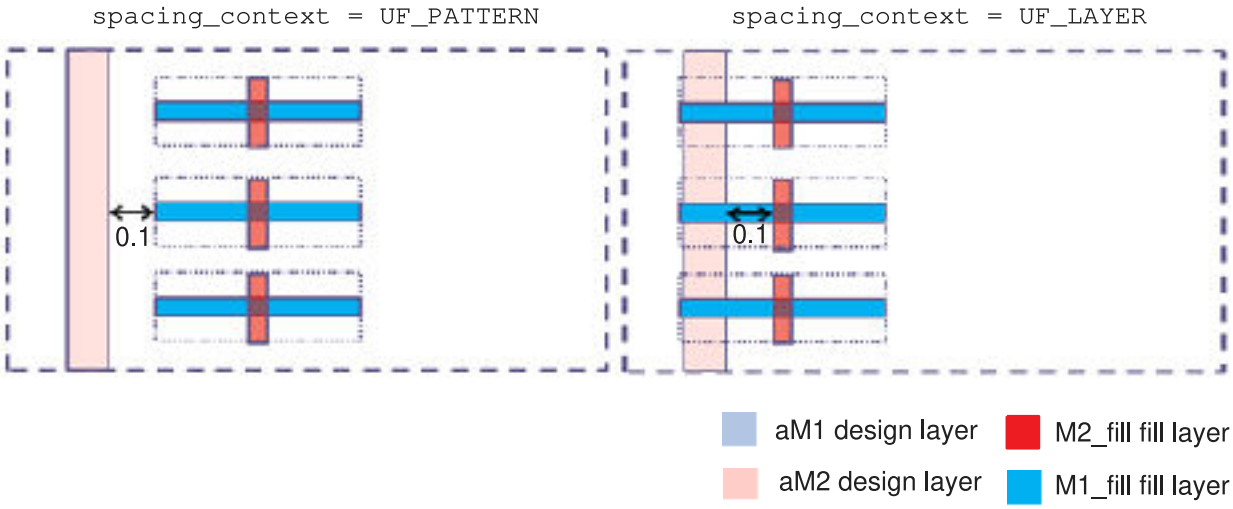
```
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
      fill_to_signal_spacing = {
         {aM1, min_space = 5.0}
      }
   }
```

Feedback

```
   polygons = DATA_ M1_fill,
};
SHAPE_M2_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M2_fill",
      fill_to_signal_spacing = {
         {aM2, min_space = 0.1}
      }
   }
   polygons = DATA_M2_fill,
};

fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill}
   }
}}
```

Figure 76 spacing_context Behavior



For the `UF_POLYGON`, `UF_STACK`, `UF_CELL`, `UF_EXPANDABLE`, and `UF_EXPANDABLE_CELL` pattern types, it is possible to specify the `fill_to_signal_spacing` option at both the `layer_spec` level and the pattern level. When the `fill_to_signal_spacing` option is specified at both levels, the most restrictive value is used. In Example 20, the M1_fill fill layer is kept away from the aM1 design layer by a minimum distance of 0.5 µm.

Example 20 fill_to_signal_spacing Defined Both in the layer_spec Option and in the Pattern

```
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
```

Feedback

```
      fill_to_signal_spacing = {
         {aM1, min_space = 0.5}
      }
   }
   polygons = DATA_ M1_fill,
};

fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      layers = {SHAPE_M1_fill,},
      fill_to_signal_spacing = {
         {aM1, min_space = 0.1}
      }
}
}}
```

In Example 21, as shown in Figure 77, multiple design layers are used with `fill_to_signal_spacing` values at different places in the fill layer definitions and the fill pattern definition.

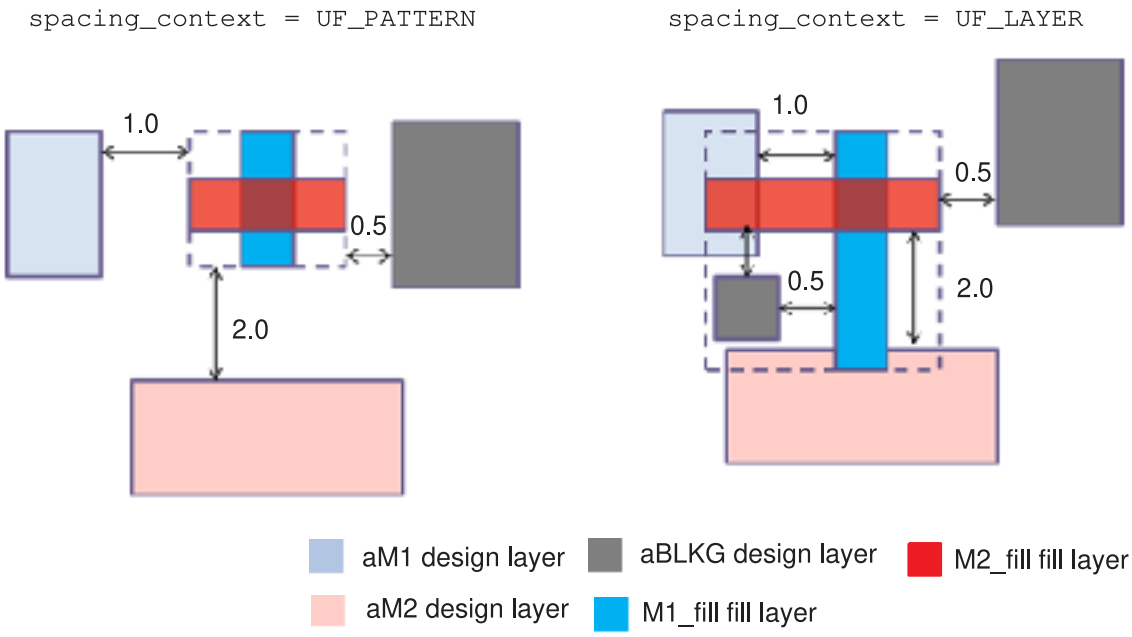
Example 21 fill_to_signal_spacing With spacing_context and Multiple Design Layers

```
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
      fill_to_signal_spacing = {
         {aM1, min_space = 1.0}
      }
   }
   polygons = DATA_ M1_fill,
};
SHAPE_M2_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M2_fill",
      fill_to_signal_spacing = {
         {aM2, min_space = 2.0}
      }
   }
   polygons = DATA_M2_fill,
};

fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill},
      fill_to_signal_spacing = {
         {aBLKG, min_space = 5.0}
      }
}
}}
```

Feedback

Figure 77 spacing_context Behavior With Multiple Design Layers



Range and Width Dependent Spacing

Apart from a fixed minimum space value in the `fill_to_signal_spacing` option, fill-to-signal spacing ranges from a fill layer to a design layer can also be specified. You can specify a list of lower bound and upper bound constraints to indicate the allowed spacing of the fill layers from the design layer. Fill patterns are placed only within these space constraints. Additionally, a list of spacing constraints can be specified based on the width of the design layer polygons.

In Example 22 spacing of the M1_fill fill layer to the aM1 design layer is

- Minimum space from M1_fill to aM1:
  - aM1 width > 0.1
  - space = 0.1
- Allowed space from M1_fill to aM1:
  - aM1 > 0.3
  - 0.2 < space < 0.5 or space >= 1.0

    That is, M1_fill is not allowed to be placed next to aM1 with

    space <= 0.2 or 0.5 <= space < 1.0

Feedback

Example 22 Range and Width Dependent Spacing Example

```
        layer_spec = {
           output_layer_key = "M1_fill",
           fill_to_signal_spacing = {
              {aM1, {
                 {width > 0.1, {>= 0.1}},
                 {width > 0.3, {(0.2, 0.5), >= 1.0}}
              }},
           }
        }
        polygons = DATA_M1_fill,
    };

    fill_patterns = {{
        type = UF_POLYGON,
        polygon_fill = {
           layers = {SHAPE_M1_fill}
        }
    }}
```

Polygon Grouping

For `UF_STACK` and `UF_CELL` fill types, the `unified_fill()` function can break the pattern definition into smaller permutations. If a pattern definition has two fill layers, for example, M1_fill and M2_fill, there can be three permutations:

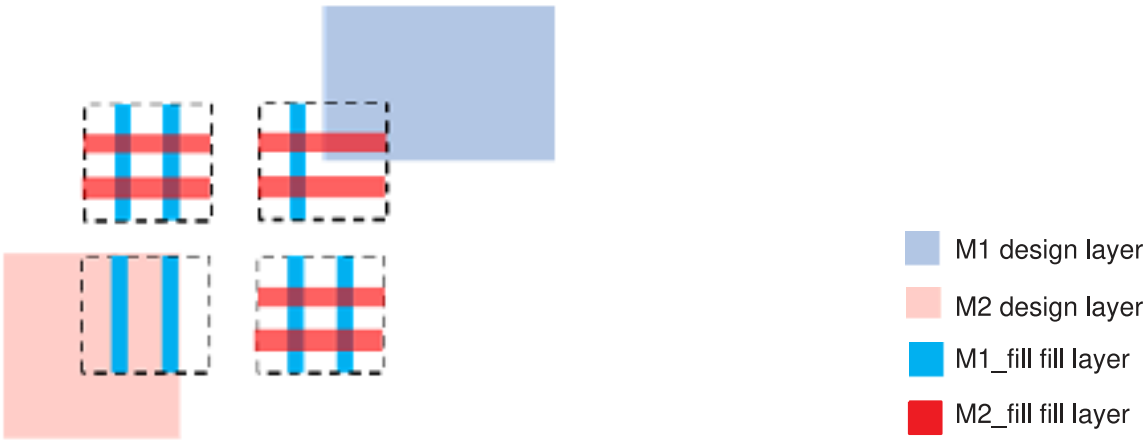
- Complete pattern
- Pattern with only M1_fill
- Pattern with only M2_fill

When an individual layer definition of a fill layer has multiple polygons, you can configure the `unified_fill()` function to break the layer definition further, so that either all or none of the fill-layer polygons are retained when pattern placements cause spacing violations. When multiple polygons constitute a layer definition in a pattern, the `unified_fill()` function retains, by default, all polygons that meet fill-to-signal spacing in every repetition of the pattern.

As shown in Figure 78, a multilayer pattern, indicated by dashed lines, is repeated inside a target region. The `spacing_context` option is `UF_LAYER`. The pattern consists of two fill layers, each composed of two rectangles. Based on fill-to-signal spacing of the fill layer to the design layer, every instance might have either all of the polygons or a partial set of polygons. Polygons causing spacing violations are removed.

Feedback

Figure 78 Multilayer Pattern Repeated Inside a Target Region



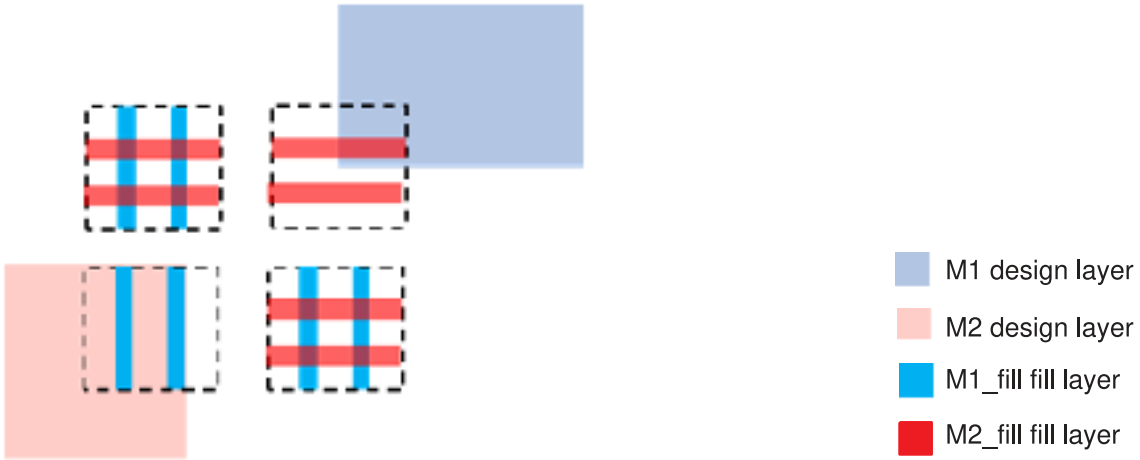
Alternatively, you can configure the `unified_fill()` function in such way that if any polygon of a fill layer inside a pattern repetition causes spacing violations, then all polygons of that fill layer are removed from that specific pattern repetition. For this behavior, set the `grouping` option to `ALL` in the definition of the fill layer. For example, as shown in Example 23 and Figure 79, both polygons of the M1_fill fill layer are removed from the top right pattern repetition, even though only one polygon violates the spacing.

Example 23 grouping = ALL Example

```
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
      grouping = ALL,
      fill_to_signal_spacing = {
         {aM1, min_space = 1.0}
      }
   }
   polygons = DATA_ M1_fill,
};
```

Feedback

Figure 79 Layer Removed From Top Right Pattern Repetition Example



The following rules and restrictions apply for polygon grouping:

- If the `spacing_context` option is `UF_PATTERN`, the `grouping` option setting of `NONE` does not apply because fill-to-signal spacing is measured from the bounding box of the pattern but not from the individual polygons. When the `spacing_context` option is `UF_PATTERN`, the `grouping` option is automatically `ALL`.
- When the `space_x` and `space_y` values are negative, the `grouping` option setting of `ALL` is not supported.
- The `grouping` option does not apply to the `UF_POLYGON` and `UF_ADJUSTABLE` types.

Signal Aligned Fills

For applications that require design layers to be protected by placing fill shapes alongside the design layer polygon, use the `signal` option in the `unified_fill()` function. This feature inserts fill shapes around the design layer shapes in the form of a ring such that the major axis of the fill shape aligns with the edge of the design layer shape. Figure 80 shows an example.

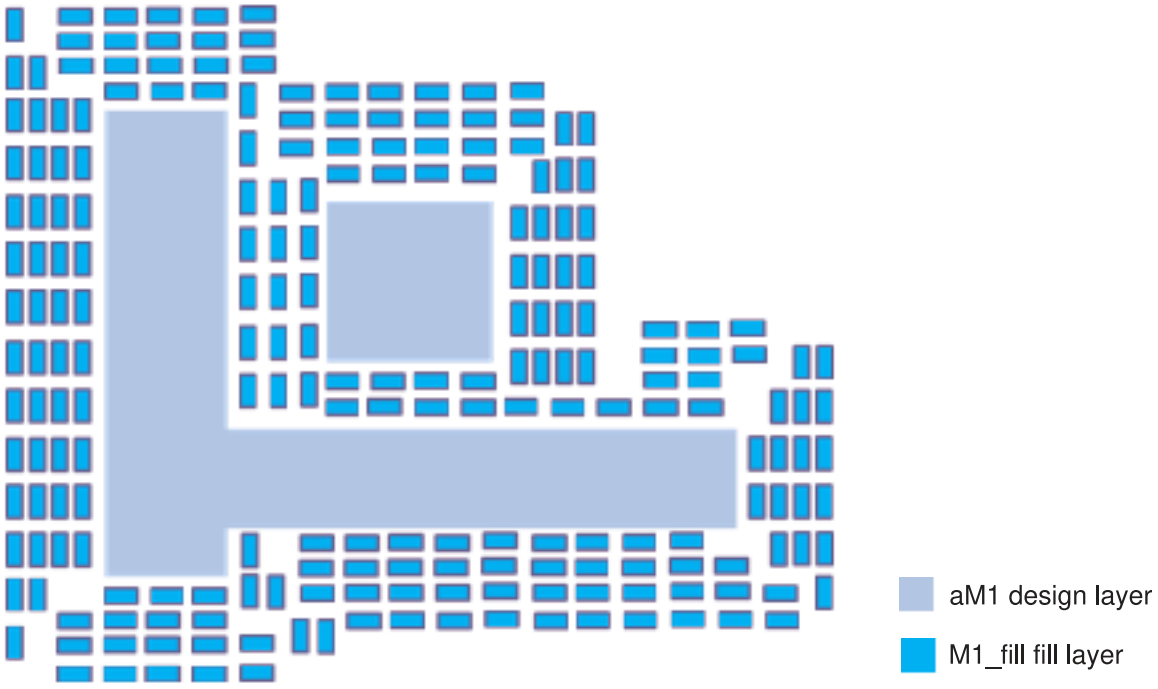
Feedback

Figure 80 Fill with ring_count = 2



Use the `signal` option only with the `UF_ADJUSTABLE` and `UF_POLYGON` types. With the `UF_POLYGON` type, the pattern must be a single layer with single shape. You must set the `fill_context` option to `SIGNAL_CONTEXT` in the definition of the `fill_patterns` argument. With the `ring_count` option of the `signal` option, you can control the number of rings of fill pattern shapes that surround each design layer shape. In Figure 80, the ring count is 2. Figure 81 and Example 24 show an example with a ring count of 4.

Figure 81 Fill with ring_count = 4



Example 24 Fill with ring_count = 4

```
DATA_M1_fill : list of list of coordinate_s = {
   { {0.0, 0.0}, {2.0, 0.0}, {2.0, 1.0}, {0.0, 1.0} }
```

Feedback

```
};
SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
      fill_to_signal_spacing = {
         {aM1, min_space = 0.1},
      }
   }
   polygons = DATA_M1_fill,
};

fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      pattern_spec = {space_x = 0.1,
                      space_y = 0.1},
      layers = {SHAPE_M1_fill},
      fill_context = SIGNAL_CONTEXT,
      signal = {layer = aM1,
                ring_count = 4}
   }
}}
```

You can use the `signal` option with the width-dependent spacing functionality that is described in the Range and Width Dependent Spacing section. Use this functionality in applications that require the fill shapes to be kept away from the design layer shape by different minimum space values that depend on the width of the design layer shape. Fill pattern rings are generated using all spacing constraints.

The following rules and restrictions apply for signal aligned fills:

- The `signal` option cannot be used with the `UF_STACK` or `UF_CELL` type.
- The `signal` option cannot be used when pattern definitions have multiple fill layers or if the fill-layer definition has multiple polygons.
- The `stagger_y` option is not supported with the `signal` option. The `stagger_y` option is ignored.
- The `fill_context` option must be `SIGNAL_CONTEXT`. With this setting, the `partition`, `insertion`, and `pitch` options are ignored.
- The `width_based_spacing` option is not supported for the design layer specified in the `signal` option. Use the `min_space` option in `fill_to_signal_spacing` to set the minimum spacing between the fill layer and the design layer specified in the `signal` option.

Feedback

Defining the Fill Pattern for Signal Aligned Fills

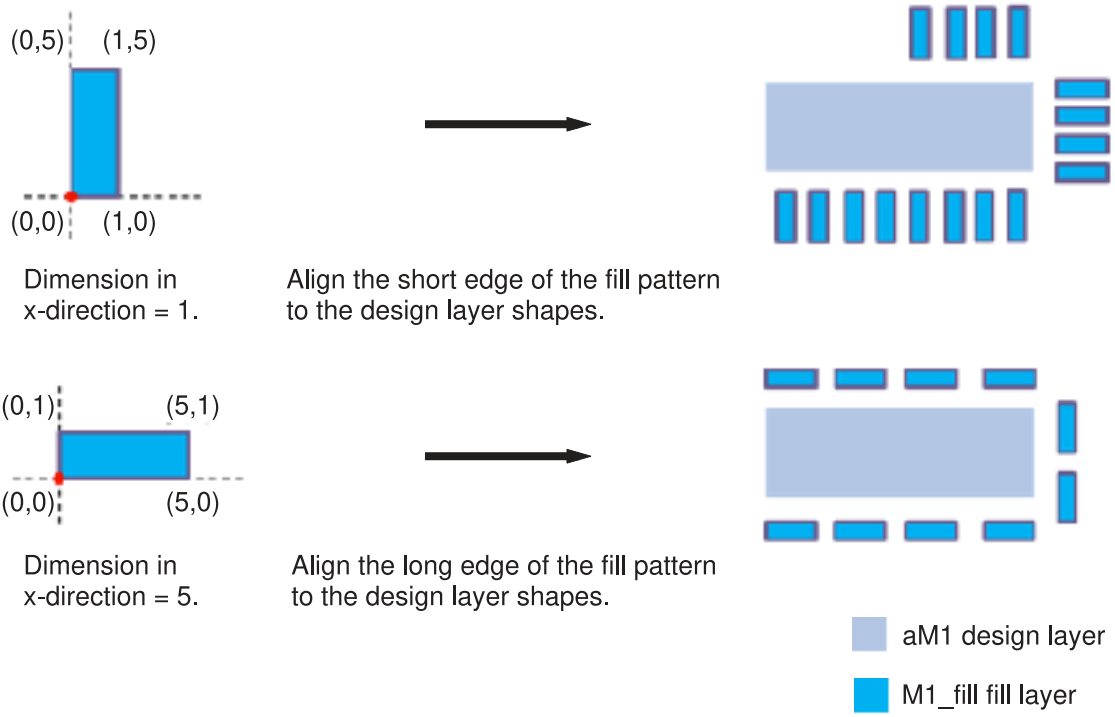
When you define a fill pattern using the coordinate list, make sure that the dimensions of the pattern in the x- and y-directions meet your requirements. The `signal` option in the `unified_fill()` function treats the x-direction as the major axis of the pattern. The tool aligns this edge of the pattern with the shapes on the design layer.

- To align the long edge of the fill pattern rectangle with the design layer shape, make sure that the dimension of the pattern in the x-direction is greater than the dimension in the y-direction.
- To align the short edge of the fill pattern rectangle with the design layer shape, make sure that the dimension of the pattern in the x-direction is less than the dimension in the y-direction.

Figure 82 illustrates the two scenarios. For simplicity, not all fill pattern placements are shown. For this example, the coordinate data is:

```
DATA_M1_fill_V : list of list of coordinates_s = {
   { {0.0, 0.0}, {1.0, 0.0}, {1.0, 5.0}, {0.0, 5.0} }
};
```

Figure 82 Aligning Short or Long Edge of the Fill Pattern

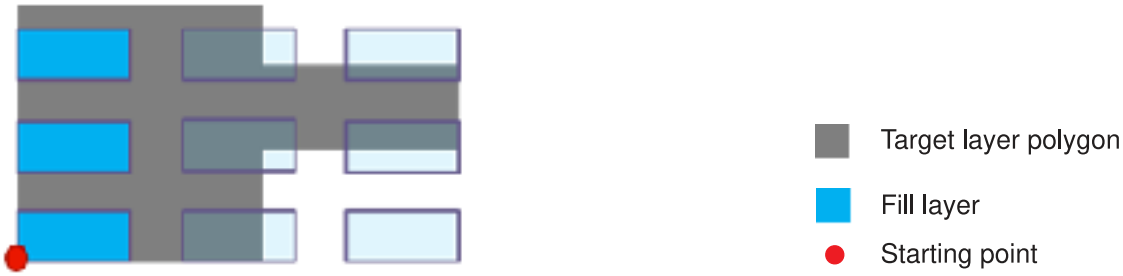


Feedback

Improving Pattern Insertion

Similar to the `fill_patterns` argument, the default starting point for pattern placement in the `unified_fill()` function is at the lower-left corner of the bounding box of the polygon on the target layer. When the polygons on the target layer are complex, nonrectangular shapes, the default placement might not always result in maximum insertion. Figure 83 shows how several patterns could not be placed in such a situation.

Figure 83 Default Placement That Is Not Maximum Insertion



The `unified_fill()` function provides several optional configurations to change the default starting point and allow more flexibility in pattern placement. These changes might improve the insertion rate.

Example 25 shows how to use the `iteration` option to improve the insertion rate.

Example 25 Using the iteration Option

```
fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      pattern_spec = {space_x = 0.1,
                      space_y = 0.1},
      layers = {SHAPE_M1_fill},
      insertion = {iterations = 3}
   }
}}
```

For a detailed description of the `insertion` option, see the `unified_fill()` function in the *IC Validator Reference Manual*.

Pitch Aligned Fills

From the `fill_pattern()` function, you can control the location of the pattern grid with the `grid_mode` option.

Feedback

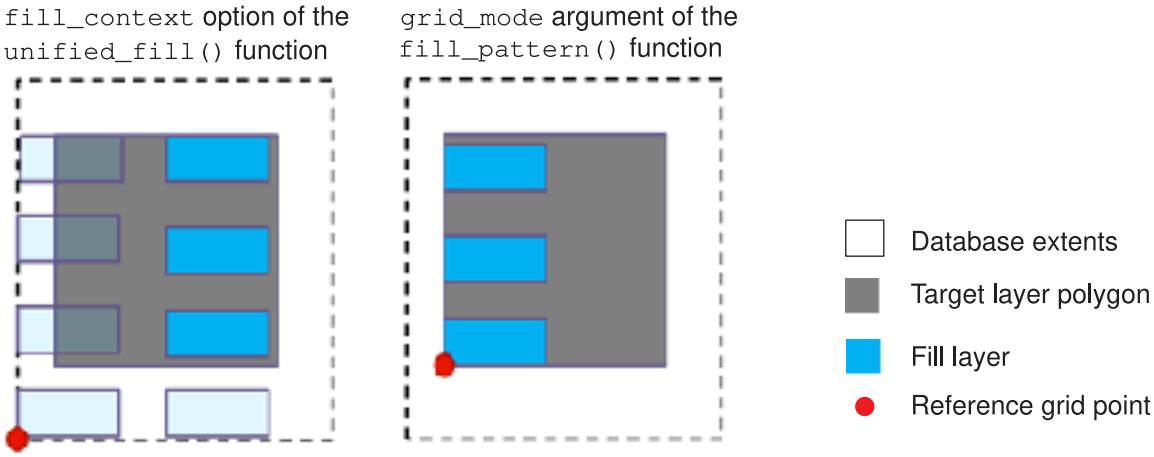
- `Specify GLOBAL` to place the fill shapes by using the lower-left of the entire database extents as the reference.
- `Specify LOCAL` to place fill shapes by allowing every polygon on the input layer to have its own unique reference grid point.

The reference grid point is the lower-left of the bounding box of every individual polygon belonging to the input layer.

With the `unified_fill()` function, in addition to the `GLOBAL` and `LOCAL` grid modes, you can enforce more complex control over the location of the pattern grid by referencing another layer. Along with the target layer, you can also specify a context layer. When a context layer is used, fill patterns are placed inside the target layer polygons in such a way that the lower-left vertex of the bounding box of the fill patterns aligns with the grid points originating from the lower-left vertex of the bounding box of each individual polygon on the context layer as the reference grid point. A context layer polygon can encompass multiple polygons on the target layer.

Figure 84 shows a comparison of the `grid_mode` argument of the `fill_pattern()` function with the `fill_context` option of the `unified_fill()` function.

Figure 84 Pitch Aligned Fills Comparison



Example 26 and Figure 85 show the behavior of the `pitch` option of the `unified_fill()` function. In this example, the `pitch` option is applied only to the y-direction grid point. The fill patterns in a given target layer polygon are placed starting from the grid point that is closest to the lower-left vertex of the bounding box of the specific target layer polygon, satisfying the specified `pitch` value.

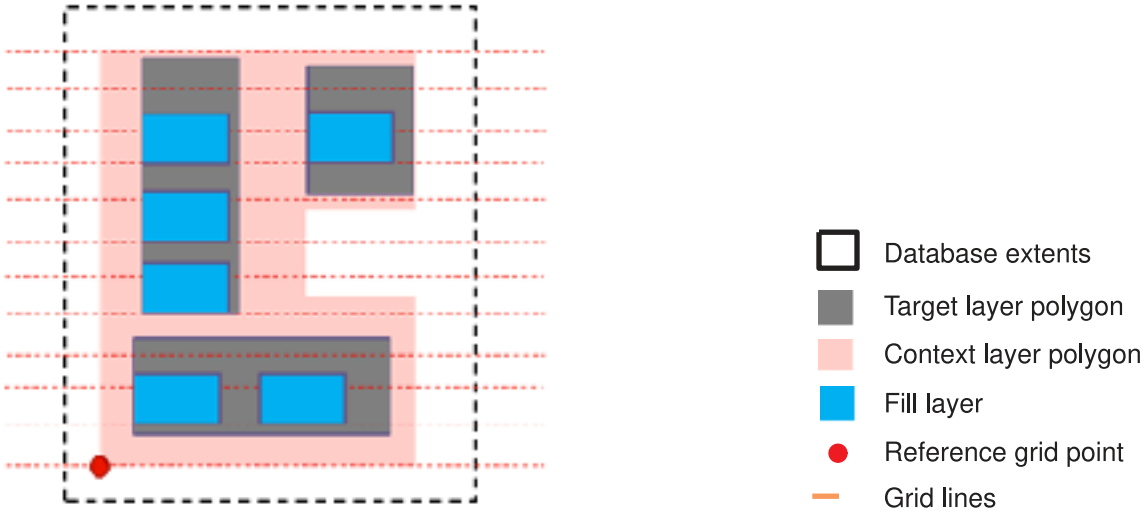
Example 26 Applying the Pitch Option Only in the Y-Direction

```
fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
```

Feedback

```
        pattern_spec = {space_x = 0.1,
                        space_y = 0.1},
        layers = {SHAPE_M1_fill},
        pitch = {y = 0.1,
                 context_layer = aFILL_CONTEXT_LAYER}
    }
}}
```

Figure 85 Applying the Pitch Option Only in the Y-Direction



The following rules and restrictions apply for pitch-aligned fills:

- Target layer polygons that are completely outside the context layer are discarded

  Fill patterns are not inserted in these polygons.

- If a target layer polygon overhangs context layer polygons, only fill-pattern polygons that are fully enclosed by the context layer are inserted
- Fill patterns are not inserted in regions of the target layer that interact with multiple polygons on the context layer
- When the `pitch` option is used, the `fill_context` option is ignored if it is set to `CHIP_CONTEXT` or `REGION_CONTEXT` because these settings are not relevant in this case
- The `pitch` value must correlate with the pattern dimension and the `space_x` and `space_y` values

  For example, the following value must be a multiple of the `pitch` value; otherwise, the resultant fill might be missing patterns:

  ((pattern dimension in x-direction) + `space_x`)

Feedback

For a higher insertion rate, you can increase the `shift_factor` value of the `insertion` option. The `unified_fill()` function automatically adjusts the fill patterns inside the target layer polygons such that they align with the grid lines of the context layer polygon. The `shift_factor` option is useful when polygons on the target layer are complex, nonrectangular shapes. Example 27 shows how to use the `shift_factor` option.

Example 27 Using the shift_factor Option

```
fill_patterns = {{
   type = UF_POLYGON,
   polygon_fill = {
      pattern_spec = {space_x = 0.1,
                      space_y = 0.1},
      layers = {SHAPE_M1_fill},
      insertion = {shift_factor = 5}
      pitch = {y = 0.1,
               context_layer = aFILL_CONTEXT_LAYER},
   }
}}
```

Criteria Analysis

With the `unified_fill()` function, you can customize the fill procedure for metrics such as density. The tool supports equation-based criteria analysis to meet requirements such as density ranges, target density, and uniform gradient density across the design. The `unified_fill()` function uses the same algorithms as the `density()` function, ensuring that the fill results meet the postfill design rule requirements.

Use the following arguments of the `unified_fill()` function to configure the fill procedure to meet your requirements:

`criteria`

`delta_window`

`delta_x`

`delta_y`

`window_layer`

`boundary`

The `criteria` option supports the following options to configure the fill procedure:

`target`

`gradient`

`fill_layer_keys`

Feedback

```
design_layers

design_layers_hash

window_function
```

The string names specified in the `output_layer_key` option of the pattern definition are used to identify the fill layers used in criteria equations. You can specify any number of criteria equations in the `criteria` argument. If you have multiple criteria equations for the same fill layer, the tool ensures that at least one of the equations is met. If there are multiple criteria equations for different fill layers (for example, aM1_FILL and aM2_FILL each have a criteria equation defined), then the tool addresses all the equations.

When the `criteria` argument is specified without a window function, the tool evaluates a default equation to check against the criteria value. This default equation is evaluated for every delta window and checked against the `target` and `gradient` values of the `criteria` argument. The fill patterns are evaluated to converge the evaluated value to the specified value. The default equation is:

[(Area of fill layers) + (Area of design layers)] / (Area of delta window)

Where,

- *Area of fill layers* is the sum of areas within the current delta window of all polygon data on all the fill layers that are specified in the `fill_layer_keys` option.
- *Area of design layers* is the sum of areas within the current delta window of all polygon data on all the design layers that are specified in the `design_layers` option.

Note:

Pattern definitions must generate fill output that allows the criteria to contribute. For example, the tool cannot meet a higher target density when the provided pattern definitions are composed of narrow, widely-spaced rectangles.

Example 28 shows a fill specification for two fill layers, M1_fill and M2_fill, that have their own criteria equations. The cumulative density of the existing aM1 design layer and M1_fill fill layer should be between 25 percent and 45 percent with a target of 35 percent, when measured in 100 µm x 100 µm windows stepped across the design in increments of 50 µm along the x- and y-directions. The M2_fill fill layer has a similar fill specification with the same density values.

Example 28 Fill Specification With Two Criteria Equations

```
DATA_M1_fill : list of list of coordinate_s = {
   { {1.0, 0.0}, {2.0, 0.0}, {2.0, 3.0}, {1.0, 3.0} }
};

SHAPE_M1_fill : polygon_layer_s = {
   layer_spec = {output_layer_key = "M1_fill"}
   polygons = DATA_ M1_fill,
```

Feedback

```
};

DATA_M2_fill : list of list of coordinate_s = {
   { {0.0, 1.0}, {3.0, 1.0}, {3.0, 2.0}, {0.0, 2.0} },
};

SHAPE_M2_fill : polygon_layer_s = {
   layer_spec = {output_layer_key = "M2_fill"}
   polygons = DATA_ M2_fill,
};

fill_patterns_m1_m2 = {{
   type = UF_POLYGON,
   polygon_fill = {
      layers = {SHAPE_M1_fill,
                SHAPE_M2_fill}
   }
}}

M1_density_criteria : fill_criteria_s =
{
   target = {range = [0.25, 0.45], target = 0.35},
   fill_layer_keys = {"M1_fill"},
   design_layers = {aM1}
};
M2_density_criteria : fill_criteria_s =
{
   target = {range = [0.25, 0.45], target = 0.35},
   fill_layer_keys = {"M2_fill"},
   design_layers = {aM2}
};
uf_fill_output_h = unified_fill(
   fill_patterns = fill_patterns_m1_m2,
   criteria = {M1_density_criteria,
               M2_density_criteria},
   delta_window = {100, 100},
   delta_x = 50,
   delta_y = 50,
   fill_boundary = {type = LAYER, layer = data_extent},
   window_layer = aWINDOW,
   grid = 0.001,
   boundary = ALIGN
);
M1_fill_output = uf_fill_output_h["M1_fill"][0];
M2_fill_output = uf_fill_output_h["M2_fill"][0];
```

Example 29 is another fill example that uses multiple design layers and fill layers in the same criteria equation. The sum of the densities in every window of M1, M2, and M3 for both design and fill should be greater than or equal to 105 percent.

Note:

For simplicity, Example 29 shows only the criteria equations.

Feedback

Example 29 Fill Specification With Multiple Layers in the Same Criteria Equation

```
M1_M2_density_criteria : fill_criteria_s =
{
   target = {range = >= 1.05},
   fill_layer_keys = {"M1_fill", "M2_fill", "M3_fill"},
   design_layers = {aM1, aM2, aM3}
};
```

For a specific fill layer, when there is more than one fill pattern specified, the `unified_fill()` function sequentially inserts the fill patterns in the order that the patterns are declared in the function. When a criteria is specified, it is possible that the `unified_fill()` function does not make use of all the fill pattern definitions. That is, the `unified_fill()` function stops inserting fill as soon as the criteria is met regardless of whether all the fill patterns have been used.

In Example 30, the procedure for the M1_fill fill layer is composed of two insertion sequences. After inserting the first pattern, if the criteria is met, the tool does not perform the second sequence.

Example 30 Sequence of Pattern Definitions

```
fill_patterns_m1 = {
{
   type = UF_POLYGON,
   polygon_fill = {layers = {SHAPE_M1_fill_TYPE1}}
},
{
   type = UF_POLYGON,
   polygon_fill = {layers = {SHAPE_M1_fill_TYPE2}}
}}
M1_density_criteria : fill_criteria_s =
{
   target = {range = [0.25, 0.45], target = 0.35},
   fill_layer_keys = {"M1_fill"},
   design_layers = {aM1}
};
uf_fill_output_h = unified_fill(
   fill_patterns = fill_patterns_m1,
   criteria = {M1_density_criteria},
   delta_window = {100, 100},
   delta_x = 50,
   delta_y = 50,
   fill_boundary = {type = LAYER, layer = data_extent},
   window_layer = aWINDOW,
   grid = 0.001,
   boundary = ALIGN
);
M1_fill_output = uf_fill_output_h["M1_fill"][0];
```

Instead of using the default criteria equation, you can specify your own criteria by defining the window function as shown in Example 31.

Feedback

Example 31 Customized Criteria Equation

```
design_layers : layer_groups_h = {
   "aM1" => aM1
};
M1_density_criteria_func : function ( void ) returning void
{
   da_aM1 = uf_area(DESIGN, "aM1");
   fa = uf_area(FILL, "M1_fill");

check_value : double = (da_aM1 + fa)) / uf_window_area();
   uf_save_window(check_value);
};
M1_density_criteria : fill_criteria_s =
{
   target = {[0.25, 0.45]},
   fill_layer_keys = {"M1_fill"},
   design_layers_hash = design_layers,
   window_function = M1_density_criteria_func
};
```

See the Unified Fill Utility Functions section in the “Utility Functions” chapter of the *IC Validator Reference Manual* for more information about the utility functions you can use in the window function.

The following rules and restrictions apply:

- Multiple fill layers cannot be used in a single window function
- If target is not specified, the `unified_fill()` function uses the midpoint of the range as its internal target
- A valid constraint that converges must be specified. For example, the following constraint is not allowed

  ```
  target = {range = <0.50}
  ```

The following examples show the behavior based for several scenarios:

- `target = {range = >0.50}`

  The `unified_fill()` function tries to achieve a target density greater than 50 percent.

- `target = {range = [0.45, 0.55]}`

  The `unified_fill()` function assumes an internal target density of 50 percent. Fill patterns are not added to those windows for which the initial density is already greater than 50 percent. Otherwise, the tool works toward reaching a target density of 50 percent, but not exceeding 50 percent.

- `target = {range = [0.45, 0.55], target = 0.48}`

Feedback

The `unified_fill()` function works toward achieving a target density of 48 percent. If the initial density in a window is 46 percent, the tool works toward reaching a density 48 percent, but not exceeding 48 percent.

- `target = {range = >0.30, target = 0.50}`

  The allowed range is greater then 30 percent and less than or equal to 100 percent. The tool works toward achieving a target density of 50 percent, but not exceeding 50 percent.

Output Layers

The `unified_fill()` function returns a hash containing layer lists, with each list having three polygon layers:

Index	Polygon layer
0	Uncolored
1	Color 1 (The `color` option must be `true` to get this layer.)
2	Color 2 (The `color` option must be `true` to get this layer.)

To retrieve a list of polygon layers, you specify the `output_layer_key` value that was used by the `layer_spec` option of the fill pattern definition. If the `color` option is `false`, only uncolored data is generated on index 0. If the `color` option is `true`, then the two colored output layers are returned on indexes 1 and 2, respectively. For example,

```
M1_fill_ALL = M1_fill_h["M1_fill"][0];
M1_fill_COLOR1 = M1_fill_h["M1_fill"][1];
M1_fill_COLOR2 = M1_fill_h["M1_fill"][2];
```

Example 32 is a procedure for two fill layers, M1_fill and M2_fill, that shows how the fill layers are accessed from the output hash generated by the `unified_fill()` function.

Example 32 Accessing Fill Layers From Hash

```
fill_patterns_m1_m2 = {{
   type = UF_POLYGON,
   polygon_fill = {
      layers = {SHAPE_M1_fill,      //output_layer_key is "M1_fill"
                SHAPE_M2_fill}      //output_layer_key is "M2_fill"
   }
}}
uf_fill_output_h = unified_fill(
   fill_patterns = fill_patterns_m1_m2
);
```

Feedback

```
M1_fill_output = uf_fill_output_h["M1_fill"][0];
M2_fill_output = uf_fill_output_h["M2_fill"][0];
```

Example 33 shows a fill specification for the M1_fill fill layer that has two pattern insertion steps. Each pattern definition has a specified color. The `unified_fill()` function creates fill shapes in each pattern insertion step by coloring them as individual sets. Furthermore, both pattern insertion steps use the same string name in the `output_layer_key` option to designate the fill layer. The `unified_fill()` function accumulates all polygon data from different pattern definitions into the same polygon layer list element in the output hash key of that fill layer. In this example, the value of list index 0 of the M1_fill output hash key has complete polygon data from both pattern insertion steps. The value of list index 1 has complete color 1 polygon data from both pattern insertion steps. Similarly, the value of list index 2 has complete color 2 polygon data from both pattern insertion steps.

Example 33 Pattern Insertion Using the color Option

```
SHAPE_M1_fill_TYPE1 : polygon_pattern_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
   }
   polygons = DATA_M1_fill_TYPE1,
};
SHAPE_M1_fill_TYPE2 : polygon_pattern_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
   }
   polygons = DATA_M1_fill_TYPE2,
};
M1_fill_h = unified_fill(
   fill_patterns = {
     {type = UF_POLYGON,
      polygon_fill = {
         layers = {SHAPE_M1_fill_TYPE1},
         color = true
      }
     },
     {type = UF_POLYGON,
      polygon_fill = {
         layers = {SHAPE_M1_fill_TYPE2},
         color = true
      }
     }
   },

   fill_boundary = {type = LAYER, layer = data_extent}
);

M1_fill_ALL    = M1_fill_h["M1_fill"][0];
M1_fill_COLOR1 = M1_fill_h["M1_fill"][1];
M1_fill_COLOR2 = M1_fill_h["M1_fill"][2];
```

Feedback

Apart from the fill layers, you can also retrieve the bounding box, that is, the extents, of one or more output fill layers by specifying one or more of the appropriate `output_layer_key` values. Example 34 and Figure 86 show the usage of the `extents_output` option.

Example 34 Using the extents_output Option

```
SHAPE_M1_fill : polygon_pattern_s = {
   layer_spec = {
      output_layer_key = "M1_fill",
   }
   polygons = DATA_M1_fill_TYPE1,
};

M1_fill_h = unified_fill(
   fill_patterns = {{
     {type = UF_POLYGON,
      polygon_fill = {
         layers = {SHAPE_M1_fill},
         color = true
      }
     }
   },

   extents_output = {
     {output_layer_key = "M1_fill_boundary",
      fill_layer_keys = {"M1_fill"}
     }
   }
);

M1_fill   = M1_fill_h["M1_fill"][0];
M1_fill_E = M1_fill_h["M1_fill_boundary"][0];
```

Figure 86 Using the extents_output Option



When the `output_layer_key` option is not specified in the `extents_output` option, the bounding box of the pattern is constructed by considering all the fill layers present in the

Feedback

pattern placement. For patterns that are generated by the `UF_STACK` or `UF_CELL` type, the `extents_output` option works in accordance with the `separate_patterns` option.

Layer Compression

The geometry data on output polygon layers generated by the `unified_fill()` function can be grouped hierarchically to reduce the size of the output file created by functions such as `write_gds()` and `write_milkyway()`. Layer compression can be achieved in two ways in an IC Validator runset:

- Specify the `hierarchical_fill` option inside pattern definitions of the `unified_fill()` function.
- Use the `compress_fill` option in the `write_gds()` and `write_milkyway()` functions.

The `hierarchical_fill` option in the `unified_fill()` function is similar to the `output_aref` option in the `fill_pattern()` function. However, for optimum output file size, use the `compress_fill` argument in the `write_gds()` and `write_milkyway()` functions for all the individual polygon layers.

12

Working With Edges

This chapter gives a brief overview of using edges in IC Validator runsets. The edge functions are described in the IC Validator Reference Manual.

The IC Validator tool provides for

- Associating Edge Layers With a Parent Layer
- Topological Operations
- Logical Operations
- Edge Manipulation

Feedback

Associating Edge Layers With a Parent Layer

Edge layers can have a parent polygon layer.

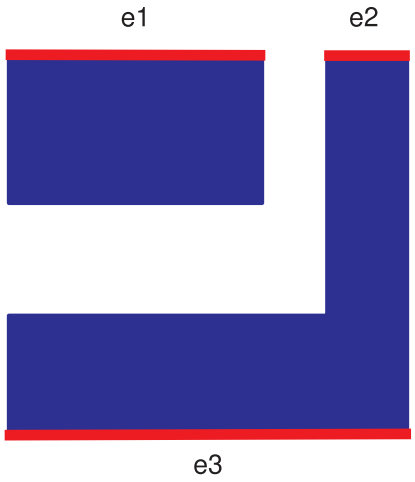
- All edge selection operations maintain the connection between an edge layer and the polygon layer from which it was derived.
- All edge creation operations result in “orphan” edge layers, that is, edge layers without a parent polygon layer.
- The parent polygon layer provides a more intuitive output by influencing dimensional function behavior for edges.

For example, Figure 87 shows edge association with a parent layer.

```
red = edge_derived_from_blue;
@ "red edge min width rule";
internal1(red, spacing < x);
```

The IC Validator tool detects the violation from e2 to e3 and avoids the violation from e1 to e3.

Figure 87 Edge Association With a Parent Layer



Topological Operations

Topological operations use the complete edges selected and do not add new endpoints. These operations include

- `adjacent_edge()` and `not_adjacent_edge()`
- `angle_edge()` and `not_angle_edge()`

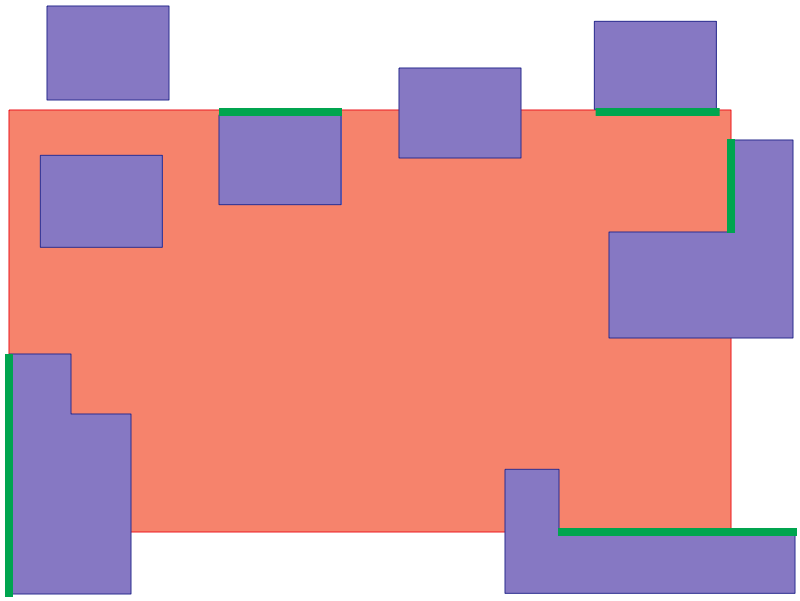
Feedback

- `inside_point_touching_edge()` and `not_inside_point_touching_edge()`
- `inside_touching_edge()` and `not_inside_touching_edge()`
- `interacting_edge()` and `not_interacting_edge()`
- `length_edge()` and `not_length_edge()`
- `outside_point_touching_edge()` and `not_outside_point_touching_edge()`
- `outside_touching_edge()` and `not_outside_touching_edge()`
- `point_touching_edge()` and `not_point_touching_edge()`
- `touching_edge()` and `not_touching_edge()`
- `vertices_edge()` and `not_vertices_edge()`

Figure 88 shows an example of touching edges:

```
green = touching_edge (layer1 = blue, layer2 = red);
```

Figure 88 Touching Edges Example



Logical Operations

Logical operations generate new endpoints. These operations include

- `coincident_edge()` and `not_coincident_edge()`
- `coincident_inside_edge()` and `not_coincident_inside_edge()`

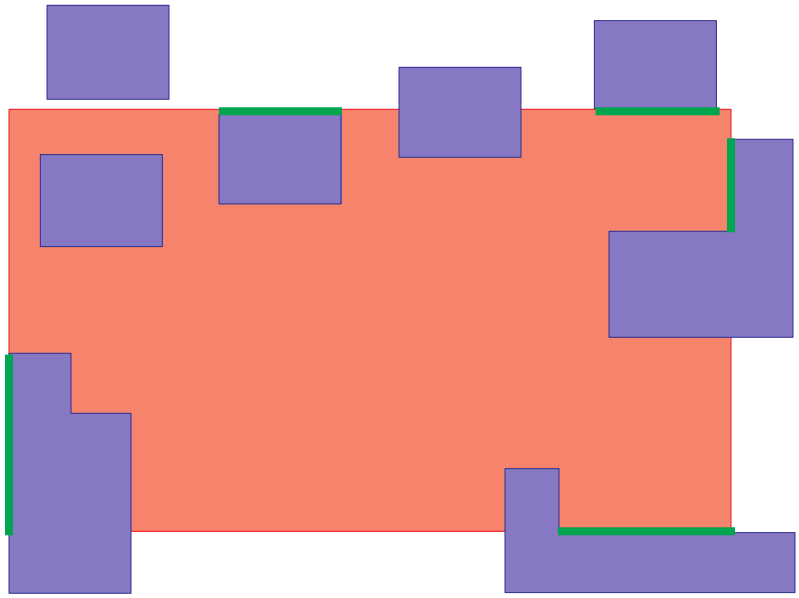
Feedback

- `coincident_outside_edge()` and `not_coincident_outside_edge()`
- `and_edge()`
- `not_edge()`
- `or_edge()`
- `xor_edge()`

Figure 89 shows an example of coincident edges:

```
green = coincident_edge (layer1 = blue, layer2 = red);
```

Figure 89 Coincident Edges Example



Edge Manipulation

These functions create edges, which are always orphan edges:

- `move_edge()`
- `snap_edge()`
- `extend_edge()`
- `copy_edge()`

This function creates polygons from edges:

- `edge_size()`

13

Critical Area Analysis Flow

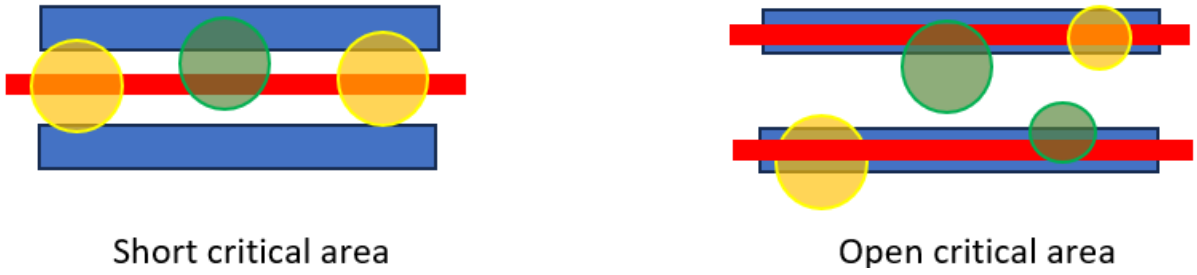
This chapter provides an introduction to the IC Validator critical area analysis flow.

The IC Validator critical area analysis (CAA) flow allows you to identify areas of layout which are sensitive to failures caused by random contaminants presenting in manufacturing facilities.

Critical area is defined as an area of layout where if center of defect of given size is located might cause short or open failure of circuit.

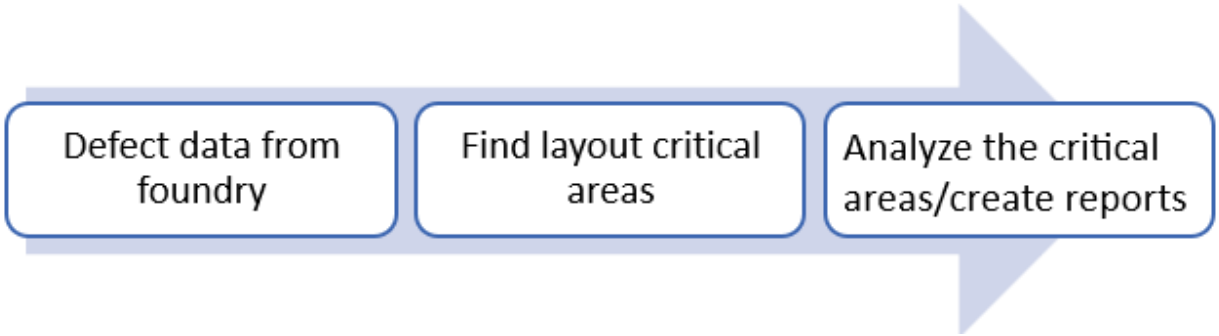
The CAA flow currently supports two type of defects – OPEN and SHORT.

Figure 90 Critical Area by Defect Types



The critical area analysis flow can be illustrated by following diagram:

Figure 91 Critical Area Analysis Flow



Feedback

1. Defect data from foundry: The starting point of CAA flow is data provided by foundry about manufacturing defects and formula for defect sizes calculation.

   Defect data includes range of defect sizes typically defined by 3 parameters - minimal and maximal defect sizes – `Dmin` and `Dmax`, number of defect size samplings, and coefficients for defect sizes calculation within given range of (`Dmin`, `Dmax`).

2. Find layout critical areas: The `critical_areas()` function is used to determine critical areas of layout. For more information see, the `critical_area()` function in the *IC Validator Reference Manual*.

3. Analyze the critical areas/create reports: After the critical areas of layout are identified and presented as polygon layers, that data can be used in the runset to calculate weighted critical area (WCA) and average quality.

   The WCA is calculated as a

   $$WCA = K^* \left(\frac{C1}{X1^3} + \frac{C2}{X2^3} + \ldots + \frac{C11}{X11\text{^}3}\right)$$

   where K-factor calculation is provided by you in the runset.

   The average quality usually is calculated as follows:

   $$AVERAGE_QUALITY = 1 - \frac{WCA}{AREA}$$

   where WCA is weighted critical area for given layout layer and AREA is the total area of layer.

   The calculated data can be used to generate reports and heat maps. The VUE tool should be used to generate heat maps.

   The simplest critical area report can be generated by the `dfm_feature()` function and output to **LAYOUT_ERRORS** file for analyzed layers.

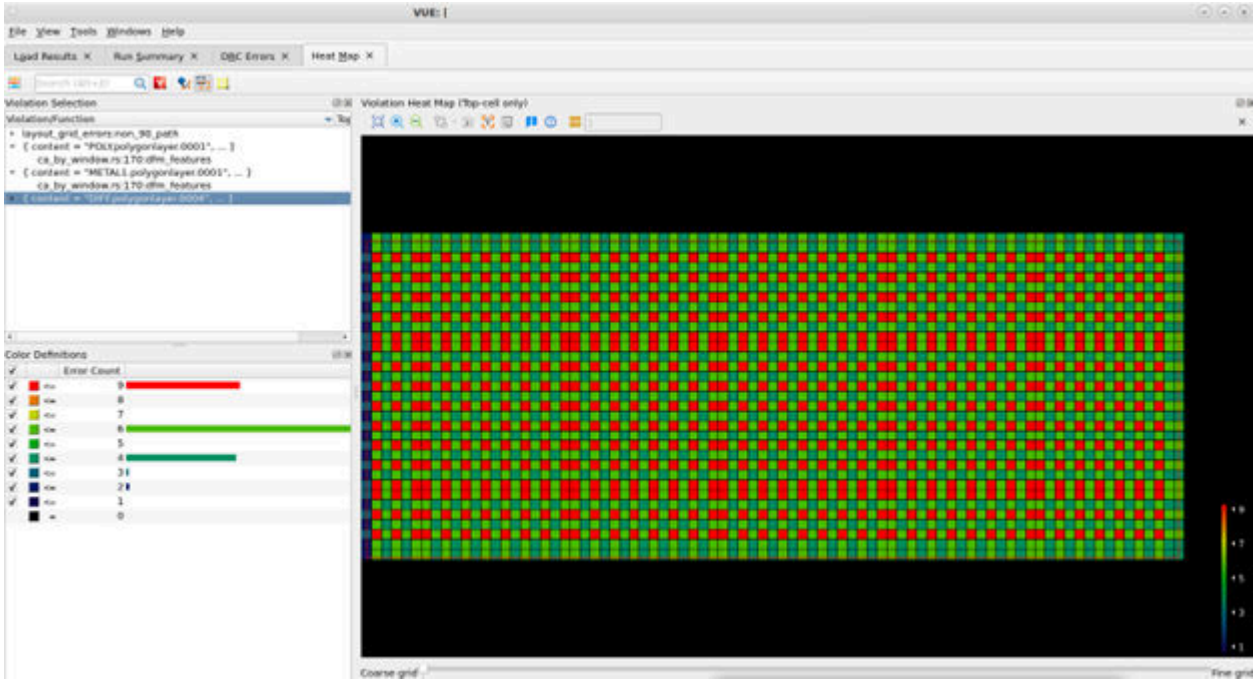
```
chip area:                  = 1543.3206
Critical_Area[0]:              = 0.0000
Defect Size[0]:                = 0.0110
Critical_Area[1]:              = 0.0000
Defect Size[1]:                = 0.0120
Critical_Area[2]:              = 0.0000
Defect Size[2]:                = 0.0190
Critical_Area[3]:              = 0.0000
Defect Size[3]:                = 0.0380
Critical_Area[4]:              = 0.0000
Defect Size[4]:                = 0.0740
Critical_Area[5]:              = 0.0000
Defect Size[5]:                = 0.1350
Critical_Area[6]:              = 1.4728
Defect Size[6]:                = 0.2250
Critical_Area[7]:              = 4.9703
```

Feedback

```
Defect Size[7]:                 = 0.3500
Critical_Area[8]:               = 17.1042
Defect Size[8]:                 = 0.5170
Critical_Area[9]:               = 90.4452
Defect Size[9]:                 = 0.7320
Critical_Area[10]:              = 200.9118
Defect Size[10]:                = 1.0000
WCA:                            = 0.0005
DIFF_CA_OPEN AVERAGE QUALITY:   = 1.0000
```

The critical area heat maps are generated by the VUE tool as shown in Figure 92.

Figure 92 Heat Maps in VUE



IC Validator Critical Area Analyzer Runset

The following PXL-structures present a possible approach to organize data in the CAA runset:

```
ca_defect_range_s : newtype struct of {
  count : integer;
  min : double;
  max : double;
};
ca_defect_density_s : newtype struct of {
  na : integer;             // dummy
  fall_power : double;
  conversion_factor : double;
};
```

Feedback

```
ca_config_s : newtype struct of {
  layer : polygon_layer;
  mode : critical_area_mode_e;
  modeling : integer     // dummy
  range : ca_defect_range_s;
  defect_density : ca_defect_density_s;
};
ca_layer_configuration_l : newtype list of ca_config_s;
```

Then the typical CAA runset would be as follows:

```
#include <icv.rh>
library("input_desing.gds", GDSII, "top");

// assign section
DIFF = assign({ { 3 } });
POLY = assign({ { 3 } });
METAL1 = assign({ { 3 } });

connect_layer = assign({ { 4 } });

// types to store defect sizes;
defect_sizes_list : newtype list of double;
defect_sizes_h : newtype hash of integer to defect_sizes_list;
defect_hash : defect_sizes_h = {};
layer_dr : list of double = {};
layer_tagname : string = "";
fall_power : double = 0.0;
ca_mode : critical_area_mode_e;

g_ca_layers : string_to_geometry_layer_h = {};

// the remote function to calculate the WCA and average quality and
 output report
get_area : function (void) returning void
{
    dfm_report_double("chip area: ", dfm_window_area());

    wca_tmp: double = 0;

    // calculate K-factor
    // put actual K-calculation code here
    K = 6.0e-07;

    // for each input layer the critical_areas() command creates
    // list of critical area layers - one layer per defect size

    for (idx = 0 to layer_dr.size() - 1) {
      this_area = dfm_sum_area(""+idx);
      dfm_report_double("Critical_Area["+idx+"]: ", this_area);    //
 output critical area value
```

Feedback

```
      dfm_report_double("Defect Size["+idx+"]: ", layer_dr[idx]);  //
 output defect size

      // calculate sum of WCAs for all defect sizes for given input layer
      wca_tmp= wca_tmp + (this_area / (layer_dr[idx]**fall_power));
    }

    // final WCA calculation
    wca = K * wca_tmp;

    dfm_report_double("WCA: " , wca);   // report WCA

    average_quality : double = 1 - (wca / dfm_window_area());      //
 calculate average quality
    dfm_report_double(layer_tagname + "_" + ca_mode + " AVERAGE QUALITY:
 ",
                                          average_quality);   // report
 average quality for one input layer

    dfm_save_data();
}


ca_defect_range_s : newtype struct of {
  count : integer;
  min : double;
  max : double;
};
ca_defect_density_s : newtype struct of {
  na : integer;            // dummy
  fall_power : double;
  conversion_factor : double;
};
ca_config_s : newtype struct of {
  layer : polygon_layer;
  mode : critical_area_mode_e;
  modeling : integer     // dummy
  range : ca_defect_range_s;
  defect_density : ca_defect_density_s;
};

cas : ca_layer_configuration_l = {
   {DIFF,CA_OPEN,MODELING,{10,0.022,2.000},{NA,2.5,1e-8}},
   {POLY,CA_OPEN,MODELING,{10,0.022,2.000},{NA,2.5,1e-8}},
   {METAL1,CA_OPEN,MODELING,{10,0.022,2.000},{NA,2.5,1e-8}},
};

get_defect_sizes : function   // function for defect size calculation
(
   min_size  : double,
   max_size  : double,
   steps     : integer,
   fall_power : double
```

Feedback

```
)  returning defect_list : defect_sizes_list
{
/*
    // put actual defect sizes calculation coded by user
*/
   defect_list = {0.011, 0.013, 0.017, 0.035, 0.073, 0.13, 0.24, 0.33,
 0.495, 0.74, 1.0};
}

di : integer = 0;
foreach (config in cas)
{
   //  calculate defect size
   one_list = get_defect_sizes(config.range.min, config.range.max,
 config.range.count,

 config.defect_density.fall_power);
   defect_hash[di] = one_list;

   di = di + 1;
}

output_layers: list of write_layer_map_s = {};


di = 0;
layer_n : integer = 1;
foreach (config in cas)
 {
  outs : list of polygon_layer;

  if(config.mode == CA_SHORT)
  {
      // for SHORT mode connect_db is required
      cdb = connect({{{config.layer}, connect_layer}});
      outs = critical_areas( config.layer, defect_hash[di], config.mode,
 cdb);
  }
  else
      outs = critical_areas( config.layer, defect_hash[di], config.mode);

  for(i = 0 to defect_hash[di].size()-1) {
    g_ca_layers[""+i] = outs[i];

    output_layers.push_back( {outs[i], {layer_n}} );
    layer_n = layer_n + 1;
  }

  g_ca_layers_keys = g_ca_layers.keys();

  layer_dr = defect_hash[di];
  fall_power = config.defect_density.fall_power;
  ca_mode = config.mode;
```

Feedback

```
  layer_tagname = config.layer.tagname;

  {@ ""+config.layer; dfm_features(g_ca_layers, get_area);}
}

// write out CA layers
gds_file = gds_library("CA_out.gds");
write_gds
(
  output_library = gds_file,
  layers = output_layers
);
```

14

DRC Error Classification

This chapter explains the DRC error classification capabilities of the IC Validator tool and how to use this feature.

The DRC error classification capabilities are described in the following sections:

- Using DRC Error Classification
- DRC Error Classification Flow
- Using PERC Error Classification

Feedback

Using DRC Error Classification

DRC error classification allows you to classify and annotate errors found during an IC Validator run and to export the errors into a database for use in subsequent IC Validator runs. Errors can be classified individually, in groups using VUE, or in bulk using the pydb_report utility.

Error classifications and user comments can be retained across multiple IC Validator runs by using a persistent error classification database (cPYDB). Error data from the subsequent run, as well as the classification data, is stored in SQLite databases.

For this flow, use the `error_options()` function. This function has arguments that are specifically for DRC error classification:

- `match_errors`: Specifies the list of error classification databases to import for matching during an IC Validator run. For each error classification database listed, the suppression of matched errors and reporting of unmatched errors can be controlled. (Unmatched errors are errors that exist in an imported error classification database but are not produced by the current IC Validator run.)
- `mustfix_errors`. Designates certain violation comments and cells as must-fix. You cannot classify these comments and cells as Ignore or Waive. This setting applies for the current run and is not retained in the error classification database.
- `comment_match_delimiter`. Specifies a delimiter string for violation comment matching in imported error classification databases.
- `match_errors_processing_mode`. Specifies whether the IC Validator tool uses cell level or hierarchical processing to match classified violations in an error classification database. The default is `CELL_LEVEL`.
- `violation_options`. Allows certain global error options to be overridden on a rule by rule basis. For example, if most rules are matched in `HIERARCHICAL` mode, but a few need to run in `CELL_LEVEL` mode, this argument allows for that.

Use the pydb_report and pydb_export utilities to classify errors, and then reuse the classification data. See pydb_report Utility and pydb_export Utility for more information. These utilities are in the IC Validator installation bin directory. You can also use VUE to classify and export errors. See the *IC Validator VUE User Guide*.

Hierarchy Considerations in CELL_LEVEL Mode

During preprocessing, the IC Validator function makes hierarchy optimizations. As a result, when error matching is done in `CELL_LEVEL` mode, errors could possibly be inconsistent between runs. To help prevent inconsistent hierarchy between runs, when an error classification database is imported, any cells defined within an error classification database are automatically protected from explosion during preprocessing. However,

Feedback

some optimizations create hierarchy, or explode or flatten individual placements of cells, and therefore, can create consistency problems.

When running the DRC error classification flow in `CELL_LEVEL` mode, always set the `flow` argument in the `hierarchy_auto_options()` function to the `ERROR_CLASSIFICATION` option.

See the `hierarchy_auto_options()` function.

Hierarchical DRC Error Classification

The IC Validator tool provides a `HIERARCHICAL` mode for error classification matching that is more robust and less sensitive to hierarchy changes than `CELL_LEVEL` mode. This mode puts no restrictions on hierarchy optimization. Cells that have classified errors in the error classification database (cPYDB) are allowed to explode.

An error is considered a match for a classified error from the cPYDB if the complete hierarchically formed shape of the errors produced by the IC Validator tool exactly overlaps the complete hierarchically formed shape of errors from the cPYDB with the same classification. If any part of the complete shape is different between the errors found by the IC Validator tool and the errors from the cPYDB, it is not a hierarchical match.

However, in this mode, errors can also be matched individually when possible. That is, when a single error, rather than the complete hierarchically formed shape consisting of multiple errors, exactly overlaps a corresponding single error in the cPYDB it is considered a match, even if it is at a different level of hierarchy.

Note:

Setting the `partially_exploded_cells` argument of the `error_options()` function to `EXPLODE` or using the `-pec EXPLODE` command-line option is recommended for all IC Validator runs that use hierarchical error classification, especially the initial IC Validator run used to generate the cPYDB. This argument ensures that errors are reported in cells for which the IC Validator tool has kept all placements from the original design hierarchy, and it reduces the chance that instance-specific errors might lead to unmatched error classifications.

Error Classification Best Practices

It is important to make the distinction between classified errors in the error classification database (cPYDB) that are referenced by rule, or violation comment, and errors produced by the IC Validator tool that are created by a command. The recommended best practice to ensure proper error matching is to output all of the violations for a rule with a single IC Validator command. This avoids the possibility of errors from different commands within the same rule interacting in the cPYDB and preventing a hierarchical match when they are

Feedback

considered for an individual command in a subsequent run. Violation comments should be made unique to the command to ensure this relationship.

It is also important to clean up invalid cPYDB data. For example if the error shapes change, but the new shapes are considered valid and are to be waived, the cPYDB entries for that violation and cell should be purged and re-created with the new error shapes. Leaving the old, invalid shapes in the cPYDB can prevent the IC Validator tool from finding a hierarchical match.

Currently, error classification is most applicable to DRC violations. Matching of previously classified errors of other types of violations, including ERC, device extraction, and density is not supported.

Database Naming Conventions

Databases are referred to by specifying a database name and database path. Error databases (PYDBs) generated by an IC Validator run are named PYDB_*top-cell* and by default, reside in run_details/pydb unless a different path is specified in the `db_path` argument of the `error_options()` function.

For example, you run the IC Validator tool on the design TOP. The directory run_details/pydb/PYDB_TOP contains the data files for the error database. The database path is run_details/pydb, and the database name is PYDB_TOP.

SQLite allows database names to consist of alphanumeric characters and the underscore character (_). Database names are transformed, as follows:

- All non alphanumeric characters are replaced with an underscore (_). For example, PYDB_A.1.2 becomes PYDB_A_1_2.
- If the database name contains only digits, leading and trailing underscores are added. For example, 123 becomes _123_.

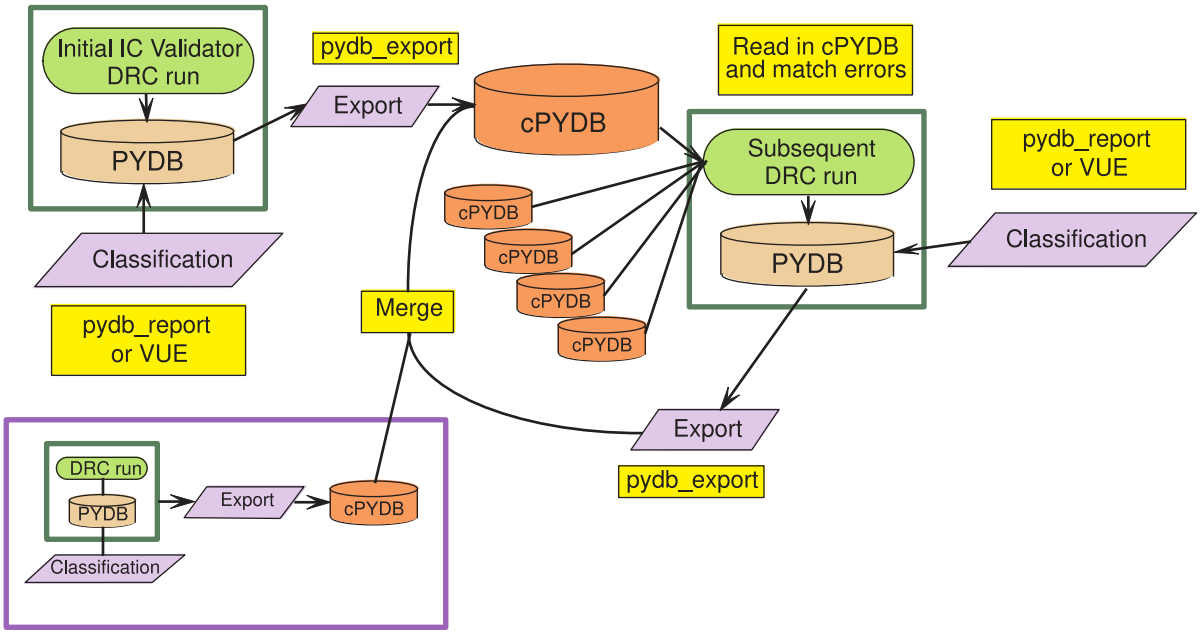
You can continue to use the original database name in runsets, in VUE, and with the pydb_report and pydb_export utilities, but the database is stored on disk with the modified name.

DRC Error Classification Flow

Figure 93 shows the DRC error classification flow.

Feedback

Figure 93 DRC Error Classification Flow



1. Run the IC Validator tool on the design. The resultant DRC errors can then be classified using VUE or the pydb_report utility.

   For reuse of classification data in future runs, export the classified errors to an error classification database using VUE or the pydb_export utility.

2. When the IC Validator tool is run again, specify the error classification database in the `match_errors` argument of the `error_options()` function. If multiple error classification databases are specified, each one is searched for a matching error in the order specified. The error is pre-classified with the data in the error classification database it matches first, and it is written to the error database for the current run. The error shows as classified in the LAYOUT_ERRORS file and in VUE.

   Using the `match_errors argument` of the `error_options()` function set to the `suppress_matched_errors` option you can specify errors that are not written to the error database.

   Note:

   The `error_limit_per_check` argument of the `error_options()` function only applies to new errors. All errors produced during this run are checked against the imported error classification database list, and all matching errors are written to error database. Errors that do not

Feedback

match an error classification database are written until the specified `error_limit_per_check` limit is reached.

3. Finally, new classifications from the current run or from other runs can be merged into an existing error classification database. Newly classified errors are appended to the database, while existing errors are updated if they have changed.

DRC Error Classification Flow Example

This section walks through an example of the DRC error classification flow using the AD4FUL design. The pydb_report and pydb_export utilities are used for classification and database management. (See pydb_report Utility on page 327 and pydb_export Utility on page 332.)

The basic steps are

1. Running the IC Validator Tool Initially
2. Creating an Error Classification Database
3. Running the IC Validator Tool Again
4. Updating the Existing Error Classification Database

Running the IC Validator Tool Initially

First, run the IC Validator tool on the design, producing DRC errors. A standard LAYOUT_ERRORS file is written.

Next, classify

- All errors in cell INVH as Ignore.
- All errors in the violation “Met1 Spacing must be >= 3” as Waive.
- Errors within a coordinate window in the cell TGATE as Watch.

This classification is accomplished by using the pydb_report utility.

```
pydb_report -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb -classify classify.csv
```

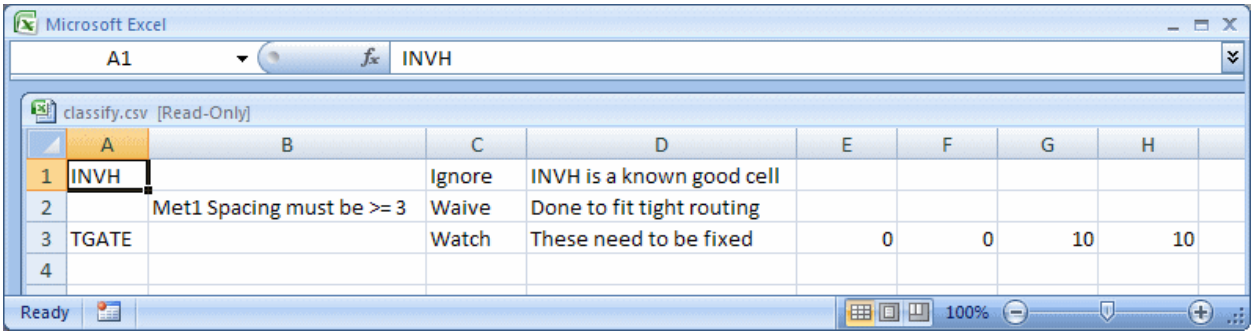
The input file to the pydb_report utility is in a standard comma-delimited format. See pydb_report Utility on page 327 for more information. The file used in this example is

```
INVH,,Ignore,INVH is a known good cell,,,,
,Met1 Spacing must be >= 3,Waive,Done to fit tight routing,,,,
TGATE,,Watch,These need to be fixed,0,0,10,10
```

Figure 94 shows the comma-delimited format in Microsoft Excel.

Feedback

Figure 94 Example Classify File in Microsoft Excel



The pydb_report utility writes a log file, pydbreport.log, to the current working directory. The same information is displayed on screen. The output from this example is shown in Example 35.

Example 35 Initial pydb_report Utility pydbreport.log

```
Connecting to DB: PYDB_AD4FUL
            path: ./run_details/pydb
Connected to DB PYDB_AD4FUL

Error database PYDB_AD4FUL summary
    Number of commands:              12
    Number of commands w/ errors:    12
    Number of dynamic tables:        5
    Total error polygons:            48
    DBU:                             0.00050
    Number of unique cells w/ errors: 6

Classifying Errors from classify.csv...
  Classifying all errors in violation/cell...
    Violation: ALL
    Cell:      INVH
    Class:     Ignore
    Comment:   INVH is a known good cell
   6 new errors classified.
   Classify time = 0:00:00  User=0.00 Sys=0.01 Mem=0.000 GB
  Classifying all errors in violation/cell...
    Violation: Met1 Spacing must be >= 3
    Cell:      ALL
    Class:     Waive
    Comment:   Done to fit tight routing
   3 new errors classified.
   3 existing errors reclassified.
   Classify time = 0:00:00  User=0.00 Sys=0.00 Mem=0.491 GB
  Classifying all errors in window...
    Violation: ALL
    Cell:      TGATE
    Window:    (0.0000, 0.0000) (10.0000, 10.0000)
    Class:     Watch
```

Feedback

```
    Comment:   These need to be fixed
   12 new errors classified.
   Classify time = 0:00:00  User=0.01 Sys=0.00 Mem=0.000 GB
Overall Classify time = 0:00:00  User=0.01 Sys=0.01 Mem=0.000 GB
Done.
```

You can get a report of the classified errors in the error database for the current run using the pydb_report utility:

```
pydb_report -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb \
   -show_classified file=classified.txt
```

Example 36 shows the first error record from the classified error report (classified.txt) for this example.

Example 36 Error Record From Classified Error Report

```
---------------------------------------------------------------------
                    Classified Error Report
---------------------------------------------------------------------
Report Violation: ALL
Report Cell:      ALL
Report Class:     ALL
=====================================================================
Violation: " toxcont must be enclosed by met1 by more than 1.0
 without touching or overlapping"
---------------------------------------------------------------------
Cell: TGATE

Class Summary:
              Class         Errors
              ------------- ------
              Watch         6

Command: runset.rs:101:enclose
 - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  ( lower left x, y ) ( upper right x, y) Distance
 - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
 (3.0000, 4.0000)    (4.0000, 5.1180)     1.0000
   * Class:            Watch
   * Created by:       juser on 2008-10-21 08:36:19
   * Last Modified by: juser on 2008-10-21 08:36:19
   * Comment by:       juser on 2008-10-21 08:36:19
       These need to be fixed
```

Feedback

Creating an Error Classification Database

To use the classification data in a subsequent run, run the pydb_export utility to create an error classification database (cPYDB). For example,

```
pydb_export -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb \
   -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb -create
```

This example creates a new error classification database called CPYDB_AD4FUL in the ./cpydb directory. This database can be used in subsequent IC Validator runs to retain the error classification data. The pydb_export utility produces a log file, pydbexport.log, in the current directory. The output from this example run is shown in Example 37.

Example 37 Initial Error Classification Database

```
Called as: pydb_export -pydb_name PYDB_AD4FUL -pydb_path
./run_details/pydb -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb
-create

Operation ..................... Create New Database
Input PYDB Path ............... ./run_details/pydb
Input PYDB Name ............... PYDB_AD4FUL
Output cPYDB Path ............. ./cpydb
Output cPYDB Name ............. CPYDB_AD4FUL
Export Matched Errors ......... FALSE

Loading global database info
 Loading database time = 0:00:00  User=0.00 Sys=0.01 Mem=0.000 GB
Exporting classified error data
  Exporting violation " toxcont must be enclosed by met1 by more
  than 1.0 without touching or overlapping"
   6 new errors exported
  Exporting time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
  Exporting violation " toxcont must be enclosed by tox by more
  than 1.5"
   6 new errors exported
  Exporting time = 0:00:00  User=0.01 Sys=0.00 Mem=0.000 GB
  Exporting violation "Met1 Spacing must be >= 3"
   6 new errors exported
  Exporting time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
  Exporting violation "ngate must be exactly 2.0 by 6.0 rectangles"
   2 new errors exported
  Exporting time = 0:00:01  User=0.00 Sys=0.00 Mem=0.000 GB
  Exporting violation "pgate must be exactly 2.0 by 12.0
  rectangles"
   1 new error exported
  Exporting time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
 Export time = 0:00:01  User=0.01 Sys=0.01 Mem=0.803
Recreating indexes
 Recreating indexes time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
Overall export time = 0:00:01  User=0.01 Sys=0.02 Mem=0.000 GB
```

Feedback

```
Export Successful.
PYDB_Export is done.
```

Running the IC Validator Tool Again

After fixing some of the errors, run the IC Validator tool again, importing the error classification database created in the last run. Using the error classification database keeps you from having to duplicate work. To use the error classification database, specify the error classification database in the `error_options()` function:

```
error_options(
   match_errors = {
      {db_name = "CPYDB_AD4FUL",
       db_path = "./cpydb"}
   }
);
```

After the IC Validator tool runs, notice in the new LAYOUT_ERRORS file that the errors were pre classified using data from the error classification database.

- The error summary section contains error counts broken out by classification tag.
- The error details section groups errors by their classification tag.

Note:

The pydb_report utility can be used to regenerate the LAYOUT_ERRORS file from the error database at any time. Also, you can use the pydb_report utility to generate the classified error report to show detailed classification information, as shown in Example 38 and Example 39.

Example 38 shows the error summary section.

Example 38 Error Summary Section

```
                    ERROR SUMMARY

 toxcont must be enclosed by met1 by more than 1.0 without touching or
 overlapping
 enclose ..................................... 9 violations found.
   Unclassified Errors ....................... 3 violations found.
   Errors classified as Watch ................ 6 violations found.
```

Example 39 shows the error details section.

Example 39 Error Details Section

```
                    ERROR DETAILS

--------------------------------------------------------------------------
 toxcont must be enclosed by met1 by more than 1.0 without touching or
 overlapping
--------------------------------------------------------------------------
```

Feedback

```
run2.rs:107:enclose
Structure ( lower left x, y ) ( upper right x, y) Distance
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
TGATE      (21.5000, 23.5000)   (22.0000, 24.9140)   0.5000
TGATE      (21.5000, 19.0860)   (22.0000, 20.5000)   0.5000
TGATE      (21.5000, 20.5000)   (22.0000, 23.5000)   0.5000

+ Errors classified as Watch
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
TGATE      (3.0000, 4.0000)     (4.0000, 5.1180)     1.0000
TGATE      (3.0000, 0.0000)     (4.0000, 1.0000)     1.0000
TGATE      (7.0000, 0.0000)     (8.1180, 1.0000)     1.0000
TGATE      (3.0000, 0.0000)     (4.0000, 1.0000)     1.0000
TGATE      (3.0000, 1.0000)     (4.0000, 4.0000)     1.0000
TGATE      (4.0000, 0.0000)     (7.0000, 1.0000)     1.0000
```

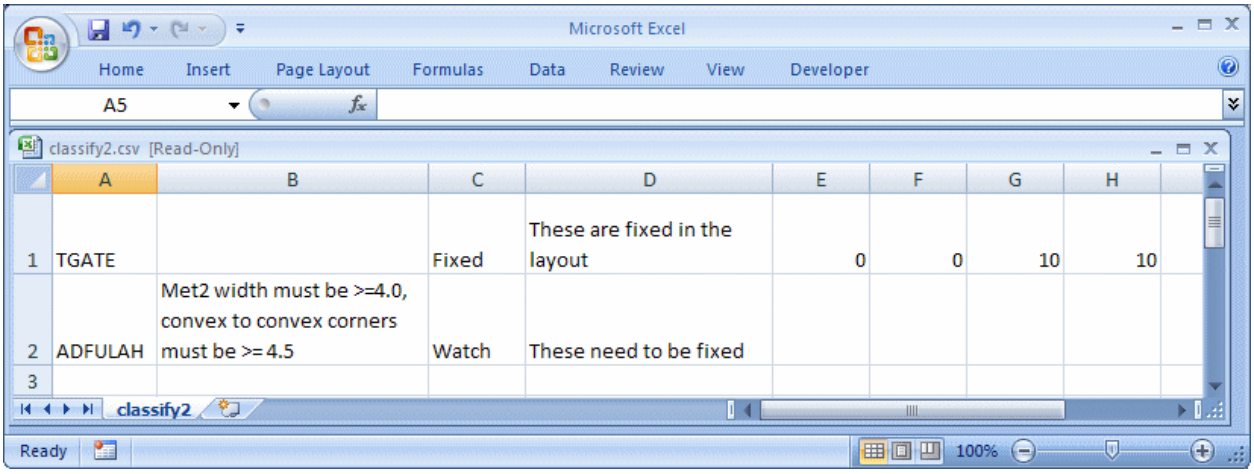
Updating the Existing Error Classification Database

After the second run, you can classify more errors and reclassify existing errors:

```
pydb_report -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb -classify classify2.csv
```

Figure 95 shows the classify file used.

Figure 95 Classify File



The log file from the pydb_report utility is shown in Example 40.

Example 40 pydb_report Log File

```
Connecting to DB: PYDB_AD4FUL
            path: ./run_details/pydb
Connected to DB PYDB_AD4FUL
```

Feedback

```
Error database PYDB_AD4FUL summary
    Number of commands:               12
    Number of commands w/ errors:     12
    Number of dynamic tables:         5
    Total error polygons:             48
    DBU:                              0.00050
    Number of unique cells w/ errors: 6

Classifying Errors from classify2.csv...
  Classifying all errors in window...
    Violation: ALL
    Cell:      TGATE
    Window:    (0.0000, 0.0000) (10.0000, 10.0000)
    Class:     Fixed
    Comment:   These are fixed in the layout
   12 existing errors reclassified.
   Classify time = 0:00:00  User=0.01 Sys=0.01 Mem=0.000 GB
  Classifying all errors in violation/cell...
    Violation: Met2 width must be >=4.0, convex to convex corners
    must be >= 4.5
    Cell:      ADFULAH
    Class:     Watch
    Comment:   These need to be fixed
   3 new errors classified.
   Classify time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
Overall Classify time = 0:00:00  User=0.01 Sys=0.01 Mem=0.000 GB
Done.
```

Merge the changes into the existing error classification database (cPYDB), using the pydb_export utility:

```
pydb_export -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb -cpydb_name CPYDB_AD4FUL \
   -cpydb_path ./cpydb -merge
```

The log file from the pydb_export utility for the merge step is shown in Example 41.

Example 41 pydb_export Log File

```
Called as: pydb_export -pydb_name PYDB_AD4FUL -pydb_path
./run_details/pydb -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb
-merge

Operation ..................... Merge
Input PYDB Path ............... ./run_details/pydb
Input PYDB Name ............... PYDB_AD4FUL
Output cPYDB Path ............. ./cpydb
Output cPYDB Name ............. CPYDB_AD4FUL
Export Matched Errors ......... FALSE

Loading global database info
 Loading database time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
Merging classified error data
```

Feedback

```
  Exporting violation " toxcont must be enclosed by met1 by more
  than 1.0 without touching or overlapping"
   6 existing errors updated
  Exporting time = 0:00:00  User=0.01 Sys=0.00 Mem=0.000 GB
  Exporting violation " toxcont must be enclosed by tox by more
  than 1.5"
   6 existing errors updated
  Exporting time = 0:00:00  User=0.00 Sys=0.00 Mem=0.000 GB
  Exporting violation "Met2 width must be >=4.0, convex to convex
  corners must be >= 4.5"
   3 new errors exported
  Exporting time = 0:00:01  User=0.00 Sys=0.00 Mem=0.000 GB
 Export time = 0:00:01  User=0.01 Sys=0.01 Mem=0.000 GB
Overall export time = 0:00:01  User=0.01 Sys=0.01 Mem=0.000 GB
Export Successful.
PYDB_Export is done.
```

Running the IC Validator tool again, the LAYOUT_ERRORS file shows that the changes made to the database are retained.

See the detailed information by running the pydb_report utility:

```
pydb_report -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb \
   -show_classified file=classified2.txt
```

Alternatively, you can use VUE to generate the report.

The new output from the classified error report for this example is similar to the error report for the initial run.

pydb_report Utility

Use the pydb_report utility with an error database to

- Generate a detailed report of classified errors.
- Regenerate the LAYOUT_ERRORS file.
- Classify errors.

When an error is classified, the classification tag, UNIX user ID, timestamp, and possible comment are stored with the error in the error database for the current run.

Additionally, the pydb_report utility can be used on an error classification database to generate a detailed report of classified errors.

Note:

See Database Naming Conventions on page 318.

Feedback

This utility is in the IC Validator installation bin directory. Table 64 defines the command-line options.

Table 64 pydb_report Utility Command-Line Options

Command-Line Option	Definition
`-64`	Runs the utility with 64-bit coordinates support. This command-line option might be necessary if coordinates in the error database exceed the capacity of a 32-bit signed integer, that is, if the IC Validator tool runs with the `-64` command-line option.
`-classify` *`input_file`*	Classifies errors in an error database based on the given input file. See Classify File Format in the following section.
`-classify_by_window [interacting | enclosed]`	Specifies how errors are selected for classification based on window coordinates in the classify file. The default is `interacting`. • `interacting`. Errors having any interaction with the window coordinates are selected for classification. • `enclosed`. Errors completely enclosed by the window coordinates are selected for classification. Selected errors can touch the enclosing window.
`-cpydb_name` *`name`*	Error classification database (cPYDB) instance name. Required if operating on an error classification database.
`-cpydb_path` *`path`*	Error classification database (cPYDB) path. Required if operating on an error classification database.
`-create_results_report` *`output_file`*	Creates a RESULTS-style file from information in the PYDB.
`-dlpc <`*`limit`*`>`	Limits the error details (coordinates) for each check in the generated LAYOUT_ERRORS or TOP_LAYOUT_ERRORS files to the given number.
`-help`	Displays the usage message.
`-le` *`output_file`*	Generates the LAYOUT_ERRORS file from the current error database.
`-no_error_details`	Does not report error details (that is, coordinates) in the LAYOUT_ERRORS or TOP_LAYOUT_ERRORS files.
`-pydb_name` *`name`*	Error database (PYDB) instance name; usually PYDB_*top_cell*. Required if operating on an error database.
`-pydb_path` *`path`*	Error database (PYDB) path; usually ./run_details/pydb unless otherwise specified in the `db_path` argument of the `error_options()` function. Required if operating on an error database.

Feedback

Table 64 pydb_report Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-show_classified [cell=cell] [violation=comment] [class=class1,..., classn] [file=file]`	Generates a detailed classified error report for an error database and error classification database. The specified parameters can restrict the report by violation, cell, and class. If the output file is not specified, output goes to the screen.
`suppress_classified_errors`	If specified, no classified errors are reported in the LAYOUT_ERRORS or TOP_LAYOUT_ERRORS file. All classified errors are suppressed, whether they come from the error classification database or an IC Validator option that classifies errors programmatically, or have been classified after the IC Validator run with VUE or the error database.
`-tle output_file`	Generates the TOP_LAYOUT_ERRORS file from the PYDB database.
`-v`	Prints the utility version.
`-verbose_results_file`	Controls the output when `pydb_report -create_results_report` is used.

Classify File Format

The format of the input file for the `-classify` command-line option is a comma-delimited file with these columns: Cell, Violation, Class, Comment, x1, y1, x2, y2. See Figure 94.

- If a cell is given but not a violation, all errors within the cell are classified.
- If a violation is given but not a cell, all errors within the violation are classified.
- If window coordinates are given, a cell is required and errors that interact with that window are classified. The default is any interaction, but you can change this behavior using the `-classify_by_window` command-line option.

The formatting of records in the file should follow the standard RFC4180 recommended guidelines for CSV files with no column headings. This formatting is supported by Microsoft Excel and other spreadsheet tools.

The classifications you can choose the classifications shown in Table 65.

Table 65 Classify File Format

Classification	Description
Error	Unclassified error (default classification)

Feedback

Table 65 Classify File Format (Continued)

Classification	Description
Ignore	Error that the designer does not care about: to be fixed later in the process flow
Waive	Intended design error. It exists at sign-off.
Watch	Error that needs to be fixed in the next design cycle.
Fixed	Error that has been fixed in the layout
Unmatched	Error that exists in an imported error classification database but was not produced by the current run

Using Regular Expression Matching

In the input file specified by the `-classify` command-line option you can now use regular expressions for matching violation comments and cell names in the error database. This matching is enabled by using leading and trailing forward slash (/) characters, with an optional modifier character after the trailing slash. The modifier characters are shown in Table 66

Table 66 Modify Characters for Use in the Classify File

Modify character	Description
m	Match case. This character is the default.
i	Ignore case.

For example, the following line in a classify file results in all violation comments starting with M1, M2, M3, or M4 in cell Cell1 being classified as ignore:

```
Cell1,"/^M[1-4].*/",Ignore,"comment"
```

The following line in a classify file results in violations in all cells containing “sram”, ignoring case, being waived:

```
"/.*sram.*/i","",Waive,"comment"
```

A violation comment that actually begins with a slash cannot be matched with a literal string, but must use GNU extended regular expression special characters. See the table of these special characters in the `text_options()` function in the *IC Validator Reference Manual* for more information. For example, the following in a classify file would match a violation comment containing “/M4 rule 5/”:

```
"/\/M4 rule 5\//"
```

Feedback

Examples

To regenerate TOP.LAYOUT_ERRORS from the current run, use this syntax:

```
pydb_report -pydb_name PYDB_TOP -pydb_path ./run_details/pydb \
   -le TOP.LAYOUT_ERRORS
```

To classify errors in the error database for the current run, use this syntax:

```
pydb_report -pydb_name PYDB_TOP -pydb_path ./run_details/pydb \
   -classify classify.csv
```

To show classification details on screen of all ignored and waived errors in the cell TOP, use this syntax:

```
pydb_report -pydb_name PYDB_TOP -pydb_path ./run_details/pydb \
   -show_classified cell=TOP class=Ignore,Waive
```

To show classification details of all waived errors in an error classification database, use this syntax:

```
pydb_report -cpydb_name CPYDB_TOP -cpydb_path ./cpydb \
   -show_classified class=Waive
```

To use `-verbose_results_file` <value[s]> with `-create_results_report`, use the following syntax:

```
pydb_report -i run_details/pydb/PYDB_TOP -verbose_results_file
ERRORS,PRE_WAIVED_ERRORS,FLAT_VIOLATION_COUNT,TOTAL_VIOLATION_COUNT,ASSI
GNS -create_results_report TOP.report
```

Note:

You need to enclose <value[s]> in double quotation marks if they contain any whitespace characters.

The `verbose_results_file` argument can have up to five values to control the verbosity of the results file generated by IC Validator. The default is `ERRORS`. These values are:

- `ERRORS`. Reports each rule name and the total number of violations for that rule.

  Note:

  When using error classification, the number of waived violations is excluded from the number of violations found in the *cell*.RESULTS file. For example, the tool detects 100 violations, but 60 of them are waived. Therefore, the number of violations reported is 40.

- `PRE_WAIVED_ERRORS`. Specifies the reporting of pre-waiver violation counts in a separate table following the current (post-waiver) violation counts table.

Feedback

- `FLAT_VIOLATION_COUNT`. Reports the flat violation count for each rule in the error details section and the total flat violation count in the error summary section.
- `TOTAL_VIOLATION_COUNT`. Reports the total number of hierarchical and flat violation counts at the top of the rules table in the error details section.
- `ASSIGNS`. Reports each assign layer name, hierarchical and flat polygon counts for the layer (or “Pruned” if the layer was pruned), and additional assignment information for the layer from the assign functions in the runset.

These enum values can be used with the `verbose_results_file` argument in the `pydb_report` utility to customize the output in the results file.

Following is a part of the `TOP.report` file for a small PYDB (after manually waiving one of the two violations belonging to "MyRule").

Note:
Waived violation counts do not show in this report.

```
...
...

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Rule                          Violations Found
TOTAL                         v = 2 (2)
MyRule                        v = 1 (1)
Violation                     v = 1 (1)
  text_net:text_short           v = 1 (1)

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Rule                          Pre-Waived Violations Found
TOTAL                         v_all = 3 (3)
MyRule                        v_all = 2 (2)
Violation                     v_all = 1 (1)
  text_net:text_short           v_all = 1 (1)
```

pydb_export Utility

Use the pydb_export utility to export error classification data from an error database to either

- Create a new error classification database (cPYDB). See Error Classification Database Format on page 337.
- Merge into an existing error classification database by appending new errors and updating existing ones.

Also, the pydb_export utility can be used to manage error classification databases by merging several error classification databases together or by purging classification data from violations and cells.

Feedback

For an error to be considered equivalent between runs, the violation comments must match exactly or match up to the first occurrence of a specified comment delimiter, and the error polygon must match at cell level.

Note:

See Database Naming Conventions on page 318.

This utility is in the IC Validator installation bin directory. Table 67 defines the command-line options.

Table 67 pydb_export Utility Command-line Options

Command-Line Option	Definition
`-64`	Runs the utility with 64-bit coordinates support. This command-line option might be necessary if coordinates in the error database exceed the capacity of a 32-bit signed integer, that is, if the IC Validator tool runs with the `-64` command-line option.
`-cell` *cell*	Limit exports to errors of the given cell. This command-line option can be given more than one time to specify multiple classes. It supports regular expressions.
`-class` *class*	Limit exports to errors of the given classification. This command-line option can be given more than one time to specify multiple classes. It supports regular expressions.
`-comment_match_delimiter` *delim*	Match violation comments up to the specified delimiter. When exporting an error database, the default of the delimiter is whatever is specified in the runset using the `comment_match_delimiter` argument of the `error_options()` function. When merging multiple error classification databases, however, this command-line option delimiter should be specified to use partial violation comment matching.
`-convert_to_64`	Convert a 32-bit or variable width coordinate error classification database (cPYDB) to a 64-bit coordinate error classification database. When used with the `-copy_cpydb` or `-create` command-line options, this command-line option converts the database to 64-bits when it is created. Otherwise, it converts an existing database in place.

Feedback

Table 67 pydb_export Utility Command-line Options (Continued)

Command-Line Option	Definition
`-convert_to_32`	Convert a 64-bit or variable width coordinate error classification database (cPYDB) to a 32-bit coordinate error classification database. When used with the `-copy_cpydb` or `-create` command-line options, this command-line option converts the database to 32-bits when it is created. Otherwise, it converts an existing database in place. **Note:** This operation fails if the error classification database contains coordinates that are too large to be represented by 32-bit integers.
`-convert_to_variable`	Convert a 64-bit or 32-bit coordinate error classification database (cPYDB) to a variable width coordinate error classification database. When used with the`-copy_cpydb` or `-create` command-line options, this command-line option converts the database to variable width coordinates when it is created. Otherwise, it converts an existing database in place.
`-copy_cpydb name=`*`db_name`* `path=`*`db_path`*	Copy an error classification database to the destination error classification database, creating a new database or overwriting an existing database.
`-cpydb_name` *`name`*	Output error classification database (cPYDB) instance name. Required.
`-cpydb_path` *`path`*	Output error classification database (cPYDB) path. Required.
`-create`	Export the error database, either creating a new error classification database or overwriting an existing error classification database. Or, merge multiple error classification databases, creating a new error classification database or overwriting an existing error classification database.
`-exclude_cell` *`cell`*	Excludes errors in the given cell from export. This command-line option can be given more than one time to specify multiple classes. It supports regular expressions. The `-exclude_cell` command-line option takes priority over the `-cell` command-line option.
`-exclude_class` *`class`*	Excludes errors of the given classification. This command-line option can be given more than one time to specify multiple classes. It supports regular expressions. The `-exclude_class` command-line option takes priority over the `-class` command-line option.

Feedback

Table 67 pydb_export Utility Command-line Options (Continued)

Command-Line Option	Definition
`-exclude_violation` *violation*	Excludes errors in the given violation comment from export. This command-line option can be given more than one time to specify multiple classes. It supports regular expressions. The `-exclude_violation` command-line option takes priority over the `-violation` command-line option.
`-export_matched`	Include matched errors. The default is to export only newly classified errors. When merging into an existing error classification database, this option specifies whether to include matched errors from error classification databases other than the destination.
`-help`	Displays the usage message.
`-host_init`	Specifies the host configuration options for distributed operations. Currently, distributed processing with this command-line option is supported only for the `-flatten` command-line option. See the `-host_init` IC Validator command-line option in Chapter 1, IC Validator Basics for more information.
`-flatten layout=`*layout* `cell=`*cell*	Flattens the output error classification database to the given top cell using the given layout. The layout and top cell used to create the error classifications being flattened should be given as the layout and cell parameters to this option. The layout format is auto-detected and is given as a path using the same conventions as the library1 and library2 command-line options of the LVL utility. Only violations in and hierarchically underneath the given cell are output. Layout grid and drawn checks on original cells are not flattened. See Flattening an Error Classification Database on page 339 for more information.
`-format CPYDB | OASIS | PERC`	Specifies the format of the output waiver database. The default is `CPYDB`.
`-i` *input db*	Input PYDB database, cPYDB, or OASIS waiver file.
`-layer` *layer number*	Specifies the layer for OASIS format waiver shapes. Waiver shapes are created on the specified layer number with a datatype based on the violation comment of the rule. The default is `87654`.
`-merge`	Merge the specified error database into the specified error classification database, updating classification of existing errors or adding new errors to the database.
`-merge_cpydb name=`*db_name* `path=`*db_path*	Merge an error classification database into the destination error classification database. You can use multiple options on a command line to merge multiple error classification databases.

Feedback

Table 67 pydb_export Utility Command-line Options (Continued)

Command-Line Option	Definition
`-merge_layout` *main_library_path*	Merge the waivers from an error classification database specified through the -i and a design layout specified by this argument into an OASIS file specified by -o. This allows a single OASIS file to contain both the layout and the waivers. This argument requires `-format` OASIS.
`-o` *output db*	Specifies the path to the output database. This command-line option can be used to specify either a cPYDB or an OASIS format output database.
`-password` *password*	Password for write access to an existing error classification database. Specify `prompt` to read the password from the command line.
`-perc_hierarchical_delimiter` *delimiter*	Hierarchical delimiter to use when exporting PERC waivers. The default is `"/"`.
`-perc_holding_cell` *cell_name*	When exporting PERC waiver files, set the given cell name as holding cell for the exported waivers.
`-perc_output_comment` *true|false*	When exporting PERC waiver files, select whether to output the informational comment, which is part of the error. The default is `true`.
`-purge` *purge_file*	Purge all classification data from violations and cells based on the given input file. See Purge File Format on page 338 for file format information. If any error databases reference the target error classification database and errors matched the violations and cells that are being purged, those errors are no longer viewable.
`-pydb_name` *name*	Input error database (PYDB) instance name; usually PYDB_*top_cell*. Required for export.
`-pydb_path` *path*	Input error database (PYDB) path; usually ./run_details/pydb unless otherwise specified in the `db_path` argument of the `error_options()` function. Required for export.
`-read_only`	Specifies that the output error classification database (cPYDB) should be read-only. When used with the `-create` or `-copy_cpydb` command-line options, the cPYDB is changed to read-only mode after it is created. Otherwise, it changes an existing database to read-only mode. A read-only database might not be modified in any way. Each IC Validator component that connects to a read-only cPYDB has its own database server process. A writable database might not be converted to read-only while other users are connected.

Feedback

Table 67 pydb_export Utility Command-line Options (Continued)

Command-Line Option	Definition
`-rule_name_delimiter` *regular expression*	Specifies the rule name delimiter regular expression used to separate the rule name from the rest of the violation comment when creating rule waiver cell names in the OASIS waiver file. The default is to use the rule name delimiter specified by `error_options(rule_name_delimiter)` in the runset. The runset rule name delimiter has a default of `/[[:space:]]+:|[[:space:]]*:[[:space:]]/`.
`-rule_name_length` *integer*	Specifies the maximum length of the rule name used to create a rule waiver cell name in the OASIS waiver file. Rule names longer than this specified value are truncated, and a suffix is added to keep them unique if there are conflicts after truncation. The default is `15`.
`-set_password` *password*	Set the password on the given error classification database. Specify `prompt` to read the password from the command line.
`-set_waiver_properties` *user_property_file.json*	Sets waiver user properties from the given JSON waiver property file.
`-select_waiver_properties` *select_properties_file.json*	Includes only waivers that match the waiver user properties specified in the given JSON property selection file.
`-V`	Prints the utility version.
`-violation` *violation*	Limit exports to the given violation comment. This command-line option can be given more than one time to specify multiple classes. It supports regular expressions.
`-waiver_property_definition_file` *property_definition_file.json*	Specifies the JSON file to be used for validating waiver user properties. This file overrides the environment setting of ICV_WAIVER_PROPERTY_DEFINITION_FILE.

Error Classification Database Format

The error classification database contains classification tags, user IDs, timestamps, comments, and error polygons for the classified errors. When an error classification database is used in a subsequent run, errors that match are annotated in the database with a reference to the error classification database. The actual classification data for these errors remains in the error classification database. Therefore, to view errors for the current run, the error classification database needs to be present in the location specified when the IC Validator tool was run. Also, if data is purged from the error classification database or is overwritten, you cannot view the matched errors from the current run.

Feedback

Purge File Format

The format of the input file for the `-purge` command-line option is a comma-delimited file with two columns: Cell and Violation.

- If a cell is specified but not a violation, all errors within the cell are purged.
- If a violation is specified but not a cell, all errors within the violation are purged.

Format the records in the file to follow the standard RFC4180 recommended guidelines for CSV files with no column headings. This formatting is supported by Microsoft Excel and other spreadsheet tools.

Examples

To create a new error classification database (cPYDB) containing classification data from the current run, use this syntax:

```
pydb_export -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb \
   -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb \
   -create
```

To set the password on an error classification database, use this syntax:

```
pydb_export -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb \
   -password oldpassword -set_password newpassword
```

To merge classification data into an existing error classification database that is password protected, use this syntax:

```
pydb_export -pydb_name PYDB_AD4FUL \
   -pydb_path ./run_details/pydb \
   -cpydb_name CPYDB_AD4FUL -cpydb_path ./cpydb \
   -merge -password password
```

To merge several error classification databases together, creating a new error classification database, use this syntax:

```
pydb_export -cpydb_name CPYDB_MY_DESIGN -cpydb_path ./cpydb \
   -merge_cpydb name=CPYDB_CELLA path=./cpydb \
   -merge_cpydb name=CPYDB_CELLB path=./cpydb \
   -merge_cpydb name=CPYDB_TOP path=./cpydb \
   -create
```

To convert an error classification database with 32-bit coordinates to one with 64-bit coordinates in place, use the following syntax. When using the `-convert_to_64` command-line option, you need exclusive access to the database; that is, the database cannot be open by VUE, any other scripts, or IC Validator jobs.

```
pydb_export -cpydb_name db name -cpydb_path db path -convert_to_64
```

Feedback

To copy an error classification database and convert it to 64-bit without modifying the original database, use this syntax:

```
pydb_export -cpydb_name db name -cpydb_path db path \
   -copy_cpydb name=input db name path=input db path \
   -convert_to_64
```

The pydb_report utility can provide information about the coordinate width of an existing error classification database:

```
pydb_report -cpydb_name CPYDB_AD4FUL64 -cpydb_path ./cpydb
...

(C) Copyright 2008-2014.
Synopsys, Inc.  All rights reserved.

cPYDB Name ............ CPYDB_AD4FUL64
cPYDB Path ............ ./cpydb
Coordinate Width ...... 64 bit
```

To export only the classes starting with “W” in all cells that do not start with "W", use this syntax:

```
pydb_export -class "/^W.*/" -exclude_cell "/^W.*/" ...
```

To export all classes except “Ignore” and “Waive”, use this syntax:

```
pydb_export -exclude_class Ignore -exclude_class Waive ...
```

To export only the top cell, use this syntax:

```
pydb_export -cell TOP ...
```

Flattening an Error Classification Database

When an error classification database is generated using block-level IC Validator results, with plans to use it for hierarchical error classification at the chip level after the block is placed, it is advisable to flatten the block-level results if there are any shared sub cells between the current block and other blocks that are placed in the top-level design. The IC Validator tool might push errors down into a sub cell as part of hierarchical DRC checking. If the block-level error classification database is not flattened, it might contain waivers in a shared sub cell that apply only in the context of the block, but are invalid after the block is placed in a larger design with other instances of the shared sub cell. Flattening the block-level error classification database avoids this situation.

The pydb_export utility flattens an error classification database, using the same layout and top cell which were used in the original IC Validator run that created the violations. The layout must be supplied to provide hierarchy information for the flattening process. Violations in the error classification database are flattened to the provided top cell. Only violations in or hierarchically underneath the provided top cell are output.

Feedback

The layout is specified as a path, using the same convention as the *library1* and *library2* command-line options of the LVL utility:

- GDSII: `/path/to/file.gds`
- OASIS: `/path/to/file.oas`
- Milkyway: `/library/path/LIBRARY_NAME`
- OpenAccess: `/path/to/lib.defs/LIBRARY_NAME`
- NDM: `/library/path/LIBRARY_NAME`

The flattening process can be distributed using the `-host_init` command-line option of the pydb_export utility. The following use cases are supported.

To flatten during export from PYDB:

```
pydb_export \
  -pydb_path path -pydb_name name \
  -cpydb_path output_path -cpydb_name output_name \
  -create \
  -flatten layout=layout cell=topcell \
  [-host_init number]
```

To copy a cPYDB and flatten the destination cPYDB:

```
pydb_export \
  -copy_cpydb path=path name=name \
  -cpydb_path output_path -cpydb_name output_name \
  -flatten layout=layout cell=topcell \
  [-host_init number]
```

To flatten a cPYDB in place:

```
pydb_export \
  -cpydb_path path -cpydb_name name \
  -flatten layout=layout cell=topcell \
  [-host_init number]
```

Using PERC Error Classification

Error classification is supported for netlist-based PERC device and net errors. The overall flow is identical to the flow for DRC error classification. Errors from an initial IC Validator run are classified using VUE or one of the PYDB utilities and are then exported to an error classification database (cPYDB) so that the classifications might be applied in subsequent IC Validator runs. The cPYDB should be specified in the `error_options(match_errors)` argument just as with DRC error classification.

Feedback

Note:

Classification is only supported for netlist-based PERC flows and is not available for layout-based flows. Only device and net errors are supported.

Matching Classified Errors

Netlist-based errors are different from DRC errors in that there are no shapes or coordinates available to uniquely identify particular error instances In addition, netlist-based errors have more metadata associated with them. Here is an example of a device error.

```
- - - -
```

Feedback

- Violation Comment / Rule
- Cell Name
- Net Name
- Comment

If all of these criteria of a new error match a classified error in the cPYDB, IC Validator applies the classification to the new error.

Hierarchical PERC Error Classification

IC Validator provides a hierarchical mode that allows classifications from errors that were reported in a lower level of hierarchy to be applied if they are reported at a higher level in a subsequent IC Validator run.

When checking device errors for classification in hierarchical mode, if the device has an instance path, such as the one in the previous example, the device error should be classified if issued at a higher level of hierarchy as long as the original instance name and cell are at the bottom of the new instance path. Nets must match in the same way. The original instance name and cell should be at the bottom of the new instance path.

Classification matches are first attempted at the cell level. If no cell-level match is found, IC Validator looks down the hierarchy to search for a classified error that matches part of the instance path. It is possible that an error might match multiple classified errors. IC Validator applies the first match it finds looking down the hierarchy, and any other potential matches are ignored.

Hierarchical Example

For this example, refer to the device error from the previous example. It is reported and waived in cell dwc_eusb2 with device name Xu_eusb /XI206/M7. If IC Validator is rerun on the same block, it is very likely that an exact cell-level match is found.

If IC Validator is run on higher-level block dwc_top and the error is reported with device name X1/X2/Xu_eusb/XI206/M7, then IC Validator searches for a matching classified error as follows:

- IC Validator searches for a cell-level match in dwc_top, and no matching classified error is found.
- X1 in dwc_top is an instance of cell dwc_mid. IC Validator searches for a classified error in cell dwc_mid with a device name of X2/Xu_eusb/XI206/M7, and no match is found.

Feedback

- X2 in dwc_mid is an instance of cell dwc_eusb2. IC Validator searches for a waiver in cell dwc_eusb2 with a device name of Xu_eusb/XI206/M7. This matches the error from the previous example, so the classification is applied to the error being reported in dwc_top.

The same methodology is used to hierarchically match net names for net errors.

15

DRC Waivers

This chapter explains the DRC waivers capabilities of the IC Validator tool and how to use this feature.

The DRC waivers capabilities are described in the following sections:

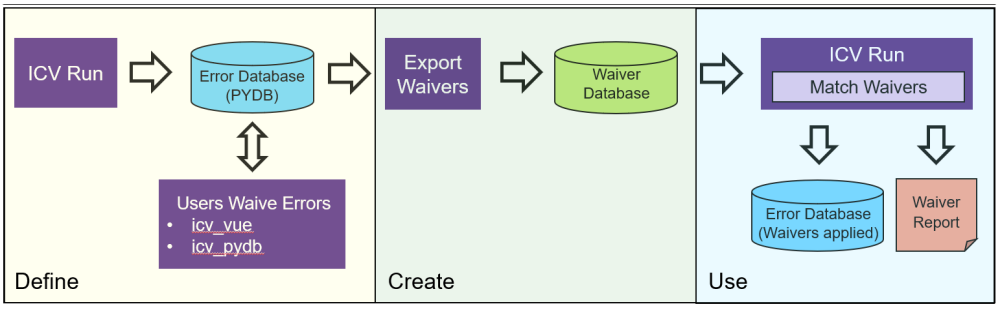
- Defining Waivers
- Creating a Waiver Database
- Using a Waiver Database

DRC waivers allow you to classify and annotate errors found during an IC Validator run and export the errors into a waiver database for use in subsequent IC Validator runs. The DRC Waivers flow is separated into three distinct steps:

- Defining which errors might be waived, and annotating the error database through VUE or the icv_pydb command-line utility
- Creating a persistent waiver database on disk
- Applying the waivers to subsequent IC Validator runs

Each of these steps is described in the following sections.

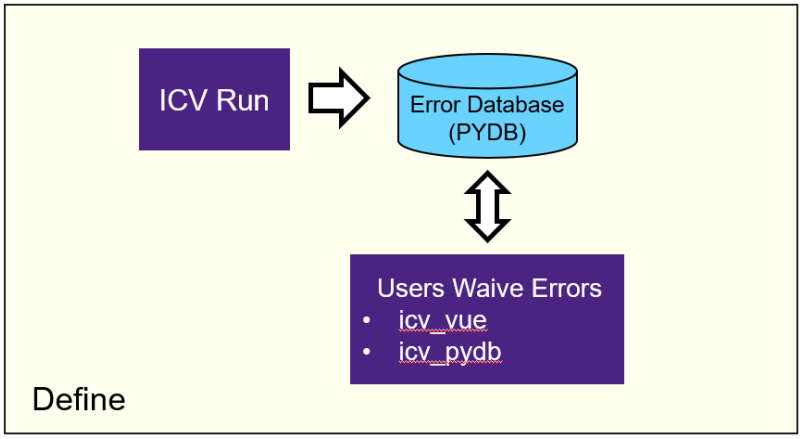
Figure 96 General Waiver Flow



Feedback

Defining Waivers

You can review design errors written to the error database (PYDB) using VUE or the ASCII Error Report file, *cell*. LAYOUT_ERRORS. In many designs, there are DRC violations that can be waived, that is, these errors do not need to be fixed in the layout. This could be a temporary waiver that must be fixed before the design is taped out, or a permanent waiver that is approved by the foundry. You can add a user comment or predetermined waiver properties to waivers to give more context to other engineers that are reviewing the waivers.



The IC Validator SQLite error database allows for concurrent access by multiple users; as errors are reviewed and waivers are applied, other users who access the database can see the updates you have applied.

IC Validator supports several waiver classes:

- Persistent waiver classes: Waive, Watch, Ignore
- Non-Persistent waiver classes: Error, Fixed

Persistent waiver classes can be exported to a waiver database. See Creating a Waiver Database. With multiple persistent waiver classes, you can segregate waived errors into different categories. This makes waiver management easier during the design process. Each of these waiver classes are treated equally. For example, you can choose Waive when the error is waived throughout the design process all the way to tapeout, and choose Watch to designate errors that can be fixed in the next iteration or two of the design.

Non-persistent waiver classes provide information to other users who debug the same error database. For example, with an error defined as Error with the user comment, “this error needs to be fixed in conjunction with rule ABCD”, anyone reviewing the error in the same error database can see this guidance.

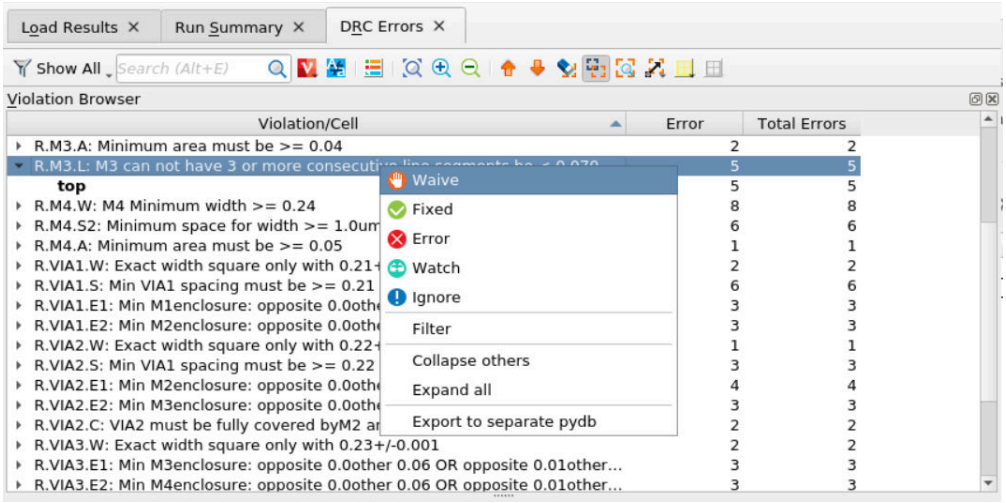
Feedback

Using VUE to Define Waivers

The most common way to define waivers is through the VUE DRC browser. There are several ways to waive errors.

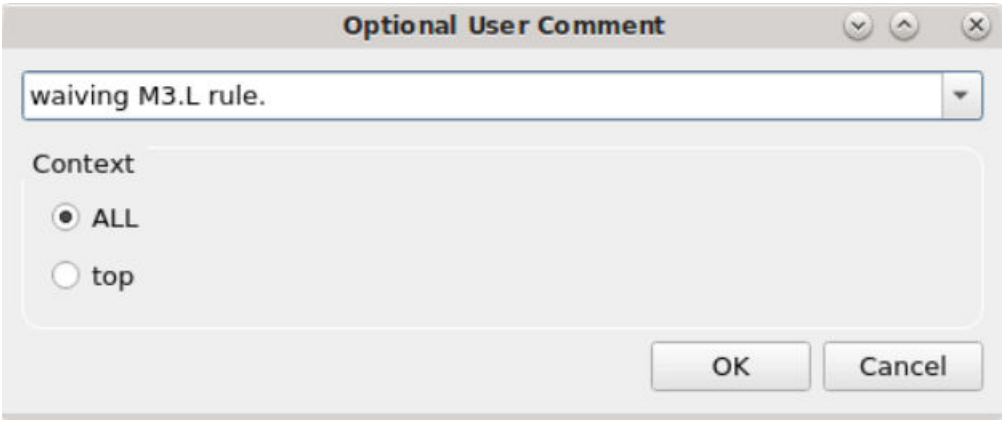
To waive all error shapes in a violation (rule), select the violation in the Violation Browser and right-click to see the menu of options. Choose the appropriate waiver class, as shown in Figure 97.

Figure 97 Violation/Cell in Violation Browser



In this example, the Waive class is selected. After selection, a window appears, which allows you to provide an optional user comment and define the waiver context.

Figure 98 Optional User Comment



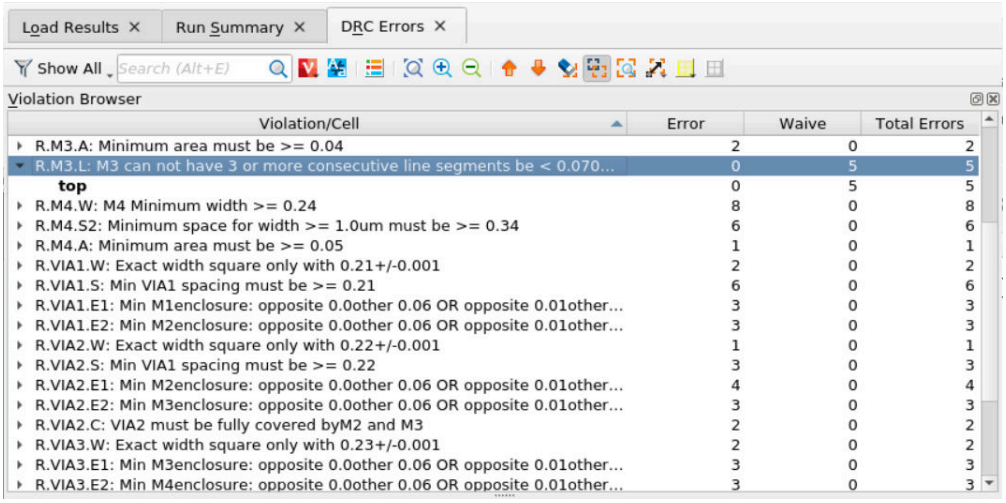
The Context option determines when these waivers apply as they are used in future IC Validator runs. When set to ALL, the waiver is always applied if matched to an error. When

Feedback

set to the top cell (in this example, the top cell name is “top”) , the waiver is applied only if the current top block is run again as the top cell.

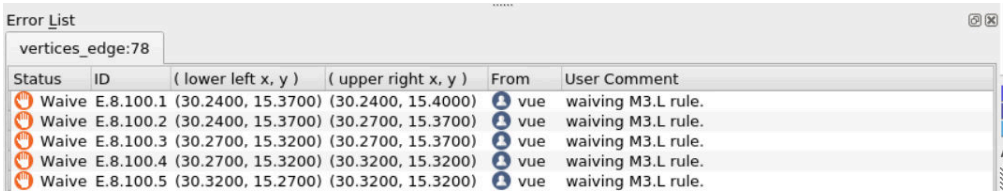
After the waivers are defined, a new column in the error browser is added to show the violations with waived errors.

Figure 99 Error Browser Column



Each error is now annotated with the waiver information.

Figure 100 Waiver Information

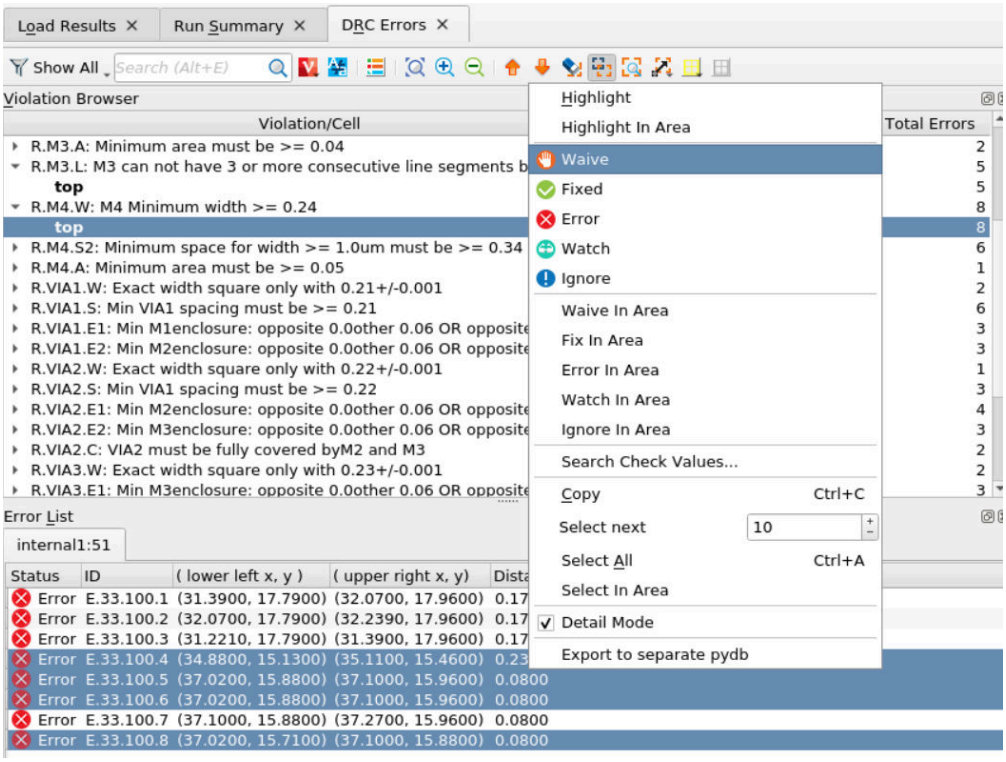


The status is now Waive. The From column shows “vue,” which indicates that the current waiver is user-applied. It was not found while IC Validator referenced a waiver database.

The Error List can be used to waive only certain errors, as shown Figure 101.

Feedback

Figure 101 Error List

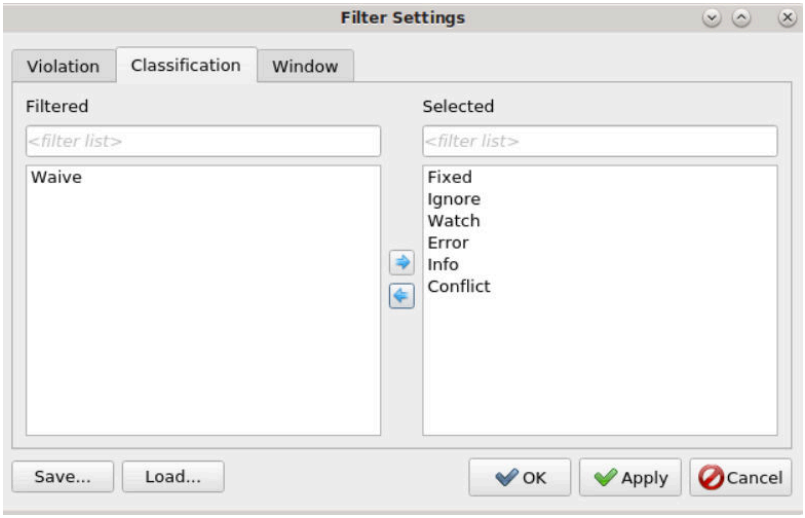


You can select individual errors of the R.M4.W violation by pressing Shift-click and Ctrl-click. Then, right-click to see the menu of waiving options. An additional option, Waive In Area, allows you to waive all errors in a specific region instead of choosing them individually from the Error List.

After the errors are waived, you can filter the error list to hide any specific waiver classes in the Error List. In DRC/Extraction VUE options, use the Enable/Disable filter functions section to select the waiver classes you want to see in the Error List.

Feedback

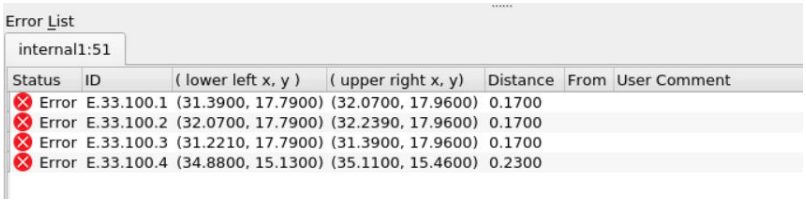
Figure 102 Filter Settings



In this example, errors classified as Waive are filtered from the Error List. All other waiver classes are displayed.

Figure 103 shows the updated Error List:

Figure 103 Updated Error List



Defining Waivers Using the Command-Line Utilities

You can define waivers using the icv_pydb command-line utility. See Error Management Utility for more information.

Custom Waiver Properties

Along with the user comment, you can define additional waiver properties for a given rule and cell. These properties are useful for adding information and providing additional ways to manage waivers across designs or process node. For example, waivers can be given a waiver property that defines a specific tracking number and its current management approval status. If the waiver is not approved, any waiver with the waiver tracking number can be removed from the waiver database using pydb_export and the `-select_waiver_properties` command-line option.

Feedback

The allowable values are defined in a JSON format file, which is read by VUE.

Example 42 JSON Property File (waiver_properties.json)

```
{
  "waiver_property_definitions": [
    {
      "property_name": "WaiverTracking#",
      "property_values": []
    },
    {
      "property_name": "Valid_through",
      "property_values": ["milestone1", "milestone2", "milestone3"]
    },
    {
      "property_name": "Status",
      "property_values": ["pending", "approved", "not approved"],
      "default_value": "pending"
    }
  ]
}
```

Each property has a property_name , property_values, and an optional default_value. The property_name follows OASIS property standards, which means it uses printable ASCII characters, excluding spaces, tabs, newlines, and control characters. The property value is not restricted.

Note:

The WAIVER_REPORT file uses ‘|’ as a delimiter between multiple property values. To avoid confusion, do not use this character.

VUE reads the JSON file when the following environment variable is set.

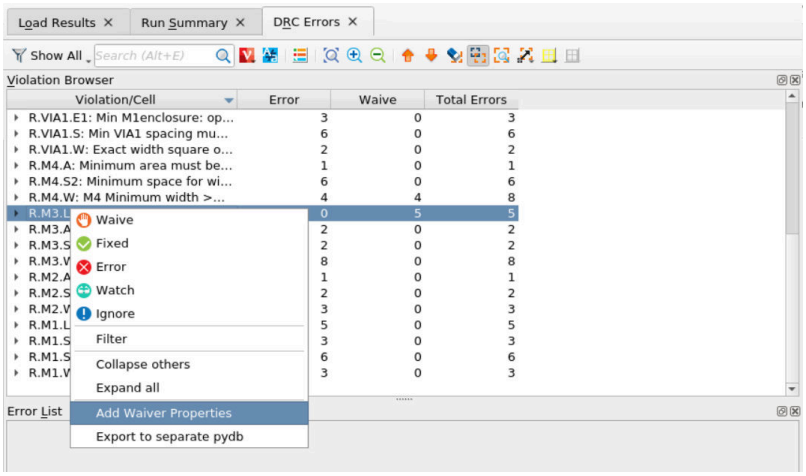
```
setenv ICV_WAIVER_PROPERTY_DEFINITION_FILE waiver_properties.json
```

It then creates a custom window with the allowable properties.

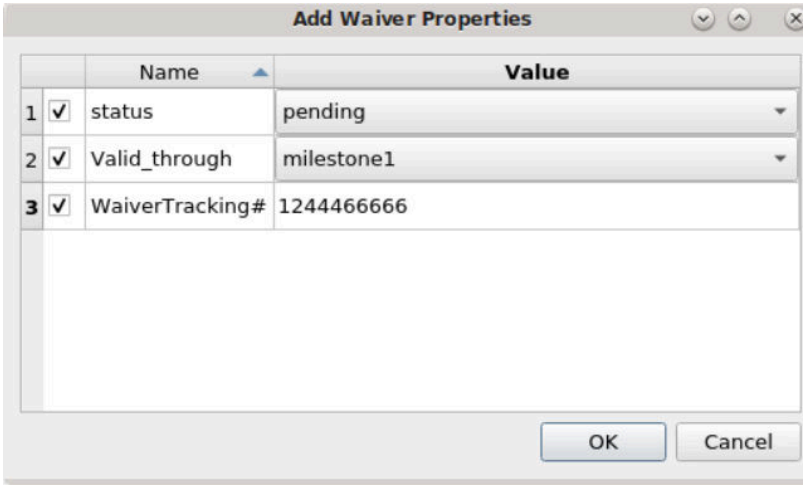
In the Violation Browser, right-click to select a rule and cell. The Add Waiver Properties option is now available.

Feedback

Figure 104 Add Waiver Properties

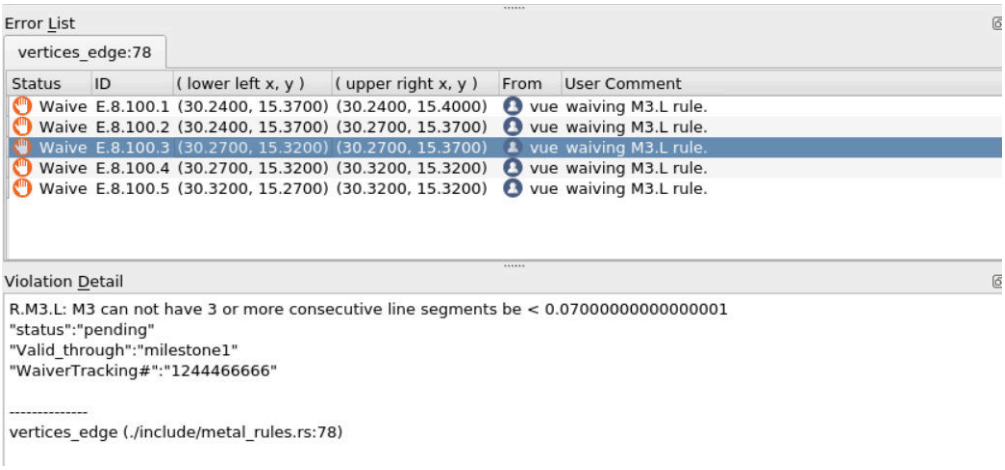


Select this option to define the user properties.



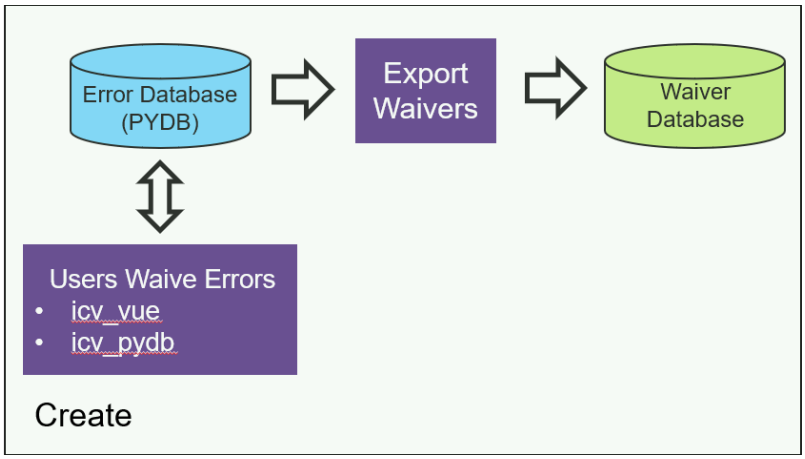
All waived errors for the rule/cell inherit the properties. You can see the properties in the Violation Detail pane, as shown here.

Feedback



Creating a Waiver Database

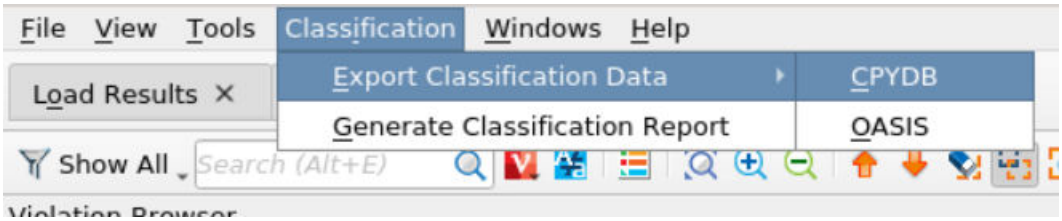
After you define all of the waivers for the current design, export the information to a standalone waiver database. The waiver database can either be in SQLite format (CPYDB) or OASIS format. The waiver database is self contained. This means that all information required for waiver errors is contained within the database, that is, user comments, user-IDs, dates, waiver properties, and so on.



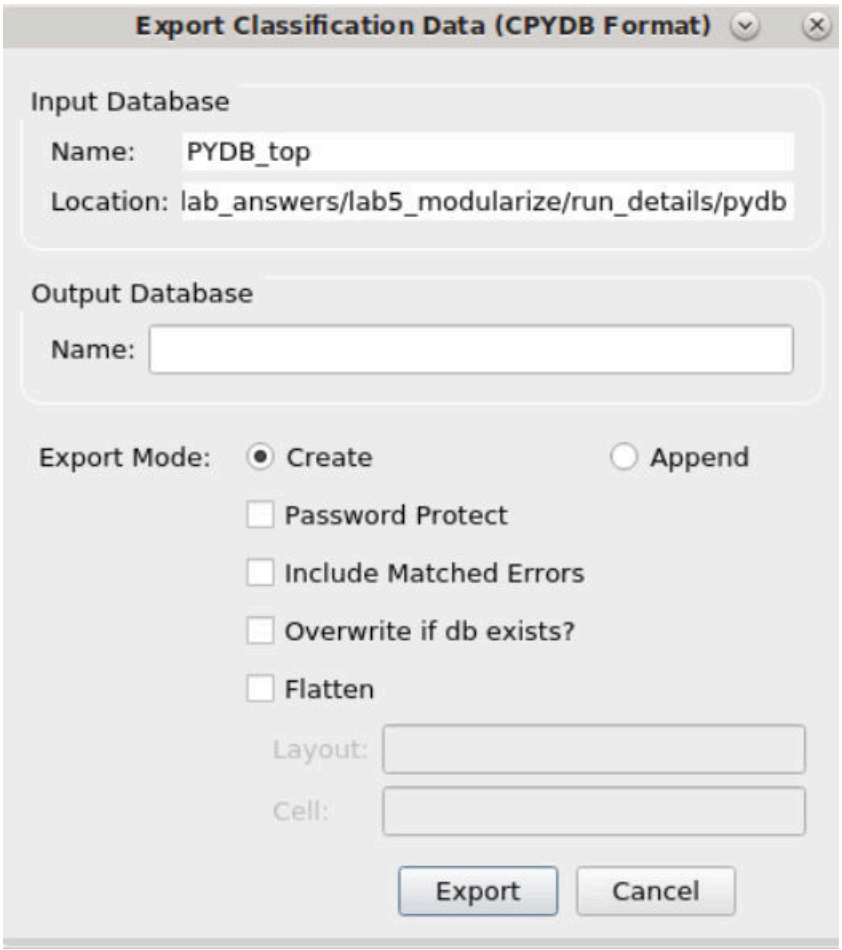
You can export a waiver database using VUE or from the pydb_export command-line utility.

To export a SQLite waiver database using VUE, select Classification > Export Classification Data (CPYDB).

Feedback



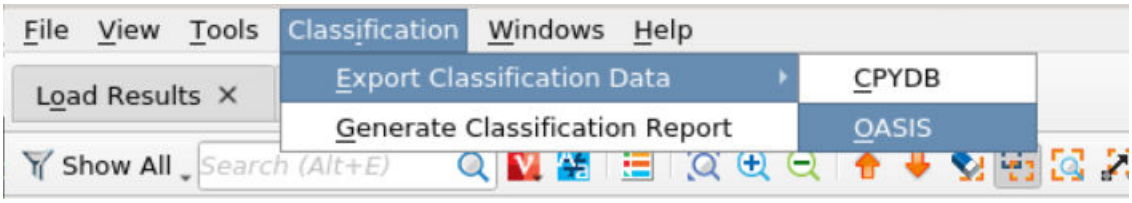
Here you can define the details of the database.



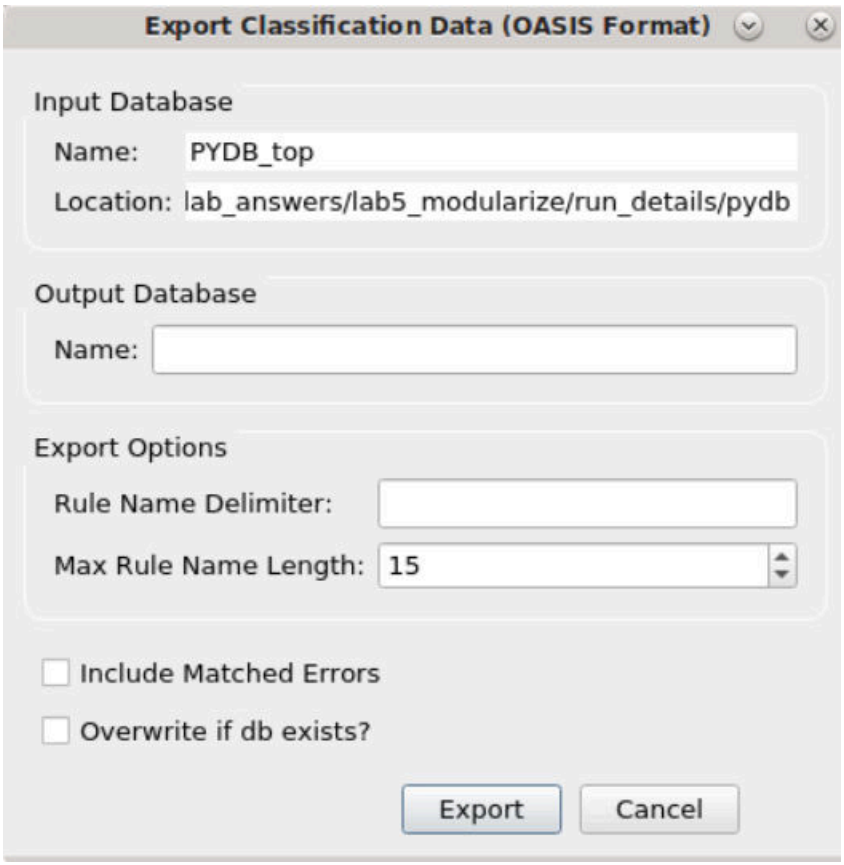
The output database name is written to the working directory where VUE was started. To write the file to another path, include that path in the output file name. By default, IC Validator creates a new waiver database. You can append this information into an existing CPYDB database by setting the Append Mode. During export, VUE exports any new waivers defined in the error database. To include all waivers, including those that were previously matched from another waiver database, check the Include Matched Errors box. The SQLite format allows you to set a password for the waiver database. This prevents modifying the waiver database unless the password is known.

Feedback

To export an OASIS format waiver database from VUE, select Classification > Export Classification Data (OASIS).



Here you see a window that provides the settings for the waiver database.



The output database name is written to the working directory where VUE was started. To write the file to another path, include that path in the output file name. The output OASIS format places waiver shapes in a specially named cell that provides the rule and cell name for each set of waivers (Rule_Name@cellname). The Rule Name Delimiter allows you to define the delimiter for your process nodes rule names. In the following examples, the rule names use the ":" character as a delimiter between rule name and rule description. When set, all characters to the left of the delimiter define the waiver cell name. If it is not set, or if the rule name is very long, the Max Rule Name Length truncates the waiver cell name

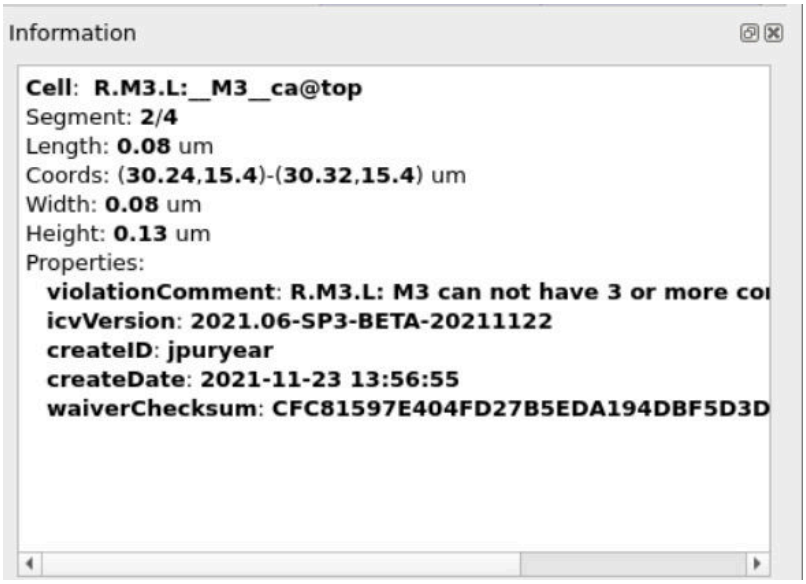
Feedback

to that length. The default is "[[:space:]]+:|[[:space:]]*:[[:space:]]", which matches most foundry runset naming conventions.

The OASIS format waiver database can now be opened IC Validator WorkBench. In the tree view for the waiver database, as shown the following figure, you can see that the database has a cell with the prefix @. This is a holding cell that contains all of the waivers defined for that cell. This cell does not contain any waiver shapes. Those are contained in the rule-specific cells that are placed within the holding cell.



Each waiver cell (rule@cell) has properties attached on the cell.



These properties are set during export. The person who exported the waiver database has their user-ID listed in the createID property as well as the timestamp of the export.

The waiverChecksum property is calculated on export for the cell. When IC Validator reads the waiver cell, it calculates a checksum and compares it to the stored property. If the properties are different, this indicates the database has been modified since it was

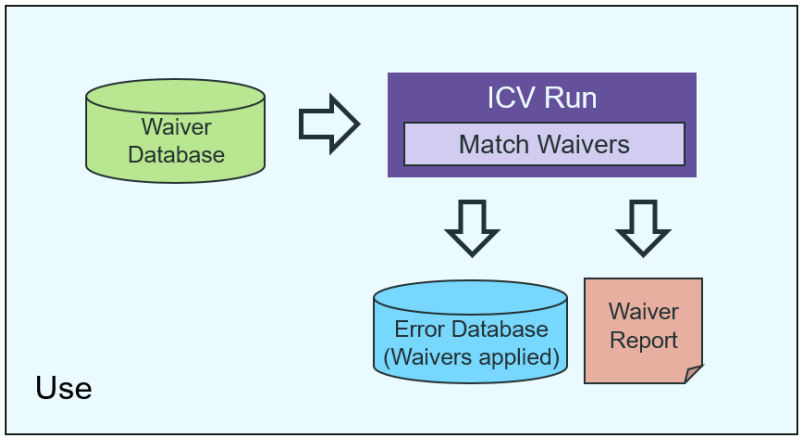
Feedback

exported. The end of the WAIVER_REPORT file includes information on whether the check sums match or not.

All waiver shapes for a given Violation/Cell are written to an arbitrary layer number. IC Validator does not use the layer number, but rather the violationComment property to know what rule the waiver shapes apply to. Therefore, no mapping file is needed.

Using a Waiver Database

After you create the waiver database, you can use it as input to other IC Validator runs. The new runs do not have to be the same top cell. IC Validator imports all waiver shapes the Cell / Violation present in the current run. IC Validator waivers are fully hierarchical.



The `waiver_options()` function provides control over how waivers are applied and reported during the IC Validator run. This command is divided into two main sections. The first section includes database management, where you provide the list of waiver databases and customize how each one is processed. The second section provides global waiver control that IC Validator uses for all waivers and reporting.

The database management is controlled with the `databases` argument. IC Validator automatically detects the database format, and both CPYDB and OASIS formats can be supplied in the same IC Validator run. Layer mapping files are not required when using OASIS format waiver databases.

For each waiver database, you can customize controls to provide flexibility across designs. This includes:

- Control over what happens if the waiver database cannot be found.
- If the errors waived with this database should be suppressed instead of classified. Suppressed means the waived errors are not written to the PYDB or LAYOUT_ERRORS file.

Feedback

- Cell prefixes or cell suffixes can also be given for a waiver database. This is useful if you have a flow that uniquifies cells when placed inside of larger macro blocks, but the waivers for the base cell can still apply to the renamed cell.
- Generating a special report that lists all waiver shapes in the database that were not matched during the IC Validator run.

The global waiver controls how waiver shapes are compared to the error shapes found during the run, including any rule-specific overrides and report generations.

Matching Modes

You can control how IC Validator matches a waiver shape to a given error shape. All matching occurs after all interacting waiver shapes have been merged together and interacting error shapes have been merged together. In some situations, to allow the merging to occur, the shapes might need to be leveled to a common parent cell. The matching is done hierarchically, that is, the level-merged waiver shapes do not need to be in the same cell as the level-merged errors shapes to find a match.

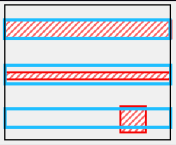
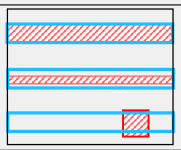
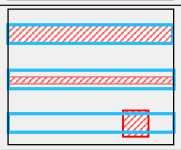
There are three matching modes available: EXACT, ENCLOSE (default) , and INTERACT.

- EXACT mode specifies that the level-merged waiver shape must exactly match the level-merged error shape for the waiver to be applied.
- ENCLOSE mode is the default matching mode. The waiver shape must enclose the error shape.
- INTERACT mode specifies that any error which interacts hierarchically with a waiver shape is waived. Use the `include_touch` argument to allow outside-touch interactions to be considered as an interaction for waiving.

Note:

Use the `tolerance` argument to provide better matching with all-angle shapes.

Feedback

Matching Mode	Cell With Errors	Waiver Result
EXACT • Exact shape match with tolerance – individual shapes or complete hierarchical shape		Waive Error Error
ENCLOSE • Default for waiver_options() • Errors enclosed by merged waiver shapes are matched		Waive Waive Error
INTERACT • Errors that interact with waiver shapes are matched. Include_touch specifies if outside-touch interactions are considered		Waive Waive Waive

The matching mode you set in the `waiver_options()` function apply to all waiver_databases. You can override the matching mode for specific rules using the `rule_overrides` argument. For example, ENCLOSE mode is allowed for all rules except R.MY.STRICT_RULE. For that rule, waivers are applied only if the waiver and error shapes exactly match. For example,

```
waiver_options(
  databases = { {"/path/to/waiver.oas"}},
  matching_mode  = ENCLOSE,
  rule_overrides = {
   violation_comments = {"R.MY.STRICT_RULE"},
   matching_mode = EXACT,
   tolerance     = 0.005
  }
 );
```

Waiver Report File

IC Validator writes all classified (waived) errors to a waiver report file in the IC Validator working directory (*cell*.WAIVER_REPORT).

The LAYOUT_ERRORS file and VUE run_summary tab state

```
ERRORS (WAIVERS APPLIED)
```

The waiver report file is generated by default with a summary of rules, including applied waivers and a list of input waiver databases.

```
--------------------------------------------------------------------------------
WAIVER SUMMARY
--------------------------------------------------------------------------------
```

Feedback

```
R.M3.L: M3 cannot have 3 or mor... | Waive : 5 | Ignore : 0 | Watch : 0 |
 WaiverConflict : 0 | Unmatched : 0
R.M4.W: M4 Minimum width >= 0.24    | Waive : 4 | Ignore : 0 | Watch : 0
 | WaiverConflict : 0 | Unmatched : 0
```

The table columns are: Rule name, Waive, Ignore, Watch Class counts, WaiverConflict count, and Unmatched waiver counts.

- A waiver conflict occurs when interacting waiver shapes with different waiver classes are level-merged together.
- Unmatched is a count of waiver shapes for the rule across all input waiver database that did not have a match in the current design.

IC Validator can also generate a detailed waiver report by setting

```
waiver_options(report_waiver_details = true)
```

This adds a table for every waiver, the cell, waiver class, matching mode, context, the waiver database with the waiver shape that matched, any waiver properties, and error details (distance, bounding box, and so on).

Note:

The `waiver_options()` function can be modified by using the `-runset_config` command-line option.

You can generate the waiver report file after an IC Validator run completes. This can be useful if you do not want to view the details that are on by default in your IC Validator runs, but want to see them after the run.

```
%icv_pydb -i ./run_details/pydb/PYDB_top \
     -create_waiver_report top.DETAILED_WAIVERS \
     -include_waiver_details
```

A JSON-formatted version of the waiver report file can also generated using the icv_pydb utility. If you need to write a script to process the waivers applied on a given design, this structured format is an easier file to parse.

```
% icv_pydb -i ./run_details/pydb/PYDB_top \
      -create_json_waiver_file top.waiver_file \
      -include_waiver_details
```

Example 43 JSON Format Waiver Report File

```
{
    "icv_run_info": {
    "library_name": "../../../lab_data/INLIB_TRAINING.GDS",
    "structure_name": "top",
    "generated_by": "IC Validator RHEL64
 S-2021.06-SP3-BETA-20211123.7086978 2021/11/23",
    "runset_name": "../modular.rs",
```

Feedback


```
    "user_name": "johndoe",
    "time_started": "2021/11/24 12:00:58PM",
    "time_ended": "2021/11/24 12:01:13PM"
  },
  "rules": [
    {"rule": "R.M3.L", "violation_comments": ["R.M3.L: M3 cannot have 3
 or more consecutive line segments be < 0.07000000000000001"]},
    {"rule": "R.M4.W", "violation_comments": ["R.M4.W: M4 Minimum width
 >= 0.24"]}
  ],
  "waiver_databases": [
    {
      "waiver_database_id": "WDB1",
      "file":
 "/path/to/johndoe/TRAINING/drc/lab_answers/lab5_modularize/mywaivers_db.
oas",
      "checksum": "PASS"
    }
  ],
  "waiver_details": [
    {
      "rule": "R.M3.L",
      "command": ".././include/metal_rules.rs:78:vertices_edge",
      "cell": "top",
      "class": "Waive",
      "mode": "Enclose",
      "context": "ALL",
      "waiver_sources": ["WDB1:top"],
      "unique_id": "E.12.100.1",
      "bbox": [30.2400, 15.3700, 30.2400, 15.4000],
      "user_comments": ["waiving M3.L rule."]
    }
],
  "unmatched_waiver_details": [  ],
  "rule_overrides": [  ]
```

16

Error Management Utility

This chapter explains the error management utility that is available with the IC Validator tool.

The error management utility is described in the following sections:

- Error Management Utility
- Splitting Error Databases
- Merging Error Databases
- Applying Waivers to an Existing Error Database
- VUE File Support
- Modifying Databases in Place
- Hierarchy Considerations
- XML Command File
- Exporting Errors to a Layout
- Managing Referenced cPYDB Paths

Error Management Utility

Use the error management utility (icv_pydb) to

- Split an error database (PYDB) from a single IC Validator run into multiple databases for debugging purposes.
- Select errors from an input PYDB and create a new PYDB with only the selected errors.
- Merge multiple PYDB databases together.
- Alter an existing PYDB database by renaming violations or cells, or classifying errors.

Feedback

- Export errors in a PYDB database to a layout.
- Group DRC errors.

The icv_pydb utility provides command-line options to enable any selection by cell (either individually or hierarchically), violation comment,violation name, error classification, or cell-level bounding box.

This utility is in the IC Validator installation bin directory. Table 68 defines the command-line options.

Table 68 Error Management Utility Command-Line Options

Command-Line Option	Definition
Input Options	
`-i input db`	Specifies path to an input database, for example run_details/pydb/PYDB_top. Required. This option must be given before any input selection options that apply to it. All other input options apply only to the input database which precedes them on the command line.
`-svc violation_comment` `-select_violation_comment violation_comment`	Includes only those violation comments which match the given pattern.
`-uvc violation_comment` `-uvc violation_comment violation_comment`	Does not include violation comments which match the given pattern.
`-select_cell cell`	Includes only those cells which match the given pattern. This command-line option does not include any cells placed at any level underneath the matching cells.
`-unselect_cell cell`	Does not include cells which match the given pattern. This command-line option does not exclude any cells placed at any level underneath the matching cells.
`-select_cell_hierarchical cell`	Includes only those cells which match the given pattern. This command-line option also includes all descendants of the matching cells.
`-unselect_cell_ hierarchical cell`	Does not include cells which match the given pattern. This command-line option also excludes all descendants of the matching cells.
`-select_interacting cell,x1,y1,x2,y2`	Includes only those errors in the given cell which interact with the given window. This command-line option is cell level and does not apply to any errors underneath the given cell hierarchically. Window selections are combined so that errors included by any of the window selections in a cell, but do not match any unselections, are included.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-unselect_interacting cell,x1,y1,x2,y2`	Excludes errors in the given cell which interact with the given window. This command-line option is cell level and does not apply to any errors underneath the given cell hierarchically.
`-select_enclosed cell,x1,y1,x2,y2`	Includes only those errors in the given cell which are enclosed by the given window. This command-line option is cell level and does not apply to any errors underneath the given cell hierarchically. Window selections are combined so that errors included by any of the window selections in a cell, but do not match any unselections, are included.
`-unselect_enclosed cell,x1,y1,x2,y2`	Excludes errors in the given cell which are enclosed by the given window. This command-line option is cell level and does not apply to any errors underneath the given cell hierarchically.
`-select_classified Waive,Ignore,Error,...`	Includes only those errors with the given classifications.
`-unselect_classified Waive,Ignore,Error,...`	Does not include errors with the given classifications.
`-sln layer_name` `-select_layer_names layer_name`	Specifies only those errors that are derived from runset layer names matching the given pattern. Layer information is available only if IC Validator was run with the `-vue` command-line option.
`-uln layer_name` `-unselect_layer_nameslayer_name`	Specifies that errors which are derived from the given layers are not included. Layer information is available only if IC Validator was run with the `-vue` command-line option.
`-sl layernum[-range] [,datatype[-range]] ...` `-select_layers layernum[-range] [,datatype[-range]] ...`	Specifies only those errors that are derived from the given layers. For OpenAccess layouts, the layers are specified by layout layer and purpose name, instead of layer and datatype number. Layer information is available only if IC Validator was run with the `-vue` command-line option.
`-ul layernum[-range] [,datatype[-range]] ...` `-unselect_layers layernum[-range] [,datatype[-range]] ...`	Specifies that errors which are derived from the given layers are not included. For OpenAccess layouts, the layers are specified by layout layer and purpose name, instead of layer and datatype number. Layer information is available only if IC Validator was run with the `-vue` command-line option.
`-set tier_number[-range] ...` `-select_explorer_tiers tier_number[-range] ...`	Includes only those errors which are part of the given Explorer tier numbers.
`-uet tier_number[-range] ...` `-unselect_explorer_tiers tier_number[-range] ...`	Does not include errors which are part of the given Explorer tier numbers.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-svn` *violation_name* `-select_violation_name` *violation_name*	Includes only violation names which match the given pattern. The violation name corresponds to the left side of the `@=` in the runset.
`-uvn` *violation_name* `-unselect_violation_name` *violation_name*	Does not include violation names which match the given pattern. The violation name corresponds to the left side of the `@=` in the runset.
`-exclude_heat_map`	Does not copy heat maps to the output database. This command-line option can be used if the heat map grids of the same cell in the input database and output database do not align to avoid errors.
`-exclude_ error_classification`	Does not copy error classification to the output database.
`-rename_cells` *rename_cells_file*	Changes cell names in the output according to the given CSV file, with the first column being the old cell name and the second column being the new cell name. This command-line option can be used to modify the input database in place if there is no output database specified. If modifying a database in place, this operation requires that no other users be connected to the database.
`-rename_violations` *rename_violations_file*	Changes violation comments in the output according to the given CSV file, with the first column being the old comment and second column being the new comment. This command-line option can be used to modify the input database in place if no output database is specified. If modifying a database in place, this operation requires that no other users be connected to the database.
`-new_top_cell` *orig_cell*`,`*new_cell*`[,`*x*`,`*y*`,`*angle*`,`*reflected*`,[flat_placement_count]]`	Specifies a different destination top cell than the source database, and optionally provides a translation from the source top cell to the destination top cell. If the new top cell is below the original top cell in the hierarchy, a translation is not needed. If the new cell is above the original top cell in the hierarchy a single instance translation of the original cell to the new top cell. The instance translations to top from the source database are updated to the context of the new top cell to enable highlighting violations in the new top cell. Optionally, a flat placement count of the old top cell within the new top cell might be provided to keep flat error counts accurate in the context of the new top cell. If no flat placement count is provided, the default is one placement.
`-classify` *classify file*	Classifies errors in the input database based on the given input file. This command-line option can only be used to modify a PYDB in place. See the "Classify File Format" section for more information.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-classify_by_window [interacting|enclosed]`	Specifies how errors are selected for classification based on window coordinates in the classify file. The default is `interacting`. • `interacting`. Errors having any interaction with the window coordinates are selected for classification. • `enclosed`. Errors completely enclosed by the window coordinates are selected for classification. Selected errors can touch the enclosing window.
`-report_user_comments`*`All | most_recent`*	Specifies which user comments are included in the JSON waiver report. The default is *`All`*.
`-apply_cell_level_waivers` *`waiver_database`*	Applies waivers from the given waiver database to the input PYDB using cell level exact matching. This operation may only be used to modify a PYDB.
`-waiver_matching_tolerance` *`microns`*	Specifies an optional tolerance value to be used when matching error shapes to waiver shapes while applying cell level waivers.
`-waiver_tool_renamed_cells` *`MATCH_NEW_NAME | MATCH_ORIGINAL_NAME`*	When applying waivers, determines how to look up cells in the waiver database when the IC Validator tool has renamed the cells. The default is `MATCH_NEW_NAME`.
`-waiver_cell_prefixes` *`prefix1,prefix2,...`*	When applying waivers, cells from the waiver database are used if they match a PYDB cell with any of the removed-listed prefixes.
`-waiver_cell_suffixes` *`suffix1,suffix2,...`*	When applying waivers, cells from the waiver database are used if they match a PYDB cell with any of the removed-listed suffixes.
`-set_waiver_context` *`cellname | ALL`*	When classifying errors with a CSV file, sets the given cell as the waiver context for the errors being classified. This means that the waivers are applied only if that cell is the top cell of the design being checked. This is typically the top cell of a block-level run for waivers that are only applicable at the block level. If `ALL` is specified, the waivers are always applied.
`-set_waiver_properties`*`user_property_file.json`*	Sets waiver user properties from the given JSON waiver property file. This operation can be used only to modify a cPYDB or OASIS waiver database in place.
`-waiver_property_definition_file` *`property_definition_file.json`*	Specifies the JSON file to be used for validating waiver user properties. This overrides the environment setting of `ICV_WAIVER_PROPERTY_DEFINITION_FILE`. See Custom Waiver Properties.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-select_waiver_properties`	Filters the given waiver database to include only waivers that match the waiver user properties specifies in the given JSON property selection file. This operation can be used only to modify a cPYDB or OASIS waiver database in place.
`-rule_name_delimiter`	Specifies the rule name delimiter regular expression used to separate the rule name from the rest of the violation comment when selecting based on rule name instead of the full violation comment, such as when dealing with waiver properties. The rule name delimiter has a default of `/[[:space:]]+:|[[:space:]]*:[[:space:]]/`
`-replace_user_ids` *`new_user_id`*	Globally replaces all user IDs in the input cPYDB or OASIS waiver database with the given value. This operation can only be used to modify a cPYDB or OASIS waiver database in place, and it requires that no other users be connected to the database.
`-delete_classification_ comments`	Deletes all classification comments in the input cPYDB or OASIS waiver database. This operation can only be used to modify a cPYDB or OASIS waiver database in place, and it requires that no other users be connected to the database.
`-replace_db_path` *`new_db_path`*	Replaces the database path stored in the input cPYDB with the given value. This operation might only be used to modify a cPYDB in place, and it requires that no other users be connected to the database.
`-rename_cpydb_paths` *`pattern new_path`*	Changes any referenced cPYDB path matching the given pattern to the specified new cPYDB path. Only the path component of the referenced cPYDB is updated. This command-line option can be used only to modify a PYDB in place.
`-group_drc_errors`	Determines unique error groups for the selected DRC errors. Errors are placed in the same group if their violation marker shape and check values are identical, regardless of orientation. Only certain commands are eligible for grouping. This process replaces any existing grouping. See DRC Error Grouping for more information.
`-minimum_errors_per_group`	Adjusts the threshold of errors needed to make a group. The minimum threshold is 3. See DRC Error Grouping for more information.
`-group_same_orientation`	Groups together errors with the same orientation. The default is to group errors regardless of orientation. See DRC Error Grouping for more information.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
Output Options	
`-o output db`	Specifies the path to an output database. If this command-line option is not given, the input database is modified in place. If the output database does not exist, it is created. If the output database does exist, errors is merged into it from the input database, unless the `-overwrite` command-line option is specified. All other output command-line options apply to the output database, which precedes them on the command line. Output to this database comes only from the input databases which precede it on the command line since the last output database was specified. When exporting to a layout using the `-format` command-line option, the `-o` command-line option specifies the output layout path.
`-overwrite`	If the output database exists, it is overwritten. If this command-line option is not specified, the default behavior is to merge errors into an existing output database, or create a new output database if none exists.
`-vue output VUE file`	Creates the given VUE file which points to the output PYDB.
`-vue_template input_vue_file`	Uses the given input VUE file as a template to create the output VUE file.
`-dbu microns`	Optionally specifies a value in microns for the database unit of the output database. This value is used when creating a new database, or if merging into an existing database, it can be used to verify the DBU of the merged database. If this command-line option is not specified, the DBU of the input database is used when creating a new output database. When exporting to an output layout, if this argument is given, it is used as the DBU of the output layout. If this argument is not given, the DBU of the original input layout (which might be coarser than the DBU of the input databases) is used.
Export Options	
`-format GDSII|OASIS`	Indicates that the output file refers to a layout to be exported from the input database, and it specifies the format.
`-layout original_layout`	Specifies the original layout from which the design hierarchy is read when exporting errors to a layout. If this command-line option is not given, the layout path is taken from the PYDB. An input layout is required for the layout export flow to work, whether or not this argument is given.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-c top_cell`	Specifies the top cell of the layout. If this option is not given, the top cell of the input PYDB is used.
`-violation_map violation mapping file`	Specifies a file that maps rules and commands to a layer and datatype pair in the output layout. This CSV-formatted file includes the following columns: “Violation Comment”,”Command”,”Layer”,”Datatype”. Wildcards are supported for the violation comment and command columns. The command column is optional and can be left blank, but if present, select only the errors when the runset file:line:function match. By default, errors without an entry in this file are not output. If a violation mapping file is not given, rules are assigned an arbitrary layer and datatype.
`-include_unmapped`	Indicates that errors without an entry in the violation mapping file are included when exporting errors to a layout. These errors are assigned an arbitrary layer and datatype, which is not already contained in the violation mapping file.
`-violation_map_output output violation mapping file`	Writes a violation mapping file containing all of the layer/datatype pairs that were used in the export. This file can be reused in subsequent runs to reproduce the same layer and datatype choices for rules.
`-command_property_number property number`	Writes the command runset file:line:function information to the given property number on each output polygon when exporting errors to a layout. This can substantially increase the size of the output layout.
`-density_property_number property number`	Writes the properties of density violations and the property constraints of unmatched density waivers to the given property number on each output polygon. This might substantially increase the size of the output layout.

`-datatype_per_classification`	Writes errors of different classifications to separate datatypes. A value corresponding to the classification status of the error is added to the output datatype, as shown in the following example:
	`Classification Status  Added to Datatype`
	`Error/Unclassified  0`
	`Ignore  1`
	`Waive  2`
	`Watch  3`
	`Unmatched  4`
	`Fixed  7`
	`WaiverConflict  9`

`-host_init host1:#cores ... [hostN:#cores]`	Assigns hosts for distributed operations.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
General Options	
`-list_cells`	Shows a list of unique cells in the input database.
`-list_cell_details`	Shows a list of unique cells in the input database, including details such as flat placement counts and instance translations.
`list_cell_descendants`	Shows a list of unique cells in the input database, including detailed information on all of the unique descendants of each cell.

`-list_layers`	Shows a list of assign layers defined in the database if IC Validator was run with the `-vue` command-line option.
	For example,
	`icv_pydb -i run_details/pydb/PYDB_AD4FUL -list_layers`
	`...`
	`Layers defined in run_details/pydb/ PYDB_AD4FUL:`
	`tox (1/0)`
	`poly (5,0)`
	`well (31,0)`
	`psel (14,0)`
	`cont (6,0)`
	`met1 (8,0)`
	`met2 (10,0)`
	`via (19,0)`

`-list_violations`	Shows a list of unique violation comments in the input database.

`-include_layers`	Includes layer information in the violation listing, if available.
	For example,
	`icv_pydb -i run_details/pydb/PYDB_AD4FUL -list_violations -include_layers`
	`...`
	`Violations in database run_details/pydb/ PYDB_AD4FUL:`
	`"Met1 Spacing must be >= 3"`
	`  met1 (8,0)`
	`"Met1 to Met2 spacing must be >= 1 and not touching "`
	`  met1 (8,0)`
	`  met2 (10,0)`
	`"Met2 Spacing must be >= 3.0 and >5.0 for longedges >=50"`
	`  met2 (10,0)`

`-list_cpydbs`	Shows a list of cPYDB databases that are used to match error classifications in the input PYDB.

Feedback

Table 68 Error Management Utility Command-Line Options (Continued)

Command-Line Option	Definition
`-list_waiver_properties`	Shows a list of violation and cell pairs from the given cPYDB or OASIS waiver database that have waiver properties defined, and lists the values of the waiver properties.
`-check_waiver_database`	Checks the compliance of the input OASIS waiver database.
`-create_waiver_report`	Creates a WAIVER_REPORT file for the input database.
`-include_waiver_details`	Includes defaults of waived errors in the WAIVER_REPORT file. This overrides the runset setting of `waiver_options(report_waiver_details)`.
`-include_unmatched_waivers`	Includes details of unmatched waivers in the WAIVER_REPORT file. This overrides the runset setting of `report_waiver_details`.
`-create_json_waiver_file`	Creates a JSON waiver details file.
`-password` *password*	Specifies the password for write access to an existing error classification database. Specify `prompt` to read the password from the command line.
`-file` *command file*	Use the given XML command file to control tool operation.
`-help`	Displays the usage message.
`-v`	Prints the utility version.

DRC Error Grouping

The error database management utility can be used to place DRC errors into groups that have the same violation marker shape and properties. The error grouping can be used in VUE to display only a representative error from each group using the Show Unique and Highlight Unique menu options. By default, the grouping is done regardless of orientation. For example:

```
icv_pydb -i run_details/pydb/PYDB_TOPCELL -group_drc_errors
```

See Table 68 for information on the error management utility command-line options.

Feedback

Splitting Error Databases

If there are multiple blocks in a design,

```
          TOP
       /    |     \
Block1   Block2   Block3
 /  \     /  \     /  \
A   B     A  C     B   D
```

The results might be split into three error databases PYDBs), one for each subblock. For example:

```
icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block1"\
  -o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue

icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block2"\
  -o new_pydb/PYDB_Block2 -vue Block2.vue -vue_template TOP.vue

icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block3"\
  -o new_pydb/PYDB_Block1 -vue Block3.vue -vue_template TOP.vue
```

Use the `-select_cell_hierarchical` command-line option to automatically pull in any descendant cells placed underneath the selected cells. In this example, the command-line options pull in A and B under Block1, A and C under Block2, and B and D under Block3. Cells can be included or excluded individually without including subtrees using the `-select_cell` or `-unselect_cell` command-line options. Alternatively, this operation can be specified by a single icv_pydb command:

```
icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block1"\
  -o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue \
  -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block2" \
  -o new_pydb/PYDB_Block2 -vue Block2.vue -vue_template TOP.vue \
  -i run_details/pydb/PYDB_TOP -select_cell_hierarchical "Block3" \
  -o new_pydb/PYDB_Block3
```

In this example, the top cell of each output database is still TOP. To make the lower-level block the top cell, the `-new_top_cell` command-line option can be used.

```
icv_pydb -i run_details/pydb/PYDB_TOP -select_cell_hierarchical \
  -new_top_cell TOP,Block1 \
  -o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue
```

Selection by violation comment and by bounding box is also allowed. For example, to split out only the metal1 rules with no cell selections, the tool might be run as follows

```
icv_pydb -i run_details/pydb/PYDB_TOP -svc "M1" \
  -o new_pydb/PYDB_TOP_m1 -vue TOP_m1.vue -vue_template TOP.vue
```

However, notice that any heat maps that were at the top level are not copied in this case, since TOP no longer exists in the output database.

Feedback

Since PYDB does not contain cell placement data, bounding-box selection is valid only within a cell, and there is no looking down the hierarchy. For example, to include top-level data over block1 when the extents of the block1 placement are (100,200),(300,400), the tool might be run this way:

```
icv_pydb -i run_details/pydb/PYDB_TOP \
  -select_cell_hierarchical "Block1 \
  -select_interacting TOP,100.0,200.0,300.0,400.0 \
  -o new_pydb/PYDB_Block1 -vue Block1.vue -vue_template TOP.vue
```

Notice that when `error_options(flatten_violations)` is set, the flattening of violations is allowed to specified blocks during the IC Validator run when the entire hierarchy is available. Using the `flatten_violations` argument in conjunction with bounding-box selection allows for geographical partitioning of the errors.

Selection can also be done by error classification using several command-line options to include or exclude errors of certain classifications, and also to drop error classification data from the output.

Merging Error Databases

The icv_pydb utility provides the ability to merge multiple error databases (PYDBs) into one, for example, when subblocks or different sets of rules are run separately. In the simple case of merging several PYDBs with the same top cell, the icv_pydb utility can be run one time for each input database. For example,

```
icv_pydb -i metal1run/run_details/pydb/PYDB_TOP -o merged_pydb/PYDB_TOP
icv_pydb -i metal2run/run_details/pydb/PYDB_TOP -o merged_pydb/PYDB_TOP
...
```

If merged_pydb/PYDB_TOP does not exist, it is created and subsequent runs merge data into the existing database. Alternatively, they might be combined on the command line:

```
icv_pydb -i metal1run/run_details/pydb/PYDB_TOP \
         -i metal2run/run_details/pydb/PYDB_TOP \
         -o merged_pydb/PYDB_TOP
```

When merging multiple databases with different top cells, provide a (single instance) translation to the common top cell, so that “highlight in top cell” works properly in VUE. This can be done using the `-new_top_cell` command-line option. For example,

```
icv_pydb -i block1run/run_details/pydb/PYDB_Block1 \
  -new_top_cell Block1,TOP,100,100,0,0 -o merged_pydb/PYDB_TOP
icv_pydb -i block2run/run_details/pydb/PYDB_Block2 \
  -new_top_cell Block2,TOP,200,100,0,0 -o merged_pydb/PYDB_TOP
```

It is possible that the PYDBs being merged could be at different DBU resolutions. When merging into an existing PYDB, the icv_pydb tool always uses the DBU of the destination database. However, you can specify a DBU on the command line so that, when creating a

Feedback

new PYDB, the specified DBU is used, and when merging into an existing PYDB, the DBU of the existing database can be verified.

The `-dbu` command-line option allows the DBU to be given as a floating-point value in microns. If merging into an existing database with a different DBU from the value given on the command line, an error is issued and the tool exits.

Note:

Runset functions are not combined during merging. If two similar runset functions in the same violation are merged, they exist as separate functions in the destination database.

Applying Waivers to an Existing Error Database

After an IC Validator run, you can apply waivers to an existing PYDB using the error management utility. For example:

```
icv_pydb -i run_details/pydb/PYDB_TOP -apply_cell_level_waivers
 waivers.oas
```

The input waiver database can be in cPYDB or OASIS format. Waivers are matched at the cell level using exact shape matching. You can use the `-waiver_matching_tolerance` argument to set a tolerance. Each waiver shape must exactly overlap the error shape in the same cell for the waiver to apply. This process is sensitive to hierarchy or data cutting changes, which can make error shapes and waiver shapes different.

VUE File Support

When a new database is created, whether by splitting or merging, you can output a VUE file which points to the new database, by specifying the `-vue` command-line option. You can also use the corresponding `-vue_template` command-line option to specify an input VUE file to be used as a template by replacing only the database name. Otherwise, relevant information is taken from the input PYDB.

Modifying Databases in Place

Certain operations are supported for modifying a PYDB or cPYDB in place, without a destination database. These are:

```
-rename_cells rename_cells_file
-rename_violations rename_violations_file
-classify classify.csv
```

Feedback

For modify-in-place operations, only the input PYDB or cPYDB is specified using the `-i` command-line option. There is no output database.

Note:

The `-classify` command-line option is supported only with PYDB and not cPYDB.

Hierarchy Considerations

The PYDB stores cell descendant information for every unique cell, including an instance translation of all its descendants This enables the splitting of a database to properly update the instance translation to top for “Highlight In Top Cell” functionality.

A common use model might be to completely split out a subblock and make it the new top cell for the destination PYDB. For example,

```
icv_pydb -i run_details/pydb/PYDB_TOP -o new_pydb/PYDB_Block1 \
  -select_cell_hierarchical "Block1" -vue Block1.vue -vue_template \
   TOP.vue \
  -new_top_cell TOP,Block1
```

In this example hierarchy, a new PYDB is created with Block1 as the top cell and all violations in Block1 and below it are selected. Since PYDB has all of the cell descendant information, a translation is not required when the new top cell is below the original top cell. An error is issued if the new top cell is below the original top cell and cell selection includes anything above the new top cell.

XML Command File

The XML command file allows any selections available on the command line as well as selection of individual errors by ID. Similar to using command-line options, the XML file can specify multiple source databases and different selections for each. The file can be gzipped.

All standard rules for XML formatting must be followed. Of particular note is that all parameter values must be enclosed in quotation marks, and any special characters such as angle brackets, ampersand, or quotes must be escaped. Example 44 shows the syntax of the XML command file.

Example 44 XML Command File Syntax

```
<?xml version="1.0"?>
<pydb>
  <pydb_operation>
     <input_pydb path="run_details/pydb/PYDB_AD4FUL">
      field_selection_mode="UNION|INTERSECTION">
```

Feedback

```
<select_cell pattern="cellA*"/>
<unselect_cell pattern="cellB*"/>
<select_cell_hierarchical pattern="cellA*"/>
<unselect_cell_hierarchical pattern="cellB*"/>
<select_violation_comment pattern="Rule1*"/>
<unselect_violation_comment pattern="Rule2*"/>
<select_enclosed cell="cellname" x1="1.0" y1="2.0" x2="3.0" \
 y2="4.0"/>
<unselect_enclosed cell="cellname" x1="1.0" y1="2.0" x2="3.0" \
 y2="4.0"/>
<select_interacting cell="cellname" x1="1.0" y1="2.0" x2="3.0" \
 y2="4.0"/>
<unselect_interacting cell="cellname" x1="1.0" y1="2.0" x2="3.0"\
 y2="4.0"/>
<select_classified>
  <class name="Waive"/>
  ...
</select_classified>
<unselect_classified>
  <class name="Waive"/>
  ...
</unselect_classified>
<select_layer_names pattern="m1*"/>
<unselect_layer_names pattern="m2*"/>
<select_layers range="10-11,0-55"/>
<unselect_layers range="10-11,0-255"/>
<select_explorer_tiers range="<range>"/>
<unselect_explorer_tiers range="<range>"/>
<rename_cell old="cellA" new="cellB"/>
<rename_cells file="rename_cells.csv"/>
<rename_violation old="Rule1" new="Rule2"/>
<rename_violations file="rename_violations.csv"/>
<top_cell old="top1" new="top2" dx="1.0" dy="2.0" angle="90" \
 reflected="1" flat_placement_count="1"/>
<!-- Select individual errors by ID -->
<select_errors command_id="1" cell_id="2">
  <error id="1"/>
  <error id="2"/>
  ...
</select_errors>
<!-- No error IDs given == all errors selected -->
<select_errors command_id="2" cell_id="2"/>
<!-- No error IDs given, no cell ID given == entire command \
 selected -->
<select_errors command_id="3"/>
<!-- Select errors by exact value of a field -->
<select_errors field="Instance Name" value="I123"/>
<select_errors field="Distance" value="4.5"/>
<!-- Select errors by wildcard pattern -->
<select_errors field="Instance Name" pattern="I4*"/>
<!-- Select errors based on a range of values -->
<select_errors field="Distance" min="1.0" max="2.0"/>
<!-- Selection conditional on another field for commands like
```

Feedback

```
            density -->
          <select_errors field="Value" min="1.0" max="2.0"
            when_field="Report" is="Area"/>
          <!-- Classify errors -->
          <classify_errors class="Waive" comment="Waive errors in IP block"
          field_selection_mode="UNION|INTERSECTION">
          <!-- Any selection options might go here -->
          </classify_errors>
          <exclude_heat_map/>
          <exclude_error_classification/>
        </input_pydb>
        ...
        <output_pydb path="/path_to_pydb" dbu="0.0001">
          <overwrite/>
          <vue file="output.vue" template="inblock.vue"/>
        </output_pydb>
        <output_layout file="output_layout_file" format="GDSII|OASIS">
          <violation_map file="mapping_file" out="output_file"/>
          <include_unmapped/>
          <input_layout file="input_layout_file" cell="top_cell"/>
          <command_property number="property_number"/>
          <host_init host="host1:#cores"/>
          <host_init host="hostN:#cores"/>
        </output_layout>
      </pydb_operation>
      ...
</pydb>
```

The XML command file syntax is defined as follows:

- `<pydb>`

  Top-level enclosing tag for the command file XML document. There must be one and only one `<pydb>` tag, as everything else is enclosed in it.

- `<pydb_operation>`

  Top-level enclosing tag of a single database operation. For example, a split, merge, or copy. There might be multiple `pydb_operation` tags in a file.

- `<input_pydb path="path/to/pydb"`
  `field_selection_mode="UNION|INTERSECTION">`

  Designates an input PYDB database, just as the `-i` command-line option does on the command line. All selections and command-line options within this section apply only to the given input database. If there is no output database specified, the input PYDB can be modified in place. Multiple input databases can be specified for a single `pydb_operation` in the XML command file.

  The optional `field_selection_mode` attribute controls how `<select_errors>` tags based on field values are combined. The default is `UNION`, which means that errors which meet any `<select_errors field...>` criteria is included. If `INTERSECTION` is

Feedback

specified, then only errors which meet all of the `<select_errors field...>` criteria is included.

- `<select_layer_names pattern="m1*"/>`

  Specifies that only layer names matching a given pattern name (wildcards allowed) are included from this database.

- `<unselect_layer_names pattern="m2*"/>`

  Specifies that no layer names matching a given pattern name (wildcards allowed) are included from this database.

- `<select_layers range="10-11,0-255"/>`

  Specifies that only layers matching the given range are included from this database.

- `<unselect_layers range="10-11,0-255*"/>`

  Specifies that no layers matching the given range are included from this database.

- `<select_cell pattern="cellA*"/>`

  Specifies that only cells matching the given pattern (wildcards allowed) are included from this database. Equivalent to the `-select_cell` command-line option, and can be specified multiple times.

- `<unselect_cell pattern="cellA*"/>`

  Specifies that no cells matching the given pattern (wildcards allowed) are included from this database. Equivalent to the `-unselect_cell` command-line option, and can be specified multiple times.

- `<select_cell_hierarchical pattern="cellA*"/>`

  Specifies that only cells matching the given pattern (wildcards allowed) and their descendants are included from this database. Equivalent to the `-select_cell_hierarchical` command-line option, and can be specified multiple times.

- `<unselect_cell_hierarchical pattern="cellA*"/>`

  Specifies that no cells matching the given pattern (wildcards allowed) or their descendants are included from this database. Equivalent to the `-unselect_cell_hierarchical` command-line option, and can be specified multiple times.

- `<select_violation_comment pattern="cellA*"/>`

  Specifies that only violation comments matching the given pattern (wildcards allowed) are included from this database. Equivalent to the `-svc` command-line option, and can be specified multiple times.

Feedback

- `<unselect_violation_comment pattern="cellA*"/>`

  Specifies that no violation comments matching the given pattern (wildcards allowed) are included from this database. Equivalent to the `-uvc` command-line option, and can be specified multiple times.

- `<select_enclosed cell="cellname" x1="1.0" y1="2.0" x2="3.0" \ y2="4.0"/>`

  Specifies that in the given cell, only errors which are enclosed by the given window coordinates are included. Equivalent to the `-select_enclosed` command-line option, and can be specified multiple times.

- `<unselect_enclosed cell="cellname" x1="1.0" y1="2.0" x2="3.0" \ y2="4.0"/>`

  Specifies that in the given cell, no errors which are enclosed by the given window coordinates are included. Equivalent to the `-unselect_enclosed` command-line option, and can be specified multiple times.

- `<select_interacting cell="cellname" "1.0" y1="2.0" x2="3.0" \ y2="4.0"/>`

  Specifies that in the given cell, only errors which interact with the given window coordinates are included. Equivalent to the `-select_interacting` command-line option, and can be specified multiple times.

- `<unselect_interacting cell="cellname" "1.0" y1="2.0" x2="3.0" \ y2="4.0"/>`

  Specifies that in the given cell, no errors which interact with the given window coordinates are included. Equivalent to the `-unselect_interacting` command-line option, and can be specified multiple times.

- `<select_classified> ... </select_classified>`

  Specifies that only errors that match the given list of classifications are included. Equivalent to the `-select_classified` command-line option.

- `<unselect_classified> ... </unselect_classified>`

  Specifies that only errors that match the given list of classifications are included. Equivalent to the `-unselect_classified` command-line option.

- `<rename_cell old="cellA" new="cellB"/>`

  Renames a single cell from this input database. Can be specified multiple times.

- `<rename_cells file="rename_cells.csv"/>`

Feedback

Renames cells from this input database according to the given CSV file. Equivalent to the `-rename_cells` command-line option.

- `<rename_violation old="Rule1" new="Rule2"/>`

  Renames a single violation from this input database. Can be specified multiple times.

- `<rename_violations file="rename_violations.csv"/>`

  Renames violations from this input database according to the given CSV file. Equivalent to the `-rename_violations` command-line option.

- ```
  <top_cell old="top1" new="top2" dx="1.0" dy="2.0" angle="90" \
   reflected="1"/>
  ```

  Specifies a different destination top cell than the source database, and provides a translation from the source top cell to the destination top cell. This tag is required if merging into a PYDB with a different top cell. The instance translations to top from the source database are updated to the context of the new top cell to enable highlighting violations in the new top cell.

- ```
  <select_errors command_id="1" cell_id="2">
   <error id="1"/>
  <error id="2"/>
   ...
  </select_errors>
  ```

  Selects individual errors to be included from this input PYDB using numeric IDs from the source database. If there are no `<error>` records within the `<select_errors>` record, all errors from the command/cell are included. If the `cell_id` is not specified, all errors from the command are included. This tag can be specified multiple times.

- ```
  <select_errors field="Instance Name" value="I123"/>

  <select_errors field="Distance" value="4.5"/>
  ```

  Specifies that errors where the given string, integer, or double field is equal to the given value are included. The field name should be the heading of the column from the LAYOUT_ERRORS file. Field-based selection options are combined according to the `field_selection_mode` attribute of the `<input_pydb>` or `<classify_errors>` section in which they occur.

- `<select_errors field="Instance Name" pattern="I4*"/>`

  Specifies that errors where the given string field matches the given wildcard pattern are included. The field name should be the heading of the column from the LAYOUT_ERRORS file. Field-based selection options are combined according to
  ```

Feedback

the field_selection_mode attribute of the <input_pydb> or <classify_errors> section in which they occur.

- <select_errors field="Distance" min="1.0" max="2.0"/>

  Specifies that errors where the given double or integer field is in the given range are included. The field name should be the heading of the column from the LAYOUT_ERRORS file. Field-based selection options are combined according to the field_selection_mode attribute of the <input_pydb> or <classify_errors> section in which they occur.

- <select_errors field="Value" min="1.0" max="2.0" \
  when_field="Report" is="Area"/>

  Specifies that errors which meet the given selection criteria are included. The addition of the when_field and is attributes make this selection conditional on another value in the row. This is useful for functions such as density(), when there might be many Report and Value pairs for a single error and the selection must be only for a particular Report. The previous example selects density errors with Area in the range of 1.0-2.0. The field name and when_field should be the headings of the appropriate columns from the LAYOUT_ERRORS file. Field-based selection options are combined according to the field_selection_mode attribute of the <input_pydb> or <classify_errors> section in which they occur.

- <classify_errors class="Waive" comment="Waive errors in IP block"
  field_selection_mode="UNION|INTERSECTION">
  <!-- Any selection options might go here -->
  </classify_errors>

  Classifies errors in this input PYDB according to the given class and applies the optional user comment. Any error selection options might be specified inside the <classify_errors> section and combined with selection options that are outside of the <classify_errors> section. Multiple <classify_errors> sections can be specified in a single <input_pydb> section.

  The optional field_selection_mode attribute controls how <select_errors> tags based on field values are combined inside this <classify_errors> section. The default is UNION, which means that errors which meet any <select_errors field...> criteria are included. If INTERSECTION is specified, then only errors which meet all of the <select_errors field...> criteria are included.

- <exclude_heat_map/>

  Specifies that heat map data is not be copied from the input database to the output database. Equivalent to the -exclude_heat_map command-line option.

Feedback

- `<exclude_error_classification/>`

  Specifies that error classification data is not be copied from the input database to the output database. Equivalent to the `-exclude_error_classification` command-line option.

- `<output_pydb path="path/to/pydb" dbu="0.0001">`

  Designates an output PYDB database, just as the `-o` command-line option does on the command line. The `dbu` parameter is optional and is equivalent to the `-dbu` command-line option. The `output_pydb` option can be specified at the most, one time within a single `pydb_operation`.

  - `<overwrite/>`

    Specifies that if the output database exists, it is overwritten. Otherwise, the data is merged into an existing output database. Equivalent to the `-overwrite` command-line option.

  - `<vue file="output.vue" template="inlib.vue"/>`

    Specifies that the given VUE file should be created which points to the new output database. Equivalent to the `-vue` command line option. The optional template parameter specifies an existing VUE file to use as a template.

- `<output_layout file="output_layout_file" format="GDSII|OASIS">`

  Specifies that violations from the input PYDB are exported to the output layout specified by the given file and format.

  - `<violation_map file="mapping_file" out="output_file"/>`

    Specifies a violation mapping file or output violation mapping file for this output layout.

  - `<include_unmapped/>`

    If a violation mapping file is used, specifies that violations without an entry in the violation mapping file are included in the output on arbitrarily assigned layers and datatypes.

  - `<include_unmapped/>`

    Specifies the input layout from which the design hierarchy is read.

  - `<command_property number="property_number"/>`

    Specifies that the runset file:line:function is added to each polygon in the output layout on the given property number. This might increase the size of the output layout substantially.

Feedback

- `<host_init host="host1:#cores"/>`

  Assigns hosts for distributed operations.

Classifying Errors With the XML Command File

The XML command file provides a way to classify errors using the icv_pydb tool. Any selection options available for other icv_pydb operations can also be used to select errors for classification. Example 45 illustrates several common waiver scenarios.

Example 45 Common Waiver Scenarios

```
<?xml version="1.0"?>
<pydb>
  <pydb_operation>
     <input_pydb path="run_details/pydb/PYDB_AD4FUL"

       <!-- Waive all the errors for an entire rule -->
       <classify_errors class="Waive" comment="Waiver for rule M1.R1">
         <select_violation_comment pattern="M1.R1*"/>
       </classify_errors>

       <!-- Ignore all the errors in particular cells -->
       <classify_errors class="Ignore" comment="Ignoring errors in IP \
        cells">
         <select_cell pattern="MYIP*"/>
        </classify_errors>

        <!-- Classify errors in a window -->
        <classify_errors class="Watch" comment="Fix in next rev">
          <select_enclosed cell="TOP" x1="0.0" y1="0.0" x2="10.0" \
           y2="5.0"/>
        </classify_errors>

        <!-- Waive metal1 spacing errors based on Distance -->
        <classify_errors class="Waive" comment="Waive these">
          <select_layer_names pattern="metal1*"/>
          <select_errors field="Distance" min="1.0" max="2.0"/>
        </classify_errors>

        <!-- Waive density errors based on Area -->
        <classify_errors class="Waive" comment="Accepted density values">
          <select_violation_comment pattern="DN*"/>
          <select_errors field="Value" min="0.0" max="0.5" \
           when_field="Report" is="Area"/>
        </classify_errors>

        <!-- Waive PERC errors based on combination of device name \
         and comment -->
        <classify_errors class="Waive" comment="Waiving \
         M1 path errors" field_selection_mode="UNION">
```

Feedback

```
            <select_violation_comment pattern="Rule_1*"/>
            <select_errors field="Device name (Type)" pattern="*/M1 *"/>
            <select_errors field="Comment" pattern="Gate has \
             no path to power, ground, or top-level port"/>
          </classify_errors>

          <!-- Waive PERC errors with certain instance names -->
          <classify_errors class="Waive" comment="Waiving specific \
           instances">
           <select_violation_comment pattern="Rule_2*"/>
           <!-- The default field_selection_mode is "UNION"-->
           <select_errors field="Reference Instance (Cell Name)" \
            pattern="XI123 *"/>
           <select_errors field="Reference Instance (Cell Name)"
            pattern="XI456 *"/>
          </classify_errors>

      </input_pydb>
    </pydb_operation>
  </pydb>
```

Exporting Errors to a Layout

The icv_pydb utility provides the capability to export errors from one or more error databases (PYDBs) to a GDSII or OASIS layout file. All selection options that the icv_pydb utility provides are supported. The design hierarchy must be read from the original input layout, which was used for the IC Validator run. Layer and datatype numbers might be specified by a violation mapping file, or they are assigned arbitrarily.

The violation mapping file is a CSV file with the following columns:

- Violation comment

  Required. The full text of the violation or rule comment. Wildcard matching is supported.

- Command

  Optional. The runset file:line:function of a specific IC Validator command within the given rule. Wildcard matching is supported. If this column is empty, all commands within the violation or rule is matched.

- Layer

  Required. The layer number that is used to output matching violations.

- Datatype

  Required. The datatype number that is used to output matching violations.

Feedback

If a violation map file is given, only the rules and commands listed in the violation mapping file are exported. To include rules and commands without a listing in the violation map file, use the `-include_unmapped` command-line option.

If no input layout is given, the icv_pydb utility attempts to use the original layout path stored with the PYDB to read the design hierarchy.

Example 46 Example of Exporting to OASIS

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL \
      -o errors.oas -format OASIS -layout AD4FUL.GDS -c AD4FUL \
      -violation_map violation_map.csv
```

Example 47 Example With Selections and Creating a Reference Violation Mapping File With the Layers and Datatypes Used

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL \
      -svc "Met1*" -select_cell "AD4FUL" \
      -o errors.oas -format OASIS \
      -violation_map_output violation_map_used.csv
```

Example 48 Example of Creating a Violation Map File With Arbitrarily Assigned Layer and Datatype Numbers From a PYDB

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL \
      -violation_map_output violation_map.csv
```

Managing Referenced cPYDB Paths

When an error classification database (cPYDB) is used to apply error classifications during an IC Validator run, the error database (PYDB) contains a link to the cPYDB. Some details such as classification comments and timestamps are not stored in the PYDB but are read from the referenced cPYDB by tools such as VUE when needed. If the cPYDB is not in the original location, these details might be missing. The icv_pydb utility provides the capability to edit the path to referenced cPYDBs so that VUE can load those classification details.

Example 49 Example of Listing the cPYDBs Referenced by a PYDB

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL -list_cpydbs

...

cPYDBs referenced by run_details/pydb/PYDB_AD4FUL:
  Path: /global/cpydb1
  Name: CPYDB1

  Path: /global/cpydb1
  Name: CPYDB1A
```

Feedback

```
  Path: /global/cpydb2
  Name: CPYDB2
```

Example 50 Example of Updating the cPYDB Paths Using the -rename_cpydb_paths Command-Line Option

```
icv_pydb -i run_details/pydb/PYDB_AD4FUL
  -rename_cpydb_paths "*cpydb1" "/new/path/cpydb1" \
  -rename_cpydb_paths "*cpydb2" "/new_path/cpydb2"

...

Renamed cPYDBs:
  Old Path: /global/cpydb1
  New Path: /new/path/cpydb1
  Name: CPYDB1

  Old Path: /global/cpydb1
  New Path: /new/path/cpydb1
  Name: CPYDB1A

  Old Path: /global/cpydb2
  New Path: /new/path/cpydb2
  Name: CPYDB2
```

17

Pattern Matching

This chapter provides an introduction to IC Validator pattern matching. The pattern matching functions are described in the IC Validator Reference Manual.

The IC Validator pattern matching feature is described in the following sections:

- Overview
- Using the Pattern Matching Flow
- Creating the Pattern Library
- Viewing and Locking a Pattern Library
- Checking a Layout
- Detecting and Repairing Hotspots Within the IC Compiler Tool

Overview

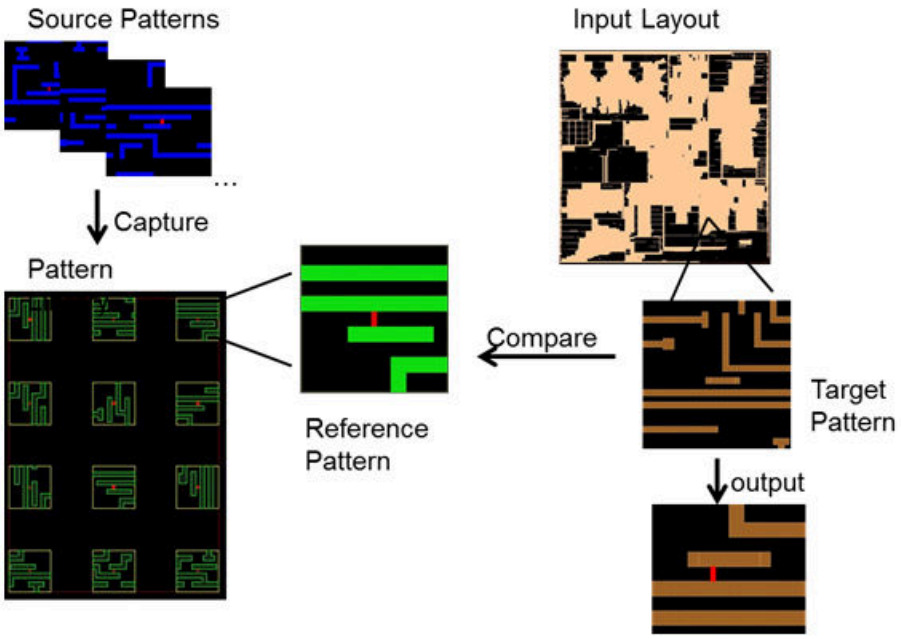
IC Validator pattern matching compares two geometrical configurations (patterns) to determine if they match. In pattern matching process, you match a target pattern from a design layout with a reference pattern from an existing pattern library to determine if the patterns match. Figure 105 illustrates this step.

The reference pattern represents any combination of geometric shapes that cannot be manufactured, such as a lithography hotspot, or are difficult to be interpreted accurately with standard DRC rules, such as a complex two-dimensional geometric measurements. Reference patterns are pre-identified through robust verification and validation techniques, and they are precisely specified and added to a pattern library before matching.

An advantage of pattern matching as a DRC-based flow is that it is much faster than the simulation-based lithography detection, which is extremely computationally expensive and time consuming. Therefore, pattern matching offers a cost-effective approach to identify lithography-induced or process-induced hotspots earlier in the design flow, and it makes real-time verification and repairing possible.

Feedback

Figure 105 Pattern Matching Flow

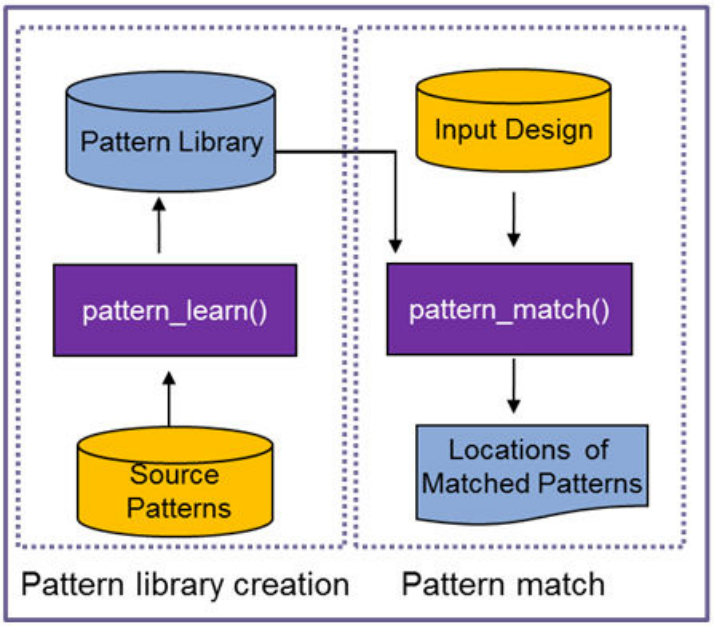


Using the Pattern Matching Flow

To use the patching matching flow, you provide a set of source patterns in the form of design pattern layers and location marker layers. These layers are identified either at a manufacturing site known as yield detractors or at a fabless company for methodology checking. Figure 106 shows a top-level IC Validator pattern-matching flow for pattern library creation and pattern matching.

Feedback

Figure 106 Pattern Matching Flowchart



During pattern library creation, pattern information is extracted from the source pattern and written to a pattern library. The pattern library is called during the pattern matching step as a reference for identifying the same or similar patterns in the input design layout.

The `pattern_learn()` function creates a pattern library. The `pattern match()` function performs pattern matching. See Chapter 1, “Tables of Runset Functions” in the *IC Validator Reference Manual* for a summary of all of the pattern matching functions.

Creating the Pattern Library

This section shows an example of the pattern library creation flow using the `pattern_learn()` function.

Preparing the Input Layout

Source patterns in the GDSII or OASIS formats are captured and registered to a pattern library through a pattern definition process, which contains specifications to precisely describe a pattern.

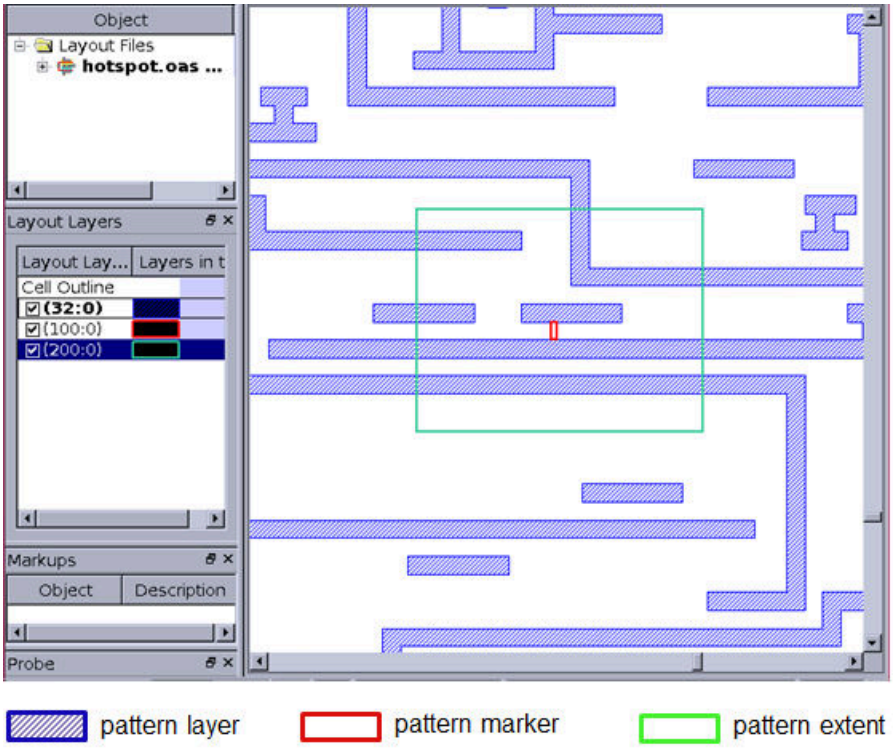
Feedback

Required Input

Three user-defined inputs are required for creating a pattern library. These inputs are part of the input layout, or they are generated through runset operations before the `pattern_learn()` function call. Figure 107 shows an example of required input.

- Pattern layers. Specifies that a pattern contains single or multiple pattern layers, such as a pattern consisting of metal and via layers.
- Pattern marker. Specifies a rectangular polygon that is placed on each source pattern to mark the location of interest, such as the pinch or bridge location, or a DRC violation. The pattern marker is output during pattern matching to report the match between the pattern library and the design.
- Pattern extent. Specifies the bounding region of a pattern. Pattern layers in this region are processed and registered to the pattern library. The pattern extent can also be generated through the `ambit_mode` and `match_ambit` arguments of the `pattern_learn()` function.

Figure 107 Required Input for Creating a Pattern Library



Feedback

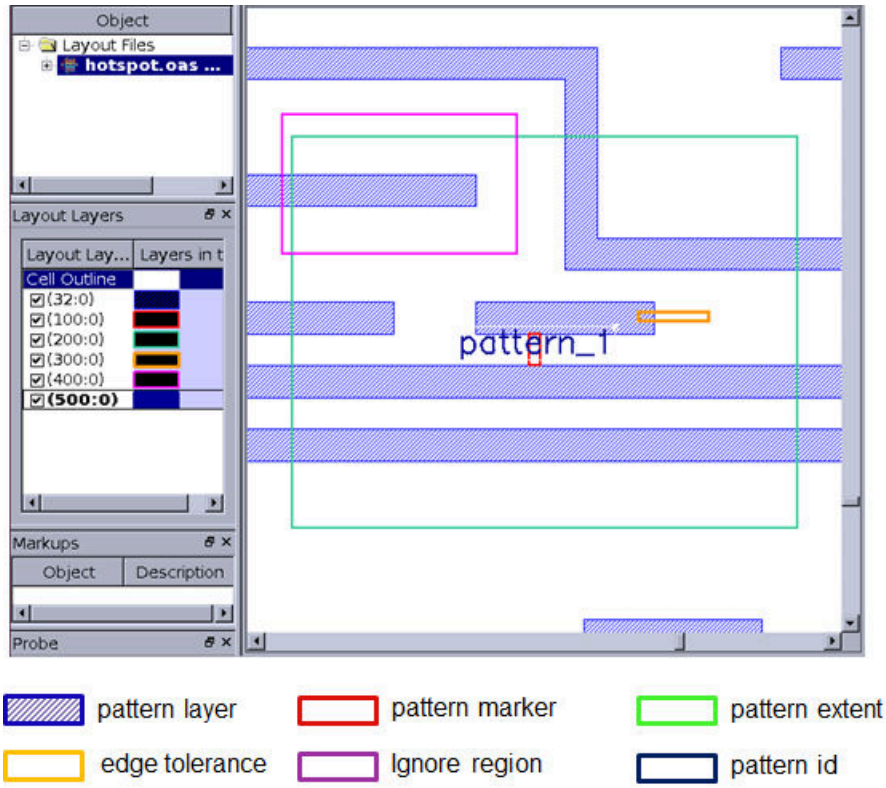
Optional Input

You might also need to provide the following specifications for your pattern definition. Figure 108 shows an example of optional input.

- Edge tolerance layers. Specifies a set of polygon layers that define the allowed edge placement variation for each pattern layer edge.
- Ignore region layers. Contains one or more rectilinear polygons defining areas on the source pattern that are excluded from the matching process.
- Optional pattern marker layers. Contains polygons that can be attached to a pattern and retrieved for certain post-pattern matching operations. For example, it can be user-defined fix guidance for a particular pattern.
- Pattern text ID layer. Contains the text ID of the source patterns. If the text ID layer is not provided, a tool-generated text ID is used.
- Pattern text property layers. Contains the text information defined for a source pattern, for example, the pattern ID, pattern type, or minimum CD. You attach these properties to the pattern during pattern library creation and retrieve them by using the `select_marker_by_double_property()` and `select_marker_by_string_property()` functions for post-pattern matching operations.

Feedback

Figure 108 Optional Input for Creating a Pattern Library



Additional Pattern Specifications

You might need to set additional pattern specifications during pattern library creation, such as:

- Pattern fuzziness. Specifies the pattern matching mode. You can choose the exact or fuzzy matching mode. The pattern fuzziness is edge-based and is specified individually and globally.
- Pattern reflection and rotation. Specifies whether to report a pattern as a match when it has a different orientation from the original shape.
- Pattern type. Specifies whether the source patterns are one-dimensional or two-dimensional patterns. Mixed pattern types are not supported.

See the *IC Validator Reference Manual* for more information about the `pattern_learn()` function.

Feedback

Example of Creating a Pattern Library

You create a standard DRC runset file to create a pattern library. The runset must include the following sections:

- Layout database specifics
- General options
- Pattern library path definition using the `pattern_options()` function
- Layer assignments
- Pattern library creation using the `pattern_learn()` function

A pattern matching runset that is used to create a pattern library is described in the following steps:

1. Beginning of the runset

   ```
   #include <icv.rh>
   ```

2. User-defined environment variables

   Environment variables in the runset are optional and can be customized.

   Two environment variables are defined for pattern library creation. The PATTERN_LIB_PATH environment variable specifies the location where the pattern library is saved. The PM_LIB_NAME environment variable specifies the name of the pattern library. For example,

   ```
   #pragma envvar default PM_LIB_PATH "/pattern_library"
   #pragma envvar default PM_LIB_NAME "MET"
   ```

3. Layout database specifications

   The `library()` function defines the design library, and it is required in the runset.

   ```
   library (
           format       = GDSII,
           library_name = "TOP.gds",
           cell         = "TOP"
   );
   ```

   Use the IC Validator command-line options to override arguments specified in the `library()` function. For example, use the `-i` command-line option to override the library name specified by `library_name` or the `-f` command-line option to override the library format specified by `format`. See the IC Validator Command-Line Options section in the “IC Validator Basics” chapter of the *IC Validator User Guide* for more information.

4. General Options

Feedback

Set `error_limit_per_check` to `ERROR_LIMIT_MAX` in the `error_options()` function to store and report all of the violations all of the errors up to a tool-defined maximum limit reported by the `pattern_learn()` and `pattern_match()` functions. The default is `100`.

```
error_options (
      error_limit_per_check = ERROR_LIMIT_MAX
      );
```

Setting `error_limit_per_check` to `ERROR_LIMIT_MAX` or other very high values can result in decreased performance and increased disk usage for very large numbers of errors, and is therefore not recommended.

5. Call the `pattern_options()` function to define the pattern library path.

   The `pattern_options()` function defines the path of a pattern library for the `pattern_learn()` and `pattern_match()` functions. The `pattern_options()` function can be called only one time in the runset. See the Runset Functions chapters of *IC Validator Reference Manual* for more information.

   ```
   pattern_options (
         pattern_library_path = $PM_LIB_PATH
         );
   ```

   Note:

   The pattern library path specified by the `pattern_library_path` argument must exist before pattern library creation. Otherwise a runtime error occurs.

6. Layer assignments

   ```
   metal : polygon_layer = assign ( {{3,0}} );
   pattern_marker : polygon_layer = assign ( {{4,0}, {5,0}} );
   pattern_text_id : text_layer = assign_text( {{6,0}} );
   error_list : list of write_error_map_s = {};
   ```

7. Call the `pattern_learn()` function to create a pattern library.

   - Define parameters for pattern library creation.

     This optional section of the runset provides access to a list of values for any parameter of the `pattern_learn()` function. See the descriptions of the `pattern_learn()` function in *IC Validator Reference Manual* for more information about the values for parameters in the `pattern_learn()` function.

     ```
     match_ambit : ambit_values_s = {0.2,0.2,0.2,0.2};
     ambit_mode : pattern_ambit_mode_e = PM_MARKER_CENTER;
     pattern_fuzziness : pattern_fuzzy_e = PM_EDGE_UNIFORM;
     uniform_fuzzy_size : double = 0.004;
     pattern_reflect : boolean = true;
     pattern_rotate : boolean = true;
     ```

Feedback

- Pattern library creation

  Call the `pattern_learn()` function to create a pattern library. You can call this function multiple times in a runset. The following example of calling the `pattern_learn()` function to create a single pattern layer library with edge uniform matching mode. The pattern extent is 0.4 x 0.4 µm and is created by the tool from the center of the pattern marker with 0.2 µm `match_abmit`.

```
tmp_violation @= {
    @"pattern_learn result";
    pattern_learn (
        pattern_library_name = $PM_LIB_NAME,
        pattern_layers = {metal},
        pattern_marker = pattern_marker,
        pattern_text_id = pattern_text_id,
        match_ambit = match_ambit,
        ambit_mode = ambit_mode,
        pattern_fuzziness = pattern_fuzziness,
        uniform_fuzzy_size = uniform_fuzzy_size,
        pattern_reflect = pattern_reflect,
        pattern_rotate = pattern_rotate
        );
}
```

  A pattern library directory with a name defined by the PM_LIB_NAME environment variable is generated and stored in the path defined by the PM_LIB_PATH environment variable.

  The error output from the `pattern_learn()` function is stored as the pattern_learn result. See “Examining the Output of Pattern Library Creation” for more information about the error output from the `pattern_learn()` function.

8. Output layout database

   In this optional section, you write the output from the `pattern_learn()` function to either a GDSII or OASIS file.

```
//DEBUG output only
error_list.push_back({tmp_violation,{0,0}});
output_lib=gds_library("out.gds");
write_gds(
        output_library=output_lib,
        errors=error_list
        );
```

Examining the Output of Pattern Library Creation

The output files produced by the IC Validator runset for pattern library creation include the pattern libraries and the same set of files as you use to perform DRC. See the Output Files chapter of the *IC Validator User Guide* for more details.

Feedback

- Error Output

  The IC Validator tool creates the *cell*.LAYOUT_ERRORS file when the DRC run completes. For pattern library creation, this file is used as a quick summary of the pattern creation statistics rather than a violation report.

  The following TOP.LAYOUT_ERRORS file reports an ERROR result:

```
                    LAYOUT ERRORS RESULTS: ERRORS

                    ##### ####  ####   ###  ####   ###
                    #     #   # #   # #   # #   # #
                    ####  ####  ####  #   # ####   ###
                    #     #  #  #  #  #   # #  #      #
                    ##### #   # #   #  ###  #   # ####
====================================================================
Library name:     ../../testcase/hotspot_clips.oas
Structure name:   TOP
Generated by:     IC Validator RHEL64 Release I-2013.12.SP1.4010 2014/
01/09
                        ERROR SUMMARY
Runset name: icv runset.rs
User name: juser
Time started: 2016/03/02 07:35:12AM
Time ended: 2016/03/02 07:35:51AM

Called as: icv runset.rs
pattern_learn result
   pattern_learn ............................... 6 violations found.

pattern_learn result/Invalid input: first pattern
layer has no corner within the pattern extent
   pattern_learn ............................... 1 violation found.

pattern_learn result:pattern_1
   pattern_learn ............................... 4 violations found.

pattern_learn result:pattern_5
    pattern_learn ............................... 2 violations found.
```

  In this report, six source patterns are processed by the `pattern_learn()` function. Two of the six patterns are identified as pattern_5 and four are identified as pattern_1. Therefore, two unique patterns are captured in the pattern library.

  The `pattern_learn()` function also identifies one pattern as an invalid two-dimensional pattern because it has no corner within the pattern extent. Therefore, it cannot be added to a two-dimensional pattern library. Check the *cell*.LAYOUT_ERRORS file after pattern library creation to determine if there are invalid patterns.

- Pattern Library

Feedback

You can store more than one pattern library under the same pattern library path defined by the `pattern_library_path` argument of the `pattern_options()` function. You call one or multiple pattern libraries during pattern matching.

The `pattern_learn()` function writes pattern information to the binary pattern.dat file, and it outputs a log file in the same directory defined by `pattern_library_path` argument.

Example 51 shows the log file generated for the pattern library, MET.

Example 51 Log File

```
[Tue Nov  5 15:19:22 2013]: Created pattern library.
version= 1.0
dbu=0.000500
learn_version=3
num_pattern_layers=1
num_optional_marker_layers=0
ambit_mode=PM_MARKER_CENTER
match_ambit.left=0.200000
match_ambit.bottom=0.200000
match_ambit.right=0.200000
match_ambit.top=0.200000
pattern_fuzziness=PM_EDGE_UNIFORM
pattern_grouping=PM_NONE
uniform_fuzzy_size=0.005000
edge_jog_size=0.000000
has_user_anchor=false
has_pattern_extent_layer=false
ignore_extra_polygons=false
pattern_type=PM_TWO_DIMENSIONAL
pattern_length=PM_GE
viewable=true
writable=true

[Tue Nov  5 15:19:22 2013]: Learned new pattern(s).
Registered 2 new unique pattern(s); total # of unique pattern(s)=2.
Created reflected/rotated pattern(s); total # of pattern(s)=16.
Derived 16 index pattern(s) using ambit = (0.195000, 0.195000,
  0.195000, 0.195000).
viewable=true; writable=true.
```

At the beginning of the log file, you see the parameters and their corresponding values applied for generating this pattern library for the first time. To add new patterns to an existing pattern library, use the same set of parameters and values. Otherwise an error occurs.

Feedback

At the end of the log file, you see the number of unique patterns that are registered for each `pattern_learn()` function call.

The last line of the log file indicates that this pattern library is writable, that is, the new patterns can be added to it. This pattern library can be converted to either the GDSII or OASIS format for viewing, if it is viewable.

Viewing and Locking a Pattern Library

You store the pattern library in a binary format. To view its content, convert it to a GDSII or OASIS file.

The `pattern_libary_read()` function is required for conversion. See the Runset Functions chapter in *IC Validator Reference Manual* for an example of how to convert a pattern library to a GDSII file.

By default, a pattern library is viewable and writable. The `pattern_library_lock()` function adds read and write protection to a pattern library that, after locked, is neither accessible by the `pattern_learn()` function for adding new patterns, nor by the `pattern_library_read()` function for converting it to a graphical format file. The following line, `viewable=false`; `writable=false`, is appended to the existing log file of the locked pattern library.

The locked pattern library is then delivered to the end users. See Chapter 3, “Runset Functions: J - Z,” in *IC Validator Reference Manual* for an example of how to lock a pattern library.

Alternatively, you can use the pdb_utility.pl script, which is in the IC Validator contrib directory, to perform the functionality of both the `pattern_library_read()` and `pattern_library_lock()` functions without executing a runset:

```
Usage: pdb_utility.pl <pdb_lib_path> <pdb_lib_name> -read/-lock
 [output_name]

Options:
<pdb_path>.............pattern library path
<pdb_name>.............pattern library name
-read..................converting a pattern library to a graphical
 output.
-lock..................adds the write and view protection to a pattern
 library.
<output_name>.........optional, specifies the graphical output file
 name, [default:pdb_view.gds]
```

This example converts the met pattern library, which is located in the /test/pattern_lib directory, to the hs.gds output GDSII file.

```
ICV_INSTALL_DIRECTORY/contrib/pdb_utility.pl /test/pattern_lib met -read
 hs.gds
```

Feedback

Checking a Layout

This section shows an example of the pattern matching flow using the `pattern_match()` function.

Runset Example of Pattern Matching

You create a standard DRC runset file to run the IC Validator tool to perform pattern matching on a design layout. The runset file must include the following sections:

- Layout database specifics
- General options
- Pattern library path definition using the `pattern_options()` function
- Layer assignments
- Performs pattern matching with the `pattern_match()` function

A runset that uses the pattern matching flow for has these components:

1. Beginning of the runset

   Include icv.rh at the beginning of the runset.

   ```
   #include <icv.rh>
   ```

2. User-defined environment variables

   Environment variables in the runset are optional and can be customized.

   ```
   #pragma envvar default PM_LIB_PATH "/pattern_library"
   #pragma envvar default PM_LIB_NAME "MET"
   #pragma envvar default PM_TOP_MET "10"
   ```

3. Layout database specifics

   The `library()` function defines the design library used in the pattern matching check.

   ```
   library (
           format       = GDSII,
           library_name = "TOP.gds",
           cell         = "TOP"
   );
   ```

   If you use a Milkyway library, set `format` to `MILKYWAY` and specify the `library_path` to the parent directory of the Milkyway library.

4. General options

Feedback

To store and report all of the violations generated by `pattern_match()` function up to a tool-defined maximum limit, set `error_limit_per_check` to `ERROR_LIMIT_MAX` in the `error_options()` function. The default is `100`.

```
error_options (
        error_limit_per_check = ERROR_LIMIT_MAX
       ;
```

Setting `error_limit_per_check` to `ERROR_LIMIT_MAX` or other very high values can result in decreased performance and increased disk usage for very large numbers of errors, and is therefore not recommended.

5. Call the `pattern_options()` function to define the pattern library path.

```
pattern_options (
   pattern_library_path = $PM_LIB_PATH
);
```

The `pattern_options()` function defines the path of a pattern library, and it can be called only one time in the runset. This function specifies the path you need to access a single or multiple pattern libraries.

6. Layer assignments

```
MET1 : polygon_layer = assign ( {{15,0}} );
MET2 : polygon_layer = assign ( {{17,0}} );
MET3 : polygon_layer = assign ( {{19,0}} );
MET4 : polygon_layer = assign ( {{21,0}} );
MET5 : polygon_layer = assign ( {{23,0}} );
MET6 : polygon_layer = assign ( {{25,0}} );
MET7 : polygon_layer = assign ( {{27,0}} );
MET8 : polygon_layer = assign ( {{29,0}} );
MET9 : polygon_layer = assign ( {{31,0}} );
MET10 : polygon_layer = assign ( {{33,0}} );

metal_layer_list : list of polygon_layer = {MET1, MET2, MET3, MET4,
 MET5, MET6, MET7, MET8, MET9, MET10};
metal_error_list : list of write_error_map_s = {};
```

In this example, MET1 through MET10 are assigned and subjected to a pattern matching check.

7. Call the `pattern_match()` function to perform pattern matching.

You must call the `pattern_match()` function to perform pattern matching. This function can be called multiple times within a runset. The following example shows the `pattern_match()` function executing pattern matching on the metal layers of the TOP.gds file.

```
for(i = 0 to strtoi($PM_TOP_MET)-1) {
    layer_index : integer = i + 1;
```

Feedback

```
        tmp_violation @= {
            @"METAL"+layer_index+" PM RESULT: ";
            pattern_match (
                pattern_library_name = $PM_LIB_NAME,
                pattern_layers = {metal_layer_list[i]}
                  ;
}
            out_layer_index = layer_index;
            metal_error_list.push_back({tmp_violation,
{out_layer_index,0}});
       }
```

8. Output layout database

   Optionally, you can write the output from the `pattern_match()` function to a GDSII or OASIS file.

```
output_lib=gds_library("out.gds");
write_gds(
        output_library=output_lib,
        errors=metal_error_list
        );
```

Examining Pattern Matching Output

The IC Validator tool creates the *cell*.LAYOUT_ERRORS error file when the DRC run completes. The beginning of the file states that the run is either CLEAN or has ERRORS. In pattern matching, the ERRORS result means that one or more matched patterns are identified. The CLEAN result means that no matched patterns are found.

The following TOP.LAYOUT_ERRORS file reports violations (matches):

```
                  LAYOUT ERRORS RESULTS: ERRORS
                  ##### ####  ####   ###  ####   ###
                  #     #   # #   # #   # #   # #
                  ####  ####  ####  #   # ####   ###
                  #     #  #  #  #  #   # #  #      #
                  ##### #   # #   #  ###  #   # ####
================================================================
Library name:     ./test.gds
Structure name:   TOP
Generated by:     IC Validator RHEL64 Release H-2013.06.SP2.4035 2013/08/
28
                         ERROR SUMMARY
METAL2 PM RESULT:
   pattern_match ..............................873 violations found.

METAL3 PM RESULT:
   pattern_match ..............................69 violations found.

METAL4 PM RESULT:
   pattern_match ..............................2 violations found.
```

Feedback

```
                          ERROR DETAILS
------------------------------------------------------------------
METAL2 PM RESULT:
------------------------------------------------------------------
/match.rs:38:pattern_match
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Structure (lower left x, y )  (upper right x, y )  Pattern   Orientation
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
TOP   (8.1600, 528.2500)   (8.4000, 528.7200)   pattern_78  R90
TOP   (2.1400, 528.6700)   (2.3800, 529.1400)   pattern_57  R90
TOP   (100.1450, 529.3000) (100.3850, 529.7700) pattern_107 R90FX
TOP   (100.1450, 529.2300) (100.3850, 529.7000) pattern_108 R90
TOP   (107.9800, 531.7500) (108.2200, 532.2200) pattern_80  R0
TOP   (83.7900, 534.7600)  (84.0300, 535.2300)  pattern_7   R180
TOP   (86.0000, 534.8300)  (86.2400, 535.3000)  pattern_94  R90FY
TOP   (87.6800, 540.2900)  (87.9200, 540.7600)  pattern_78  R90
……
```

The error summary section contains the number of matched patterns on each metal layer produced by the `pattern_match()` function. The error details section lists the coordinates of the pattern marker and pattern ID of each matched pattern.

You can review pattern matching results in VUE, just as you do for other DRC violations.

Post Pattern-Matching Operations

Use the following functions in the post pattern-matching process to select matched patterns based on their properties or retrieve optional pattern markers of certain patterns for additional DRC operations.

- `marker_merge()`. Merges the marker layer into the polygon layer. The marker layer does not merge; it can be processed only by several IC Validator runset functions. The marker layer must be converted to a polygon layer to be processed by other DRC functions.
- `optional_pattern_markers()`. Retrieves all optional pattern markers attached to a pattern. Optional pattern marker layers are polygon layers.
- `select_marker_by_string_property()`. Retrieves the pattern markers based on the specified string properties.
- `select_marker_by_double_property()`. Retrieves the pattern markers based on the specified double properties.
- `milkyway_route_directives()`. Passes user-defined fix guidance to the router for hotspot repair.
- `drc_features_marker()`. Outputs the marker layer.

See the *IC Validator Reference Manual* for examples of each function.

Feedback

Detecting and Repairing Hotspots Within the IC Compiler Tool

To meet the design-for-manufacturing requirements for advanced nodes, the IC Validator tool provides In-Design DFM technology in the implementation environment. This physical verification approach enables early detection and repairing of DRC violations, with negligible physical or timing impacts within the IC Compiler tool. For more information about the Automatic DRC Repair (ADR) flow, see the Synopsys white paper, “In-Design Physical Verification - Automatic DRC Repair (ADR).”

The ADR flow enables In-Design hotspot detection and correction using IC Validator pattern matching during design implementation. With a usage model similar to DRC, you can use a foundry-certified DFM package (containing pattern library and matching runset) or your own yield detractor pattern library to search a design for potential manufacturing related hotspots to perform automatic fixing and revalidation for all errors. This improves the overall manufacturability of the design.

You run IC Validator pattern matching within the IC Compiler tool using the same set of IC Compiler commands used to perform signoff DRC checking. You use the `signoff_drc` command to do pattern matching and the `signoff_autofix_drc` command to automatically fix the violations (hotspots) detected by the `signoff_drc` command. See the *IC Compiler Implementation User Guide*, which is available on SolvNetPlus, or the IC Compiler man pages for detailed command syntax and usage.

The following sections describe how to detect and correct hotspots using the signoff DRC commands in the IC Compiler tool (not applicable in the IC Compiler II tool).

Setting Up the Physical Signoff Options

Before you use the `signoff_drc` and `signoff_autofix_drc` commands to run pattern matching, set up the physical signoff options that apply to these commands.

For more information, see the “Performing Signoff Design Rule Checking” section in the *IC Compiler Implementation User Guide*.

Running Pattern Matching for Hotspot Detection

Use the `signoff_drc` command to perform hotspot detection on the routing layers.

For more information, see the “Performing Signoff Design Rule Checking” section in the *IC Compiler Implementation User Guide*.

Automatically Fixing Hotspot Patterns

Use the `signoff_drc` command to automatically fix detected hotspots.

For more information, see the “Performing Signoff Design Rule Checking” section in the *IC Compiler Implementation User Guide*.

Feedback

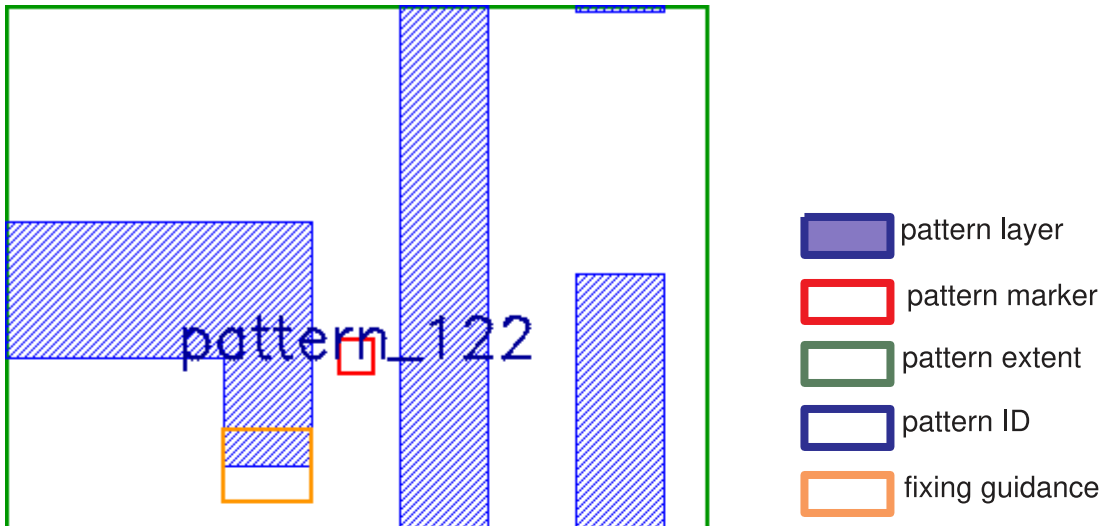
Repairing Hotspots With User-Defined Fix Guidance

In addition to the default ADR correction, the IC Validator tool also enables you to implement user-defined fix guidance. The steps for implementing fix guidance in the pattern matching flow are:

1. Specify fix guidance as the optional pattern markers during pattern library creation. A single pattern can have multiple guidance.

   The pattern in Figure 109 shows a single user-defined fix guidance that to add to a rectangle at the end of the L-shape polygon.

Figure 109 User-Defined Fix Guidance



Example 52 Attaching Fix Guidance During Pattern Library Creation

```
//Layer Assignment
metal : polygon_layer = assign ( {{3,0}} );
pattern_marker : polygon_layer = assign ( {{1,0}} );
pattern_extent : polygon_layer = assign ( {{2,0}} );
pattern_add : polygon_layer = assign ( {{4,0}} );
optional_markers: list of polygon_layer =
 {pattern_extent,pattern_marker,pattern_add};

//Create Pattern Library
pattern_learn (
```

Feedback

```
            pattern_library_name = "met",
            pattern_layers = {metal},
            pattern_marker = marker_layer,
            optional_pattern_markers = optional_markers,
            pattern_text_id = text_layer,
            pattern_extent = pattern_extent,
            pattern_fuzziness = PM_EXACT,
            pattern_reflect = true,
            pattern_rotate = true
            );
```

2. Retrieve and pass the fix guidance after pattern matching.

   You use the `optional_markers()` function to retrieve the fix guidance and the `milkyway_route_directives()` function to pass the fixing guidance to the router.

Example 53 Retrieving and Passing Fix Guidance

```
//Output hotspot marker layer
hotspot_marker = pattern_match (
            pattern_library_name = "met",
            pattern_layers = {metal_layer_list[i]});

//Retrieve all the optional pattern marker layers defined in step 1
optional_markers = optional_pattern_markers("met", hotspot_marker);

//Retrieve the add shape layer
pattern_add = optional_markers[2];

//Pass fixing guidance to IC Compiler
milkyway_route_directives(
                  display_marker = optional_markers[1],
                  group_marker = optional_markers[0],
                  error_layers = { {optional_markers[1]} },
                  route_directives = { {pattern_add, dummy_sub} } );
```

3. Run ADR within the IC Compiler tool for hotspot detection and fixing.

   Write an IC Compiler TCL script to execute ADR in the icc_shell, or use the IC Compiler GUI to execute the commands.

Example 54 IC Compiler TCL Script

```
open_mw_lib ./design
copy_mw_cel -from eco_route_routeopt -to pm_flow_add
open_mw_cel pm_flow_add

set_physical_signoff_options -exec icv -drc_runset {/runset/ \
match_add.rs}
signoff_drc -run_dir {./signoff_drc_run} -ignore_child_cell_errors

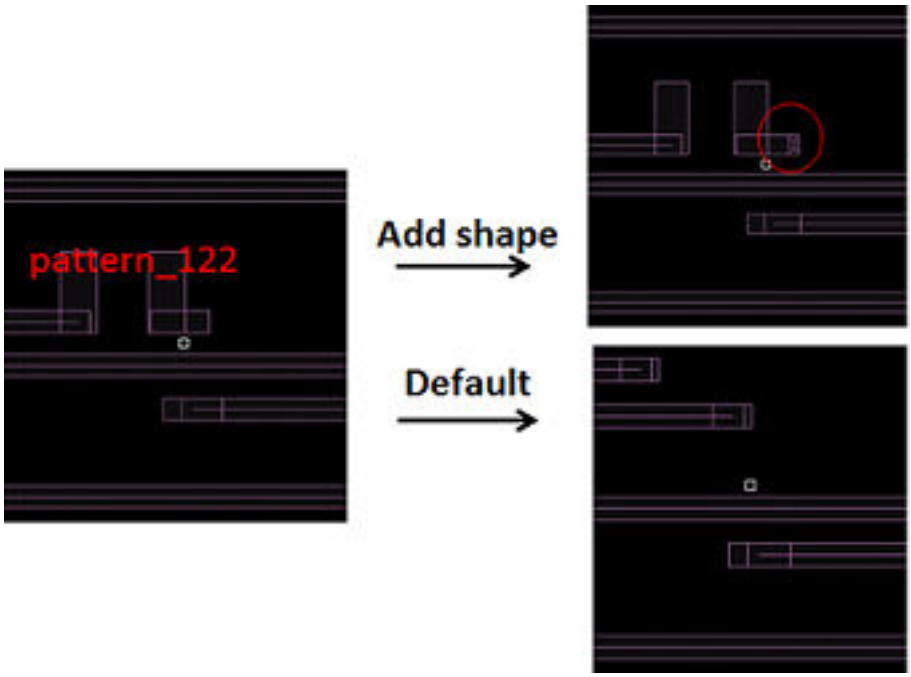
signoff_autofix_drc -run_dir {./signoff_autofix_drc_run} \
 -init_drc_error_db signoff_drc_run
```

Feedback

```
save_mw_cel -as pm_flow_add
close_mw_cel
```

The IC Compiler tool starts with user-defined fix guidance based on a defined order. In Figure 110, you see the corrected results of pattern_122 by the user-defined guidance and the default ADR flow, respectively.

Figure 110 Example of Hotspot Correction



18

Pattern Library Manager

This chapter provides an introduction to IC Validator Pattern Library Manager.

The IC Validator Pattern Library Manager is described in the following sections:

- Getting Started
- Creating a New Pattern Library
- Editing a Pattern
- Exporting and Importing a Pattern Library
- Running Pattern Matching in the Pattern Library Manager
- Restrictions and Limitations

Getting Started

The IC Validator Pattern Library Manager (PLM) is a graphical user interface (GUI) that provides an alternate way to build a pattern library for an IC Validator pattern matching run instead of using the batch mode for pattern library generation with the `pattern_learn()` function.

Supported Versions of IC Validator WorkBench

The Pattern Library Manager communicates with the IC Validator WorkBench tool through a socket or port connection.

IC Validator N-2017.12-SP2 and later versions support IC Validator WorkBench J-2014.06 and later versions.

Invoking the Pattern Library Manager

Ensure that the *`ICV_INSTALL_DIRECTORY`* installation directory is set properly.

Launch the Pattern Library Manager either at the shell command or in the IC Validator WorkBench layout editor.

1. The command-line syntax is

   ```
   icv_plm
   ```

   Note:

   If you launch the Pattern Library Manager and the IC Validator WorkBench tool separately from the command line, both of them must be in the same directory path (on the same machine) to establish a connection. For information about how to connect the Pattern Library Manager to IC Validator WorkBench, see the Communicating With IC Validator WorkBench section.

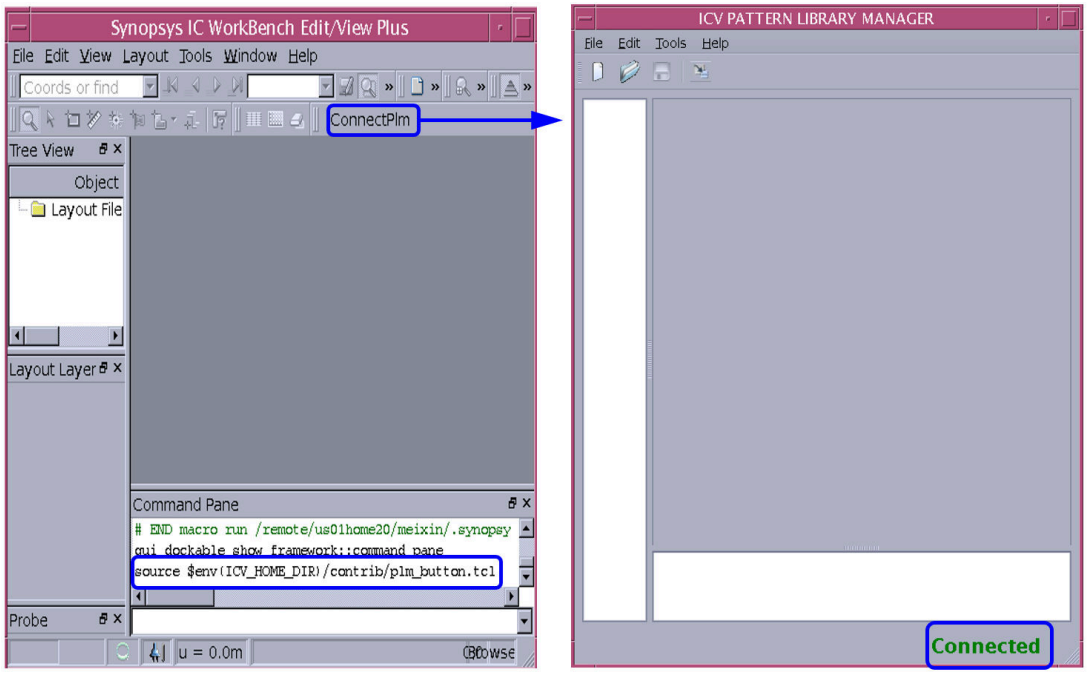
2. To access the Pattern Library Manager from IC Validator WorkBench, source the IC Validator plm_button.tcl file in the command window.

   ```
   % source $env(ICV_INSTALL_DIRECTORY)/contrib/plm_button.tcl
   ```

   The ConnectPlm button is added to the toolbar. Click this button to launch the Pattern Library Manager and establish a connection, as shown in Figure 111.

Feedback

Figure 111 ConnectPlm Button



Note:

You can also put this plm_button.tcl file in your IC Validator WorkBench startup file to avoid setting up the connection each time.

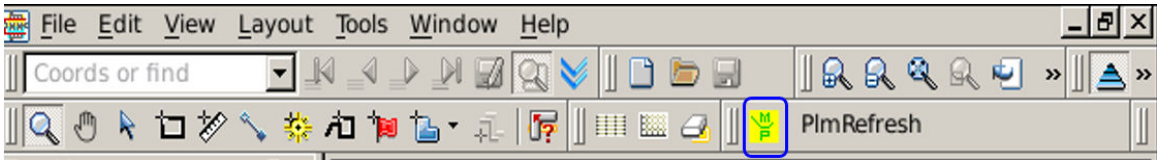
The IC Validator WorkBench tool connects to Pattern Library Manager automatically when you set the `ICVWB_USER` environment variable:

For M-2016.03-SP2 and newer versions of IC Validator WorkBench:

```
setenv ICVWB_USER ICV_PLM
```

A yellow button is added to the toolbar. Click this button to start the Pattern Library Manager and establish a connection.

Figure 112 Pattern Library Manager Toolbar



Feedback

Communicating With IC Validator WorkBench

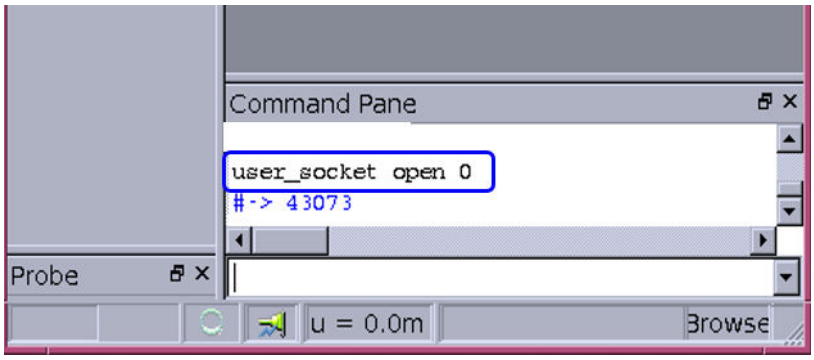
To establish a connection with IC Validator WorkBench after the Pattern Library Manager has already started,

1. Type the following line in the IC Validator WorkBench command window:

   ```
   user_socket open 0
   ```

   This command returns a number. This is the open socket that IC Validator WorkBench established.

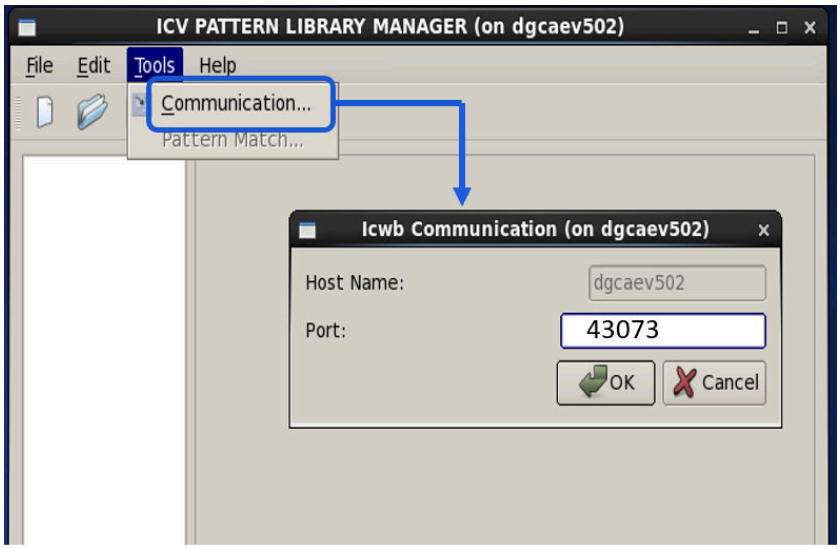
Figure 113 Open Socket Connection



2. In the Pattern Library Manager, choose Tools > Communication. Enter the port number and click OK.

Feedback

Figure 114 Communication Window



3. The Pattern Library Manager is now connected to IC Validator WorkBench.

Pattern Library Manager GUI

The main GUI is displayed in Table 69 shows the main function of the four pull-down menus of this GUI.

Table 69 Pull-Down Menu of the Pattern Library Manager

Menu	Description
File	Creates, opens, or saves a pattern library. Imports a Pattern Database format pattern library, and exports a Pattern Library Manager format pattern library
Edit	Creates, adds, or edits patterns in a pattern library
Tool	Establishes communication with IC Validator WorkBench and performs pattern matching in the Pattern Library Manager
Help	Displays information about the interface language and the user guide

Creating a New Pattern Library

The File menu allows you to create a new pattern library for adding new patterns. Table 70 summarizes the available options.

Feedback

Table 70 File Menu Commands

Option	Description
New	Creates a new pattern library
Open	Opens an existing pattern library
Save	Saves the current library
Export	Output a pattern library to Pattern Database format
Import	Reads in a pattern database format library to the Pattern Library Manager
Close	Closes the current library
Exit	Exits the Pattern Library Manager

Creating a New Pattern Library Name

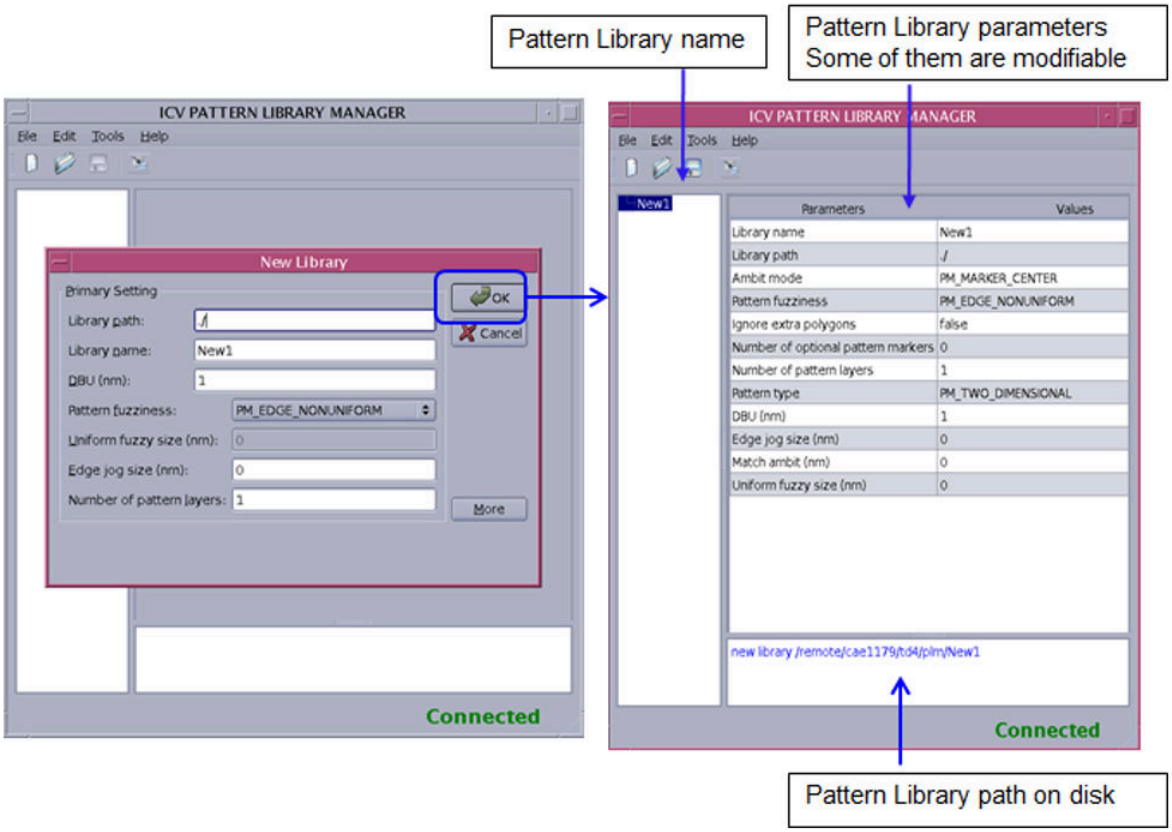
Before adding new patterns to a new pattern library, you must create a pattern library name.

1. Choose File > New. A dialog box appears. A new library name is created with the default name, New1.
2. Change the library name if necessary.
3. Specify the path for saving the pattern library.
4. Fill in the required pattern library attributes, such as pattern fuzziness, number of pattern layers, and uniform fuzzy size (for `PM_EDGE_UNIFORM` mode only).
5. Click More to open advanced settings, such as pattern type and number of optional pattern marker layers.
6. Click OK to create the new pattern library (name). A corresponding empty layout (named with the pattern library name) is created in the IC WorkBench EV Plus window with multiple layers. A summary table listed the defined pattern library parameters appears in Pattern Library Manager main window, as shown in Figure 115.

Feedback

7. You can now add patterns to the new pattern library. As you add new patterns, each of them appears under the pattern library you have created.

Figure 115 Creating a New Pattern Library



Adding New Patterns

You can add new patterns to a pattern library using three Edit menu commands, as shown in Table 71.

Table 71 Edit Menu Commands

Command	Description
New Pattern	Create a pattern layout and adds it as a new pattern
New Pattern By Clip	Generates a pattern through a clipping window on a layout
New Pattern By Layer	Extracts multiple patterns from the selected area of a layout
Copy	Copies a pattern from a pattern library

Feedback

Table 71 Edit Menu Commands (Continued)

Command	Description
Paste	Pastes the copied pattern to another pattern library
Rename	Renames a pattern
Delete	Deletes a pattern from a pattern library

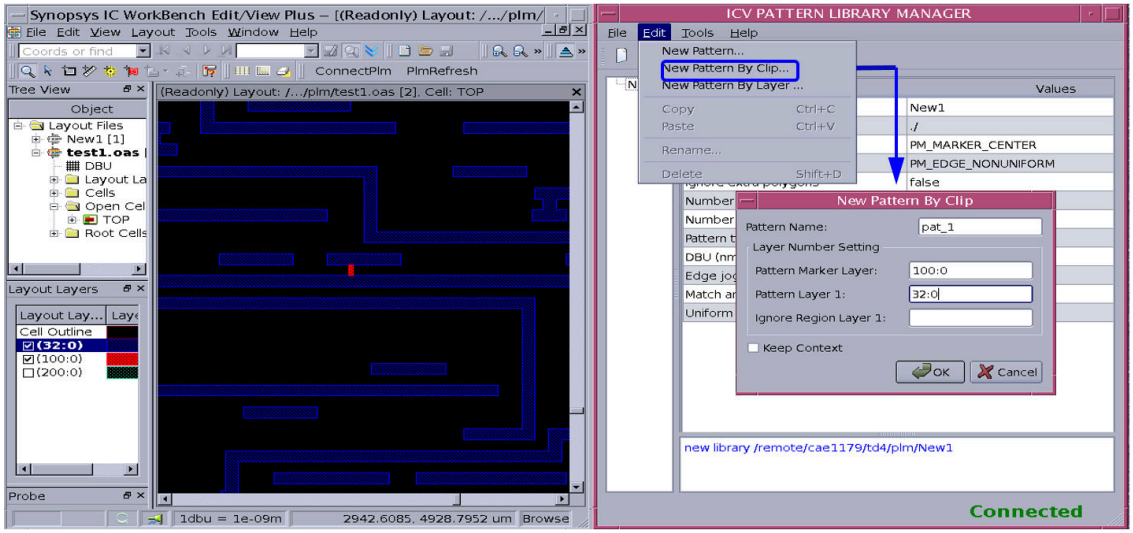
Capturing a New Pattern by Clipping

You can create a new pattern from a layout that has pattern layers and a pattern marker already defined. The pattern boundary is defined as the clipping window.

To capture a new pattern, start with a layout that contains polygons.

1. In the Pattern Library Manager, open an existing pattern library or create a new pattern library entry.
2. Open the layout in IC WorkBench EV Plus.
3. Browse to the location of the pattern, and zoom to an appropriate display to have all the polygons to be extracted shown on the display window.
4. As shown in Figure 116, choose Edit > New Pattern By Clip in the Pattern Library Manager. A dialog box appears. Enter a name for this pattern and specify the layer number and data type for the pattern marker layer and pattern layer. Click OK.

Figure 116 Capturing a New Pattern By Clipping



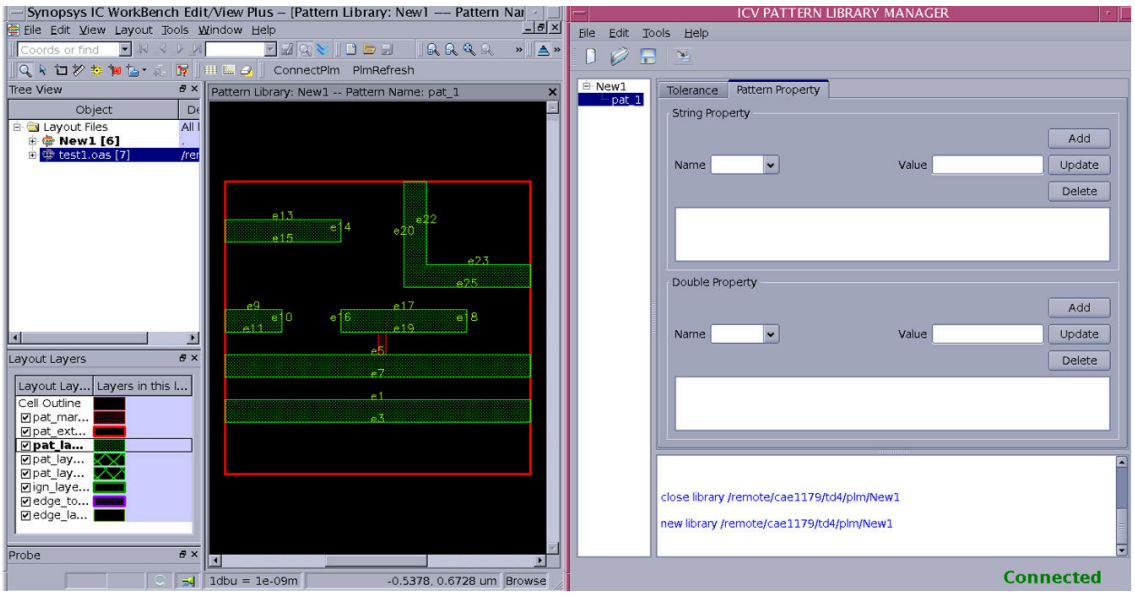
Feedback

Note:

By default, polygons outside of the pattern extent are not captured in the generated pattern. To maintain the surrounding area outside the pattern extent for further modification before the pattern is finalized, choose Keep Context before clicking OK.

5. Draw a clipping window in IC WorkBench EV Plus to define the pattern extent. A new layout window appears. Each polygon edge is labeled with a number starting from 0. Edge labels are created as text layers under the layer list. A new pattern is added to the pattern library, as shown in Figure 117.

Figure 117 Drawing a Clipping Window



Note:

The only allowed operation in IC WorkBench EV Plus after clicking OK in step 4 is to draw a clipping window to define the pattern extent.

6. Define pattern properties in the Pattern Property tab if needed. For more information, see the Attaching Pattern Properties to a Pattern section.

7. Add edge constraints in the Tolerance tab if needed. For more information, see the Adding an Edge Tolerance to a Pattern section.

8. Repeat the previous steps to generate additional new patterns if there are multiple locations in the same layout. Upon completion, choose File > Save in the Pattern Library Manager to save the changes to your pattern library file.

A pattern library with one or multiple patterns from the layout is generated.

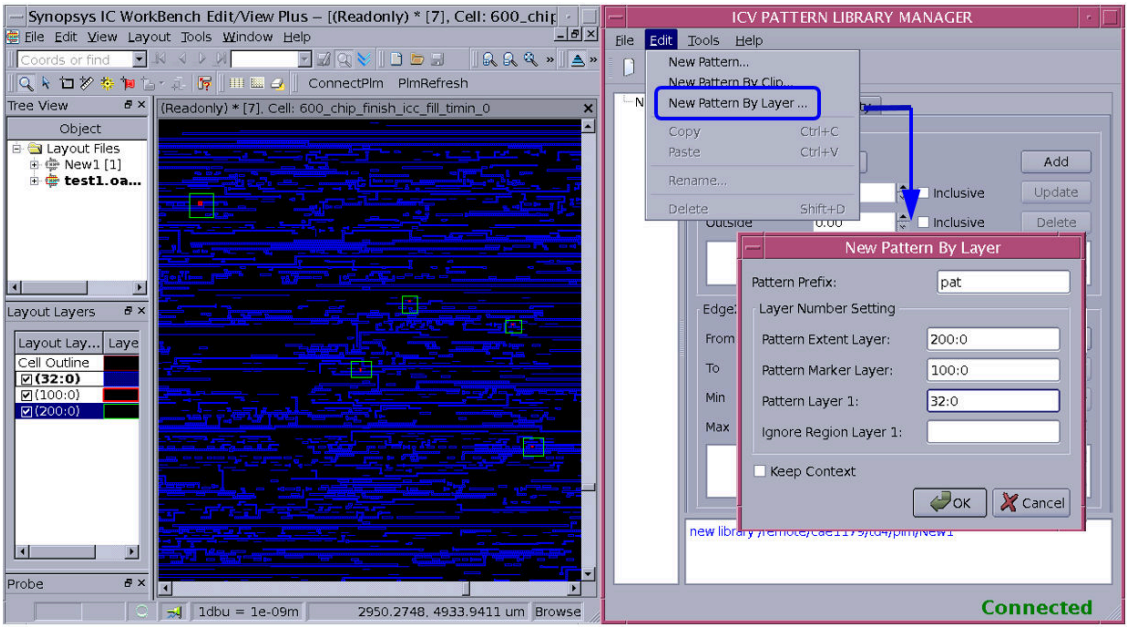
Feedback

Extracting Patterns in a Layout

You can create multiple patterns from a layout that has pattern layers, pattern marker, and the pattern extent layer already defined.

1. In the Pattern Library Manager, open an existing pattern library or create a new pattern library entry.
2. Open the layout in IC WorkBench EV Plus.
3. Browse to the location of the layout, and zoom to an appropriate display, to have the right part of the layout shown on the display window.
4. As shown in Figure 118, choose Edit > New Pattern By Layer in the Pattern Library Manager. A dialog box appears. Enter a name for the pattern prefix and specify the layer number and data type for the pattern extent, pattern marker, and pattern layer. The optional ignore region can also be specified if defined. Click OK.

Figure 118 Extracting Patterns in a Layout



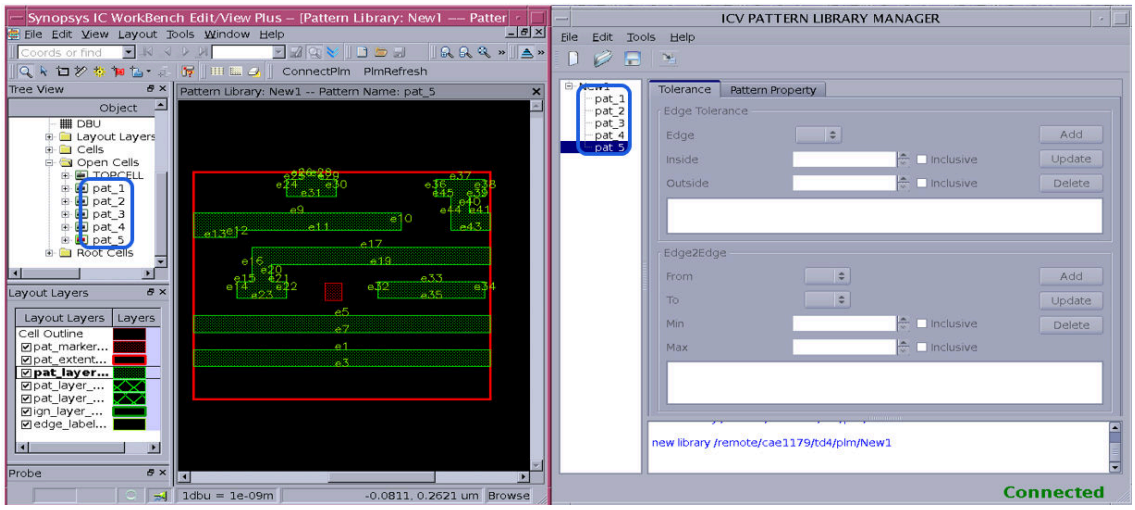
Note:

By default, polygons outside of the pattern extent are not included in the generated pattern. To keep the surrounding area outside of the pattern extent, choose Keep Context before clicking OK.

Feedback

5. Draw a clipping window in IC WorkBench EV Plus to select the part of the layout for generating new patterns. Patterns are extracted and listed under the currently open or selected pattern library, as shown in Figure 119.

Figure 119 Selecting Part of Layout for Generating New Patterns



Note:

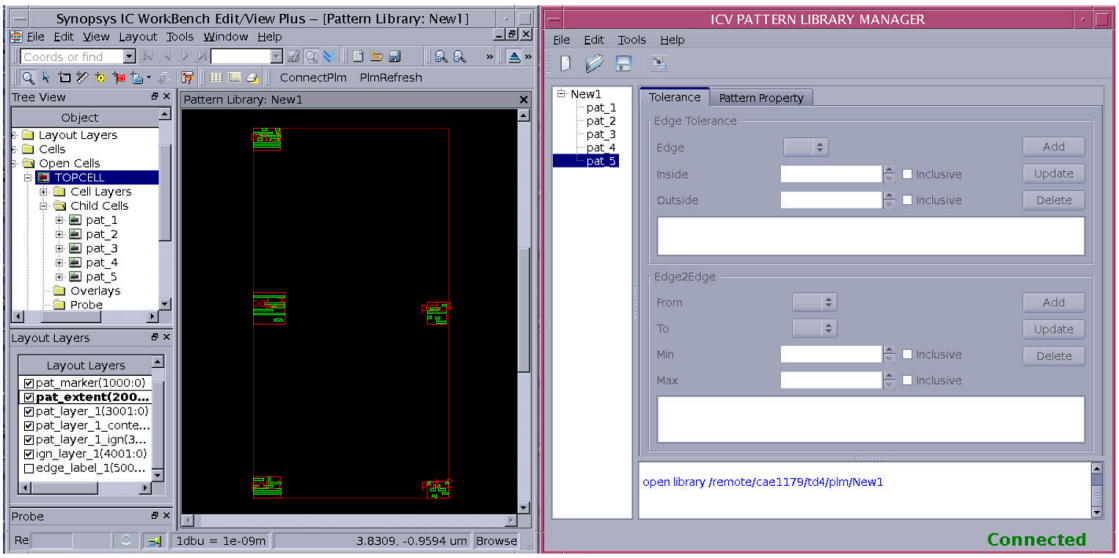
The only allowed operation in IC WorkBench EV Plus after clicking OK in step 4 is to draw a clipping window to define the area of the layout for pattern extraction.

6. Define pattern properties in the Pattern Property tab if needed. For more information, see the Attaching Pattern Properties to a Pattern section.
7. Add edge constraints in the Tolerance tab if needed. For complete information, see the Adding an Edge Tolerance to a Pattern section.
8. Upon completion, choose File > Save in the Pattern Library Manager to save the changes to your pattern library file.

A pattern library with multiple patterns from the layout is generated, as shown in Figure 120.

Feedback

Figure 120 Pattern Library With Multiple Patterns



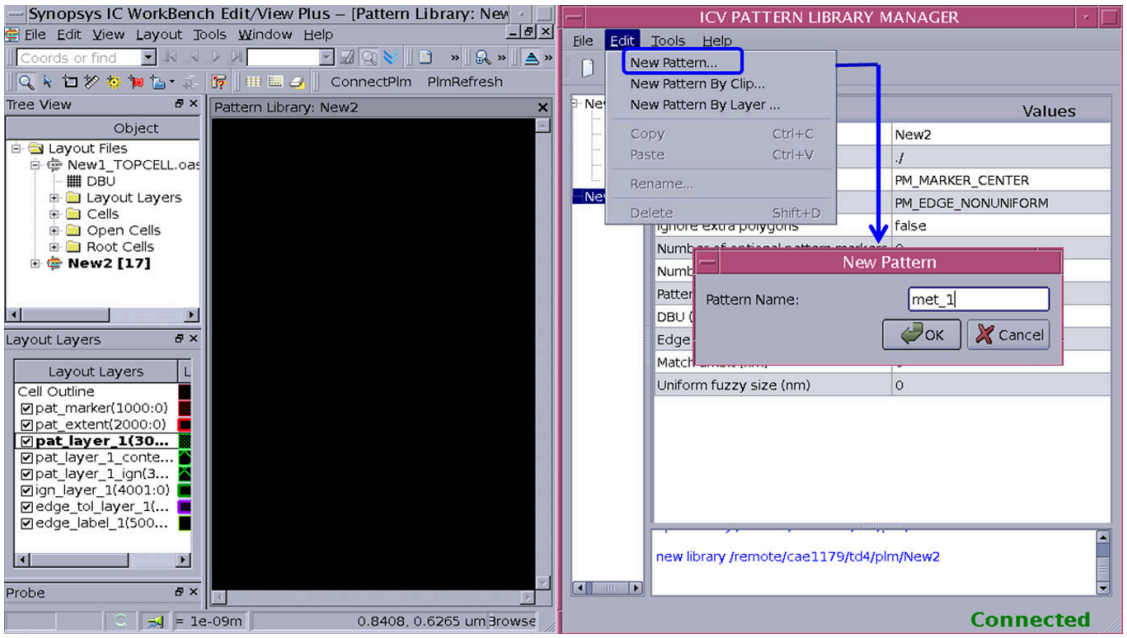
Generating a New Pattern From Scratch

You can draw a pattern layout in ICWBEV to create a new pattern when there is no existing layout for pattern extraction.

1. In the Pattern Library Manager, open an existing pattern library or create a new pattern library entry.
2. In the Pattern Library Manager, choose Edit > New Pattern. A dialog box appears. Enter a name for the pattern to be generated. Then, click OK, as shown in Figure 121.

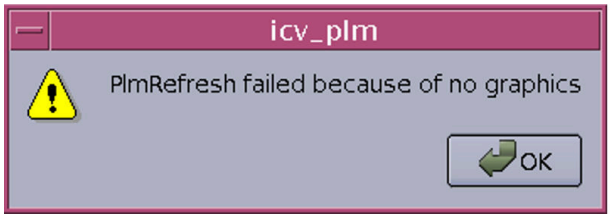
Feedback

Figure 121 Generating a New Pattern From Scratch



3. A warning message appears. This is because there are no graphics in IC WorkBench EV Plus associated with the newly created pattern name. Click OK to close this message.

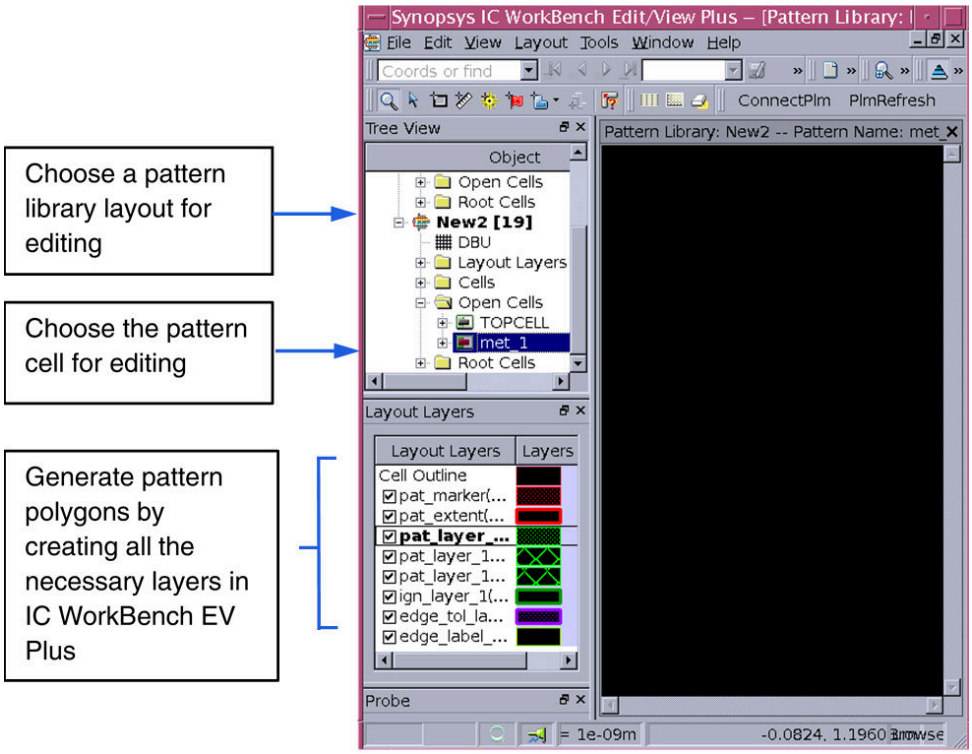
Figure 122 No Graphics Warning Message



4. Create the pattern layout in IC WorkBench EV Plus, as shown in Figure 123.

Feedback

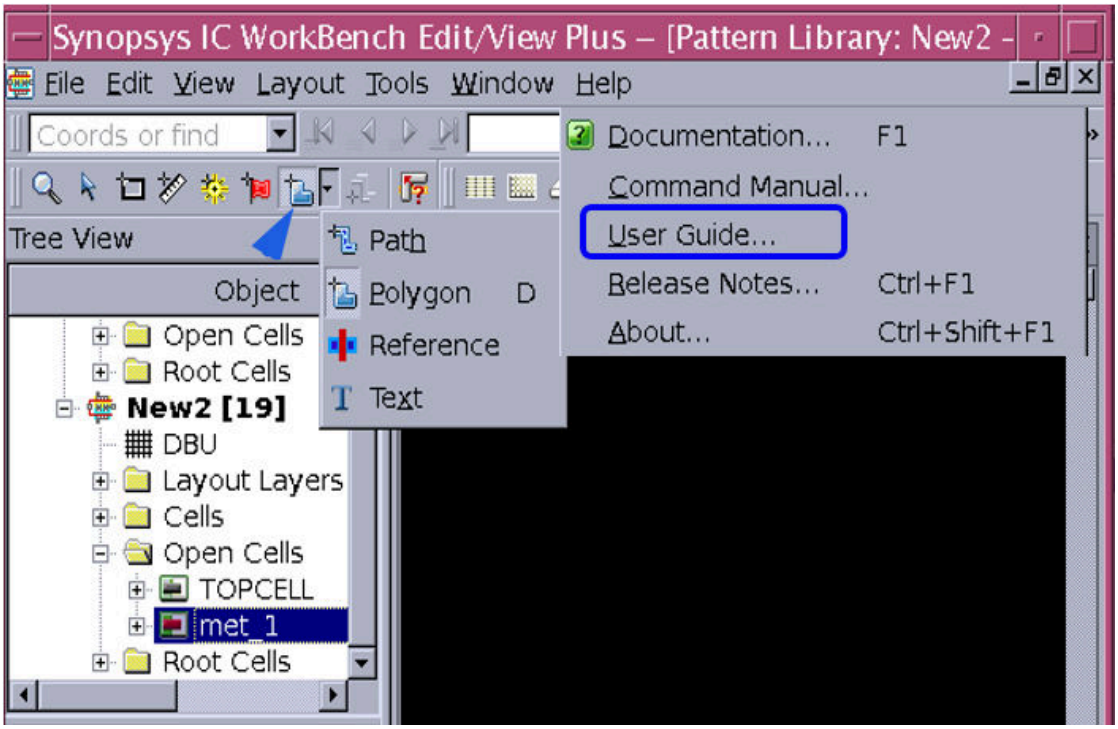
Figure 123 Creating the Pattern Layout



5. For more information about layout creation and editing, see the *IC WorkBench EV Plus User Guide*, which can be found on the Help menu. See Figure 124.

Feedback

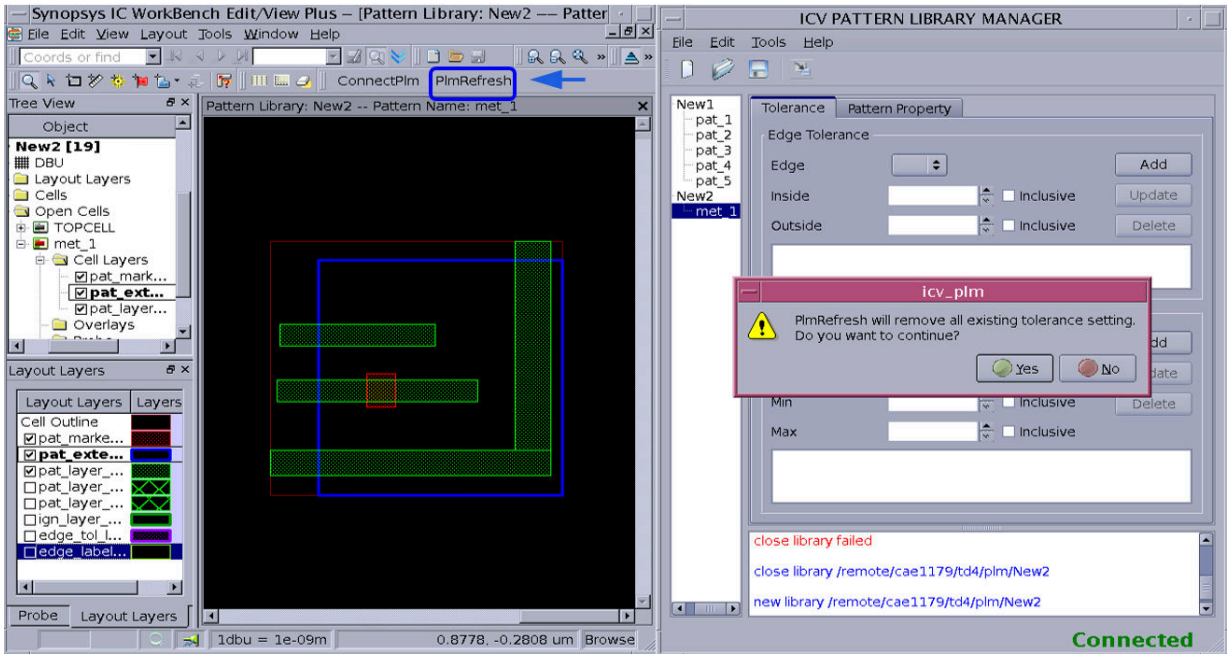
Figure 124 IC WorkBench EV Plus User Guide Help Menu



6. Click PlmRefresh in IC WorkBench EV Plus to update the pattern library with all of the layers created for this pattern. A dialog box appears in the Pattern Library Manager, as shown in Figure 125. Click Yes.

Feedback

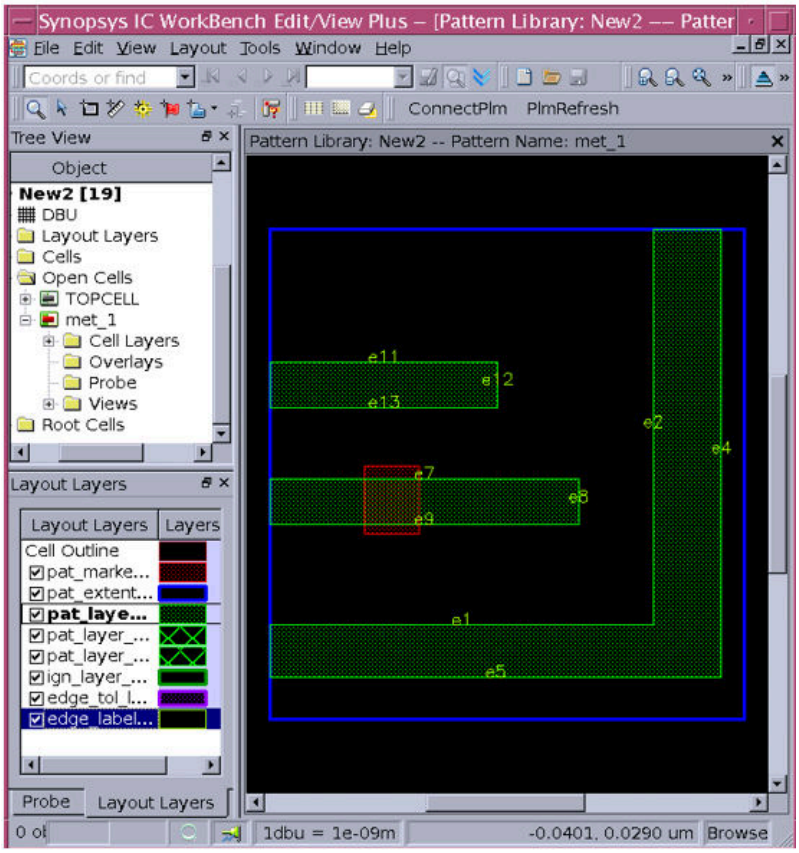
Figure 125 Dialog Box for Updating Pattern Library



The pattern is added in the pattern library, as shown in Figure 126.

Feedback

Figure 126 Pattern Added to Pattern Library



7. Define a pattern property in the Pattern Property tab if needed. See the Attaching Pattern Properties to a Pattern section for more information.
8. Add edge constraints in the Tolerance tab if needed. See the Adding an Edge Tolerance to a Pattern section for more information.
9. Repeat the previous steps to generate additional patterns under the same pattern library file. Upon completion, choose File > Save in the Pattern Library Manager to save the changes to your pattern library file.

Editing a Pattern

You can edit a pattern inside of the Pattern Library Manager by attaching one or more text properties to this pattern, or by specifying an edge tolerance for non-uniform matching mode.

Feedback

You can also edit a pattern layout in IC WorkBench EV Plus by adding, removing, or moving polygons, edges, and vertices.

Adding an Edge Tolerance to a Pattern

The Pattern Library Manager allows you to specify an edge tolerance (constraint) for non-uniform fuzzy matching mode. Constraints enable you to match different variations of a pattern to the defined pattern. Each edge should not have more than one edge constraint.

To specify an edge tolerance:

1. Open a pattern library that contains the patterns for editing.
2. Select the pattern for editing by clicking on the pattern name. The Tolerance and Pattern Property tabs appear in the Pattern Library Manager and the pattern layout is displayed in IC WorkBench EV Plus.
3. Choose an edge to specify the tolerance from the drop-down box. If the edge you choose is horizontal, the edge is restricted to move vertically. If the edge you choose is vertical, the edge is restricted to move horizontally.

   Note:

   You can use the edge numbers displayed in IC WorkBench EV Plus as a reference to identify the edge number.

4. Click the increase or decrease button to specify an inside and outside limit in nanometers. Select Inclusive if the tolerance interval includes the value, as shown in Figure 127.

Figure 127 Adding an Edge Tolerance

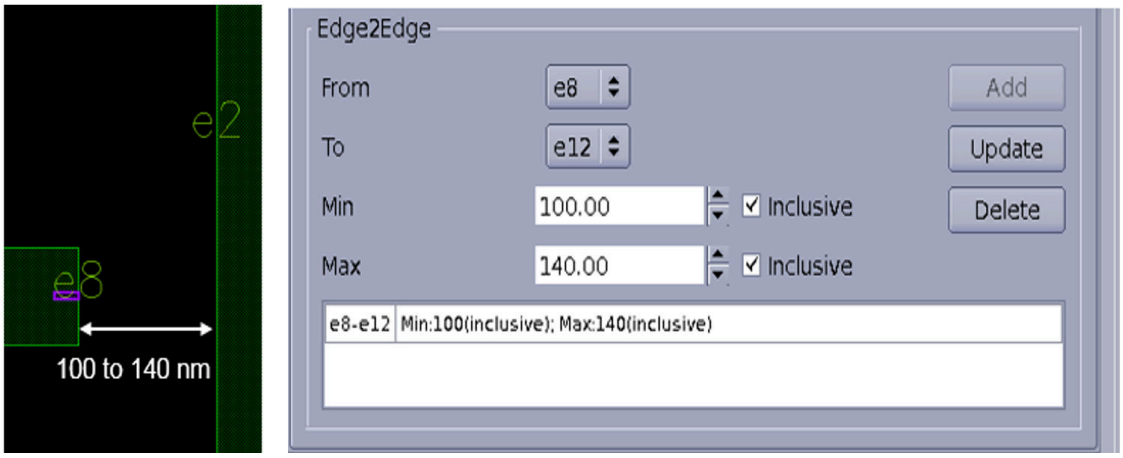


5. Click Add to add the tolerance to the pattern library. Edge tolerances appear on layer (9001:0) in IC WorkBench EV Plus.
6. Choose File > Save to save changes to the pattern library.

Feedback

7. Click update to modify an existing edge tolerance and Delete to remove an edge tolerance.
8. You can also specify edge-to-edge tolerance in the Edge2Edge pane by selecting two edges, as shown in Figure 128.

Figure 128 Adding an Edge-to-Edge Tolerance



Note:

For the edge-to-edge constraint to be recognized, at least one of the two selected edges must be defined with an edge tolerance.

Attaching Pattern Properties to a Pattern

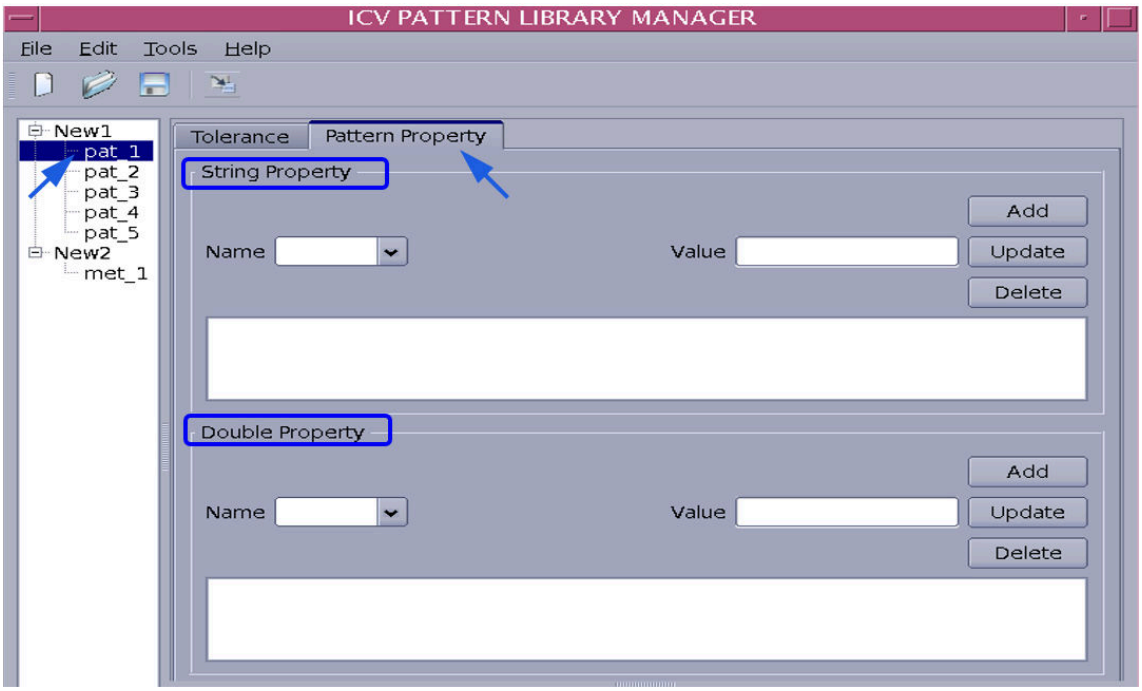
The Pattern Library Manager allows you to attach properties to a pattern during pattern library creation. Those properties are passed to the output pattern marker when a match is found. This gives you the ability to do post processing of the output pattern markers based on property names and property values using other IC Validator runset functions.

To attach properties to a pattern:

1. Open a pattern library that contains the patterns for editing.
2. Select the pattern for editing by clicking on the pattern name. The Tolerance and Pattern Property tabs appear. Click the Pattern Property tab to activate it, as shown in Figure 129.

Feedback

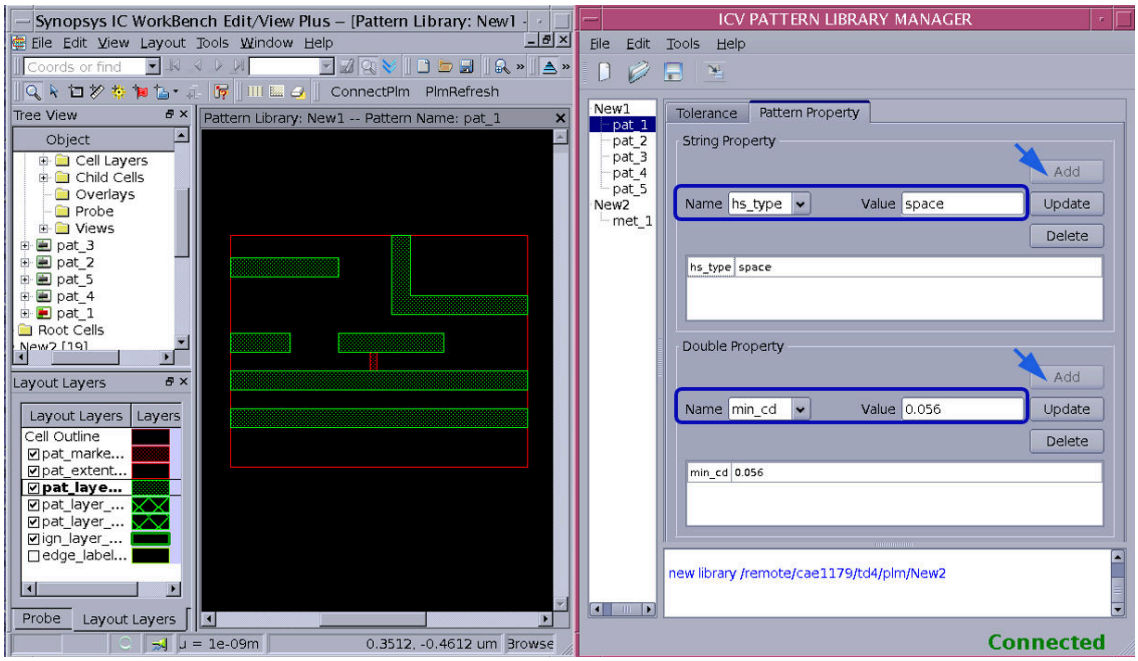
Figure 129 Pattern Property Tab



3. Add a property name and value to the String Property field if the property is a string type, or to the Double Property field if it is a double type. For the example in Figure 130, the `hs_type` property is a string type, and it is assigned a value of `space`. The `min_cd` property is a double type, and it is assigned a value of `0.056`.

Feedback

Figure 130 Attaching Pattern Properties to a Pattern



4. Click Add to add the properties to the pattern library.
5. Choose File > Save to save changes to the pattern library.
6. Click Update to modify an existing property or Delete to remove a property.

Editing a Pattern Layout in IC WorkBench EV Plus

You can modify a pattern layout in IC WorkBench EV Plus by adding, moving, or removing polygons and shapes for pattern layers, pattern extent, pattern marker, and ignore region layers. See the *IC WorkBench EV Plus User Guide*, which can be found on the Help menu, for more details on how to edit a layout.

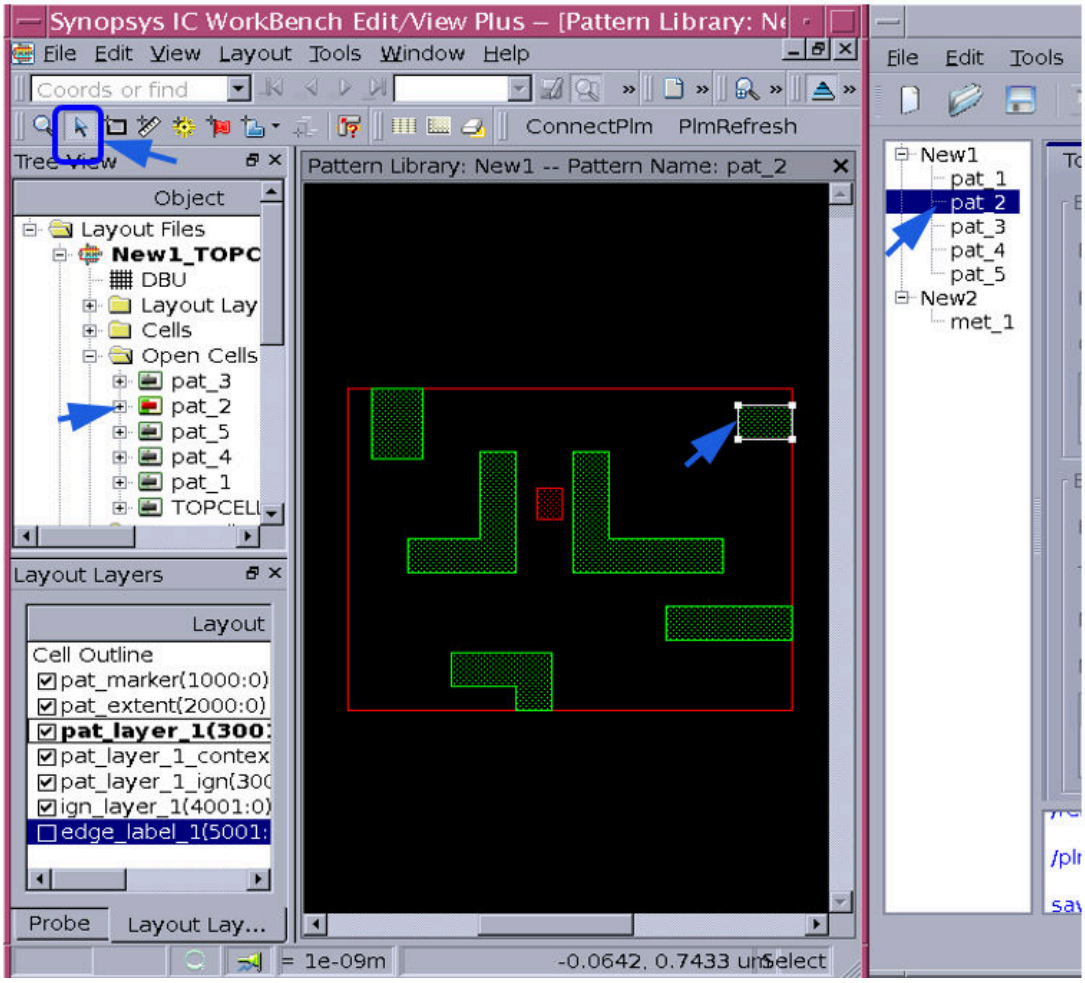
Example of removing a sliver:

1. Open a pattern library that contains the patterns for editing.
2. Select the pattern for editing by clicking the pattern name. The pattern cell is selected and appears in IC WorkBench EV Plus.

Feedback

3. Edit this pattern accordingly. For example, if you want to remove the sliver in `pat_2` as shown in the Figure 131, perform the following steps:

Figure 131 Editing a Pattern Layout



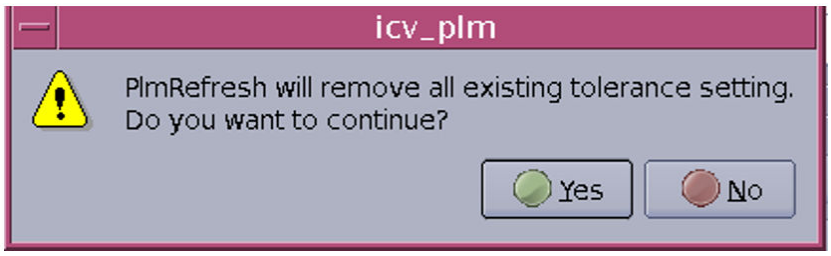
a. To select the shape, choose Tool > Select, or click the Select button on the Tools toolbar and click the shape.

b. To remove the selected shape from `pat_2`, choose Edit > Delete or click the Delete key on the keyboard.

c. Click PlmRefresh in IC WorkBench EV Plus to update the change made to `pat_2`. A message appears, as shown in Figure 132. Click Yes.

Feedback

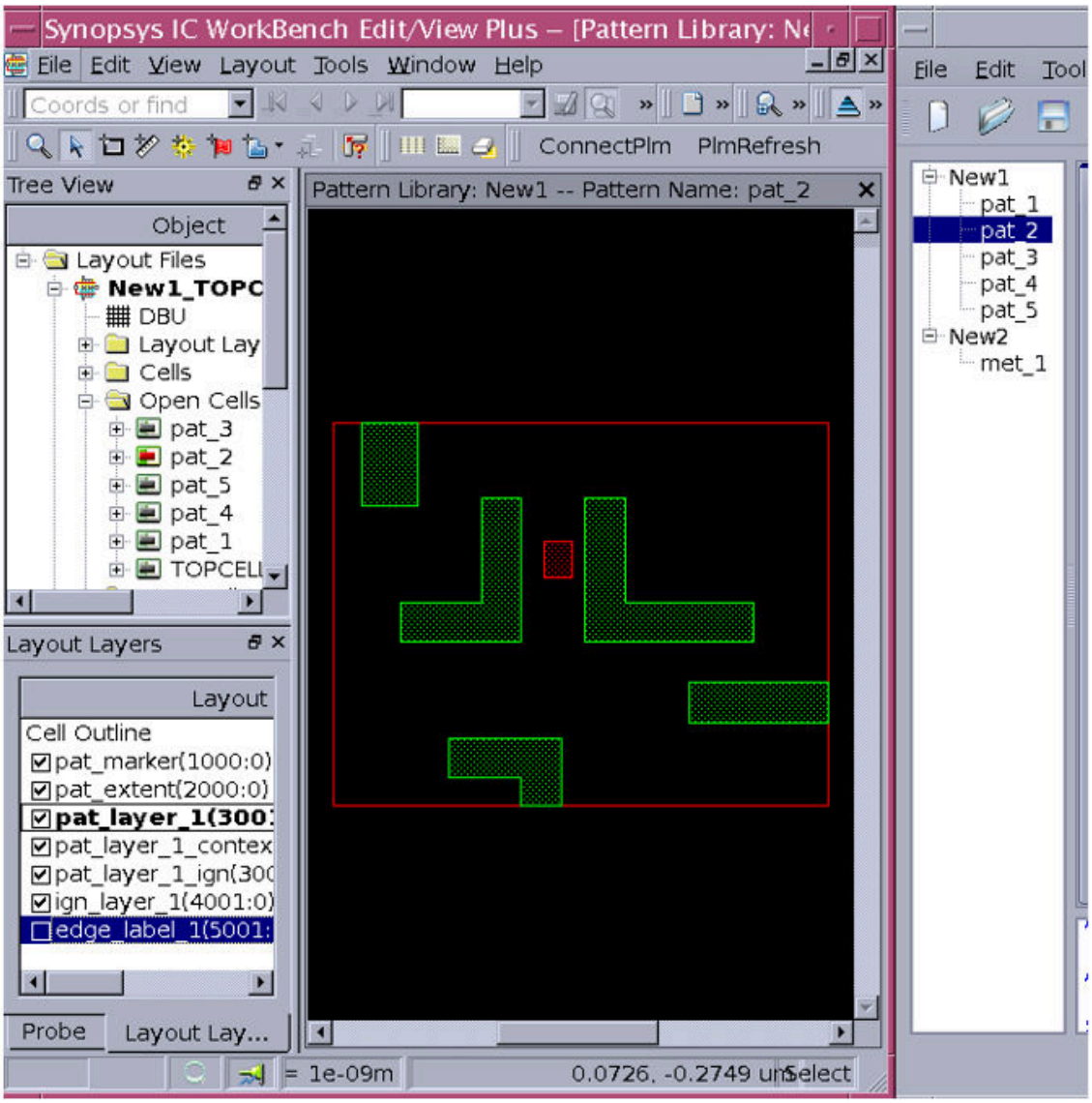
Figure 132 Pattern Library Manager Refresh Button



Feedback

4. In the Pattern Library Manager, choose File > Save to save the pattern library, as shown in Figure 133.

Figure 133 Saving the Pattern Library



Exporting and Importing a Pattern Library

In the IC Validator tool, you can use the GUI-based Pattern Library Manager to create a pattern library. You can also create a pattern library using the `pattern_learn()` function by running an IC Validator job.A pattern library generated in the Pattern Library Manager consists two parts: An OASIS directory to store the graphical information and a text file

Feedback

in XML format to store edge tolerances and edge-to-edge constraints. A pattern library created using the `pattern_learn()` function by running an IC Validator job consists of three parts: a binary file named pattern.dat, a text file named log to record pattern library creation parameters, and pattern statistics, and an optional text file in XML format to store edge tolerances and edge-to-edge constraints when the pattern matching mode is set to `PM_EDGE_NONUNIFORM`.The `pattern_match()` function executes pattern matching in an IC Validator run. The pattern library must be ready before a pattern matching run, and it must be in the binary format, which is also addressed as a pattern database (PDB) format. Table 72 shows a summary comparison of these two types of pattern library formats.

Table 72 Summary Comparison of Pattern Library Formats

Mode	Format	Import/Export
Pattern library created using `pattern_learn()` function	Pattern Database format includes: 1.pattern.dat (binary) 2.log 3.optional XML file to store edge tolerance when matching mode is PM_EDGE_NONUNIFORM	Must be imported in the Pattern Library Manager for editing
Pattern library created in Pattern Library Manager	Pattern Library Manager format includes: 1.OASIS to store graphical information 2.XML file to store edge and edge2edge tolerance	Must be exported from the Pattern Library Manager to use the pattern library in an IC Validator pattern matching run

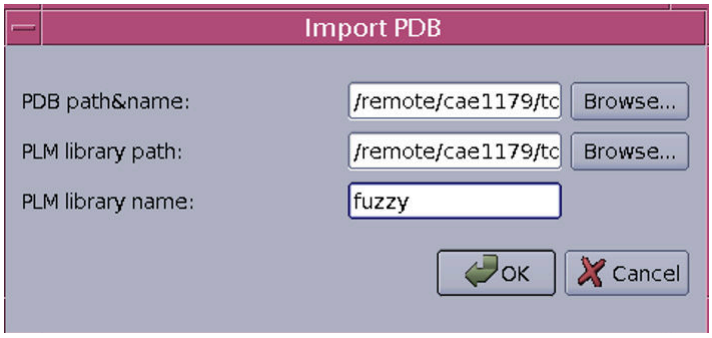
Importing a Pattern Database Format

A pattern library in a Pattern Database format must be imported to use the Pattern Library Manager for editing

1. In the Pattern Library Manager, choose File > Import.
2. Browse to the location of the pattern library in Pattern Database format. Browse to the location to save the pattern library to be converted to Pattern Library Manager format and enter a name for it, as shown in Figure 134.

Feedback

Figure 134 Importing a Pattern Library



Note:

If the pattern library in a Pattern Database format is locked (encrypted), the import fails.

3. Click OK. The pattern library is converted to a Pattern Library Manager format and uploaded in the Pattern Library Manager.

Exporting a Pattern Library Manager Format

A pattern library in a Pattern Library Manager format must be exported to use the pattern library in an IC Validator pattern matching run.

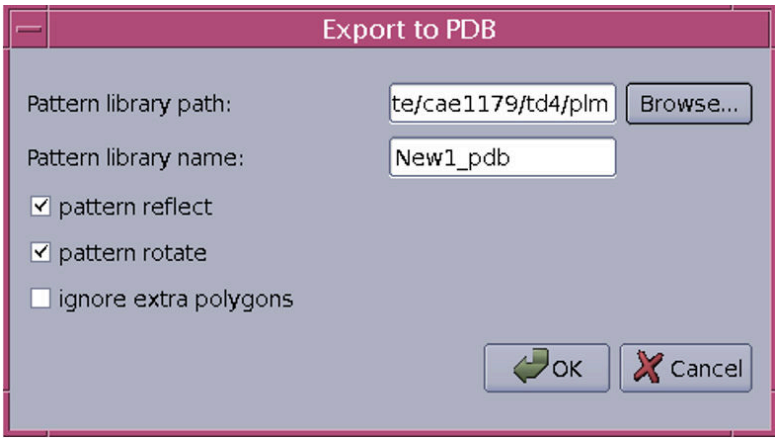
1. In the Pattern Library Manager, choose File > Open to load a pattern library for exporting or click to select a pattern library that is already open in the Pattern Library Manager.
2. Choose File > Export.
3. Browse to the location to save the pattern library to be converted to Pattern Database format and enter a name for it, as shown in Figure 135.

   Select pattern reflect and pattern rotate if the pattern in a different orientation is also considered as a match.

   Select ignore extra polygons if it is allowed for `PM_EXACT` matching mode.

Feedback

Figure 135 Exporting a Pattern Library



4. Click OK. The pattern library is converted to the Pattern Database format and saved on the disk with the name and path defined in step 3. An Export Result window appears, as shown in Figure 136. Click OK to close this window and complete the exporting process.

Figure 136 Pattern Library Export Result



This pattern library is ready to use in an IC Validator run.

Feedback

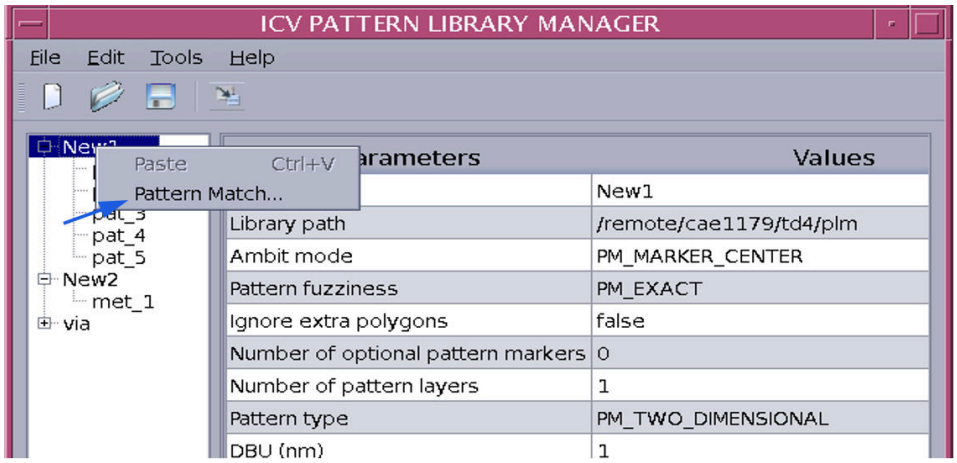
Running Pattern Matching in the Pattern Library Manager

The IC Validator tool uses the `pattern_match()` function to execute pattern matching in an IC Validator run. This means that an IC Validator runset must be created. See the Pattern Matching chapter of the *IC Validator User Guide* for more information about how to create an IC Validator rule file and run pattern matching in batch mode.

In addition to the batch mode, the Pattern Library Manager provides a convenient way to run pattern matching on a design layout with a generated pattern library without leaving the GUI.

1. In the Pattern Library Manager, click Pattern Match on the selected pattern library, as shown in Figure 137.

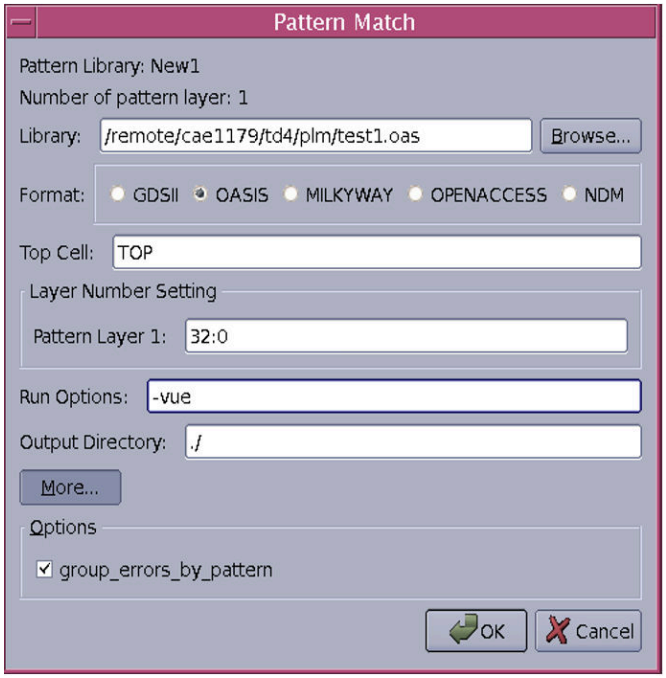
Figure 137 Performing Pattern Matching on the Selected Pattern Library



2. The Pattern Match window pops up, as shown in Figure 138.

Feedback

Figure 138 Pattern Match Window



a. Browse to the location of the input layout. Select the design format. Enter the top-cell name. Assign the pattern layers.

b. You can specify the IC Validator command-line option in the Run Options column. See the IC Validator Basics chapter of the *IC Validator User Guide* for more information about command-line options.

c. Specify the path to the output directory.

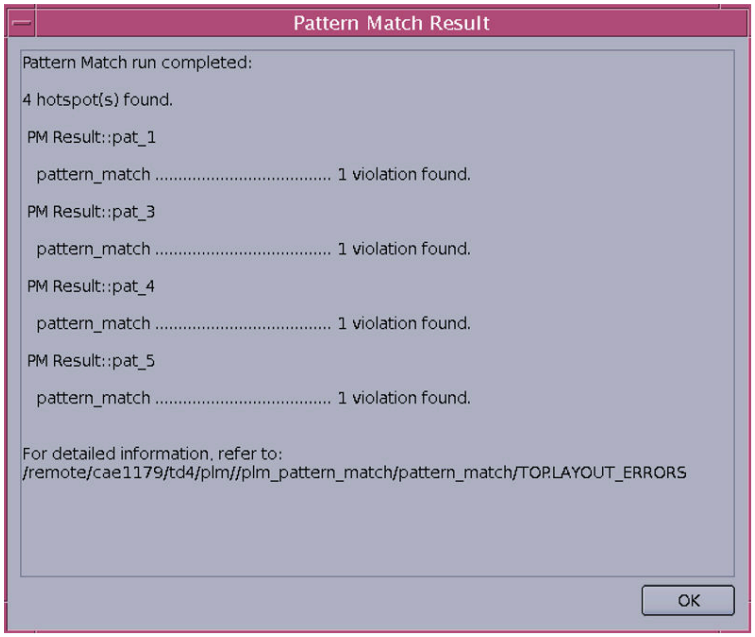
d. Click More to show additional options. Select the `group_errors_by_pattern` check box if you want to group the results based on the pattern name.

e. Click **OK** to execute pattern matching on the specified layout.

Feedback

3. Pattern matching results appear in a dialog box, as shown in Figure 139. Detailed information is saved in the output directory. Click OK to close this dialog box.

Figure 139 Pattern Matching Results Dialog Box



Restrictions and Limitations

The following features are not currently available in the Pattern Library Manager, but are supported by the `pattern_learn()` function in batch mode:

- Specifies that the IC Validator tool generates a pattern extent based on the user-defined ambit value and `ambit_mode`.
- Specifies the edge tolerance as user-defined input layers.
- Specifies the optional region layers.
- Specifies auto anchoring.

A

IC Validator Architecture

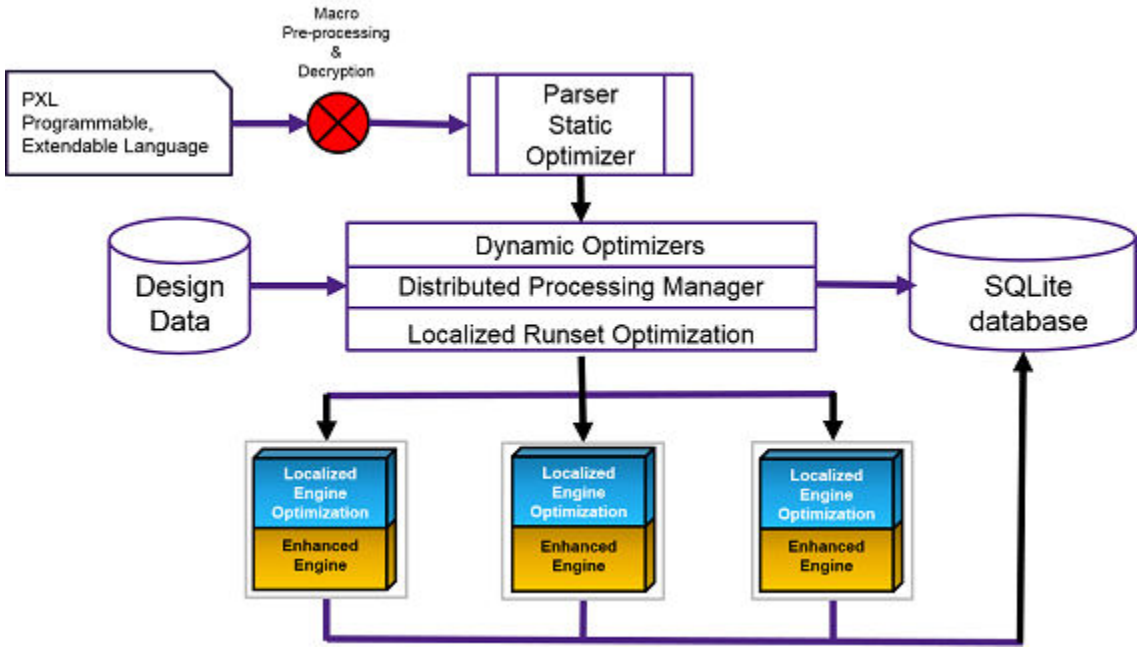
This appendix shows the basic IC Validator architecture.

Feedback

Basic Architecture

Figure 140 shows the basic IC Validator architecture.

Figure 140 Basic IC Validator Architecture



B

LVL Utility

This appendix discusses the Layout-Versus-Layout (LVL) utility and the QuickLVL utility that are available with the IC Validator tool.

The LVL utility is described in the following sections:

- Description
- Command-Line Options
- Using the OpenAccess Library Format
- Layer Files
- Comparing a Subset of Layers
- Handling of Text
- LVL Output
- Use Models

The QuickLVL utility is described in the following sections:

- QuickLVL Utility
- Cell-Level Quick LVL
- Differences Between LVL and QuickLVL
- Auto-Incremental DRC Focused on Differing Layers and Regions

Feedback

Description

The LVL utility allows you to

- Generate an assign file from a single library.
- Compare two layouts of a design using either an XOR-based comparison or a NOT-based comparison. The errors that are found can be viewed in VUE using the *cell1*_vs_*cell2*.vue file. See the *IC Validator VUE User Guide* for more information.

  Note:

  The LVL utility supports text with any text character, except the NULL character.

  For the NOT-based comparison,

  - The baseline layer is compared using a NOT operation with the check library layer.
  - The check library layer is compared using a NOT operation with the baseline layer.

  The errors from each NOT operation are shown separately in the *cell1*_vs_*cell2*.LVL_ERRORS file.

The LVL utility supports input in the following layout formats:

- GDSII
- Milkyway
- OASIS
- OpenAccess
- NDM

Note:

The LVL utility supports text with any text character, except the NULL character.

Command-Line Options

The LVL utility command-line syntax is

```
icv_lvl library1 library2 -c cell | -c1 cell1 -c2 cell2 [options]
```

Table 73 describes the basic command-line options and the command-line options available for distributed processing with the LVL utility.

Feedback

Table 73 Command-Line Options for the LVL Utility

Syntax	Description
`-64`	Runs the utility with 64-bit coordinates support.
`-af` *`filename`*	Name of the file where the assign layers in the library are stored. This file is generated by the LVL utility, and it has the same syntax as an IC Validator runset ASSIGN section. Include a path if you want the file in a directory that is not in the current directory.
`-c` *`cell`*	Cell name of both libraries, *library1* and *library2*.[6] The cell name is also needed for generating an assign file with the `-af` option.
`-c1` *`cell1`*	Cell name of the first library, *`library1`*.[6]
`-c2` *`cell2`*	Cell name of the second library, `library2`.[6]
`-cell_map_file`	Supports the mapping of cell names in the second input library to cell names in the first input library.
`-cr` *`filename`* `-create_runset` *`filename`*	Creates a LVL runset with the given name that can be run directly by the IC Validator tool.
`-ctp` `-compare_text_properties`	Compares GDSII magnitude, angle, and reflection text properties when performing text comparison.
`-elpc` *`num`* `-error_limit_per_check` *`num`*	Sets the error limit for each type of check. The default is 100. **Note:** Setting this command-line option to `ERROR_LIMIT_MAX` causes all errors up to a tool-defined maximum limit to be stored. The limit can be set above the tool-defined maximum limit by giving a specific number. However, setting the limit above the tool-defined maximum might result in decreased performance and increased disk usage for large numbers of errors, and therefore is not recommended. Setting it to `0` suppresses storage of all errors.
`-flat`	Flattens all text and geometry to the top cell and performs all layer comparisons in the top cell.
`-flatten`	Flattens all output to the top cell.
`-h`	Displays the usage message.
`-host_add hostname | number_of_cpus | hostname:number_of_cpus | LSF | SGE | NB`	Adds more hosts (including remote hosts) to an `icv_lvl` job that is already running. The usage is the same as the usage for `-host_init`. See Specifying Host Names and CPU Counts for more information.

6. *If the libraries have the same cell name, provide the cell name with the `-c` command-line option. If the libraries have different cell names, you must provide both cell names using the `-c1` and `-c2` command-line options.*

Feedback

Table 73 Command-Line Options for the LVL Utility (Continued)

Syntax	Description
`-host_init`	Launches an IC Validator job on a single host or multiple hosts. See Specifying Host Names and CPU Counts for more information. The priority of the remote login protocol used to log in to remote hosts is described in the `-host_login` command-line option. See the `-host_init` IC Validator command-line option in Chapter 1, IC Validator Basics for more information.
`-host_init LSF`	Acquires hosts through an LSF batch queuing system by reading either the LSB_HOSTS or LSB_MCPU_HOSTS environment variables. The priority of the remote login protocol used to log in to remote hosts is described in the `-host_login` command-line option. See the `-host_init LSF` IC Validator command-line option in Chapter 1, IC Validator Basics for more information.
`-host_init SGE`	Acquires hosts through an SGE batch queuing system by reading the PE_HOSTFILE environment variable. The priority of the remote login protocol used to log in to remote hosts is described in the `-host_login` command-line option. See the `-host_init SGE` IC Validator command-line option in IC Validator Basics" for more information.
`-host_disk`	Specifies a local group directory for all hosts specified with `-host_init` or `-host_add`. See the `-host_disk` IC Validator command-line option in IC Validator Basics for more information.
`-host_login`	Enables the IC Validator tool to log in to remote hosts to start processes for a distributed job. Ensure that the tool locates the binary/script supplied to `-host_login`. The default is `rsh`. See the `-host_login` IC Validator command-line option in IC Validator Basics for more information.
`-host_memory` *`Memory_Value`* `M|G|T`	Attempts to maintain the user-specified target memory for the hosts used in the IC Validator job by rescheduling and threading. You cannot use this switch to reduce the memory of a single command. See the `-host_memory` IC Validator command-line option in IC Validator Basics for more information.
`-ignore_text_case`	Instructs the utility to ignore case when comparing text. The default behavior is that text comparison considers case.
`-al`	Used with the `-layer_file` arguments. Adds any missing layers to the LVL run. There is a performance impact as the input files need to be scanned to determine whether layers are missing from the layer file. This argument can be used only when the files being checked are GDSII or OASIS.
`-lf` *`filename`* `-layer_file` *`filename`*	Layer file for both libraries that is used for layer mapping, layer merging, and layer naming. It must contain a valid PXL runset ASSIGN section.[7] Include the path if the library is not in the current directory. See Layer Files on page 450 for more information.

7. The layer file can be a full runset. The LVL utility, however, ignores everything but the ASSIGN section.

Feedback

Table 73 Command-Line Options for the LVL Utility (Continued)

Syntax	Description
`-lf1` *filename* `-layer_file1` *filename*	Layer file for the first library.[7] Always use this command-line option along with `-lf2`. Include the path if the library is not in the current directory. See Layer Files on page 450 for more information.
`-lf2` *filename* `-layer_file2` *filename*	Layer file for the second library.[7] Always use this command-line option along with `-lf1`. Include the path if the library is not in the current directory. See Layer Files on page 450 for more information.
`-oa_color_mapping` *option*	Specifies how to use the locked and unlocked anchor bit for the OpenAccess color layer mapping. The selections for *option* are • `ignore`. Uses the color mappings for which the locked or unlocked keyword is not specified, and ignores the color mappings for which locked or unlocked is specified. • `strict`. Uses the color mappings for which the locked or unlocked keyword is specified, and ignores the color mappings for which locked or unlocked is not specified. • `full`. Sets all shapes that are read with color to locked. The mapping for this setting is the same as the mapping for the `compatibility` setting. • `compatibility`. Follows the industry standard. Ignores color mappings for which the colorAnnotate keyword is located after the color specification. When no color mapping rule matches a color shape, the mapping is used without color. • `lockOnly`. Specifies that for a colorable layer, if the `-fullColor` option is `true` (the default) in the technology database for that layer, only shapes that are colored and locked are mapped. Uncolored or unlocked shapes on that layer are discarded. If the `-fullColor` option is `false` for a layer, the layer mapping is honored for that layer.
`-oa_color_mapping1` *ignore* | *strict* | *full* | *compatibility* | `lockOnly`	Specifies how to use the locked and unlocked anchor bit for the OpenAccess color layer mapping form, OpenAccess library1.
`-oa_color_mapping2` *ignore* | *strict* | *full* | *compatibility* | `lockOnly`	Specifies how to use the locked and unlocked anchor bit for the OpenAccess color layer mapping form, OpenAccess library2.
`-of` *options_file*	Optional file containing an IC Validator runset OPTIONS section. This file can contain any of the OPTIONS section functions, including the `gds_options()`, `milkyway_options()`, and `oasis_options()` functions.
`-olf | -output_library_format` *<format>*	Specifies the output library format. If not provided, the baseline library format is used. The supported format includes GDSII, OASIS, NDM, and OPENACCESS.

Feedback

Table 73 Command-Line Options for the LVL Utility (Continued)

Syntax	Description
`-rsi`	Reports the following input library information. Supports only GDSII and OASIS input formats. Reports • GDSII last modified and last accessed • md5sum computed with md5sum -b *file* • format of the input library • path to the input library • command used for the library input Enumerators must be in a comma-separated list, and the entire list must be within quotation marks. See the `report_streamfile_information` argument of the `run_options()` function. **Note:** The `-rsi` command-line option overrides its runset setting in the `run_options()` function.
`library1`	Baseline library name. Include the path if the library is not in the current directory. When one library is specified with the `-af` option, the LVL utility generates an assign file. You cannot generate an assign file and compare two layouts in one run of the LVL utility. The following examples show how to specify input layouts for various formats: • GDSII. `/path/to/file.gds` • OASIS. `/path/to/file.oas` • Milkyway. `/library/path/LIBRARY_NAME` • OpenAccess. `/path/to/lib.defs/LIBRARY_NAME` • NDM. `/library/path/LIBRARY_NAME`
`library2`	Check library name. Include the path if the library is not in the current directory. Do not specify this file when using the `-af` command-line option. You cannot generate an assign file and compare two layouts in one run of the LVL utility. The following examples show how to specify input layouts for various formats: • GDSII. `/path/to/file.gds` • OASIS. `/path/to/file.oas` • Milkyway. `/library/path/LIBRARY_NAME` • OpenAccess. `/path/to/lib.defs/LIBRARY_NAME` • NDM. `/library/path/LIBRARY_NAME`
`-mf` *`filename`* `-map_file` *`filename`*	Specifies the layer mapping file applied to both libraries. See Mapping Layers on page 451 for more information.
`-mf1` *`filename`* `-map_file1` *`filename`*	Specifies the layer mapping file applied to the first library. See Mapping Layers on page 451 for more information.

Feedback

Table 73 Command-Line Options for the LVL Utility (Continued)

Syntax	Description
`-mf2 filename` `-map_file2 filename`	Specifies the layer mapping file applied to the second library. See Mapping Layers on page 451 for more information.
`-ndl label` `-ndm_design_label`	Design label for both libraries, *library1*[6] and *library2*.[8]
`-ndl1 label` `-ndm_design_label1`	Design label for the first library, *library1*.[6]
`-ndl2 label` `-ndm_design_label2`	Design label for the second library, *library2*.[6]
`-not`	Instructs the utility to do a two NOT-based comparison rather than an XOR-based comparison.
`-oa_dm4`	Uses OpenAccess shared libraries that are compatible with DM4 plug-ins.
`-outside`	Instructs the utility to do a two OUTSIDE-based comparison rather than an XOR-based comparison. When the `not_interacting()` function is executed, any data that touches is not considered for this operation. Use the `-report_touch` command-line option along with the `-outside` option to allow touches to be considered as not interacting.
`-quick`	Performs cell-level Quick LVL. See QuickLVL Utility on page 459 for more information. **Note:** The Quick LVL (`-quick`) utility works only with a single host. If multiple hosts are acquired from the batch queuing systems, Quick LVL errors out.
`-resolution`	Allows you to specify the internal resolution for the IC Validator run from within the LVL utility.
`-rt ALL | POINT` `-report_touch ALL | POINT`	Specifies how point touches are treated. This option is used with the `-outside` option. • `ALL`. Treat polygon line touches and edge point touches as outside. • `POINT`. Treat point touches as outside.
`-sel "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"` `-select_edge_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`	Specifies the edge layers and datatypes that are used in the comparison. Ranges can be used. If a datatype is not specified, it defaults to all datatypes.[9]

8. *If the libraries have the same cell name, provide the cell name with the `-c` command-line option. If the libraries have different cell names, you must provide both cell names using the `-c1` and `-c2` command-line options.*
9. *If you are comparing two OpenAccess libraries, you can use the layer name and purpose name instead of layer number and datatype number.*

Feedback

Table 73 Command-Line Options for the LVL Utility (Continued)

Syntax	Description
`-small_cell`	Forces small cell mode to run as fast as possible on small designs for the comparison part of an LVL run. To prevent slowdown on larger designs, the IC Validator tool exits with an error if it determines that the input design is too large.
`-sn layer_name1 ...` `-select_names layer_name1 ...`	Specifies, by name, the layers on which the utility is run.
`-spl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"` `-select_polygon_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`	Specifies the polygon layers and datatypes that are used in the comparison. Ranges can be used. If a datatype is not specified, it defaults to all datatypes. To select text layers for comparison, the `-stl` command-line option should be used.[9]
`-split_output`	Specifies that when used with the `-not` command-line option, the LVL utility writes `not12` results to `Cell1_vs_Cell2_not12` and writes `not21` results to `Cell1_vs_Cell2_not21`. The `not12` notation corresponds to the results of (BaselineLayer not Checklayer) for each layer in the baseline library (the first library listed on the icv_lvl command line) and each corresponding layer in the check library (the second library on the icv_lvl command line). Likewise, the `not21` notation corresponds to the results of (CheckLayer not BaselineLayer) for each compared layer.
`-stl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"` `-select_text_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`	Specifies the text layers and datatypes that are used in the comparison. Ranges can be used. If a datatype is not specified, it defaults to all datatypes. To select polygon layers for comparison, the `-spl` command-line option should be used.[9] [10]
`-sub_cell`	Indicates a run on a small subcell inside a larger OASIS layout. Avoids reading the entire layout.
`-suppress_empty_ layer_errors`	Suppresses the reporting of empty-layer error polygons.
`-text`	Instructs the utility to compare text between two layouts. See Handling of Text on page 454 for more information. When this command-line option is not used, only geometry is compared, not text.
`-top_cell_only`	Compares only the data in the top cells from each library. All lower-level cell data is dropped.

10. Text matches only when both location and text string match. Text differences appear as an X in VUE. Text comparison is only a cell-level operation. For cases of hierarchy mismatch, hierarchy and layout are compared first followed by text. If the hierarchy and layout do not compare, the text cannot be accurately compared. In general, if two cells do not have the same name, size, and placement, their text is not compared.

Feedback

Table 73 Command-Line Options for the LVL Utility (Continued)

Syntax	Description
`-uel "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"` `-unselect_edge_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`	Specifies the edge layers and datatypes that are excluded from the comparison. Ranges can be used. If a datatype is not specified, it defaults to all datatypes.[9]
`-un layer_name1 ...` `-unselect_names layer_name1 ...`	Specifies by name the layers that are excluded when the utility is run.
`-upl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"` `-unselect_polygon_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`	Specifies the polygon layers and datatypes that are excluded from the comparison. Ranges can be used. If a datatype is not specified, it defaults to all datatypes.[9]
`-utl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"` `-unselect_text_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`	Specifies the text layers and datatypes that are excluded from the comparison. Ranges can be used. If a datatype is not specified, it defaults to all datatypes.[9][10]
`-transform [RX,] [0|90|180|270|DX,DY]`	Indicates that the top cell of the check layout is transformed as described on the command line. `RX`: Indicates reflection around X-axis 0|90|180|270; counterclockwise rotation angle `dx dy`; translation in X and Y directions, in microns. See the following examples: • `-transform 90,0,0` • `-transform RX,90,1,-1.2` • `-transform RX` See QuickLVL Utility on page 459 for more information.
`baseline_layout`	Baseline library name. Include the path if the library is not in the current directory. The following examples show how to specify input layouts for each format: • OASIS. `/path/to/file.oas` • GDSII. `/path/to/file.gds` See QuickLVL Utility on page 459 for more information.
`check_layout`	Check library name. Include the path if the library is not in the current directory. The following examples show how to specify input layouts for each format: • OASIS. `/path/to/file.oas` • GDSII. `/path/to/file.gds` See QuickLVL Utility on page 459 for more information.
`-V`	Prints the utility version.

Feedback

Using the OpenAccess Library Format

The LVL utility command-line options are useful when you are comparing layouts that are in the OpenAccess format. The OpenAccess command-line options are:

- `-oalm` *`filename`*
  `-oa_layer_map` *`filename`*
- `-oalm1` *`filename`*
  `-oa_layer_map1` *`filename`*
- `-oalm2` *`filename`*
  `-oa_layer_map2` *`filename`*
- `-oacm` *`cellmap`*
  `-oa_cell_map` *`cellmap`*
- `-oacm1` *`cellmap1`*
  `-oa_cell_map1` *`cellmap1`*
- `-oacm2` *`cellmap2`*
  `-oa_cell_map2` *`cellmap2`*
- `-oa_color_mapping` *`ignore`* | *`strict`* | *`full`* | *`compatibility`*
- `-oa_color_mapping1` *`ignore`* | *`strict`* | *`full`* | *`compatibility`*
- `-oa_color_mapping2` *`ignore`* | *`strict`* | *`full`* | *`compatibility`*
- `-oav` *`viewname`*
  `-oa_view` *`viewname`*
- `-oav1` *`viewname`*
  `-oa_view1` *`viewname`*
- `-oav2` *`viewname`*
  `-oa_view2` *`viewname`*

If you are comparing an OpenAccess library to another OpenAccess library, then you do not need to specify a layer mapping file. The LVL utility matches the layer/purpose pairs and runs the comparison. However, if you are comparing an OpenAccess library to a GDSII, OASIS, Milkyway, or NDM library, then a layer mapping file is needed to map the OpenAccess layer/purpose pair to a layer/datatype in the GDSII, Milkyway, Oasis, or NDM library. For example,

```
% icv_lvl lib.defs/OA_LIB gds_lib.gds -c top -oalm layer_map.txt
```

Feedback

As shown in this example,

- The OpenAccess library (OA_LIB), which is listed first, is compared to the GDSII library (gds_lib.gds).
- The lib_defs file lists all libraries needed for the design including the OpenAccess library. The file name must be lib.defs. If the file is not located in the current directory, specify the path leading to it followed by the file name, (libs.def). For example, datapath1/testcase1/lib.defs/OA_LIB

The following is an example of the contents of the lib.defs file:

```
DEFINE analogLib ./PDK/lib/analogLib
ASSIGN analogLib libMode shared
DEFINE basic ./PDK/lib/basic
ASSIGN basic libMode shared
DEFINE device_lib ./PDK/lib/device_lib
ASSIGN device_lib libMode shared
DEFINE OA_LIB ./OA_LIB
```

OpenAccess Layer Mapping File

If you are comparing an OpenAccess database to another OpenAccess database, then you do not need to provide a layer mapping file. The LVL utility matches the layer/purpose pairs and runs a comparison.

However, if you are comparing an OpenAccess database to a GDSII, OASIS, Milkyway, or NDM library, then you need to provide a layer mapping file to map the OpenAccess layer/purpose pairs to a layer/datatype in the GDSII, OASIS, Milkyway, or NDM file.

If both OpenAccess libraries use the same layer mapping file, specify:

```
-oalm filename
```

If each input OA database has a unique layer mapping file, specify the databases individually:

```
-oalm1 filename1
-oalm2 filename2
```

For more information, see the definition of the `layer_mapping_file` argument of the `openaccess_options()` function in the *IC Validator Reference Manual*.

The following is an example of the OpenAccess layer mapping format.

```
#OA Layer Name    OA Purpose Name    Stream layer    Stream Datatype
Layer1            drawing            1               0
Layer2            drawing            2               1
Layer3            drawing            3               2
Layer4            drawing            4               0
Layer4            boundary           4               1
```

Feedback

Cell Mapping

When you are comparing an OpenAccess database to a different format, you might need to provide the cell mapping to prevent flattening. The OpenAccess library specifies unique cells by a library/cell/view triplet. The IC Validator tool and the LVL utility must assign a unique cell name to each unique library/cell/view triplet. The LVL utility flattens hierarchies with mismatched cell names until the two hierarchies match. Using a cell mapping file can ensure that cells are named consistently between the two libraries being compared, and prevent this flattening.

For more information, see the definition of the `cell_mapping_file` argument of the `openaccess_options()` function in the *IC Validator Reference Manual*.

If both OpenAccess libraries use the same cell mapping, specify:

```
-oacm cellmap
```

If each input OpenAccess database has a unique mapping, specify the databases individually:

```
-oacm1 cellmap1
-oacm2 cellmap2
```

The following is an example cell mapping file.

```
#Library Name   Cell Name   View Name   Stream Name
des_lib         nand2       layout      nand2_1
sdes_lib        nand2       layout      nand2_2
des_lib         cell_nu     layout      cell_nu
sdes_lib        cell_nu     layout      cell_new
```

Error Output Format for OpenAccess Baseline Library

The error output written to an OpenAccess library in the current working directory named *cell1*_vs_*cell2*.

The error data in the output library uses layer names from the baseline library, when possible.

The general format for layer name when a layer file is not given is

L*layer*P*purpose*

The output purpose name is Xor, BaselineNotCheck, or CheckNotBaseline, depending on the LVL method chosen.

The following is an example output.

```
Checking LxxxPyyy_Baseline XOR LaaaPbbb_Check, output to (LxxxPyyy; Xor)
Checking LxxxPyyy_Baseline NOT LaaaPbbb_Check, output to (LxxxPyyy; BaselineNo
tCheck)
```

Feedback

```
Checking LaaaPbbb_Check NOT LxxxPyyy_Baseline, output to (LaaaPbbb; CheckNotBa
seline)
```

Setting the Cell View Name

When specifying an OpenAccess library on the command line, you might need to set the view name to something other than the default view of “layout” for the top cell.

If both OpenAccess libraries use the same view name, specify

```
-oav viewname
```

If each input OpenAccess database has a different view name for the top cell, specify the databases individually:

```
-oav1 viewname1
-oav2 viewname2
```

Layer Files

By default, all layers of the layout databases are compared. However, by using command-line options, you can run the utility on a subset of these layers. You can also specify mapping files to use instead of mapping files included internally in the Milkyway and NDM libraries.

Layers are mapped before layers are selected of filtered using command-line options. For example, if `-lf layerfile` and `-mf mapfile` are used at the same time, then layer numbers and datatypes are first mapped using the *mapfile* file before interpreting the *layerfile* file.

Selecting a Subset of Layers

When you specify a layer file using the `-lf` command-line option or using both the `-lf1` and `-lf2` command-line options, only those layers that appear in the layer file are compared. No comparison is performed on layer numbers that do not appear in any of the assign statements.

These restrictions apply to the layer files:

- When a layer/datatype pair is used multiple times, any redundant assigns are ignored.

  In this example, the third assignment, L3D0, is ignored.

  ```
  L1D0 = assign({{1,0}});
  L2D0 = assign({{2,0}});
  L3D0 = assign({{1,0}});
  ```

Feedback

- The LVL utility stops with an error if a layer file contains multiple definitions of a layer name.

  In this example, layer A is defined twice. Therefore, the LVL utility reports an error. You must remove one of the definitions.

```
A = assign({{1,0}});
B = assign({{2,0}});
A = assign({{10,0}});
```

Mapping Layers

The mapping file can be included internally as part of the Milkyway and NDM libraries. Internal mapping files are used by default. Using the `-mf`, `-mf1`, and `-mf2` command-line options, you can map layers using an external file. When you specify a mapping file on the command-line, it is used instead of the internal mapping file.

Note:

To map layers using the `-mf`, `-mf1`, and `-mf2` command-line options on an OpenAccess library, you must also use the `-oalm`, `-oalm1`, and `-oalm2` command-line options to convert LayerName/Purpose string pairs to LayerNumber/Datatype numbered pairs.

The mapping file can be in two of three formats: the Milkyway format, the NDM format, or the IC Validator mapping file format. See the *Milkyway Database Application Note* and the *Library Data Preparation for IC Compiler User Guide*, which are available on SolvNetPlus, for information about the Milkyway and NDM formats. See Layer Mapping for information about the IC Validator mapping file format.

The `-lf`, `-lf1`, `-lf2`, `-spl`, `-upl`, `-sel`, `-uel`, `-stl`, `-utl`, `-sn`, and `-un` command-line options operate on layers after they have been mapped.

If do not want to use internal mapping files, you can turn them off using the mapping file command-line options. Use `-mf1""` to turn it off for the first library. Use `-mf2 ""` to turn it off for the second library. Use `-mf ""` to turn it off for both libraries.

For example, consider two input libraries:

- one.gds. This library contains data on layer 1, datatype 0.
- two.gds. This library contains data on layer 2, datatype 0, which has the same data as on layer 1 in the one.gds library.

The mf2 mapping file has

D 2:0 1 0

The LVL utility command is

Feedback

```
icv_lvl one.gds two.gds -mf2 mf2 -c top
```

The `-mf2` command-line option allows you to map layer 2 data in the two.gds library to layer 1 data. The result is that both libraries end up having their data on the same layer and the LVL comparison is clean.

Alternatively, you could use layer files to accomplish the same thing. For example,

- file1 contains

  ```
  Metal1 = assign({{1, 0}});
  ```

- file2 contains

  ```
  Metal1 = assign({{2, 0}});
  ```

The LVL utility command is

```
icv_lvl one.gds two.gds -lf1 file1 -lf2 file2 -c top
```

The first method, which uses mapping files, is easier if you already have IC Validator layer mapping files for IC Validator runs that can be used with the LVL utility mapping file command-line options.

Comparing a Subset of Layers

You can use filtering command-line options to compare a subset of polygon and edge layers, with or without a layer file. These rules apply:

- You cannot use a select option with the corresponding unselect (exclude) option.
- You cannot use filtering by name with filtering by number.

The filtering options are shown in Table 74. See the following sections for more information.

Table 74 Filtering Options

Data filtering category	By name	By number

Select	`-sn`	`-spl`
		`-sel`
		`-stl`

Unselect	`-un`	`-upl`
		`-uel`
		`-utl`

Feedback

Note:

You cannot use both the `-sn` and `-un` options in the same run of the utility.

You cannot use both the `-spl` and `-upl` options in the same run of the utility.

You cannot use both the `-sel` and `-uel` options in the same run of the utility.

You cannot use both the `-stl` and `-utl` options in the same run of the utility.

Comparing by Name

You can use command-line options to limit the polygon and edge layers being compared when a layer file is provided using the `-lf` command-line option or using both the `-lf1` and `-lf2` command-line options. This filtering is referred to as filtering by name.

The command-line options used to limit by name the layers being compared are:

- `-sn` *`layer_name1`* `...`
  `-select_names` *`layer_name1`* `...`

- `-un` *`layer_name1`* `...`
  `-unselect_names` *`layer_name1`* `...`

In this example, the LVL utility runs on the layers POLY, NWELL, and METAL1 layers:

```
-sn POLY NWELL METAL1
```

In this example, the LVL utility runs on all the layers except METAL2:

```
-un METAL2
```

Comparing by Number

You can use command-line options to limit the layers being compared when no layer file is provided. This filtering is referred to as filtering by number. Polygon, edge, and text layers are filtered separately.

The command-line options used to limit by number the layers being compared are:

- `-sel "`*`layer1`*`[-`*`layer1max`*`] [,`*`dtype1`*`[-`*`dtype1max`*`]] [...]"`
  `-select_edge_layers "`*`layer1`*`[-`*`layer1max`*`] [,`*`dtype1`*`[-`*`dtype1max`*`]] [...]"`

- `-uel "`*`layer1`*`[-`*`layer1max`*`] [,`*`dtype1`*`[-`*`dtype1max`*`]] [...]"`
  `-unselect_edge_layers "`*`layer1`*`[-`*`layer1max`*`] [,`*`dtype1`*`[-`*`dtype1max`*`]] [...]"`

- `-spl "`*`layer1`*`[-`*`layer1max`*`] [,`*`dtype1`*`[-`*`dtype1max`*`]] [...]"`
  `-select_polygon_layers "`*`layer1`*`[-`*`layer1max`*`] [,`*`dtype1`*`[-`*`dtype1max`*`]] [...]"`

Feedback

- `-upl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`
  `-unselect_polygon_layers "layer1[-layer1max] [,dtype1[-dtype1max]]`
  `[...]"`
- `-stl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`
  `-select_text_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`
- `-utl "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`
  `-unselect_text_layers "layer1[-layer1max] [,dtype1[-dtype1max]] [...]"`

In the following example, the LVL utility runs on the following polygon layers:

- 1 through 4, all datatypes
- 5, datatypes 0 through 7
- 6, all datatypes
- 7 and 8, datatypes 0 through 7

```
-spl 1-4  5,0-7  6  7-8,0-7
```

In the next example, the LVL utility runs on all polygon layers and datatypes except for datatypes 5, 6, and 7 in polygon layers 1, 2, and 3.

```
-upl 1-3,5-7
```

In this example, the LVL utility runs on the following edge layers:

- 1 through 5, datatypes 0 through 7
- 6 through 8, datatypes 5 through 7

```
-sel 1-5,0-7  6-8,5-7
```

In the next example, the LVL utility runs on all the edge layers and datatypes except edge layer 2 and datatypes 5 and 6 in edge layer 4:

```
-uel 2 4,5-6
```

Handling of Text

The `-text` command-line option instructs the LVL utility to compare text between two layouts. If one or both databases have multiple text at the same coordinate, (x1,y1), the utility matches as many text strings as possible at (x1,y1) between the two databases. The errors are reported as follows:

- If there is only one unmatched text at (x1, y1) in each database:

```
Violations Found:TEXT_XOR
- - - - - - - - - - - - - - - - - - - - - - - - - -
```

Feedback

```
Structure (Position x, y)  BaselineText CheckText
- - - - - - - - - - - - - - - - - - - - - - - - - - -
A         (x1, y1)         base_text    check_text
```

- If there is one unmatched baseline text and no unmatched checked text, or vice versa:

```
Violations Found:TEXT_XOR
- - - - - - - - - - - - - - - - - - - - - - - - - - -
Structure (Position x, y)  BaselineText CheckText
- - - - - - - - - - - - - - - - - - - - - - - - - - -
A         (x1, y1)         base_text
```

- If there are multiple unmatched text at (x1,y1) in the baseline database and at least one unmatched text at (x1,y1) in the check library:

```
Violations Found:TEXT_XOR
- - - - - - - - - - - - - - - - - - - - - - - - - - -
Structure (Position x, y)  BaselineText CheckText
- - - - - - - - - - - - - - - - - - - - - - - - - - -
A         (x1, y1)         base_text1   check_text1
A         (x1, y1)         base_text1   check_text2
A         (x1, y1)         base_text2   check_text1
A         (x1, y1)         base_text2   check_text2
A         (x1, y1)         base_text3   check_text1
A         (x1, y1)         base_text3   check_text2
```

  In the preceding example, there are three unmatched baseline text and two unmatched checked text. In general, all possible combinations of unmatched text are reported.

LVL Output

The LVL utility errors are saved in the files described in Error Files.

The output library has the same format as the baseline library. The default output cell is from the baseline library. VUE overlays differences to the baseline library.

- If the input design files are in either the GDSII or OASIS format and LVL runs with the layer file, differences are reported on layers that are listed in the LVL_ERRORS file, which might be different than the baseline layer or check layout.

  For example, a difference between baseline layer {31,0} and check layer {131, 0}, which is mapped to the same layer using the LVL command-line options, can be reported in output layer {1, 23}.

- If the input design files are in either the GDSII or OASIS format and LVL runs with the layer mapping file and without a layer file, differences are reported on a layer determined by the name generated based on the mapping file.

  For example, if the layer name is L10D15, the difference is reported in layer {10, 15}.

Feedback

- If the input design files are in either the GDSII or OASIS format and LVL runs without the layer mapping file or layer file, differences between the two layers are reported on the same layer of the output library.

  For example, a difference between baseline layer {31, 0} and check layer {31, 0} is reported in output layer {31, 0}.

If one of the input files (for example, OpenAccess or Milkyway) is not in the GDSII or OASIS format, the differences are reported on the layers that are listed in the LVL_ERRORS file, which might be different from the baseline layer or check layout.

For example, a difference between baseline layer {31,0} and check layer {131, 0}, which is mapped to the same layer using the LVL command-line options, can be reported in output layer {1, 23}.

Exit Status

The exit status values of the `icv_lvl` executable are shown in Table 75.

Table 75 Exit Status Values

Exit status value	Definition
0	Ran successfully and found no differences.
1	Ran successfully and found differences.
negative number	An error occurred.

In this example, the `icv_lvl` executable ran successfully and found no differences:

```
% icv_lvl lib1 lib2 -c cell
% echo $?
0
```

Error Files

Table 76 describes the error files that are generated and stored in the current working directory.

Note:

You can specify the maximum number of errors for each type of check using the `-elpc` option. See Table 73.

The LVL utility also provides screen output that contains real-time results from each XOR or NOT operation.

Feedback

Table 76 Error Files

File name	Description
*cell1*_vs_*cell2*.LVL_ERRORS	Error file containing all comparison errors segregated by layer.
*cell1*_vs_*cell2*.LVL_RESULTS	Results file that gives a brief description of each step of the run and indicates whether there are any errors. When there is an error, the results file points you to the specific *cell*.RESULTS file that is located in the run_details directory of the failed run. See the `run_options()` function in the *IC Validator Reference Manual* for information about specifying the *cell*.RESULTS file.
*cell1*_vs_*cell2*.gds *cell1*_vs_*cell2*.oasis *cell1*_vs_*cell2*/	Output libraries containing comparison differences in GDSII, OASIS, Milkyway, OpenAccess, and NDM formats. For OpenAccess libraries, the lib.defs file in the current directory is created or updated with the output library definition.
lvl_details/pydb	SQLite database (the error database, PYDB) containing the comparison errors segregated by each XOR or NOT operation performed by the LVL utility.
lvl_details/*cell1*_vs_*cell2*.lvl_sum	Summary file containing the following information for each run of the LVL utility: • The `icv_lvl` executable command line. • Indication of success or failure. • Purpose of the run. • Elapsed time and peak memory of the run.
*cell1*_vs_*cell2*.vue	File with overlay differences to the baseline library. This file allows you to view the errors using VUE.

Output Directories

Table 77 describes the directories that are created by the LVL utility. They are in the current working directory.

Table 77 Output Directories

Directory name	Description
lvl_details/compare_layers/	Directory containing additional diagnostic information about the comparison process between the baseline library and the check library.
lvl_details/extract_layers/	Directory containing summary information regarding preprocessing of the check library.

Feedback

Table 77 Output Directories (Continued)

Directory name	Description
lvl_details/layer_list1/	Automatically generated layer list for the baseline library if a layer list is not present. Summary information of layer list creation is included for diagnostic purposes.
lvl_details/layer_list2/	Automatically generated layer list for the check library if a layer list is not present. Summary information of layer list creation is included for diagnostic purposes.

Use Models

This section shows two simple use models of the LVL utility.

- The following use model compares the baseline library with the check library using two NOT operations. All of the layers in *library1* are compared with all of the layers in *library2*.

  ```
  icv_lvl library1 library2 -c cell -not
  ```

- The following use model generates a layer file from the input library using *cell* as the top cell. The output file contains an assign statement for each layer/datatype found in the input library under the *cell* subtree.

  ```
  icv_lvl library -af output_file -c cell
  ```

Feedback

QuickLVL Utility

This section describes two use models in the Quick LVL utility allow you to quickly read two layouts and compare the layout elements cell by cell.

- Cell-level Quick LVL: Standalone run of Quick LVL.
- Auto-Incremental DRC: The IC Validator tool calls Quick LVL first to determine differences with respect to the baseline. Results are passed to the tool in an IC Validator readable internal format without user intervention.

The Quick LVL utility supports input in the following layout formats:

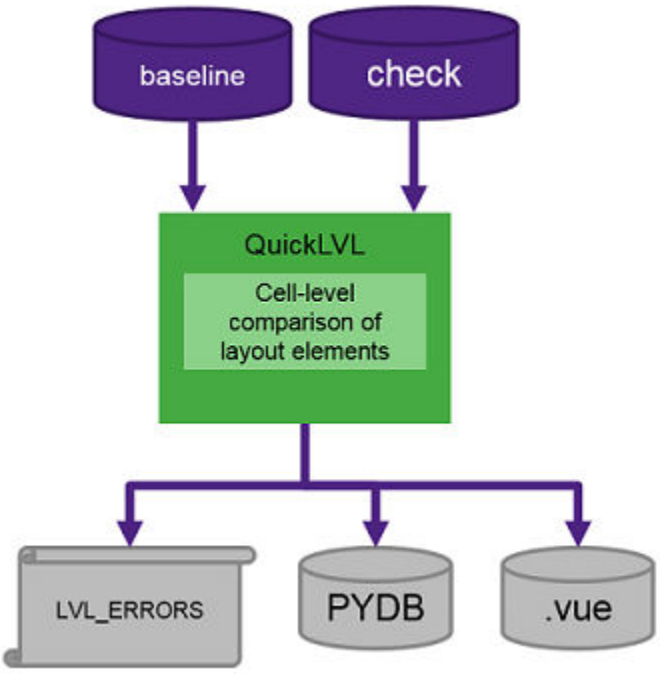
- OASIS
- GDSII

Note:

The Quick LVL utility supports text with any text character, except the NULL character.

Figure 141 shows the Quick LVL utility.

Figure 141 Quick LVL Flowchart



Feedback

Cell-Level Quick LVL

The Quick LVL utility reads two layouts and compares layout elements on a cell-by-cell basis. The results are stored in the error database, PYDB. This mode produces output files in the LVL_ERRORS and VUE formats.

The command-line syntax is

```
icv_lvl baseline_layout check_layout -c top_cell [other options] -quick
```

Table 73 describes the command-line options for the QuickLVL utility.

Differences Between LVL and QuickLVL

Cell-level Quick LVL provides a list of differences between the `baseline_layout` and `check_layout` library names. This differs from traditional full-layout-LVL in the following ways:

- The results are reported at the cell level.
- Element-wise differences are reported; not geometrical differences. For example, if a rectangle is bigger in one layout, both rectangle elements are reported entirely, not their geometrical difference.
- If a transformation is defined on the command line, the tool transforms the coordinates in the checked layout's top cell before trying to match the baseline layout.
- If an element is represented in different formats (for example, path versus rectangle or AREF versus a set of SREFs) in two layouts, they are not matched, even if they geometrically represent identical shapes. These elements are reported as “missing” and “new” elements.
- QuickLVL internally tries to match elements between layouts. Elements with certain similarities are reported as “different” between the baseline and the check. Other elements are reported as missing in the check layout or as new shapes in the baseline layout.
- It is possible to have some false-positive results. This is traded for faster speed.
- QuickLVL works only with a single host.

This tool provides the following output from the LVL_ERRORS file:

```
Path Mismatch (5:0)
  Baseline shapes different in Check ............. 25 violations found.

Placement Mismatch
  Baseline shapes missing in Check ............... 20 violations found.
  New shapes in Check not in Baseline ............ 150 violations found.
```

Feedback

```
Path Mismatch (10:0)
  New shapes in Check not in Baseline ..... 75 violations found.
```

Auto-Incremental DRC Focused on Differing Layers and Regions

With the `-idrc_i` command-line option, the IC Validator tool first runs cell-level Quick LVL to compare the current layout to the baseline layout. Differences between these layouts are used to improve IC Validator runtime by focusing on layers and windows that contain differences. The window ambit can be adjusted by setting `incremental_options(window_ambit = …);`.

Using options of the `incremental_options()` function other than the `window_ambit` option is not recommended in the Auto-Incremental DRC flow. The Auto-Incremental DRC flow requires the baseline layout and checked layout to have the same top-cell name. Results from an Auto-Incremental DRC run might not be a complete replacement for a full signoff run.

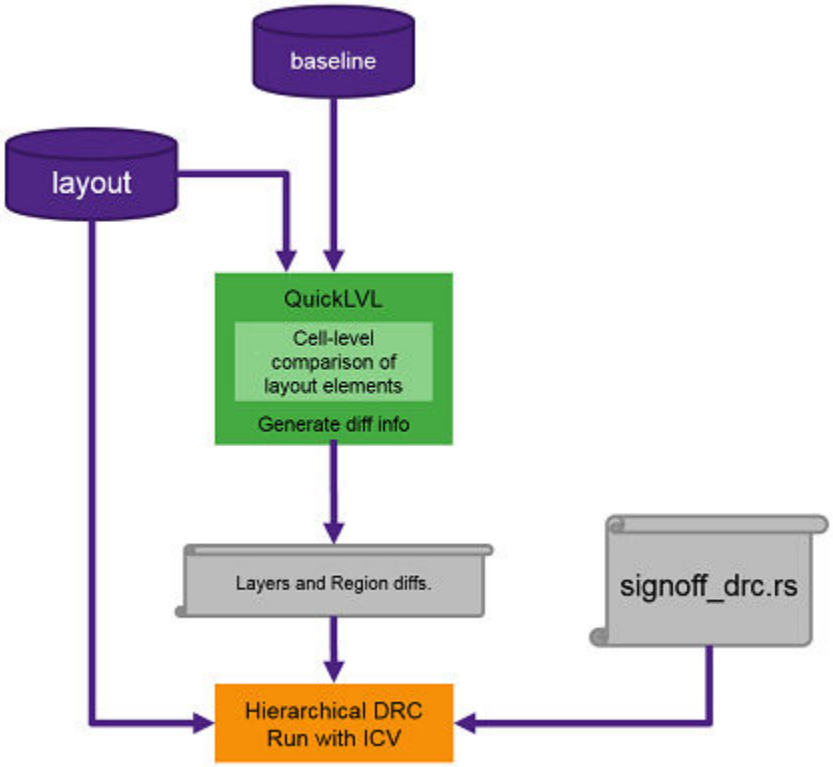
The Auto-Incremental DRC flow supports the baseline layout in the following layout formats:

- OASIS
- GDSII

Figure 142 shows the Auto-Incremental DRC Flow.

Feedback

Figure 142 Auto-Incremental DRC Flow



C

Layout Integrity Management

This appendix describes the layout integrity management feature that is available with the IC Validator tool.

The layout integrity management feature is described in the following sections:

- Using Layout Integrity Management
- Layout Integrity Management Examples
- The icv_lidb Utility
- The icv_lidb_report Utility

Using Layout Integrity Management

Layout integrity management allows you to verify the integrity of golden intellectual property (IP), standard cell layout, or any other layout cell. It provides a convenient way to verify that certain layout cells have not changed without doing a full layout-versus-layout comparison. The icv_lidb utility creates a layout integrity database (LIDB) from an input layout. The resultant LIDB can then be used in an IC Validator run to check layout cells as the design is read. The icv_lidb_report utility views the content of an LIDB.

Note:

Although false matches are extremely unlikely, this flow is not a substitute for a full verification run before tapeout.

For each cell, the LIDB contains:

- One checksum per layer of all geometric data and text on the layer.
- One checksum per unique child cell name of all references to the child cell.

Each cell in the LIDB can have multiple sets of checksums. The design cell is considered a match if it matches any set of checksums for that cell stored in the LIDB. This matching allows a single LIDB to serve equivalent cells with different checksums. For example, different layout formats can generate slightly different checksums, or a tool in the flow can change the data, such as by merging adjacent or overlapping shapes. In these cases, a

Feedback

new set of checksums can be merged into an existing LIDB so that any version of the cell passes the layout integrity check.

There are three ways that cells in a design can be checked for layout integrity:

- Use the icv_lidb utility to validate an input layout against an existing LIDB without doing a full IC Validator run. The utility matches layout cells to a set of checksums in the LIDB by cell name.
- Use the `layout_integrity_by_cell()` function in an IC Validator runset to check cells by name. A design cell is considered a match if its calculated checksums match any set of checksums for a cell of the same name in any input LIDB specified in the `layout_integrity_options()` function.
- Use the `layout_integrity_by_marker_layer()` function in an IC Validator runset to check cells by marker layer. A design cell is considered a match if its calculated checksums match any set of checksums for a cell with data present on the marker layer in any input LIDB specified in the `layout_integrity_options()` function.

Layout integrity mismatches found during an IC Validator run are stored in the error database, are reported in the LAYOUT_ERRORS file, and are visible in VUE. There are no specific coordinates associated with the errors, so there is no highlighting of these errors in VUE.

The results of layout integrity checking can be used to control the action taken on DRC errors subsequently found in those cells. The errors can be discarded, reported as normal, or classified using DRC classification.

Layout Integrity Management Examples

The following section provides several examples of the layout integrity management feature:

- Basic Example
- Running on a Different Layout Format
- Reporting of Mismatches From a Merged LIDB
- Marker Layer Check
- Mixed Milkyway and GDSII Flow

Basic Example

The following example shows basic layout integrity checking by cell name. First, create an LIDB from the input layout.

Feedback

```
icv_lidb -i EX_ADDER_3.GDS -o orig.lidb
```

Next, add the following to your runset, and the IC Validator tool runs as planned.

```
layout_integrity_options(
  databases = {
    { db_name = "orig.lidb" }
  }
);

{
  @ "Layout Integrity Check";
  layout_integrity_by_cell(
    cells = {"*"}
  );
}
```

For this run, no layout integrity mismatches are reported in the LAYOUT_ERRORS file because you are running from the same input layout from which the LIDB is generated. However, you can see that the layout integrity checking is performed by examining the run_details/lidb.log file.

```
--------------------------------------------------------------------------------
                       Layout Integrity Processing Log
                            11/08/2010 08:12:50
--------------------------------------------------------------------------------

Processing layout cell VIA
  Cell check:
    Result       = Matched
    Matched Cell = VIA
    Database     = /home/user/orig.lidb

Processing layout cell INV
  Cell check:
    Result       = Matched
    Matched Cell = INV
    Database     = /home/user/orig.lidb
...
```

Also, the layout integrity status of each cell that was checked can be seen in the run_details/hierarchy/*cell*.tree0 file.

```
CELL = AD4FUL
        Cell Status             = USED
        GDSII Library           = MAINLIB
        Layout Integrity Check  = Passed Completely
```

Feedback

There are three possible values for the layout integrity check status in the *cell*.tree0 file.

- Passed Completely. The cell and all cells placed underneath it were checked and found to be a match.
- Passed. The cell itself passed the layout integrity check, but at least one child cell was not checked or failed the check.
- Failed. No layout integrity match was found for the cell.

Running on a Different Layout Format

The following example shows the results of running on a different layout format of the same design. Using a different layout format can result in different checksum data. Common reasons for this difference are:

- Some formats, such as Milkyway, can have a cell boundary layer while others, such as GDSII, do not.
- Some formats represent cell placements differently. For example, OASIS does not directly support the concept of an AREF. Instead arrayed cell placements are represented as SREFs with a repetition placing the cell multiple times. The IC Validator tool, however, might interpret these cell placements as AREFs internally.
- There are several ways in which geometric data can differ between formats while remaining physically equivalent. Point ordering, merging of overlapping shapes, and collinear point removal are examples of how geometric data can differ. The layout integrity management feature checks the original input layout as is, without attempting to transform it in any way.

When the IC Validator run from the previous example is given an equivalent Milkyway input layout, layout integrity mismatches because of the cell boundary layer are reported.

```
...

 Layout Integrity Check
   layout_integrity_by_cell:layer_mismatch ....... 9 violations found.
...

runset1.rs:17:layout_integrity_by_cell:layer_mismatch
- - - - - - - - - - - - - - - - - - - - - - - -
Structure Layer   LIDB      Golden Layout
- - - - - - - - - - - - - - - - - - - - - - - -
AD4FUL    (255;0) orig.lidb
VIA       (255;0) orig.lidb
ADFULAH   (255;0) orig.lidb
POLYHD    (255;0) orig.lidb
INV       (255;0) orig.lidb
INVH      (255;0) orig.lidb
DGATE     (255;0) orig.lidb
```

Feedback

```
CONT      (255;0) orig.lidb
TGATE     (255;0) orig.lidb
```

When running on an equivalent OASIS layout, layout integrity mismatches because of cell placements being interpreted as AREFs are reported.

```
...
 Layout Integrity Check
   layout_integrity_by_cell:placement_mismatch .... 6 violations found.
...

runset1.rs:17:layout_integrity_by_cell:placement_mismatch
- - - - - - - - - - - - - - - - - - - - - - - -
Structure Child Cell LIDB      Golden Layout
- - - - - - - - - - - - - - - - - - - - - - - -
AD4FUL    ADFULAH    orig.lidb
AD4FUL    VIA        orig.lidb
ADFULAH   INV        orig.lidb
ADFULAH   INVH       orig.lidb
DGATE     TGATE      orig.lidb
DGATE     VIA        orig.lidb
```

Because these three layouts are equivalent, they can be merged into a single LIDB. The LIDB passes on all three formats.

```
icv_lidb -i EX_ADDER_3 EX_ADDER_3.GDS EX_ADDER_3.oas -o merged.lidb
```

Reporting of Mismatches From a Merged LIDB

Because a single cell can have multiple sets of checksums, some cells checksums can be matched but not others. A cell is considered a match only if all layer and placement checksums are a match for one set of checksums in the LIDB. The following example shows the differences between layout integrity error reporting in different scenarios in a merged LIDB.

```
validate_layout.rs:14:layout_integrity_by_cell:layer_mismatch
- - - - - - - - - - - - - - - - - - - - - - - -
Structure Layer LIDB        Golden Layout
- - - - - - - - - - - - - - - - - - - - - - - -
TOP       (1;0) merged.lidb 2.gds
TOP       (2;0) merged.lidb 1.gds
TOP       (3;0) merged.lidb
```

In the preceding example, there is no complete match found for the cell TOP. Layer 1 matches cell TOP in the layout 1.gds, but does not match 2.gds, so the mismatch is reported against the golden layout 2.gds. Layer 2 matches cell TOP in 2.gds, but does not match 1.gds, so the mismatch is reported against the golden layout 1.gds. Layer 3 does not match any golden layout, so that column is left blank.

Cell placement mismatches for the same scenario are shown in the following example.

Feedback

```
validate_layout.rs:15:layout_integrity_by_cell:placement_mismatch
- - - - - - - - - - - - - - - - - - - - - - - - - - -
Structure Child Cell LIDB        Golden Layout
- - - - - - - - - - - - - - - - - - - - - - - - - - -
TOP       A          merged.lidb LIB2
TOP       B          merged.lidb LIB1
TOP       C          merged.lidb
```

In the preceding example, no complete match found for the cell TOP. Placements of child cell A match the placements in cell TOP of LIB1, but not LIB2, so the mismatch is reported against LIB2. Placements of child cell B match the placements in cell TOP of LIB2 but not LIB1, so the mismatch is reported against LIB1. Placements of child cell C do not match any golden layout, so that column is left blank.

Hierarchical Mode

The IC Validator tool has two modes of operation for layout integrity checking:

- **Cell level**. Each cell is considered individually without regard to whether any children in the hierarchy passed or failed the check. This is the default mode.
- **Hierarchical**. The status of child cells in the hierarchy is considered. Any layout integrity mismatch of a child cell is reported also as a placement mismatch in the parent cell. This reporting continues all the way up the hierarchy.

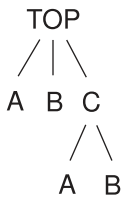
Mismatches are only reported in cells specified in the check function:

- `layout_integrity_by_cell()`. The cells are specified by the `cells` argument.
- `layout_integrity_by_marker_layer()`. The cells are those that contain a polygon on the layer specified by the `marker_layer` argument.

Hierarchical Cell Check Example

This example shows behavior of checks for the `layout_integrity_by_cell()` function with hierarchical mode. Consider the design hierarchy shown in Figure 143.

Figure 143 Design Hierarchy for Hierarchical Cell Check Example



Feedback

In this example, there is a difference in a layer shape in cell A from the checksum in the layout integrity database.

Note:

Although the example uses placed cells with matching cell names, this is not required in hierarchical mode. Cells placed under the check cell must have the same content but not the same cell name.

The following code checks all cells in hierarchical mode:

```
layout_integrity_by_cell(
   cells = { "*" },
   processing_mode = HIERARCHICAL
);
```

The difference is reported in A, C, and TOP. The output in the LAYOUT_ERRORS file is:

```
test.rs:15:layout_integrity_by_cell:layer_mismatch
- - - - - - - - - - - - - - - - - - - - -
Structure Layer  LIDB       Golden Layout
- - - - - - - - - - - - - - - - - - - - -
A         (31;0) test.lidb

test.rs:15:layout_integrity_by_cell:placement_mismatch
- - - - - - - - - - - - - - - - - - - - - - -
Structure Child Cell LIDB      Golden Layout
- - - - - - - - - - - - - - - - - - - - - - -
TOP       A          test.lidb
TOP       C          test.lidb
C         A          test.lidb
```

The following code checks cells C and TOP, but not cell A:

```
layout_integrity_by_cell(
   cells = { "*", "!A" },
   processing_mode = HIERARCHICAL
);
```

The difference is still reported in C and TOP. The hierarchical mode specifies that everything below a particular cell being checked is also be considered. The output is:

```
test.rs:15:layout_integrity_by_cell:placement_mismatch
- - - - - - - - - - - - - - - - - - - - - - -
Structure Child Cell LIDB      Golden Layout
- - - - - - - - - - - - - - - - - - - - - - -
TOP       A          test.lidb
TOP       C          test.lidb
C         A          test.lidb
```

The run_details/lidb.log file contains additional details that indicate the status of each cell in the hierarchy. In this example, cell A is processed, but there is no "Cell check" entry because cell A was not specified in the `layout_integrity_by_cell()` function call.

Feedback

```
------------------------------------------------------------------------
                      Layout Integrity Processing Log
                           04/23/2014 06:13:36
------------------------------------------------------------------------
Processing layout cell A
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found

Processing layout cell B
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
  Cell check (hierarchical):
    Result       = Matched
    Matched Cell = B
    Database     = test.lidb

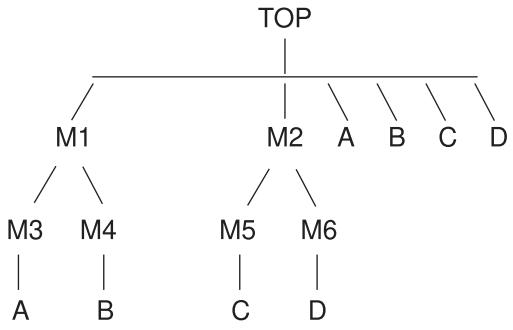
Processing layout cell C
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
  Cell check (hierarchical):
    Result       = No Match Found

Processing layout cell TOP
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
  Cell check (hierarchical):
    Result       = No Match Found
```

Hierarchical Marker Layer Check Example

This example shows behavior of checks for the `layout_integrity_by_marker_layer()` function with hierarchical mode. Consider the design hierarchy shown in Figure 144.

Figure 144 Design Hierarchy for Hierarchical Layer Check Example



In this example, only cells M1-M6 have data on the marker layer. Cell B has a difference on layer 3. Cell M5 has a difference on layer 4.

Feedback

Note:

Although the example uses placed cells with matching cell names, this is not required in hierarchical mode. Cells placed under the check cell must have the same content but not the same cell name.

The following shows several hierarchical mode checks:

```
// This reports mismatches in M1, M2, M4, and M5
layout_integrity_by_marker_layer(
  marker_layer = {1},
  processing_mode = HIERARCHICAL
);

// This check only reports mismatches in M2 and M5 since the only
// layer with a difference being checked is layer 4
layout_integrity_by_marker_layer(
  marker_layer = {1},
  check_layers = { {2}, {4} },
  processing_mode = HIERARCHICAL
);

// This check only reports mismatches in M1 and M4 since the only
// layer with a difference being checked is layer 3
layout_integrity_by_marker_layer(
  marker_layer = {1},
  check_layers = { {3} },
  processing_mode = HIERARCHICAL
);

// This check reports no mismatches, since no layers being checked
// contain a difference
layout_integrity_by_marker_layer(
  marker_layer = {1},
  check_layers = { {2} },
  processing_mode = HIERARCHICAL
);
```

The mismatches are reported in the LAYOUT_ERRORS file:

```
test2.rs:17:layout_integrity_by_marker_layer:cell_mismatch
- - - - - - - - - - -
Structure Marker Layer
- - - - - - - - - - -
M1        (1;0)
M2        (1;0)
M4        (1;0)
M5        (1;0)

test2.rs:22:layout_integrity_by_marker_layer:cell_mismatch
- - - - - - - - - - -
Structure Marker Layer
- - - - - - - - - - -
```

Feedback

```
M2          (1;0)
M5          (1;0)

test2.rs:28:layout_integrity_by_marker_layer:cell_mismatch
- - - - - - - - - - -
Structure Marker Layer
- - - - - - - - - - -
M1          (1;0)
M4          (1;0)
```

Details on the status of each cell are reported in the run_details/lidb.log file:

```
Processing layout cell C
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = No Data on Marker Layer

Processing layout cell A
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = No Data on Marker Layer

Processing layout cell B
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
    Layers (3;0): No hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = No Data on Marker Layer

Processing layout cell D
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = No Data on Marker Layer

Processing layout cell M6
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
```

```
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (1;0)
    Matched Cell = M6
    Database     = test.lidb

Processing layout cell M5
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): No hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (1;0)
    Matched Cell = M5
    Database     = test.lidb

Processing layout cell M4
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
    Layers (3;0): No hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (1;0)
    Matched Cell = M4
    Database     = test.lidb

Processing layout cell M3
  Database: test.lidb
    All Layers: Hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (1;0)
    Matched Cell = M3
    Database     = test.lidb

Processing layout cell M2M
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
    Layers (3;0): Hierarchically equivalent cells found
    Layers (4;0) (2;0): No hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (1;0)
    Matched Cell = M2
```

Feedback

```
    Database     = test.lidb

Processing layout cell M1
  Database: test.lidb
    All Layers: No hierarchically equivalent cells found
    Layers (3;0): No hierarchically equivalent cells found
    Layers (4;0) (2;0): Hierarchically equivalent cells found
    Layers (2;0): Hierarchically equivalent cells found
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (1;0)
    Matched Cell = M1
    Database     = test.lidb

Processing layout cell TOP
  Marker Layer Check:
    Result       = No Data on Marker Layer
```

Marker Layer Check

The `layout_integrity_by_marker_layer()` function does not check cells by name, but rather checks any cell with data present on the specified marker layer. It looks for any matching set of checksums for any cell in the input LIDB that also contains data on the marker layer.

```
{
  @ "Layout Integrity Check";
  layout_integrity_by_marker_layer(
    marker_layer = { 8,0 }
  );
}
```

This example checks cells with data on layer 8 as the marker layer. The following example shows a matching cell and a mismatched cell in the run_details/lidb.log file.

```
--------------------------------------------------------------------------
                       Layout Integrity Processing Log
                            11/08/2010 09:45:33
--------------------------------------------------------------------------

Processing layout cell VIA
  Marker Layer Check:
    Result       = No Match Found
    Marker Layer = (8;0)

Processing layout cell INV
  Marker Layer Check:
    Result       = Matched
    Marker Layer = (8;0)
    Matched Cell = INV
    Database     = /home/user/orig.lidb
```

Feedback

You can see from the *cell*.LAYOUT_ERRORS file that mismatches for the marker layer check are reported on a pass or fail basis. No individual layer or cell reference mismatches are reported.

```
...

 Layout Integrity Check
   layout_integrity_by_marker_layer:cell_mismatch ... 3 violations found.
...

runset8.rs:17:layout_integrity_by_marker_layer:cell_mismatch
- - - - - - - - - - -
Structure Marker Layer
- - - - - - - - - - -
AD4FUL    (8;0)
VIA       (8;0)
TGATE     (8;0)
```

If you want to check only specific layers in the marked cells, use the `check_layers` argument.

```
{
  @ "Layout Integrity Check";
  layout_integrity_by_marker_layer(
    marker_layer = { 8,0 },
    check_layers = { {==6}, {==8}, {==10} }
  );
}
```

In the preceding example, if a cell matches on layers 6, 8, and 10, it is considered a complete match.

Mixed Milkyway and GDSII Flow

For a mixed Milkyway and GDSII flow, the `layout_integrity_options()` function specifies the LIDB to be used for checking Milkyway cells. For layout integrity checking to be performed on replacement GDSII or OASIS files, the LIDB must be specified for each replacement library in the `milkyway_merge_library_options()` function.

```
library(
  library_name = "EX_ADDER_3",
  format = MILKYWAY,
  cell = "AD4FUL"
);

milkyway_merge_library_options(
  libraries = {
    {
      library_name = "EX_ADDER_3",
      replacement_libraries = {
        {
```

Feedback

```
            file = "EX_ADDER_3.GDS",
            format = GDSII,
            layout_integrity = {
              { db_name = "gds.lidb" }
            }
          }
        }
      }
    }
  }
);

layout_integrity_options(
  databases = {
    { db_name = "milkyway.lidb" }
        }
);

{
  @ "Layout Integrity Check";
  layout_integrity_by_cell(
    cells = {"*"}
  );
}
```

After the IC Validator run, the run_details/lidb.log file shows that the Milkyway cell is checked against the milkyway.lidb file, while the GDSII cells are checked against the gds.lidb file.

```
--------------------------------------------------------------------------------
                        Layout Integrity Processing Log
                             11/08/2010 09:56:43
--------------------------------------------------------------------------------

Processing layout cell AD4FUL
  Cell check:
    Result       = Matched
    Matched Cell = AD4FUL
    Database     = /home/user/milkyway.lidb

Processing layout cell VIA
  Cell check:
    Result       = Matched
    Matched Cell = VIA
    Database     = /home/user/gds.lidb
```

Feedback

The icv_lidb Utility

The icv_lidb utility manages LIDBs. Use it to:

- Create an LIDB from an input layout.
- Merge multiple LIDBs.
- Quickly validate an input layout using an existing LIDB.

Command-Line Options

The icv_lidb utility command-line syntax supports its three simple use models.

- To generate an LIDB from an input layout:

  ```
  icv_lidb -i input_layout -o output_lidb [-c top_cell]
  ```

- To validate an input layout using an existing LIDB:

  ```
  icv_lidb -i input_layout -validate golden_lidb [-c top_cell]
  ```

- To merge multiple LIDBs together:

  ```
  icv_lidb -merge lidb1 lidb2... -o output_lidb
  ```

Table 78 describes the command-line options of the utility.

Table 78 icv_lidb Utility Command-Line Options

Syntax	Description
`-c cell`	The top cell of the input layout. If omitted, all cells in the library are processed.
`-elpc num` `-error_limit_per_check num`	Sets the error limit for each type of check. The default is 100. **Note:** Setting this command-line option to `ERROR_LIMIT_MAX` causes all errors up to a tool-defined maximum limit to be stored. The limit can be set above the tool-defined maximum limit by giving a specific number. However, setting the limit above the tool-defined maximum might result in decreased performance and increased disk usage for large numbers of errors, and therefore is not recommended. Setting it to `0` suppresses storage of all errors.
`-help`	Displays a list of the command-line options and exits.

Feedback

Table 78 icv_lidb Utility Command-Line Options (Continued)

Syntax	Description
`-i input_layout ... input_layout`	Input layout name when creating an LIDB or validating a layout against an existing LIDB. If creating an LIDB, you can specify multiple files to incorporate checksums from all layouts into the output LIDB. The following examples show how to specify input layouts for various formats: • GDSII: `icv_lidb -i /path/to/file.gds` • OASIS: `icv_lidb -i /path/to/file.oas` • Milkyway: `icv_lidb -i /library/path/LIBRARY_NAME` • OpenAccess: `icv_lidb -i /path/to/lib.defs/LIBRARY_NAME`
`-lf filename`	Selects a subset of all layers and object types (polygons, edges, and text) to be written to the layout integrity database. For example, edges on layer1 are written only to the layout integrity database only if the layer file contains an `assign_edge()` function that includes layer1. The layer file is stored in the layout integrity database so that any validation runs against this database are restricted to the layers and object types found in the database of the layer file. The layer file must not contain options functions or runset functions. If the file is empty or does not contain assign statements, this command-line option is ignored and the IC Validator tool behaves as if it is not present.
`-merge lidb ... lidb`	Merges the specified input LIDB into the output database.
`-o output_lidb`	The output layout integrity database (LIDB) that is generated. If the file exists, it is overwritten.
`-oa_layer_map layer_mapping_file`	Optional OpenAccess layer mapping file for OpenAccess databases. Data on any layers not present in the mapping file is not read. See the `layer_mapping_file` argument of the `openaccess_options()` function for information about the mapping file.
`-oa_view view_name`	Optional view of the top cell for OpenAccess databases.
`-of options_file`	Optional file containing an IC Validator runset OPTIONS section. This file can contain any of the OPTIONS section functions, including the `gds_options()`, `milkyway_options()`, and `oasis_options()` functions.
`-v`	Displays the utility version and exits.
`-validate golden_lidb`	Validates the input layout by performing layout integrity checking against the specified golden LIDB. Results are in the *cell*.LAYOUT_ERRORS file.

Feedback

Exit Status

The exit status values of the `icv_lidb` executable are shown in Table 79.

Table 79 icv_lidb Utility Exit Status Values

Exit status value	Definition
0	Ran successfully. If validating an input layout, no differences were found.
1	Ran successfully and found differences.
negative number	An error occurred.

Output Files

The output files of the icv_lidb utility are shown in Table 80.

Table 80 icv_lidb Utility Output Files

File name	Description
lidb_details/icv_lidb.sum	Summary file containing information about the run. All output from the screen is captured in this file. It contains • The command-line options used for the run. • Information about input and output files. • Runtime and memory used. • Success or failure indication.
lidb_details/icv_lidb.log	A log of all cells processed for layout validation or database generation runs. For layout validation runs, this file might indicate whether a design cell was a match for a golden LIDB cell. It might also contain details on which golden LIDB cell the design cell matches.
cell.LAYOUT_ERRORS	Error file containing all layout integrity errors found during a layout validation run. If no top cell is given, the file is named *cell*.LAYOUT_ERRORS.

The icv_lidb_report Utility

The icv_lidb_report utility reports the content of an LIDB. For each set of checksums contained in the database, this utility shows the golden layout path and the user ID and timestamp, and lists the cells with entries. Output is sent to the screen as well as the icv_lidb_report.sum summary file in the current directory.

Feedback

Command-Line Options

The icv_lidb_report utility command-line syntax is:

```
icv_lidb_report -i input_lidb
```

Table 81 describes the command-line options of the utility.

Table 81 icv_lidb_report Utility Command-Line Options

Syntax	Description
`-help`	Displays a list of the command-line options and exits.
`-i input_lidb`	Input layout integrity database to report.
`-v`	Displays the utility version and exits.

Output Files

The output files of the icv_lidb_report utility are shown in Table 82.

Table 82 icv_lidb_report Utility Output Files

File name	Description
icv_lidb_report.sum	Summary file containing information about the run. All output from the screen is captured in this file. It contains • The command-line options used for the run. • Information about input and output files.

D

IC Validator PXL Debugger

The IC Validator PXL Debugger assists you in constructing a runset or part of a runset by allowing you to look at the intermediate stages in each of the design rules or device configuration functions you are writing.

The PXL Debugger is described in the following sections:

- Running the PXL Debugger
- PXL Debugger Commands

Feedback

Running the PXL Debugger

To run the PXL Debugger use the `-debug` command-line option with the IC Validator command:

```
% icv -debug runset.rs [-debug_source file]
```

The `-debug_source` command-line option allows you to run the PXL Debugger in batch mode. Specify a file written with the `record` command in a previous debug run. The output of the PXL Debugger displays on screen.

When runset execution starts,

- A message on screen indicates the start of the PXL Debugger.
- The run pauses at the first breakpoint. Five lines before and four lines after the executable line are displayed.
- The PXL Debugger prompt (ICVD>>) displays.

For example,

```
Starting ICV Debugger
      13 |           t = t + dtoi((x * y**2) / z);
      14 |       }
      15 |       t = dtoi(t / x);
      16 |       note (" f2(3) = "+ t);
      17 | }
  --> 18 | xx:integer;
      19 | f2(3);
      20 | note("end. ");
ICVD>>
```

At the prompt enter the action you want the PXL Debugger to take. See the following PXL Debugger Commands section.

Note:

The PXL Debugger is designed to execute PXL code sequentially. If distributed processing options are enabled, a warning is generated and the options are ignored.

The PXL Debugger steps through code in runset and user functions, but not in remote functions.

PXL Debugger Commands

The IC Validator PXL Debugger has the commands shown in Table 83.

Feedback

Table 83 PXL Debugger Commands

Commands[11]	Definition
`h[elp]`	Print a listing of the PXL Debugger commands.
`s[tep]`	Step into the function call. See the `step` command description.
`n[ext]`	Function is executed all at once. See the `next` command description.
`q[uit]`	Quit the PXL Debugger and the IC Validator run. See the `quit` command description.
`c[ont]`	Continue until the next breakpoint or the end of run. See the `cont` command description.
`fin[ish]`	Execute until the current function returns. See the `finish` command description.

`b[reak]`
`b[reak]`
`[file:]line`
`b[reak] function`
	Set a breakpoint. The default is at the current file:line. See the `break` command description.

`info b[reak]` `[number]`	Show the breakpoint status. The default is all breakpoints. See the `info break` command description.
`dis[able]` `[number]`	Disable the breakpoint. The default is disable all breakpoints. See the `disable` command description.
`en[able] [number]`	Enable the breakpoint. The default is enable all breakpoints. See the `enable` command description.
`d[elete] number`	Delete the specified breakpoint. See the `delete` command description.

`l[ist]` `l[ist]`
`[file:]line` `l[ist]`
`function`
	List the source. The default is the current file and line. See the `list` command description.

`where`	Show the function call stack. See the `where` command description.

`pwd`	Prints the current working directory of the IC Validator PXL Debugger. For example:
	`ICVD>> pwd`
	`Working directory /remote/data/ICV`

`p[rint]` `expression`	Print the value of the expression. See the `print` command description.
`record [file]`	Save the commands from the current debug session. The default is ICV*pid*.pdi. See the `record` command description.

11. The commands can be abbreviated as shown with the brackets.

Feedback

Table 83 PXL Debugger Commands (Continued)

Commands[11]	Definition
`source file`	Execute commands from the file. See the `source` command description.

11. The commands can be abbreviated as shown with the brackets.

Feedback

break

The `break` command sets breakpoints. There are four ways to set breakpoints:

1. `break`

   When specified without any options, `break` sets a breakpoint at the current line in the current file.

```
       4 |
       5 | f2 : function (
       6 |     x : integer
       7 |     ) returning ans : integer
       8 | {
  -->  9 |      ans = x;
      10 |      note("0. ans = " + ans);
      11 | }
      12 |
      13 | xx: integer = 0;
ICVD>>  b
Breakpoint #1,  f2() at test.rs:9
ICVD>>
```

2. `break line`

   Sets a breakpoint at a specific line number in the current file.

```
ICVD>>  b 13
Breakpoint #1,  f2() at test.rs:13
ICVD>>
```

3. `break file:line`

   Sets a breakpoint at the specified line number in the specified file.

```
ICVD>>  b test.rs:15
Breakpoint #2,  _MAIN_() at     test.rs:15
ICVD>>
```

4. `break function`

   Sets the breakpoint in the first executable line within the specified function.

```
       8 | {
       9 |      ans = x;
      10 |      note("0. ans = " + ans);
      11 | }
      12 |
  --> 13 | xx: integer = 0;
      14 | xx = f2(3);
      15 | note("1. f2(3)= " + xx);
ICVD>>  b f2
```

Feedback

```
Breakpoint #1,  f2() at test.rs:9
ICVD>>
```

This method allows setting breakpoints on overloaded functions. In this example, f2 is overloaded as it can return either an integer or void. Specifying 2 selected option 2; f2 returns an integer.

```
ICVD>>  b f2
[0] cancel
[1] all
[2] f2() returning integer, at test.rs:9
[3] f2() returning void, at test.rs:16
> 2
```

In this example, specifying 1 selected option 1; f2 returns an integer and void.

```
Breakpoint #1,  f2() at test.rs:9
ICVD>>  b f2
[0] cancel
[1] all
[2] f2() returning integer, at test.rs:9
[3] f2() returning void, at test.rs:16
> 1
Breakpoint #2,  f2() at test.rs:9
Breakpoint #3,  f2() at test.rs:16
ICVD>>
```

Feedback

cont

The `cont` command executes the runset to completion or stops at a specified breakpoint.

In this example, the runset completed the debugger run.

```
       1 | #include <math.rh>
       2 | #include <diagnostics.rh>
       3 |
  -->  4 | f: list of integer = {1, 1};
       5 |
       6 | for (i in 2 thru 19) {
       7 |         f.push_back(f[i-2] + f[i-1]);
       8 | }
ICVD>>  c

Completed ICV Debugger Run
ICVD>>
```

In this example, execution stops at a breakpoint that is in a loop. (See the `break` command description for information about the `break` command.)

```
ICVD>>  b 13
Breakpoint #1,  f2() at test.rs:13
ICVD>>  c
       8 | {
       9 |       t : integer = 0;
      10 |       y : integer = 10;
      11 |       z : integer = 100;
      12 |       for ( i=1 to x) {
B --> 13 |           t = t + dtoi((x * y**2) / z);
      14 |       }
      15 |       t = dtoi(t / x);
      16 |       note (" f2(3) = "+ t);
      17 | }
Breakpoint #1,  f2() at test.rs:13
ICVD>>  c
       8 | {
       9 |       t : integer = 0;
      10 |       y : integer = 10;
      11 |       z : integer = 100;
      12 |       for ( i=1 to x) {
B --> 13 |           t = t + dtoi((x * y**2) / z);
      14 |       }
      15 |       t = dtoi(t / x);
      16 |       note (" f2(3) = "+ t);
      17 | }
Breakpoint #1,  f2() at test.rs:13
ICVD>>
```

Feedback

delete

The `delete` command deletes the specified breakpoint. The command format is

```
delete number
```

```
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Enabled
Breakpoint #2,  _MAIN_() at     test.rs:16    Enabled
ICVD>>  d 1
ICVD>>  info b
Breakpoint #2,  _MAIN_() at     test.rs:16    Enabled
ICVD>>
```

Feedback

disable

The `disable` command disables the specified breakpoint. There are two ways to disable breakpoints:

1. `disable`

   Using the command without a breakpoint number disables all breakpoints.

```
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Enabled
Breakpoint #2,  _MAIN_() at     test.rs:18    Enabled
ICVD>>  dis
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Disabled
Breakpoint #2,  _MAIN_() at     test.rs:18    Disabled
```

2. `disable` *`number`*

   If a breakpoint number is specified, only that breakpoint is disabled.

```
ICVD>>  dis 1
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:13    Disabled
Breakpoint #2,  _MAIN_() at     test.rs:16    Enabled
ICVD>>
```

Feedback

enable

The `enable` command enables a previously disabled breakpoint. There are two ways to enable breakpoints:

1. `enable`

   Using the command without a breakpoint number enables all breakpoints.

```
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Disabled
Breakpoint #2,  _MAIN_() at     test.rs:18    Disabled
ICVD>>  en
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Enabled
Breakpoint #2,  _MAIN_() at     test.rs:18    Enabled
ICVD>>
```

2. `enable` *`number`*

   If a breakpoint number is specified, only that breakpoint is disabled.

```
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Disabled
ICVD>>  en 1
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Enabled
ICVD>>
```

Feedback

finish

The `finish` command executes to the end of the function it is in. Execution does stop, however, at breakpoints.

In this example, the run continues until the current function is completed.

```
       8 | {
       9 |       t : integer = 0;
      10 |       y : integer = 10;
      11 |       z : integer = 100;
      12 |       for ( i=1 to x) {
  --> 13 |           t = t + dtoi((x * y**2) / z);
      14 |       }
      15 |       t = dtoi(t / x);
      16 |       note (" f2(3) = "+ t);
      17 | }
ICVD>>  fin
test.rs:16  f2(3) = 3
      15 |       t = dtoi(t / x);
      16 |       note (" f2(3) = "+ t);
      17 | }
      18 | xx:integer;
      19 | f2(3);
  --> 20 | note("end. ");
```

In this example, the run continues to until a breakpoint is encountered.

```
ICVD>>  fin
test.rs:11 fibonacci(16) = 1597
       6 | for (i in 2 thru 19) {
       7 |       f.push_back(f[i-2] + f[i-1]);
       8 | }
       9 |
      10 | for (i in 0 thru 19) {
B --> 11 |       note("fibonacci(" + i + ") = " + f[i]);
      12 | }
Breakpoint #2,  _MAIN_() at     test.rs:11
ICVD>>  fin
test.rs:11 fibonacci(17) = 2584
       6 | for (i in 2 thru 19) {
       7 |       f.push_back(f[i-2] + f[i-1]);
       8 | }
       9 |
      10 | for (i in 0 thru 19) {
B --> 11 |       note("fibonacci(" + i + ") = " + f[i]);
      12 | }
Breakpoint #2,  _MAIN_() at     test.rs:11
ICVD>>
```

Feedback

info break

The `info break` command lists out information about the breakpoints set during the PXL Debugger session. There are two ways to list the breakpoint information:

1. `info break`

   When specified without a breakpoint number, all breakpoints are listed.

```
ICVD>>  info b
Breakpoint #1,  _MAIN_() at     test.rs:7     Enabled
Breakpoint #2,  _MAIN_() at     test.rs:16    Enabled
```

2. `info break` *`number`*

   If a breakpoint number is specified, only the breakpoint information for it is listed.

```
ICVD>>  info b 2
Breakpoint #2,  _MAIN_() at     test.rs:16    Enabled
ICVD>>
```

Feedback

list

The `list` command list lines from the source code. There are four ways to list lines:

1. `list`

   When specified without any option, the command lists 10 lines from the last printed line.

```
       1 | #include <math.rh>
       2 | #include <diagnostics.rh>
       3 |
  -->  4 | f: list of integer = {1, 1};
       5 |
       6 | for (i in 2 thru 19) {
       7 |         f.push_back(f[i-2] + f[i-1]);
       8 | }
ICVD>>  list
       9 |
      10 | /* comment 1
      11 |    comment 2
      12 |    comment 3
      13 |    comment 4
      14 |    comment 5
      15 | */
      16 |
      17 | for (i in 0 thru 19) {
      18 |       note("fibonacci(" + i + ") = " + f[i]);
ICVD>>
```

2. `list` *`line`*

   Print outs, from the current file, five lines before and four lines after the specified line number.

3. `list` *`file`*`:`*`line`*

   Print outs, from the specified file, five lines before and four lines after the specified line number.

```
ICVD>>  list test.rs:4
      1 | #include <math.rh>
      2 | #include <diagnostics.rh>
      3 |
  --> 4 | f: list of integer = {1, 1};
      5 |
      6 | for (i in 2 thru 19) {
      7 |         f.push_back(f[i-2] + f[i-1]);
      8 | }
ICVD>>
```

Feedback

4. `list` *`function`*

   Prints out five lines before and four lines after the first executable line within the specified function.

```
ICVD>>  list f2
       4 |
       5 | f2 : function (
       6 |      x : integer
       7 |      ) returning void
       8 | {
       9 |       t : integer = 0;
      10 |       y : integer = 10;
      11 |       z : integer = 100;
      12 |       for ( i=1 to x) {
      13 |           t = t + dtoi((x * y**2) / z);
ICVD>>
```

Feedback

next

The `next` command moves the PXL Debugger prompt to the next executable line in the runset. A `next` on a function call completes function execution, and then moves to the next executable line in the runset.

In this example, the `next` command is given at an executable line in the runset:

```
      13 |           t = t + dtoi((x * y**2) / z);
      14 |      }
      15 |      t = dtoi(t / x);
      16 |      note (" f2(3) = "+ t);
      17 | }
  --> 18 | xx:integer;
      19 | f2(3);
      20 | note("end. ");
ICVD>>  n
      14 |      }
      15 |      t = dtoi(t / x);
      16 |      note (" f2(3) = "+ t);
      17 | }
      18 | xx:integer;
  --> 19 | f2(3);
      20 | note("end. ");
ICVD>>
```

In this example, the `next` command is given on a function.

```
       8 | {
       9 |      ans = x;
      10 | }
      11 |
      12 | xx: integer = 0;
  --> 13 | xx = f2(3);
      14 | note("1. f2(3)= " + xx);
ICVD>>  n
       9 |      ans = x;
      10 | }
      11 |
      12 | xx: integer = 0;
      13 | xx = f2(3);
  --> 14 | note("1. f2(3)= " + xx);
ICVD>>
```

Feedback

print

The `print` command prints values of variables and simple expressions. The command format is

```
print expression
```

In this example, the value of a variable is printed:

```
       5 | f2 : function (
       6 |      x : integer
       7 |      ) returning ans : integer
       8 | {
B      9 |       ans = x;
  --> 10 |       note("0. ans = " + ans);
      11 | }
      12 |
      13 | xx: integer = 0;
      14 | xx = f2(3);
ICVD>>  p ans
ans = 3
```

In this example, the value of a loop variable is printed:

```
       2 | #include <diagnostics.rh>
       3 |
       4 | f: list of integer = {1, 1};
       5 |
       6 | for (i in 2 thru 19) {
B -->  7 |        f.push_back(f[i-2] + f[i-1]);
       8 | }
       9 |
      10 | /* comment 1
      11 |    comment 2
Breakpoint #1,  _MAIN_() at     test.rs:7
ICVD>>  p i
i = 3
```

In this example, the value of an expression is printed.

```
      19 | */
      20 |       t = dtoi(t / x);
      21 |       note (" f2(3) = "+ t);
      22 | }
      23 | xx:integer = 0;
  --> 24 | f2(3);
      25 | note("end. ");
ICVD>>  p xx
xx = 0
ICVD>>  p (xx+7)*20
(xx+7)*20 = 140
ICVD>>
```

Feedback

quit

The `quit` command ends the debugging session and the IC Validator run.

```
       1 | #include <math.rh>
       2 | #include <diagnostics.rh>
       3 |
  -->  4 | f: list of integer = {1, 1};
       5 |
       6 | for (i in 2 thru 19) {
       7 |         f.push_back(f[i-2] + f[i-1]);
       8 | }
ICVD>>  q
IC Validator Run: Time=  0:00:04 User=0.06 System=0.01
Overall icv_engine time = 0:00:04
IC Validator is done.
```

Feedback

record

The `record` command saves the commands from the current debugging session into an ASCII file. There are two ways to save commands:

1. `record`

   If a file name is not specified, the commands are saved in ICV*pid*.pdi, where *pid* is the UNIX process ID.

   ```
   ICVD>>  record
   Saving Commands to 'ICV22347.pdi'
   ICVD>>
   ```

2. `record` *`file`*

   You can specify a file name, with or without a path.

   ```
   ICVD>>  record myfile.cmd
   Saving Commands to 'myfile.cmd'
   ICVD>>
   ```

The ASCII file stores all commands, except break and list, verbatim from the PXL Debugger. The `break` and `list` commands are stored with the resolved file path. For example,

```
Starting ICV Debugger
       2 | #include "math.rh"
       3 | #include "string.rh"
       4 |
       5 | #include "b.rs"
       6 |
-->    7 | xx: integer = 0;
       8 | xx = f2(3);
       9 | note("2. f2(3)= " + xx);
ICVD>>  record
Saving Commands to 'ICV21589.pdi'
ICVD>>  b f2
Breakpoint #1,  f2() at b.rs:5
ICVD>>  list f2
       1 | f2 : function (
       2 |      x : integer
       3 |      ) returning ans : integer
       4 | {
B      5 |      ans = x;
       6 |      note("0. ans = " + ans);
       7 | }
ICVD>>  b 6
Breakpoint #2,  f2() at b.rs:6
ICVD>>  q
```

Feedback

The recorded file (ICV21589.pdi) contains:

```
b f2
list f2
b b.rs:6
q
```

Feedback

source

The `source` command executes the PXL Debugger commands in the specified file. After the commands are completed, control of the PXL Debugger returns to the user unless the PXL Debugger completed because of a `quit` command specified in the source file. The command format is

```
source file
```

For example,

```
ICVD>>  source ICVDebug9398.pdi
ICVD>>  n
       2 | #include <diagnostics.rh>
       3 |
       4 | f: list of integer = {1, 1};
       5 |
       6 | for (i in 2 thru 19) {
  -->  7 |       f.push_back(f[i-2] + f[i-1]);
       8 | }
       9 |
      10 | /* comment 1
      11 |    comment 2
ICVD>>  b 7
Breakpoint #1,  _MAIN_() at     test.rs:7
ICVD>>  b 16
Breakpoint #2,  _MAIN_() at     test.rs:16
ICVD>>  c
       2 | #include <diagnostics.rh>
       3 |
       4 | f: list of integer = {1, 1};
       5 |
       6 | for (i in 2 thru 19) {
B -->  7 |       f.push_back(f[i-2] + f[i-1]);
       8 | }
       9 |
      10 | /* comment 1
      11 |    comment 2
Breakpoint #1,  _MAIN_() at     test.rs:7
End of commands from commandfile
ICVD>>
```

Feedback

step

The `step` command allows you to step into functions calls, executing each line of within the function. The cursor moves to the first executable line in the function.

In this example, the PXL Debugger steps through f2, where the first executable line is 9.

```
      14 |       }
      15 |       t = dtoi(t / x);
      16 |       note (" f2(3) = "+ t);
      17 | }
      18 | xx:integer;
  --> 19 | f2(3);
      20 | note("end. ");
ICVD>>  s
       4 |
       5 | f2 : function (
       6 |     x : integer
       7 |     ) returning void
       8 | {
  -->  9 |       t : integer = 0;
      10 |       y : integer = 10;
      11 |       z : integer = 100;
      12 |       for ( i=1 to x) {
      13 |           t = t + dtoi((x * y**2) / z);
ICVD>>
```

Feedback

where

The `where` command prints the function call stack of a statement in the runset. In this example the command shows that there have been two function calls.

```
ICVD>>  s
      22 |          c = a + b;
      23 |      };
      24 |
      25 | sum: function (x: integer, y: integer) returning z: integer
      26 |      {
  --> 27 |          z = add(x,y);
      28 |       note("z = " + z);
      29 |      }
      30 |
      31 | q = add(x,y);
ICVD>>  where
#0    | sum()    at test.rs:27
#1    | _MAIN_()         at test.rs:32
```

In this example the command shows that there have been three function calls.

```
ICVD>>  s
      17 | note("x is now #2 = " + x);
      18 | note("y is now #2 = " + y);
      19 |
      20 | add: function (a: integer, b: integer) returning c: integer
      21 |      {
  --> 22 |          c = a + b;
      23 |      };
      24 |
      25 | sum: function (x: integer, y: integer) returning z: integer
      26 |      {
ICVD>>  where
#0    | add()    at test.rs:22
#1    | sum()    at test.rs:27
#2    | _MAIN_()         at test.rs:32
ICVD>>
```

E

PYDB Perl API

This appendix describes how to access the IC Validator error database (PYDB) using a Perl application programming interface (API) that is provided with the IC Validator tool.

The PYDB Perl API gives you direct access to the error data produced by the IC Validator tool. For example, you can

- Generate error information for a specific violation.
- Count violations for tracking purposes.

The database and the Perl API are described in the following sections:

- PYDB Perl API Overview
- Accessing the PYDB
- Examples of Common Tasks
- PYDB Schema

Feedback

PYDB Perl API Overview

Using the PYDB Perl API you can access information about violations found with the IC Validator tool and process the data for your needs. The IC Validator tool stores only violations and cells that contain error data in the PYDB. The LAYOUT_ERRORS file typically shows most of the information that is available in the PYDB.

Error data produced by the IC Validator tool is stored in tables in an SQLite database; this database is the PYDB. The structure of the tables and how they are related to each other is called the database schema. See “PYDB Schema” for information about the schema.

To properly query a complex database, you must know the database schema. You can use SQLite commands to access the database and return information. These commands can be complex depending on the information you want to be returned.

For convenient access to error data without needing to learn the database schema, The provided PYDB Perl API connects to and retrieves data from a PYDB. It makes interfacing with the error database much easier than using SQLite commands. It has built-in functions for

- Connecting to the PYDB server.
- Submitting queries and ensuring that multiple database servers are not competing with each other.
- Providing high-level functions to retrieve PYDB specific information without needing to create complex SQLite queries.
- Processing the results.

The PYDB Perl API is located in *ICV_INSTALL_DIRECTORY*/contrib/pydb/. The subdirectories are shown in Table 84.

Table 84 Subdirectories of the pydb Directory

Subdirectory	Description
bin/	Contains example scripts. These scripts show examples of how to use the PYDB modules.
lib/	Contains public library modules. Each of these files has a short description followed by detailed descriptions of all the class methods provided by the module.

Feedback

Accessing the PYDB

The PYDB Perl API provides you with a high level interface that allows you to work with error data, as well as providing the ability to run direct SQLite queries on the error database, if required.

Note:

The PYDB schema and server communication protocol can change between versions. Therefore, to access the PYDB you must use the API that is from the IC Validator version which generated the PYDB.

To access the PYDB using the PYDB Perl API provided with the IC Validator tool, follow these steps:

1. Load the PYDB library.

   Include the following code in your Perl script before any database commands. This code imports the needed modules from the IC Validator installation into the script.

   ```
   push @INC, $ENV{"ICV_INSTALL_DIRECTORY"}."/contrib/pydb/lib";
   require "PYDB.pm";
   ```

2. Connect to the database.

   Include the following code in your Perl script to connect to the database.

   ```
   $dbh = PYDB::Connect($db_path,$db_name);
   if (! $dbh) {
      print "ERROR: Error connecting to database: $PYDB::errstr\n";
      exit;
   }
   ```

   If the connection succeeds, a database handle is returned.

3. Perform database queries.

   Using the PYDB Perl API you can access information about the errors and process the data for your needs. Only violations and cells that contain error data is stored in the database. The LAYOUT_ERRORS file typically shows most of the information that is available in the PYDB. See “Examples of Common Tasks” for examples.

4. Disconnect from the database.

   Include the following code in your Perl script to disconnect from the database.

   ```
   $dbh->Disconnect();
   ```

Feedback

Examples of Common Tasks

The following examples show some common tasks.

- List violations in the database.

```
# All violations stored in the PYDB
foreach $vio ($dbh->Violations()) {
   print "Violation: \"".$vio->Comment()."\"\n";
}

# Find a specific violation
$vio = $dbh->GetViolationByComment("M1.S1.1: Metal 1 space");
if ($vio) {
   print "Found M1 violation, with ".$vio->ErrorCount().
      " error(s).\n";
}

# Find violations that match a regular expression
foreach $vio ($dbh->GetViolationsByCommentRegex(qr/M1\..*/)) {
   print "M1 rule: \"".$vio->Comment()."\"\n";
}
```

- List cells in the database.

```
# All cells stored in the PYDB
foreach $cell ($dbh->Cells()) {
   print "Cell: \"".$cell->CellName()."\"\n";
}

# Find a specific cell
$cell = $dbh->GetCellByName("TOP");
if ($cell) {
   print "Found top cell.\n";
}

# Find cells that match a regular expression
foreach $cell ($dbh->GetCellsByNameRegex(qr/STDCELL.*/)) {
   print "Found cell: \"".$cell->CellName()."\"\n";
}
```

- Access errors in a specific violation.

```
$vio = $dbh->GetViolationByComment("M1.S1.1: Metal 1 space");
if ($vio) {

   foreach $command ($vio->Commands()) {
      print "Command ".$command->Comment().
         " produced ".$command->NumErrorsUnlimited().
         " errors, details are available for ".
         $command->NumErrors()."\n";
```

Feedback

```
        foreach $cell ($command->Cells()) {
           # Errors are stored by command/cell, the error
           # iterator can be used to access them
           $iter = $dbh->ListErrors($command, $cell);
           if (! $iter) {
              print "ERROR: $PYDB::errstr\n";
              $dbh->Disconnect();
              exit;
           }

           while ($error = $iter->NextError()) {
              # Bounding box of error
              @bbox = $error->GetBBox();
              # Error polygons
              foreach $polygon ($error->Polygons()) {
                 # List of points making up the polygon
                 @points = $polygon->Points();
              }
              # Access to specific data fields by column name
              $distance = $error->Get("Distance");
              # Certain types of errors might have child errors
              foreach $child_error ($error->Children()) {
                 # ...
              }
           }
        }
     }
}
```

- Perform a direct SQLite query. The Perl API provides an interface similar to the DBI module that comes with most Perl installations.

```
# Prepare the query
$sth = $dbh->prepare("SELECT CellID, CellName FROM pyCellNames");
if (! $sth) {
   print "ERROR: $PYDB::errstr\n";
   $dbh->Disconnect();
   exit;
}

# Execute the query
if (! $sth->execute()) {
   print "ERROR: $PYDB::errstr\n";
   $dbh->Disconnect();
   exit;
}

# Retrieve the data
while ($ref = $sth->fetchrow_hashref()) {
   print "Cell name=".$ref->{'CellName'}.
      ", id=".$ref->{'CellID'}."\n";
}
```

Feedback

PYDB Schema

In the PYDB schema, errors are stored in error tables.

- Each table is made up of rows of data that can reference several related columns, similar to a spreadsheet.
- Each table needs at least one unique key to identify individual records. Some tables reference several keys.

There is at least one table per command type, with additional tables for commands with child errors. Error classification information is also stored in the PYDB in several tables. Table 85 shows some of the commonly used tables in the PYDB.

Table 85 PYDB Organization

Table	Information stored
pyCellNames	Contains cell information. Each command can write errors from different cells.
pyCommandCells	Lists the cells that contain errors for a command.
pyCommands	Contains one record for each runset command that produced errors. Multiple commands, for example, the `internal1()` and `external2()` functions, can write to the same violation comment.
pyErrors_<command type>	Contains individual error data. The error tables for a particular command type are created on the fly and contain data columns specific to that type of command.
pyErrorTableFormat	Defines the data columns of an error table.
pyErrorTables	Defines the hierarchy of error tables associated with a type of command.
pyPolygons_<command type>	Contains individual error shapes.
pyRunInfo	Contains general information.
pyViolations	Defines the violation rule comments for which errors are stored in the PYDB.

F

Third-Party Licenses

This appendix provides the license information for third-party software that is used in the IC Validator tool.

- Licensing Overview
- OSS Package Notices
  - ANTLR
  - Boost
  - cx_Freeze
  - Flatbuffers
  - libdp
  - libxml2
  - MariaDB Connector
  - MariaDB
  - MCPP Public Domain Code
  - NumPy*
  - patchELF
  - pyparsing*
  - Python 3.6 License
  - scikit-learn
  - SciPy*
  - Shroud-1.0
  - SQLite
  - TclLib

Feedback

- Tcl/Tk

  Note:

  The tbcload code is part of Tcl.

- zlib
- Zstandard

- Standard OSS License Text
  - Apache-2.0
  - Artistic-1.0-Perl
  - GPL-1.0
  - GPL-3.0

Licensing Overview

This document includes licensing information relating to free and open-source software ("OSS") included with Synopsys's® IC Validator products (the "SOFTWARE"). The terms of the applicable OSS license(s) govern Synopsys's® distribution and your use of the SOFTWARE. Synopsys® and the third-party authors, licensors, and distributors of the SOFTWARE disclaim all warranties and all liability arising from any and all use and distribution of the SOFTWARE. To the extent the OSS is provided under an agreement with Synopsys® that differs from the applicable OSS license(s), those terms are offered by Synopsys® alone.

Synopsys® has reproduced below copyright and other licensing notices appearing within the OSS packages. While Synopsys® seeks to provide complete and accurate copyright and licensing information for each OSS package, Synopsys® does not represent or warrant that the following information is complete, correct, or error-free. SOFTWARE recipients are encouraged to (a) investigate the identified OSS packages to confirm the accuracy of the licensing information provided herein and (b) notify Synopsys® of any inaccuracies or errors found in this document so that Synopsys® might update this document accordingly.

Certain OSS licenses (such as the GNU General Public Licenses, GNU Library/Lesser General Public Licenses, Affero General Public Licenses, Mozilla Public Licenses, Common Development and Distribution Licenses, Common Public License, and Eclipse Public License) require that the source code corresponding to distributed OSS binaries be made available to recipients or other requestors under the terms of the same OSS license.

Recipients or requestors who would like to receive a copy of such corresponding source code should submit a request to Synopsys® by post at:

Synopsys

Attn: Open Source Requests

690 E. Middlefield Road

Mountain View, CA 94043

Provide the following information in all submitted OSS requests:

- The OSS packages for which you are requesting source code;
- The Synopsys® product (and any available version information) with which the requested OSS packages are distributed;
- An email address at which Synopsys® might contact you regarding the request (if available); and
- The postal address for delivery of the requested source code.

An asterisk (*) has been added to OSS package names to indicate that those components might be included in software or hardware products developed using ZeBu. The OSS licensing information for those components is also reproduced in the IC Validator redistributable OSS notices text file included with the SOFTWARE.

OSS Package Notices

ANTLR

ANTLR 2 License

[The BSD License]

Copyright (c) 2012 Terence Parr and Sam Harwell

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the author nor the names of its contributors might be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Boost

Boost Software License - Version 1.0 - August 17th, 2003

Permission is hereby granted, free of charge, to any person or organization obtaining a copy of the software and accompanying documentation covered by this license (the “Software”) to use, reproduce, display, distribute, execute, and transmit the Software, and to prepare derivative works of the Software, and to permit third-parties to whom the Software is furnished to do so, all subject to the following:

The copyright notices in the Software and this entire statement, including the above license grant, this restriction and the following disclaimer, must be included in all copies of the Software, in whole or in part, and all derivative works of the Software, unless such copies or derivative works are solely in the form of machine-executable object code generated by a source language processor.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE COPYRIGHT HOLDERS OR ANYONE

Feedback

DISTRIBUTING THE SOFTWARE BE LIABLE FOR ANY DAMAGES OR OTHER LIABILITY, WHETHER IN CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

---

cx_Freeze

Project homepage/download site: http://cx-freeze.readthedocs.io/; https://pypi.python.org/pypi/cx_Freeze/4.3.4

Project licensing notices: http://cx-freeze.readthedocs.io/en/latest/license.html

```
Copyright © 2007-2017, Anthony Tuininga.
Copyright © 2001-2006, Computronix (Canada) Ltd., Edmonton, Alberta,
 Canada.
All rights reserved.

NOTE: this license is derived from the Python Software Foundation License
which can be found at http://www.python.org/psf/license

License for cx_Freeze 6.0b1

1. This LICENSE AGREEMENT is between the copyright holders and the
Individual or Organization ("Licensee") accessing and otherwise
using cx_Freeze software in source or binary form and its associated
documentation.

2. Subject to the terms and conditions of this License Agreement,
the copyright holders hereby grant Licensee a nonexclusive, royalty-
free, world-wide license to reproduce, analyze, test, perform and/or
display publicly, prepare derivative works, distribute, and otherwise
use cx_Freeze alone or in any derivative version, provided, however,
that this License Agreement and this notice of copyright are retained in
cx_Freeze alone or in any derivative version prepared by Licensee.
```

3. In the event Licensee prepares a derivative work that is based on or incorporates cx_Freeze or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to cx_Freeze.

4. The copyright holders are making cx_Freeze available to Licensee on an “AS IS” basis. THE COPYRIGHT HOLDERS MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, THE COPYRIGHT HOLDERS MAKE NO AND DISCLAIM ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF CX_FREEZE WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

Feedback

5. THE COPYRIGHT HOLDERS SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF CX_FREEZE FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING CX_FREEZE, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between the copyright holders and Licensee. This License Agreement does not grant permission to use copyright holder’s trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.

8. By copying, installing or otherwise using cx_Freeze, Licensee agrees to be bound by the terms and conditions of this License Agreement. Computronix® is a registered trademark of Computronix (Canada) Ltd.

Flatbuffers

Project homepage/download site: http://google.github.io/flatbuffers/; https://github.com/google/flatbuffers

Project licensing notices: https://github.com/google/flatbuffers/blob/master/LICENSE.txt

LICENSE.txt:

See Apache-2.0 in the Standard OSS License Text on page 552."

---

libdp

1. The "Software", below, refers to the Tcl-DP system, developed by the Tcl-DP group (in either source-code, object-code or executable-code form), and related documentation, and a "work based on the Software" means a work based on either the Software, on part of the Software, or on any derivative work of the Software under copyright law: that is, a work containing all or a portion of the Tcl-DP system, either verbatim or with modifications. Each licensee is addressed as "you" or "Licensee."
2. Cornell University as the parent organization of the Tcl-DP group holds copyrights in the Software. The copyright holder reserves all rights except those expressly granted to licensees, and U.S. Government license rights.
3. Permission is hereby granted to use, copy, modify, and to redistribute to others. If you distribute a copy or copies of the Software, or you modify a copy or copies of the

Feedback

Software or any portion of it, thus forming a work based on the Software, and make and/or distribute copies of such work, you must meet the following conditions:

1. If you make a copy of the Software (modified or verbatim) it must include the copyright notice and this license.

2. You must cause the modified Software to carry prominent notices stating that you changed specified portions of the Software.

4. LICENSEE AGREES THAT THE EXPORT OF GOODS AND/OR TECHNICAL DATA FROM THE UNITED STATES might REQUIRE SOME FORM OF EXPORT CONTROL LICENSE FROM THE U.S. GOVERNMENT AND THAT FAILURE TO OBTAIN SUCH EXPORT CONTROL LICENSE might RESULT IN CRIMINAL LIABILITY UNDER U.S. LAWS.

5. Portions of the Software resulted from work developed under a U.S. Government Contract and are subject to the following license: the Government is granted for itself and others acting on its behalf a paid-up, nonexclusive, irrevocable worldwide license in this computer software to reproduce, prepare derivative works, and perform publicly and display publicly.

6. Disclaimer of warranty: Licensor provides the software on an ``as is'' basis. Licensor does not warrant, guarantee, or make any representations regarding the use or results of the software with respect to its correctness, accuracy, reliability or performance. The entire risk of the use and performance of the software is assumed by licensee. ALL WARRANTIES INCLUDING, WITHOUT LIMITATION, ANY WARRANTY OF FITNESS FOR A PARTICULAR PURPOSE OR MERCHANTABILITY ARE HEREBY EXCLUDED.

7. Lack of maintenance or support services: Licensee understands and agrees that licensor is under no obligation to provide maintenance, support or update services, notices of latent defects, or correction of defects for the software.

8. Limitation of liability, indemnification: Even if advised of the possibility of damages, under no circumstances shall licensor be liable to licensee or any third party for damages of any character, including, without limitation, direct, indirect, incidental, consequential or special damages, loss of profits, loss of use, loss of goodwill, computer failure or malfunction. Licensee agrees to indemnify and hold harmless licensor for any and all liability licensor might incur as a result of licensee's use of the software.

libxml2

Libxml2 is the XML C parser and toolkit developed for the Gnome project (but usable outside of the Gnome platform), it is free software available under the MIT License. The MIT license:

The MIT License (MIT)

Copyright (c) <year> <copyright holders>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

MariaDB Connector

Synopsys modifications: NoneMariaDB Connector is licensed under the GNU Lesser General Public License, version 2.1, (LGPL-2.1). Software licensed under the LGPL-2.1 is free software that comes with ABSOLUTELY NO WARRANTY. You might modify, redistribute, or otherwise use the GPL software under the terms of the LGPL-2.1 (http://www.gnu.org/licenses/old-licenses/lgpl-2.1.html).

GNU LESSER GENERAL PUBLIC LICENSE

```
Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc. 51 Franklin
Street, Fifth Floor, Boston, MA 02110-1301 USA Everyone is permitted
to copy and distribute verbatim copies of this license document, but
changing it is not allowed.
```

Feedback

```
[This is the first released version of the Lesser GPL. It also counts as
the successor of the GNU Library Public License, version 2, hence the
version number 2.1.]
```

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it might be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You might copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

   You might charge a fee for the physical act of transferring a copy, and you might at your option offer warranty protection in exchange for a fee.

2. You might modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

   - The modified work must itself be a software library.
   - You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
   - You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
   - If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

   (For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

   These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

   Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

   In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You might opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Feedback

After this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You might copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

   If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

   However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

   When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work might be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

   If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

   Otherwise, if the work is a derivative of the Library, you might distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you might also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit

Feedback

modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)
- Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.
- Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.
- If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.
- Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It might happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

Feedback

7. You might place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:
    - Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.
    - Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.
8. You might not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.
9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.
10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You might not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.
11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you might not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

Feedback

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License might add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation might publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but might differ in detail to address new problems or concerns.

    Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you might choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

- BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT

Feedback

WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

- IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO might MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

MariaDB

Synopsys modifications: None

MariaDB is licensed under the GNU General Public License, version 2, (GPL-2.0). Software licensed under the GPL-2.0 is free software that comes with ABSOLUTELY NO WARRANTY. You might modify, redistribute, or otherwise use the GPL software under the terms of the GPL-2.0 (http://www.gnu.org/licenses/old-licenses/gpl-2.0.html)."

GNU GENERAL PUBLIC LICENSE

```
Version 2.1, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc. 51 Franklin
Street, Fifth Floor, Boston, MA 02110-1301, USA

Everyone is permitted to copy and distribute verbatim copies of this
license document, but changing it is not allowed.
```

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it might be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You might copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

   You might charge a fee for the physical act of transferring a copy, and you might at your option offer warranty protection in exchange for a fee.

2. You might modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

   - You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.
   - You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.
   - If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users might redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

     These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Feedback

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You might copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

   Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

   Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

   Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

   The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

   If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You might not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.
6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You might not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.
7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you might not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

   If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

   It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

   This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.
8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License might add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation might publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but might differ in detail to address new problems or concerns.

   Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you might choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

- BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.
- IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO might MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

MCPP Public Domain Code

Copyright (c) 1998, 2002-2007 Kiyoshi Matsui <kmatsui@t3.rim.or.jp>
All rights reserved.

Feedback

This software including the files in this directory is provided under the following license.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NumPy*

Project homepage/download site: http://www.numpy.org/; https://github.com/numpy/numpy

Project licensing notices: https://numpy.org/doc/stable/license.html

```
LICENSE.txt

Copyright (c) 2005-2017, NumPy Developers. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:

* Redistributions of source code must retain the above copyright notice,
this list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in the
documentation and/or other materials provided with the distribution.

* Neither the name of the NumPy Developers nor the names of any
contributors might be used to endorse or promote products derived from
this software without specific prior written permission.
```

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The NumPy repository and source distributions bundle several libraries that are compatibly licensed. We list these here.

Name: Numpydoc Files: doc/sphinxext/numpydoc/* License: 2-clause BSD For details, see doc/sphinxext/LICENSE.txt

Name: scipy-sphinx-theme Files: doc/scipy-sphinx-theme/* License: 3-clause BSD, PSF and Apache 2.0 For details, see doc/sphinxext/LICENSE.txt

Name: lapack-lite Files: numpy/linalg/lapack_lite/* License: 3-clause BSD For details, see numpy/linalg/lapack_lite/LICENSE.txt

Name: tempita Files: tools/npy_tempita/* License: BSD derived For details, see tools/npy_tempita/license.txt

Name: dragon4 Files: numpy/core/src/multiarray/dragon4.c License: One of a kind For license text, see numpy/core/src/multiarray/dragon4.cdoc/sphinxext/LICENSE.txt:

The files - numpydoc.py - docscrape.py - docscrape_sphinx.py - phantom_import.py have the following license: Copyright (C) 2008 Stefan van der Walt <stefan@mentat.za.net>, Pauli Virtanen <pav@iki.fi>

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

```
THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS OR
IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT,
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING,
BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF
USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF
THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

-------------------------------------------------------------------------

The files - compiler_unparse.py - comment_eater.py - traitsdoc.py have
the following license:

This software is OSI Certified Open Source Software. OSI Certified is
a certification mark of the Open Source Initiative. Copyright (c) 2006,
Enthought, Inc. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:

* Redistributions of source code must retain the above copyright notice,
this list of conditions and the following disclaimer. * Redistributions
in binary form must reproduce the above copyright notice, this list
of conditions and the following disclaimer in the documentation and/
or other materials provided with the distribution. * Neither the name
of Enthought, Inc. nor the names of its contributors might be used to
endorse or promote products derived from this software without specific
prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

-------------------------------------------------------------------------
```

The file - plot_directive.py originates from Matplotlib (http://matplotlib.sf.net/) which has the following license: Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved.

1. This LICENSE AGREEMENT is between John D. Hunter ("JDH"), and the Individual or Organization ("Licensee") accessing and otherwise using matplotlib software in source or binary form and its associated documentation.

2. Subject to the terms and conditions of this License Agreement, JDH hereby grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use matplotlib 0.98.3 alone or in any derivative version, provided, however, that JDH's License Agreement and JDH's notice of copyright, i.e., "Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved" are retained in matplotlib 0.98.3 alone or in any derivative version prepared by Licensee.

3. In the event Licensee prepares a derivative work that is based on or incorporates matplotlib 0.98.3 or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to matplotlib 0.98.3.

4. JDH is making matplotlib 0.98.3 available to Licensee on an "AS IS" basis. JDH MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, JDH MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF MATPLOTLIB 0.98.3 WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. JDH SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF MATPLOTLIB 0.98.3 FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING MATPLOTLIB 0.98.3, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between JDH and Licensee. This License Agreement does not grant permission to use JDH trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.

8. By copying, installing or otherwise using matplotlib 0.98.3, Licensee agrees to be bound by the terms and conditions of this License Agreement.

numpy/linalg/lapack_lite/LICENSE.txt:

Copyright (c) 1992-2013 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved. Copyright (c) 2000-2013 The University of California Berkeley. All rights reserved. Copyright (c) 2006-2013 The University of Colorado Denver. All rights reserved.

$COPYRIGHT$

Additional copyrights might follow

$HEADER$

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.

- Neither the name of the copyright holders nor the names of its contributors might be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

```
tools/npy_tempita/license.txt:

License

-------

Copyright (c) 2008 Ian Bicking and Contributors

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to the
following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN
NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE
USE OR OTHER DEALINGS IN THE SOFTWARE.

numpy/core/src/multiarray/dragon4.c:

/*
 * Copyright (c) 2014 Ryan Juckett
 * http://www.ryanjuckett.com/
 *
 * This software is provided 'as-is', without any express or implied
 * warranty. In no event will the authors be held liable for any damages
 * arising from the use of this software.
 *
 * Permission is granted to anyone to use this software for any purpose,
 * including commercial applications, and to alter it and redistribute it
 * freely, subject to the following restrictions:
 *
 * 1. The origin of this software must not be misrepresented; you must
 not
 *    claim that you wrote the original software. If you use this
 software
 *    in a product, an acknowledgment in the product documentation would
 be
 *    appreciated but is not required.
 *
 * 2. Altered source versions must be plainly marked as such, and must
 not be
```

Feedback

```
 *    misrepresented as being the original software.
 *
 * 3. This notice might not be removed or altered from any source
 *    distribution.
 */
```

patchELF

Project homepage/download site: https://nixos.org/patchelf.html

Project licensing notices:

COPYING:

See GPL-3.0 in the Standard OSS License Text on page 552."

pyparsing*

Project homepage/download site: https://pypi.python.org/pypi/pyparsing

Project licensing notices:

```
# module pyparsing.py
#
# Copyright (c) 2003-2016 Paul T. McGuire
#
# Permission is hereby granted, free of charge, to any person obtaining
# a copy of this software and associated documentation files (the
# "Software"), to deal in the Software without restriction, including
# without limitation the rights to use, copy, modify, merge, publish,
# distribute, sublicense, and/or sell copies of the Software, and to
# permit persons to whom the Software is furnished to do so, subject to
# the following conditions:
#
# The above copyright notice and this permission notice shall be
# included in all copies or substantial portions of the Software.
#
# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
# EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
# MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
# IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
# CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
# TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
# SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
#
```

Python 3.6 License

PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2

Feedback

---

1. This LICENSE AGREEMENT is between the Python Software Foundation ("PSF"), and the Individual or Organization ("Licensee") accessing and otherwise using this software ("Python") in source or binary form and its associated documentation.

2. Subject to the terms and conditions of this License Agreement, PSF hereby grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use Python alone or in any derivative version, provided, however, that PSF's License Agreement and PSF's notice of copyright, i.e., "Copyright (c) 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014 2015, 2016 Python Software Foundation; All Rights Reserved" are retained in Python alone or in any derivative version prepared by Licensee.

3. In the event Licensee prepares a derivative work that is based on or incorporates Python or any part thereof, and wants to make the derivative work available to others as provided herein, then Licensee hereby agrees to include in any such work a brief summary of the changes made to Python.

4. PSF is making Python available to Licensee on an "AS IS" basis. PSF MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED. BY WAY OF EXAMPLE, BUT NOT LIMITATION, PSF MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF PYTHON WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. PSF SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create any relationship of agency, partnership, or joint venture between PSF and Licensee. This License Agreement does not grant permission to use PSF trademarks or trade name in a trademark sense to endorse or promote products or services of Licensee, or any third party.

8. By copying, installing or otherwise using Python, Licensee agrees to be bound by the terms and conditions of this License Agreement.

Feedback

scikit-learn

Project homepage/download site: http://scikit-learn.org/

Project licensing notices:

```
New BSD License

Copyright (c) 2007-2018 The scikit-learn developers. All rights reserved.
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:

a. Redistributions of source code must retain the above copyright notice,
this list of conditions and the following disclaimer.

b. Redistributions in binary form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in the
documentation and/or other materials provided with the distribution.

c. Neither the name of the Scikit-learn Developers nor the names of its
contributors might be used to endorse or promote products derived from
this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS
IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS
BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF
THE POSSIBILITY OF SUCH DAMAGE.
```

SciPy*

Project homepage/download site: https://www.SciPy.org; https://github.com/SciPy/SciPy

Project licensing notices: https://github.com/SciPy/SciPy/blob/master/LICENSE.txt

```
Copyright (c) 2001, 2002 Enthought, Inc. All rights reserved.
Copyright (c) 2003-2017 SciPy Developers. All rights reserved.
Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
```

```
a. Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer.
b. Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer in the
 documentation and/or other materials provided with the distribution.
c. Neither the name of Enthought nor the names of the SciPy Developers
 might be used to endorse or promote products derived from this software
 without specific prior written permission. THIS SOFTWARE IS PROVIDED
 BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS
 OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
 DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDERS OR CONTRIBUTORS
 BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
 CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
 SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
 INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
 CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
 ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF
 THE POSSIBILITY OF SUCH DAMAGE.
SciPy bundles a number of libraries that are compatibly licensed.  We
 list these here.
Name: Numpydoc
Files: doc/sphinxext/numpydoc/*
License: 2-clause BSD
  For details, see doc/sphinxext/LICENSE.txt
Name: scipy-sphinx-theme
Files: doc/scipy-sphinx-theme/*
License: 3-clause BSD, PSF and Apache 2.0
  For details, see doc/sphinxext/LICENSE.txt
Name: Six
Files: scipy/_lib/six.py
License: MIT
  For details, see the header inside scipy/_lib/six.py
Name: Decorator
Files: scipy/_lib/decorator.py
License: 2-clause BSD
  For details, see the header inside scipy/_lib/decorator.py
Name: ID
Files: scipy/linalg/src/id_dist/*
License: 3-clause BSD
  For details, see scipy/linalg/src/id_dist/doc/doc.tex
Name: L-BFGS-B
Files: scipy/optimize/lbfgsb/*
License: BSD license
  For details, see scipy/optimize/lbfgsb/README
Name: SuperLU
Files: scipy/sparse/linalg/dsolve/SuperLU/*
License: 3-clause BSD
  For details, see scipy/sparse/linalg/dsolve/SuperLU/License.txt
Name: ARPACK
Files: scipy/sparse/linalg/eigen/arpack/ARPACK/*
License: 3-clause BSD
  For details, see scipy/sparse/linalg/eigen/arpack/ARPACK/COPYING
```

```
Name: Qhull
Files: scipy/spatial/qhull/*
License: Qhull license (BSD-like)
  For details, see scipy/spatial/qhull/COPYING.txt
Name: Cephes
Files: scipy/special/cephes/*
License: 3-clause BSD
  Distributed under 3-clause BSD license with permission from the author,
  see https://lists.debian.org/debian-legal/2004/12/msg00295.html
  Cephes Math Library Release 2.8:  June, 2000
  Copyright 1984, 1995, 2000 by Stephen L. Moshier
  This software is derived from the Cephes Math Library and is
  incorporated herein by permission of the author.
  All rights reserved.
  Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
* Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer in the
 documentation and/or other materials provided with the distribution.
* Neither the name of the <organization> nor the names of its
 contributors might be used to endorse or promote products derived from
 this software without specific prior written permission.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
 LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
 PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL <COPYRIGHT HOLDER>
 BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
 CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
 SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
 INTERRUPTION) HOWEVER CAUSED AND  ON ANY THEORY OF LIABILITY, WHETHER IN
 CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
 ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF
 THE POSSIBILITY OF SUCH DAMAGE.
Name: Faddeeva
Files: scipy/special/Faddeeva.*
License: MIT
  Copyright (c) 2012 Massachusetts Institute of Technology
  Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to permit
persons to whom the Software is furnished to do so, subject to the
following conditions:
The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN
NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
```

```
DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE
USE OR OTHER DEALINGS IN THE SOFTWARE.
doc/sphinxext/LICENSE.txt:
------------------------------------------------------------------------
------
The files - numpydoc.py - docscrape.py - docscrape_sphinx.py -
phantom_import.py have the following license:
Copyright (C) 2008 Stefan van der Walt <stefan@mentat.za.net>, Pauli
Virtanen <pav@iki.fi>
Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
1. Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer in the
 documentation and/or other materials provided with the distribution.
THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS
 OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT,
 INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
 SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
 STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING
 IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
 POSSIBILITY OF SUCH DAMAGE.
------------------------------------------------------------------------
------
The files - compiler_unparse.py - comment_eater.py - traitsdoc.py have
 the following license:
This software is OSI Certified Open Source Software. OSI Certified is a
 certification mark of the Open Source Initiative.
Copyright (c) 2006, Enthought, Inc. All rights reserved.
Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
 * Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer. * Redistributions
 in binary form must reproduce the above copyright notice, this list of
 conditions and the following disclaimer in the documentation and/or
 other materials provided with the distribution. * Neither the name
 of Enthought, Inc. nor the names of its contributors might be used to
 endorse or promote products derived from this software without specific
 prior written permission.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
 LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
 A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
 OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
 SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
```

```
 TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
 PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
 LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
 NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
--------------------------------------------------------------------------
------
The file - plot_directive.py originates from Matplotlib
 (http://matplotlib.sf.net/) which has the following license:
Copyright (c) 2002-2008 John D. Hunter; All Rights Reserved.
1. This LICENSE AGREEMENT is between John D. Hunter ("JDH"), and
 the Individual or Organization ("Licensee") accessing and otherwise
 using matplotlib software in source or binary form and its associated
 documentation.
2. Subject to the terms and conditions of this License Agreement, JDH
 hereby grants Licensee a nonexclusive, royalty-free, world-wide license
 to reproduce, analyze, test, perform and/or display publicly, prepare
 derivative works, distribute, and otherwise use matplotlib 0.98.3 alone
 or in any derivative version, provided, however, that JDH's License
 Agreement and JDH's notice of copyright, i.e., "Copyright (c) 2002-2008
 John D. Hunter; All Rights Reserved" are retained in matplotlib 0.98.3
 alone or in any derivative version prepared by Licensee.
3. In the event Licensee prepares a derivative work that is based on
 or incorporates matplotlib 0.98.3 or any part thereof, and wants to
 make the derivative work available to others as provided herein, then
 Licensee hereby agrees to include in any such work a brief summary of
 the changes made to matplotlib 0.98.3.
4. JDH is making matplotlib 0.98.3 available to Licensee on an "AS IS"
 basis. JDH MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.
 BY WAY OF EXAMPLE, BUT NOT LIMITATION, JDH MAKES NO AND DISCLAIMS
 ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS FOR
 ANY PARTICULAR PURPOSE OR THAT THE USE OF MATPLOTLIB 0.98.3 WILL NOT
 INFRINGE ANY THIRD PARTY RIGHTS.
5. JDH SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF MATPLOTLIB
 0.98.3 FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS
 AS A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING MATPLOTLIB
 0.98.3, OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY
 THEREOF.
6. This License Agreement will automatically terminate upon a material
 breach of its terms and conditions.
7. Nothing in this License Agreement shall be deemed to create any
 relationship of agency, partnership, or joint venture between JDH and
 Licensee. This License Agreement does not grant permission to use JDH
 trademarks or trade name in a trademark sense to endorse or promote
 products or services of Licensee, or any third party.
8. By copying, installing or otherwise using matplotlib 0.98.3, Licensee
 agrees to be bound by the terms and conditions of this License
 Agreement.
numpy/linalg/lapack_lite/LICENSE.txt:
Copyright (c) 1992-2013 The University of Tennessee and The University
 of Tennessee Research Foundation. All rights reserved. Copyright (c)
 2000-2013 The University of California Berkeley. All rights reserved.
```

```
 Copyright (c) 2006-2013 The University of Colorado Denver. All rights
 reserved.
$COPYRIGHT$
Additional copyrights might follow
$HEADER$
Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
- Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer listed in
 this license in the documentation and/or other materials provided with
 the distribution.
- Neither the name of the copyright holders nor the names of its
 contributors might be used to endorse or promote products derived from
 this software without specific prior written permission.
The copyright holders provide no reassurances that the source code
 provided does not infringe any patent, copyright, or any other
 intellectual property rights of third parties. The copyright holders
 disclaim any liability to any recipient for claims brought against
 recipient by any third party for infringement of that parties
 intellectual property rights.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
 LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
 A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
 OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
 SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
 TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
 PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
 LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
 NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
tools/npy_tempita/license.txt:
License
-------
Copyright (c) 2008 Ian Bicking and Contributors
Permission is hereby granted, free of charge, to any person obtaining
 a copy of this software and associated documentation files (the
 "Software"), to deal in the Software without restriction, including
 without limitation the rights to use, copy, modify, merge, publish,
 distribute, sublicense, and/or sell copies of the Software, and to
 permit
persons to whom the Software is furnished to do so, subject to the
following conditions:
The above copyright notice and this permission notice shall be included
 in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
 EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
 MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
 IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
 CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
```

```
 TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
 SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
scipy/_lib/six.py
"""Utilities for writing code that runs on Python 2 and 3"""
# Copyright (c) 2010-2012 Benjamin Peterson
#
# Permission is hereby granted, free of charge, to any person obtaining a
 copy of
# this software and associated documentation files (the "Software"), to
 deal in
# the Software without restriction, including without limitation the
rights to
# use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of
# the Software, and to permit persons to whom the Software is furnished
to do so,
# subject to the following conditions:
#
# The above copyright notice and this permission notice shall be included
in all
# copies or substantial portions of the Software.
#
# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR
# IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS
# FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR
# COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER
# IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
# CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
scipy/_lib/decorator.py
# ############################     LICENSE     ############################
#
# Copyright (c) 2005-2015, Michele Simionato
# All rights reserved.
# Redistribution and use in source and binary forms, with or without
# modification, are permitted provided that the following conditions are
# met:
#   Redistributions of source code must retain the above copyright
#   notice, this list of conditions and the following disclaimer.
#   Redistributions in bytecode form must reproduce the above copyright
#   notice, this list of conditions and the following disclaimer in
#   the documentation and/or other materials provided with the
#   distribution.
# THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
# "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
# LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
# A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
# HOLDERS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT,
# INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING,
```

```
# BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS
# OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND
# ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR
# TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE
# USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH
# DAMAGE.
scipy/linalg/src/id_dist/doc/doc.tex
The present document and all of the software
in the accompanying distribution (which is contained in the directory
{\tt id\_dist} and its subdirectories, or in the file
{\tt id\_dist.tar.gz})\, is
\bigskip
Copyright \copyright\ 2014 by P.-G. Martinsson, V. Rokhlin,
Y. Shkolnisky, and M. Tygert.
\bigskip
All rights reserved.
\bigskip
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:
\begin{enumerate}
\item Redistributions of source code must retain the above copyright
notice, this list of conditions, and the following disclaimer.
\item Redistributions in binary form must reproduce the above copyright
notice, this list of conditions, and the following disclaimer in the
documentation and/or other materials provided with the distribution.
\item None of the names of the copyright holders might be used to endorse
or promote products derived from this software without specific prior
written permission.
\end{enumerate}
\bigskip
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS ``AS IS'' AND ANY
EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNERS BE
LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
PROCUREMENT

 OF
SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR
BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE
 OR
OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
scipy/optimize/lbfgsb/README
License of L-BFGS-B (Fortran code)
==================================
The version included here (in lbfgsb.f) is 3.0 (released April 25, 2011).
It was written by Ciyou Zhu, Richard Byrd, and Jorge Nocedal
<nocedal@ece.nwu.edu>. It carries the following condition for use:
"""
```

```
This software is freely available, but we expect that all publications
describing work using this software, or all commercial products using it,
quote at least one of the references given below. This software is
released under the BSD License.
References
* R. H. Byrd, P. Lu and J. Nocedal. A Limited Memory Algorithm for Bound
Constrained Optimization, (1995), SIAM Journal on Scientific and
Statistical Computing, 16, 5, pp. 1190-1208.
* C. Zhu, R. H. Byrd and J. Nocedal. L-BFGS-B: Algorithm 778: L-BFGS-B,
FORTRAN routines for large scale bound constrained optimization (1997),
ACM Transactions on Mathematical Software, 23, 4, pp. 550 - 560.
* J.L. Morales and J. Nocedal. L-BFGS-B: Remark on Algorithm 778:
L-BFGS-B, FORTRAN routines for large scale bound constrained optimization
(2011), ACM Transactions on Mathematical Software, 38, 1.
"""
License for the Python wrapper
==============================
Copyright (c) 2004 David M. Cooke <cookedm@physics.mcmaster.ca>
Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to permit
persons to whom the Software is furnished to do so, subject to the
following conditions:
The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN
NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE
USE OR OTHER DEALINGS IN THE SOFTWARE.
scipy/sparse/linalg/dsolve/SuperLU/License.txt
Copyright (c) 2003, The Regents of the University of California, through
Lawrence Berkeley National Laboratory (subject to receipt of any required
approvals from U.S. Dept. of Energy)
All rights reserved.
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:
(1) Redistributions of source code must retain the above copyright
 notice, this list of conditions and the following disclaimer.
(2) Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer in the
 documentation and/or other materials provided with the distribution.
(3) Neither the name of Lawrence Berkeley National Laboratory, U.S. Dept.
 of Energy nor the names of its contributors might be used to endorse
 or promote products derived from this software without specific prior
 written permission.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
```

```
 LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
 A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
 OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
 SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
 TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
 PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
 LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
 NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
scipy/sparse/linalg/eigen/arpack/ARPACK/COPYING
BSD Software License
Pertains to ARPACK and P_ARPACK
Copyright (c) 1996-2008 Rice University.
Developed by D.C. Sorensen, R.B. Lehoucq, C. Yang, and K. Maschhoff.
All rights reserved.
Arpack has been renamed to arpack-ng.
Copyright (c) 2001-2011 - Scilab Enterprises
Updated by Allan Cornet, Sylvestre Ledru.
Copyright (c) 2010 - Jordi Gutiérrez Hermoso (Octave patch)
Copyright (c) 2007 - Sébastien Fabbro (gentoo patch)
Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions are
 met:
- Redistributions of source code must retain the above copyright notice,
 this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright
 notice, this list of conditions and the following disclaimer listed in
 this license in the documentation and/or other materials provided with
 the distribution.
- Neither the name of the copyright holders nor the names of its
 contributors might be used to endorse or promote products derived from
 this software without specific prior written permission.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
 LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
 A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
 OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
 SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
 TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
 PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
 LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
 NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
scipy/spatial/qhull/COPYING.txt
Qhull, Copyright (c) 1993-2015
C.B. Barber
Arlington, MA
and
The National Science and Technology Research Center for Computation and
 Visualization of Geometric Structures
 (The Geometry Center)
University of Minnesota
email: qhull@qhull.org
```

Feedback

```
This software includes Qhull from C.B. Barber and The Geometry Center.
Qhull is copyrighted as noted above.  Qhull is free software and might
 be obtained via http from www.qhull.org.  It might be freely copied,
 modified, and redistributed under the following conditions:
1. All copyright notices must remain intact in all files.
2. A copy of this text file must be distributed along with any copies
 of Qhull that you redistribute; this includes copies that you have
 modified, or copies of programs or other software products that include
 Qhull.
3. If you modify Qhull, you must include a notice giving the name of the
 person performing the modification, the date of modification, and the
 reason for such modification.
4. When distributing modified versions of Qhull, or other software
 products that include Qhull, you must provide notice that the original
 source code might be obtained as noted above.
5. There is no warranty or other guarantee of fitness for Qhull, it
 is provided solely "as is".  Bug reports or fixes might be sent to
 qhull_bug@qhull.org; the authors might or might not act on them as they
 want.
```

Shroud-1.0

Project homepage: http://www.cpan.org/authors/id/C/CR/CRAIC/shroud-1.0

Project license:

```
#!/usr/bin/perl
# shroud
# Copyright 2000 Robert Jones, Craic Computing, All rights reserved.
# This program is free software; you can redistribute it and/or modify it
# under the same terms as Perl itself.
# The software is supplied as is, with absolutely no warranty.
Perl5 is Copyright (C) 1993-2005, by Larry Wall and others.

It is free software; you can redistribute it and/or modify it under the
terms of either:

a) the GNU General Public License as published by the Free Software
Foundation; either external link version 1, or (at your option) any later
versionexternal link, or

b) the "Artistic License".

For those of you that choose to use the GNU General Public License, my
interpretation of the GNU General Public License is that no Perl script
falls under the terms of the GPL unless you explicitly put said script
under the terms of the GPL yourself.

Furthermore, any object code linked with perl does not automatically
fall under the terms of the GPL, provided such object code only adds
definitions of subroutines and variables, and does not otherwise impair
```

Feedback

```
the resulting interpreter from executing any standard Perl script.
I consider linking in C subroutines in this manner to be the moral
equivalent of defining subroutines in the Perl language itself. You
might sell such an object file as proprietary provided that you provide
or offer to provide the Perl source, as specified by the GNU General
Public License. (This is merely an alternate way of specifying input to
the program.) You might also sell a binary produced by the dumping of
a running Perl script that belongs to you, provided that you provide or
offer to provide the Perl source as specified by the GPL. (The fact that
a Perl interpreter and your code are in the same binary file is, in this
case, a form of mere aggregation.)

This is my interpretation of the GPL. If you still have concerns or
difficulties understanding my intent, feel free to contact me. Of course,
the Artistic License spells all this out for your protection, so you
might prefer to use that.

-- Larry Wall
```

See Artistic-1.0-Perl in the Standard OSS License Text on page 552."See GPL-1.0 in the Standard OSS License Text on page 552."

SQLite

Project homepage/download site: https://www.sqlite.org/index.html

Project licensing notices: https://www.sqlite.org/copyright.html

```
SQLite Is Public Domain

All of the code and documentation in SQLite has been dedicated to the
public domain by the authors. All code authors, and representatives
of the companies they work for, have signed affidavits dedicating
their contributions to the public domain and originals of those signed
affidavits are stored in a firesafe at the main offices of Hwaci. Anyone
is free to copy, modify, publish, use, compile, sell, or distribute the
original SQLite code, either in source code form or as a compiled binary,
for any purpose, commercial or non-commercial, and by any means.

The previous paragraph applies to the deliverable code and documentation
in SQLite - those parts of the SQLite library that you actually bundle
and ship with a larger application. Some scripts used as part of the
build process (for example the "configure" scripts generated by autoconf)
might fall under other open-source licenses. Nothing from these build
scripts ever reaches the final deliverable SQLite library, however, and
so the licenses associated with those scripts should not be a factor in
assessing your rights to copy and use the SQLite library.
```

All of the deliverable code in SQLite has been written from scratch. No code has been taken from other projects or from the open internet. Every line of code can be traced back to its original author, and all of those authors have public domain dedications on file. So the SQLite code base is clean and is uncontaminated with licensed code from other projects.

Open-Source, not Open-Contribution

SQLite is open-source, meaning that you can make as many copies of it as you want and do whatever you want with those copies, without limitation. But SQLite is not open-contribution. In order to keep SQLite in the public domain and ensure that the code does not become contaminated with proprietary or licensed content, the project does not accept patches from unknown persons.

All of the code in SQLite is original, having been written specifically for use by SQLite. No code has been copied from unknown sources on the internet.

Warranty of Title

SQLite is in the public domain and does not require a license. Even so, some organizations want legal proof of their right to use SQLite. Circumstances where this occurs include the following:

Your company desires indemnity against claims of copyright infringement.

You are using SQLite in a jurisdiction that does not recognize the public domain.

You are using SQLite in a jurisdiction that does not recognize the right of an author to dedicate their work to the public domain.

You want to hold a tangible legal document as evidence that you have the legal right to use and distribute SQLite.

Your legal department tells you that you have to purchase a license.

If any of the above circumstances apply to you, Hwaci, the company that employs all the developers of SQLite, will sell you a Warranty of Title for SQLite. A Warranty of Title is a legal document that asserts that the claimed authors of SQLite are the true authors, and that the authors have the legal right to dedicate the SQLite to the public domain, and that Hwaci will vigorously defend against challenges to those claims. All proceeds from the sale of SQLite Warranties of Title are used to fund continuing improvement and support of SQLite.

Contributed Code

Feedback

```
In order to keep SQLite completely free and unencumbered by copyright,
the project does not accept patches. If you would like to make a
suggested change, and include a patch as a proof-of-concept, that would
be great. However do not be offended if we rewrite your patch from
scratch.
```

TclLib

This software is copyrighted by Ajuba Solutions and other parties.The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software might be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Tcl/Tk

The following terms apply to the all versions of the core Tcl/Tk releases, the Tcl/Tk browser plug-in version 2.0, Tcllib, and TclBlend and Jacl version 1.0. Note that the TclPro tools are under a different license agreement. This agreement is part of the standard Tcl/Tk distribution as the file named "license.terms".

Feedback

Tcl/Tk License Terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, Ajuba Solutions, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software might be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

---

zlib

License

/* zlib.h -- interface of the 'zlib' general purpose compression library version 1.2.8, April 28th, 2013

Copyright (C) 1995-2013 Jean-loup Gailly and Mark Adler

Feedback

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgement in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice might not be removed or altered from any source distribution.

Jean-loup Gailly (jloup@gzip.org) Mark Adler (madler@alumni.caltech.edu)

*/

Zstandard

Project homepage: https://github.com/facebook/zstd

Project license: https://github.com/facebook/zstd/blob/master/LICENSE

```
BSD License

For Zstandard software

Copyright (c) 2016-present, Facebook, Inc. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:

* Redistributions of source code must retain the above copyright notice,
this list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in the
documentation and/or other materials provided with the distribution.

* Neither the name Facebook nor the names of its contributors might be
used to endorse or promote products derived from this software without
specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
```

Feedback

```
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

Standard OSS License Text

Apache-2.0

Project homepage/download site: http://google.github.io/flatbuffers/; https://github.com/google/flatbuffers

Project licensing notices: https://github.com/google/flatbuffers/blob/master/LICENSE.txt

LICENSE.txt

```
Apache LicenseVersion 2.0, January 2004 http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction,and
distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the
copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other
entities that control, are controlled by, or are under common control
with that entity. For the purposes of this definition, "control" means
(i) the power, direct or indirect, to cause the direction or management
of such entity, whether by contract or otherwise, or (ii) ownership
of fifty percent (50%) or more of the outstanding shares, or (iii)
beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising
permissions granted by this License.

"Source" form shall mean the preferred form for making modifications,
including but not limited to software source code, documentation source,
and configuration files.
```

Feedback

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell,

Feedback

sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You might reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You might add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You might add Your own copyright statement to Your modifications and might provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and

Feedback

distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you might have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You might choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You might act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such

Feedback

Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS\

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright 2014 Google Inc.

Licensed under the Apache License, Version 2.0 (the "License"); you might not use this file except in compliance with the License. You might obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Artistic-1.0-Perl

The "Artistic License"

Preamble

The intent of this document is to state the conditions under which a Package might be copied, such that the Copyright Holder maintains some semblance of artistic control over the development of the package, while giving the users of the package the right to use and distribute the Package in a more-or-less customary fashion, plus the right to make reasonable modifications.

Definitions:

"Package" refers to the collection of files distributed by the Copyright Holder, and derivatives of that collection of files created through textual modification.

Feedback

"Standard Version" refers to such a Package if it has not been modified, or has been modified in accordance with the wishes of the Copyright Holder as specified below.

"Copyright Holder" is whoever is named in the copyright or copyrights for the package.

"You" is you, if you're thinking about copying or distributing this Package.

"Reasonable copying fee" is whatever you can justify on the basis of media cost, duplication charges, time of people involved, and so on. (You will not be required to justify it to the Copyright Holder, but only to the computing community at large as a market that must bear the fee.)

"Freely Available" means that no fee is charged for the item itself, though there might be fees involved in handling the item. It also means that recipients of the item might redistribute it under the same conditions they received it.

- You might make and give away verbatim copies of the source form of the Standard Version of this Package without restriction, provided that you duplicate all of the original copyright notices and associated disclaimers.
- You might apply bug fixes, portability fixes and other modifications derived from the Public Domain or from the Copyright Holder. A Package modified in such a way shall still be considered the Standard Version.
- You might otherwise modify your copy of this Package in any way, provided that you insert a prominent notice in each changed file stating how and when you changed that file, and provided that you do at least ONE of the following:
    - place your modifications in the Public Domain or otherwise make them Freely Available, such as by posting said modifications to Usenet or an equivalent medium, or placing the modifications on a major archive site such as uunet.uu.net, or by allowing the Copyright Holder to include your modifications in the Standard Version of the Package.
    - use the modified Package only within your corporation or organization.
    - rename any non-standard executables so the names do not conflict with standard executables, which must also be provided, and provide

Feedback

a separate manual page for each non-standard executable that clearly documents how it differs from the Standard Version.

- make other distribution arrangements with the Copyright Holder.

- You might distribute the programs of this Package in object code or executable form, provided that you do at least ONE of the following:
  - distribute a Standard Version of the executables and library files, together with instructions (in the manual page or equivalent) on where to get the Standard Version.
  - accompany the distribution with the machine-readable source of the Package with your modifications.
  - give non-standard executables non-standard names, and clearly document the differences in manual pages (or equivalent), together with instructions on where to get the Standard Version.
  - make other distribution arrangements with the Copyright Holder.
- You might charge a reasonable copying fee for any distribution of this Package. You might charge any fee you choose for support of this Package. You might not charge a fee for this Package itself. However, you might distribute this Package in aggregate with other (possibly commercial) programs as part of a larger (possibly commercial) software distribution provided that you do not advertise this Package as a product of your own. You might embed this Package's interpreter within an executable of yours (by linking); this shall be construed as a mere form of aggregation, provided that the complete Standard Version of the interpreter is so embedded.
- The scripts and library files supplied as input to or produced as output from the programs of this Package do not automatically fall under the copyright of this Package, but belong to whoever generated them, and might be sold commercially, and might be aggregated with this Package. If such scripts or library files are aggregated with this Package via the so-called "undump" or "unexec" methods of producing a binary executable image, then distribution of such an image shall neither be construed as a distribution of this Package nor shall it fall under the restrictions of Paragraphs 3 and 4, provided that you do not represent such an executable image as a Standard Version of this Package.
- subroutines (or comparably compiled subroutines in other languages) supplied by you and linked into this Package in order to emulate subroutines and variables of the language defined by this Package shall not be considered part of this Package, but are the equivalent

Feedback

of input as in Paragraph 6, provided these subroutines do not change the language in any way that would cause it to fail the regression tests for the language.

- Aggregation of this Package with a commercial distribution is always permitted provided that the use of this Package is embedded; that is, when no overt attempt is made to make this Package's interfaces visible to the end user of the commercial distribution. Such use shall not be construed as a distribution of this Package.
- The name of the Copyright Holder might not be used to endorse or promote products derived from this software without specific prior written permission.
- THIS PACKAGE IS PROVIDED "AS IS" AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTIBILITY AND FITNESS FOR A PARTICULAR PURPOSE.

The End

GPL-1.0

GNU GENERAL PUBLIC LICENSE Version 1, February 1989

Copyright (C) 1989 Free Software Foundation, Inc. 59 Temple Place, Suite 330, Boston, MA 02111-1307, USA Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The license agreements of most software companies try to keep users at the mercy of those companies. By contrast, our General Public License is intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users. The General Public License applies to the Free Software Foundation's software and to any other program whose authors commit to using it. You can use it for your programs, too.

When we speak of free software, we are referring to freedom, not price. Specifically, the General Public License is designed to make sure that you have the freedom to give away or sell copies of free software, that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

Feedback

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of a such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must tell them their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

The precise terms and conditions for copying, distribution and modification follow. GNU GENERAL PUBLIC LICENSE TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any program or other work which contains a notice placed by the copyright holder saying it might be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any work containing the Program or a portion of it, either verbatim or with modifications. Each licensee is addressed as "you".

- You might copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this General Public License and to the absence of any warranty; and give any other recipients of the Program a copy of this General Public License along with the Program. You might charge a fee for the physical act of transferring a copy.

Feedback

- You might modify your copy or copies of the Program or any portion of it, and copy and distribute such modifications under the terms of Paragraph 1 above, provided that you also do the following:
  - cause the modified files to carry prominent notices stating that you changed the files and the date of any change; and
  - cause the whole of any work that you distribute or publish, that in whole or in part contains the Program or any part thereof, either with or without modifications, to be licensed at no charge to all third parties under the terms of this General Public License (except that you might choose to grant warranty protection to some or all third parties, at your option).
  - If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the simplest and most usual way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users might redistribute the program under these conditions, and telling the user how to view a copy of this General Public License.
  - You might charge a fee for the physical act of transferring a copy, and you might at your option offer warranty protection in exchange for a fee.

    Mere aggregation of another independent work with the Program (or its derivative) on a volume of a storage or distribution medium does not bring the other work under the scope of these terms. 3. You might copy and distribute the Program (or a portion or derivative of it, under Paragraph 2) in object code or executable form under the terms of Paragraphs 1 and 2 above provided that you also do one of the following:
  - accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Paragraphs 1 and 2 above; or,
  - accompany it with a written offer, valid for at least three years, to give any third party free (except for a nominal charge for the cost of distribution) a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Paragraphs 1 and 2 above; or,
  - accompany it with the information you received as to where the corresponding source code might be obtained. (This alternative is

Feedback

allowed only for noncommercial distribution and only if you received the program in object code or executable form alone.)

Source code for a work means the preferred form of the work for making modifications to it. For an executable file, complete source code means all the source code for all modules it contains; but, as a special exception, it need not include source code for modules which are standard libraries that accompany the operating system on which the executable file runs, or for standard header files or definitions files that accompany that operating system.

4. You might not copy, modify, sublicense, distribute or transfer the Program except as expressly provided under this General Public License. Any attempt otherwise to copy, modify, sublicense, distribute or transfer the Program is void, and will automatically terminate your rights to use the Program under this License. However, parties who have received copies, or rights to use copies, from you under this General Public License will not have their licenses terminated so long as such parties remain in full compliance.

5. By copying, distributing or modifying the Program (or any work based on the Program) you indicate your acceptance of this license to do so, and all its terms and conditions.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You might not impose any further restrictions on the recipients' exercise of the rights granted herein. 7. The Free Software Foundation might publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but might differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of the license which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the license, you might choose any version ever published by the Free Software Foundation.

8. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of

preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

9. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

10. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO might MODIFY AND/ OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS Appendix: How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to humanity, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

Copyright (C) 19yy

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 1, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

Feedback

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307, USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

Gnomovision version 69, Copyright (C) 19xx name of author Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `show w'. This is free software, and you are welcome to redistribute it under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, the commands you use might be called something other than `show w' and `show c'; they could even be mouse-clicks or menu items--whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here a sample; alter the names:

Yoyodyne, Inc., hereby disclaims all copyright interest in the program `Gnomovision' (a program to direct compilers to make passes at assemblers) written by James Hacker.

, 1 April 1989 Ty Coon, President of Vice

That's all there is to it!

GPL-3.0

GNU GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <http://fsf.org/>
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users. We, the Free Software Foundation, use the

Feedback

GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could

make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" might be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees might convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

Feedback

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its

Feedback

content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You might make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You might convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You might convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You might charge any price or no price for each copy that you convey, and you might offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You might convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You might convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.

b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you

Feedback

offer spare parts or customer support for that product model, to
give anyone who possesses the object code either (1) a copy of the
Corresponding Source for all the software in the product that is covered
by this License, on a durable physical medium customarily used for
software interchange, for a price no more than your reasonable cost of
physically performing this conveying of source, or (2) access to copy the
Corresponding Source from a network server at no charge.

c) Convey individual copies of the object code with a copy of the written
offer to provide the Corresponding Source. This alternative is allowed
only occasionally and noncommercially, and only if you received the
object code with such an offer, in accord with subsection 6b.

d) Convey the object code by offering access from a designated
place (gratis or for a charge), and offer equivalent access to the
Corresponding Source in the same way through the same place at no further
charge. You need not require recipients to copy the Corresponding Source
along with the object code. If the place to copy the object code is a
network server, the Corresponding Source might be on a different server
(operated by you or a third party) that supports equivalent copying
facilities, provided you maintain clear directions next to the object
code saying where to find the Corresponding Source. Regardless of what
server hosts the Corresponding Source, you remain obligated to ensure
that it is available for as long as needed to satisfy these requirements.

e) Convey the object code using peer-to-peer transmission, provided
you inform other peers where the object code and Corresponding Source
of the work are being offered to the general public at no charge under
subsection 6d.

A separable portion of the object code, whose source code is excluded
from the Corresponding Source as a System Library, need not be included
in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any
tangible personal property which is normally used for personal, family,
or household purposes, or (2) anything designed or sold for incorporation
into a dwelling. In determining whether a product is a consumer product,
doubtful cases shall be resolved in favor of coverage. For a particular
product received by a particular user, "normally used" refers to a
typical or common use of that class of product, regardless of the
status of the particular user or of the way in which the particular user
actually uses, or expects or is expected to use, the product. A product
is a consumer product regardless of whether the product has substantial
commercial, industrial or non-consumer uses, unless such uses represent
the only significant mode of use of the product.

Feedback

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network might be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part might be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you might at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions might be written to require their own removal

Feedback

in certain cases when you modify the work.) You might place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you might (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

d) Limiting the use for publicity purposes of names of licensors or authors of the material; or

e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you might remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you might add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Feedback

Additional terms, permissive or non-permissive, might be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You might not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

Feedback

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You might not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you might not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you might not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy,

Feedback

free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You might not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that might otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence

Feedback

you might not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation might publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but might differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you might choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions might give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY

Feedback

OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Feedback

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <http://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

<program> Copyright (C) <year> <name of author> This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'. This is free software, and you are welcome to redistribute it under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <http://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you might consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, read <http://www.gnu.org/philosophy/why-not-lgpl.html>.

Glossary

BEOL

The portion of the semiconductor manufacturing process that involves the creation of metal interconnects and vias to connect transistors and other components on a chip.

BEOL (Back End of Line)

BSD

A description language used to define the boundary scan architecture of a device, enabling testing and debugging of interconnects on printed circuit boards.

BSD (Boundary Scan Description)

DBI

A technique used to reduce power consumption and electromagnetic interference in data buses by inverting the data bits when the number of transitions exceeds a threshold.

DBI (Data Bus Inversion)

DBU

The smallest unit of measurement used in a design database, typically defined in microns or nanometers, to ensure precision in layout and design rule checking.

DBU (Database Unit)

DFM

A set of design practices and methodologies aimed at optimizing a design for ease of manufacturing, improving yield, and reducing production costs.

DFM (Design for Manufacturability)

EERC

A comprehensive reliability solution that extends traditional electrical rule checking to include netlist domain, mixed mode, and current density/point-to-point resistance checks.

EERC (Extended electrical rule checking)

Feedback

FEOL

The portion of semiconductor manufacturing that involves the creation of transistors and other active devices on the silicon wafer before interconnect layers are added.

FEOL (Front-End-Of-Line)

GDS

A data format used for representing the physical layout of integrated circuits. It is widely used in the semiconductor industry for chip design and mask generation.

GDS (Graphical Design System)

GPL

A widely used open-source license that allows users to freely use, modify, and distribute software, provided that the same rights are preserved in derivative works.

GPL (GNU General Public License)

ICVD

A debugging tool integrated with IC Validator that allows users to analyze and resolve errors detected during physical verification processes.

ICVD (IC Validator Debugger)

JSON

A lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.

JSON (JavaScript Object Notation)

LGPL

A more permissive version of the GPL that allows linking to proprietary software under certain conditions.

LGPL (GNU Limited General Public License)

MRI

A set of physical verification checks that ensure the layout adheres to specific mask manufacturing requirements.

MRI (Mask Rule Inspection)

NDM

A hierarchical data model used in IC design to represent layout and connectivity information.

Feedback

NDM (Newton Data Model)

NFS

A distributed file system protocol that allows users to access files over a network as if they were on a local disk.

NFS (Network file system)

NMOS

A type of MOSFET where the majority carriers are electrons, and the channel is formed in an n-type semiconductor.

NMOS (N-type Metal-Oxide-Semiconductor)

OOS

Software that is freely available for use, modification, and distribution, often under licenses like the GNU General Public License.

OSS (Open source software)

PDK

A collection of files and data provided by a semiconductor foundry that contains the process-specific information needed for IC design, including design rules, device models, and layout information.

PDK (Process Design Kit)

PERC

A customizable verification process that checks for electrical rule violations, such as current density and point-to-point resistance.

PERC (Programmable Electrical Rule Checking)

POLY

A mask layer used in semiconductor manufacturing to define the gate region of a MOSFET.

POLY (Polysilicon)

PVE

A framework integrated with IC design tools that provides a set of utilities and configurations, often defined in files like pdk.xml, to facilitate the verification of semiconductor processes, ensuring compliance with design and manufacturing rules.

PVE (Process Verification Environment)

PXL

A scripting language used in IC Validator for creating custom runsets and automating verification tasks.

PXL (Programmable and Extensible Language)

RAM

A type of computer memory that can be accessed randomly, allowing data to be read or written in any order.

RAM (Random Access Memory)

SGE

A distributed resource management system that schedules and manages the execution of jobs on a computing cluster.

SGE (Sun Grid Engine)

SMT

A technique that allows multiple threads to execute simultaneously on a single processor core, improving performance and resource utilization.

SMT (Simultaneous Multi-Threading)

SVS

A verification process that compares two schematic netlists to ensure they are functionally equivalent.

SVS (Schematic vs schematic)

TCL

A scripting language commonly used for automation and configuration in electronic design automation (EDA) tools.

TCL (Tool Command Language)

VUE

VUE is a flow-based graphical tool within IC Validator that guides users through the entire physical verification flow.

VUE (Verification User Environment)

XML

A markup language used for encoding documents in a format that is both human-readable and machine-readable.

Feedback

XML (Extensible Markup Language)